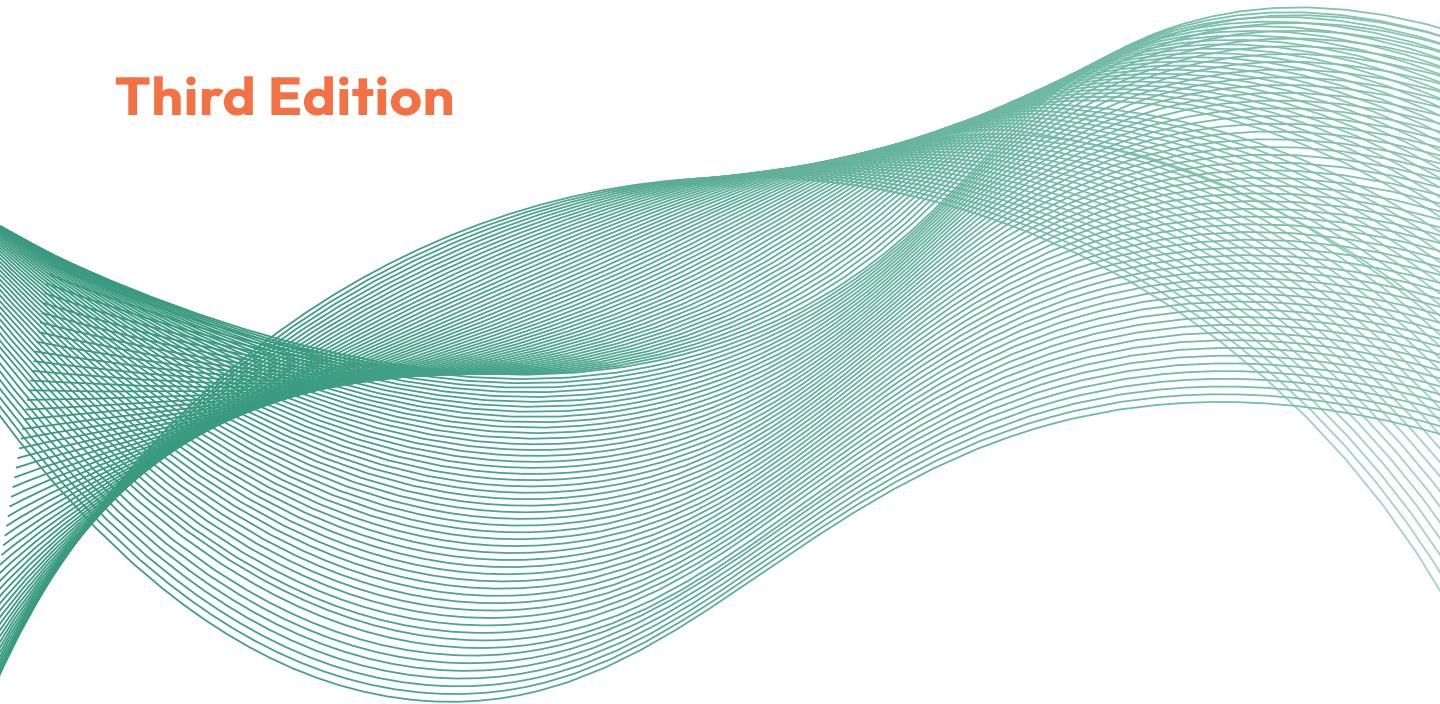


EXPERT INSIGHT

Web Development with Django 6

**A definitive guide to building modern
Python web applications using Django 6**

Third Edition



Chris Guest | Mark Walker | Ben Shaw
Saurabh Badhwar | Bharath Chandra K S

<packt>

Web Development with Django 6

Third Edition

A definitive guide to building modern Python web applications using Django 6

**Chris Guest, Mark Walker, Ben Shaw, Saurabh Badhwar,
Bharath Chandra K S**

<packt>

Web Development with Django

Third Edition

Copyright © 2026 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Ashwin Nair

Relationship Lead: Sneha Shinde

Content Engineer: Kinnari Chohan

Project Manager: Vishnu Priya R

Technical Editor: Arjun Varma

Copy Editor: Safis Editing

Indexer: Rekha Nair

Proofreader: Kinnari Chohan

Production Designer: Ponraj Dhandapani

Growth Lead: Anamika Singh

First published: January 2021

Second edition: May 2023

Third edition: March 2026

Production reference: 1130326

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83620-207-3

www.packtpub.com

I would like to dedicate this book to Karen. I have immense gratitude for the patience, support, and understanding of a fellow writer.

– Chris Guest

Contributors

About the authors

Chris Guest is an Australian software developer who has worked with Python for 26 years in health, manufacturing, publishing, and logistics. He uses Django as his go-to platform for web development. He is currently researching natural language processing with ancient Anatolian languages.

I am thankful to all my loved ones for being understanding of the time that I have taken from them in order to write this book. It would not be possible without the prior work of Ben Shaw, Saurabh Badhwar, Andrew Bird, and Bharath Chandra K S—my co-authors of the first and second editions. Lastly, I am forever grateful to the editorial team at Packt for honing my skills as a writer.

Mark Walker has been building production Django systems since 2012. As technical lead of Django CMS and an active open source contributor for over a decade, he has helped shape tools used by developers worldwide. He has architected high-traffic platforms for some of the world's largest events and now builds large-scale fantasy sports games serving millions of users, with a focus on scalability, performance, and pragmatic system design.

Ben Shaw is a software engineer based in Auckland, New Zealand. He has worked as a developer for over 16 years and has been building websites with Django since 2007. In that time, his experience has helped many different types of companies, ranging in size from start-ups to large enterprises. He is also interested in machine learning, data science, automating deployments, and DevOps. When not programming, Ben enjoys outdoor sports and spending time with his partner and son.

Saurabh Badhwar is an infrastructure engineer who builds tools and frameworks to enhance developer productivity. A major part of his work involves using Python to develop services that scale to thousands of concurrent users. He is currently employed at LinkedIn, where he works on infrastructure performance tools and services.

Bharath Chandra K S is a passionate software developer with over 14 years of experience in the industry, currently residing in Sydney, Australia. He specializes in Python stack development, including frameworks such as Flask and Django, and has extensive experience with both monolithic and microservice architectures. Bharath has developed various public-facing applications and data-processing backend systems. When not creating software, he enjoys cooking delicious food.

About the reviewers

Anna Makarudze is a software engineer, technical writer, and longtime contributor to the Django community. She has worked with the Django Girls Foundation since 2017 as fundraising coordinator and, since 2020, has served as the lead maintainer of the Django Girls repository and as a trustee of the foundation. She is a former president of the Django Software Foundation. Anna chairs the organizing committee for DjangoCon Africa. She specializes in web development with Python, Django, Django REST Framework, HTML, and JavaScript. She has organized numerous Django Girls events to introduce women to programming with Python and Django. She is committed to helping newcomers learn web development through structured, accessible tutorials.

Abigail Afi Gbadago is vice president of the Django Software Foundation Board and a fellow of the Python Software Foundation, shaping technologies used by millions of developers worldwide. As a senior developer advocate for Django at MongoDB, she drives global adoption and community growth. With over seven years of experience across backend engineering, open source governance, and strategic partnerships, Abigail operates at the intersection of technical systems and business strategy. She served as program chair for DjangoCon US 2025 and regional executor for Black Python Devs, is a Microsoft MVP, and is enrolled in the Stanford LEAD Executive Program (Stanford GSB). Outside work, Afi recharges by returning to her roots in Ghana, spending time with loved ones, and drawing on the vibrant community's good vibes.

Anthony Addae is a Python and Django engineer who cares deeply about open source and the communities that make it thrive. His journey has taken him from mentoring and speaking to contributing more actively within the Django ecosystem across conferences and community initiatives in Ghana and beyond. As a member of the Django Software Foundation and a Djangonaut Space graduate, Anthony enjoys building reliable backend systems while helping create spaces where more people can learn, grow, and contribute.

Table of Contents

Preface	xxiii
Free benefits with your book.....	xxvii
Chapter 1: Introduction to Django	1
Technical requirements • 2	
Your purchase includes a free PDF copy + code bundle	3
Creating a virtual environment with pyenv	3
Installing pyenv on Windows • 3	
Installing pyenv on macOS and Linux • 4	
Creating a virtual environment with pyenv • 4	
Scaffolding a Django project and app	5
Exercise 1.01 – creating a project and app, and starting the development server • 7	
Understanding the model-view-template paradigm	10
Models • 10	
Views • 11	
Templates • 12	
MVT in practice • 12	
An introduction to HTTP • 15	
Processing a request • 20	
Exploring the Django project structure	22
The myproject directory • 23	
Django development server • 24	
Django apps • 25	
PyCharm setup • 26	
Exercise 1.02 – project setup in PyCharm • 26	
Introducing Django views.....	35
Exploring URL mapping details	36

Exercise 1.03 – writing a view and mapping a URL to it • 37	
Working with GET, POST, and QueryDict objects	39
Exercise 1.04 – exploring GET values and QueryDict objects • 42	
Exploring Django settings.....	44
Referring to Django settings in your code • 45	
Finding HTML templates in app directories.....	47
Exercise 1.05 – creating a templates directory and a base template • 47	
Rendering a template with the render function • 49	
Exercise 1.06 – rendering a template in a view • 49	
Rendering variables in templates • 50	
Exercise 1.07 – using variables in templates • 51	
Debugging and dealing with errors	53
Exceptions • 53	
Exercise 1.08 – generating and viewing exceptions • 55	
Debugging • 57	
Exercise 1.09 – debugging your code • 58	
Activity 1.01 – creating a site welcome screen	63
Activity 1.02 – a book search scaffold.....	63
Summary	65
Get this book's PDF copy, code bundle, and more • 65	
 Chapter 2: Models and Migrations	 67

Technical requirements	67
Exploring Django ORM	68
Database configuration and creating Django applications • 68	
Django migration • 70	
Creating Django models and migrations.....	73
Field options • 74	
Primary keys • 77	
Relationships • 79	
<i>Many to one</i> • 80	
<i>Many to many</i> • 81	

<i>One-to-one relationships</i> • 83	
Adding the Review model • 84	
Model methods • 85	
Migrating the reviews app • 87	
Django's database CRUD operations	89
Exercise 2.01 – Creating an entry in the bookr database • 90	
Exercise 2.02 – Using the create() method to create an entry • 92	
Creating an object with a foreign key • 92	
Exercise 2.03 – Creating records for a many-to-one relationship • 93	
Exercise 2.04 – Creating records with many-to-many relationships • 94	
Exercise 2.05 – A many-to-many relationship using the add() method • 95	
Using the create() and set() methods for many-to-many relationships • 96	
Read operations • 97	
Exercise 2.06 – Using the get() method to retrieve an object • 97	
Returning an object using the get() method • 98	
Exercise 2.07 – Using the all() method to retrieve a set of objects • 99	
Retrieving objects by filtering • 100	
Exercise 2.08 – Using the filter() method to retrieve objects • 100	
Filtering by field lookups • 101	
Using pattern matching for filtering operations • 102	
Retrieving objects by using the exclude() method • 103	
Retrieving objects using the order_by() method • 103	
Querying across relationships • 106	
Querying using foreign keys • 107	
Querying using the model name • 107	
Querying across foreign key relationships using the object instance • 107	
Exercise 2.09 – Querying across a many-to-many relationship using the field lookup • 108	
Exercise 2.10 – A many-to-many query using objects • 109	
Exercise 2.11 – A many-to-many query using the set() method • 109	
Exercise 2.12 – Using the update() method • 110	
Exercise 2.13 – Using the delete() method • 111	

Bulk create and bulk update operations	112
Exercise 2.14 – Create multiple records using bulk_create • 112	
Exercise 2.15 – Update multiple records using bulk_update • 113	
Performing complex lookups using Q objects	114
Exercise 2.16 – Perform a complex query using a Q object • 114	
Exercise 2.17 – Verify whether a queryset contains a given object • 115	
Activity 2.01 – Create models for a project management application	116
Populating the Bookr project's database	117
Summary	118
Get this book's PDF copy, code bundle, and more	118
 Chapter 3: Django Views, URL Configuration, and Templates	 119
Technical requirements	120
Working with Django views.....	120
Understanding function-based views • 120	
Understanding class-based views • 121	
URL configuration	122
Exercise 3.01 – Implementing a simple function-based view • 124	
Working with Django templates	127
Exercise 3.02 – Using templates to display a greeting message • 129	
Django template language • 132	
<i>Template variables</i> • 132	
<i>Template tags</i> • 132	
<i>Comments</i> • 133	
<i>Filters</i> • 133	
Exercise 3.03 – Displaying a list of books and reviews • 134	
Template inheritance • 138	
Template styling with Bootstrap • 139	
Exercise 3.04 – Adding template inheritance and a Bootstrap navigation bar • 141	
Activity 3.01 – Implementing the book details view	144
Summary	145
Get this book's PDF copy, code bundle, and more	146

Chapter 4: An Introduction to Django Admin	147
Technical requirements	147
Introducing the Django admin app	148
Creating a superuser account.....	149
Exercise 4.01 – Creating a superuser account •	149
CRUD operations using the Django admin app	152
Create •	152
<i>Activity 4.01 – Creating a user account •</i>	<i>154</i>
Retrieve •	155
Update •	156
Delete •	158
Managing Django users and groups.....	159
Exercise 4.02 – Adding and modifying users and groups through the admin app •	160
Registering models with the admin app	165
Registering the reviews model •	166
Change lists •	167
The Change publisher page •	168
The book change page •	173
Exercise 4.03 – Foreign keys and deletion behavior in the admin app •	177
Customizing the admin interface	179
Site-wide Django admin customizations •	179
<i>Examining the AdminSite object from the Python shell •</i>	<i>181</i>
<i>Subclassing AdminSite – a first approach •</i>	<i>182</i>
<i>Subclassing AdminSite – a better way •</i>	<i>185</i>
<i>Activity 4.02 – Customizing the AdminSite object •</i>	<i>186</i>
Customizing the ModelAdmin classes	190
The list display fields •	191
The display decorator •	197
The filter •	198
Exercise 4.04 – Adding the date_list_filter and date_hierarchy filters •	200
The search bar •	203

Excluding and grouping fields • 206	
Activity 4.03 – Customizing the model admins • 209	
Summary	212
Get this book's PDF copy, code bundle, and more	212
Chapter 5: Serving Static Files	213
Technical requirements	215
Django and static file serving	215
Introduction to static files finders	216
Static file finders – used during a request • 217	
AppDirectoriesFinder • 217	
<i>Static file namespacing • 218</i>	
Exercise 5.01 – serving a file from an app directory • 220	
Generating static URLs with the static template tag	224
The static template tag • 225	
An example of the static template tag • 226	
Exercise 5.02 – using the static template tag • 229	
FileSystemFinder.....	232
Exercise 5.03 – serving from a project static directory • 233	
Static file finders – use during collectstatic.....	236
Exercise 5.04 – collecting static files for production • 238	
STATICFILES_DIRS prefixed mode	240
The findstatic command.....	243
Exercise 5.05 – finding files using findstatic • 244	
Serving the latest files (for cache invalidation)	248
Exercise 5.06 – exploring the ManifestFilesStorage storage engine • 251	
Custom storage engines.....	255
Activity 5.01 – adding a Reviews logo • 258	
Activity 5.02 – CSS enhancements • 260	
Activity 5.03 – adding a global logo • 262	
Summary	264
Get this book's PDF copy, code bundle, and more	265

Chapter 6: Forms	267
Technical requirements	268
Background to forms	268
The form element • 268	
Form security	270
Input validation • 270	
Cross-site request forgery protection • 270	
SQL injection prevention • 271	
Content Security Policy (CSP) and XSS attacks • 272	
HTTPS and security headers • 272	
Exercise 6.01 – Building a form in HTML • 273	
Accessing data in the view • 280	
Exercise 6.02 – Working with POST data in a view • 281	
Choosing between GET and POST • 286	
Why use GET when we can put parameters in the URL mappings? • 288	
The Django Forms library	289
Defining a form • 290	
Rendering a form in a template • 301	
Exercise 6.03 – Building and rendering a Django form • 307	
Validating forms and retrieving Python values	311
Exercise 6.04 – Validating forms in a view • 315	
Built-in field validation • 318	
Exercise 6.05 – Adding extra field validation • 319	
Activity 6.01 – Book Search	322
Summary	326
Get this book's PDF copy, code bundle, and more	327
Chapter 7: Advanced Form Validation and Model Forms	329
Technical requirements	330
Custom field validation and cleaning	330
Custom validators • 331	

Cleaning methods • 332	
Multi-field validation • 334	
Exercise 7.01 – Custom clean and validation methods • 338	
Adding placeholders and initial values	346
Exercise 7.02 – Placeholders and initial values • 348	
Creating or editing Django models	350
The ModelForm class • 352	
Exercise 7.03 – Creating and editing a publisher • 357	
Activity 7.01 – Styling and integrating the publisher form • 363	
Activity 7.02 – Review creation UI • 366	
Summary	372
Get this book's PDF copy, code bundle, and more	373
 Chapter 8: Media Serving and File Uploads	 375
Technical requirements	376
Settings for media uploads and serving	376
Serving media files in development • 376	
Exercise 8.01 – configuring media storage and serving • 377	
Context processors and using MEDIA_URL in templates • 381	
Exercise 8.02 – template settings and using MEDIA_URL in templates • 384	
File uploads using HTML forms.....	386
Working with uploaded files in a view • 388	
<i>Security and trust of browsers' sent values • 390</i>	
Exercise 8.03 – file upload and download • 392	
File uploads with Django forms	395
Exercise 8.04 – file uploads with a Django form • 397	
Image uploads with Django forms.....	402
Resizing an image with Pillow • 404	
Exercise 8.05 – image uploads using Django forms • 405	
Serving uploaded (and other) files using Django.....	408
Storing files on model instances • 410	
Storing images on model instances • 413	

Working with FieldFile • 414	
<i>Custom storage engines • 415</i>	
<i>Reading a stored FieldFile • 415</i>	
<i>Storing existing files or content on FileField • 416</i>	
<i>Writing PIL images to ImageField • 418</i>	
Referring to media in templates • 421	
Exercise 8.06 – FileField and ImageField on models • 421	
ModelForm and file uploads 426	
Exercise 8.07 – file and image uploads using ModelForm • 429	
Handling file saving • 432	
Activity 8.01 – image and PDF upload for books..... 433	
Activity 8.02 – displaying the cover and sample link..... 438	
Summary 440	
Get this book's PDF copy, code bundle, and more 441	
 Chapter 9: Sessions and Authentication 443	
Technical requirements 444	
Middleware..... 444	
Middleware modules • 445	
Implementing authentication views and templates • 447	
Exercise 9.01 – Repurposing the Admin app login template • 453	
Password storage in Django 457	
The profile page and the request.user object • 457	
Exercise 9.02 – Adding a profile page • 457	
Authentication decorators and redirection 459	
Exercise 9.03 – Adding authentication decorators to the views • 463	
Enhancing templates with authentication data..... 466	
Exercise 9.04 – Toggling login and logout links in the base template • 466	
Activity 9.01 – Authentication-based content using conditional blocks in templates • 468	
Sessions 470	
The session engine • 470	

Do you need to flag cookie content? • 471	
Pickle or JSON storage? • 472	
Exercise 9.06 – Examining the session key • 473	
Storing data in sessions • 478	
Exercise 9.06 – Storing recently viewed books in sessions • 479	
Exercise 9.07 – Using session storage for the Search for Books page • 484	
Summary	485
Get this book's PDF copy, code bundle, and more	486
 Chapter 10: Advanced Django Admin and Customizations	 487

Technical requirements	488
Customizing the admin site	488
A customized user model	488
Extending the user model • 489	
Subclassing AbstractUser.....	490
<i>Scenario 1: Adding additional fields to the User class • 492</i>	
<i>Scenario 2: Replacing the Username key • 493</i>	
<i>Scenario 3: Modifying the password • 493</i>	
<i>Steps in replacing the auth user model • 493</i>	
Exercise 10.1: Replacing User with a TitledUser model • 494	
Activity 10.1: Implementing the reviewer profile model • 497	
Form enhancements with ModelAdmin attributes • 498	
<i>filter_horizontal / filter_vertical • 498</i>	
<i>Inline forms • 499</i>	
<i>radio_fields • 503</i>	
<i>readonly_fields • 503</i>	
<i>get_readonly_fields • 503</i>	
<i>list_editable • 504</i>	
Exercise 10.2 • 504	
Pagination customizations • 506	
Search and filter customizations • 507	
Activity 10.2 • 507	

Adding views to the admin site	508
Creating the view function • 509	
Accessing common template variables • 510	
Mapping URLs for the custom view • 510	
Restricting custom views to the admin site • 511	
Exercise 10.03 – adding custom views to the admin site • 511	
Passing additional keys to templates using template variables • 515	
Configuring an email server with Django	516
Automating admin with Django Tasks	518
Activity 10.03 – building a custom admin dashboard with a built-in search • 521	
Exercise 10.5 • 523	
Summary	528
Get this book's PDF copy, code bundle, and more • 529	
 Chapter 11: Advanced Templating and Class-Based Views	 531
Technical requirements • 532	
Template filters	532
Custom template filters	534
Creating custom template filters • 534	
<i>Setting up the directory for storing template filters • 534</i>	
<i>Setting up the template library • 535</i>	
Implementing the custom filter function • 535	
Using custom filters inside templates • 536	
String filters	536
Exercise 11.01 – Creating a custom template filter • 537	
Template tags.....	541
The types of template tags • 542	
Simple tags • 542	
Creating a simple template tag • 542	
<i>Setting up the directory • 543</i>	
<i>Setting up the template library • 543</i>	
<i>Implementing a simple template tag • 543</i>	

<i>Using simple tags inside templates • 544</i>	
Exercise 11.02 – Creating a custom simple tag • 544	
<i>Passing the template context in a custom template tag • 547</i>	
Inclusion tags • 548	
<i>Implementing inclusion tags • 548</i>	
<i>Using an inclusion tag inside a template • 549</i>	
Exercise 11.03 – Building a custom inclusion tag • 549	
Activity 11.01 – Rendering details on the user profile page using inclusion tags • 552	
Template partials 554	
Activity 11.02 – Rendering a book list item with template partials • 556	
Django views..... 556	
Class-based views..... 557	
Exercise 11.04 – Creating a book catalog using a CBV • 559	
CRUD operations with CBVs • 565	
<i>The Create view • 565</i>	
The Read view • 566	
<i>The Update view • 567</i>	
<i>The Delete view • 569</i>	
Summary 570	
Get this book's PDF copy, code bundle, and more • 571	
 Chapter 12: Building a REST API 573	
Technical requirements 574	
Understanding REST APIs..... 574	
DRF 575	
Installation and configuration • 575	
Functional API views • 576	
Exercise 12.01 – Creating a simple REST API • 576	
Understanding serializers..... 578	
Exercise 12.02 – Creating an API view to display a list of books • 579	
Class-based API views and generic views • 582	
Model serializers • 583	

Exercise 12.03 – Creating class-based API views and model serializers • 584	
Activity 12.01 – Creating an API endpoint for a top contributors pageShould • 587	
Simplifying the code using ViewSets	588
URL configuration using routers and ViewSets • 588	
Exercise 12.04 – Using ViewSets and routers • 589	
Implementing authentication.....	592
Token-based authentication • 593	
Exercise 12.05 – Implementing token-based authentication for bookr APIs • 593	
Summary	599
Get this book's PDF copy, code bundle, and more • 599	
 Chapter 13: Generating CSV, PDF, and Other Binary Files	 601
Working with CSV files in Python	602
Working with Python's csv module.....	602
Writing to CSV files using Python • 602	
Exercise 13.01 – Create a review summary CSV file • 604	
Binary file formats for data exports • 607	
Working with XLSX files using the XlsxWriter package • 608	
<i>XLSX files • 608</i>	
<i>The XlsxWriter Python package • 609</i>	
<i>Creating a workbook • 610</i>	
<i>Creating a worksheet • 610</i>	
<i>Writing data to the worksheet • 610</i>	
<i>Writing data to the workbook • 611</i>	
<i>Formatting a spreadsheet • 611</i>	
<i>Exercise 13.02 – Developing a formatted spreadsheet • 615</i>	
ZIP files	619
Working with PDF files in Python	621
Exercise 13.03 – Generating a PDF version of a web page in Python • 622	
Activity 13.01 – Adding additional functionality to a View • 626	
Generating graphs with Plotly	628
<i>Installation • 629</i>	

<i>Setting up a figure</i> • 629	
<i>Generating a plot</i> • 630	
<i>Rendering a plot on a web page</i> • 631	
Advanced graphs – bubble plots • 631	
Exercise 13.4 – Generating graphs in Python • 633	
Generating images in Python with Pillow • 636	
Activity 13.02 – Creating dynamic graphics in Django with Pillow • 638	
Summary	641
 Chapter 14: Testing Your Django Applications	 643
Importance of testing	644
Automation testing	644
Testing in Django	645
Implementing test cases • 646	
Unit testing in Django • 646	
Utilizing assertions • 646	
Exercise 14.01 – Writing a simple unit test • 648	
<i>Types of assertions</i> • 649	
Performing pre-test setup and cleanup after every test case run • 650	
Testing Django models	652
Exercise 14.02 – Testing Django models • 652	
Testing Django views.....	656
Exercise 14.03 – Writing unit tests for Django views • 657	
Testing views with authentication • 661	
Exercise 14.04 – Writing test cases to validate authenticated users • 661	
Django RequestFactory	666
Exercise 14.05 – Using RequestFactory to test views • 666	
Testing class-based views • 669	
Test case classes in Django	670
The SimpleTestCase class • 670	
The TransactionTestCase class • 671	
The LiveServerTestCase class • 671	

Modularizing test code • 671	
Running tests faster using parallelization	672
Activity 14.01 – Testing models and views in Bookr	673
Summary	674
Get this book's PDF copy, code bundle, and more	675
 Chapter 15: Deploying a Django Project	 677
Technical requirement	677
Server architecture	677
Gunicorn • 679	
Whitenoise • 679	
The deployment checklist.....	679
Run manage.py check --deploy • 680	
Switch away from manage.py runserver • 681	
Critical settings • 681	
<i>SECRET_KEY</i> • 682	
<i>DEBUG</i> • 682	
Environment-specific settings • 682	
<i>ALLOWED_HOSTS</i> • 682	
<i>CACHES</i> • 683	
<i>DATABASES</i> • 683	
<i>EMAIL_BACKEND</i> and related settings • 683	
<i>HTTPS</i> • 684	
<i>Performance optimizations</i> • 684	
<i>Error reporting</i> • 685	
<i>Customize the default error views</i> • 685	
Exercise 15.01 – Configuring the Bookr project for deployment • 686	
Deployment options for Django	690
Manual deployment • 690	
Containerized deployment • 691	
Platform-as-a-service • 691	
Exercise 15.02 – Deploying the Bookr project • 692	

Summary	710
Get this book's PDF copy, code bundle, and more • 711	
Other Books You May Enjoy	714

Index	717
--------------	------------

Preface

Do you want to develop reliable and secure applications that stand out from the crowd without spending hours on boilerplate code? You've made the right choice by trusting the Django framework, and this book will tell you why. Often referred to as a "batteries included" web development framework, Django comes with all the core features needed to build a standalone application. *Web Development with Django 6* will take you through all the essential concepts and help you explore its power to build real-world applications using Python.

Throughout the book, you'll get to grips with the major features of Django by building a website called Bookr – a repository for book reviews. This end-to-end case study is split into a series of bite-sized projects presented as exercises and activities, allowing you to challenge yourself in an enjoyable and attainable way. As you advance, you'll acquire various practical skills, including how to serve static files to add CSS, JavaScript, and images to your application, implement forms to accept user input, and manage sessions to ensure a reliable user experience. You'll cover everyday tasks that are part of the development cycle of a real-world web application.

By the end of this Django book, you'll have the skills and confidence to creatively develop and deploy your own projects.

Who this book is for

This book is for programmers looking to enhance their web development skills using the Django framework. To fully understand the concepts explained in this book, basic knowledge of Python programming, as well as familiarity with JavaScript, HTML, and CSS, is assumed.

What this book covers

Chapter 1, An Introduction to Django, starts by getting a Django project set up almost immediately. You'll learn how to bootstrap a Django project, respond to web requests, and use HTML templates.

Chapter 2, Models and Migrations, introduces Django data models, the method of persisting data to a SQL database.

Chapter 3, URL Mapping, Views, and Templates, builds on the techniques that were introduced in *Chapter 1, An Introduction to Django*, and explains in greater depth how to route web requests to Python code and render HTML templates.

Chapter 4, An Introduction to Django Admin, shows how to use Django's built-in Admin GUI to create, update, and delete data stored by your models.

Chapter 5, Serving Static Files, explains how to enhance your website with styles and images and how Django makes managing these files easier.

Chapter 6, Forms, shows you how to collect user input through your website by using Django's Forms module.

Chapter 7, Advanced Form Validation and Model Forms, builds upon *Chapter 6, Forms*, by adding more advanced validation logic to make your forms more powerful.

Chapter 8, Media Serving and File Uploads, shows how to further enhance sites by allowing your users to upload files and serve them with Django.

Chapter 9, Sessions and Authentication, introduces the Django session and shows you how to use it to store user data and authenticate users.

Chapter 10, Advanced Django Admin and Customization, continues from *Chapter 4, An Introduction to Django Admin*. Now that you know more about Django, you can customize Django admin with advanced features. You will also learn about configuring SMTP for Django and Django's Tasks framework.

Chapter 11, Advanced Templating and Class-Based Views, lets you see how to reduce the amount of code you need to write using some of Django's advanced templating features and classes.

Chapter 12, Building a REST API, looks at how to add a REST API to Django to allow programmatic access to your data from different applications.

Chapter 13, Generating CSV, PDF, and Other Binary Files, further expands the capabilities of Django by showing how you can use it to generate more than just HTML.

Chapter 14, Testing Your Django Applications, is an important part of real-world development. This chapter shows how to use the Django and Python testing frameworks to validate your code.

Chapter 15, Deploying a Django Project, finishes the project by showing you how to deploy your project to a virtual server.

To get the most out of this book

In order to follow the instructions in this book, it is assumed that you will be acquainted with writing simple Python code and debugging errors. Also, a basic familiarity with HTML, CSS, and JavaScript is assumed.

The code in this book was tested with Python 3.12 and Django 6.0. For best results, work with these versions.

Download the example code files

This book includes a complete downloadable code bundle containing all the example projects and files used throughout the chapters. We recommend downloading the bundle so you can follow along smoothly and experiment with the examples.

Use the bundle as a practical starting point. Modify it, extend it, and apply what you learn by creating your own variations as you progress through the chapters.

Get the code bundle

If you bought the book directly from Packt:

1. Go to packtpub.com
2. Click your profile picture and select Your Orders
3. Find this book and click Download Code

If you bought this book from Amazon or any other channel partner:

1. Go to packtpub.com/unlock or scan the following QR code:



2. Search for this book
3. Sign up or log in to your free Packt account
4. Upload your proof of purchase and download the code bundle locally

Usage note: You're free to use and modify this code for personal learning and non-commercial projects.

Download the color images

Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to *Free benefits with your book section* at the end of the *Preface* to unlock your PDF copy.

Conventions used

There are a number of text conventions used throughout this book.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "Execute the `manage.py runserver` command."

A block of code is set as follows:

```
from django.apps import AppConfig

class ReviewsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'reviews'
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
```

Any command-line input or output is written as follows:

```
django-admin startproject bookr
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: "Open the **Preferences** (macOS) or **Settings** (Windows/Linux) window inside PyCharm."

Note

Warnings or important notes appear like this.

Tip

Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at customercare@packt.com and mention the book's title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packt.com/submit-errata>, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packt.com/>.

Free benefits with your book

This book includes free benefits designed to support your learning and help you apply what you learn effectively. Activate them now for instant access (see the *How to unlock* section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:



Complete Code Bundle

Download the full source code for this book's examples and projects.



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

How to unlock

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

Share your thoughts

Once you've read *Web development with Django 6*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.



<https://packt.link/r/1836202075>

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

1

Introduction to Django

"The web framework for perfectionists with deadlines." It's a tagline that aptly describes Django, a framework that has been around for nearly 20 years. It is battle-tested and widely used, with more and more people using it every day. All this might make you think that Django is old and no longer relevant. On the contrary, its longevity has proved that its API is reliable and consistent, and even those who learned Django v1.0 in 2007 can mostly write the same code for Django 6 today. Django is still in active development, with bug fixes and security patches being released regularly.

Like Python, the language in which it is written, Django is easy to learn, yet powerful and flexible enough to grow with your needs. It is a "batteries-included" framework – in other words, you do not have to find and install many other libraries or components to get your application up and running. Other frameworks, such as Flask or Pylons, require manually installing third-party frameworks for database connections or template rendering. Instead, Django has built-in support for database querying, URL mapping, and template rendering (we'll go into detail on what these mean soon). However, just because Django is easy to use doesn't mean it is limited. Django is used by many large sites, including Disqus, Instagram, Mozilla, Spotify, Open edX, OpenStack, and National Geographic.

Where does Django fit into the web? When talking about web frameworks, you might think of frontend JavaScript frameworks such as ReactJS, Angular, or Vue. These frameworks are used to enhance or add interactivity to already-generated web pages. Django sits at the layer beneath these tools and is instead responsible for routing a URL, fetching data from databases, rendering templates, and handling form input from users. However, this does not mean you must pick one or the other; JavaScript frameworks can be used to enhance the output from Django or interact with a REST API generated by Django.

In this book, we will build a Django project using the same methods that professional Django developers use every day. The project is called **Bookr** and allows you to browse and add books

and book reviews. This book is divided into four parts. In the first part, we'll start with the basics of scaffolding a Django app, quickly build some pages, and serve them with the Django development server. You'll be able to add data to the database using the Django admin site.

This chapter introduces you to Django and its role in web development. You will begin by understanding how the model-view-template paradigm works and how Django processes HTTP requests and responses. Equipped with the basic concepts, you'll create your first Django project, called Bookr, an application for adding, viewing, and managing book reviews. It's an application you'll keep enhancing and adding features to throughout this book. You will then learn about the `manage.py` command (used to orchestrate Django actions). You will use this command to start the Django development server and test whether the code you wrote so far works as expected. You will also learn how to work with PyCharm, a popular Python IDE, which you'll be using throughout this book. You will use it to write code that returns a response to your web browser. Finally, you'll learn how to use PyCharm's debugger to troubleshoot problems with your code. By the end of this chapter, you'll have the necessary skills to start creating projects using Django.

We will cover the following topics in this chapter:

- Scaffolding a Django project and app
- Understanding the model-view-template paradigm
- Exploring the Django project structure
- Introducing Django views
- Exploring URL mapping details
- Exploring Django settings
- Finding HTML templates in app directories
- Debugging and dealing with errors
- Activity 1.01 – creating a site welcome screen
- Activity 1.02 – a book search scaffold

By the end of the book, you will have enough experience to design and build your own Django project from start to finish.

Technical requirements

Throughout this book, you will be writing code. If you need to refer to the completed code for this chapter, you can find it in the `Chapter01` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Your purchase includes a free PDF copy + code bundle

Your purchase includes a DRM-free PDF copy of this book, the code bundle, and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

Creating a virtual environment with pyenv

Before diving deeply into the theory behind Django paradigms and HTTP requests, we'll show you how easy it is to get a Django project up and running. After this first section and exercise, you will have created a Django project, made a request to it with your browser, and seen the response.

As a Python developer, you may be working with many applications on your system that have different and even conflicting dependencies. The examples in this book use Python 3.12 and Django 6.0. However, you may need to maintain other versions of Python for your other projects that may not even be compatible with Django 6.0.

In addition, operating systems such as macOS and Linux ship with Python installed, and it is frequently used as a scripting language by operating systems. For this reason, it is best practice to develop with a different Python installation that will not disrupt operating system functionality if conflicts are introduced.

For these reasons, virtual environments are an ideal technique for developing with multiple environments. When you install Python, Django, and its dependencies, they are quarantined in their own user-specific environment and do not affect the behavior of other Python software that you are developing and using.

In this book, we will use pyenv to configure Python virtual environments.

Installing pyenv on Windows

pyenv for Windows is available from <https://github.com/pyenv-win/pyenv-win>. It can be installed from the PowerShell with this command:

```
Invoke-WebRequest-UseBasicParsing-Uri"https://raw.githubusercontent.com/pyenv-win/pyenv-win/master/pyenv-win/install-pyenv-win.ps1"-OutFile"./install-pyenv-win.ps1"; &"./install-pyenv-win.ps1"
```

Installing pyenv on macOS and Linux

If you use the Homebrew package manager on macOS, you can install these two packages for the required functionality:

```
brew install pyenv pyenv-virtualenv
```

Otherwise, on macOS and Linux, pyenv can be installed by running the following at the command line.

```
curl https://pyenv.run | bash
```

On Linux, you will need to include these lines in your `.bashrc` file.

```
export PYENV_ROOT="$HOME/.pyenv"
[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

On macOS, include these lines in your `.zshrc` file.

```
export PYENV_ROOT="$HOME/.pyenv"
command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

Creating a virtual environment with pyenv

Now that we have installed pyenv, we can use it to create an environment for our Django project. As we are using Python 3.12 in this book, we will start by installing this version under our local pyenv environment. We do this by using the `pyenv install` command.

```
pyenv install 3.12
```

We can check the setup using the `pyenv versions` command.

```
% pyenv versions
* system (set by /Users/chrisguest/.pyenv/version)
  3.12.0
```

With our desired Python version installed, we can create a virtual environment. We will call it `djangoenv`.

```
pyenv virtualenv 3.12 djangoenv
```

Now, with our virtual environment created, we will activate it so that subsequent commands in the terminal are interpreted using this environment using the `pyenv local` command.

```
pyenv local djangoenv
```

With our environment created, we can install the Python modules that we will need for the project. In our case, as we are developing Django projects using Django 5.1, we will install this version of Django using the `pip install` command.

```
pip install django==6.0
```

Most of the exercises and activities in this book have the necessary modules listed in a `requirements.txt` file. For example, this chapter has this file:

```
Web-Development-with-Django-Third-Edition/Chapter01/requirements.txt
```

In order to install the modules from a specific `requirements.txt` file, you can use this command:

```
pip install -r requirements.txt
```

If you need to switch back to the system version of Python, you can use this command.

```
% pyenv global system
```

With a virtual environment created and activated, we are ready to get working with Django.

Scaffolding a Django project and app

Before diving deeply into the theory behind Django paradigms and HTTP requests, we'll show you how easy it is to get a Django project up and running. After this first section and exercise, you will have created a Django project, made a request to it with your browser, and seen the response.

A Django project is a directory that contains all the data for your project – code, settings, templates, and assets. It is created and scaffolded by running the `django-admin` command on

the command line with the `startproject` argument and providing the project name. For example, to create a Django project with the name `myproject`, the command that is run is as follows:

```
django-admin startproject myproject
```

This will create the `myproject` directory, which Django populates with the necessary files to run the project. Inside the `myproject` directory are two files (shown in *Figure 1.1*):

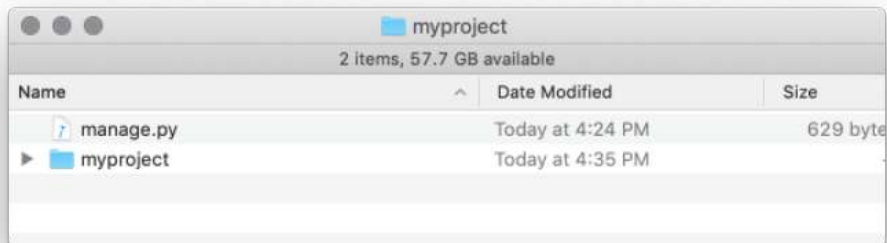


Figure 1.1 – The project directory for myproject

`manage.py` is a Python script that is executed at the command line to interact with your project. We will use it to start the **Django development server**, a development web server you will use to interact with your Django project on your local computer. Like `django-admin`, commands are passed in on the command line. Unlike `django-admin`, this script is not mapped in your system path, so we must execute it using Python. We will need to use the command line to do that. For example, inside the project directory, we run the following command:

```
python manage.py runserver
```

This passes the `runserver` command to the `manage.py` script, which starts the Django dev server. We will examine more of the commands that `manage.py` accepts later. When interacting with `manage.py` in this way, we call these management commands. For example, we might say that we are "executing the `runserver` management command."

The `startproject` command also created a directory with the same name as the project – in this case, `myproject`. This is a Python package that contains settings and some other configuration files that your project needs to run. We will examine its contents later.

After starting the Django project, the next thing to do is to start a Django app. We should try to segregate our Django project into different apps, grouped by functionality. For example, with Bookr, we will have a *reviews* app. This will hold all the code, HTML, assets, and database classes specific to working with book reviews. If we decide to expand Bookr to sell books as well, we might add a store application, containing the files for the bookstore. Apps are created with the `startapp` management command, passing in the application name. Here is an example:

```
python manage.py startapp myapp
```

This creates the app directory (`myapp`) inside the project directory. Django automatically populates this with files for the app that are ready to be filled in when you start developing. We'll examine these files and discuss what makes a good app in the *Django apps* section.

Now that we've introduced the basic commands to scaffold a Django project and application, let's put them into practice by starting the *bookr* project in the first exercise of this book.

Exercise 1.01 – creating a project and app, and starting the development server

As mentioned before, throughout this book, we will be building a book review website named Bookr. It will allow you to add fields for publishers, contributors, books, and reviews. A publisher will publish one or more books, and each book will have one or more contributors (author, editor, co-author, and so on). Only admin users will be allowed to modify these models. Once a user has signed up for an account on the site, they will be able to start adding reviews for a book.

In this exercise, you will scaffold the *bookr* Django project, test that Django is working by running the development server, and then create the *reviews* Django app.

You should already have a virtual environment set up with Django installed. To know how to do that, you can refer to the *Preface*. Once you're ready, let's start by creating the *bookr* project:

1. Open the terminal and run the command to create the *bookr* project directory and the default subfolders:

```
django-admin startproject bookr
```

This command does not generate any output but will create a folder called `bookr` inside the directory in which you ran the command. You can look inside this directory and see the items we described before for the `myproject` example – the `bookr` package directory and the `manage.py` file.

We can now test that the project and Django are set up correctly by running the Django development server. Starting the server is done with the `manage.py` script.

2. In your terminal (or Command Prompt), change to the bookr project directory (using the `cd` command), and then run the `manage.py runserver` command as follows:

```
python manage.py runserver
```

This command starts the Django development server. You should get output similar to the following:

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly
until you apply the migrations for app(s): admin, auth, contenttypes,
sessions.
Run 'python manage.py migrate' to apply them.
December 06, 2025 - 13:10:57
Django version 6.0, using settings 'bookr.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

You will probably receive some warnings about unapplied migrations, but that's okay for now.

3. Open up a web browser and go to `http://127.0.0.1:8000/`, which will show you the Django welcome screen (*Figure 1.2*). If you see this, you will know that your Django project was created successfully and all is working fine for now:

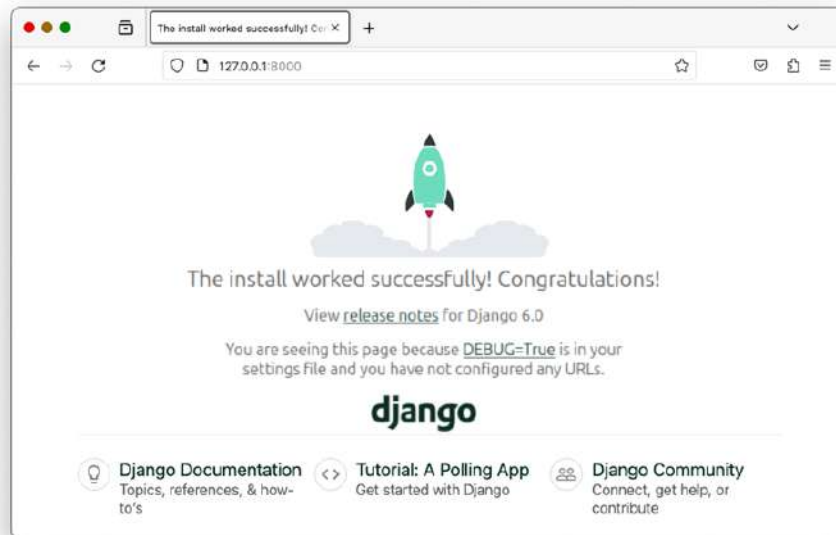


Figure 1.2 – The Django welcome screen

4. Go back to your terminal and stop the development server running by typing `Ctrl + C`.
5. We'll now create the *reviews* app for the *bookr* project. In your terminal, make sure you are in the *bookr* project directory, and then execute the following command to create the *reviews* app:

```
python manage.py startapp reviews
```

Note

After creating the *reviews* app, the files in your *bookr* project directory will look like the ones in the Chapter01 folder in this book's GitHub repo.

There is no output if the command was successful, but a *reviews* app directory has been created. You can look inside this directory to see the files that were created – the migrations directory, `admin.py`, `models.py`, and so on. We'll examine these in detail in the *Django apps* section.

In this exercise, we created the *bookr* project, tested that the project was working by starting the Django development server, and then created the *reviews* app for the project. Now that

you've had some hands-on time with a Django project, we'll return to some of the theory behind Django's design, and HTTP requests and responses.

Understanding the model-view-template paradigm

A common design pattern in application design is **Model View Controller (MVC)**, where the model of an application (its data) is displayed in one or more views, and a controller marshals interaction between the model and view. Django follows a different, yet similar, paradigm called **Model-View-Template (MVT)**.

Like MVC, MVT also uses models for storing data. However, with MVT, a view will query a model and then render it with a template. Usually, with MVC languages, all three components need to be developed with the same language. With MVT, the template can be in a different language. In the case of Django, models and views are written in Python, and the template is written in HTML. This means that a Python developer could work on the models and views, while a specialist HTML developer works on the HTML. We'll first explain models, views, and templates in more detail and then look at some example scenarios where they are used.

Models

Django models define data for your application and provide an abstraction layer to a SQL database accessed through an **Object Relational Mapper (ORM)**. An ORM lets you define your data schema (classes, fields, and their relationships) using Python code, without needing an understanding of the underlying database. Basically, this means you can define your database layer in Python code and Django will take care of generating SQL queries for you. The ORM will be discussed in detail in *Chapter 2, Models and Migrations*.

Note

SQL stands for **Structured Query Language** and is a way of describing a type of database that stores its data in tables, with each table having several rows. Think of each table as being like an individual spreadsheet. Unlike a spreadsheet, though, relationships can be defined between the data in each table. You can interact with data by executing SQL queries (often referred to as just queries when talking about databases). Queries allow you to retrieve data (`SELECT`), add or change data (`INSERT` and `UPDATE`, respectively), and remove data (`DELETE`). There are many SQL database servers to choose from, such as SQLite, PostgreSQL, MySQL, and Microsoft SQL Server. Much of the SQL syntax is similar between each database, but there can be some differences in dialect. Django's ORM takes care of these differences for you – when we start coding, we will use the SQLite database to store data on disk, but later, when we deploy to a server, we will switch to PostgreSQL, and won't need to make any code changes.

Normally, when querying a database, the results come back as primitive Python objects (for example, lists of strings, ints, floats, or bytes). When using the ORM, results are automatically converted into instances of the model classes you have defined. Using an ORM means that you are automatically protected from a type of vulnerability known as a SQL injection attack.

If you're more familiar with databases and SQL, you always have the option of writing your own queries too.

Views

A Django view is where most of the logic for your application is defined. When a user visits your site, their web browser will send a request to retrieve data from your site (we will go into more detail on what an HTTP request is and what information it contains in the next section). A view is a function that you write that will receive this request in the form of a Python object (specifically, a Django `HttpRequest` object). It is up to your view to decide how it should respond to the request and what it should send back to the user. Your view must return an `HttpResponse` object that encapsulates all the information being provided to the client – content, HTTP status, and other headers.

The view can also optionally receive information from the URL of the request – for example, an ID number. A common design pattern of a view is to query a database via the Django ORM, using an ID that is passed into your view. Then, the view can render a template (there'll be more on this shortly) by providing it with data from the model retrieved from the database.

The rendered template becomes the content of the `HttpResponse` object and is returned from the view function. Django takes care of the communication of the data back to the browser.

Templates

A template is a **HyperText Markup Language (HTML)** file (usually, any text file can be a template) that contains special placeholders that are replaced by variables your application provides. For example, your application could render a list of items in either a gallery layout or a table layout. Your view would fetch the same models for either one, but would be able to render a different HTML file with the same information to present the data differently. Django emphasizes safety, so it will take care of automatically escaping variables for you. For example, the `<` and `>` symbols (among others) are special characters in HTML. If you try to use them in a variable, then Django automatically encodes them so that they render correctly in a browser.

MVT in practice

We'll now look at some examples to illustrate how MVT works in practice. In the examples, we have a `Book` model that stores information about different books, and a `Review` model that stores information about different reviews of books.

In the first example, we want to be able to edit the information about `Book` or `Review`. Take the first scenario of editing a book's details. We would have a view to fetch the `Book` data from the database and provide the `Book` model. Then, we would pass context information containing the `Book` object (and other data) to a template that would show a form to capture the new information. The second scenario (editing a review) is similar – fetch a `Review` model from the database and then pass the `Review` object and other data to a template to display an edit form. These scenarios might be so similar that we can reuse the same template for both, as shown in *Figure 1.3*.

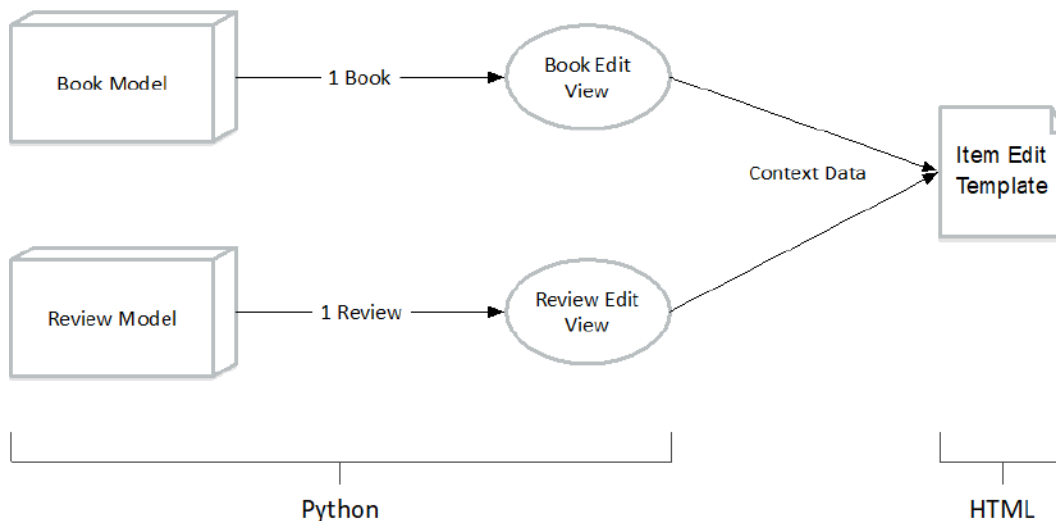


Figure 1.3 – Editing a single book or review

You can see here that we use two models, two views, and one template. Each view fetches a single instance of its associated model, but they can both use the same template, which is a generic HTML page to display a form. Views can provide extra context data to slightly alter the display of the template for each model type. Also illustrated in the diagram are the parts of the code that are written in Python and those that are written in HTML.

In the second example, we want to be able to show a user a list of the books or reviews that are stored in the application. Furthermore, we want to allow the user to search for books and get a list of all that match their criteria. We will use the same two models as the previous example (Book and Review), but we will create new views and templates. Since there are three scenarios, we'll use three views this time – the first fetches all books, the second fetches all reviews, and the last searches for books based on some search criteria. Once again, if we write a template well, we might be able to just use a single HTML template again, as shown in *Figure 1.4*.

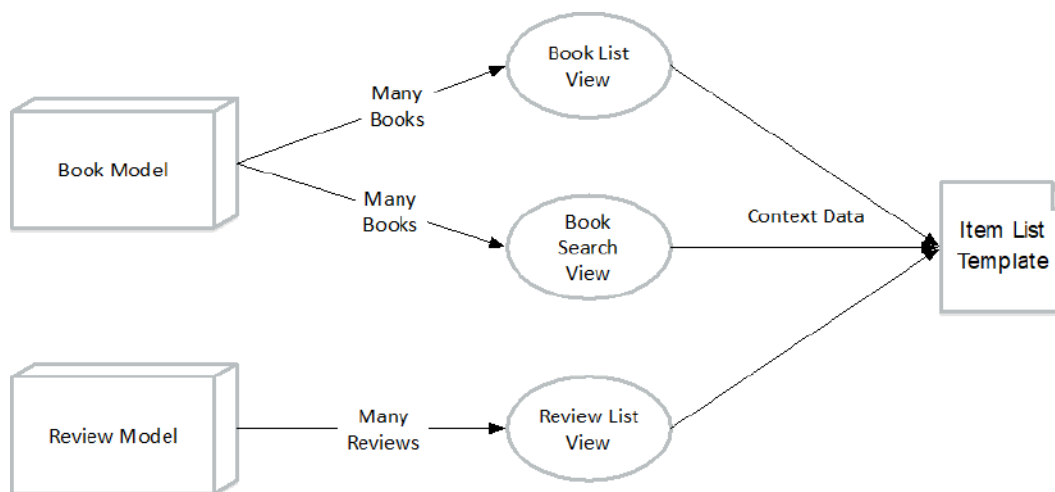


Figure 1.4 – Viewing multiple books or reviews

The Book and Review models remain unchanged from the previous example; the three views will fetch many (zero or more) books or reviews. Then, each view can use the same template, which is a generic HTML file that iterates over a list of objects it is provided and renders them. Once again, the views can send extra data in the context to alter how the template behaves, but the majority of the template will be as generic as possible.

In Django, a model does not always need to be used to render an HTML template. A view can generate the context data itself and render a template with it, without requiring any model data. Figure 1.5 shows a view sending data straight to a template.

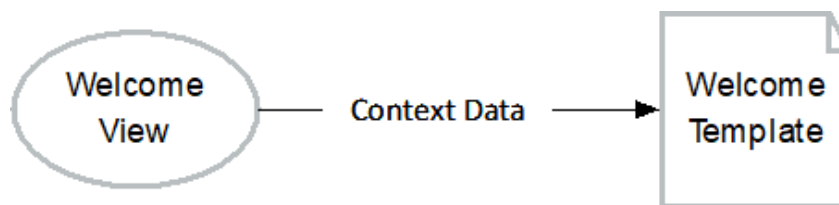


Figure 1.5 – A view sending data to a template without a model

In this example, there is a **Welcome View** to welcome a user to the site. It doesn't need any information from the database, so it can just generate the context data itself. The context data depends on the type of information you want to display – for example, you could pass the user information to greet them by name if they are logged in.

It is also possible for a view to render a template without any context data. This can be useful if you have static information in an HTML file that you want to serve.

Now that you have been introduced to MVT in Django, we can look at how Django processes an HTTP request and generates an HTTP response. However, we first need to explain in more detail what HTTP requests and responses are and what information they contain. We will see this in the next section.

An introduction to HTTP

Let's say someone wants to visit your web page. They type in its URL or click a link to your site from a page they are already on. Their web browser creates an HTTP request, which is sent to the server hosting your website. Once a web server receives the HTTP request from your browser, it can interpret it and then send back a response. The response that the server sends might be simple, such as just reading an HTML or image file from disk and sending it. Alternatively, the response might be more complex, maybe using server-side software (such as Django) to dynamically generate the content before sending it.

The following diagram shows the direction of the transmission of HTTP requests and HTTP responses between a browser and a web server.

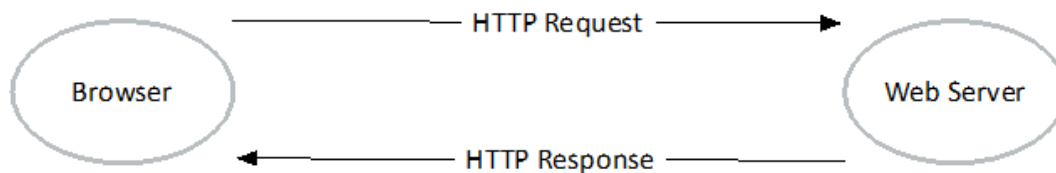


Figure 1.6 – An HTTP request and an HTTP response

The request is made up of four main parts: the method, the path, headers, and the body. Some types of requests don't have a body. If you just visit a web page, your browser will not send a body, whereas if you are submitting a form (for example, by logging into a site or performing a search), then your request will have a body containing the data you're submitting. We'll look at two example requests now to illustrate this.

The first request will be to an example page with the following URL: `https://www.example.com/page`. When your browser visits that page, behind the scenes, this is what it's sending:

```
GET /page HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/109.0
Cookie: sessid=abc123def456
```

The first line contains the method (GET) and the path (/page). It also contains the HTTP version – in this case, 1.1, although you don't have to worry about this. There are many different HTTP methods that can be used, depending on how you want to interact with the remote page. Some common ones are GET (to retrieve the remote page), POST (to send data to the remote page), PUT (to create a remote page), and DELETE (to delete the remote page). Note that the descriptions of the actions are somewhat simplified – the remote server can choose how it responds to different methods, and even experienced developers can disagree on the correct method to implement for a particular action. It's also important to note that even if a server supports a particular method, you will probably need the correct permissions to perform that action – you can't just delete a web page you don't like, for example.

When writing a web application, the vast majority of the time, you will only deal with GET requests. When you start accepting forms, you'll also have to use POST requests. It is only when you are working with advanced features, such as creating REST APIs, that you will have to worry about PUT, DELETE, and other methods.

Referring to the example request again, line 2 onwards features the headers of the request. The headers contain extra metadata about the request. Each header is on its own line, with the header name and its value separated by a colon. Most are optional (except for Host – there'll be more on that soon). Header names are not case-sensitive. For the sake of this example, we're only showing three common headers here. Let's look at the example headers in order:

- **Host:** As mentioned, this is the only header that is required (for HTTP 1.1 or later). It is needed for the web server to know which website or application should respond to the request if there are multiple sites hosted on a single server.
- **User-Agent:** Your browser usually sends to the server a string identifying its version and operating system. Your server application could use this to serve different pages to different devices (for example, a mobile-specific page for smartphones).
- **Cookie:** You have probably seen a message when visiting a web page that lets you know that it is storing a cookie in the browser. These are small pieces of information that a website can store in your browser and use to identify you or save settings for when you return to the site. If you were wondering about how your browser sends these cookies back to the server, it is through this header.

There are many other standard headers defined, and it would take up too much space to list them all. They can be used to authenticate to the server (*Authorization*), tell the server what kind of data you can receive (*Accept*), or even what language you'd like for the page (*Accept-Language*), although this will only work if the page creator has made the content available in

the particular language you request). You can even define your own headers that only your application knows how to respond to.

Now, let's look at a slightly more advanced request – one that sends some information to a server and thus (unlike the previous example) contains a body. In this example, we are logging into a web page by sending a username and password – for example, you visit `https://www.example.com/login`, and it displays a form to enter a username and password. After you click the **Login** button, this is the request that is sent to the server:

```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 32

username=user&password=password1
```

As you can see, this looks similar to the first example, but there are a few differences. The method is now POST, and two new headers have been introduced (you can assume that your browser will still be sending the other headers that were in the previous example too):

- **Content-Type:** This tells the server the type of data that is included in the body. In the case of `application/x-www-form-urlencoded`, the body is a set of key-value pairs. An HTTP client could set this header to tell the server if it was sending other types of data, such as JSON or XML.
- **Content-Length:** For the server to know how much data to read, the client must tell it how much data is being sent. The Content-Length header contains the length of the body. If you count the length of the body in this example, you'll see that it's 32 characters.

The headers are always separated from the body by a blank line. By looking at the example, you should be able to tell how the form data is encoded in the body – username has the user1 value and password has the password1 value.

These requests were quite simple, but most requests don't get much more complicated. They might have different methods and headers, but should follow the same format. Now that we've seen requests, we'll take a look at HTTP responses that come back from the server.

An HTTP response looks similar to a request and consists of three main parts: a status, headers, and a body. Like a request, though, depending on the type of response, it might not have a body. The first response example is a simple, successful response:

```
HTTP/1.1 200 OK
Server: nginx
Content-Length: 18132
Content-Type: text/html
Set-Cookie: sessid=abc123def46

<!DOCTYPE html><html><head>...
```

The first line contains the HTTP version, a numeric status code (200), and then a text description of what the code means (OK – the request was a success). We'll show some more statuses after the next example. Lines 2 to 5 contain headers, similar to a request. Some headers you may have seen before; we will explain them all in this context:

- **Server:** This is similar but the opposite of the User-Agent header – this is the server telling the client what software it is running.
- **Content-Length:** The client uses this value to determine how much data to read from the server to get the body.
- **Content-Type:** The server uses this header to indicate to the client what type of data it is sending. The client can then choose how it will display the data – an image must be displayed differently from HTML, for example.
- **Set-Cookie:** We saw in the first request example how a client sends a cookie to the server. This is the corresponding header that a server sends to set that cookie in the browser.

After the headers is a blank line, and then the body of the response. We haven't shown it all here, just the first few characters of the HTML that is being received, out of the 18,132 that the server has sent.

Next, we'll show an example of a response that is returned if a requested page is not found:

```
HTTP/1.1 404 Not Found
Server: nginx
Content-Length: 55
Content-Type: text/html

<!DOCTYPE html><html><body>Page Not Found</body></html>
```

It is similar to the previous example, but the status is now `404 Not Found`. If you've ever been browsing the internet and received a `404` error, this is the type of response your browser received. The various status codes are grouped by the type of success or failure they indicate:

- **100–199:** The server sends codes in this range to indicate protocol changes or that more data is required. You don't have to worry about these status codes.
- **200–299:** A status code in this range indicates successful handling of a response. As we saw, the most common one you will deal with is `200 OK`.
- **300–399:** A status code in this range means the page you are requesting has moved to another address. An example of this is a URL shortening service that would redirect you from the short URL to the full one when you visit it. Common responses are `301 Moved Permanently` or `302 Found`. When sending a redirect response, the server will also include a `Location` header that contains the URL you should be redirected to.
- **400–499:** A status code in this range means that the request could not be handled because there was a problem with what the client sent. This is in contrast to a request not being able to be handled due to a problem on the server (we will discuss those soon). We've already seen a `404 Not Found` response; this is due to a bad request because the client is requesting a document that does not exist. Some other common responses are `401 Unauthorized` (the client should log in) or `403 Forbidden` (the client is not allowed to access the specific resource). Both of these problems could be avoided by getting the client to log in, hence them being considered client-side (request) problems.
- **500–599:** Status codes in this range indicate an error on the server's side. The client shouldn't expect to be able to adjust a request to fix the problem. When working with Django, the most common server error status you will see is `500 Internal Server Error`. This will be generated if your code raises an exception. Another common one is `504 Gateway Timeout`, which might occur if your code is taking too long to run. The other variants that are common to see are `502 Bad Gateway` and `503 Service Unavailable`, which generally mean there is a problem with your application's hosting in some way.

These are only some of the most common HTTP statuses. You can find a more complete list at <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>. Like HTTP headers, though, statuses are arbitrary, and an application can return custom statuses. It is up to the server and clients to decide what these custom statuses and codes mean.

If this is your first time being introduced to the HTTP protocol, there's quite a lot of information to take in. Luckily, Django does all the hard work and encapsulates the incoming data into an `HttpRequest` object. Most of the time, you don't need to know about most of the information coming in, but it's available if you need it. Likewise, when sending a response, Django encapsulates your data in an `HttpResponse` object. Normally, you just set the content to return, but you also have the freedom to set HTTP status codes and headers. We will discuss how to access and set the information in `HttpRequest` and `HttpResponse` later in this chapter. In the next section, we will look at how Django receives, parses, and responds to an HTTP request.

Processing a request

This is a basic timeline of the request and response flows so that you can get an idea of what the code you'll be writing does at each stage. In terms of writing code, the first part you will write is your view. The view you create will perform some actions, such as querying a database for data. Then, the view will pass this data to another function to render a template, finally returning the `HttpResponse` object that encompasses the data you want to send back to the client.

Next, Django needs to know how to map a specific URL to your view so that it can load the correct view for the URL it receives as part of a request. You will write this URL mapping in a URL configuration Python file.

When Django receives a request, it parses the URL config file and then finds the corresponding view. It calls the view, passing in the `HttpRequest` object that represents the request. Your view will return its `HttpResponse`, and then Django takes over again to send this data to its host web server and back out to the client that requested it.

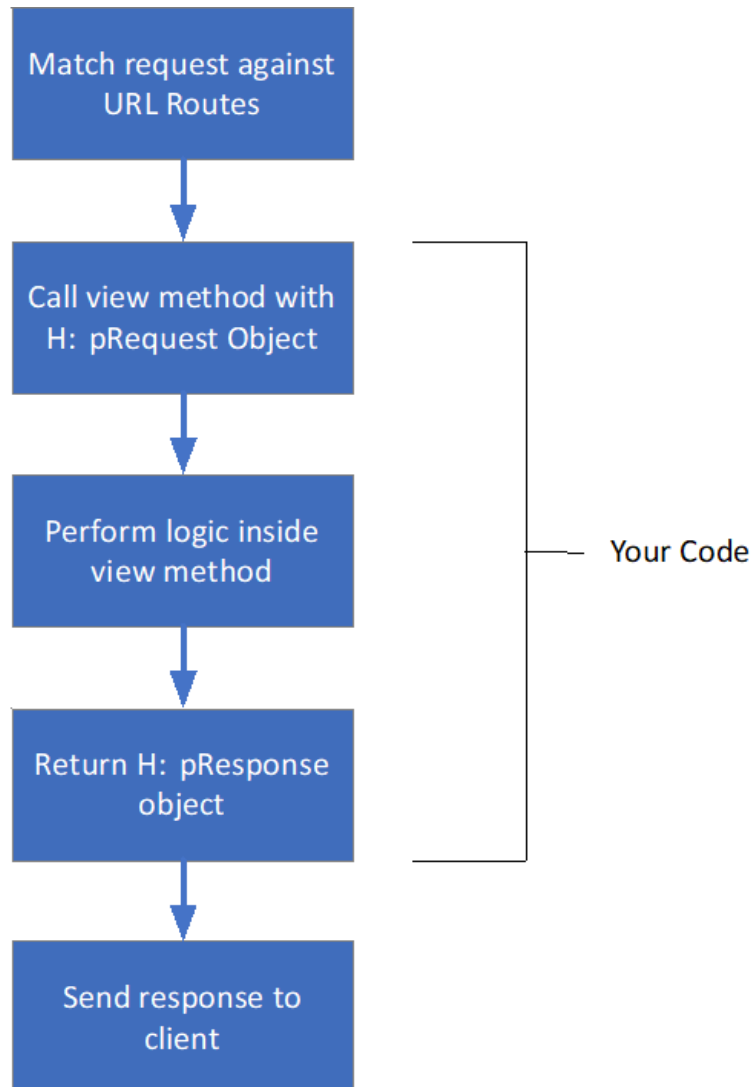


Figure 1.7 – The request and response flow

The request and response flow is illustrated in *Figure 1.7*; the sections indicated as **Your Code** are for the code that you write, and the first and last steps are taken care of by Django. Django does the URL matching for you, calls your view code, and then handles passing the response back to the client.

In this section, we learned about the structure of HTTP requests and responses, including what different types of requests are used for. We saw how HTTP status codes can be used to denote

errors in HTTP requests or responses. We also got an overview of how Django can process an HTTP request. In the next section, we'll explore the Django project structure and learn what the various files and directories are used for.

Exploring the Django project structure

We already introduced Django projects in the *Scaffolding a Django project and app* section. Let's remind ourselves of what happens when we run `startproject` (for a project named `myproject`) – the command creates a `myproject` directory with one file called `manage.py`, and a directory called `myproject` (this matches the project name; in *Exercise 1.01 – creating a project and app, and starting the development server*, this folder was called `bookr`, the same as the project). The directory layout is shown in *Figure 1.8*. We'll now examine the `manage.py` file and `myproject` package contents in more detail.

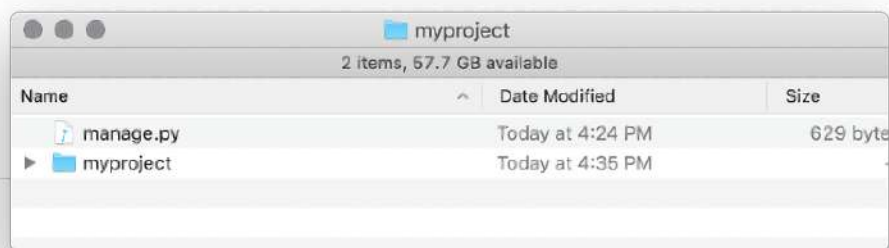


Figure 1.8 – The project directory for `myproject`

As the name suggests, `manage.py` is a script that is used to manage your Django project. Most of the commands that are used to interact with your project will be supplied to this script on the command line. The commands are supplied as an argument to this script – for example, if we want to run the `manage.py runserver` command, it would mean running the `manage.py` script like this:

```
python manage.py runserver
```

There are a number of useful commands that `manage.py` provides. You will be introduced to them in more detail throughout the book; some of the common ones are as follows:

- `runserver`: This starts the Django development HTTP server to serve the Django app on your local computer.
- `startapp`: This creates a new Django app in your project. We'll talk about what apps are in more depth soon.
- `shell`: This starts a Python interpreter with the Django settings pre-loaded. This is useful for interacting with your application without having to manually load your Django settings.
- `dbshell`: This starts an interactive shell connected to your database, using the default parameters from your Django settings. You can run manual SQL queries this way.
- `makemigrations`: This generates database change instructions from your model definitions. You will learn what this means and how to use this command in *Chapter 2*.
- `migrate`: This applies migrations generated by the `makemigrations` command. You will use this in *Chapter 2*, as well.
- `test`: This runs automated tests that you have written. You'll use this command in *Chapter 14*.

A full list of all commands is available at <https://docs.djangoproject.com/en/3.0/ref/django-admin/>.

In the following sections, we will explore the contents of the project directory, learn what Django app directories contain, and see how to load up your project in PyCharm.

The myproject directory

As well as the `manage.py` file, the other file item created by `startproject` is the `myproject` directory. This is the actual Python package for your project. It contains settings for the project, some configuration files for your web server, and the global URL maps. Inside the `myproject` directory are five files:

- `__init__.py`: This is an empty file that lets Python know that the `myproject` directory is a Python module. You'll be familiar with these files if you've worked with Python before.
- `settings.py`: This contains all the Django settings for your application. We will explain the contents soon.

- `urls.py`: This has the global URL mappings that Django will initially use to locate views or other child URL mappings. You will add a URL map to this file soon.
- `asgi.py` and `wsgi.py`: These files are what ASGI or WSGI web servers use to communicate with your Django app when you deploy it to a production web server. You normally don't need to edit these at all, and they aren't used in day-to-day development. Their use will be discussed more in *Chapter 18*.

Django development server

You already started the Django development server in *Exercise 1.01 – creating a project and app, and starting the development server*. As we mentioned previously, it is a web server intended to only be run on a developer's machine during development. It is not intended for use in production.

By default, the server listens on port 8000 on localhost (127.0.0.1), but this can be changed by adding a port number, or address and port number, after the `runserver` argument:

```
python manage.py runserver 8001
```

This will have the server listen on port 8001 on localhost (127.0.0.1).

You can also have it listen on a specific address if your computer has more than one, or 0.0.0.0 for all addresses:

```
python manage.py runserver 0.0.0.0:8000
```

This will have the server listen on all your computer's addresses on port 8000, which can be useful if you want to test your application from another computer or your smartphone.

The development server watches your Django project directory and will restart automatically every time you save a file, so that any code changes you make are automatically reloaded into the server. However, you still have to manually refresh your browser to see changes there.

When you want to stop the `runserver` command, it can be done in the usual way for stopping processes in the terminal – by typing *Ctrl + C*.

Django apps

Now that we've covered most of the theory about apps, we can be more specific about their purpose. An app directory contains all the models, views, templates (and more) that they need to provide application functionality. A Django project will contain at least one app (unless it has been heavily customized to not rely on a lot of Django functionality). If well-designed, an app should be able to be removed from a project and moved to another project without modification. Usually, an app will contain models for a single design domain, and this can be a useful way of determining whether your app should be split into multiple apps.

Your app can have any name as long as it is a valid Python module name (i.e., only letters, numbers, and underscores) and does not conflict with other files in your project directory. For example, as we have seen, there is already a directory called `myproject` in the project directory (containing the `settings.py` file), so you could not have an app called `myproject`. As we saw in *Exercise 1.01 – creating a project and app, and starting the development server*, creating an app uses the `manage.py startapp appname` command, as shown here:

```
python manage.py startapp myapp
```

The `startapp` command creates a directory within your project with the name of the app specified. It also scaffolds files for the app. Inside the app directory are a number of files and a folder, as shown in *Figure 1.9*:

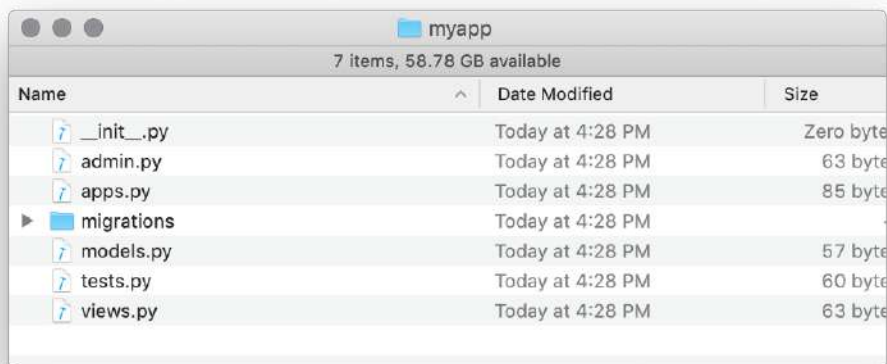


Figure 1.9 – The contents of the `myapp` app directory

- `__init__.py`: An empty file, indicating that this directory is a Python module.

- `admin.py`: Django has a built-in admin site for viewing and editing data with a GUI. In this file, you will define how your app's models are exposed in the Django admin site. We'll cover this in more detail in *Chapter 4*.
- `apps.py`: This contains some configuration for the metadata of your app. You won't need to edit this file.
- `models.py`: This is where you will define the models for your application. You'll read about this in more detail in *Chapter 2*.
- `migrations`: Django uses migration files to automatically record changes to your underlying database as models change. They are generated by Django when you run the `manage.py makemigrations` command and stored in this directory. They do not get applied to the database until you run `manage.py migrate`. They will also be covered in *Chapter 2*.
- `tests.py`: In order to test that your code is behaving correctly, Django supports writing tests (unit, functional, or integration) and will look for them inside this file. We will write some tests throughout this book and cover testing in detail in *Chapter 14*.
- `views.py`: Your Django views (the code that responds to HTTP requests) will go in here. You will create a basic view soon, and views will be covered in more detail in *Chapter 3*.

We will examine the contents of these files in more detail later, but for now, we'll get PyCharm up and running.

PyCharm setup

We confirmed in *Exercise 1.01 – creating a project and app, and starting the development server* that the *bookr* project has been set up properly (since the development server runs successfully), so we can now start using PyCharm to run and edit our project. PyCharm is an IDE for Python development and includes features such as code completion, automatic style formatting, and a built-in debugger. We will then use PyCharm to start writing our own URL maps, views, and templates. It will also be used to start and stop the development server, which will allow debugging of your code by setting breakpoints.

Exercise 1.02 – project setup in PyCharm

In this exercise, we will open the *bookr* project in PyCharm and set up the project interpreter so that PyCharm can run and debug the project. Follow these steps:

1. Open PyCharm. When you first open PyCharm, you will be shown the **Welcome to PyCharm** screen, which asks you what you want to do:

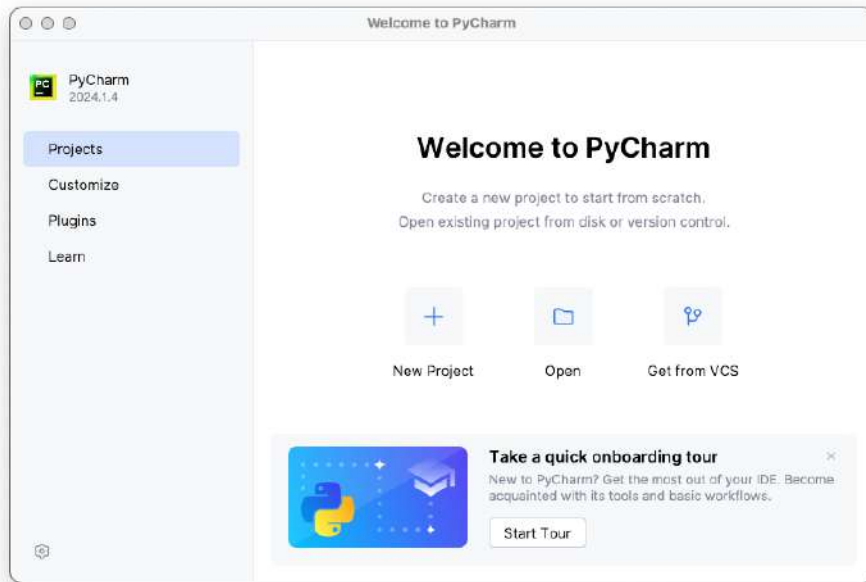


Figure 1.10 – The PyCharm welcome screen

2. Click on **Open**, then browse to the bookr project you just created, then open it. Make sure you are opening the bookr project directory and not the bookr package directory inside.
3. You will be asked if you want to trust the bookr project. Since you just created it, it's safe, so click on **Trust Project**:

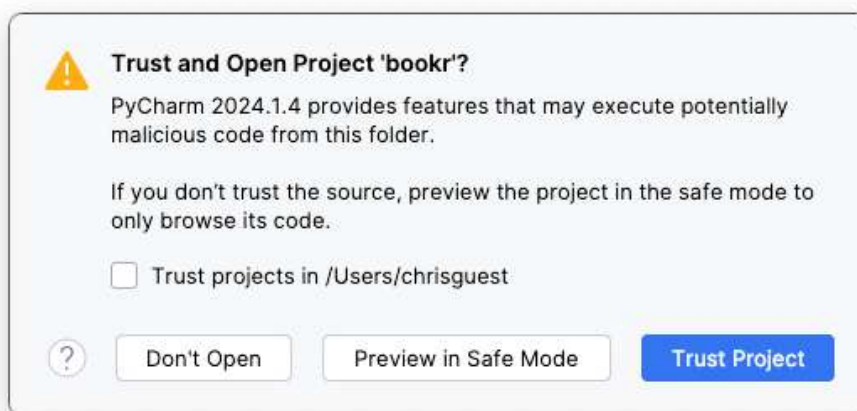


Figure 1.11 – PyCharm's Trust Project dialog

If you haven't used PyCharm before, it will ask you about what settings and themes you want to use, and once you have answered all those questions, you will see your *bookr* project structure open in the **Project** pane on the left of the window:

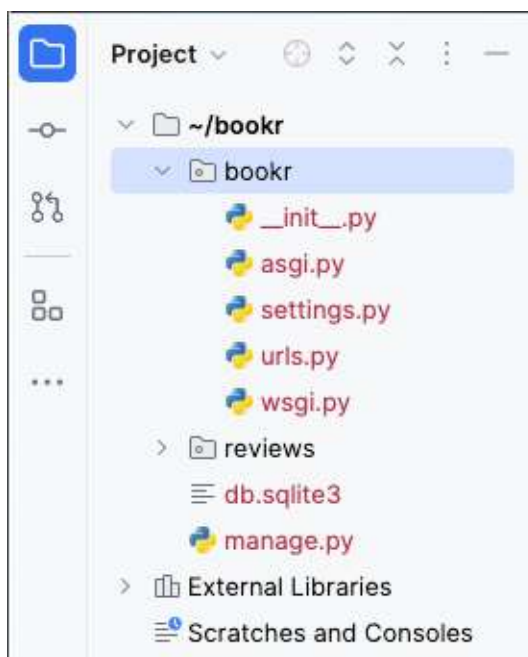


Figure 1.12 – The PyCharm Project pane

Your **Project** pane should look like *Figure 1.13* and show the `bookr` and `reviews` directories, and the `manage.py` file. If you do not see these and instead see `asgi.py`, `settings.py`, `urls.py`, and `wsgi.py`, then you have opened the `bookr` package directory instead. Select **File** | **Open**, and then browse and open the `bookr` project directory instead.

Before PyCharm knows how to execute your project to start the Django development server, the interpreter must be set to the Python binary inside your virtual environment. This is done first by adding the interpreter to the global interpreter settings.

4. Open the **Preferences** (macOS) or **Settings** (Windows/Linux) window inside PyCharm. Here's how you can get to these windows:
 - macOS: **PyCharm** menu | **Preferences**
 - Windows and Linux: **File** | **Settings**
5. In the preferences list pane on the left, open the **Project: bookr** item, and then click **Python Interpreter**:

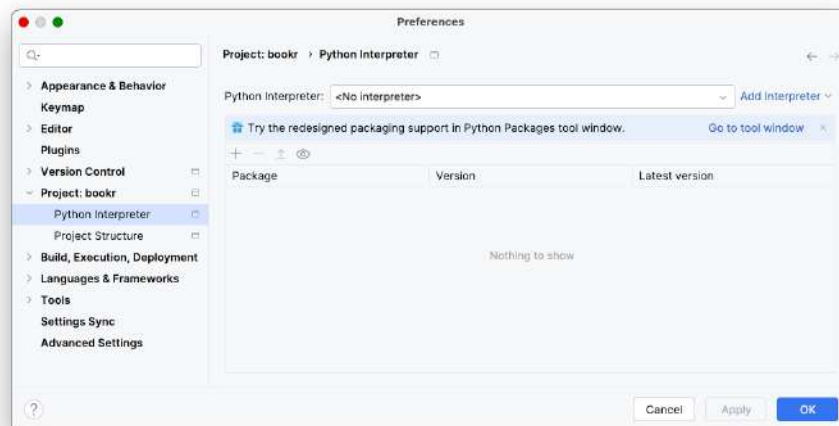


Figure 1.13 – The Python Interpreter settings

6. Sometimes, PyCharm is able to automatically determine the virtual environments, so in this case, the project interpreter may already be populated with the correct interpreter. If it has, and you see Django in the list of packages, click **OK** to close the window and complete this exercise.

7. In most cases, though, the Python interpreter must be set manually. Click the cog icon next to the **Python Interpreter** dropdown, and then click **Add...**
8. The **Add Python Interpreter** window is now displayed. Select the **Existing Environment** radio button, and then click the ellipses (...) next to the **Interpreter** selection. You should then browse and select the Python interpreter for your virtual environment.

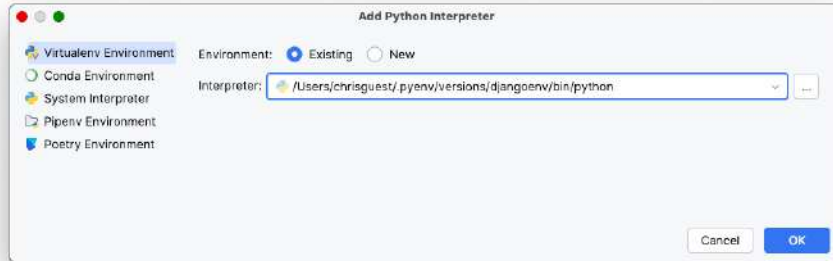


Figure 1.14 – The Add Python Interpreter window

9. On macOS (assuming you called the virtual environment `djangoenv`), the path is usually `/Users/<yourusername>/.pyenv/djangoenv/bin/python`. Similarly, in Linux, it should be in `/home/<yourusername>/.pyenv/djangoenv/bin/python`.
10. If you're unsure, you can run the `which python` command in the terminal where you previously ran the `python manage.py` command, and it will tell you the path to the Python interpreter:

```
which python
/Users/chrisguest/.pyenv/djangoenv/bin/python
```

On Windows, it will be wherever you created your virtual environment with the `pyenv` command.

After selecting the interpreter, your **Add Python Interpreter** window should look like *Figure 1.15*.

11. Click **OK** to close the **Add Python Interpreter** window. You should now see the main preferences window, and Django (and other packages in your virtual environment) will be listed (see *Figure 1.16*).

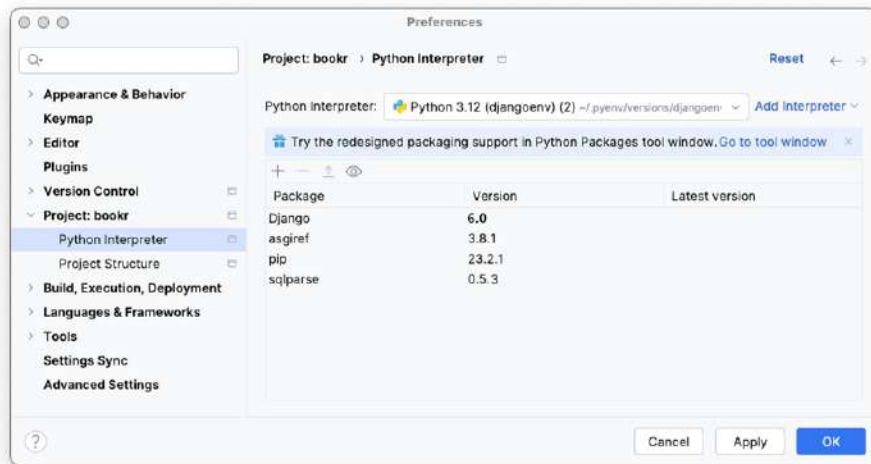


Figure 1.15 – Packages in the virtual environment are listed

12. Click **OK** in the main **Preferences** window to close it. PyCharm will now take a few seconds to index your environment and the libraries installed. You can see the process in its bottom-right status bar. Wait for this process to finish, and the progress bar will disappear.

In order to run the Django development server, Python needs to be configured with a run configuration. We will set this up now:

13. Click **Add Configuration...** at the top right of the PyCharm project window to open the **Run/Debug Configurations** window:

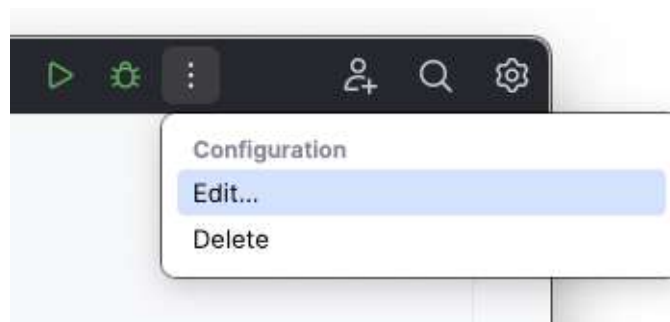


Figure 1.16 – The Add Configuration... button at the top right of the PyCharm window

14. Click the **+** button at the top left of this window, and select **Python** from the drop-down menu.

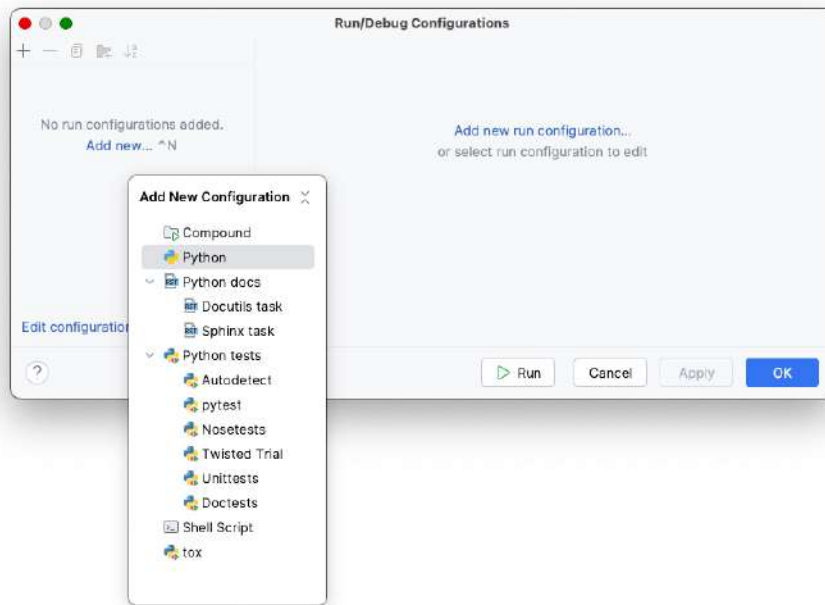


Figure 1.17 – Adding a new Python configuration in the Run/Debug Configurations window

15. A new configuration panel with fields regarding how to run your project will display on the right of the window. Go ahead and fill out the fields as follows:

- The **Name** field can be anything, but should be something understandable. Enter Django Dev Server.
- **Script path** is the path to your `manage.py` file. If you click the folder icon in this field, you can browse your filesystem to select the `manage.py` file inside the bookr project directory.
- **Parameters** refers to the arguments that come after the `manage.py` script, the same as if we were running it from the command line. We will use the same argument here to start the server, so enter `runserver`.

Note

As mentioned earlier, the `runserver` command can also accept an argument for the port or address to listen to. If you want to, you can add this argument after `runserver` in the same Parameters field.

16. The **Python Interpreter** setting should have been automatically set to the one that was set in *Exercise 1.02*. If not, click the arrow dropdown on the right to select it.
The **Working directory** setting should be the bookr project directory. This has probably already been set correctly.
17. Make sure that **Add content roots to PYTHONPATH** and **Add source roots to PYTHONPATH** are checked. This will ensure PyCharm adds your bookr project directory to PYTHONPATH (the list of paths that the Python interpreter searches when loading a module). Without these checked, the imports from your project will not work correctly.

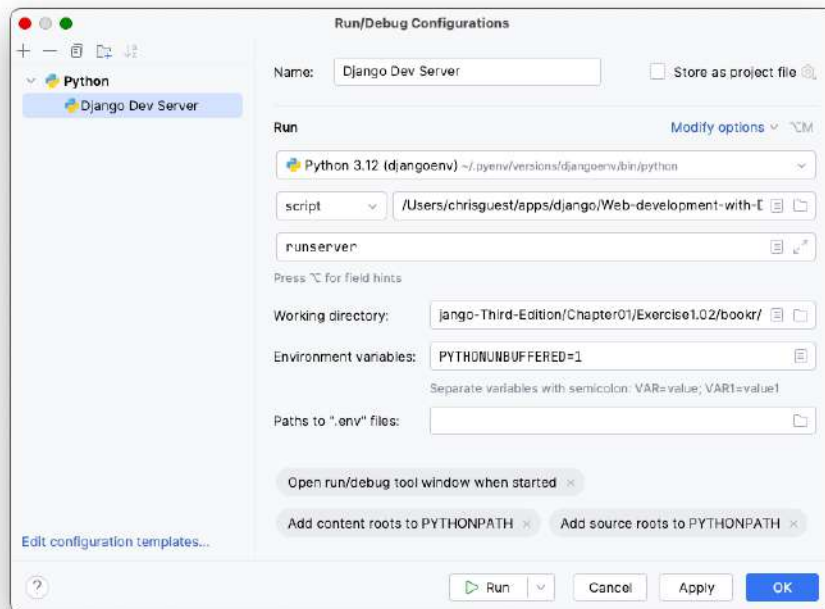


Figure 1.18 – The configuration settings

18. Check that your **Run/Debug Configurations** window looks similar to *Figure 1.19*, and then click **OK** to save the configuration.
19. Now, instead of starting the Django development server in a terminal, click the **Play** icon at the top right of the project window to start it (see *Figure 1.20*).



Figure 1.19 – Django development server configuration with the Play, Debug, and Stop buttons

20. Click the **Play** icon to start the Django development server.

Note

Make sure you stop any other instances of the Django development server that are running (such as in a terminal); otherwise, the one you are starting will not be able to bind to port 8000 and will fail to start.

A console will open at the bottom of the PyCharm window, which will show output indicating that the development server has started (*Figure 1.21*).

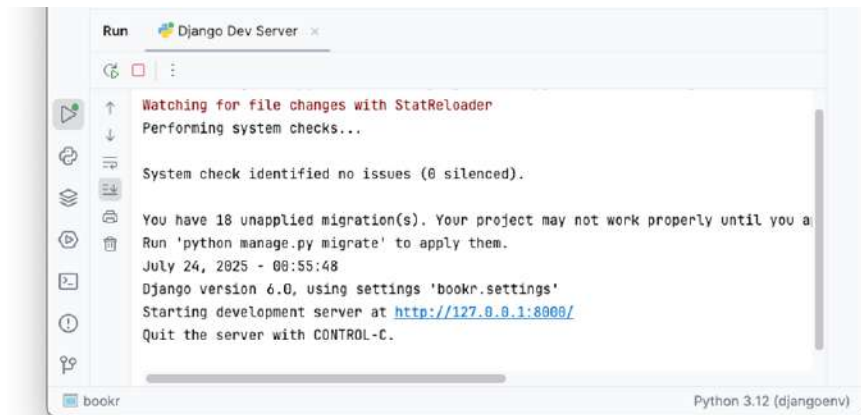


Figure 1.20 – The console with the Django development server running

21. Open a web browser and navigate to `http://127.0.0.1:8000`. You should see the same Django example screen that you saw earlier, in *Exercise 1.01 – creating a project and app, and starting the development server* (*Figure 1.21*), which will confirm that, once again, everything is set up correctly.

In this exercise, we opened the *Bookr* project in PyCharm and then set the Python interpreter for our project. We then added a run configuration in PyCharm, which allows us to start and stop the Django development server from within PyCharm. We will also be able to debug our project later by running it inside PyCharm's debugger.

In this section, we explored the Django folder structure and saw the purpose of the directories and folders that were scaffolded. We also saw how to create a Django app and the files that it contains. Finally, we got a Django project up and running in PyCharm.

Introducing Django views

You now have everything set up to start writing your own Django views and configure the URLs that will map to them. As we saw earlier in this chapter, a view is simply a function that takes an `HttpRequest` instance (built by Django) and (optionally) some parameters from the URL. It will then perform some operations, such as fetching data from a database. Finally, it returns `HttpResponse`.

To use our *bookr* project as an example, we might have a view that receives a request for a certain book. It queries the database for this book and then returns a response containing an HTML page, showing information about the book. Another view could receive a request to list all the books, and then return a response with another HTML page containing this list. Views can also create or modify data; another view could receive a request to create a new book, and it would then add the book to the database and return a response with HTML that displays the new book's information.

In this chapter, we will only be using functions as views, but Django also supports class-based views that allow you to leverage object-oriented paradigms (such as inheritance). This allows you to simplify code used in multiple views that have the same business logic. For example, you might want to show all books or just books by a certain publisher. Both views need to query a list of books from the database and render them to a book list template. One `View` class could inherit from another and just implement the data fetching differently and leave the rest of the functionality (such as rendering) identical. Class-based views can be more powerful but also harder to learn. They will be introduced later, in *Chapter 11*, when you have more experience with Django.

The `HttpRequest` instance that is passed to the view contains all the data related to the request, with attributes such as the following:

- **method:** A string containing the HTTP method the browser used to request the page, usually `GET`, but it will be `POST` if a user has submitted a form. You can use this to change the flow of the view – for example, show an empty form on `GET`, or validate and process a form submission on `POST`.
- **GET:** This is a `QueryDict` class containing the parameters used in the URL query string. This is the part of the URL after `?`, if it contains one. We will go further into `QueryDict`

classes soon. Note that this attribute is always available, even if the request was not GET.

- **POST:** This is another `QueryDict` containing the parameters sent to the view in a POST request, such as from a form submission. Usually, you would use this in conjunction with a Django form, which will be covered in *Chapter 6, Forms*.
- **headers:** This is a case-insensitive key dictionary with the HTTP headers from the request. For example, you could vary the response with different content for different browsers based on the *User-Agent* header. We discussed some HTTP headers that are sent by the client earlier in this chapter.
- **path:** This is the path used in the request. Normally, you don't need to examine this because Django will automatically parse the path and pass it to the view function as parameters, but it can be useful in some instances.

We won't be using all these attributes yet, and there are others that will be introduced later, but you can now see what role the `HttpRequest` argument plays in your view.

Next, we'll see how Django determines what view to use for a specific URL.

Exploring URL mapping details

We briefly mentioned URL maps earlier, in the *Processing a request* section. Django does not automatically know which view function should be executed when it receives a request for a particular URL. The role of URL mapping is to build a link between a URL and a view. For example, in Bookr, you might want to map the `/books/` URL to a `books_list` view that you have created.

The URL-to-view mapping is defined in the file that Django automatically created, called `urls.py`, inside the bookr package directory (although a different file can be set in `settings.py`; there'll be more on that later).

This file contains a variable, `urlpatterns`, which is a list of paths that Django evaluates in turn until it finds a match for the URL being requested. The match will either resolve to a view function or another `urls.py` file, also containing a `urlpatterns` variable, which will be resolved in the same manner. URL files can be chained in this manner for as long as you want. This way, you can split URL maps into separate files (such as one or more per app) so that they don't become too large. Once a view has been found, Django calls it with `HttpRequest` and any parameters parsed from the URL.

Rules are set by calling the `path` function, which takes the path of the URL as the first argument. The path can contain named parameters that will be passed to a view as function parameters. Its second argument is either a view or another file also containing `urlpatterns`.

There is also the `re_path` function, which is similar to `path`, except that it takes a regular expression as the first argument for more advanced configuration. There is much more to URL mapping; it will be covered in *Chapter 3*. To illustrate these concepts, *Figure 1.22* shows the default `urls.py` file that Django generates:

```
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

Figure 1.21 – The default `urls.py` file

You can see the `urlpatterns` variable, which lists all the URLs that are set up. Currently, there is only one rule set up, which maps any path starting with `admin/` to the admin URL maps (the `admin.site.urls` module). This is not a mapping to a view; instead, it is an example of chaining URL maps together – the `admin.site.urls` module will define the remainder of the paths (after `admin/`) that map to the admin views. We will cover the Django admin site later in the book, starting in *Chapter 4*.

We will now write a view and set up a URL map to it, to see these in action.

Exercise 1.03 – writing a view and mapping a URL to it

Our first view will be very simple and will just return some static text content. In this exercise, we will see how to write a view and set up a URL map to resolve to a view:

Note

As you make changes to files in your project and save them, you might see the Django development server automatically restarting in the terminal or console in which it is running. This is normal – it automatically restarts to load any code changes that you

make. Please also note that it won't automatically apply changes to the database if you edit models or migrations – there'll be more on this in *Chapter 2*.

1. In PyCharm, expand the reviews folder in the **Project** browser on the left, and then double-click on the `views.py` file inside to open it. In the right (editor) pane in PyCharm, you should see the placeholder text automatically generated by Django:

```
from django.shortcuts import render

# Create your views here.
```

2. Remove this placeholder text from `views.py` and instead insert this content:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world!")
```

First, the `HttpResponse` class needs to be imported from `django.http`. This is what is used to create the response that goes back to the web browser. You can also use it to control things such as the HTTP headers or status code. For now, it will just use the default headers and the 200 Success status code. Its first argument is the string content to send as the body of the response.

Then, the view function returns an `HttpResponse` instance with the content we defined (Hello, world!).

3. We will now set up a URL pattern that maps the `index` view. This will be very simple and will not contain any parameters. Expand the `bookr` directory in the **Project** pane, and then open `urls.py`. Django has automatically generated this file. For now, we'll just add a simple URL to replace the default index that Django provides.
4. Import your views into the `urls.py` file by adding this line after the other existing imports:

```
import reviews.views
```

5. Add a map to the index view to the `urlpatterns` list by adding a call to the `path` function, with an empty string and a reference to the index function:

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', reviews.views.index),  
]
```

Make sure you don't add brackets after the index function (i.e., it should be `reviews.views.index` and not `reviews.views.index()`), as we are passing a reference to a function rather than calling it.

6. Switch back to your web browser and refresh. The Django default welcome screen should be replaced with the text defined in the view, **Hello, world!**.



Figure 1.22 – The web browser should now display the *Hello, world!* message

We just saw how to write a view function and map a URL to it. We then tested the view by loading it in a web browser.

Next, we'll see how to work with data that is sent in an HTTP request, but not in the URL path.

Working with GET, POST, and QueryDict objects

Data can come through an HTTP request as parameters on a URL or inside the body of a POST request. You might have noticed parameters in a URL when browsing the web – the text after `?` – for example, `http://www.example.com/?parameter1=value1¶meter2=value2`. We also saw earlier in this chapter an example of form data in a POST request to log in a user (the request body was `username=user&password=password1`).

Django automatically parses these parameter strings into `QueryDict` objects. The data is then available on the `HttpRequest` object that is passed to your view – specifically, in the `HttpRequest.GET` and `HttpRequest.POST` attributes for URL parameters and body parameters,

respectively. QueryDict objects mostly behave like dictionaries, except that they can contain multiple values for a key.

To show different methods of accessing items, we'll use a simple QueryDict (the `qd` variable) with only one key (`k`) as an example. The `k` item has three values in a list – the `a`, `b`, and `c` strings. The following code snippets show output from a Python interpreter.

First, the QueryDict `qd` variable is constructed from a parameter string:

```
>>> from django.http.request import QueryDict
>>> qd = QueryDict("k=a&k=b&k=c")
```

When accessing items with square bracket notation or the `get` method, the last value for that key is returned:

```
>>> qd["k"]
'c'
>>> qd.get("k")
'c'
```

To access all the values for a key, the `getlist` should be used, as follows:

```
>>> qd.getlist("k")
['a', 'b', 'c']
```

`getlist` will always return a list; it will be empty if the key does not exist:

```
>>> qd.getlist("bad key")
[]
```

While `getlist` does not raise an exception for keys that do not exist, accessing a key that does not exist with square bracket notation will raise `KeyError`, like a normal dictionary. Use the `get` method to avoid this error.

The QueryDict objects for GET and POST are immutable (they cannot be changed), so the `copy` method should be used to get a mutable copy if you need to change its values, as follows:

```
>>> qd["k"] = "d"
AttributeError: This QueryDict instance is immutable
>>> qd2 = qd.copy()
>>> qd2
```

```
<QueryDict: {'k': ['a', 'b', 'c']}>
>>> qd2["k"] = "d"
>>> qd2["k"]
"d"
```

To give an example of how QueryDict is populated from a URL, let's imagine an example URL: `http://127.0.0.1:8000?val1=a&val2=b&val2=c&val3`.

Behind the scenes, Django passes the query from the URL (everything after `?`) to instantiate a QueryDict object and attach it to request, which is passed to the view function. It goes something like this:

```
request.GET = QueryDict("val1=a&val2=b&val2=c&val3")
```

Remember, this is done to request before you receive it inside your view function, so you do not need to do this.

In the case of the preceding example URL, we can access the parameters inside the view function as follows:

```
request.GET["val1"]
```

Using standard dictionary access, it would return the "a" value:

```
request.GET["val2"]
```

Again, using standard dictionary access, there are two values set for the val2 key, so it would return the last value, "c":

```
request.GET.getlist("val2")
```

This would return a list of all the values for val2 – ["b", "c"]:

```
request.GET["val3"]
```

This key is in the query string but has no value set, so this returns an empty string:

```
request.GET["val4"]
```


This key is not set, so `KeyError` will be raised. Use `request.GET.get("val4")` instead, which will return `None`:

```
request.GET.getlist("val4")
```

Since this key is not set as an empty list, `[]` will be returned.

We will now look at `QueryDict` in action using the GET parameters. We will examine POST parameters further in *Chapter 6*.

Exercise 1.04 – exploring GET values and QueryDict objects

We will now make some changes to our index view from the previous exercise to read values from the URL in the GET attribute, and then we will experiment with passing different parameters to see the result:

1. Open the `views.py` file in PyCharm. Add a new variable called `name`, which reads the user's name from the GET parameters. Add this line after the index function definition:

```
name = request.GET.get("name") or "world"
```

2. Change the return value so that the `name` is used as part of the content that is returned:

```
return HttpResponse(f"Hello, {name}!")
```

Visit `http://127.0.0.1:8000/?name=Ben` in your browser. Note that the page still says **Hello, world!**. This is because we have not supplied a `name` parameter. You can add your name to the URL – for example, `http://127.0.0.1:8000/?name=Ben`. This can be seen in the following screenshot:



Figure 1.23 – Setting the name in the URL

3. Try adding two names – for example, `http://127.0.0.1:8000/?name=Ben&name=John`. As we mentioned, the last value for the parameter is retrieved with the `get` function, so you should see **Hello, John!**:



Figure 1.24 – Setting multiple names in the URL

4. Try setting no name, like this – `http://127.0.0.1:8000/?name=`. The page should go back to displaying **Hello, world!**:



Figure 1.25 – No name set in the URL

Note

You might wonder why we set `name` to the default **world** by using `or`, instead of passing `'world'` as the default value to `get`. Consider what happened in *step 6*, when we passed in a blank value for the `name` parameter. If we had passed `'world'` as a default value for `get`, then the `get` function would still have returned an empty string. This is because a value *is* set for `name`; it's just that it's blank. Keep this in mind when developing your views, as there is a difference between no value being set and a blank value being set. Depending on your use case, you might choose to pass the default to `get`.

In this exercise, we retrieved values from the URL in our view using the `GET` attribute of the incoming request. We saw how to set default values and which value is retrieved if multiple

values are set for the same parameter. The view returned `HttpResponse`, containing the message we wanted to send to the browser.

In the next section, we'll learn about how to update and use Django's settings.

Exploring Django settings

We haven't yet looked at how Django stores its settings. Now that we've seen the different parts of Django, it is a good time to examine the `settings.py` file. There are many settings Django contains that can be used to customize it. A default `settings.py` file was created for you when you started the *bookr* project. We will discuss some of the more important settings in the file now, and a few others that might be useful as you become more fluent with Django. You should open your `settings.py` file in PyCharm and follow along so that you can see where and what the values are for your project.

Each setting in this file is just a file-global variable. The order in which we will discuss the settings is the same order in which they appear in this file, although we may skip over some – for example, there is the `ALLOWED_HOSTS` setting between `DEBUG` and `INSTALLED_APPS`, which we won't cover in this part of the book (you'll see it in *Chapter 18*):

```
SECRET_KEY = '...'
```

This is an automatically generated value that shouldn't be shared with anyone. It is used for hashing, tokens, and other cryptographic functions. If you had existing sessions in a cookie and changed this value, the sessions would no longer be valid:

```
DEBUG = True
```

With this value set to `True`, Django will automatically display exceptions to the browser to allow you to debug any problems you encounter. It should be set to `False` when deploying your app to production:

```
INSTALLED_APPS = [...]
```

When you write your own Django apps (such as the reviews app) or install third-party applications (which will be covered in *Chapter 15*), they should be added to this list. As we've seen, it is not strictly necessary to add them here (our index view worked without our reviews

app being in this list). However, for Django to be able to automatically find the app's templates, static files, migrations, and other configurations, it must be listed here:

```
ROOT_URLCONF = 'bookr.urls'
```

This is the Python module that Django will first load to find URLs. Note that it is the file we added our index view URL map to previously:

```
TEMPLATES = [...]
```

Right now, it's not too important to understand everything in this setting, as you won't be changing it; the important line to point out is this one:

```
'APP_DIRS': True,
```

This tells Django that it should look in a `templates` directory inside each `INSTALLED_APP` when loading a template to render. We don't have a `templates` directory for `reviews` yet, but we will add one in the next exercise.

Django has more settings available that aren't listed in the `settings.py` file, so it will use its built-in defaults in these cases. You can also use the file to set arbitrary settings that you make up for your application. Third-party applications might want settings to be added here as well. In later chapters, we will add settings here for other applications. You can find a list of all settings, and their defaults, at <https://docs.djangoproject.com/en/5.1/ref/settings/>.

Referring to Django settings in your code

It can sometimes be useful to refer to settings from `settings.py` in your own code, whether they are Django's built-in settings or ones you have defined yourself. You might be tempted to write code such as these import settings:

```
from bookr import settings

# check if running in DEBUG mode
if settings.DEBUG:
    do_some_logging()
```

This method is incorrect, for several reasons:

- It is possible to run Django and specify a different settings file to read from, in which case the previous code would cause an error, as it would not be able to find that file.

Alternatively, if the file exists, the import would succeed but contain the wrong settings.

- Django has settings that might not be listed in the `settings.py` file, and if they aren't, it will use its own internal default. For example, if you removed the `DEBUG = True` line from your `settings.py` file, Django would fall back to using its internal value for `DEBUG` (which is `False`). You would get an error if you tried to access it using `settings.DEBUG` directly though.
- Third-party libraries can change how your settings are defined, so your `settings.py` file would look completely different. None of the expected variables may exist at all. The behavior of all these applications is beyond the scope of this book, but it is something to be aware of.

The preferred way is to use the `django.conf` module instead, like this:

```
# import settings from here instead
from django.conf import settings

if settings.DEBUG:
    do_some_logging()
```

When importing settings from `django.conf`, Django mitigates the three issues we just discussed:

- Settings are read from whatever Django settings file has been specified
- Any default settings values are interpolated
- Django takes care of parsing any settings defined by a third-party library

In our new, shorter example code snippet, even if `DEBUG` is missing from the `settings.py` file, it will fall back to the default value that Django has internally (which is `False`). The same is true for all other settings that Django defines; however, if you define your own custom settings in this file, Django will not have internal values for them, so in your code, you should have some provision for them not existing – how your code behaves is your choice and beyond the scope of this book.

Next, we'll see how Django locates template files.

Finding HTML templates in app directories

There are many options that are available to tell Django how to find templates, which can be set in the `TEMPLATES` setting of `settings.py`, but the easiest one (for now) is to create a templates directory inside the reviews directory. Django will look in this (and in other apps' templates directories) because of `APP_DIRS` being `True` in the `settings.py` file, as we saw in the previous section. However, for Django to know that the reviews directory is an app, we need to configure it in the settings. We'll do that in the next exercise.

Exercise 1.05 – creating a templates directory and a base template

In this exercise, you will create a templates directory for the *reviews* app. Then, you will add an HTML template file that Django will be able to render to an HTTP response.

We discussed `settings.py` and its `INSTALLED_APPS` setting in the *Exploring Django settings* section. We need to add the reviews app to `INSTALLED_APPS` for Django to be able to find templates. Follow these steps:

1. Open `settings.py` in PyCharm. Update the `INSTALLED_APPS` setting and add reviews to the end. It should look like this:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'reviews',  
]
```

2. Save and close `settings.py`.
3. In the PyCharm **Project** browser, right-click the reviews directory and select **New | Directory**.

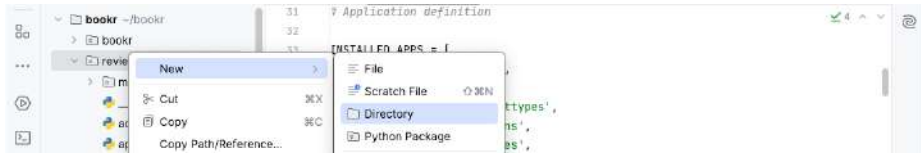


Figure 1.26 – Creating a new directory inside the reviews directory

4. Enter the name `templates` and press *Enter/Return* to create it.
5. Right-click the newly created `templates` directory and select **New | HTML File**.

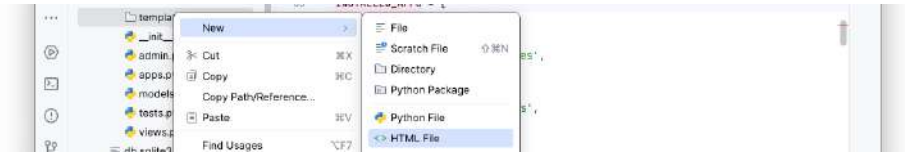


Figure 1.27 – Creating a new HTML file in the templates directory

6. In the window that appears, enter the name `base.html` and then press *Enter/Return* to create the file:
7. After PyCharm creates the file, it automatically opens it too. It will have this content:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>

</body>
</html>
```

8. Between the `<body>` and `</body>` tags, add a short message to validate that the template is being rendered: `Hello from a template!`
9. Your full code should look like this:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
<title>Title</title>
</head>
<body>
    Hello from a template!
</body>
</html>
```

In this exercise, we created a `templates` directory for the `reviews` app and added an HTML template to it. The HTML template will be rendered once we implement the use of the `render` function on our view.

Rendering a template with the `render` function

We now have a template to use, but we need to update our `index` view so that it renders the template, instead of returning the *Hello (name)!* text that it is currently displaying (refer to *Figure 1.30* to see how it currently looks). We will do this by using the `render` function and providing the name of the template. `render` is a shortcut function that returns `HttpResponse`. There are other ways to render a template that provide more control over how it is rendered, but for now, this function is fine for our needs. `render` takes at least two arguments – the first is always the request that was passed to the view, and the second is the name/relative path of the template being rendered. We will also call it with a third argument, the render context that contains all the variables that will be available in the template – there'll be more on this in *Exercise 1.07 – using variables in templates*.

Exercise 1.06 – rendering a template in a view

In this exercise, you will update your `index` view function to render the HTML template you created in *Exercise 1.05 – creating a templates directory and a base template*. You will make use of the `render` function, which loads your template from disk, renders it, and sends it to the browser. This will replace the static text you are currently returning from the `index` view function:

1. In PyCharm, open `views.py` in the `reviews` directory.
2. We no longer manually create `HttpResponse`, so remove the `HttpResponse` import line:

```
from django.http import HttpResponse
```

3. Replace it with an import of the `render` function from `django.shortcuts`:

```
from django.shortcuts import render
```


4. Update the `index` function so that, instead of returning `HttpResponse`, it's returning a call to `render`, passing in `request` and the template name.
5. The `views.py` file now contains this code:

```
from django.shortcuts import render

def index(request):
    return render(request, "base.html")
```

6. Start the development server if it's not already running. Then, open your web browser and refresh `http://127.0.0.1:8000`. You should see the **Hello from a template!** message rendered, as shown in *Figure 1.38*:



Figure 1.28 – Your first rendered HTML template

Rendering variables in templates

Templates aren't just static HTML. Most of the time, they will contain variables that are interpolated as part of the rendering process. They are passed from a view to a template using a context – a dictionary (or dictionary-like object) that contains names of all the variables a template can use. Let's take Bookr again as an example. Without variables in your template, you would need a different HTML file for each book you wanted to display. Instead, we use a variable such as `book_name` inside the template, and then the view provides the template with a `book_name` variable, set to the title of the book model it has loaded. When displaying a different book, the HTML does not need to change; the view just passes a different book to it. You can see how models, views, and templates are all now coming together.

Unlike some other languages, such as PHP, variables must be explicitly passed to a template, and variables in a view aren't automatically available to the template. This is for security reasons, as well as to avoid accidentally polluting the template's namespace (we don't want any unexpected variables in the template).

Inside a template, variables are denoted by double braces – `{{ }}`. While not strictly a standard, this style is quite common and used in other templating tools, such as Vue.js and Mustache. Symfony (a PHP framework) also uses double braces in its Twig templating language, so you might have seen them used similarly there.

To render a variable in a template, simply wrap it with braces – `{{ book_name }}`. Django will automatically escape HTML in output so that you can include special characters (such as `<` or `>`) in your variable without worrying about it garbling your output. If a variable is not passed to a template, Django will simply render nothing at that location, instead of throwing an exception.

There are many more ways to render a variable differently using filters, but these will be covered in *Chapter 3*.

Exercise 1.07 – using variables in templates

We'll put a simple variable inside the `base.html` file to demonstrate how Django's variable interpolation works:

1. In PyCharm, open the `base.html` file.
2. Update the `<body>` element so that it contains a place to render the `name` variable:

```
<body>
Hello, {{ name }}!
</body>
```

3. Go back to your web browser and refresh (you should still be at `http://127.0.0.1:8000`). You will see that the page now displays **Hello, !**. This is because we have not set the `name` variable in the rendering context.



Figure 1.29 – No value is rendered in the template because no context was set

4. Open `views.py` and add a variable called `name`, set to the "world" value, inside the `index` function:

```
def index(request):  
    name = "world"  
    return render(request, "base.html")
```

5. Refresh your browser again. Note that nothing has changed – anything we want to render must be explicitly passed to the render function as **context**. This is the dictionary of variables that are made available when rendering.
6. Add the context dictionary as the third argument to the render function. Change your render line to this:

```
return render(request, "base.html", {"name": name})
```

The completed `views.py` file will contain this code:

```
from django.shortcuts import render  
  
def index(request):  
    name = "world"  
    return render(request, "base.html",  
                  {"name": name})
```

7. Refresh your browser again, and you'll see it now says **Hello, world!**.



Figure 1.30 – A template rendered with a variable

In this exercise, we combined the template we created in the previous exercise with the render function to render an HTML page, with the `name` variable that was passed to it inside a context dictionary.

In the next section, we'll look at some ways you can debug your code and handle errors.

Debugging and dealing with errors

When programming, unless you're the perfect programmer who never makes mistakes, you'll probably have to deal with errors or debug your code at some point. When there is an error in your program, there are usually two ways to tell – either your code will raise an exception, or you will get unexpected output or results when viewing the page. You will probably see exceptions more often, as there are many accidental ways to cause them. If your code is generating unexpected output but not raising any exceptions, you will probably want to use the PyCharm debugger to find out why.

We'll start with an overview of some Python exceptions and how to handle them, along with an exercise that demonstrates how Django shows exceptions. Then, we'll look at running Django inside the PyCharm debugger so that you can peek inside your program while it executes.

Exceptions

If you have worked with Python or other programming languages before, you have probably come across exceptions. If not, here's a quick introduction. Exceptions are *raised* (or *thrown* in other languages) when an error occurs. The execution of a program stops at that point in the code, and the exception travels back up the function call chain until it is caught. If it is not caught, then the program will crash, sometimes with an error message describing the exception and where it occurred. There are exceptions that are raised by Python itself, and your code can raise exceptions to quickly stop execution at any point. Some common exceptions that you might see when programming Python are as follows:

- `IndentationError`: Python will raise this if your code is not correctly indented or has mixed tabs and spaces
- `SyntaxError`: Python raises this error if your code has invalid syntax:

```
>>> a == 1
      File "<stdin>", line 1
        a == 1
          ^
      SyntaxError: invalid syntax
```

- **ImportError:** This is raised when an import fails – for example, if trying to import from a file that does not exist or trying to import a name that is not set in a file:

```
>>> import missing_file
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ImportError: No module named missing_file
```

- **NameError:** This is raised when trying to access a variable that has not yet been set:

```
>>> a = b + 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'b' is not defined
```

- **KeyError:** This is raised when accessing a key that is not set in a dict (or a dict-like object):

```
>>> d = {'a': 1}
>>> d['b']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'b'
```

- **IndexError:** This is raised when accessing an index outside the length of a list:

```
>>> l = ['a', 'b']
>>> l[3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

- **TypeError:** This is raised when trying to perform an operation on an object that does not support it, or when using two objects of the wrong type – for example, trying to add a string to an int:

```
>>> 1 + '1'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Django also raises its own custom exceptions, and you will be introduced to them throughout the book.

When running the Django development server with `DEBUG = True` in your `settings.py` file, Django will automatically capture exceptions that occur in your code (instead of crashing). It will then generate an HTTP response, showing you a stack trace and other information to help you debug the problem. When running in production, `DEBUG` should be set to `False`. Django will then return a standard internal server error page, without any sensitive information. You also have the option to display a custom error page.

Exercise 1.08 – generating and viewing exceptions

Let's create a simple exception in our view so that you are familiar with how Django displays them. In this case, we'll try to use a variable that doesn't exist, which will raise `NameError`:

1. In PyCharm, open `views.py`. In the `index` view function, change the context being sent to the render function so that it's using a variable that doesn't exist. We'll try to send `invalid_name` in the context dictionary, instead of `name`. Don't change the context dictionary key, just its value:

```
return render(request, "base.html",  
              {"name": invalid_name})
```

2. Go back to your browser and refresh the page. You should see a screen like *Figure 1.31*:

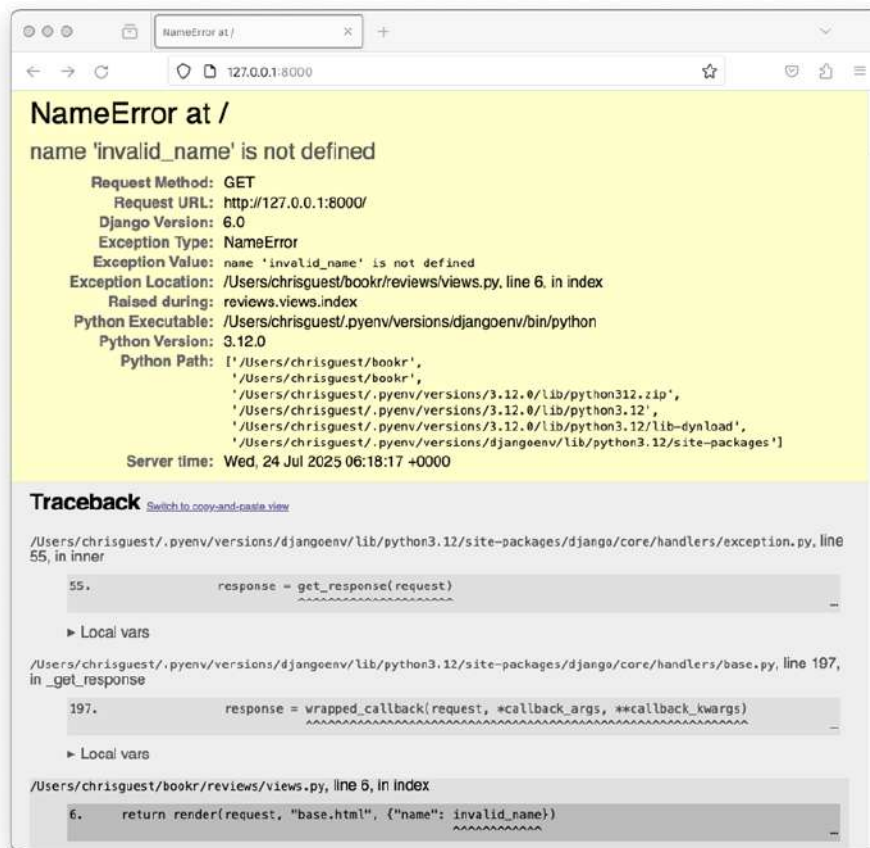


Figure 1.31 – A Django exception screen

The first two header lines on the page inform you of the error that occurred:

```
NameError at /
name 'invalid_name' is not defined
```

- Below the header is a traceback to where the exception occurred. Click on the various lines of code to expand them and see the surrounding code, or click **Local vars** for each frame to expand them and see what the values of the variables are:

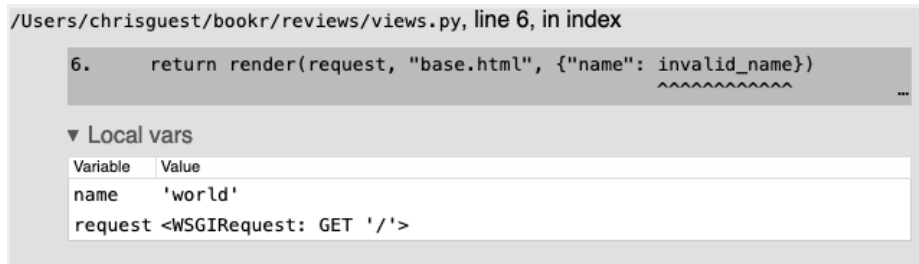


Figure 1.32 – The line that caused the exception

In our case, we can see the exception was raised on line six of our `views.py` file, and by expanding its **Local vars** dropdown, we see that `name` has the value `world`, and the only other variable is the incoming request (Figure 1.43).

4. Go back to `views.py` and fix `NameError` by renaming `invalid_name` back to `name`.
5. Save the file and refresh your browser. **Hello, world!** should display again (as shown in Figure 1.41).

In this exercise, we made our Django code raise an exception (`NameError`) by trying to use a variable that had not been set. We saw that Django automatically sent details of this exception and a stack trace to the browser to help us find the cause of the error. We then reverted our code change to make sure our view worked properly again.

Debugging

When you're trying to find problems in your code, it can help to use a debugger. This is a tool that lets you go through your code line by line, rather than executing it all at once. Each time the debugger is paused on a particular line of code, you can see the values of all the current variables. This is very useful for finding errors in your code that don't raise exceptions.

For example, in `Bookr`, we talked about having a view that fetches a list of books from the database and renders them in an HTML template. If you view the page in the browser, you might see only one book when you expect several. You could have the execution pause inside your view function and see what values were fetched from the database. If your view is only receiving one book from the database, you know there is a problem with your database querying somewhere. If your view is successfully fetching multiple books but only one is being rendered, then it's probably a problem with the template. Debugging helps you narrow down faults like this.

PyCharm has a built-in debugger to make it easy to step through your code and see what is happening on each line. To tell the debugger where to stop the execution of code, you need to

set a *breakpoint* on one or more lines of code. They are named as such because the execution of code will *break* (stop) at that *point*.

For breakpoints to be activated, PyCharm needs to be set to run your project in its debugger. There is a small performance penalty, but it is usually not noticeable, so you might choose to always run your code inside the debugger so that you can quickly set a breakpoint, without having to stop and restart the Django development server.

Running the Django development server inside the debugger is as simple as clicking the **Debug** icon instead of the **Play** icon (see *Figure 1.20*) to start it.

Exercise 1.09 – debugging your code

In this exercise, you will learn the basics of the PyCharm debugger. You will run the Django development server in the debugger and then set a breakpoint in your view function to pause execution so that you can examine the variables:

1. If the Django development server is running, stop it by clicking the **Stop** button in the top-right corner of the PyCharm window:



Figure 1.33 – The Stop button in the top-right corner of the PyCharm window

2. Start the Django development server again inside the debugger by clicking the **Debug** icon just to the left of the **Stop** button (*Figure 1.44*).
The server will take a few seconds to start, and then you should be able to refresh the page in your browser to make sure it's still loading. You shouldn't notice any changes; all the code is executed the same as before.
3. Now, we can set a breakpoint that will cause execution to stop so that we can see the state of the program. In PyCharm, click just to the right of the line numbers, on line 5, in the gutter on the left of the editor pane. A red circle will appear to indicate that the breakpoint is now active:



Figure 1.34 – A breakpoint on line 5

4. Go back to your browser and refresh the page. Your browser will not display any content; instead, it will just continue to try to load the page. Depending on your operating system, PyCharm should become active again; if not, bring it to the

foreground. You should see that line 5 is highlighted, and at the bottom of the window, the debugger is shown. The stack frames (the chain of functions that were called to get to the current line) are on the left, and the current variables of the function are on the right.

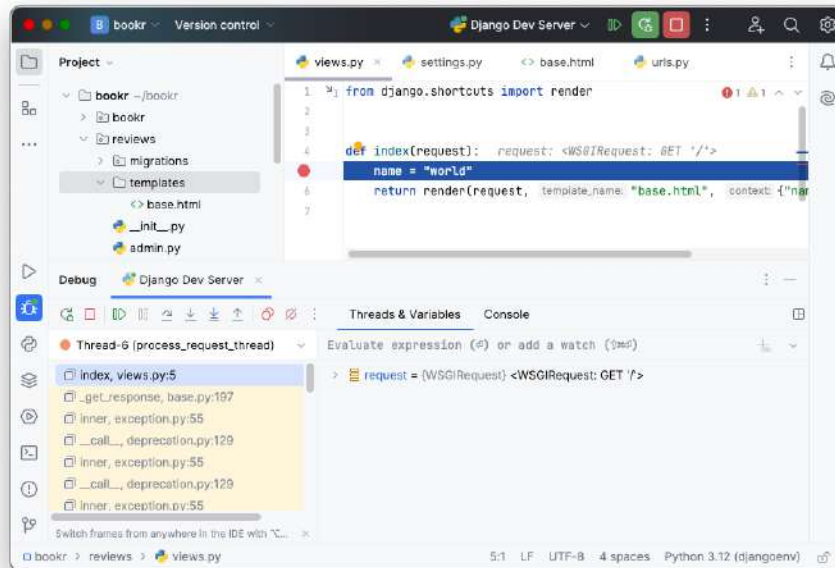


Figure 1.35 – The debugger paused with the current line (5) highlighted

There is currently one variable in scope, `request`. If you click the toggle triangle to the left of its name, you can show or hide the attributes it has set.

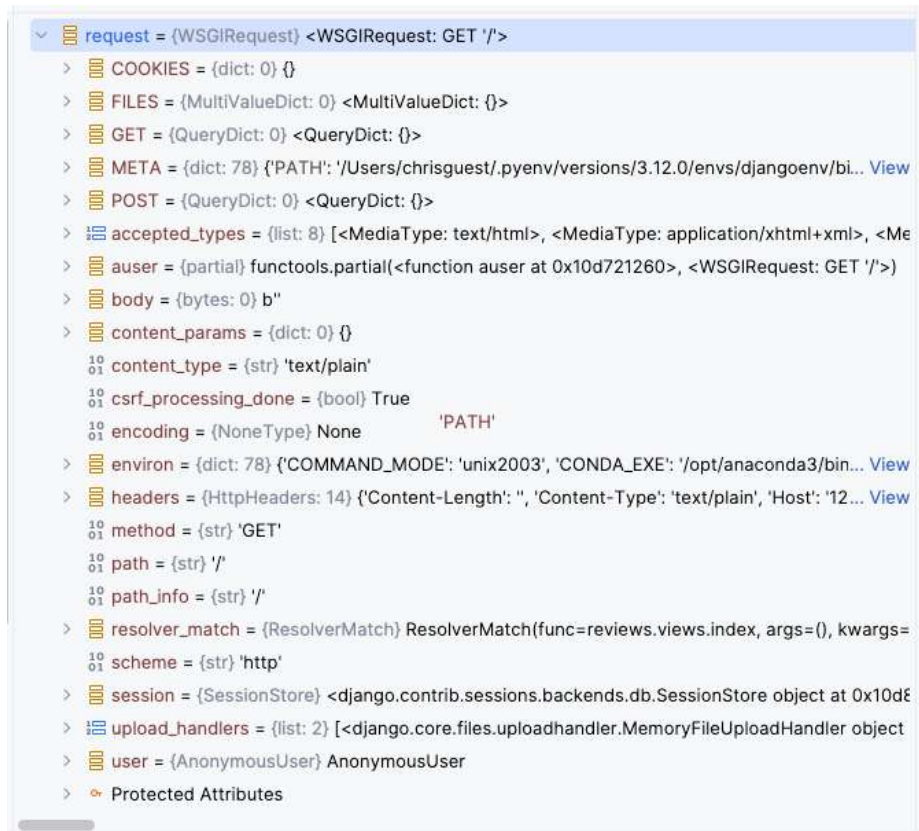


Figure 1.36 – The attributes of the request variable

For example, if you scroll down through the list of attributes, you can see that method is GET and path is /.



Figure 1.37 – The actions bar

5. The actions bar, shown in *Figure 1.37*, is above the stack frames and variables. Its buttons (from left to right) are as follows:
 - **Rerun:** This stops the program and restarts it.
 - **Stop:** This stops the debugger.
 - **Resume Program:** This continues running until the next breakpoint.
 - **Pause Program:** This button breaks the program at its current execution point.

- **Step Over:** This executes the current line of code and continues to the next line.
- **Step Into:** This steps into the current line. For example, if the line contained a function, it would continue with the debugger inside this function.
- **Step into My Code:** This steps into the line being executed but continues until it finds the code you have written. For example, if you're stepping into third-party library code that later calls your code, it will not show you the third-party code, instead continuing through until it returns to the code that you have written.
- **Step Out:** This returns from the current code to the function or the method that called it. It is the opposite of the **Step In** action.
- **View Breakpoints:** This opens a window to see all the breakpoints you have set.
- **Mute Breakpoints:** This toggles all breakpoints on or off but does not remove them.

Note that not all buttons are useful all the time. For example, it can be easy to step out of your view and end up confusing Django library code.

6. Click the **Step Over** button once to execute line 5.
7. You can see that the name variable has been added to the list of variables in the debugger view, and its value is world.



```
10 01 request.method = {str} 'GET'
10 01 name = {str} 'world'
> [icon] request = {WSGIRequest} <WSGIRequest: GET '/'>
```

Figure 1.38 – The new variable name is now in scope, with the world value

We are now at the end of our `index` view function, and if we were to step over this line of code, it would jump to Django library code, which we don't want to see. To continue executing and to send the response back to your browser, click the **Resume Program** button, the third from the left on the debugger toolbar (Figure 1.39). You should see that your browser has now loaded the page again:

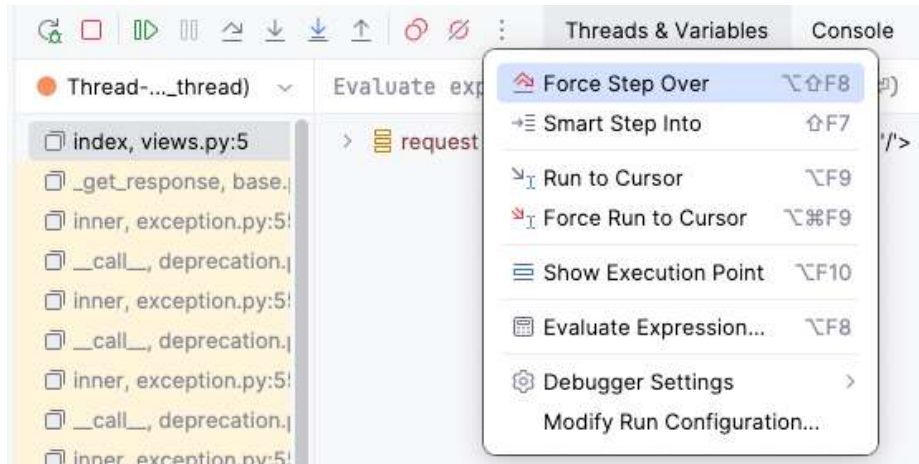


Figure 1.39 – Actions to control execution – the green play icon is the Resume Program button

Clicking on the vertical ellipsis button, `:`, reveals more buttons (see Figure 1.50); from the top, they are **Force Step Over** (steps over the current code line, ignoring any method calls to the next line), **Smart Step Into** (allows cursor selection of a method from the current line of code), **Run to Cursor** (keeps running the code to the current line under the cursor, suspending the program at any breakpoints), **Force Run to Cursor** (keeps running the code to the current line under the cursor, ignoring any breakpoints), **Show Execution Point** (focuses on the execution point in the editor window and the stack frame in the panel), **Evaluate Expression...** (evaluates an expression from the editor window), **Debugger Settings** (menu for configurable settings), and **Modify Run Configuration...** (opens the run configuration window).

8. For now, turn off the breakpoint in PyCharm by clicking on it (the red circle next to line 5).

```

4     def index(request):
5         name = "world"
6         return render(request, template_name="base.html", context={"name":

```

Figure 1.40 – Clicking the breakpoint that was on line 5 disables it

This is just a quick introduction to how to set breakpoints in PyCharm. If you have used debugging features in other IDEs, then you should be familiar with the concepts – you can step through code, step in and out of functions, or evaluate expressions. Once you have set a breakpoint, you can right-click on it to change the options. For example, you can make the breakpoint conditional so that the execution only stops under certain circumstances. All these

are beyond the scope of this book, but are useful to know about when trying to solve problems in your code.

We're going to finish the chapter with two activities. First, you'll build a welcome screen for Bookr, and then you'll scaffold the book-searching page.

Activity 1.01 – creating a site welcome screen

The Bookr website that we are building needs to have a splash page that welcomes users and lets them know what site they are on. It will also contain links to other parts of the site, but these will be added in later chapters. For now, you will create a page with a welcome message.

These steps will help you complete the activity:

1. In your `index` view, render the `base.html` template.
2. Update the `base.html` template to contain the welcome message. It should be in both the `<title>` tag in `<head>` and in a new `<h1>` tag in the body.

After completing the activity, you should be able to see something like this:



Figure 1.41 – The Bookr welcome page

In the next activity, you will build a basic page to show the results of a book search in Bookr.

Activity 1.02 – a book search scaffold

A useful feature of a site such as Bookr is the ability to search through data to find something on the site quickly. Bookr will implement book searching to allow users to find a particular book by part of its title. While we don't have any books to find yet, we can still implement a page that shows the text a user searched for. The user enters the search string as part of the URL parameters. We will implement searching and a form for easy text entry in *Chapter 6, Forms*.

These steps will help you complete the activity:

1. Create a search result HTML template. It should include a variable placeholder to show the search word(s) that were passed in through the render context. Show the passed-in variable in the `<title>` and `<h1>` tags. Use an `<em>` tag around the search text in the body to make it italic.
2. Add a search view function in `views.py`. The view should read a search string from the URL parameters (in the request's GET attribute). It should then render the template you created in the previous step, passing in the search value to be substituted, using the context dictionary.
3. Add a URL mapping to your new view to `urls.py`. The URL can be something such as `/book-search`.

After completing this activity, you should be able to pass in a search value through the URL's parameters and see it rendered on the resulting page. It should look like this:



Figure 1.42 – Searching Django workshop

You should also be able to pass in special HTML characters such as `<` and `>` to see how Django automatically escapes them in the template.



Figure 1.43 – Note how HTML characters are escaped so that we are protected from tag injection

Summary

This chapter was a quick introduction to Django. You first got up to speed on the HTTP protocol and the structure of HTTP requests and responses. We then saw how Django uses the MVT paradigm, and then how it parses a URL, generates an HTTP request, and sends it to a view to get an HTTP response. We scaffolded the *bookr* project and then created the *reviews* app for it. We then built two example views to illustrate how to get data from a request and use it when rendering templates. You also experimented to see how Django escapes output in HTML when rendering a template.

We did all this with the PyCharm IDE, and you learned how to set it up to debug your application. The debugger will help you find out why things aren't working as they should.

In the next chapter, you will start to learn about Django's database integration and its model system, so you can start storing and retrieving real data for your application.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

2

Models and Migrations

In this chapter, we will learn about Django's **object-relational mapping (ORM)**, through which our application can interact and seamlessly work with a relational database using simple Python code, eliminating the need to run complex SQL queries. We will learn about models and migrations, which are part of Django's ORM, used to propagate database schema changes from the application to the database, and also perform database **Create, Retrieve, Update, Delete (CRUD)** operations. Toward the end of the chapter, we will study the various types of database relationships and use that knowledge to perform queries across related records.

We will cover the following topics in this chapter:

- Exploring Django ORM
- Creating Django models and migrations
- Django's database CRUD operations
- Bulk create and bulk update operations
- Performing complex lookups using Q objects
- Activity 2.01 – Create models for a project management application
- Populating the Bookr project's database

By the end of this chapter, we will have a good understanding of databases and how Django can be used to implement database operations for a web application.

Technical requirements

Find the solution in the Chapter02 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Exploring Django ORM

Web applications constantly interact with databases, and one of the ways to do so is by using SQL. If you decide to write a web application without a web framework, such as Django, and instead use Python alone, Python libraries such as `psycopg2` (for Postgres DB) could be used to interact directly with the databases using SQL commands. But while developing a web application with multiple tables and fields, SQL commands can easily become overly complex and thus difficult to maintain. For this reason, popular web frameworks such as Django provide a level of abstraction that allows us to easily work with databases. The part of Django that helps us do this is called the ORM.

Django's ORM converts object-oriented Python code into actual database constructs, such as database tables with data type definitions, and facilitates all the database operations via simple Python code. Because of this, we do not have to deal with SQL commands while performing database operations. This helps in faster application development and ease in maintaining the application source code.

Django supports relational databases such as SQLite, PostgreSQL, Oracle Database, and MySQL. Django's database abstraction layer ensures that the same Python/Django source code can be used across any of the preceding relational databases with very little modification to the project settings. Since SQLite is part of the Python libraries and Django is configured by default to use SQLite, for the scope of this chapter, we shall use SQLite while we learn about Django models and migrations. Next, we will understand how database configuration is done with Django.

Database configuration and creating Django applications

As we have already seen in *Chapter 1*, when we create a Django project and run the Django server, the default database configuration is SQLite3. The database configuration will be present in the project directory, in the `settings.py` file.

Note

Make sure you examine the `settings.py` file for the `bookr` app. Going through the entire file once will help you understand the concepts that follow. You can find the file in the `Chapter02` folder in this book's GitHub repo.

So, for our example project, the database configuration will be present at the following location: `bookr/settings.py`. The default database configuration present in this file when a Django project is created is as follows:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

The `DATABASES` variable is assigned a dictionary containing the database details for the project. Inside the dictionary, there is a nested dictionary with a key of `default`. This holds the configuration of a default database for the Django project. The reason we have a nested dictionary with `default` as a key is that a Django project could potentially interact with multiple databases, and the default database is the one used by Django for all operations unless explicitly specified. The `ENGINE` key represents which database engine is being used; in this case, `sqlite3`.

The `NAME` key defines the name of the database, which can have any value. But for `SQLite3`, since the database is created as a file, `NAME` can have the full path of the directory where the file needs to be created. The full path of the `db` file is processed by joining (or concatenating) the previously defined path in `BASE_DIR` with `db.sqlite3`. Note that `BASE_DIR` is the project directory as already defined in the `settings.py` file. Because `BASE_DIR` is defined as `pathlib.Path`, the `/` operator in this line of code is the overloaded `pathlib.Path.__truediv__` method, rather than a numerical operator. This operator is used to join two paths so that `pathlib.Path('a') / 'b'` is equal to `pathlib.Path('a/b')`.

If you are using other databases, such as PostgreSQL, MySQL, and so on, changes will have to be made in the preceding database settings, as shown here:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'bookr',
        'USER': 'django',
        'PASSWORD': '0b3ef4b0d80c28db',
        'HOST': '127.0.0.1',
        'PORT': '5432', }}
}
```

Here, changes have been made to `ENGINE` to use PostgreSQL. The host IP address and port number of the server need to be provided for `HOST` and `PORT`, respectively. As the names suggest, `USER` is the database username, and `PASSWORD` is the database password. In addition to changes in the configuration, we will have to install the database drivers or bindings along with the database host and credentials. Note that this code block is just an example to show the changes you'll need to make to use a different database, such as PostgreSQL, but since we are using SQLite, we shall use the database configuration that exists already, and there is no need to make any modifications to the database settings. Next, we will briefly read about Django apps and some of the default apps present in Django.

Django migration

As we have learned before, Django's ORM helps make database operations simpler. A major part of the operation is transforming the Python code into database structures, such as database fields with stated data types and tables. In other words, the transformation of Python code into database structures is known as **migration**. For example, instead of creating database tables by running SQL commands, you would write models for them in Python; something we'll learn to do in the upcoming *Creating models and migrations* section. These models will have **fields** that form the blueprints of database tables. The fields, in turn, will have different field types, giving us more information about the type of data stored there (recall how we specified the data type of our field as `TEXT` in *step 4 of Exercise 2.01, Creating an entry in the bookr database*).

Since we have a Django project set up, let's perform our first migration. Although we have not added any code to our project, we can migrate the applications listed in `INSTALLED_APPS`. This is necessary because Django's installed apps need to store the relevant data in the database for their operations, and migration will create the required database tables to store the data in the database.

The following is a snippet from the Bookr project's `settings.py` file:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'reviews',  
]
```

This code block contains a set of installed or default apps used for the admin site, authentication, content types, sessions, messaging, static file management, and the review app that we started in *Chapter 1*.

The following command should be entered in the terminal or shell to do this:

```
python manage.py migrate
```

By executing this command, we create all the database structures required by the installed apps.

For better visualization of the data in the SQLite database, we will use an open source tool called **DB Browser for SQLite**. This tool helps visualize the data and provides a shell to execute the SQL commands.

If you haven't done so already, visit the <https://sqlitebrowser.org> URL, and from the downloads section, install the application as per your operating system and launch it.

Let's take a look at the database for which changes have been made after executing the `migrate` command using DB Browser for SQLite.

The database file will be created in the project directory under the `db.sqlite3` name.

Note

Database operations can also be performed using a command-line shell by using the `sqlite3` command `sqlite3 db.sqlite3`.

Follow these steps to verify that migration has been successful and database tables have been created:

1. Open DB Browser, click **Open Database**, navigate until you find the `db.sqlite3` file, and open it. You should see a set of newly created tables created by the Django migration. It will look as follows in DB Browser:

Name	Type	Schema
Tables (11)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIMARY KEY
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT NULL
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL PRIMAR
auth_user		CREATE TABLE "auth_user" ("id" integer NOT NULL PRIMARY KEY
auth_user_groups		CREATE TABLE "auth_user_groups" ("id" integer NOT NULL PRIM
auth_user_user_permissions		CREATE TABLE "auth_user_user_permissions" ("id" integer NOT
django_admin_log		CREATE TABLE "django_admin_log" ("id" integer NOT NULL PRIM
django_content_type		CREATE TABLE "django_content_type" ("id" integer NOT NULL PRI
django_migrations		CREATE TABLE "django_migrations" ("id" integer NOT NULL PRIM
django_session		CREATE TABLE "django_session" ("session_key" varchar(40) NOT
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq)
Indices (15)		
Views (0)		
Triggers (0)		

Figure 2.1: Contents of the db.sqlite3 file

- Now, browse through the newly created database structure by clicking the database tables, and we should see the following:

Name	Type	Schema
Tables (11)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIM
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
name	varchar(150)	"name" varchar(150) NOT NULL UNIQUE
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
group_id	integer	"group_id" integer NOT NULL
permission_id	integer	"permission_id" integer NOT NULL
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL F
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
content_type_id	integer	"content_type_id" integer NOT NULL
codename	varchar(100)	"codename" varchar(100) NOT NULL
name	varchar(255)	"name" varchar(255) NOT NULL

Figure 2.2: Browsing through the newly created database structure

- Notice that the database tables created have different fields, each with its respective data types. Click on the **Browse data** tab in DB Browser and select a table from the drop-down menu. For instance, after clicking the auth_group_permissions table, you should see something like this:

Table:	auth_group_permissions				
id	group_id	permission_id			
Filter	Filter	Filter			

Figure 2.3: Viewing the auth_group_permissions table

You will see that there is no data available for these tables yet because Django migration only creates the database structure or the blueprint, and the actual data in the database is stored during the operation of the application. Now that we have migrated the built-in or default Django apps, let's try to create an app and perform a Django migration.

Creating Django models and migrations

A Django model is essentially a Python class that holds the blueprint for creating a table in a database. The `models.py` file can have many such models, and each model transforms into a database table. The attributes of the class form the fields and relationships of the database table as per the model definitions.

For our reviews application, we need to create the following models and their database tables:

- **Book:** This should store information about books
- **Contributor:** This should store information about the person(s) who contributed to writing the book, such as the author, co-author, or editor
- **Publisher:** As the name implies, this refers to the book's publisher
- **Review:** This should store all the book reviews written by the users of the application

Every book in our application will need to have a publisher, so let's create `Publisher` as our first model. Enter the following code in `reviews/models.py`:

```
from django.db import models

class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
        max_length=50,
        help_text="The name of the Publisher.")
    website = models.URLField(
        blank=True,
        help_text="The Publisher's website.")
    email = models.EmailField(
        blank=True,
        help_text="The Publisher's email address.")
```


Note

You can take a look at the complete `models.py` file for the bookr app in the `Chapter02` folder in this book's GitHub repo.

The first line of code imports the Django `models` module. While this line will be autogenerated at the time of the creation of the Django app, do make sure you add it if it is not present. Following the import, the rest of the code defines a class named `Publisher`, a subclass of Django's `models.Model`. Furthermore, this class will have attributes or fields such as `name`, `website`, and `email`. The following are the field types used while creating this model.

As we can see, each of these fields is defined to have the following types:

- `CharField`: This field type stores shorter string fields, such as `Packt Publishing`. For very large strings, we use `TextField`.
- `EmailField`: This is similar to `CharField` but validates whether the string represents a valid email address – for example, `customersupport@packtpub.com`.
- `URLField`: Again, this is similar to `CharField`, but validates whether the string represents a valid URL – for example, `https://www.packtpub.com`.

Next, we will look at the field options used while creating each of these fields.

Field options

Django provides a way to define field options for a model's field. These field options are used to set a value or a constraint, and so on. For example, we can set a default value for a field using `default=<value>` to ensure that every time a record is created in the database for the field, it is set to a default value specified by us. The following are the three field options that we have used while defining the `Publisher` model:

- `blank`: The default is `False`. If this option is set to `True`, then blank values will be allowed for this field during validation of the Django form.
- `help_text`: This field option helps us add descriptive text for a field that gets automatically included for Django forms.

- `max_length`: This option is provided to `CharField`, where it defines the maximum length of the field in terms of the number of characters.
- Other field options that are frequently used include the following:
 - `null`: The default is `False`. If this option is set to `True`, then a `NULL` value is allowable for the field at the database level.
 - `unique`: The default is `False`. If this option is set to `True`, then the database column corresponding to the field will be defined as unique. Duplicate values for this field will lead to a validation error.

Django has many more field types and field options that can be explored from the extensive official Django documentation. As we go about developing our sample book review application, we will learn about the types and fields used for the project. Now, let's migrate the Django models into the database by following these steps:

1. Execute the following command in the shell or terminal to do that (run it from the folder where your `manage.py` file is stored):

```
python manage.py makemigrations reviews
```

The output of the command looks like this:

```
Migrations for 'reviews':
  reviews/migrations/0001_initial.py
    - Create model Publisher
```

The `makemigrations <appname>` command creates the migration scripts for the given app – in this case, for the `reviews` app. Notice that after running `makemigrations`, there is a new file created under the `migrations` folder:



Figure 2.4: New file under the migrations folder

This is the migration script created by Django. When we run `makemigrations` without the app name, the migration scripts will be created for all the apps in the project.

2. Next, let's list the project migration status. Remember that earlier, we applied migrations to Django's installed apps, and now we have created a new app, reviews. Run the following command in the shell or terminal, and it will show the status of model migrations throughout the project (run it from the folder where your `manage.py` file is stored):

```
python manage.py showmigrations
```

The output for the preceding command is as follows:

```
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
reviews
[ ] 0001_initial
sessions
[X] 0001_initial
```

Here, the [X] mark indicates that the migrations have been applied. Notice the difference that all the other apps' migrations have applied except that of reviews. The

showmigrations command can be executed to understand the migration status, but this is not a mandatory step while performing model migrations.

3. Next, let's understand how Django transforms a model into an actual database table. Django provides the sqlmigrate command to show the actual SQL generated by a migration. This isn't a mandatory step in model migrations, but it can be useful for further understanding.

We should see the following output:

```
BEGIN;
--
-- Create model Publisher
--
CREATE TABLE "reviews_publisher" ("id" integer
NOT NULL PRIMARY KEY AUTOINCREMENT, "name"
varchar(50) NOT NULL, "website" varchar(200)
NULL, "email" varchar(254) NULL);
COMMIT;
```

The preceding snippet shows the SQL command equivalent used when Django migrates the database. In this case, we are creating the reviews_publisher table with the name, website, and email fields with the defined field types. Furthermore, the name field is defined as NOT NULL, implying that the entries for this field cannot be null and should have a value. The field options null=True or unique=True will lead to a SQL definition with NULL or UNIQUE, respectively.

In the next section, we will learn about primary keys and their importance while storing data in a database.

Primary keys

Let's assume that a database table called users, as its name suggests, stores information about users. Let's say it has more than 1,000 records, and there are at least 3 users with the same name, Joe Burns. How do we uniquely identify these users from the application? The solution is to have a way to uniquely identify each record in the database. This is done using **primary keys**. A primary key is unique for a database table, and as a rule, a table cannot have two rows with the same primary key. In Django, when the primary key is not explicitly mentioned in the database models, Django automatically creates id as the primary key (of type integer), which auto-increments as new records are created.

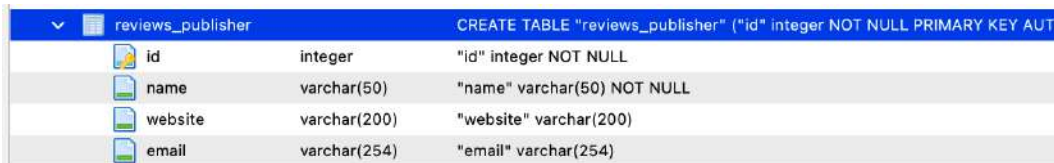
In the previous section, notice the output of the `python manage.py sqlmigrate` command. While creating the `Publisher` table, the SQL `CREATE TABLE` command added one more field called `id` to the table. An `id` is defined as `PRIMARY KEY AUTOINCREMENT`. In relational databases, a primary key is used to uniquely identify an entry in the database. For example, the `book` table has `id` as the primary key, which has numbers starting from 1. This value increments by one as new records are created. The integer value of `id` is always unique across the `book` table. Since the migration script has already been created by executing `makemigrations`, let's now migrate the newly created model in the `reviews` app by executing the following command:

```
python manage.py migrate reviews
```

We should get the following output:

```
Operations to perform:
  Apply all migrations: reviews
Running migrations:
  Applying reviews.0001_initial... OK
```

This operation creates the database table for the `reviews` app. The following is a snippet from DB Browser indicating the new `reviews_publisher` table has been created in the database:

A screenshot of a database browser interface showing the structure of the 'reviews_publisher' table. The table is expanded, showing four columns: 'id', 'name', 'website', and 'email'. Each column has a data type and a description. The 'id' column is an integer, not null, and is the primary key. The 'name', 'website', and 'email' columns are varchar types with lengths of 50, 200, and 254 respectively, and are not null.

reviews_publisher		CREATE TABLE "reviews_publisher" ("id" integer NOT NULL PRIMARY KEY AUTOINCR...
id	integer	"id" integer NOT NULL
name	varchar(50)	"name" varchar(50) NOT NULL
website	varchar(200)	"website" varchar(200)
email	varchar(254)	"email" varchar(254)

Figure 2.5: The `reviews_publisher` table created after executing the migration command

So far, we have explored how to create a model and migrate it into the database. Let's now work on creating the rest of the models for our book review application. As we've already seen, the application will have database tables for `Book`, `Publisher`, `Contributor`, and `Review`.

Let's add the `Book` and `Contributor` models, as shown in the following code snippet, into `reviews/models.py`:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField()
```

```
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")

class Contributor(models.Model):
    """A contributor to a Book, e.g. author, editor,
        co-author."""
    first_names = models.CharField(
        max_length=50, help_text="The contributor's first
        name or names.")
    last_names = models.CharField(
        blank=True, max_length=50,
        help_text="The contributor's last name or names.")
    email = models.EmailField(
        blank=True,
        help_text="The contact email for the contributor.")
```

The code is self-explanatory. The Book model has the title, publication_date, and isbn fields. The Contributor model has the first_names and last_names fields and the email ID of the contributor. There are also some newly added models, apart from the ones we have seen in the Publisher model. They have DateField as a new field type, which, as the name suggests, is used to store a date. A new field option called verbose_name is also used. It provides a descriptive name for the field. Next, we will see how relationships work in a relational database.

Relationships

One of the powers of relational databases is the ability to establish relationships between data stored across database tables. Relationships help maintain data integrity by establishing the correct references across tables, which in turn helps maintain the database. Relationship rules, on the other hand, ensure data consistency and prevent duplicates.

In a relational database, there can be the following types of relations:

- Many to one
- Many to many
- One to one

Let's explore each relationship in detail in the following sections.

Many to one

In this relationship, many records (rows/entries) from one table can refer to one record (row/entry) in another table. For example, there can be many books produced by one publisher. This is an example of a **many-to-one relationship**. To establish this relationship, we need to use the database's foreign keys. A foreign key in a relational database establishes the relationship between a field from one table and a primary key from a different table.

For example, say we have data about employees belonging to different departments stored in a table called `employee_info` with their employee ID as the primary key alongside a column that stores their department name; this table also contains a column that stores that department's department ID. Now, there's another table called `departments_info`, which has the department ID as the primary key. In this case, then, the department ID is a foreign key in the `employee_info` table.

In our bookr app, the `Book` model can have a foreign key referring to the primary key of the `Publisher` table. Since we have already created the models for `Book`, `Contributor`, and `Publisher`, let's now establish a many-to-one relationship across the `Book` and `Publisher` models. For the `Book` model, add the last line:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
```

Now the newly added `publisher` field is establishing a many-to-one relationship between `Book` and `Publisher` using a foreign key. This relationship ensures the nature of a many-to-one relationship, which is that many books can have one publisher:

- `models.ForeignKey`: This is the field option to establish a many-to-one relationship.

- **Publisher:** When we establish relationships with different tables in Django, we refer to the model that creates the table; in this case, the Publisher table is created by the Publisher model (or the Publisher Python class).
- **on_delete:** This is a field option that determines the action to be taken upon the deletion of the referenced object. In this case, the `on_delete` option is set to `CASCADE(models.CASCADE)`, which deletes the referenced objects.

For example, assume a publisher has published a set of books. For some reason, if the publisher has to be deleted from the application, the next action is `CASCADE`, which means deleting all the referenced books from the application. There are many more `on_delete` actions, such as the following:

- **PROTECT:** This prevents the record deletion unless all the referenced objects are deleted
- **SET_NULL:** This sets a null value if the database field has been previously configured to store null values
- **SET_DEFAULT:** Sets to a default value on the deletion of the referenced object

We will only use the `CASCADE` option for our book review application.

Many to many

In this relationship, multiple records in a table can have a relationship with multiple records in a different table. For example, a book can have multiple co-authors, and each author (contributor) could have written multiple books. So, this forms a **many-to-many relationship** between the Book and Contributor tables:

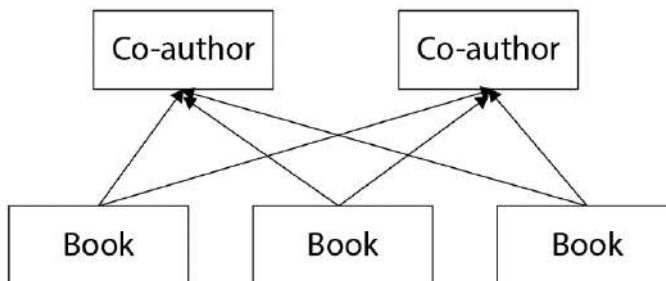


Figure 2.6: Many-to-many relationship between books and co-authors

In `models.py`, for the Book model, add the last line as shown here:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
```



```

        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
    contributors = models.ManyToManyField(
        'Contributor', through="BookContributor")

```

The newly added `contributors` field establishes a many-to-many relationship with `Book` and `Contributor` using the `ManyToManyField` field type:

- `models.ManyToManyField`: This is the field type to establish a many-to-many relationship.
- `through`: This is a special field option for many-to-many relationships. When we have a many-to-many relationship across two tables, if we want to store some extra information about the relationship, then we can use this to establish the relationship via an intermediary table.

For example, we have two tables, namely `Book` and `Contributor`, where we need to store the information on the type of contributor for the book, such as `Author`, `Co-author`, or `Editor`. Then the type of contributor is stored in an intermediary table called `BookContributor`. Here is how the `BookContributor` table/model looks. Make sure you include this model in `reviews/models.py`:

```

class BookContributor(models.Model):
    class ContributionRole(models.TextChoices):
        AUTHOR = "AUTHOR", "Author"
        CO_AUTHOR = "CO_AUTHOR", "Co-Author"
        EDITOR = "EDITOR", "Editor"

    book = models.ForeignKey(
        Book, on_delete=models.CASCADE)
    contributor = models.ForeignKey(
        Contributor, on_delete=models.CASCADE)
    role = models.CharField(
        verbose_name="The role this contributor had in"

```

```
" the book.", choices=ContributionRole.choices,  
max_length=20)
```

An intermediary table, such as `BookContributor`, establishes relationships by using foreign keys to both the `Book` and `Contributor` tables. It can also have extra fields that can store information about the relationship the `BookContributor` model has with the following fields:

- `book`: This is a foreign key to the `Book` model. As we saw previously, `on_delete=models.CASCADE` will delete an entry from the relationship table when the relevant book is deleted from the application.
- `Contributor`: Again, this is a foreign key to the `Contributor` model/table. This is also defined as `CASCADE` upon deletion.
- `role`: This is the field of the intermediary model, which stores the extra information about the relationship between `Book` and `Contributor`.
- `class ContributionRole(models.TextChoices)`: This can be used to define a set of choices by creating a subclass of `models.TextChoices`. For example, `ContributionRole` is a subclass created out of `TextChoices`, which is used by the `roles` field to define `Author`, `Co-Author`, and `Editor` as a set of choices.
- `choices`: This refers to a set of choices defined in the models, and they are useful when creating Django Forms using the models.

Note

When the `through` field option is not provided while establishing a many-to-many relationship, Django automatically creates an intermediary table to manage the relationship.

One-to-one relationships

In this relationship, one record in a table will have a reference to only one record in a different table. For example, a person can have only one driver's license, so a person with a driver's license could form a one-to-one relationship:

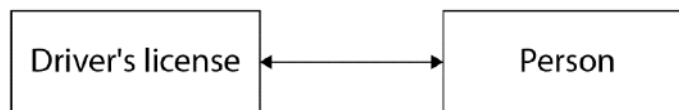


Figure 2.7: Example of a one-to-one relationship

OneToOneField can be used to establish a one-to-one relationship, as shown here:

```
class DriverLicence(models.Model):
    person = models.OneToOneField(
        Person, on_delete=models.CASCADE)
    licence_number = models.CharField(max_length=50)
```

Now that we have explored database relationships, let's come back to our bookr application and add one more model there.

Adding the Review model

We've already added the Book and Publisher models to the reviews/models.py file. The last model that we are going to add is the Review model. The following code snippet should help us do this:

```
from django.contrib import auth
class Review(models.Model):
    content = models.TextField(
        help_text="The Review text.")
    rating = models.IntegerField(
        help_text="The rating the reviewer has given.")
    date_created = models.DateTimeField(
        auto_now_add=True,
        help_text="The date and time the"
        " review was created.")
    date_edited = models.DateTimeField(
        null=True,
        help_text="The date and time the review"
        " was last edited.")

    creator = models.ForeignKey(
        auth.get_user_model(), on_delete=models.CASCADE)
    book = models.ForeignKey(
        Book, on_delete=models.CASCADE,
        help_text="The Book that this review is for.")
```

The Review model/table will be used to store user-provided review comments and ratings for books. It has the following fields:

- **content:** This field stores the text for a book review; hence, the field type used is `TextField`, as this can store a large amount of text.
- **rating:** This field stores the review rating of a book. Since the rating will be an integer, the field type used is `IntegerField`.
- **date_created:** This field stores the time and date when the review was written; hence, the field type is `DateTimeField`.
- **date_edited:** This field stores the date and time whenever a review is edited. Again, the field type is `DateTimeField`.
- **creator:** This field specifies the review creator or the person who writes the book review. Notice that this is a foreign key to `auth.get_user_model()`, which refers to the `User` model from Django's built-in authentication module. It has a `on_delete=models.CASCADE` field option. This explains that when a user is deleted from the database, all the reviews written by that user will be deleted.
- **book:** Reviews have a field called `book`, a foreign key to the `Book` model. This is because reviews have to be written for a book review application, and a book can have many reviews, so this is a many-to-one relationship. This is also defined with an `on_delete=models.CASCADE` field option because once the book is deleted, there is no point in retaining the reviews in the application. So, when a book is deleted, all the reviews referring to the book will also get deleted.

In the next section, we will learn about and implement model methods.

Model methods

In Django, we can write methods inside a model class. These are called **model methods**, and they can be custom or special methods that override the default methods of Django models. One such method is `__str__()`. This method returns the string representation of the `Model` instances and can be especially useful while using the Django shell. In the following example, where the `__str__()` method is added to the `Publisher` model, the string representation of the `Publisher` object will be the publisher's name; hence, `self.name` is returned, with `self` referring to the `Publisher` object:

```
class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
```

```
        max_length=50,
        help_text="The name of the Publisher.")
website = models.URLField(
    help_text="The Publisher's website.")
email = models.EmailField(
    blank=True,
    help_text="The Publisher's email address.")

def __str__(self):
    return self.name
```

Add the `__str__()` methods to `Contributor` and `Book` as well, as follows:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
    contributors = models.ManyToManyField(
        'Contributor', through="BookContributor")

    def __str__(self):
        return self.title

class Contributor(models.Model):
    """A contributor to a Book, e.g. author, editor,
    co-author."""
    first_names = models.CharField(
        max_length=50,
        help_text="The contributor's first name or names.")
    last_names = models.CharField(
        blank=True, max_length=50,
        help_text="The contributor's last name or names.")
    email = models.EmailField(
```

```
blank=True,  
help_text="The contact email for the contributor.")  
  
def __str__(self):  
    return self.first_names
```

Similarly, the string representation of book is title, so the returned value is `self.title`, with `self` referring to the Book object. The string representation of Contributor is the first name of the contributor; hence `self.first_names` is returned. Here, `self` is referring to the Contributor object. Next, we will look at migrating the reviews app.

Migrating the reviews app

Since we have the entire model file ready, let's now migrate the models into the database, similar to what we did before with the installed apps. Since the reviews app has a set of models created by us, it is important to create the migration scripts before running the migration. Migration scripts help in identifying any changes to the models and will propagate these changes into the database while running the migration. Follow these steps to create migration scripts and then migrate models into the database:

1. Execute the following command to create the migration scripts:

```
python manage.py makemigrations reviews
```

We should get an output similar to this:

```
reviews/migrations/0002_auto_20191007_0112.py  
- Create model Book  
- Create model Contributor  
- Create model Review  
- Create model BookContributor  
- Add field contributors to book  
- Add field publisher to book
```

Migration scripts will be created in a folder named migrations in the application folder.

2. Next, migrate all the models into the database using the migrate command:

```
python manage.py migrate reviews
```

We should see the following output:

```
Operations to perform:
  Apply all migrations: reviews
Running migrations:
  Applying reviews.0002_auto_20191007_0112... OK
```

After executing this command, we successfully created the database tables defined in the reviews app. You may use DB Browser for SQLite to explore the tables you have just created after the migration.

3. To do so, open DB Browser for SQLite, click on the **Open Database** button (*Figure 2.17*), and navigate to your project directory.
4. Select the `db.sqlite3` database file to open it.

We should now be able to browse the new sets of tables created. The following screenshot shows the database tables defined in the reviews app:

▼	reviews_book		CREATE TABLE "reviews_book" ("id" Integer NOT NULL PRIMARY KEY,
	id	Integer	"id" Integer NOT NULL
	title	varchar(70)	"title" varchar(70) NOT NULL
	publication_date	date	"publication_date" date NOT NULL
	isbn	varchar(20)	"isbn" varchar(20)
	publisher_id	bigint	"publisher_id" bigint NOT NULL
▼	reviews_bookcontributor		CREATE TABLE "reviews_bookcontributor" ("id" Integer NOT NULL PRI
	id	Integer	"id" Integer NOT NULL
	role	varchar(20)	"role" varchar(20) NOT NULL
	book_id	bigint	"book_id" bigint NOT NULL
	contributor_id	bigint	"contributor_id" bigint NOT NULL
▼	reviews_contributor		CREATE TABLE "reviews_contributor" ("id" Integer NOT NULL PRIMAR
	id	Integer	"id" Integer NOT NULL
	first_names	varchar(50)	"first_names" varchar(50) NOT NULL
	last_names	varchar(50)	"last_names" varchar(50)
	email	varchar(254)	"email" varchar(254)
▼	reviews_publisher		CREATE TABLE "reviews_publisher" ("id" Integer NOT NULL PRIMARY
	id	Integer	"id" Integer NOT NULL
	name	varchar(50)	"name" varchar(50) NOT NULL
	website	varchar(200)	"website" varchar(200)
	email	varchar(254)	"email" varchar(254)
▼	reviews_review		CREATE TABLE "reviews_review" ("id" Integer NOT NULL PRIMARY KEY
	id	Integer	"id" Integer NOT NULL
	content	text	"content" text NOT NULL
	rating	Integer	"rating" Integer NOT NULL
	date_created	datetime	"date_created" datetime NOT NULL
	date_edited	datetime	"date_edited" datetime
	book_id	bigint	"book_id" bigint NOT NULL
	creator_id	Integer	"creator_id" Integer NOT NULL

Figure 2.8: Database tables as defined in the reviews app

In this section, we learned more about Django models and migrations and how Python's simple classes can transform themselves into database tables. We also learned about how various class attributes translate into appropriate database columns following the defined field types. Later, we learned about primary keys and different types of relationships that can exist in a database. We also created models for the book review application and migrated those models, translating them into database tables. In the next section, we will learn about how to perform database CRUD operations using Django's ORM.

Django's database CRUD operations

As we have created the necessary database tables for the book review application, let's work on understanding the basic database operations with Django.

CRUD is an acronym for **Create, Read, Update, Delete** – the four primary operations of a database system. In SQL, these operations are implemented with the `INSERT`, `SELECT`, `UPDATE`, and `DELETE` statements.

Django's ORM provides the same functionality without having to deal with SQL statements. Django's database operations are simple Python code, hence we overcome the hassle of maintaining SQL statements within the Python code. Let's take a look at how these are performed.

To execute the CRUD operations, we will enter Django's command-line shell by executing the following command:

```
python manage.py shell
```

When the interactive console starts, it looks as follows:

```
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
```

In the following exercise, we shall perform a create operation.

Exercise 2.01 – Creating an entry in the bookr database

In this exercise, we will create a new entry in the database by saving a model instance. In other words, we will create an entry in a database table without explicitly running a SQL query. Follow these steps to do this:

1. First, import the `Publisher` class/model from `reviews.models`:

```
>>> from reviews.models import Publisher
```

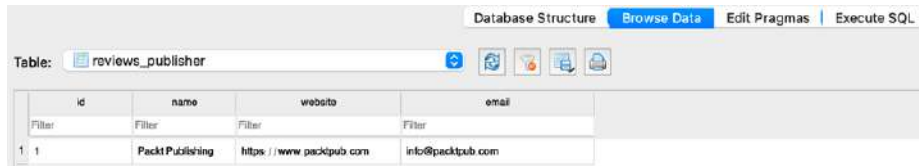
2. Create an object or an instance of the `Publisher` class by passing all the field values (name, website, and email) required by the `Publisher` model:

```
>>> publisher = Publisher(name='Packt Publishing',
                          website='https://www.packtpub.com',
                          email='info@packtpub.com')
```

3. Next, to write the object into the database, it is important to call the `save()` method, because until this is called, there will not be an entry created in the database:

```
>>> publisher.save()
```

Now we can see a new entry created in the database using DB Browser:



The screenshot shows the DB Browser interface with the 'reviews_publisher' table selected. The table has four columns: id, name, website, and email. A single entry is visible with id 1, name 'Packt Publishing', website 'https://www.packtpub.com', and email 'info@packtpub.com'.

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com	info@packtpub.com

Figure 2.9: An entry created in the database

4. Use the object attributes to make any further changes to the object and save the changes to the database:

```
>>> publisher.email = 'info@packtpub.com'
>>> publisher.email = 'customersupport@packtpub.com'
>>> publisher.save()
```

We can see the changes using DB Browser as follows:



The screenshot shows the DB Browser interface with the 'reviews_publisher' table selected. The table has four columns: id, name, website, and email. A single entry is visible with id 1, name 'Packt Publishing', website 'https://www.packtpub.com', and email 'customersupport@packtpub.com'.

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com	customersupport@packtpub.com

Figure 2.10: An entry with the updated email field

In this exercise, we created an entry in the database by creating an instance of the model object and used the `save()` method to write the model object into the database.

Note that by following the preceding method, the changes to the class instance are not saved until the `save()` method is called. However, if we use the `create()` method, Django saves the changes to the database in a single step. We'll use this method in the exercise that follows.

Exercise 2.02 – Using the create() method to create an entry

Unlike the previous exercise, where we executed two separate steps by first creating an object and later using the save() method to create an entry in the database, in this exercise, we will create a record in the contributor table using the create() method in a single step:

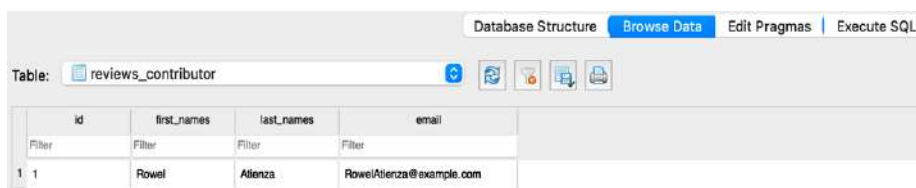
1. First, import the Contributor class as before:

```
>>> from reviews.models import Contributor
```

2. Invoke the create() method to create an object in the database in a single step. Ensure that you pass all the required parameters (first_names, last_names, and email):

```
>>> contributor = Contributor.objects.create(
    first_names="Rowel", last_names="Atienza",
    email="RowelAtienza@example.com")
```

3. Use DB Browser to verify that the contributor record has been created in the database. If your DB Browser is not already open, open the db.sqlite3 database file as we just did in the previous section. Click **Browse Data** and select the desired table – in this case, the reviews_contributor table from the **Table** drop-down menu, as shown in the screenshot – and verify the newly created database record:



The screenshot shows the DB Browser interface with the 'reviews_contributor' table selected. The table has four columns: id, first_names, last_names, and email. A single record is displayed with id 1, first_names 'Rowel', last_names 'Atienza', and email 'RowelAtienza@example.com'.

id	first_names	last_names	email
1	Rowel	Atienza	RowelAtienza@example.com

Figure 2.11: Verifying the creation of the record in DB Browser

In this exercise, we learned that using the create() method, we can create a record for a model in a database in a single step.

Creating an object with a foreign key

Similar to how we created a record in the Publisher and Contributor tables, let's now create one for the Book table. If you recall, the Book model has a foreign key to Publisher that cannot have a null value. So, a way to populate the publisher's foreign key is by providing the created publisher object in the book's publisher field, as shown in the following exercise.

Exercise 2.03 – Creating records for a many-to-one relationship

In this exercise, we will create a record in the Book table, including a foreign key to the Publisher model. As you already know, the relationship between Book and Publisher is a many-to-one relationship, so you have to first fetch the Publisher object and then use it while creating the book record. To do that, follow these steps:

1. First, import the Publisher class:

```
>>> from reviews.models import Book, Publisher
```

2. Retrieve the publisher object from the database using the following command. The `get()` method is used to retrieve an object from the database. We still haven't explored database read operations. For now, use the following command; we will go deeper into database read/retrieve in the next section:

```
>>> publisher = Publisher.objects.get(
    name='Packt Publishing')
```

3. When creating a book, we need to supply a date object, as `publication_date` is a date field in the Book model. So, import date from datetime so that a date object can be supplied when creating the book object, as shown in the following code:

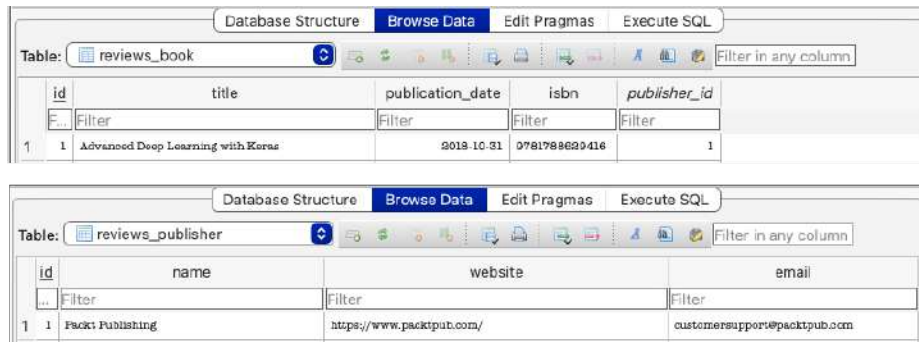
```
>>> from datetime import date
```

4. Use the `create()` method to create a record of the book in the database. Ensure that you pass all the fields, namely `title`, `publication_date`, `isbn`, and the `publisher` object:

```
>>> book = Book.objects.create(
    title="Advanced Deep Learning with Keras",
    publication_date=date(2018, 10, 31),
    isbn="9781788629416",
    publisher=publisher)
```

Note that since `publisher` is a foreign key and it is not nullable (cannot hold a null value), it is mandatory to pass a publisher object. When the mandatory foreign key object `publisher` is not provided, the database will throw an integrity error.

Figure 2.12 shows the Book table, where the first entry is created. Notice that the foreign key field (`publisher_id`) points to the `id` (primary key) of the Publisher table. The `publisher_id` entry in the book's record is pointing to a Publisher record that has `id` (primary key) 1, as shown in the following two screenshots:



The figure consists of two screenshots from a database browser interface. The top screenshot shows the 'reviews_book' table with columns: id, title, publication_date, isbn, and publisher_id. It contains one record with id 1, title 'Advanced Deep Learning with Keras', publication_date '2019-10-31', isbn '0781788620416', and publisher_id 1. The bottom screenshot shows the 'reviews_publisher' table with columns: id, name, website, and email. It contains one record with id 1, name 'Packt Publishing', website 'https://www.packtpub.com/', and email 'customersupport@packtpub.com'.

id	title	publication_date	isbn	publisher_id
1	Advanced Deep Learning with Keras	2019-10-31	0781788620416	1

id	name	website	email
1	Packt Publishing	https://www.packtpub.com/	customersupport@packtpub.com

Figure 2.12: The foreign key `publisher_id` of `reviews_book` points to the primary key of `reviews_publisher`

In this exercise, we learned that while creating a database record, an object can be assigned to a field if it is a foreign key. We know that the Book model also has a many-to-many relationship with the Contributor model. Let's now explore the ways to establish many-to-many relations as we create records in the database.

Exercise 2.04 – Creating records with many-to-many relationships

In this exercise, we will create a many-to-many relationship between Book and Contributor using the BookContributor relationship model:

1. In case you have restarted the shell and lost the publisher and book objects, retrieve them from the database by using the following set of Python statements:

```
>>> from reviews.models import Book
>>> from reviews.models import Contributor
>>> contributor = Contributor.objects.get(
    first_names='Rowel')
>>> book = Book.objects.get(
    title="Advanced Deep Learning with Keras")
```

2. The way to establish a many-to-many relationship is by storing the information about the relationship in the intermediary model or the relationship model; in this case, BookContributor. Since we have already fetched the book and the contributor records

from the database, let's use these objects while creating a record for the BookContributor relationship model. To do so, first, create an instance of the BookContributor relationship class and then save the object to the database. While doing so, ensure you pass the required fields, namely the book object, the contributor object, and role:

```
>>> from reviews.models import BookContributor
>>> book_contributor = BookContributor(book=book,
    contributor=contributor, role='AUTHOR')
>>> book_contributor.save()
```

Notice that we specified the role as AUTHOR while creating the book_contributor object. This is a classic example of storing relationship data while establishing a many-to-many relationship. The role can be AUTHOR, CO_AUTHOR, or EDITOR.

This established the relationship between the *Advanced Deep Learning with Keras* book and the contributor Rowel (Rowel being the author of the book).

In this exercise, we established a many-to-many relationship between Book and Contributor using the BookContributor relationship model. With regards to the verification of the many-to-many relationship that we just created, we will see this in detail in a few exercises later on in this chapter.

Exercise 2.05 – A many-to-many relationship using the add() method

In this exercise, we will establish a many-to-many relationship using the add() method. When we don't use the relationship to create objects, we can use through_default to pass in a dictionary with the parameters defining the required fields. Continuing from the previous exercise, let's add one more contributor to the book titled *Advanced Deep Learning with Keras*. This time, the contributor is an editor of the book:

1. If you have restarted the shell, run the following two commands to import and fetch the desired book instance:

```
>>> from reviews.models import Book, Contributor
>>> book = Book.objects.get(
    title="Advanced Deep Learning with Keras")
```

2. Use the `create()` method to create a contributor, as shown here:

```
>>> contributor = Contributor.objects.create(
    first_names='Packt',
    last_names='Example Editor',
    email='PacktEditor@example.com')
```

3. Add the newly created contributor to the book using the `add()` method. Ensure you provide the role relationship parameter as dict. Enter the following code:

```
>>> book.contributors.add(contributor,
    through_defaults={'role': 'EDITOR'})
```

Thus, we used the `add()` method to establish a many-to-many relationship between the book and the contributor while storing the relationship data role as Editor. Let's now take a look at other ways of doing this.

Using the `create()` and `set()` methods for many-to-many relationships

Assume the *Advanced Deep Learning with Keras* book has a total of two editors. Let's use the following method to add another editor to the book. If the contributor is not already present in the database, then we can use the `create()` method to simultaneously create an entry, as well as to establish the relation with the book:

```
>>> book.contributors.create(first_names='Packtp',
    last_names='Editor Example',
    email='PacktEditor2@example.com',
    through_defaults={'role': 'EDITOR'})
```

Similarly, we can also use the `set()` method to add a list of contributors for a book. Let's create a publisher, a set of two contributors who are the co-authors, and a book object. First, import the Publisher model, if not already imported, using the following code:

```
>>> from reviews.models import Publisher
```

The following code will help us do so:

```
>>> publisher = Publisher.objects.create(
    name='Pocket Books',
```

```
website='https://pocketbookssampleurl.com',
email='pocketbook@example.com')
>>> contributor1 = Contributor.objects.create(
    first_names='Stephen', last_names='Stephen',
    email='StephenKing@example.com')
>>> contributor2 = Contributor.objects.create(
    first_names='Peter', last_names='Straub',
    email='PeterStraub@example.com')
>>> book = Book.objects.create(title='The Talisman',
    publication_date=date(2012, 9, 25),
    isbn='9781451697216', publisher=publisher)
```

Since this is a many-to-many relationship, we can add a list of objects in just one go using the `set()` method. We can use `through_defaults` to specify the role of the contributors; in this case, they are co-authors:

```
>>> book.contributors.set([contributor1, contributor2],
    through_defaults={'role': 'CO_AUTHOR'})
```

Read operations

Django provides us with methods that allow us to read/retrieve from the database. We can retrieve a single object from the database using the `get()` method. We have already created a few records in the previous sections, so let's use the `get()` method to retrieve an object.

Exercise 2.06 – Using the `get()` method to retrieve an object

In this exercise, we will retrieve an object from the database using the `get()` method. Follow these steps:

1. Fetch a `Publisher` object that has a `name` field with the `Pocket Books` value:

```
>>> from reviews.models import Publisher
>>> publisher = Publisher.objects.get(
    name='Pocket Books')
```

2. Re-enter the retrieved publisher object and press *Enter*:

```
>>> publisher
<Publisher: Pocket Books>
```


Notice that the output is displayed in the shell. This is called a string representation of an object. It is the result of adding the `__str__()` model method as we did in the *Model methods* section for the `Publisher` class.

3. Upon retrieving the object, we have access to all the object's attributes. Since this is a Python object, the attributes of the object can be accessed by using dot notation (`.` followed by the attribute name). So, we can retrieve the publisher's name with the following command:

```
>>> publisher.name
'Pocket Books'
```

4. Similarly, to retrieve the publisher's website, use the following:

```
>>> publisher.website
'https://pocketbookssampleurl.com'
```

The publisher's email address can be retrieved as follows:

```
>>> publisher.email
'pocketbook@example.com'
```

In this exercise, we learned how to fetch a single object using the `get()` method. There are several disadvantages to using this method, though. Let's find out what they are next.

Returning an object using the `get()` method

It is important to note that the `get()` method can only fetch one object. If there is another object carrying the same value as the field mentioned, then we can expect a *returned more than one* error message. For example, if there are two entries in the `Publisher` table with the same value for the `name` field, we can expect an error. In such cases, there are alternate ways to retrieve those objects, which we will explore in the subsequent sections.

We can also get a *matching query does not exist* error message when there are no objects returned from the `get()` query. The `get()` method can be used with any of the object's fields to retrieve a record. In the following case, we are using the `website` field:

```
>>> publisher = Publisher.objects.get(
    website='https://pocketbookssampleurl.com')
```

After retrieving the object, we can still get the publisher's name, as shown here:

```
>>> publisher.name
'Pocket Books'
```

Another way to retrieve an object is by using its primary key, `pk`, as can be seen here:

```
>>> Publisher.objects.get(pk=2)
<Publisher: Pocket Books>
```

Using `pk` for the primary key is a more generic way of using the primary key field. But for the `Publisher` table, since we know that `id` is the primary key, we can simply use the `id` field name to create our `get()` query:

```
>>> Publisher.objects.get(id=2)
<Publisher: Pocket Books>
```

For `Publisher` and all the other tables, the primary key is `id`, which Django automatically created. This happens when a primary key field is not mentioned at the time of the creation of the table. But there can be instances where a field can be explicitly declared as a primary key.

In the next exercise, we will see how to query and retrieve all the objects for a given model.

Exercise 2.07 – Using the `all()` method to retrieve a set of objects

We can use the `all()` method to retrieve a set of all objects. In this exercise, we will use this method to retrieve the names of all contributors. To do that, follow these steps:

1. Add the following code to retrieve all the objects from the `Contributor` table:

```
>>> from reviews.models import Contributor

>>> Contributor.objects.all()
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>, <Contributor:
Packtp>, <Contributor: Stephen>, <Contributor: Peter>]>
```

Upon execution, we will get `QuerySet` of all the objects.

2. We can use list indexing to look up a specific object or to iterate over the list using a loop to do any other operation:

```
>>> contributors = Contributor.objects.all()
```

3. Since Contributor is a list of objects, we can use indexing to access any element in the list, as shown in the following command:

```
>>> contributors[0]
<Contributor: Rowel>
```

In this case, the first element in the list is a contributor with a `first_names` value of 'Rowel' and a `last_names` value of 'Atienza', as we can see from the following code:

```
>>> contributors[0].first_names
'Rowel'
>>> contributors[0].last_names
'Atienza'
```

In this exercise, we learned how to retrieve all the objects using the `all()` method and how to use the retrieved set of objects as a list.

Retrieving objects by filtering

If we have more than one object for a field value, we cannot use the `get()` method since the `get()` method can return only one object. For such cases, we have the `filter()` method, which can retrieve all the objects that match a specified condition.

Exercise 2.08 – Using the `filter()` method to retrieve objects

In this exercise, we will use the `filter()` method to get a specific set of objects for a certain condition. Specifically, we will retrieve all the contributors' names whose first name is Peter. To do that, follow these steps:

1. First, create two more contributors:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.create(first_names='Peter',
                               last_names='Wharton',
                               email='PeterWharton@example.com')
>>> Contributor.objects.create(first_names='Peter',
```

```
last_names='Tyrrell',  
email='PeterTyrrell@example.com')
```

2. To retrieve those contributors who have a `first_names` value of Peter, add the following code:

```
>>> Contributor.objects.filter(first_names='Peter')  
<QuerySet [<Contributor: Peter>, <Contributor: Peter>,  
<Contributor: Peter>]>
```

3. The `filter()` method returns the object even if there is only one. We can see this here:

```
>>> Contributor.objects.filter(first_names='Rowel')  
<QuerySet [<Contributor: Rowel>]>
```

4. Furthermore, the `filter()` method returns an empty `QuerySet` if none matches the query. This can be seen here:

```
>>> Contributor.objects.filter(first_names='Nobody')  
<QuerySet []>
```

In this exercise, we saw the use of filters to retrieve a set of a few objects filtered by a certain condition. In the next section, we will learn about filtering by using field lookups.

Filtering by field lookups

Now, let's suppose we want to filter and query a set of objects using the objects' fields by providing certain conditions. In a case, we can use a **double-underscore lookup**. For example, the `Book` object has a `publication_date` field; let's say we want to filter and fetch all the books that were published after 01-01-2014. We can easily look these up by using the double-underscore method. To do this, we will first import the `Book` model:

```
>>> from reviews.models import Book  
>>> book = Book.objects.filter(  
    publication_date__gt=date(2014, 1, 1))
```

Here, `publication_date__gt` indicates the publication date, which is greater than (`gt`) a certain specified date – in this case, 01-01-2014. Similar to this, we have the following abbreviations:

- `lt`: Less than

- lte: Less than or equal to
- gte: Greater than or equal to

The result after filtering can be seen here:

```
>>> book
<QuerySet [(<Book: Advanced Deep Learning with Keras>)]>
```

Here is the publication date of the book that is part of the query set, which confirms that the publication date was after 01-01-2014:

```
>>> book[0].publication_date
datetime.date(2018, 10, 31)
```

Using pattern matching for filtering operations

For filtered results, we can also look up whether the parameter contains a part of the string we are looking for:

```
>>> book = Book.objects.filter(
    title__contains='Deep learning')
```

Here, title__contains looks for all those objects with titles containing 'Deep learning' as a part of the string:

```
>>> book
<QuerySet [(<Book: Advanced Deep Learning with Keras>)]>

>>> book[0].title
'Advanced Deep Learning with Keras'
```

Similarly, we can use icontains if the string match needs to be case-insensitive. Using startswith matches any string starting with the specified string.

Retrieving objects by using the `exclude()` method

In the previous section, we learned about fetching a set of objects by matching a certain condition. Now, suppose we want to do the opposite; that is, we want to fetch all those objects that do not match a certain condition. In such cases, we can use the `exclude()` method to exclude a certain condition and fetch all the required objects. This will be clearer with an example. The following is a list of all contributors:

```
>>> Contributor.objects.all()
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>,
<Contributor: Packtp>, <Contributor: Stephen>,
<Contributor: Peter>, <Contributor: Peter>,
<Contributor: Peter>]>
```

Now, from this list, we will exclude all those contributors who have the `first_names` value of Peter:

```
>>> Contributor.objects.exclude(first_names='Peter')
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>,
<Contributor: Packtp>, <Contributor: Stephen>]>
```

We see here that the query returned all those contributors whose first name is not Peter.

Retrieving objects using the `order_by()` method

We can retrieve a list of objects while ordering by a specified field using the `order_by()` method. For example, in the following code snippet, we order the books by their publication date:

```
>>> books = Book.objects.order_by("publication_date")
>>> books
<QuerySet [<Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

Let's examine the order of the query. Since the query set is a list, we can use indexing to check the publication date of each book:

```
>>> books[0].publication_date
datetime.date(2012, 9, 25)
>>> books[1].publication_date
datetime.date(2018, 10, 31)
```

Notice that the publication date of the first book with a 0 index is older than the publication date of the second book with a 1 index. So, this confirms that the queried list of books has been properly ordered as per their publication dates. We can also use a prefix with the negative sign for the field parameter to order results in descending order. This can be seen from the following code snippet:

```
>>> books = Book.objects.order_by("-publication_date")
>>> books

<QuerySet [
<Book: Advanced Deep Learning with Keras>,
<Book: The Talisman>]>
```

Since we have prefixed a negative sign to the publication date, notice that the queried set of books has now been returned in the opposite order, where the first book object with a 0 index has a more recent date than the second book:

```
>>> books[0].publication_date
datetime.date(2018, 10, 31)

>>> books[1].publication_date
datetime.date(2012, 9, 25)
```

We can also order by using a string field or a numerical one. For example, the following code can be used to order books by their primary key or id:

```
>>> books = Book.objects.order_by('id')
>>> books

<QuerySet [
<Book: Advanced Deep Learning with Keras>,
<Book: The Talisman>]>
```

The queried set of books has been ordered as per book id in ascending order:

```
>>> books[0].id
1
>>> books[1].id
2
```

Again, to order in descending order, the negative sign can be used as a prefix, as follows:

```
>>> books = Book.objects.order_by('-id')
>>> books
<QuerySet [ <Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

Now, the queried set of books has been ordered as per book id in descending order:

```
>>> books[0].id
2
>>> books[1].id
1
```

To order by a string field in alphabetical order, we can do something like this:

```
>>> books = Book.objects.order_by('title')
>>> books
<QuerySet [ <Book: Advanced Deep Learning with Keras>, <Book: The Talisman>]>
```

Since we have used the title of the book to order by, the query set has been ordered in alphabetical order. We can see this as follows:

```
>>> books[0]
<Book: Advanced Deep Learning with Keras>
>>> books[1]
<Book: The Talisman>
```

Similar to what we've seen for the previous ordering types, the negative sign prefix can help us sort in reverse alphabetical order, as we can see here:

```
>>> books = Book.objects.order_by('-title')
>>> books
<QuerySet [ <Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

This will lead to the following output:

```
>>> books[0]
<Book: The Talisman>
>>> books[1]
<Book: Advanced Deep Learning with Keras>
```


Yet another useful method offered by Django is `values()`. It helps us get a query set of dictionaries instead of objects. In the following code snippet, we're using this for a `Publisher` object:

```
>>> publishers = Publisher.objects.all().values()

>>> publishers
<QuerySet [{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}, { 'id': 2, 'name':
'Pocket Books', 'website':
'https://pocketbookssampleurl.com',
'email': 'pocketbook@example.com'}]>
>>> publishers[0]
{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}
>>> publishers[0]
{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}
```

In the next few sections, we will explore how to query across relationships.

Querying across relationships

As we have studied in this chapter, the reviews app has two kinds of relationships: many-to-one and many-to-many. So far, we have learned various ways of making queries using `get()`, filters, field lookups, and so on. Now let's study how to perform queries across relationships. There are several ways to go about this – we could use foreign keys, object instances, and more. Let's explore these with the help of some examples.

Querying using foreign keys

When we have relationships across two models/tables, Django provides a way to perform a query using the relationships. The command shown in this section will retrieve all the books published by Packt Publishing by performing a query using model relationships. Similar to what we've seen previously, this is done using the double-underscore lookup. For example, the Book model has a foreign key of publisher pointing to the Publisher model. Using this foreign key, we can perform a query using double underscores and the name field in the Publisher model. This can be seen from the following code:

```
>>> Book.objects.filter(publisher__name='Packt Publishing')
<QuerySet [ <Book: Advanced Deep Learning with Keras> ]>
```

Querying using the model name

Another way of querying is using a relationship to do the query backward, using the model name in lowercase. For instance, let's say we want to query the publisher who published the *Advanced Deep Learning with Keras* book using model relationships in the query. For this, we can execute the following statement to retrieve the Publisher information object:

```
>>> Publisher.objects.get(
    book__title='Advanced Deep Learning with Keras')
<Publisher: Packt Publishing>
```

Here, book is the model's name in lowercase. As we already know, the Book model has a publisher foreign key with a name value of Packt Publishing.

Querying across foreign key relationships using the object instance

We can also retrieve the information using the object's foreign key. Suppose we want to query the publisher's name for the title *The Talisman*:

```
>>> book = Book.objects.get(title='The Talisman')
>>> book.publisher
<Publisher: Pocket Books>
```

Using the object here is an example of where we use the reverse direction to get all the books published by a publisher by using the `set.all()` method:

```
>>> publisher = Publisher.objects.get(name='Pocket Books')
>>> publisher.book_set.all()
<QuerySet [<Book: The Talisman>]>
```

We can also create queries using chains of queries:

```
>>> Book.objects.filter(
    publisher__name='Pocket Books').filter(
    title='The Talisman')
<QuerySet [<Book: The Talisman>]>
```

Let's perform some more exercises to shore up our knowledge of the various kinds of queries we have learned about so far.

Exercise 2.09 – Querying across a many-to-many relationship using the field lookup

We know that `Book` and `Contributor` have a many-to-many relationship. In this exercise, without creating an object, we will perform a query to retrieve all the contributors who contributed to writing the book titled *The Talisman*:

1. First, import the `Contributor` class:

```
>>> from reviews.models import Contributor
```

2. Now, add the following code to query for the set of contributors on *The Talisman*:

```
>>> Contributor.objects.filter(
    book__title='The Talisman')
```

We should see the following:

```
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

From the preceding output, we can see that Stephen and Peter are the contributors who contributed to writing the book *The Talisman*. The query uses the book model (written in lowercase) and does a field lookup for the title field using the double underscore, as shown in the command.

In this exercise, we learned how to perform queries across many-to-many relationships using a field lookup. Let's now look at using another method to carry out the same task.

Exercise 2.10 – A many-to-many query using objects

In this exercise, using a Book object, search for all the contributors who contributed to writing the book with the title *The Talisman*. The following steps will help us do that:

1. Import the Book model:

```
>>> from reviews.models import Book
```

2. Retrieve a book object with the title *The Talisman*, by adding the following line of code:

```
>>> book = Book.objects.get(title='The Talisman')
```

3. Then retrieve all the contributors who worked on the book *The Talisman* using the book object. Add the following code to do so:

```
>>> book.contributors.all()  
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

Again, we can see that Stephen and Peter are the contributors who worked on the book *The Talisman*. Since the book has a many-to-many relationship with contributors, we have used the `contributors.all()` method to get a query set of all those contributors who worked on the book. Now, let's try using the set method to perform a similar task.

Exercise 2.11 – A many-to-many query using the set() method

In this exercise, we will use a contributor object to fetch all the books written by the contributor named Rowel:

1. Import the Contributor model:

```
>>> from reviews.models import Contributor
```

2. Fetch a contributor object whose `first_names` is 'Rowel' using the `get()` method:

```
>>> contributor = Contributor.objects.get(
    first_names='Rowel')
```

3. Using the contributor object and the `book_set()` method, get all those books written by the contributor:

```
>>> contributor.book_set.all()
<QuerySet [Book: Advanced Deep Learning with Keras]>
```

Since `Book` and `Contributor` have a many-to-many relationship, we can use the `set()` method to query a set of objects associated with the model. In this case, `contributor.book_set.all()` returned all the books written by the contributor.

Exercise 2.12 – Using the `update()` method

In this exercise, we will use the `update()` method to update an existing record:

1. Change `first_names` for a contributor who has the last name `Tyrrell`:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.filter(
    last_names='Tyrrell').update(
    first_names='Mike')
```

1

The return value shows the number of records that have been updated. In this case, one record has been updated.

2. Fetch the contributor that was just modified using the `get()` method and verify that the first name has been changed to Mike:

```
>>> Contributor.objects.get(
    last_names='Tyrrell').first_names
'Mike'
```

Note

If the filter operation has more than one record, then the `update()` method will update the specified field in all the records returned by the filter.

In this exercise, we learned how to use the `update()` method to update a record in the database. Now, finally, let's try deleting a record from the database using the `delete()` method.

Exercise 2.13 – Using the `delete()` method

An existing record in the database can be deleted using the `delete()` method. In this exercise, we will delete a record from the `contributors` table that has a `last_name` value of `Wharton`:

1. Fetch the object using the `get` method and use the `delete` method, as shown here:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.get(
    last_names='Wharton').delete()
(1, {'reviews.Contributor': 1})
```

Notice that we called the `delete()` method without assigning the contributor object to a variable. Since the `get()` method returns a single object, we can access the object's method without actually creating a variable for it.

2. Verify that the contributor object with `last_name` as `'Wharton'` has been deleted:

```
>>> Contributor.objects.get(last_names='Wharton')
Traceback (most recent call last):
File "<console>", line 1, in <module>
File "../site-packages/django/db/models/manager.py",
line 82, in manager_method
return getattr(self.get_queryset(), name)(*args,
**kwargs)
File "../site-packages/django/db/models/query.py",
line 417, in get
self.model._meta.object_name
reviews.models.Contributor.DoesNotExist: Contributor
matching query does not exist.
```

As you can see, upon running the query, we got an *object does not exist* error. This is expected since the record has been deleted. In this exercise, we learned how to use the `delete` method to delete a record from the database.

Bulk create and bulk update operations

When we have a large set of records that needs to be created or updated, we can perform bulk create and bulk update operations using the `bulk_create()` and `bulk_update()` methods, respectively. When we have a large number of records to be created or updated, using methods such as these can be efficient while performing database operations.

Typically, here is how a `bulk_create` method is called by supplying a list of objects:

```
Person.objects.bulk_create([
    Person(name='Robert',
          address='5 Falcon Street, Byron Bay, NSW 2481'),
    Person(name='Mark',
          address="31 West Street, Orange, NSW 2800"),])
```

Likewise, the bulk update is also performed on a list of similar objects. The object's attributes can be changed as shown in the following example and updated in the database in a single command using the `bulk_update()` method, as shown in the following snippet:

```
persons[0].address = "2 Church Street, Byron Bay, NSW 2481"
persons[1].address = "35 Steel Street, Orange, NSW 2800"
Person.objects.bulk_update(persons, ["address"])
```

We will see further examples of both of these in the following exercises.

Exercise 2.14 – Create multiple records using `bulk_create`

In this exercise, we will use the `bulk_create()` method to create multiple entries in the `Publisher` table in one go. To do that, follow these steps:

1. Import the `Publisher` model if you have not already imported it into your Django shell:

```
>>> from reviews.models import Publisher
```

2. Create multiple records in the `Publisher` table by passing a list of the `Publisher` objects into the `bulk_create()` method:

```
>>> Publisher.objects.bulk_create([
    Publisher(name="New Town Publisher",
              website="www.newtown.example.com",
              email='newtown@example.com'),
```

```
Publisher(name="Byron Bay Press",
website="www.byronbay.example.org",
email='byronbay@example.org'),
Publisher(name="Katoomba Publisher",
website="www.katoomba.example.net",
email='katoomba@example.net'))

[<Publisher: New Town Publisher>, <Publisher: Byron Bay Press>, <Publisher:
Katoomba Publisher>]
```

The method returns a list of objects created as entries in the database. If a large number of entries need to be created, this might be one of the most efficient ways of carrying out the task.

Next, we shall try to bulk update records efficiently using the `bulk_update` method.

Exercise 2.15 – Update multiple records using `bulk_update`

Similar to the previous exercise, we can update multiple records in one go by using the `bulk_update` method:

1. First, import the `Publisher` model if you have not already imported it:

```
>>> from reviews.models import Publisher
```

2. In the next step, we will pick a couple of `Publisher` objects as a list.
3. Assuming both of these publishers got merged into a single company, and they need to share the same website; now let us update both the objects in a query set:

```
>>> publishers = [
    Publisher.objects.get(name='New Town Publisher'),
    Publisher.objects.get(name='Byron Bay Press')]
>>> publishers[0].website = "nswpublisher.example.com"
>>> publishers[1].website = "nswpublisher.example.com"

>>> Publisher.objects.bulk_update(publishers,
    ["website"])
2
```

Upon running the `bulk_update` method, it returns the number of objects updated; in this case, it is two objects. Again, this is an efficient way to update more than one object in one go.

Performing complex lookups using Q objects

Q objects are used to perform complex queries, especially when a query involves the AND or OR operations in a WHERE clause. For instance, if we need to do a query similar to the SQL query shown here:

```
SELECT * FROM Person
WHERE name LIKE 'Rob%' OR name LIKE 'Bob%';
```

The preceding SQL statement queries for any person whose name either starts with Rob or Bob.

LIKE 'Rob%' – Here, the LIKE keyword is pattern-matching a string to check whether the string starts with the specified Rob value.

Q objects use the & and | operators for the AND and OR operations while combining the WHERE clauses. The preceding query can be written as follows using Q objects:

```
Person.objects.get(Q(name__startswith='Rob'), | Q(name__startswith='Bob'))
```

Exercise 2.16 – Perform a complex query using a Q object

Next, using the concept learned before, Q objects let us perform a query for two of the publishers:

1. Import Publisher if you have not already done so before, and also import Q from `django.db.models`, as shown in the following command:

```
>>> from django.db.models import Q
>>> from reviews.models import Publisher
```

2. Execute the Q object query as shown next. Here, we check whether any of the Publisher objects has name starting with either New or Idea:

```
>>> Publisher.objects.filter(Q(name__startswith="New")
    | Q(name__startswith="Idea"))
<QuerySet [Publisher: New Town Publisher]>>
```

Since there is no publisher with name starting with Idea and there was an object with name starting with New, one object was returned.

This is an example Q object query with the AND operator. In this query, we are querying a Publisher object whose name starts with New and also whose name ends with Publisher:

```
>>> Publisher.objects.filter(Q(name__startswith="New") &
                             Q(name__endswith="Publisher"))
<QuerySet [<Publisher: New Town Publisher>]>
```

In this case, only one object was returned, which is New Town Publisher, which satisfies the conditions stated in the query. This way, multiple such where clauses can be combined using the AND or OR operators to get the desired results.

Exercise 2.17 – Verify whether a queryset contains a given object

In this exercise, we will use a contains method to check whether the query contains a specified object:

1. Import the Publisher model if you have not already imported it:

```
>>> from reviews.models import Publisher
```

2. Run a query, as shown next. In this example, we are executing a Q object query, and the result has two publishers:

```
>>> publishers = Publisher.objects.filter(
    Q(name__startswith="New")
    | Q(name__endswith="Publisher"))
>>> publishers
<QuerySet [<Publisher: New Town Publisher>, <Publisher: Katoomba
Publisher>]>
```

3. Fetch a new publisher object and verify whether it is a part of the previous query set:

```
>>> new_town_publisher = Publisher.objects.get(
    name='New Town Publisher')
>>> new_town_publisher
<Publisher: New Town Publisher>
>>> publishers.contains(new_town_publisher)
True
```

If the object is present, it returns a Boolean value such as True or False.

Here is an example when a certain query set does not have the specified object, in which case the result returned is `False`:

```
>>> publishers.contains(Publisher.objects.get(
    name='Byron Bay Press'))
False
```

In this exercise, we learned how a simple method, `contains()`, can be used to check whether an object is part of a query set.

Overall, in this entire section, we learned about and used Django's command-line interactive shell. Using the models created in the previous section for the book review application, we used the Django interactive shell to perform CRUD operations. We also explored various ways to filter and query (Read) from the database. All of these database operations will come in handy while developing any Django application. Using the concepts learned so far in this chapter, in the next section, we will create models for a sample application and perform some of the CRUD operations.

Activity 2.01 – Create models for a project management application

Imagine we are developing a project management application called Juggler. Juggler is an application that can track multiple projects, and each project can have multiple tasks associated with it. The following steps will help us complete this activity:

1. Using the techniques we have learned so far, create a Django project called `juggler`.
2. Create a Django app called `projectp`.
3. Add the app projects in the `juggler/settings.py` file.
4. Create two related model classes called `Project` and `Task` in `projectp/models.py`.
5. Create migration scripts and migrate the models' definitions to the database.
6. Open the Django shell and import the models.
7. Populate the database with an example and write a query displaying the list of tasks associated with a given project.

Note

The full solution for this activity can be found at in the `Chapter02` folder in this book's GitHub repo.

Populating the Bookr project's database

Although we know how to create individual database records, it will be good to populate the Bookr database with a dataset that exemplifies real-world publications. We have included a JSON file that can be used to load publication data and reviews into the database using the `loaddata` subcommand of the `manage.py` script.

Note

You will find the `reviews.json` file in the `Chapter02` folder in this book's GitHub repo.

Follow the next few steps to populate the project's database:

1. Copy the `reviews.json` file into the `reviews` app folder. This file can be found in the GitHub repository for this book.
2. Now, let's recreate a fresh database. Delete your SQL database file present in the project folder:

```
rm db.sqlite3
```

3. To create a fresh database with the correct schema, execute the Django migrate command:

```
python manage.py migrate
```

Now we can see the newly created `db.sqlite3` file under the `bookr` folder.

4. Execute the `loaddata` management command to populate the database:

```
python manage.py loaddata reviews/reviews.json
```

5. Using DB Browser for SQLite, verify that all the tables created by the `bookr` project are populated.

With this, we have successfully populated our application's database, which will come in handy to work with in the upcoming chapters of this book.

Summary

In this chapter, we learned how Django provides a valuable abstraction layer called the ORM that helps us interact seamlessly with relational databases using simple Python code without having to compose SQL commands. As a part of the ORM, we learned about Django models, migrations, and how they help propagate changes to Django models in a database.

We shored up our knowledge of databases by learning about database relationships and their key types in relational databases. We also worked with the Django shell, using Python code to perform the same CRUD queries we performed earlier using SQL. Later, we learned how to retrieve our data in a more refined manner using pattern matching and field lookups. As we learned about these concepts, we also made considerable progress on our Bookr application. We created models for our reviews app and gained all the skills we needed to interact with the data stored inside the app's database. In the next chapter, we will learn how to create Django views, URL routing, and templates.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

3

Django Views, URL Configuration, and Templates

In *Chapter 2*, we were introduced to databases, and we learned how to store, retrieve, update, and delete records from a database. We also learned how to create Django models and apply database migrations.

However, these database operations alone cannot display the application's data to a user. We need a way to display all the stored information in a meaningful way to the user – for example, displaying all the books present in our Bookr application's database, in a browser and in a presentable format. This is where Django views, URL configuration, and templates come into play. Views are one of the most important parts of a Django application, where the application logic is written. This application logic controls interactions with the database, such as creating, reading, updating, or deleting records from the database. Server requests are routed to views using URL mapping. Views also control how the data can be displayed to the user. This is done with the help of Django HTML templates.

You will start by exploring the two main types of views in Django: function-based views and class-based views. Next, you will learn the basics of Django template language and template inheritance. Using these concepts, you will create a page to display the list of all the books in the Bookr application. You will also create another page to display the details, review comments, and ratings of books.

In this chapter, we will be studying the following topics:

- Working with Django views
- URL configuration
- Working with Django templates

By using and learning about these topics, by the end of this chapter, you will have implemented a way to list all the books and display details of any given book in the **Bookr** application.

Technical requirements

Find the solution in the `Chapter03` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Working with Django views

Views are the part of a Django application that takes in a web request, processes it and provides a web response. For example, a user clicks on a **Product details** link:

The browser sends a request to a specific URL (e.g. `/products/12/`), which Django maps to a view.

The view goes to the database to fetch product 12 and prepares data from that object to be used in the view's template. This data is referred to as context data and is used alongside the template to create HTML for the response.

The response takes the rendered template to display the HTML in the user's browser. Django views can be broadly classified into two types: **function-based views** and **class-based views**. Let's take a look at each in detail.

Note

In this chapter, we will learn more about function-based views in Django. Class-based views, which are a more advanced topic, will be discussed in detail in *Chapter 11*. Asynchronous views will be discussed in *Chapter 17*.

Understanding function-based views

As the name implies, function-based views are implemented as *Python functions*. To understand how they work, consider the following snippet, which shows a simple view function named `home_page`:

```
from django.http import HttpResponseRedirect

def home_page(request):
    message = "<html><h1>Welcome to my Website</h1></html>"
    return HttpResponseRedirect(message)
```

The view function defined here, named `home_page`, takes a request object as an argument and returns an `HttpResponse` object, having the `Welcome to my Website` message. The advantage of using function-based views is that, since they are implemented as simple Python functions, they are easier to learn and also easily readable for other programmers. The major disadvantage of function-based views is that the code cannot be reused and made as concisely as class-based views for generic use cases. The next section is a brief introduction to class-based views.

Understanding class-based views

As the name implies, class-based views are implemented as *Python classes*. Using the principles of class inheritance, these classes are implemented as subclasses of Django's generic view classes. Unlike function-based views, where all the view logic is expressed explicitly in a function, Django's generic view classes come with various pre-built properties and methods that can provide shortcuts to writing clean, reusable views. This property comes in handy quite often during web development; for example, developers often need to render an HTML page without needing any data inserted from the database or any customization specific to the user. In this case, it is possible to simply inherit from Django's `TemplateView` and specify the path of the HTML file. The following is an example of a class-based view that can display the same message as in the function-based view example:

```
from django.views.generic import TemplateView

class HomePage(TemplateView):
    template_name = 'home_page.html'
```

In the preceding code snippet, `HomePage` is a class-based view inheriting Django's `TemplateView` from the `django.views.generic` module. The `template_name` class attribute defines the template to render when the view is invoked. For the template, we add an HTML file to our templates folder with the following content:

```
<html><h1>Welcome to my Website</h1></html>
```

This is a very basic example of class-based views, which will be explored further in *Chapter 11*. The major advantage of using class-based views is that fewer lines of code need to be used to implement the same functionality as compared to function-based views. Also, by inheriting Django's generic views, we can keep the code concise and avoid the duplication of code. However, a disadvantage of class-based views is that the code is often less readable for someone new to Django, which means that learning about these is usually a longer process compared to function-based views.

In the next section, we will learn about URL configuration.

URL configuration

Django views cannot work on their own in a web application. When a web request is made to the application, Django's **URL configuration** takes care of routing the request to the appropriate view function to process the request. A typical URL configuration in the `urls.py` file in Django looks like this:

```
from . import views

urlpatterns = [
    path('url-path/' views.my_view, name='my-view'),
]
```

Here, `urlpatterns` is the variable defining the list of URL paths, and `'url-path/'` defines the path to match.

`views.my_view` is the view function to invoke when there is a URL match, and `name='my-view'` is the name of the view function used to refer to the view. There may be a situation wherein, elsewhere in the application, we want to get the URL of this view. We wouldn't want to hardcode the value, as it would then have to be specified twice in the code base. Instead, we can access the URL by using the name of the view, as follows:

```
from django.urls import reverse

url = reverse('my-view')
```

If needed, we can also use a regular expression in a URL path to match string patterns using `re_path()`:

```
urlpatterns = [
    re_path(r'^url-path/(?P<name>pattern)/$',
           views.my_view, name='my-view')
]
```

Here, `name` refers to the pattern name, which can be any Python regular expression pattern, and this needs to be matched before calling the defined view function. You can also pass parameters from the URL into the view itself, as shown in this example:

```
urlpatterns = [  
    path('url-path/<int:id>/', views.my_view,  
        name='my-view')  
]
```

In the preceding example, `<int:id>` tells Django to look for URLs that contain an integer at this position in the string, and to assign the value of that integer to the `id` argument. This means that if the user navigates to `/url-path/14/`, the `id=14` keyword argument is passed to the view. This is often useful when a view needs to look up a specific object in the database and return corresponding data. For example, suppose we had a `User` model, and we wanted the view to display the user's name. The view could be written as follows:

```
def my_view(request, id):  
    user = User.objects.get(id=id)  
    return HttpResponse(f"This user's name is { user.first_name }  
    { user.last_name }")
```

When the user accesses `/url-path/14/`, `my_view` is called, and the `id=14` argument is passed into the function.

Here is the typical workflow when a URL such as `http://0.0.0.0:8000/url-path/` is invoked using a web browser:

1. An HTTP request would be made to the running application for the URL path. Upon receiving the request, the server reaches for the `ROOT_URLCONF` setting present in the `settings.py` file:

```
ROOT_URLCONF = 'project_name.urls'
```

This determines the URL configuration file that will be used first. In this case, it is the URL file present in the project directory, `project_name/urls.py`.

2. Next, Django goes through the list named `urlpatterns`, and once it matches `url-path/` with the path present in the URL, `http://0.0.0.0:8000/url-path/`, it invokes the corresponding view function.

URL configuration is sometimes also referred to as **URL conf** or **URL mapping**, and these terms are often used interchangeably. Overall, we can say that URL configuration is present to route the URL request posted on a browser to an appropriate view function. To understand views and URL configuration better, let's start with a simple exercise.

Exercise 3.01 – Implementing a simple function-based view

In this exercise, we will write a very basic function-based view and use the associated URL configuration to display the `Welcome to Bookr!` message in a web browser. We will also tell the user how many books we have in the database. Let's get started with the steps:

1. First, ensure that `ROOT_URLCONF` in `bookr/settings.py` is pointing to the project's URL file by adding the following command:

```
ROOT_URLCONF = 'bookr.urls'
```

2. Open the `bookr/reviews/views.py` file and add the following code snippet:

```
from django.http import HttpResponse
from .models import Book

def home(request):
    message = f"""<html><h1>Welcome to Bookr!</h1>
    <p>{Book.objects.count()} books and counting!</p></html>"""
    return HttpResponse(message)
```

In the preceding snippet, we first import the `HttpResponse` class from the `django.http` module. Next, we define the `home` function, which can display the `Welcome to Bookr!` message in a web browser. The `request` object is a function parameter that carries the HTTP request object. The next line defines the `message` variable, which contains HTML that displays the header, followed by a line that counts the number of books available in the database.

In the last line, we return an `HttpResponse` object with the string associated with the `message` variable. When the `home` view function is called, it will display the message `Welcome to Bookr! 2 Books and counting!` in the web browser.

3. Now, create the URL configuration to call the newly created view function. Open the project URL file, `bookr/urls.py`, and add the list of `urlpatterns` as follows:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('reviews.urls'))
]
```

The first line in the list of `urlpatterns`, that is, `path('admin/', admin.site.urls)` routes to the admin URLs if `admin/` is present in the URL path (for example, `http://0.0.0.0:8000/admin`).

Similarly, consider the second line, `path('', include('reviews.urls'))`. Here, the path mentioned is an empty string, `''`. If the URL does not have any specific path after `http://hostname:port-number/` (for example, `http://0.0.0.0:8000/`), it includes `urlpatterns` present in `review.urls`.

The `include` function is a shortcut that allows you to combine URL configurations. It is common to keep one URL configuration per application in your Django project. Here, we've created a separate URL configuration for the reviews app and have added it to our project-level URL configuration.

4. Since we do not have the `reviews.urls` URL module yet, create a file called `bookr/reviews/urls.py`, and add the following lines of code:

```
from django.contrib import admin
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Here, we have used an empty string again for the URL path. So, when `http://0.0.0.0:8000/` is invoked, after getting routed from `bookr/urls.py` into `bookr/reviews/urls.py`, this pattern invokes the home view function.

5. After making changes to the two files, we have the necessary URL configuration ready to call the home view. Now, start the Django server with `./manage.py runserver` and type `http://0.0.0.0:8000` or `http://127.0.0.1:8000` in your web browser. You should be able to see the **Welcome to Bookr!** message:

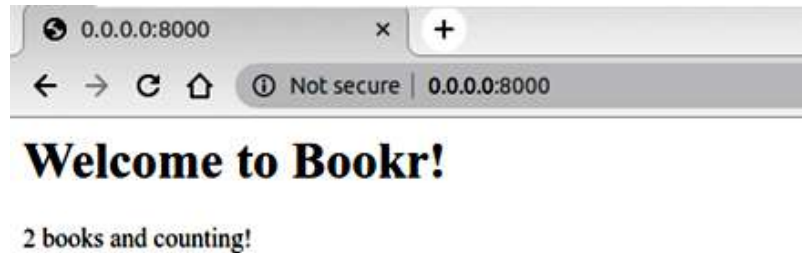


Figure 3.1 – Displaying "Welcome to Bookr!" and the number of books on the home page

Note

If there is no URL match, Django invokes error handling, such as displaying a **404 Page not found** message or something similar.

In development, with the `DEBUG` setting enabled, Django's error page will provide information to help understand the error and help fix it. In the example of a 404, it will show the URL patterns that it has tried to match against to find a route to a view for the request.

In this exercise, we learned how to write a basic view function and do the associated URL configuration. We have created a web page that displays a simple message to the user and reports how many books are currently in our database.

However, the astute reader will have noticed that it doesn't look very nice to have HTML code sitting inside our Python function as in the preceding example. As our views get bigger, this will become even more unsustainable. Therefore, in the next section, we will turn our attention to where our HTML code is supposed to be – inside templates.

Working with Django templates

In *Exercise 3.01, Implementing a simple function-based view*, we saw how to create a view, configure the URL, and display a message in the browser. But if you recall, we hardcoded the Welcome to Bookr! HTML message in the view function itself and returned an `HttpResponse` object, as follows:

```
message = f"""<html><h1>Welcome to Bookr!</h1>
<p>{Book.objects.count()} books and counting!</p></html>"""
return HttpResponse(message)
```

Hardcoding of HTML inside Python modules is not a good practice, because as the content to be rendered on a web page increases, so does the amount of HTML code we need to write for it. Having a lot of HTML code among Python code can make the code hard to read and maintain in the long run.

For this reason, Django **templates** provide us with a better way to write and manage HTML templates. Django's templates not only work with static HTML content but also with dynamic HTML templates.

Django's template configuration is done in the `TEMPLATES` variable present in the `settings.py` file. This is how the default configuration looks:

```
TEMPLATES = [
    {
        'BACKEND':
            'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

Let's go through each keyword present in the preceding snippet:

- `'BACKEND': 'django.template.backends.django.DjangoTemplates'`: This refers to the template engine to be used. A template engine is an API used by Django to work with HTML templates. Django is built with Jinja2 and the DjangoTemplates engine. The default configuration is the DjangoTemplates engine and Django template language. However, this can be changed to use a different one if required, such as Jinja2 or any other third-party template engine. For our Bookr application, though, we will leave this configuration as it is.
- `'DIRS': []`: This refers to the list of directories where Django searches for the templates in the given order.
- `'APP_DIRS': True`: This tells the Django template engine whether it should look for templates in the installed apps defined under `INSTALLED_APPS` in the `settings.py` file. The default option for this is `True`.
- `'OPTIONS'`: This is a dictionary containing template-engine-specific settings. Inside this dictionary, there is a default list of context processors, which helps the Python code to interact with templates to create and render dynamic HTML templates.

The current default settings are mostly fine for our purposes. However, in the next exercise, we will create a new directory for our templates, and we will need to specify the location of this folder. For example, if we have a directory called `my_templates`, we need to specify its location by adding it to the `TEMPLATES` settings as follows:

```
TEMPLATES = [  
    {  
        'BACKEND':  
            'django.template.backends.django.DjangoTemplates',  
        'DIRS': [os.path.join(BASE_DIR, 'my_templates')],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

Note

When defining the directories to collect templates from here, the Python `os` module is used, so remember to use `import os` in your settings file.

`BASE_DIR` is the directory path to the project folder. This is defined in the `settings.py` file. The `os.path.join()` method joins the project directory with the templates directory, returning the full path for the templates directory. In the following exercise, we shall use a template to display a message to the user.

Exercise 3.02 – Using templates to display a greeting message

In this exercise, we will create our first Django template, and just as we did in the previous exercise, we will display the `Welcome to Bookr!` message using the templates. Let's get started with the steps:

1. Create a directory called `templates` in the `bookr` project directory, and inside it, create a file called `base.html`. If you already have a `templates` directory, consider removing it or adapting `base.html` if you have one. The directory structure should look like the following figure:

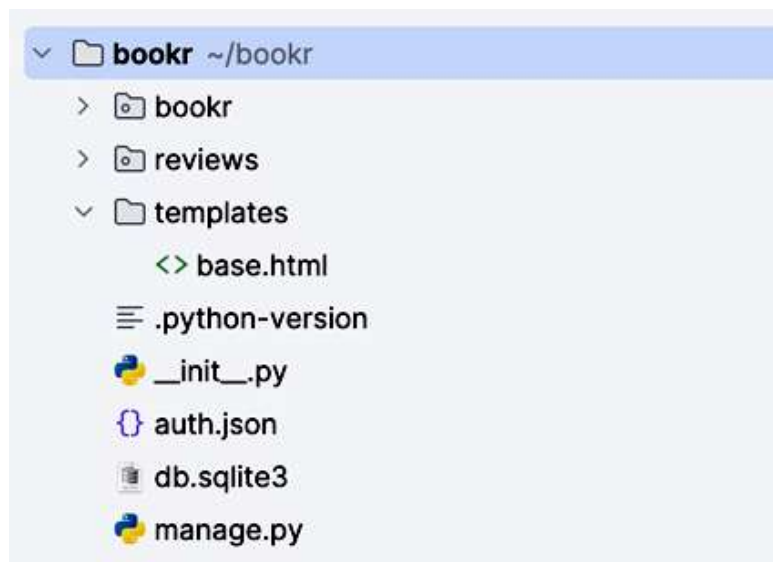


Figure 3.2 – Directory structure for `bookr`

Note

When the default configuration is used (that is, when `DIRS` is an empty list), Django searches for templates present only in the app folders' `template` directory (the `reviews/templates` folder in the case of a book review application). Since we included the new template directory in the main project directory, Django's template engine would not be able to find the directory unless the directory is included in the `'DIRS'` list.

2. Add the folder to the `TEMPLATES` settings:

```
TEMPLATES = [  
    {  
        'BACKEND':  
            'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

3. Add the following lines of code to the `base.html` file:

```
<!doctype html>  
<html lang="en">  
<head>  
    <meta charset="utf-8">  
    <title>Home Page</title>  
</head>  
    <body>  
        <h1>Welcome to Bookr!</h1>
```

```
</body>  
</html>
```

This is simple HTML that displays the `Welcome to Bookr!` message in the header.

4. Modify the code inside `bookr/reviews/views.py` so that it looks as follows:

```
from django.shortcuts import render  
  
def home(request):  
    return render(request, 'base.html')
```

Since we have already configured the `'templates'` directory in the `TEMPLATES` configuration, `base.html` is available for use with the template engine. The code renders the `base.html` file using the imported `render` method from the `django.shortcuts` module.

5. Save the files, run `./manage.py runserver`, and open the `http://0.0.0.0:8000/` or `http://127.0.0.1:8000/` URL to check the newly added template loading in the browser:



Figure 3.3 – Displaying "Welcome to Bookr!" on the home page

In this exercise, we created an HTML template and used Django templates and views to return the `Welcome to Bookr!` message.

Overall, we can say that Django templates help us to work with HTML templates and present information to the user in a desired format.

Next, we will learn about the Django template language, which can be used to render the application's data along with HTML templates.

Django template language

Django views not only return static HTML templates but can also add dynamic application data while generating the templates. Along with data, we can also include some programmatic elements in the templates. All of these put together form the basics of **Django's template language**. The following few sections look at some of the basic parts of the Django template language, such as template variables, template tags, comments, and filters.

Template variables

A **template variable** is represented between two curly braces, as shown here:

```
{{ variable }}
```

When this is present in the template, the value carried by the variables will be replaced in the template. Template variables help in adding the application's data into the templates. They can be added by defining context data for a view, modifying the code inside `bookr/reviews/views.py` so that it looks as follows:

```
from django.shortcuts import render

def home(request):
    context_data = {
        "template_variable": "I am a template variable."
    }
    return render(request, 'base.html', context=context_data)
```

Then, in `base.html`, the variable would be included as follows:

```
<body>
    {{ template_variable }}
</body>
```

Template tags

A **tag** is similar to a programmatic control flow, such as an `if` condition or a `for` loop. A tag is represented between two curly braces and percentage signs, as shown. Here is an example of a `for` loop iterating over a list using template tags:

```
{% for element in element_list %}

{% endfor %}
```

Unlike Python programming, we also add the end of the control flow by adding the end tag, such as `{% endfor %}`. This can be used along with template variables to display the elements in the list, as shown here:

```
<ul>
  {% for element in element_list %}
    <li>{{ element.title }}</li>
  {% endfor %}
</ul>
```

Comments

Comments in the Django template language can be written as shown here; anything in between `{% comment %}` and `{% endcomment %}` will be commented out:

```
{% comment %}
  <p>This text has been commented out</p>
{% endcomment %}
```

Filters

Filters can be used to modify a variable to represent it in a different format. The syntax for a filter is a variable separated from the filter name using a pipe (|) symbol:

```
{{ variable|filter }}
```

Here are some examples of built-in filters:

- `{{ variable|lower }}`: This converts the variable string into lowercase.
- `{{ variable|title }}`: This converts the first letter of every word into uppercase.

Let's use the concepts we have learned up till now to develop the book review application.

Exercise 3.03 – Displaying a list of books and reviews

In this exercise, we will create a web page that can display a list of all books, their ratings, and the number of reviews present on the book review application. For this, we will be using some features of the Django template language, such as template variables and tags, to pass the book review application data into the templates to display meaningful data on the web page:

1. Create a file called `utils.py` under `bookr/reviews/` and add the following code:

```
def average_rating(rating_list):
    if not rating_list:
        return 0

    return round(sum(rating_list) / len(rating_list))
```

`average_rating` is a helper method that will be used to calculate the average rating of a book.

2. Include these `import` statements at the start of `bookr/reviews/views.py`:

```
from django.shortcuts import render

from .models import Book
from .utils import average_rating
```

3. This is to import the Django `render` function, the `Book` model class, and the helper method we just added.
4. Add this function to `bookr/reviews/views.py`:

```
def book_list(request):
    books = Book.objects.all()
    book_list = []
    for book in books:
        reviews = book.review_set.all()
        if reviews:
            book_rating = average_rating([
                review.rating for review in reviews])
            number_of_reviews = len(reviews)
        else:
            book_rating = None
```

```
        number_of_reviews = 0
    book_list.append({
        'book': book,
        'book_rating': book_rating,
        'number_of_reviews': number_of_reviews})
    context = {
        'book_list': book_list
    }
    return render(request, 'reviews/book_list.html', context)
```

5. This is a view to display the list of books for the Bookr application.

Note

This example includes a reviews folder in the template name for the render function. This folder wasn't created in the initial configuration of the templates directory with the base.html template, so the code will likely throw a `TemplateDoesNotExist` error. It is good practice to create a folder with the same name as the application for its templates. These application-specific templates can inherit from the same base template, which covers the basic setup for the whole project, such as style sheets and JavaScript libraries common between all templates.

6. Django will read all template directories and collect the files together, so without a folder structure, there is no differentiation between two templates of the same name. Here, `book_list` is the view function. In this function, we start by querying the list of all books. Next, for every book, we calculate the average rating and the number of reviews posted. All this information for each book is appended to a list called `book_list` as a list of dictionaries. This list is then added to a dictionary named `context` and is passed to the render function.

The render function takes three parameters: the first one is the request object that was passed into the view, the second is the `book_list.html` HTML template, which will display the list of books, and the third is `context`, which we pass to the template.

Since we have passed `book_list` as a part of the context, the template will use this to render the list of books using template tags and template variables.

7. Create the `book_list.html` file in the `bookr/reviews/templates/reviews/` `book_list.html` path and add the following HTML code to the file:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Bookr</title>
</head>
<body>
  <h1>Book Review application</h1>
  <hr>
```

You can find the complete code in the `Chapter03` folder in this book's GitHub repo.

This is a simple HTML template with template tags and variables iterating over `book_list` to display the list of books.

8. In `bookr/reviews/urls.py`, add to `urlpatterns` the following URL pattern to invoke the `book_list` view:

```
path('books/', views.book_list, name='book_list'),
```

This configures the URL for the `book_list` view function.

9. Save all the modified files and wait for the Django service to restart. Open `http://0.0.0.0:8000/books/` in the browser, and you should see something similar to the following figure:

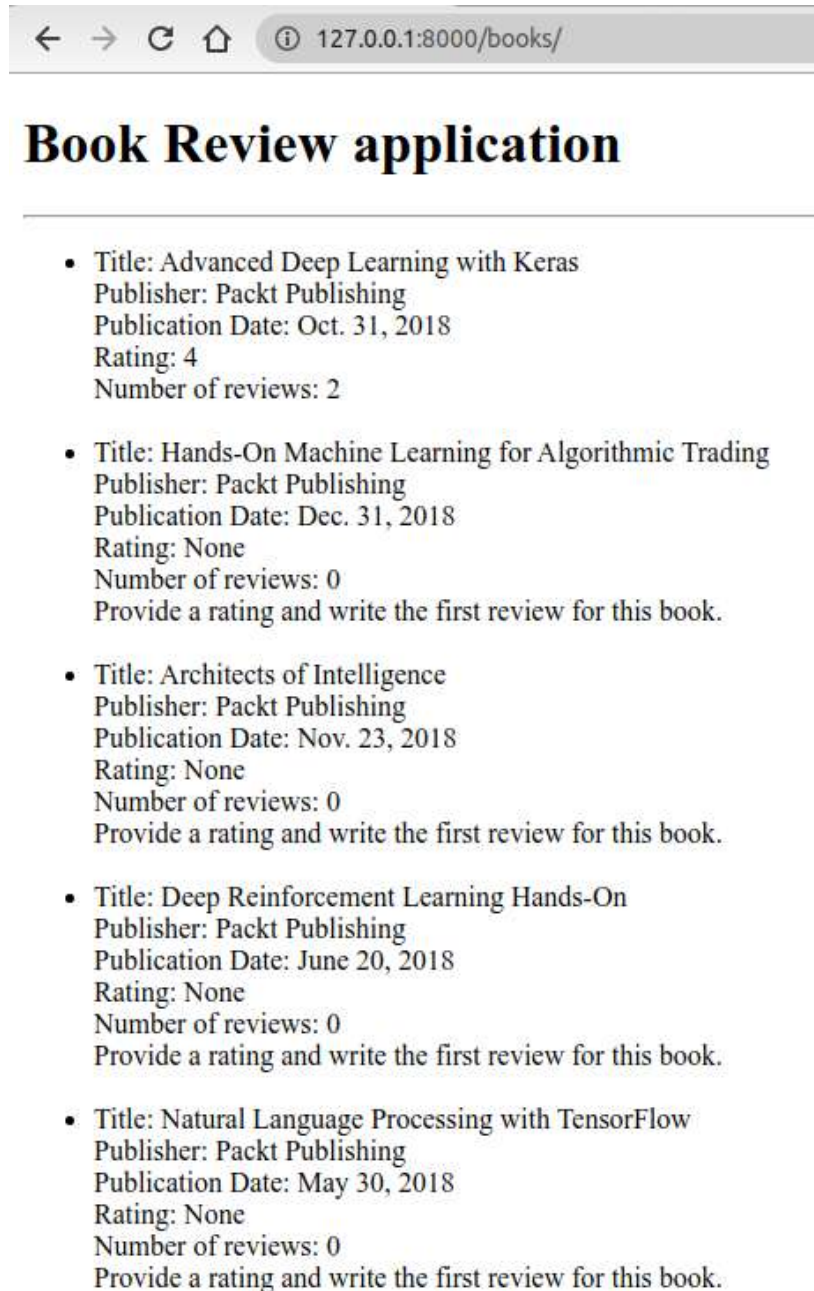


Figure 3.4 – List of books present in the book review application

In this exercise, we created a view function, created templates, and added a URL configuration, which can display a list of all books present in the application. Although we were able to display a list of books using a single template, next, let's explore a bit about how to work with multiple templates in an application that has common or similar code.

Template inheritance

As we build the project, the number of templates will increase. It is highly probable that when we design the application, some of the pages will look similar and have common HTML code for certain features. Using template inheritance, we can inherit the common HTML code in other HTML files. This is similar to class inheritance in Python, where the parent class has all the common code, and the child class has those extras that are unique to the child's requirements.

Note

It would be good practice to update the home view to use an `home.html` that is a child of `base.html`. Having the base template as the common ancestor to templates allows for all the boilerplate, style sheets, JavaScript, and so on to be defined once and then shared between all templates.

For example, let's consider the following to be a parent template that is named `base.html`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>{% block title %}Hello World{% endblock %}</title>
</head>
<body>
    <h1>Hello World using Django templates!</h1>
    {% block content %}
    {% endblock %}
</body>
</html>
```

The following is an example of a child template:

```
{% extends 'base.html' %}
{% block content %}
```

```
<h1>How are you doing?</h1>
{% endblock %}
```

In the preceding snippet, the `{% extends 'base.html' %}` line extends the template from `base.html`, which is the parent template. After extending from the parent template, any HTML code in between the block content will be displayed along with the parent template. Once the child template is rendered, here is how it looks in the browser:

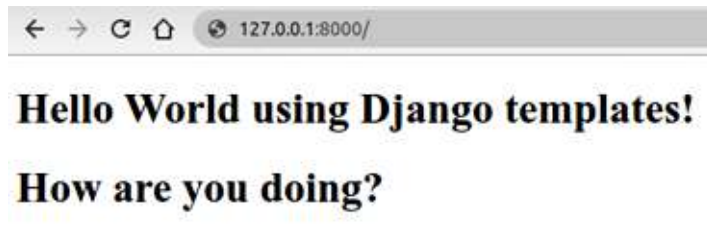


Figure 3.5 – Greeting message after extending the `base.html` template

In the next section, we will do some template styling using Bootstrap.

Template styling with Bootstrap

We have seen how to display all the books using views, URL configuration, and templates. Although we were able to display all the information in the browser, it would be even better if we could add some styling and make the web page look better. For this, we can add a few elements of **Bootstrap**. Bootstrap is an open source **Cascading Style Sheets (CSS)** framework that is particularly good for designing responsive pages that work across desktop and mobile browsers.

Using Bootstrap is simple. First, you need to add the Bootstrap CSS to your HTML. You can experiment for yourself by creating a new file called `example.html`. Populate it with the following code and open it in a browser:

```
<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
      initial-scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href=
      "https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css"
```

```
integrity="sha384-Vkoo8x4CGs03+Hhxv8T/
Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh" crossorigin="anonymous">

</head>
<body>
    Content goes here
</body>
</html>
```

The Bootstrap CSS link in the preceding code adds the Bootstrap CSS library to your page. This means that certain HTML element types and classes will inherit their styles from Bootstrap. For example, if you add the `btn-primary` class to the class of a button, the button will be rendered blue with white text. Try adding the following between `<body>` and `</body>`:

```
<h1>Welcome to my Site</h1>
<button type="button" class="btn btn-primary">
    Checkout my Blog!</button>
```

You will see that the title and button are both styled nicely, using Bootstrap's default styles:



Figure 3.6 – Display after applying Bootstrap

This is because in the Bootstrap CSS code, they specify the color of the `btn-primary` class with the following code:

```
.btn-primary {
    color: #fff;
    background-color: #007bff;
    border-color: #007bff
}
```

You can see that using third-party CSS libraries such as Bootstrap allows you to quickly create nicely styled components without needing to write too much CSS. Next, we shall use template inheritance from a base template and also add some styling elements, such as a navigation bar.

Note

We recommend that you explore Bootstrap further with its tutorial here: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>.

Exercise 3.04 – Adding template inheritance and a Bootstrap navigation bar

In this exercise, we will use template inheritance to inherit the template elements from a base template and reuse them in the `book_list` template to display the list of books. We will also use certain elements of Bootstrap in the base HTML file to add a navigation bar to the top of our page. The Bootstrap code for `base.html` was taken from <https://getbootstrap.com/docs/5.3/getting-started/introduction/> and <https://getbootstrap.com/docs/5.3/components/navbar/>. Let's get started with the steps:

1. Open the `base.html` file from the `bookr/templates/base.html` location. Remove any existing code and replace it with the following code:

```
<!doctype html>
{% load static %}
<html lang="en">
<head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
        initial-scale=1, shrink-to-fit=no">
    <!-- Bootstrap CSS -->
```

You can view the entire code for this file in the `Chapter03` folder in this book's GitHub repo. This is a `base.html` file with all the Bootstrap elements for styling and the navigation bar.

2. Next, open the template at `bookr/reviews/templates/reviews/book_list.html`, remove all the existing code, and replace it with the following code:

```
{% extends 'base.html' %}

{% block content %}
<ul class="list-group">
  {% for item in book_list %}
    <li class="list-group-item">
      <span class="text-info">Title: </span> <span>{{
        item.book.title }}</span>
      <br>
      <span class="text-info">Publisher:
        </span><span>{{
          item.book.publisher }}</span>
    </li>
  {% endfor %}
</ul>
```

You can view the complete code for this file in the `Chapter03` folder in this book's GitHub repo. This template has been configured to inherit the `base.html` file, and it has also been added with a few styling elements to display the list of books. The part of the template that helps in inheriting the `base.html` file is as follows:

```
{% extends 'base.html' %}

{% block content %}
{% endblock %}
```

3. After adding the two new templates, open either of the URLs (`http://0.0.0.0:8000/books/` or `http://127.0.0.1:8000/books/`) in your web browser to see the book list page, which should now look neatly formatted:

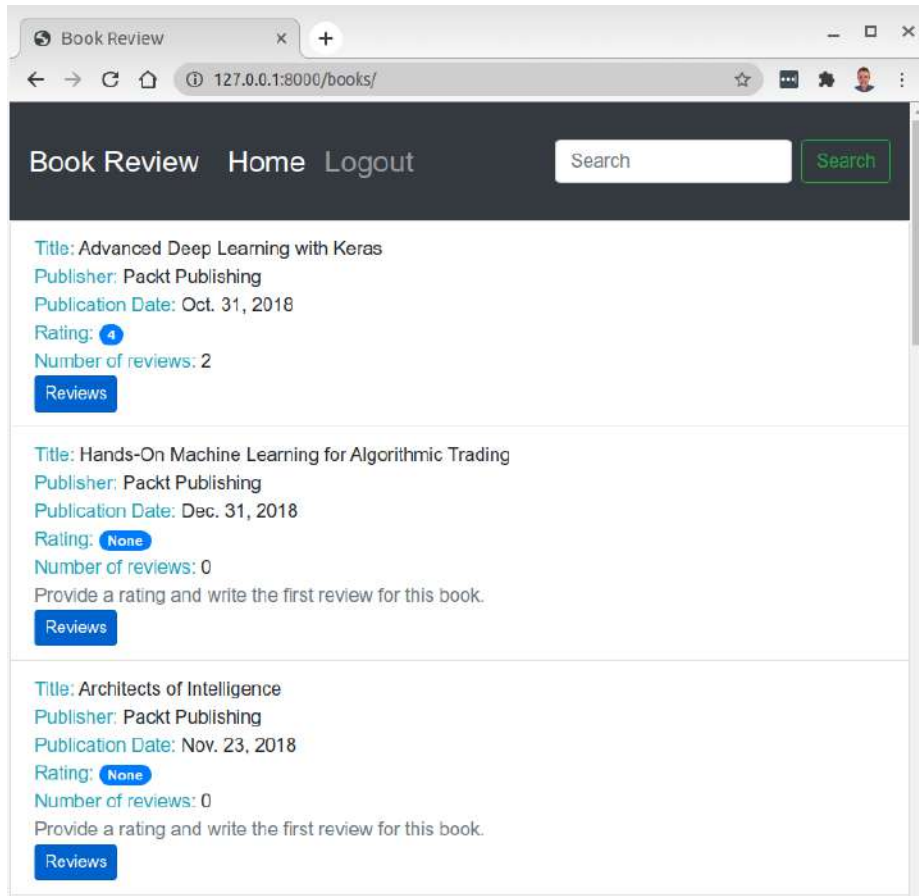


Figure 3.7 – Neatly formatted book list page

In this exercise, we added some styling to the application using Bootstrap, and we also used template inheritance while we displayed the list of books from the book review application. So far, we have worked extensively on displaying all the books present in the application. In the next activity, you will display the details and reviews of an individual book.

Activity 3.01 – Implementing the book details view

In this activity, you will implement a new view, template, and URL configuration to display these details of a book: title, publisher, publication date, and overall rating. In addition to these details, the page should also display all the review comments, specifying the name of the commenter and the dates on which the comments were written and (if applicable) modified. The following steps will help you complete this activity:

1. Create a book details endpoint that extends the base template.
2. Create a book details view that takes a specific book's primary key as the argument and returns an HTML page listing the book's details and any associated reviews.
3. Add the required URL configuration in `urls.py`. The book details view URL should be `http://0.0.0.0:8000/books/1/` (where 1 will represent the ID of the book being accessed). You can use the `get_object_or_404` method to retrieve the book with the given primary key.

Note

The `get_object_or_404` function is a useful shortcut for retrieving an instance based on its primary key. You could also do this using the `.get()` method described in *Chapter 2*: `Book.objects.get(pk=pk)`. However, `get_object_or_404` has the added advantage of returning an HTTP 404 Not Found response if the object does not exist. If we simply use `get()` and someone attempts to access an object that does not exist, our Python code will hit an exception and return an HTTP 500 Server Error response. This is undesirable because it looks as though our server has failed to handle the request correctly.

4. At the end of the activity, you should be able to click the **Reviews** button on the book list page and get a detailed view of the book. The detailed view should have all the details displayed, as shown in the following screenshot:

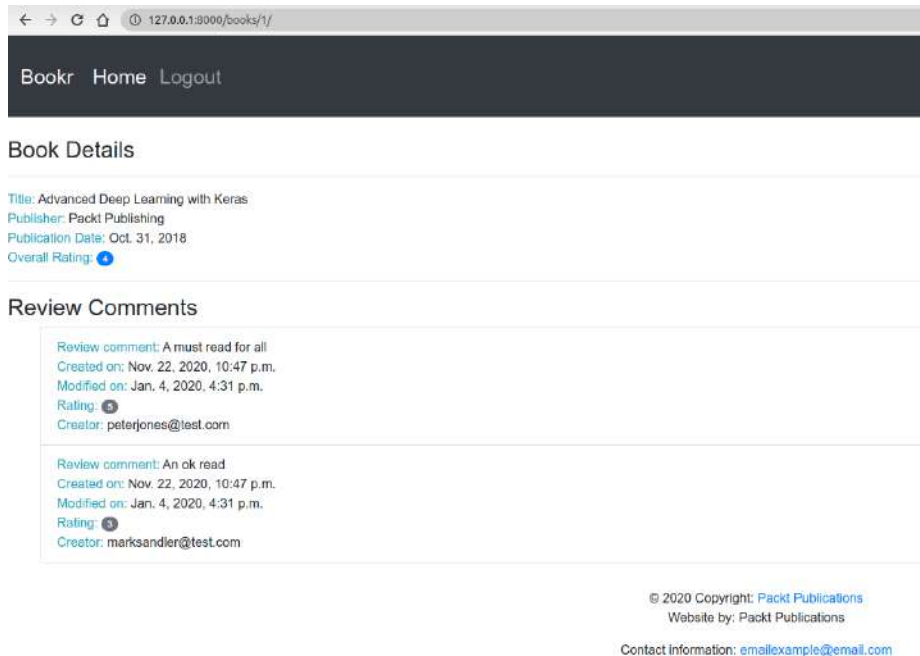


Figure 3.8 – Page displaying the book details

So far in this section, we have learned how to use Django templates to present static information to the user, template language to display the application's dynamic data, and Bootstrap to add styling to the information and make it presentable to users.

Summary

This chapter covered the core infrastructure required to handle an HTTP request to our website. The request is first mapped via URL patterns to an appropriate view. Parameters from the URL are also passed into the view to specify the object displayed on the page.

The view is responsible for compiling any necessary information to display on the website, and then passes this dictionary through to a template, which renders the information as HTML code that can be returned as a response to the user. We covered both class- and function-based views and learned about the Django template language and template inheritance. We created two new pages for the book review application, one displaying all the books present and the other being the book details view page.

In the next chapter, we will learn about Django admin and superuser, registering models, and performing **create, read, update, and delete (CRUD)** operations using the admin site.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

4

An Introduction to Django Admin

This chapter introduces you to the basic functionality of the Django admin app. You will start by creating superuser accounts for the Bookr app, before moving on to executing **Create, Read, Update, and Delete (CRUD)** operations with the admin app. You will learn how to integrate your Django app with the admin app, and you'll also look at the behavior of `ForeignKey` in the admin app. At the end of this chapter, you will see how you can customize the admin app according to a unique set of preferences by subclassing the `AdminSite` and `ModelAdmin` classes to make its interface more intuitive and user-friendly.

In this chapter, we will cover the following topics:

- Introduce the Django admin app
- Creating a superuser account
- CRUD operations using the Django admin app
- Managing Django users and groups
- Registering models with the admin app
- Customizing the admin interface

By the end of this chapter, you will be able to create superuser accounts, user accounts, and groups, and assign permissions to them. You will be able to register your models with the admin app, customize the functionality of change list pages and forms for administering your models, as well as customize the global properties of the admin app.

Technical requirements

Find the solution in the `Chapter04` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Introducing the Django admin app

When developing an app, there is often a need to populate it with data and then alter that data. We have already seen, in *Chapter 2*, how this can be done in the command line using the Python `manage.py` shell. In *Chapter 3*, we learned how to develop a web form interface for our model using Django's views and templates. But neither of these approaches is ideal for administering the data from the classes in `reviews/models.py`. Using the shell to manage data is too technical for non-programmers, and building individual web pages would be a laborious process, as it would see us repeating the same view logic and very similar template features for each table in the model. Fortunately, a solution to this problem was devised in the early days of Django when it was still being developed.

Django admin is actually written as a Django app. It offers an intuitively rendered web interface to give administrative access to the model data. The admin interface is designed to be used by the website administrators. It is not intended to be used by non-privileged users who interact with the site. In our case of a book review system, the general population of book reviewers will never encounter the admin app. They will see the app pages, like those we built with views and templates in *Chapter 3*, and will write their reviews on the pages.

Also, while developers put a lot of effort into creating a simple and inviting web interface for general users, the admin interface, aimed at administrative users, maintains a utilitarian feel that typically displays the intricacies of the models that make up the application and are registered with the admin. It may have escaped your attention, but you already have an admin app in your Bookr project. Look at the list of installed apps in `bookr/bookr/settings.py`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    ...  
]
```

Now, look at the URL patterns in `bookr/bookr/urls.py`:

```
urlpatterns = [  
    path("admin/", admin.site.urls),  
    ...  
]
```

If we put this path into our browser, we can see that the link to the admin app on the development server is `http://127.0.0.1:8000/admin/`. Before we make use of it, though, we need to create a superuser through the command line.

Creating a superuser account

Our Bookr application has just found a new user. Her name is Alice, and she wants to start adding her reviews right away. Bob, who is already using Bookr, has just informed us that his profile seems incomplete and needs to be updated. Eve no longer wants to use the application and wants their account to be deleted. For security reasons, we do not want just any user performing these tasks for us. That's why we need to create a **superuser** with elevated privileges. Let's start by doing just that.

In Django's authorization model, a superuser is one with all permissions. A user with the staff flag set is able to log in to the admin site, but then their specific permissions dictate what they can see and do with the models registered. We will examine this later in the chapter and learn more about this authorization model in *Chapter 9*.

We can create a superuser by using the `manage.py` script, which we explored in earlier chapters. Again, we need to be in the project directory when we enter the command. We will use the `createsuperuser` subcommand by entering the following command in the command line:

```
python manage.py createsuperuser
```

Note

In this chapter, we will use email addresses that fall under the `example.com` domain. This follows an established convention to use this reserved domain for testing and documentation. You could use your own email addresses if you prefer.

Let's go ahead and create our superuser.

Exercise 4.01 – Creating a superuser account

In this exercise, you will create a superuser account that lets the user log in to the admin site. This functionality will also be used in the upcoming exercises to implement changes that only a superuser can. The following steps will help you complete this exercise:

1. Enter the following command to create a superuser:

```
python manage.py createsuperuser
```

On executing the preceding command, you will be prompted to create a superuser. This command will prompt you for a superuser name, an optional email address, and a password.

2. Add the username and email for the superuser as follows. Here, we enter `bookadmin` (highlighted) at the prompt and press the *Enter* key. Similarly, at the next prompt, which asks you to enter your email address, you can add `bookadmin@example.com` (highlighted). Press the *Enter* key to continue:

```
Username (leave blank to use 'django'): bookadmin
Email address: bookadmin@example.com
Password:
```

This will assign the name `bookadmin` to the superuser. Note that you won't see any output immediately.

3. The next prompt in the shell is for your password. Add a strong password and press the *Enter* key to confirm it once again:

```
Password:
Password (again):
```

You should see the following message on your screen:

```
Superuser created successfully.
```

Note that the password is validated as per the following criteria:

- It cannot be among the 20 thousand most common passwords
- It should have a minimum of eight characters
- It cannot be only numerical characters
- It cannot be derived from the username, first name, last name, or email address of the user

These criteria are determined by `AUTH_PASSWORD_VALIDATORS` in the `settings.py` file. It is possible to change the required password strength by configuring these validators or writing custom validators.

With this, you have created a superuser named `bookadmin` who can log in to the admin app. The following screenshot shows how this looks in the shell:

```
> python manage.py createsuperuser
Username (leave blank to use 'django'): bookradmin
Email address: bookradmin@example.com
Password:
Password (again):
Superuser created successfully.
>
```

Figure 4.1 – Creating a superuser

4. Visit the admin app at <http://127.0.0.1:8000/admin> and log in with the superuser account that you have created:

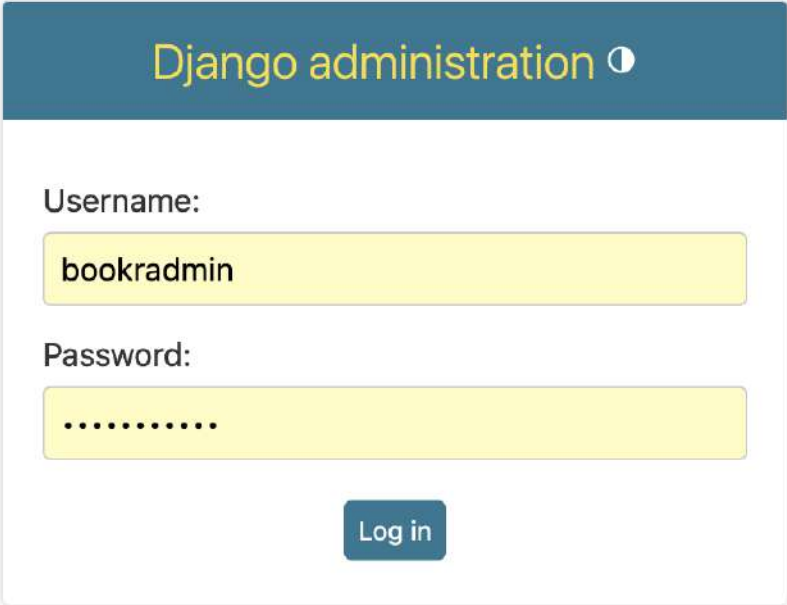
The image shows a web browser window displaying the Django administration login page. At the top, there is a dark blue header with the text "Django administration" in yellow, followed by a small circular icon. Below the header, the page has a white background. There are two labels: "Username:" and "Password:". Under "Username:", there is a yellow rectangular input field containing the text "bookradmin". Under "Password:", there is a yellow rectangular input field filled with ten dots. Below these fields is a blue button with the text "Log in" in white.

Figure 4.2 – The Django administration login form

In this exercise, you created a superuser account that we will be using for the rest of this chapter to assign or remove privileges as needed.

With a superuser account created, we are now able to log into the Django admin app and learn about the CRUD facilities that it provides.

CRUD operations using the Django admin app

Let's get back to the requests we got from Bob, Alice, and Eve. As a superuser, your tasks will involve creating, updating, retrieving, and deleting various user accounts, reviews, and title names. This set of activities is collectively termed CRUD. CRUD operations are central to the behavior of the admin app. It turns out that the admin app is already aware of the models from another Django app, Authentication and Authorization – referenced in `INSTALLED_APPS` as `'django.contrib.auth'`. When logging in to `http://127.0.0.1:8000/admin/`, we are presented with the models from the authorization application, as shown in *Figure 4.3*:

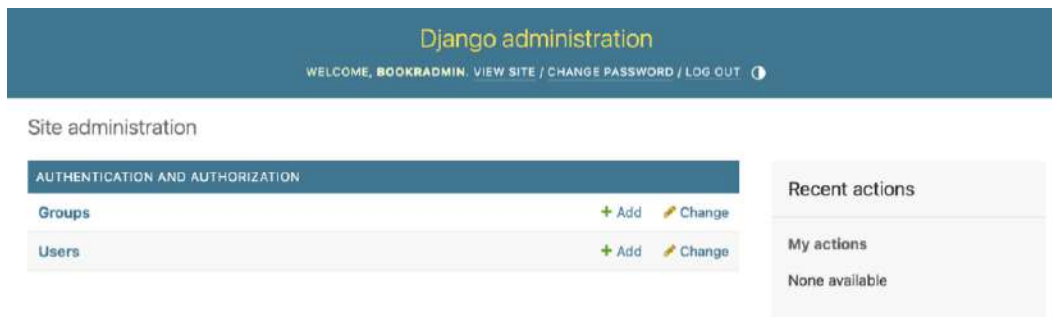


Figure 4.3 – The Django administration window

When the admin app is initialized, it calls its `autodiscover()` method to detect whether any other installed apps contain an admin module. If so, these admin models are imported. In our case, it has discovered `'django.contrib.auth.admin'`. Now that the modules are imported and our superuser account is ready, let's start working on the requests from Bob, Alice, and Eve.

Create

Before Alice starts writing her reviews, we need to create an account for her through the admin app. Once that is done, we can look at the administration access levels we can assign to her. To create a user, we need to click the + **Add** option next to **Users** (refer to *Figure 4.3*) and fill out the form, as shown in *Figure 4.4*.

Note

We don't want any random user to have access to the Bookr users' accounts. Therefore, it is imperative that we choose strong, secure passwords.

Home > Authentication and Authorization > Users > Add user

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

Add user

First, enter a username and password. Then, you'll be able to edit more user options.

Username:
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password-based authentication: ☒ Enabled ☐ Disabled
Whether the user will be able to authenticate using a password or not. If disabled, they may still be able to authenticate using other backends, such as Single Sign-On or LDAP.

Password:
Your password can't be too similar to your other personal information.
 Your password must contain at least 8 characters.
 Your password can't be a commonly used password.
 Your password can't be entirely numeric.

Password confirmation:
Enter the same password as before, for verification.

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

Figure 4.4 – The Add user page

There are three buttons at the bottom of the form:

- **Save and add another:** This creates the user and renders the same **Add user** page again, with blank fields.
- **Save and continue editing:** This creates the user and loads the **Change user** page. The **Change user** page lets you add additional information that wasn't present on the **Add user** page, such as **First name**, **Last name**, and more (see *Figure 4.5*). Note that **Password** does not have an editable field on the form. Instead, it shows information about the hashing technique that it is stored with, in addition to a link to a separate **Reset password** form:

 The user "alice" was added successfully. You may edit it again below.

Change user

alice HISTORY

Username:

alice

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

algorithm: pbkdf2_sha256 iterations: 870000 salt: 0QXJFj*****
hash: HM/vXt*****

Reset password

Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Alice

Last name:

White

Email address:

alice.white@example.com

Permissions

☒ Active

Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ Staff status

Designates whether the user can log into this admin site.

☐ Superuser status

Designates that this user has all permissions without explicitly assigning them.

Figure 4.5 – The Change user page presented after clicking Save and continuing editing

- **SAVE:** This creates the user and lets the user navigate to the **Select user to change** list page, as depicted in *Figure 4.6*.

Activity 4.01 – Creating a user account

We have now seen how to create user accounts through the Django admin interface.

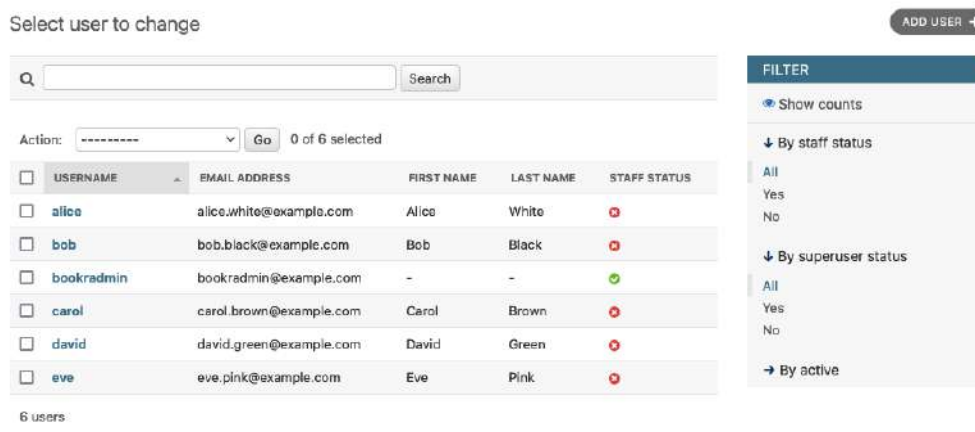
It is time to put your knowledge into practice to create an account for a new user, David Green, using the email address `david.green@example.com`.

As was the case with Alice White's account, just set this one as an active user without ticking the **Staff status** or **Superuser status** boxes.

Now that you have created a user through the Django admin interface, we can examine the other three CRUD operations: retrieve, update, and delete.

Retrieve

The administrative tasks need to be divided among some users, and for this, the admin (the person with the superuser account) would like to view those users whose email addresses end with `n@example.com` and assign the tasks to these users. This is where the **retrieve** functionality can come in handy. After we have clicked the **SAVE** button on the **Add user** page (refer to *Figure 4.4*), we are taken to the **Select user to change** list page (as shown in *Figure 4.6*), which carries out the retrieve operation. Note that the **Create** form is also reachable by clicking on the **ADD USER +** button on the **Select user to change** list page. So, after we have added a few more users, the change list will look something like this:



Select user to change

ADD USER +

Search

Action: ----- Go 0 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✗
☐	bob	bob.black@example.com	Bob	Black	✗
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗
☐	eve	eve.pink@example.com	Eve	Pink	✗

6 users

FILTER

Show counts

By staff status

All
Yes
No

By superuser status

All
Yes
No

By active

Figure 4.6 – The Select user to change page

At the top of the form is a **Search** bar that searches the content, such as username, email address, and first and last names of users. On the right-hand side is a **FILTER** panel that narrows down the selection based on the values of **By staff status**, **By superuser status**, and **By active**. In *Figure 4.7*, you can see what happens when we search the `n@example.com` string. This will return only the names of the users whose email addresses consist of a username ending with an `n` and a domain starting with `example.com`. We can only see three users with email addresses matching this requirement – `bookradmin@example.com`, `carol.brown@example.com`, and `david.green@example.com`:

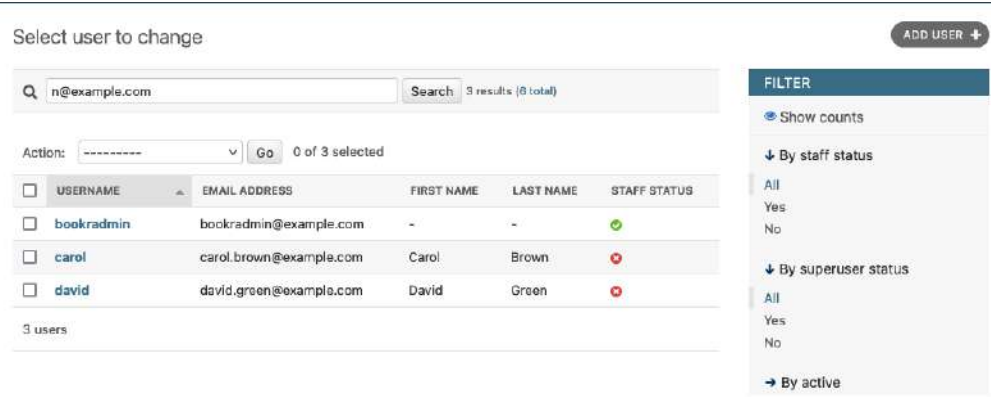


Figure 4.7 – Searching for users by a portion of their email address

The retrieve functionality, such as the list page and search bar, allows us to obtain records for update and delete operations.

Update

Remember that Bob wanted his profile to be updated. Let's update Bob's unfinished profile by clicking the **bob** username link in the **Select user to change** list:

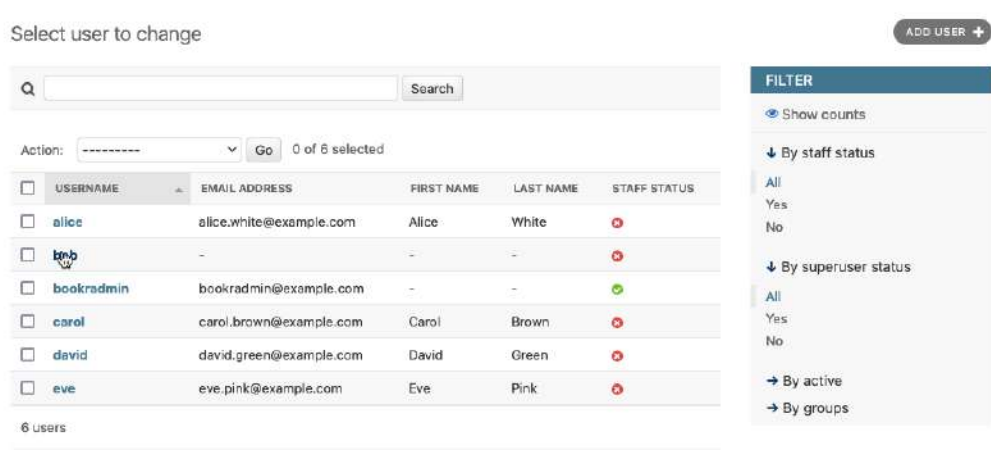


Figure 4.8 – Selecting bob from the Select user to change list

This will take us back to the **Change user** form, where the values for **First name**, **Last name**, and **Email address** can be entered:

Django administration

WELCOME, BOOKADMIN. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#) ⓘ

Home > Authentication and Authorization > Users > bob

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

Change user

bob [HISTORY](#)

Username:

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password: **invalid password format or unknown hashing algorithm.**

[Reset password](#)

Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Last name:

Email address:

Figure 4.9 – Adding personal info

As can be seen from *Figure 4.9*, we are adding personal information about Bob here – his name, surname, and email address, specifically.

Another type of update operation is *soft deleting*. The **Active** Boolean property allows us to deactivate a user rather than deleting the entire record and losing all the data that has dependencies on the account. This practice of using a Boolean flag to denote a record as inactive or removed (and subsequently filtering these flagged records out of queries) is referred to as a **soft delete**. Similarly, we can upgrade the user to **Staff status** or **Superuser status** by ticking the respective checkboxes for those:

Permissions

☒ Active
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ Staff status
Designates whether the user can log into this admin site.

☐ Superuser status
Designates that this user has all permissions without explicitly assigning them.

Figure 4.10 – The Active, Staff status, and Superuser status permissions

Delete

Eve no longer wants to use the Bookr application and has requested that we delete their account. The auth admin caters to this, too. Select a user or user records on the **Select user to change** list page and choose the **Delete selected users** option from the **Action** drop-down menu. Then, hit the **Go** button (Figure 4.11):

Select user to change

ADD USER +

Q

Search

Action:

✓ Delete selected users

1 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✖
☐	bob	bob.black@example.com	Bob	Black	✖
☐	bookradmin	bookradmin@example.com	-	-	✔
☐	carol	carol.brown@example.com	Carol	Brown	✖
☐	david	david.green@example.com	David	Green	✖
☒	eve	eve.pink@example.com	Eve	Pink	✖

6 users

FILTER

Show counts

↓ By staff status

All

Yes

No

↓ By superuser status

All

Yes

No

→ By active

→ By groups

Figure 4.11 – Deleting from the Select user to change list page

You will be presented with a confirmation screen and taken back to the **Select user to change** list once you have deleted the object:

Are you sure?

Are you sure you want to delete the selected user? All of the following objects and their related items will be deleted:

Summary

- Users: 1
- Reviews: 1

Objects

- User: eve
 - Review: Review object (3)

Yes, I'm sure No, take me back

Figure 4.12 – User deletion confirmation

You will see the following message once the user is deleted:

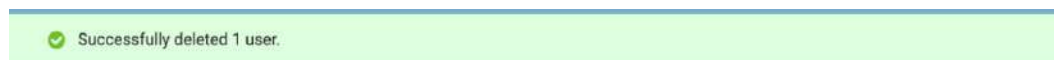


Figure 4.13 – User deletion notification

After that confirmation, you will find that Eve's account no longer exists.

So far, we have learned how we can add a new user, get the details of another user, make changes to the data for a user, and delete a user. These skills helped us cater to Alice, Bob, and Eve's requests. As the number of users of our app grows, managing requests from hundreds of users will eventually become quite difficult. One way around this problem would be to delegate some of the administrative responsibilities to a selected set of users. We'll learn how to do that in the section that follows.

Managing Django users and groups

Django's authentication model consists of users, groups, and permissions. Users can belong to many groups, which is a way of categorizing users. It also streamlines the implementation of permissions by allowing permissions to be assigned to collections of users as well as individuals.

In the *CRUD operations using the Django admin app* section, we saw how a superuser account could cater to Alice, David, and Bob's requests to make modifications to their profiles. It was quite easy to do, and our application seems well-equipped to handle their requests.

What will happen when the number of users grows? Will the admin user be able to manage 100 or 150 users at once? As you can imagine, this can be quite a complicated task. To overcome

this, we can give elevated permissions to a certain set of users, and they can help ease the admin's tasks. And that's where groups come in handy. Though we'll learn more about users, groups, and permissions in *Chapter 9*, we can start understanding groups and their functionality by creating a **Help Desk User group** that contains accounts that have access to the admin interface but lack many powerful features, such as the ability to add, edit, or delete groups or to add or delete users.

Exercise 4.02 – Adding and modifying users and groups through the admin app

In this exercise, we will grant a certain level of administrative access to one of our Bookr users, Carol. First, we will define the level of access for a group, and then we will add Carol to the group. This will allow Carol to update user profiles and check user logs. The following steps will help you implement this exercise:

1. Visit the admin interface at `http://127.0.0.1:8000/admin/` and log in as bookradmin using the account set up with the superuser command.
2. In the admin interface, follow the links to **Home | AUTHENTICATION AND AUTHORIZATION | Groups:**

Authentication and Authorization administration

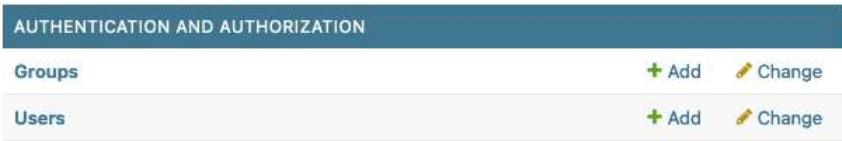


Figure 4.14 – The Groups and Users options on the AUTHENTICATION AND AUTHORIZATION page

3. Use **ADD GROUP +** in the top right-hand corner to add a new group:



Figure 4.15 – Adding a new group

4. Name the group **Help Desk User** and give it the following permissions, as shown in *Figure 4.16*:

- Can view log entry

- Can view permission
- Can change user
- Can view user

Add group

Name:

Permissions:

Available permissions

Q Filter

- Authentication and Authorization | group | Can change group
- Authentication and Authorization | group | Can delete group
- Authentication and Authorization | group | Can view group
- Authentication and Authorization | permission | Can add permission
- Authentication and Authorization | permission | Can change permission
- Authentication and Authorization | permission | Can delete permission
- Authentication and Authorization | permission | Can view permission**
- Authentication and Authorization | user | Can add user
- Authentication and Authorization | user | Can change user**
- Authentication and Authorization | user | Can delete user
- Authentication and Authorization | user | Can view user**

Choose all

Hold down "Control", or "Command" on a Mac, to select more than one.

Chosen permissions

Q Filter

Remove all

Figure 4.16 – Selecting permissions

This can be done by selecting the permissions from **Available permissions** and clicking the right arrow in the middle so that they appear under **Chosen permissions**. Note that to add multiple permissions at a time, you can hold down the *Ctrl* key (or *Command* for Macintosh) to select more than one:

Help Desk User

Name:

Permissions:

Available permissions

Q Filter

- Authentication and Authorization | group | Can view group
- Authentication and Authorization | permission | Can add permission
- Authentication and Authorization | permission | Can change permission
- Authentication and Authorization | permission | Can delete permission
- Authentication and Authorization | user | Can add user
- Authentication and Authorization | user | Can delete user
- Content Types | content type | Can add content type
- Content Types | content type | Can change content type
- Content Types | content type | Can delete content type
- Content Types | content type | Can view content type

Choose all

Hold down "Control", or "Command" on a Mac, to select more than one.

Chosen permissions

Q Filter

- Administration | log entry | Can view log entry
- Authentication and Authorization | permission | Can view permission
- Authentication and Authorization | user | Can change user
- Authentication and Authorization | user | Can view user

Remove all

Figure 4.17 – Adding selected permissions to Chosen permissions

Once you click the **SAVE** button, you will see a confirmation message stating **The group "Help Desk User" was added successfully:**



Figure 4.18 – The group Help Desk User was added successfully

5. Now, navigate to **Home | AUTHENTICATION AND AUTHORIZATION | Users** and click the link of the user with the first name **carol**:

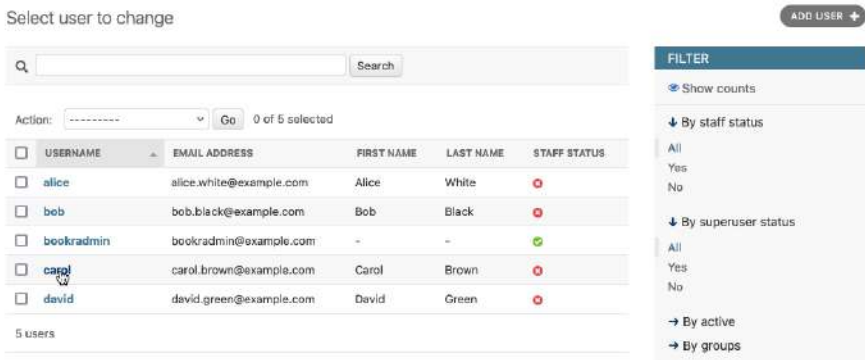
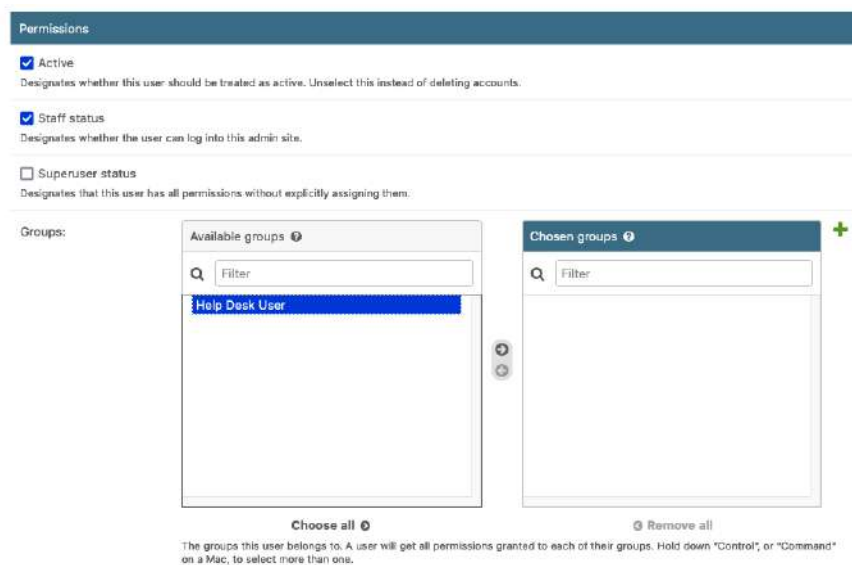


Figure 4.19 – Clicking on the carol username

6. Scroll down to the **Permissions** fields set and select the **Staff status** checkbox. This is required for Carol to be able to log in to the admin app:



Permissions

☒ **Active**
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☒ **Staff status**
Designates whether the user can log into this admin site.

☐ **Superuser status**
Designates that this user has all permissions without explicitly assigning them.

Groups:

Available groups

Help Desk User

Choose all

Chosen groups

Remove all

The groups this user belongs to. A user will get all permissions granted to each of their groups. Hold down "Control", or "Command" on a Mac, to select more than one.

Figure 4.20 – Clicking the Staff status checkbox

7. Add Carol to the **Help Desk User** group that we created in the previous steps by selecting it from the **Available groups** selection box (refer to Figure 4.20) and clicking the right arrow to shift it into her list of **Chosen groups** (as shown in Figure 4.21). Note that unless you do this, Carol won't be able to log in to the admin interface using her credentials:

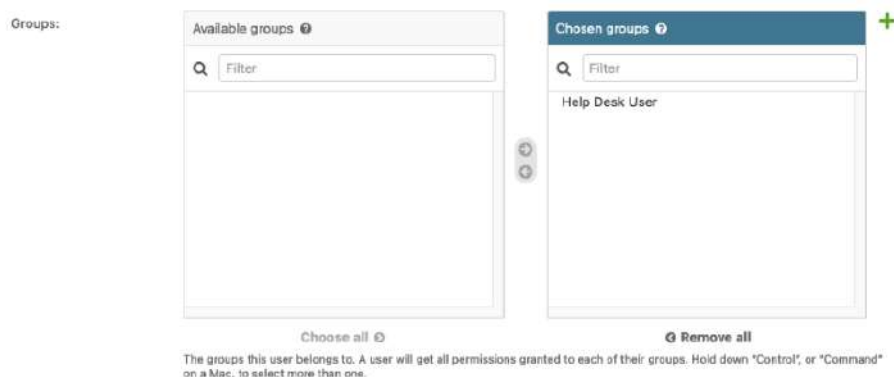


Figure 4.21 – Shifting the Help Desk User group into the Chosen groups list for Carol

8. Let's test whether what we've done up till now has yielded the right outcome. To do this, log out of the admin site and log in again as Carol. Upon logging out, you should see the following on your screen:

Logged out

Thanks for spending some quality time with the web site today.

[Log in again](#)

Figure 4.22 – The Logged out screen

Note

If you don't recall the password that you initially gave her, you can change the password in the command line by typing `python manage.py changepassword carol`.



Figure 4.23 – The admin dashboard

As we did not assign any group permissions, not even **auth | group | Can view group**, to the **Help Desk User** group, when Carol logs in, the **Groups** admin interface is not available to her. Similarly, when you navigate to **Home | AUTHENTICATION AND AUTHORIZATION | Users** and click a user link, you will see that there are no options to edit or delete the user. This is because of the permissions that were granted to the **Help Desk User** group, of which Carol is a member. The members of the group can view and edit users, but cannot add or delete any user.

In this exercise, we learned how to grant a certain number of administrative privileges to users of our Django app.

Now that we have an understanding of how the admin app works with the **AUTHENTICATION AND AUTHORIZATION** models for **Users** and **Groups**, we can turn our attention to registering the models that we have developed in the reviews app.

Registering models with the admin app

The Django admin app produces workable CRUD interfaces to Django objects based on the characteristics of the models, with a minimal amount of coding. In this section, we will first look at how to register the models with the admin app and how elements of the interface, such as change lists and change pages, are derived from the model properties. We will conclude with an important exercise that demonstrates how foreign key settings in the model determine the behavior of the object deletion process.

Registering the reviews model

Let's say that Carol is tasked with improving the **Reviews** section in Bookr; that is, only the most relevant and comprehensive reviews should be shown, and duplicate or spam entries should be removed. For this, she will need access to the reviews model. As we have seen previously with our investigation of groups and users, the admin app already contains admin pages for the models from the *Authentication and Authorization* app, but it does not yet reference the models in our *Reviews* app. To make the admin app aware of the models, we need to explicitly register them with the admin app. Fortunately, we don't need to modify the admin app's code to do so, as we can instead import the admin app into our project and use its API to register our models. This has already been done in the *Authentication and Authorization* app, so let's try it with our *Reviews* app. Our aim is to be able to use the admin app to edit the data in our reviews model.

Take a look at the `reviews/admin.py` file. It is a placeholder file that was generated with the `startapp` subcommand that we used in *Chapter 1*, and currently contains these lines:

```
from django.contrib import admin

# Register your models here.
```

Now, we can try to expand this. To make the admin app aware of our models, we can modify the `reviews/admin.py` file and import the models. Then, we can register the models with the `AdminSite` object, `admin.site`. The `AdminSite` object contains the instance of the Django admin application (later, we will learn how to subclass this `AdminSite` object and override many of its properties). Then, `reviews/admin.py` will look as follows:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor,
                             Book, BookContributor, Review)

admin.site.register(Publisher)
admin.site.register(Contributor)
admin.site.register(Book)
admin.site.register(BookContributor)
admin.site.register(Review)
```

The `admin.site.register` method makes the models available to the admin app by adding it to a registry of classes contained in `admin.site._registry`. If we chose not to make a model accessible through the admin interface, we would simply not register it. When you reload

`http://127.0.0.1:8000/admin/` in your browser, you will see the following on the admin app landing page:



Figure 4.24 – Admin app landing page

Note the change in the appearance of the admin page after the reviews model has been imported.

With the five models of the *Reviews* app registered with the admin app, we can examine some of the default interfaces that it has created for us.

Change lists

We have seen several examples of change list pages so far, as we took a tour of the Django admin app.

A change list is a page that displays a list of links to objects that are retrieved by a query, and also options to perform actions on multiple objects (such as deletion), add new ones, filter, or sort the objects being displayed.

Now that we have registered our models with the admin app, we can take a look at the default change lists that have been created. If we click the **Publishers** link, we will be taken to `http://127.0.0.1:8000/admin/reviews/publisher` and see a change list containing links to the publishers. These links are designated by the `id` field of the `Publisher` objects. If your database

has been populated with the script in *Chapter 3*, you will see a list with seven publishers, as shown in *Figure 4.25*.

Note

Depending on the state of your database and based on the activities you have completed, the object IDs, URLs, and links in your session may be ordered and numbered differently from the following screenshots.

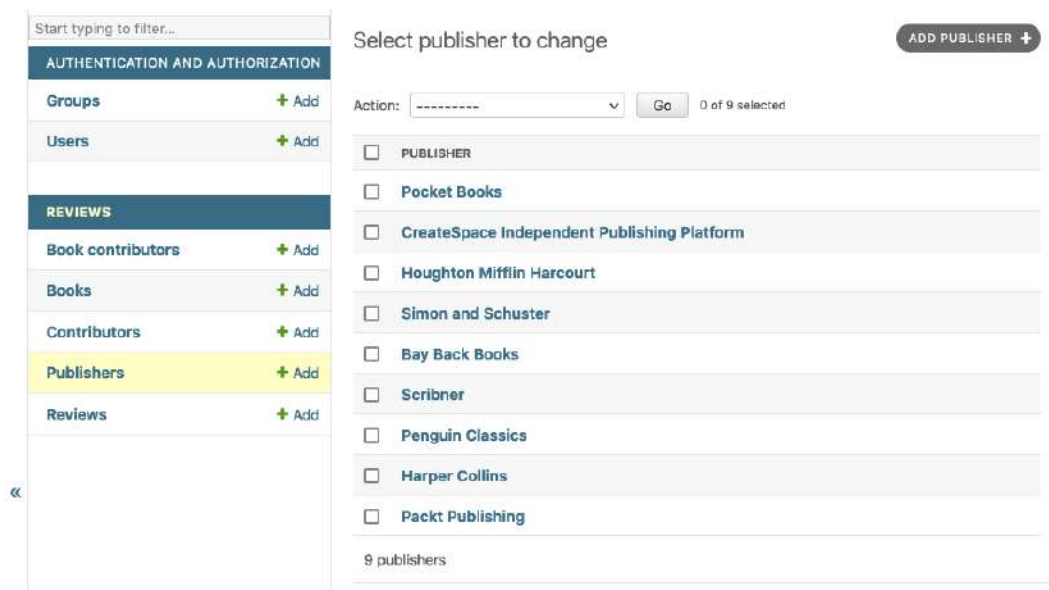


Figure 4.25 – The Select publisher to change list

We can follow one of the links in the change list and examine the **Change publisher** page.

The Change publisher page

The **Change publisher** page at <http://127.0.0.1:8000/admin/reviews/publisher/1> contains what we might expect (see *Figure 4.26*). There is a form for editing the publisher's details. These details have been derived from the `reviews.models.Publisher` class:

Change publisher

Packt Publishing HISTORY

Name:
The name of the Publisher.

Website: **Currently:** <https://www.packtpub.com/>
Change:
The Publisher's website.

Email:
The Publisher's email address.

SAVE Save and add another Save and continue editing Delete

Figure 4.26 – The Change publisher page

If we had clicked the **ADD PUBLISHER +** button, the admin app would have returned a similar form for adding a publisher. The beauty of the admin app is that it gives us all of this CRUD functionality with just one line of coding – `admin.site.register(Publisher)` – using the definition of the `reviews.models.Publisher` attributes as a schema for the page content:

```
class Publisher(models.Model):
    """A company that publishes books."""
    Name = models.CharField(max_length=50,
                           help_text="The name of the
                           Publisher.")
    website = models.URLField(
        help_text="The Publisher's website.")
    email = models.EmailField(
        help_text="The Publisher's email address.")
```

The publisher **Name** field is constrained to 50 characters as specified in the model. The help text that appears in gray below each field is derived from the `help_text` attributes specified on

the model. We can see that `models.CharField`, `models.URLField`, and `models.EmailField` are rendered in HTML as input elements of the text, url, and email types, respectively.

The fields in the form come with validation where appropriate. Unless model fields are set to `blank=True` or `null=True`, the form will throw an error if the field is left blank, as is the case for the `Publisher.name` field. Similarly, as `Publisher.website` and `Publisher.email` are respectively defined as instances of `models.URLField` and `models.EmailField`, they are validated accordingly. In *Figure 4.27*, we can see the validation of **Name** as a required field, **Website** as a URL, and **Email** as an email address:

Add publisher

Please correct the errors below.

Name:

This field is required.

The name of the Publisher.

Website:

Enter a valid URL.

The Publisher's website.

Email:

Enter a valid email address.

The Publisher's email address.

Figure 4.27 – Field validation

It is useful to examine how the admin app renders elements of the models to understand how it functions. In your browser, right-click **View Page Source** and examine the HTML that has been rendered for this form. You will see a browser tab displaying HTML code similar to that in *Figure 4.28*.

```
<fieldset class="module aligned ">
  <div class="form-row errors field-name">
    <ul class="errorlist"><li>This field is
      required.</li></ul>
    <div>
      <label class="required" for="id_name">Name:</label>
      <input type="text" name="name" class="vTextField"
        maxlength="50" required="" id="id_name">
      <div class="help">The name of the Publisher.</div>
    </div>
  </div>
  <div class="form-row errors field-website">
    <ul class="errorlist"><li>Enter a valid URL.</li></ul>
    <div>
      <label class="required"
        for="id_website">Website:</label>
      <input type="url" name="website" value="packtcom"
        class="vURLField" maxlength="200" required=""
        id="id_website">
      <div class="help">The Publisher's website.</div>
    </div>
  </div>
  <div class="form-row errors field-email">
    <ul class="errorlist"><li>Enter a valid email
      address.</li></ul>
    <div>
      <label class="required" for="id_email">Email:</label>
      <input type="email" name="email"
        value="infoatpackt.com"
        class="vTextField" maxlength="254"
        required="" id="id_email">
      <div class="help">The Publisher's email
        address.</div>
    </div>
  </div>
</fieldset>
```

Figure 4.28 – Browser displaying HTML code

The form has a `publisher_form` ID, and it contains a field set with HTML elements corresponding to the data structure of the `Publisher` model in `reviews/models.py`, shown as follows:

```
class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(max_length=50,
                           help_text="The name of the
                           Publisher.")
    website = models.URLField(
        help_text="The Publisher's website.")
    email = models.EmailField(
        help_text="The Publisher's email address.")
```

Note that for the name, the input field is rendered like this:

```
<input type="text" name="name" value="Packt Publishing"
class="vTextField" maxlength="50"
required="" id="id_name">
```

It is a required field, and it has a text type and `maxlength` of 50, as defined by the `max_length` parameter in the model definition:

```
name = models.CharField(max_length=50,
                        help_text="The name of the
                        Publisher.")
```

Similarly, we can see the website and email being defined in the model, as `URLField` and `EmailField` are rendered in HTML as input elements of the `url` and `email` types, respectively:

```
<input type="url" name="website"
value="https://www.packtpub.com/"
class="vURLField" maxlength="200" required=""
id="id_website">
<input type="email" name="email" value="info@packtpub.com"
class="vTextField" maxlength="254"
required="" id="id_email">
```

We have learned that the Django admin app derives sensible HTML representations of Django models based on the model definitions that we have provided.

Now let's examine the book change page.

The book change page

Similarly, there is a change page that can be reached by selecting **Books** from the **Site administration** page and then selecting a specific book in the change list:



Figure 4.29 – Selecting Books from the Site administration page

After clicking **Books**, as shown in the preceding screenshot, you will see the following on your screen:

Select book to change ADD BOOK +

Action: 0 of 18 selected

☐	BOOK
☐	The Talisman
☐	Paul Clifford
☐	Animal Farm: A Fairy Story
☐	1984
☐	Pride and Prejudice
☐	Fahrenheit 451
☐	The Catcher in the Rye
☐	The Great Gatsby
☐	To Kill A Mocking Bird
☐	For Whom The Bell Tolls
☐	The Grapes of Wrath
☐	Brave New World
☐	Hands-On Reinforcement Learning with Python
☐	Natural Language Processing with TensorFlow
☐	Deep Reinforcement Learning Hands-On
☐	Architects of Intelligence
☐	Hands-On Machine Learning for Algorithmic Trading
☐	Advanced Deep Learning with Keras

18 books

Figure 4.30 – The Select book to change page

In this instance, selecting the book *Architects of Intelligence* will take us to `http://127.0.0.1:8000/admin/reviews/book/3/change/`. In the previous example, all the model

fields were rendered as simple HTML text widgets. The rendering of some other subclasses of `django.db.models.Field` used in `models.Book` are worthy of closer examination:

The screenshot shows the Django Admin interface for changing a book. The header is dark blue with 'Django administration' in yellow. Below it, a navigation bar shows 'Home > Reviews > Books > Architects of Intelligence'. The main content area is white. At the top, 'Change book' is the title, and 'Architects of Intelligence' is the object name. A 'HISTORY' button is in the top right. The form has four fields: 'Title' (text input with value 'Architects of Intelligence'), 'Date the book was published.' (date selection widget showing '2018-11-23'), 'ISBN number of the book.' (text input with value '9781789954531'), and 'Publisher:' (dropdown menu with value 'Packt Publishing'). At the bottom, there are four buttons: 'SAVE', 'Save and add another', 'Save and continue editing', and 'Delete'.

Figure 4.31 – The Change book page

Here, `publication_date` is defined using `models.DateField`. It is rendered using a date selection widget. The visual representation of the widgets will vary among operating systems and the choice of browser:

Title:

Architects of Intelligence

Date the book was published:

2018-11-23

Today 

ISBN number of the book:

9781789954531

Publisher:

Packt Publishing

NOVEMBER 2018

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

Yesterday | Today | Tomorrow

Cancel

Figure 4.32 – Date selection widget

As Publisher is defined as a foreign key relation, it is rendered as a **Publisher** drop-down menu with a list of the Publisher objects:

ISBN number of the book.

Publisher:

Figure 4.33 – Publisher drop-down menu

This brings us to how the admin app handles deletion. The admin app takes its cue from the models' foreign key constraints when determining how to implement deletion functionality. In the `BookContributor` model, `Contributor` is defined as a foreign key. The code in `reviews/models.py` looks as follows:

```
contributor = models.ForeignKey(Contributor,
    on_delete=models.CASCADE)
```

By setting `on_delete=CASCADE` on a foreign key, the model specifies the database behavior required when a record is deleted; the deletion is cascaded to other objects that are referenced by the foreign key.

It is important to understand the impact of foreign key settings on the flow of the deletion process, and the following exercise examines this.

Exercise 4.03 – Foreign keys and deletion behavior in the admin app

At present, all the `ForeignKey` relations in the reviews models are defined with an `on_delete=CASCADE` behavior. For instance, think of a case wherein an admin deletes one of the publishers. This would delete all the books that are associated with the publisher. We do not want that to happen, and that is precisely the behavior that we will be changing in this exercise:

1. Visit the **Contributors** change list at `http://127.0.0.1:8000/admin/reviews/contributor/` and select a contributor to delete. Make sure that the contributor is the author of a book.
2. Click the **Delete** button, but don't click **Yes, I'm sure** on the confirmation dialog. You will see a message like the one in *Figure 4.34*:

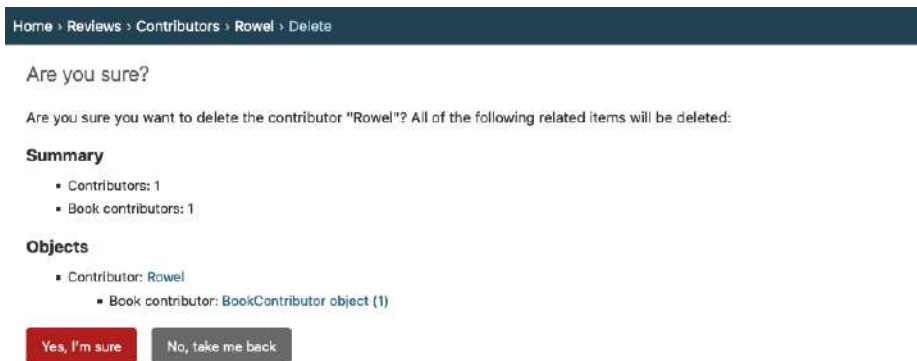


Figure 4.34 – Cascading delete confirmation dialog

In accordance with the `on_delete=CASCADE` argument to the foreign key, we are warned that deleting this `Contributor` object will have a cascading effect on a `BookContributor` object.

3. In the `reviews/models.py` file, modify the `Contributor` attribute of `BookContributor` to the following and save the file:

```
contributor = models.ForeignKey(Contributor,  
                                on_delete=models.PROTECT)
```

4. Now, try deleting the `Contributor` object again. You will see a message similar to the one in *Figure 4.35*:



Figure 4.35 – Foreign key protection error

Because the `on_delete` argument is `PROTECT`, our attempt to delete the object with dependencies will throw an error. If we used this approach in our model, we would need to delete objects in the `ForeignKey` relation before we deleted the original object. In this case, it would mean deleting the `BookContributor` object before deleting the `Contributor` object.

5. Now that we have learned about how the admin app handles the `ForeignKey` relations, let's revert the `ForeignKey` definition in the `BookContributor` class to the following:

```
contributor = models.ForeignKey(Contributor,  
                                on_delete=models.CASCADE)
```

We have examined how the admin app's behavior adapts to the `ForeignKey` constraints that are expressed in model definitions. If the `on_delete` behavior is set to `models.PROTECT`, the admin app returns an error explaining why a protected object is blocking the deletion. This functionality can come in handy while building real-world apps, as there is often a chance of a manual error inadvertently leading to the deletion of important records. In the next section, we will look at how we can customize our admin app interface for a smoother user experience.

Customizing the admin interface

When first developing an application, the convenience of the default admin interface is excellent for building a rapid prototype of the app. Indeed, for many simpler applications or projects that require minimal data maintenance, this default admin interface may be entirely adequate. However, as the application matures to the point of release, the admin interface will generally need to be customized to facilitate more intuitive use and to robustly control data, subject to user permissions. You might want to retain certain aspects of the default admin interface and, at the same time, make some tweaks to certain features to better suit your purposes. For example, you would want the **Publisher** list to show the complete names of the publishing houses instead of `Publisher(1)`, `Publisher(2)`, and so on. In addition to the aesthetic appeal, this makes it easier to use and navigate through the app.

In this section, we will be examining the `AdminSite` object and learning about some of its important properties that can be used to modify the global appearance of the admin app. Then, we will examine approaches to subclassing `AdminSite` within our project so that we can customize it from its default appearance. We will end by testing our knowledge of these techniques by creating a new project and customizing its admin interface.

Site-wide Django admin customizations

We saw in *Figure 4.2* a page titled **Log in | Django site admin** containing a **Django Administration** form. However, an administrative user of the Bookr application may be somewhat perplexed by all this Django jargon, and it would be very confusing and a recipe for error if they had to deal with multiple Django apps that all had identical admin apps. As a developer of an intuitive and user-friendly application, you should customize this. Global properties such as these are specified as attributes of the `AdminSite` object. The following table details some of the simplest customizations to improve the usability of your app's admin interface:

AdminSite Attribute	Base Value	Description
<code>site_title</code>	"Django site admin"	Populates the <code><title></code> tag on each page of the admin interface.
<code>site_header</code>	"Django administration"	Sets the header on the login form.

AdminSite Attribute	Base Value	Description
index_title	"Site administration"	Sets the heading on the admin index page (where the models are listed).
index_template	None	Provides the path to find the admin index template. If unset, the admin/index.html template is used.
app_index_template	None	Provides the path to find the app admin index template. If unset, the admin/app_index.html template is used.
login_template	None	Provides the path to find the login template. If unset, the admin/login.html template is used.
logout_template	None	Provides the path to find the logout template. If unset, the registration/logged_out.html template is used.
password_change_template	None	Provides the path to find the password change template. If unset, the registration/password_change_form.html template is used.

AdminSite Attribute	Base Value	Description
password_change_done_template	None	Provides the path to find the password change done template. If unset, the registration/password_change_done.html template is used.

Figure 4.36 – Important AdminSite attributes

If this seems a little abstract, it might help if we examine the properties in this table using the Django shell.

Examining the AdminSite object from the Python shell

Let's take a deeper look at the AdminSite class. We have already encountered an object of the AdminSite class. It is the `admin.site` object that we used in the *Registering the reviews model* section. If the development server is not running, start it now with the `runserver` subcommand as follows :

```
python manage.py runserver
```

We can examine the `admin.site` object by importing the `admin` app in the Django shell, using the `manage.py` script again:

```
python manage.py shell
>>> from django.contrib import admin
```

We can interactively examine the default values of `site_title`, `site_header`, and `index_title` and see that they match the expected values of 'Django site admin', 'Django administration', and 'Site administration' that we have already observed on the rendered web pages of the Django admin app:

```
>>> admin.site.site_title
'Django site admin'
>>> admin.site.site_header
'Django administration'
>>> admin.site.index_title
'Site administration'
```

The `AdminSite` class also specifies which forms and views are used to render the admin interface and determine its global behavior.

Now that we have examined the object's properties, we will attempt to subclass it so that we can customize them in our Django project.

Subclassing `AdminSite` — a first approach

We can make some modifications to the `reviews/admin.py` file. Instead of importing the `django.contrib.admin` module and using its `site` object, we will import `AdminSite`, subclass it, and instantiate our customized `admin_site` object. Consider the following code snippet. Here, `BookrAdminSite` is a subclass of `AdminSite` that contains custom values for `site_title`, `site_header`, and `index_title`; `admin_site` is an instance of `BookrAdminSite`, and we can use this instead of the default `admin.site` object to register our models. The `reviews/admin.py` file will look as follows:

```
from django.contrib.admin import AdminSite
from reviews.models import (Publisher, Contributor, Book,
                             BookContributor, Review)

class BookrAdminSite(AdminSite):
    title_header = 'Bookr Admin'
    site_header = 'Bookr administration'
    index_title = 'Bookr site admin'

admin_site = BookrAdminSite(name='bookr')

admin_site.register(Publisher)
admin_site.register(Contributor)
admin_site.register(Book)
admin_site.register(BookContributor)
admin_site.register(Review)
```

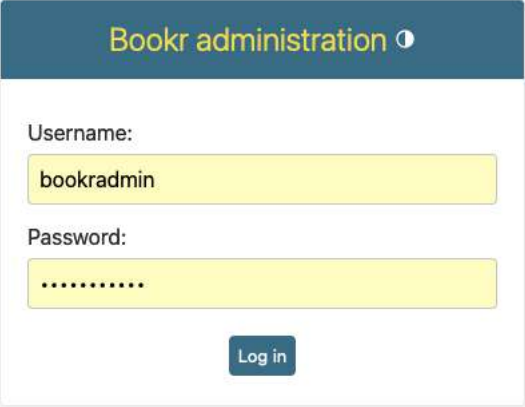
Now that we have created our own `admin_site` object that overrides the behavior of the `admin.site` object, we need to remove the existing references in our code to the `admin.site` object. In `bookr/bookr/urls.py`, we need to point `admin` to the new `admin_site` object and update our URL patterns. Otherwise, we would still be using the default admin site, and our customizations would be ignored. The change will look as follows:

```
from reviews.admin import admin_site
import reviews.views
```

```
from django.urls import include, path

urlpatterns = [
    path("admin/", admin_site.urls),
    path("book-search", reviews.views.book_search),
    path("", include("reviews.urls")),
]
```

This produces the expected results on the login screen:



The screenshot shows a web browser window displaying the 'Bookr administration' login page. The page has a dark blue header with the text 'Bookr administration' and a small circular icon. Below the header is a white box containing the login form. The form has two input fields: 'Username:' with the value 'bookradmin' and 'Password:' with a masked password represented by dots. A 'Log in' button is located at the bottom of the form.

Figure 4.37 – Customizing the login screen

However, now there is a problem; we have lost the interface for auth objects. Previously, the admin app discovered the models registered in `reviews/admin.py` and `django.contrib.auth.admin` through the auto-discovery process, but now we have overridden this behavior by creating a new `AdminSite`.



Figure 4.38 – Customized AdminSite is missing authentication and authorization

We could go down the path of referencing both the AdminSite objects to URL patterns in `bookr/bookr/urls.py`, but this approach would mean that we would end up with two separate admin apps for authentication and reviews. So, the URL `http://127.0.0.1:8000/admin` will take you to the original admin app derived from the `admin.site` object, while `http://127.0.0.1:8000/bookradmin` will take you to our `BookrAdminSite`, `admin_site`. This is not what we want to do, as we are still left with the admin app without the customizations that we added when we subclassed `BookrAdminSite`:

```
from django.contrib import admin
from reviews.admin import admin_site
from django.urls import path
urlpatterns = [path('admin/', admin.site.urls),
               path('bookradmin/', admin_site.urls),]
```

To avoid this problem, let's see what we can do in the next subsection.

Subclassing AdminSite – a better way

This has been a clumsy problem with the Django admin interface that has led to a lot of ad hoc solutions in earlier versions. However, since Django 2.1 came out, there has been a simple way of integrating a customized interface for the admin app without breaking auto-discovery or any of its other default features. Let's see how to do this with the following steps:

1. As BookrAdminSite is project-specific, the code does not really belong under our reviews folder. So, move BookrAdminSite to a new file, bookr/bookr/admin.py, at the top level of the Bookr project directory:

```
from django.contrib import admin

class BookrAdminSite(admin.AdminSite):
    title_header = 'Bookr Admin'
    site_header = 'Bookr administration'
    index_title = 'Bookr site admin'
```

The URL settings path in bookr/bookr/urls.py changes to path('admin/', admin.site.urls), and we define ReviewsAdminConfig.

2. Add a file called reviews/adminconfig.py containing these lines:

```
from django.contrib.admin.apps import AdminConfig

class ReviewsAdminConfig(AdminConfig):
    default_site = "admin.BookrAdminSite"
```

3. Replace django.contrib.admin with reviews.apps.ReviewsAdminConfig, so that INSTALLED_APPS in the bookr/bookr/settings.py file will look as follows:

```
INSTALLED_APPS = [
    'reviews.adminconfig.ReviewsAdminConfig',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'reviews']
```

With the ReviewsAdminConfig specification of default_site, we no longer need to replace references to admin.site with a custom AdminSite object, admin_site. We can

replace those `admin_site` calls with the `admin.site` calls that we had originally. Now, `reviews/admin.py` reverts to the following:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor,
                             Book, BookContributor, Review)

admin.site.register(Publisher)
admin.site.register(Contributor)
admin.site.register(Book, BookAdmin)
admin.site.register(BookContributor)
admin.site.register(Review)
```

There are other aspects of `AdminSite` that we can customize, but we will look at these in *Chapter 9*, once we have a fuller understanding of Django's templates and forms.

The following activity is aimed at testing the skills that you have learned to configure and customize the Django admin app for a new Django project.

Activity 4.02 – Customizing the AdminSite object

You have learned how to modify attributes of the `AdminSite` object in a Django project. This activity will challenge you to use these skills to customize a new project and override its site title, site header, and index header. Also, you will replace the logout message by creating a project-specific template and setting it in our custom `AdminSite` object. You are developing a Django project that implements a message board called *Comment8or*. *Comment8or* is geared toward a technical demographic, so you need to make the phraseology succinct and abbreviated:

- The *Comment8or* admin site will be referred to as `c8admin`. This will appear on the site header and index title.
- For the title header, it will say `c8 site admin`.
- The default Django admin logout message is `Thanks for spending some quality time with the Web site today. In Comment8or, it will say Bye from c8admin.`

These are the steps that you need to follow to complete this activity:

1. Following the process that you learned in *Chapter 1*, create a new Django project called `comment8or`, an app called `messageboard`, and run the migrations. Create a superuser called `c8admin`.

2. In the Django source code, there is a template for the logout page located in `django/contrib/admin/templates/registration/logged_out.html`. Make a copy of it in your project's directory under `bookr/comment8or/templates/comment8or` and modify the message in the template following the requirements.
 3. Inside the project, create an `admin.py` file that implements a custom `AdminSite` object. Set the appropriate values for the `index_title`, `title_header`, `site_header`, and `logout_template` attributes, based on the requirements.
 4. Add a custom `AdminConfig` subclass to `bookr/messageboard/adminconfig.py`.
 5. Replace the admin app with the custom `AdminConfig` subclass in `bookr/comment8or/settings.py`.
 6. Configure the `TEMPLATES` setting so that the project's template is discoverable.
- When the project was first created, the login page looked like this:

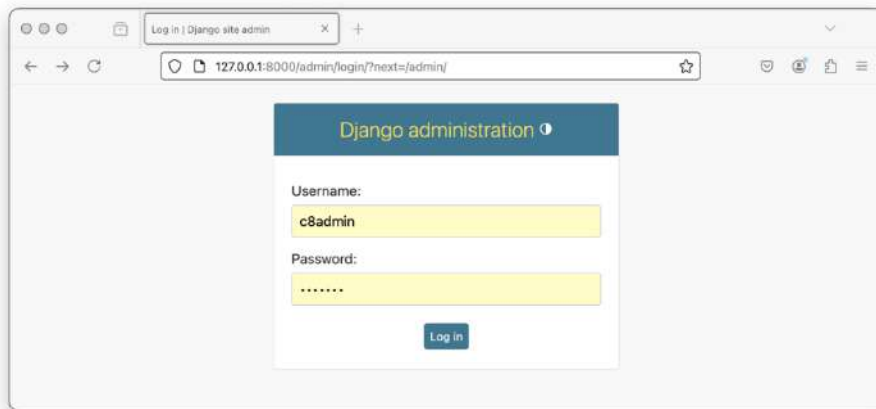


Figure 4.39 – Login page for the project

The app index page looked like this:

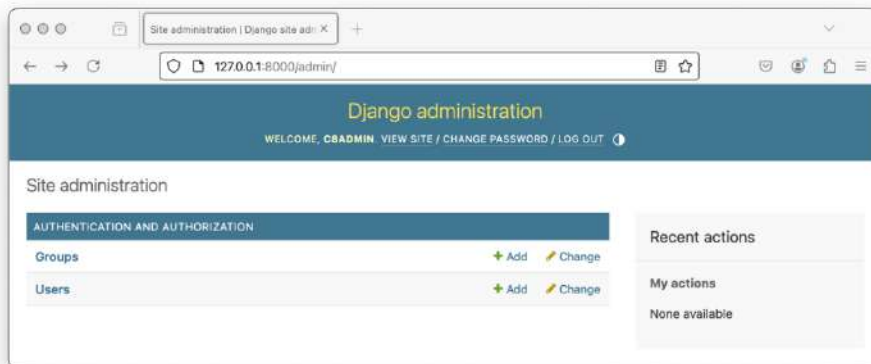


Figure 4.40 – App index page for the project

Finally, the logout page looked like this:

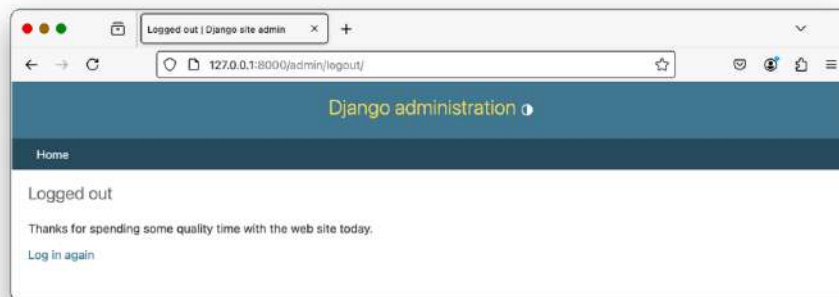


Figure 4.41 – Logout page for the project

After you have completed this activity with all the customizations, the login page will look like this:



Figure 4.42 – Login page after customization

The app index page will look like this:

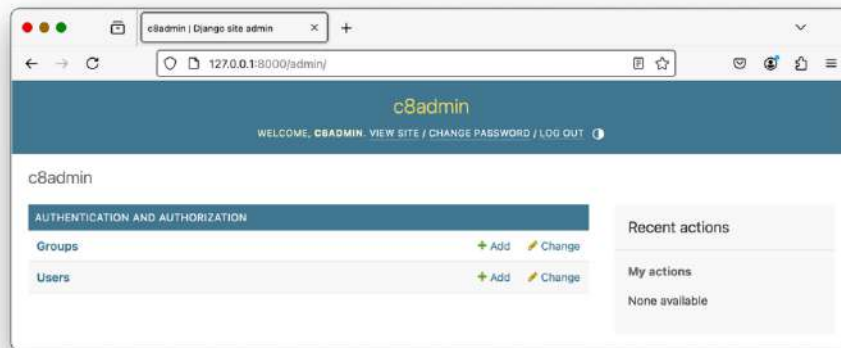


Figure 4.43 – App index page after customization

Finally, the logout page will look like this:



Figure 4.44 – Logout page after customization

You have successfully customized the admin app by subclassing `AdminSite`.

In the next section, we will examine the `AdminSite` object and learn about some of its important properties that can be used to modify the global appearance of the admin app. Then, we will examine approaches to subclassing `AdminSite` within our project so that we can customize it away from its default appearance. We will end by testing our knowledge of these techniques by creating a new project and customizing its admin interface.

Customizing the `ModelAdmin` classes

Now that we've learned how a subclassed `AdminSite` can be used to customize the global appearance of the admin app, we will look at how to customize the admin app's interface to individual models. Owing to the admin interface being generated automatically from the models' structure, it has an overly generic appearance and needs to be customized for the sake of aesthetics and usability. Click one of the **Books** links in the admin app and compare it to the **Users** link. Both links take you to the change list pages. These are the pages that a Bookr administrator visits when they want to add new books or add or alter the privileges of a user. As explained previously in the *Change lists* section, a change list page presents a list of model objects with the option of selecting a group of them for bulk deletion (or other bulk activity), examining an individual object to edit it, or adding a new object. Notice the difference between the two change list pages with a view to making our vanilla **Books** page as fully featured as the **Users** page. The following screenshot from the **Authentication and Authorization** app contains useful features such as a search bar, sortable column headers for important user fields, and a result filter:

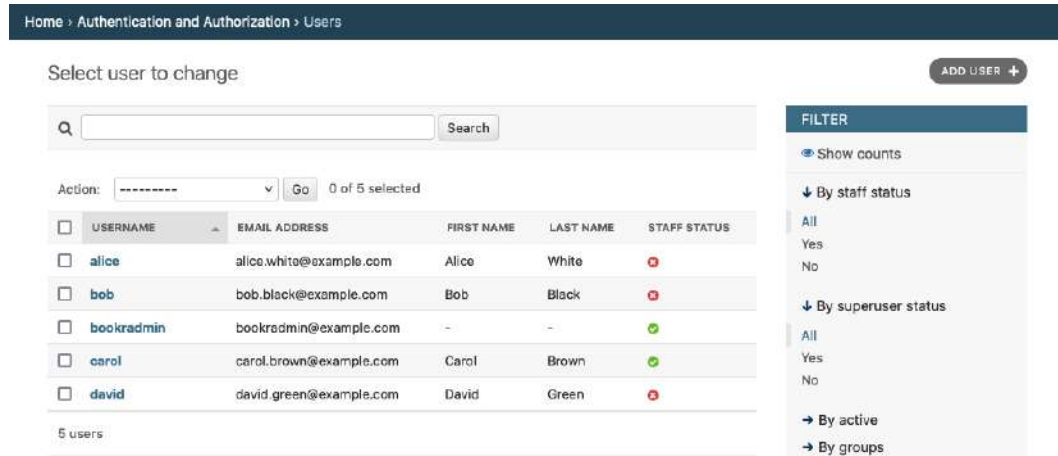


Figure 4.45 – The Users change list contains the customized ModelAdmin features

By subclassing `ModelAdmin`, we can create a class that sets the functionality for a specific model's admin pages.

In this section, we will learn how the `ModelAdmin` classes can be developed to customize the fields on the change list page and to implement filters and search bars. We will look at how model fields can be grouped in admin CRUD forms or be excluded from the form if the fields should retain a default or system-generated value.

The list display fields

On the **Users** change list page, you will see the following:

- There is a list of user objects presented, summarized by their **USERNAME**, **EMAIL ADDRESS**, **FIRST NAME**, **LAST NAME**, and **STAFF STATUS** attributes.
- These individual attributes are sortable. The sorting order can be changed by clicking the headers.
- There is a search bar at the top of the page.
- In the right-hand column, there is a selection filter that allows the selection of several user fields, including some that do not appear in the list display.

However, the behavior for the **Books** change list page is a lot less helpful. The books are listed by their titles but not in alphabetical order. The title column is not sortable, and there are no filter or search options present:

Home > Reviews > Books

Select book to change ADD BOOK +

Action: ----- Go 0 of 18 selected

- ☐ BOOK
- ☐ The Talisman
- ☐ Paul Clifford
- ☐ Animal Farm: A Fairy Story
- ☐ 1984
- ☐ Pride and Prejudice

Figure 4.46 – The Books change list

Recall from *Chapter 2*, that we defined the `__str__` methods on the `Publisher`, `Book`, and `Contributor` classes. In the case of the `Book` class, it had a `__str__()` representation that returns the book object's title:

```
class Book(models.Model):  
    ...  
    def __str__(self):  
        return self.title
```

If we had not defined the `__str__()` method on the `Book` class, it would have inherited it from the base `Model` class, `django.db.models.Model`.

This base class provides an abstract way to give a string representation of an object. For example, when we have `Book` with a primary key, in this case, the `id` field with a value of 17, then we will end up with a string representation of `Book object (17)`:

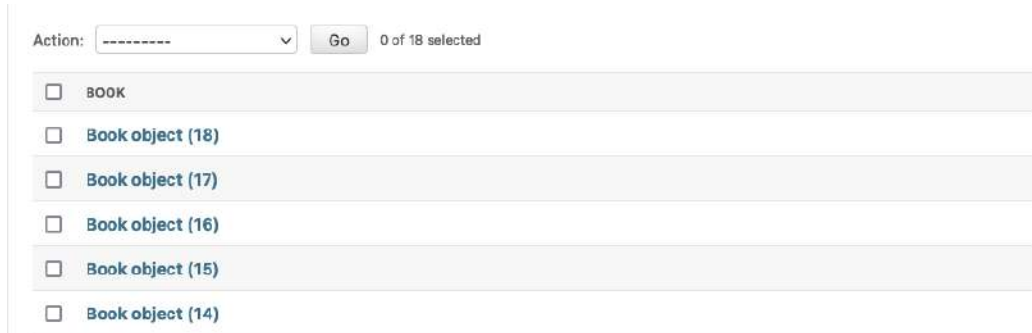


Figure 4.47 – The Books change list using the model `__str__` representation

It could be useful in our application to represent a Book object as a composite of several fields. For example, if we wanted the books to be represented as Title (ISBN), the following code snippet would produce the desired results:

```
class Book(models.Model):
    ...
    def __str__(self):
        return f"{self.title} ({self.isbn})"
```

This is a useful change in and of itself, as it makes the representation of the object more intuitive in the app:



Figure 4.48 – A portion of the Books change list with the custom string representation

We are not limited to using the `__str__` representation of the object in the `list_display` field. The columns that appear in the list display are determined by the `ModelAdmin` class of the

Django admin app. In the Django shell, we can import the `ModelAdmin` class and examine its `list_display` attribute:

```
python manage.py shell
>>> from django.contrib.admin import ModelAdmin
>>> ModelAdmin.list_display
('__str__',)
```

This explains why the default behavior of `list_display` is to display a single-column table of the objects' `__str__` representations so that we can customize the list display by overriding this value. The best practice is to subclass `ModelAdmin` for each object. If we wanted the book list display to contain two separate columns for **Title** and **ISBN**, rather than having a single column containing both values, as in *Figure 4.48*, we would subclass `ModelAdmin` as `BookAdmin` and specify the custom `list_display`. The benefit of doing this is that we are now able to sort books by title and by ISBN. We can add this class to `reviews/admin.py`:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ("title", "isbn")
```

Now that we've created a `BookAdmin` class, we should reference it when we register our `reviews.models.Book` class with the admin site. In the same file, we also need to modify the model registration to use `BookAdmin` instead of the default value of `admin.ModelAdmin`, so the `admin.site.register` call now becomes the following:

```
admin.site.register(Book, BookAdmin)
```

Once these two changes have been made to the `reviews/admin.py` file, we will get a **Books** change list page that looks like this:

Action: 0 of 18 selected

☐	TITLE	ISBN NUMBER OF THE BOOK.
☐	The Talisman	9781451697216
☐	Paul Clifford	9781719053167
☐	Animal Farm: A Fairy Story	9780151002177
☐	1984	9781328869333
☐	Pride and Prejudice	9780141439518

Figure 4.49 – A portion of the Books change list with a two-column list display

This gives us a hint as to how flexible `list_display` is. It can take four types of values:

- It takes field names from the model, such as `title` or `isbn`.
- It takes a function that takes the model instance as an argument, such as this function that gives an initialized version of a person's name:

```
def initialled_name(obj):
    """ obj.first_names='Jerome David',
    obj.last_names='Salinger'=> 'Salinger, JD'
    obj.first_names='Plato', obj.last_names=''
    => 'Plato'
    """
    initials = "".join([name[0] for name in
                        obj.first_names.split(" ")])
    if obj.last_names:
        return "{obj.last_names}, {initials}"
    return obj.first_names

class ContributorAdmin(admin.ModelAdmin):
    list_display = (initialled_name,)
```

- It takes a method from the `ModelAdmin` subclass that takes the model object as a single argument. Note that this needs to be specified as a string argument, as it would be out of scope and undefined within the class:

```
class BookAdmin(admin.ModelAdmin):

    list_display = ("title", "isbn13")
```

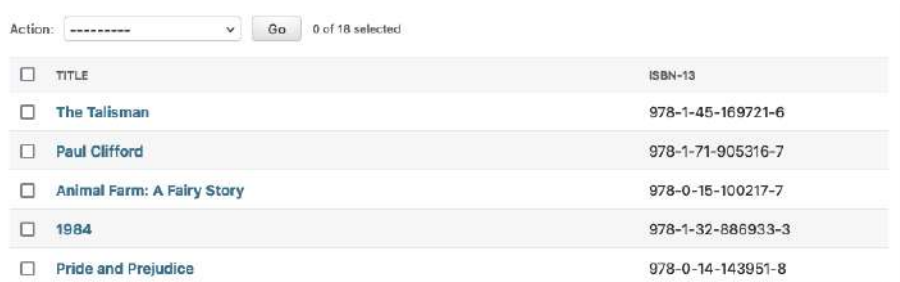
```
def isbn13(self, obj):
    """ '9780316769174' => '978-0-31-676917-4'
        '0316769174' => '0316769174'
        None => '' """
    if obj.isbn:
        if len(obj.isbn)==13:
            return "-".join([obj.isbn[0:3],
                             obj.isbn[3:4], obj.isbn[4:6],
                             obj.isbn[6:12], obj.isbn[12:13]])
        return obj.isbn
    return ""
```

- It takes a method (or a non-field attribute) of the model class, such as `__str__`, as long as it accepts the model object as an argument. For example, we could convert `isbn13` to a method on the `Book` model class:

```
class Book(models.Model):

    def isbn13(self):
        """ '9780316769174' => '978-0-31-676917-4'
            '0316769174' => '0316769174'
            None => '' """
        if self.isbn:
            if len(self.isbn)==13:
                return "-".join([self.isbn[0:3],
                                 self.isbn[3:4], self.isbn[4:6],
                                 self.isbn[6:12],self.isbn[12:13]])
            return self.isbn
        return ""
```

Now, when viewing the **Books** change list at <http://127.0.0.1:8000/admin/reviews/book>, we can see the hyphenated ISBN13 field:



☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8

Figure 4.50 – A portion of the Books change list with the hyphenated ISBN13

It is worth noting that computed fields such as `__str__` or our `isbn13` methods do not make for sortable fields on the summary page unless specified. Also, we cannot include fields of the `ManyToManyField` type in `list_display`.

While columns that are derived from model attributes derive their column headers and properties from the field attributes, we can specify properties of display columns that are computed fields using the display decorator.

The display decorator

When using a callable in `list_display`, as in the cases of `initialled_name` and `isbn13`, we can use the `admin.display` decorator to specify the column name that will appear in the header of the change list using the `description` argument. We can also use it to get around the limitation of calculated fields not being sortable by specifying `ordering` on the callable. The `empty_value` argument can be used to specify how a `None` value or an empty string is displayed. The default `empty_value` display is a single dash character:

```
@admin.display(
    ordering="isbn",
    description="ISBN-13",
    empty_value="-"
)
def isbn13(self, obj):
    """ '9780316769174' => '978-0-31-676917-4'
        '0316769174' => '0316769174'
        None => ' ' """
    ...
```


Filters are implemented using the `list_filter` attribute of a `ModelAdmin` subclass. In our Bookr app, filtering by book title or ISBN would be impractical, as it would produce a large list of filter options that return only one record. The filter that would occupy the right-hand side of the page would take up more space than the actual change list. A practical option would be to filter books by publisher. We defined a custom `__str__` method for the `Publisher` model that returns the publisher's name attribute, so our filter options will be listed as publisher names.

We can specify our change list filter in `reviews/admin.py` in the `BookAdmin` class like so:

```
list_filter = ("publisher",)
```

Here is how the **Books** change list page should look now:

The screenshot shows the Django Admin interface for the 'Books' model. At the top, there's a header 'Select book to change' and an 'ADD BOOK +' button. Below the header, there's an 'Action:' dropdown menu, a 'Go' button, and a status '0 of 6 selected'. The main content is a table with two columns: 'TITLE' and 'ISBN NUMBER OF THE BOOK.'. The table lists six books, each with a checkbox in the first column. To the right of the table, there's a 'FILTER' sidebar with a 'By publisher' section. This section has a list of publisher names: 'All', 'Packt Publishing', 'Harper Collins', 'Penguin Classics', 'Scribner', 'Bay Back Books', 'Simon and Schuster', 'Houghton Mifflin Harcourt', and 'HardPress'. The 'All' option is currently selected. At the bottom of the table, it says '6 books'.

☐	TITLE	ISBN NUMBER OF THE BOOK.
☐	Hands-On Reinforcement Learning with Python	9781788836524
☐	Natural Language Processing with TensorFlow	9781788478311
☐	Deep Reinforcement Learning Hands-On	9781788834247
☐	Architects of Intelligence	9781789954531
☐	Hands-On Machine Learning for Algorithmic Trading	9781789346411
☐	Advanced Deep Learning with Keras	9781788629416

6 books

Figure 4.52 – The Books change list page with the publisher filter

With that line of code, we have implemented a useful publisher filter on the **Books** change list page.

In order to consolidate our knowledge of filters, we will add further filters to the **Books** change list page.

Exercise 4.04 – Adding the date list_filter and date_hierarchy filters

We have seen that the `admin.ModelAdmin` class provides useful attributes to customize filters on change list pages. For example, filtering by date is a crucial functionality for many applications and can also help us make our app more user-friendly. In this exercise, we will examine how date filtering can be implemented by including a date field in the filter and look at the `date_hierarchy` filter:

1. Edit the `reviews/admin.py` file and modify the `list_filter` attribute in the `BookAdmin` class to include `'publication_date'`:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ("title", "isbn13")
    list_filter = ("publisher", "publication_date")
```

2. Reload the **Books** change list page and confirm that the filter now includes date settings:

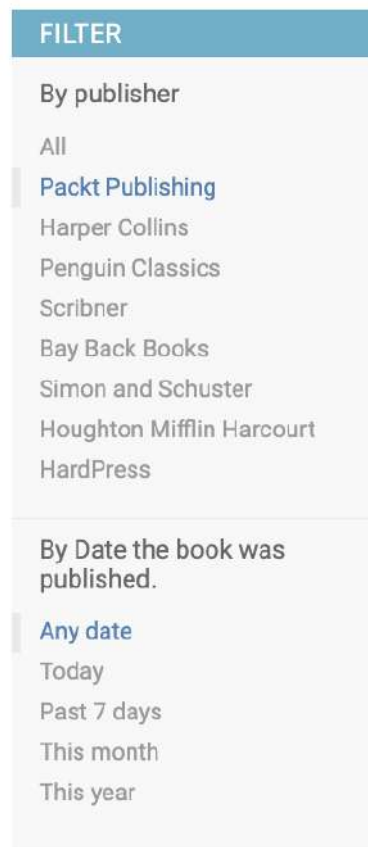


Figure 4.53 – Confirming that the Books change list page includes date settings

This publication date filter would be convenient if the Bookr project were receiving a lot of new releases, and we wanted to filter books by what was published in the last seven days or a month. Sometimes, though, we might like to filter by a specific year or a specific month in a specific year. Fortunately, the `admin.ModelAdmin` class comes with a custom filter attribute geared toward navigating hierarchies of temporal information. It is called `date_hierarchy`.

3. Add a `date_hierarchy` attribute to `BookAdmin` and set its value to `publication_date`:

```
class BookAdmin(admin.ModelAdmin):  
    date_hierarchy = "publication_date"  
    list_display = ("title", "isbn13")  
    list_filter = ("publisher", "publication_date")
```


4. Reload the **Books** change list page and confirm that the date hierarchy appears above the **Action** drop-down menu:

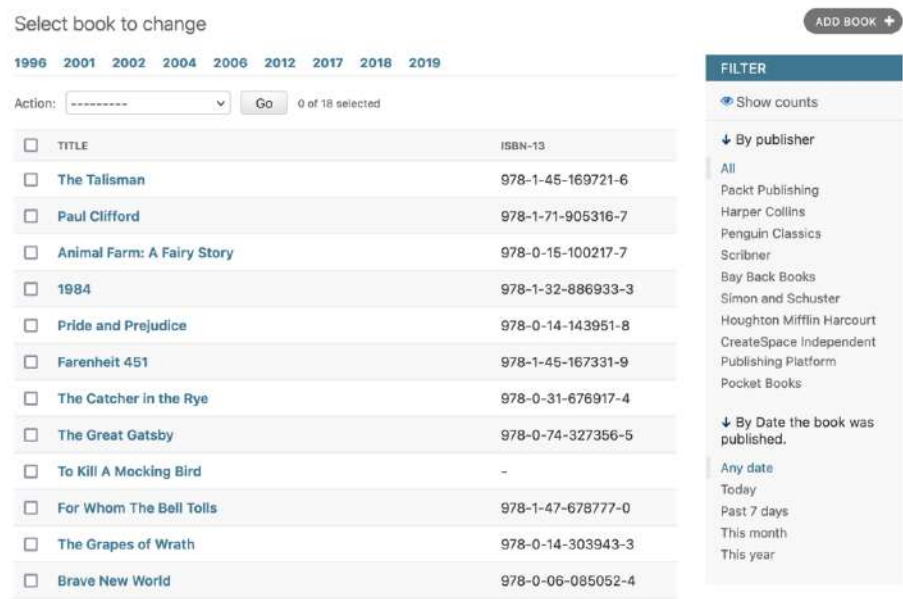


Figure 4.54 – Confirming that the date hierarchy appears above the Action drop-down menu

5. Select a year from the date hierarchy and confirm that it contains a list of months in that year containing book titles and a total list of books:

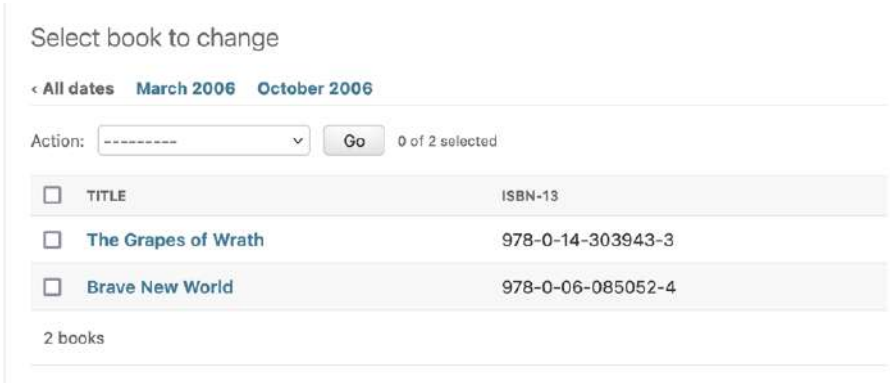


Figure 4.55 – Confirming that the selection of a year from the date hierarchy shows the books published that year

6. Confirm that selecting one of these months further filters down to days in the month:

Select book to change

< 2006 **March 28**

Action: 0 of 1 selected

☐	TITLE	ISBN-13
☐	The Grapes of Wrath	978-0-14-303943-3

1 book

Figure 4.56 – Filtering months down to days in the month

The `date_hierarchy` filter is a convenient way of customizing a change list that contains a large set of time-sortable data in order to facilitate faster record selection, as we saw in this exercise.

Let's now look at the implementation of a search bar in our app.

The search bar

This brings us to the remaining piece of functionality that we wanted to implement – the search bar. Like filters, a basic search bar is quite simple to implement. We only need to add the `search_fields` attribute to the `ModelAdmin` class. The obvious character fields in our `Book` class to search on are `title` and `isbn`. At present, the **Books** change list appears with a date hierarchy across the top of the change list. The search bar will appear above this:

Select book to change

ADD BOOK +

Q Search

1996 2001 2002 2004 2006 2012 2017 2018 2019

Action: Go 0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	-
☐	For Whom The Bell Tolls	978-1-47-678777-0
☐	The Grapes of Wrath	978-0-14-303943-3

FILTER

Show counts

↓ By publisher

All

- Packt Publishing
- Harper Collins
- Penguin Classics
- Scribner
- Bay Back Books
- Simon and Schuster
- Houghton Mifflin Harcourt
- CreateSpace Independent Publishing Platform
- Pocket Books

↓ By Date the book was published.

Any date

- Today
- Past 7 days
- This month
- This year

Figure 4.57 – The Books change list before the search bar is added

We can start by adding this attribute to BookAdmin in reviews/admin.py and examining the result:

```
search_fields = ("title", "isbn")
```

The result would look like this:

Select book to change

ADD BOOK +

Q Search

1996 2001 2002 2004 2006 2012 2017 2018 2019

Action: Go 0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	978-0-06-093546-7
☐	For Whom The Bell Tolls	978-1-47-678777-0

FILTER

Show counts

By publisher

- All
- Packt Publishing
- Harper Collins
- Penguin Classics
- Scribner
- Bay Back Books
- Simon and Schuster
- Houghton Mifflin Harcourt
- CreateSpace Independent Publishing Platform
- Pocket Books

By Date the book was published.

- Any date
- Today
- Past 7 days
- This month
- This year

Figure 4.58 – The Books change list with the search bar

Now, we can perform a simple text search on fields that match the title field or ISBN. This search requires precise string matches, so "color" won't match "colour." It also lacks the deep semantic processing that we expect from more sophisticated search facilities such as **Elasticsearch**. ISBN lookup is a very good feature if you happen to have a barcode scanner. Limiting our search to fields on the Books model is quite restrictive. We might want to search by publisher name, too. Fortunately, `search_fields` is flexible enough to accomplish this. To search on `ForeignKeyField` or `ManyToManyField`, we just need to specify the field name on the current model and the field on the related model, separated by two underscores. In this case, Book has a foreign key, `publisher`, and we want to search on the `Publisher.name` field so it can be specified as `'publisher__name'` on `BookAdmin.search_fields`:

```
search_fields = ("title", "isbn", "publisher__name")
```

If we wanted to restrict a search field to an exact match rather than return results that contain the search string, then the field can be suffixed with `'__exact'`. So, replacing `'isbn'` with `'isbn__exact'` will require the complete ISBN to be matched, and we won't be able to get a match using a portion of the ISBN.

Similarly, we constrain the search field to only return results that start with the search string by using the '`__startswith`' suffix. Qualifying the publisher name search field as '`publisher__name__startswith`' means that we will get results searching for "pack" but not for "ackt."

This concludes our examination of common customizations of the change list pages. We may also want to customize the behavior of the create and update forms in our app.

Excluding and grouping fields

There are occasions when it is appropriate to restrict the visibility of some of the fields in the model in the admin interface. This can be achieved with the `exclude` attribute.

This is the review form screen with the **Date edited** field visible. Note that the **Date created** field does not appear – it is already a hidden view because `date_created` is defined on the model with the `auto_now_add` parameter:

Add review

Content:
The Review text.

Rating:
The rating the reviewer has given.

Date edited: Date: Today | 📅
 Time: Now | ⌚
Note: You are 10 hours ahead of server time.
 The date and time the review was last edited.

Creator: ✎ + 👁

Book: ✎ + 👁
The Book that this review is for.

Figure 4.59 – The review form

If we wanted to exclude the **Date edited** field from the review form, we would do this in the `ReviewAdmin` class:

```
exclude = ("date_edited",)
```

Then, the review form would appear without **Date edited**:

Add review

Content:
The Review text.

Rating:
The rating the reviewer has given.

Creator:   

Book:   
The Book that this review is for.

Figure 4.60 – The review form with the Date edited field excluded

Conversely, it might be more prudent to restrict the admin fields to those that have been explicitly permitted. This is achieved with the `fields` attribute. The advantage of this approach is that if new fields are added in the model, they won't be available in the admin form unless they have been added to the `fields` tuple in the `ModelAdmin` subclass:

```
fields = ("content", "rating", "creator", "book")
```

This will give us the same result that we saw earlier.

Another option is to use the `fieldsets` attribute of the `ModelAdmin` subclass to specify the form layout as a series of grouped fields. Each grouping in `fieldsets` consists of a title followed by a dictionary containing a `"fields"` key pointing to a list of field name strings:

```
fieldsets = (
    ("Linkage", {"fields": ("creator", "book")}),
    ("Review content", {"fields": ("content", "rating")}),
)
```

The review form should look as follows:

Add review

Linkage

Creator:   

Book:   
The Book that this review is for.

Review content

Content:
The Review text.

Rating:
The rating the reviewer has given.

SAVE

Save and add another

Save and continue editing

Figure 4.61 – The review form with fieldsets

If we want to omit the title on a fieldset, we can do so by assigning the None value to it:

```
fieldsets = (  
    (None, {"fields": ("creator", "book")}),  
    ("Review content", {"fields": ("content", "rating")}),  
)
```

Now, the review form should appear as shown in the following screenshot:

Add review

Creator:   

Book:   
The Book that this review is for.

Review content

Content: 
The Review text.

Rating: 
The rating the reviewer has given.

Figure 4.62 – The review form with the first fieldset untitled

Now that we have learned about change lists and form customizations, we can put our knowledge into practice with a comprehensive activity.

Activity 4.03 – Customizing the model admins

In our data model, the Contributor class is used to store data for book contributors; they can be authors, contributors, or editors. This activity focuses on modifying the Contributor class and adding a ContributorAdmin class to improve the user-friendliness of the admin app. At present, the Contributor change list defaults to a single column, FirstNames, based on the `__str__` method created in *Chapter 2*. We will investigate some alternative ways of representing this. These steps will help you complete the activity:

1. Edit `reviews/models.py` to add additional functionality to the Contributor model.
2. Add an `initialled_name` method to Contributor that takes no arguments (like the `Book.isbn13` method).
3. The `initialled_name` method will return a string containing Contributor.`last_names` followed by a comma and the initials of the given names. For example, for a Contributor object with `first_names` of Jerome David and `last_names` of Salinger, `initialled_name` will return Salinger, JD. If the

`Contributor.last_names` is blank, then `Contributor.first_names` will be returned. Here is an example:

```
>>> Contributor(first_names="Plato",
                 last_names="").initialled_name()
'Plato'
```

4. Replace the `__str__` method for `Contributor` with one that calls `initialled_name()`. At this point, the **Contributors** display list will look like this:

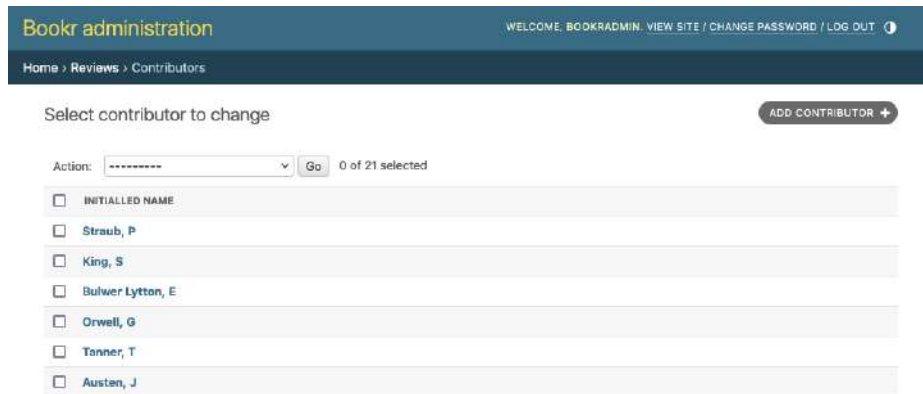


Figure 4.63 – The Contributors display list

5. Edit the `reviews/admin.py` file. We will modify `ContributorAdmin`, which was subclassed from `admin.ModelAdmin` in the *The list display fields* section.
6. Modify it so that on the Contributors change list, records are displayed with two sortable columns: Last Names and First Names. We can remove the existing `list_display` attribute from `ContributorAdmin` along with the `initialled_name` function.
7. Add a search bar that searches on Last Names and First Names. Modify it so that the Last Names search matches the start of Last Names.
8. Add a filter on Last Names.

By completing the activity, you should see something like this:

The screenshot shows the Django Admin interface for 'Bookr administration'. The top navigation bar includes 'Home > Reviews > Contributors' and a 'WELCOME, BOOKRADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT' link. A 'Select contributor to change' header is present above the main content area. On the right, there is an 'ADD CONTRIBUTOR +' button.

The main content area features a search bar with a magnifying glass icon and a 'Search' button. Below the search bar is an 'Action:' dropdown menu set to '-----' and a 'Go' button, followed by the text '0 of 21 selected'.

The central table lists 21 contributors with checkboxes in the first column. The table has two columns: 'LAST NAMES' and 'FIRST NAMES'. The contributors listed are: Straub, King, Bulwer Lytton, Orwell, Tanner, Austen, Bradbury, Salinger, Fitzgerald, Lee, Hemingway, Steinbeck, Huxley, Ravichandiran, Ganegedara, Lapan, Ford, Stefan, Editor Example, Example Editor, and Atienza.

On the right side, there is a 'FILTER' sidebar. It includes a 'Show counts' link and a 'By last names' dropdown menu. Below the dropdown is a list of names: All, Atienza, Austen, Bradbury, Bulwer Lytton, Editor Example, Example Editor, Fitzgerald, Ford, Ganegedara, Hemingway, Huxley, King, Lapan, Lee, Orwell, Ravichandiran, Salinger, Stefan, Steinbeck, Straub, and Tanner.

At the bottom of the table, it says '21 contributors'.

Figure 4.64 – Expected output

Changes such as these can be made to improve the functionality of the admin user interface. By implementing the First Names and Last Names columns as separate columns in the Contributors change list, we are giving the user an option to sort on either of the fields. By considering what columns are most useful in search retrieval and filter selections, we can improve the efficient retrieval of records.

Summary

In this chapter, we saw how to create superusers through the Django command line and how to use them to access the admin app. Then, after a brief tour of the admin app's basic functionality, we examined how to register our models with it to produce a CRUD interface for our data.

Then, we learned how to refine this interface by modifying site-wide features. We altered how the admin app presents model data to the user by registering custom model admin classes with the admin site. This allowed us to make fine-grained changes to the representation of our models' interfaces. These modifications included customizing change list pages by adding additional columns, filters, date hierarchies, and search bars. We also modified the layout of the model admin pages by grouping and excluding fields.

This was only a very shallow dive into the functionality of the admin app. We will revisit the rich functionality of `AdminSite` and `ModelAdmin` in *Chapter 10*. But first, we need to learn some more intermediate features of Django. In the next chapter, we will learn how to organize and serve static content, such as CSS, JavaScript, and images, from a Django app.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

5

Serving Static Files

A web application with just plain **HyperText Markup Language (HTML)** is quite limiting. However, we can enhance the look of web pages with **Cascading Style Sheets (CSS)** and images, and add interaction with JavaScript. We call all these kinds of files "static files," which are developed and then deployed as part of the application. We can compare this to dynamic responses, which are generated in real time when a request is made. All the views you have written generate a dynamic response by rendering a template.

Note

We will not consider templates to be static files as they are not sent verbatim to a client; instead, they are rendered first and sent as part of a dynamic response.

During development, the static files are created on the developer's machine, and then must be moved to the production web server. If you must move an application to production in a short timeframe (say, a few hours), then it can be time-consuming to collect all the static assets, move them to the correct directory, and upload them to the server. When developing web applications using other frameworks or languages, you might need to manually put all of your static files into a specific directory that your web server hosts. Making changes to the URL from which static files are served might mean updating values throughout your code.

Django can manage static assets for us to make this process easier. It provides tools for serving them with its development server during development. When your application goes to production, it can also collect all your assets and copy them to a folder for a dedicated web server to host. This allows you to keep your static files segregated in a meaningful way during development, and automatically bundle them for deployment.

This functionality is provided by Django's built-in `staticfiles` app. It adds several useful features for working with and serving static files:

- The `static` template tag automatically builds the static URL for an asset and includes it in your HTML.
- A view (called `static`) serves static files in development.
- Static file finders can be used to customize where assets are found on your filesystem.
- The `collectstatic` management command finds all static files and moves them into a single directory for deployment.
- The `findstatic` management command shows which static file on the disk has been loaded for a particular request. This also helps us debug if a particular file is not being loaded.

In the exercises and activities of this chapter, we will be adding static files (images and CSS) to the Bookr application. Each file will be stored inside the Bookr project directory during development. We will need to generate a URL for each so that the templates can reference them and the browser can download them. Once the URL has been generated, Django will need to serve these files. When we deploy the Bookr application to production, all the static files need to be found and moved to a directory where they can be served by the production web server. If there are static files that are not loading as expected, we need a method of determining what the cause is.

In this chapter, you will start by learning the difference between static and dynamic responses. You will then see how the Django `staticfiles` app helps manage static files. As you continue to work on the Bookr app, you will enhance it with images and CSS. You'll see the different ways you can lay out your static files for your project and examine how Django consolidates them for production deployment. Django includes tools for referencing static files in templates; you'll see how these tools help reduce the amount of work you must do when deploying the application to production. After this, you'll explore the `findstatic` command, which can be used to debug issues with your static files. Later, you'll get an overview of how to write code for storing static files on a remote service. Finally, you'll look at caching web assets and how Django can help with cache invalidation.

We will cover the following main topics in this chapter:

- Static file serving
- Introduction to static files finders
- Generating static URLs with the static template tag
- FileSystem Finder

- Static file finders – use during collectstatic
- STATICFILES_DIRS prefixed mode
- The findstatic command
- Serving the latest files (for cache validation)
- Custom storage engines

Technical requirements

Find the solution in the Chapter05 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Django and static file serving

In the introduction, we mentioned that Django includes a view function called `static` that serves static files. The first important point to make is that *Django does not intend to serve static files in production*. It is not Django's role, and in production, this is better suited to the web server. If Django is reading from the filesystem and sending out a file, then it has no advantage over a normal web server, which will probably be more performant at this task. Further, if you serve static files with Django, you will keep the Python process busy for the duration of this request, and it will be unable to serve the dynamic requests for which it is more suited.

For these reasons, the Django `static` view is designed only for use during development and will not work if your `DEBUG` setting is `False`. Since, during development, we usually only have one person accessing the site at a time (the developer), then Django is fine to serve static files. We will discuss how the `staticfiles` app supports production deployment more shortly. Plus, the entire production deployment process will be covered in *Chapter 18, Deploying Django*.

A URL mapping to the `static` view is automatically set up when running the Django development server, provided that your `settings.py` file consists of the following elements:

- Has `DEBUG` set to `True`
- Contains `"django.contrib.staticfiles"` in its `INSTALLED_APPS`

Both of these settings are configured on a newly created app.

The URL mapping that is created is roughly equivalent to having the following map in your `urlpatterns`:

```
path(settings.STATIC_URL, django.conf.urls.static)
```

Any URL starting with `settings.STATIC_URL` (which is `/static/` by default) gets mapped to the static view.

Note

You could use the static view without having staticfiles in `INSTALLED_APPS`, but you must set up an equivalent URL mapping manually.

Next, we'll talk about how Django locates static files using Static Files Finder.

Introduction to static files finders

Static files finders can be thought of as a plugin component of the Django architecture. They are classes that implement methods for converting URL paths into file paths, iterating through the project directory to find static files. There are three situations when Django needs to locate static files on disk, using static file finders to do so:

- The first is when the Django static view receives a request to load a particular static file; here, it needs to convert the path in the URL into a location on disk. For example, let's say the URL's path is `/static/logo.png`, and it is converted into the `bookr/static/logo.png` path on the disk. As we noted in the previous section, this only happens during development. On a production server, Django should not receive this request as it will be handled directly by the web server.
- The second is when using the `collectstatic` management command. This gathers all the static files in the project directory and copies them to a single directory to be served by the production web server. `bookr/static/logo.png` will get copied to the web server root – for example, `/var/www/bookr/static/logo.png`. The static file finder contains code for locating all the static files inside your project directory.
- The third use of static file finders is while executing the `findstatic` management command. This is similar to the first usage, where it accepts a static file name (such as `logo.png`), but it outputs the full path (`bookr/static/logo.png`) to the terminal instead of loading the file's content.

Django comes with some built-in finders, but you can also write your own if you want to store static files in a custom directory layout. The list of finders Django uses is defined by the `STATICFILES_FINDERS` setting in `settings.py`. In this chapter, we will cover the behavior of the default static file finders, `AppDirectoriesFinder` and `FileSystemFinder`, in the *AppDirectoriesFinder* subsection and the *FileSystemFinder* section, respectively.

Note

If you look in `settings.py`, you won't see the `STATICFILES_FINDERS` setting defined by default. This is because Django will use its built-in default for the setting, which is defined as `["django.contrib.staticfiles.finders.FileSystemFinder", "django.contrib.staticfiles.finders.AppDirectoriesFinder"]`. If you add the `STATICFILES_FINDERS` setting to your `settings.py` file to include a custom finder, be sure to include these defaults if you're using them.

In this section, we will first discuss static file finders and their use in the first case – responding to a request. Then, we will introduce some more concepts and return to the behavior of `collectstatic` and how it uses static file finders.

Static file finders – used during a request

When Django receives a request for a static file (remember, Django will only serve static files during development), each static file finder that has been defined will be queried until a file on the disk has been found. Or, if none of the finders can locate a file, the static view will return an HTTP 404 Not Found response.

For example, the URL of the request will be something such as `/static/main.css` or `/static/reviews/logo.png`. Each finder will be queried in turn with the path from the URL and will return a path such as `bookr/static/main.css` for the first file and `bookr/reviews/static/reviews/logo.png` for the second.

Each finder will use its logic to convert from a URL path into a filesystem path – we will discuss this logic in the *AppDirectoriesFinder* section (which comes next) and the *FileSystemFinder* section (which will come later).

AppDirectoriesFinder

The `AppDirectoriesFinder` class is used to find static files inside each app directory, in a directory called `static`. The application must be listed in the `INSTALLED_APPS` setting in your `settings.py` file (we did this in *Chapter 1, An Introduction to Django*). As we also mentioned in *Chapter 1, An Introduction to Django*, it is good for apps to be self-contained. By letting each application have a static directory, we can continue the self-contained design by also storing app-specific static files inside the app directory.

Before we use `AppDirectoriesFinder`, we will explain a problem that can occur if multiple static files have the same name, and also how to solve this problem.

Static file namespacing

In the prior section, *Static file finders – used during a request*, we discussed serving a file named `logo.png`. This would provide a logo for the reviews application. The filename (`logo.png`) could be quite common – you could imagine that if we added a store app (for purchasing books), it would also have a logo. Not to mention that third-party Django apps might also want to use a common name, such as `logo.png`. The problem we are about to describe could apply to any static file that has a common name, such as `styles.css` or `main.js`.

Let's consider the reviews and stores examples. We can add a static directory in each of these apps. Then, each static directory would have a `logo.png` file (although it would be a different logo). The directory structure is shown in *Figure 5.1*:

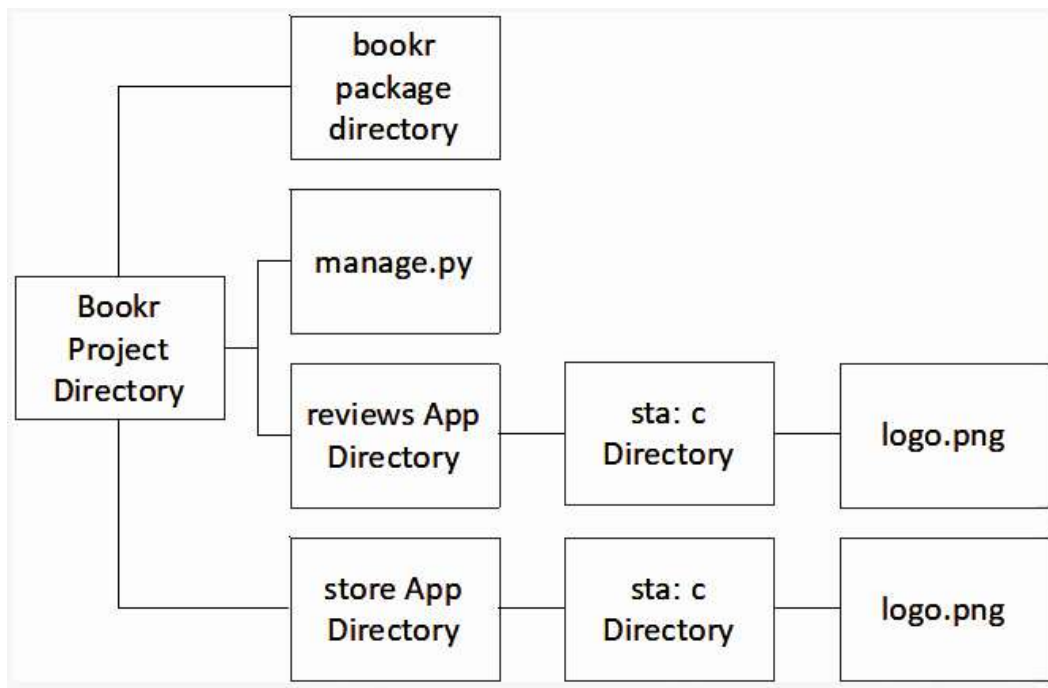


Figure 5.1 – Directory layout with static directories inside app directories

The URL path that we use to download the static file is relative to the static directory. Therefore, it is unclear which `logo.png` is being referenced if we make an HTTP request for `/static/logo.png`. Django will check the static directory for each application in turn (in the order they are specified in the `INSTALLED_APPS` setting). The first `logo.png` that it locates, it will serve. There is no way, in this directory layout, to specify which `logo.png` you want to load.

We can solve this problem by **namespacing** our static files. This is the process of using another directory inside the static directory, named the same as the app. The reviews app has a reviews directory inside its static directory, and the store app has a store directory inside its static directory. The respective `logo.png` files are then moved inside these sub-directories.

The new directory layout is shown in *Figure 5.2*:

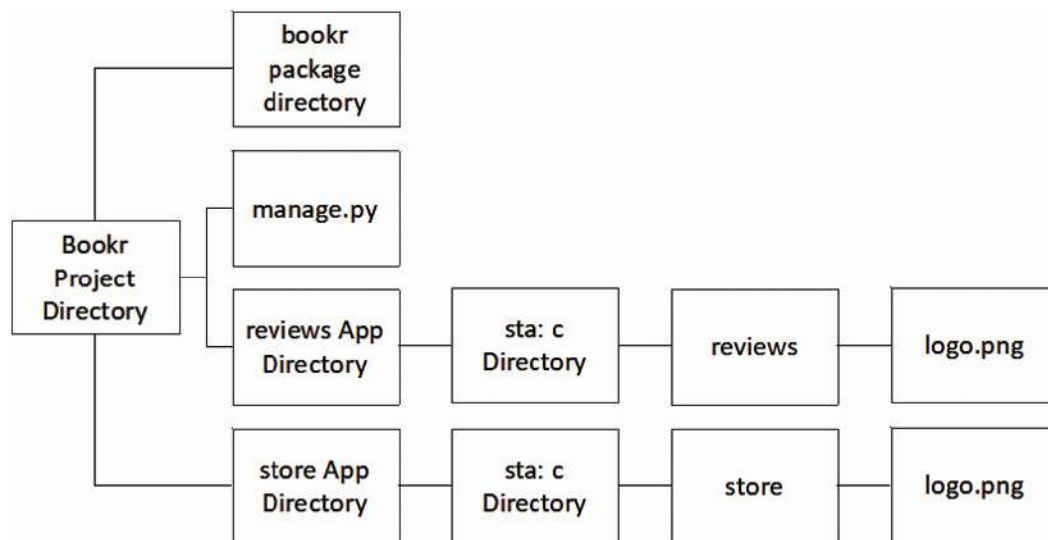


Figure 5.2 – Directory layout with namespaced directories

To load a specific file, we must include the namespaced directory too. For the reviews logo, the URL path is `/static/reviews/logo.png`, which maps to `bookr/reviews/static/reviews/logo.png` on the disk. Similarly, for the store logo, the path is `/static/store/logo.png`, which maps to `bookr/store/static/store/logo.png`. You might have noticed that the example paths for the `logo.png` file were already namespaced in the *Static file finders – used during a request* section.

Now that we have introduced `AppDirectoriesFinder` and static file namespacing, we can use them to serve our first static file. In the first exercise of this chapter, we will create a new Django project for a basic business site. We will then serve a logo file from an app called `landing` that we will create in this project. The `AppDirectoriesFinder` class is used to find static files inside each app directory, in a directory called `static`. The application must be listed in the `INSTALLED_APPS` setting in your `settings.py` file.

As we mentioned in *Chapter 1, An Introduction to Django*, it is good for apps to be self-contained. By letting each application have a `static` directory, we can continue the self-contained design by also storing app-specific static files inside the app directory.

The easiest way to serve a static file is from an app directory. This is because we don't need to make any settings changes. Instead, we just need to create the files in the correct directory, and they will be served using the default Django configuration.

Note

The business site project

For the exercises in this chapter, we'll create a new Django project and use it to demonstrate the static file concepts. The project will be a basic business site with a simple landing page that has a logo. The project will have one app, called *landing*.

You can refer to *Exercise 1.01 – Creating a project and app, and starting the dev server* from *Chapter 1, An Introduction to Django*, to refresh your memory on creating a Django project.

For detailed instructions on how to set up and configure PyCharm to work with your Django project, refer to *Exercise 1.02 – project setup in PyCharm*, in *Chapter 1, An Introduction to Django*.

Exercise 5.01 – serving a file from an app directory

In this exercise, you will add a logo file for the *landing* app. This will be done by putting a `logo.png` file in a `static` directory inside the *landing* app directory. Once you've done this, you can test that the static file is being served correctly and confirm the URL that will serve it:

1. Start by creating the new Django project. You can reuse the Bookr virtual environment that already has Django installed.
2. Open a new terminal and activate the virtual environment (refer to the *Preface* for instructions on how to create and activate a virtual environment).
3. Run the `django-admin` command in the terminal (or Command Prompt) to start a Django project named `business_site`.

```
django-admin startproject business_site
```

There won't be any output. This command will scaffold the Django project in a new directory named `business_site`.

4. Create a new Django app in this project by using the startapp management command. The app should be called *landing*. To do this, cd into the `business_site` directory, then run the following:

```
python manage.py startapp landing
```

Note that there won't be any output again. The command will create the `landing` app directory inside the `business_site` directory.

5. Launch PyCharm and open the `business_site` directory:
 - If you already have a project open, you can do this by clicking **File | Open**.
 - Otherwise, just click **Open** in the **Welcome to PyCharm** window. Navigate to the `business_site` directory, select it, then click **Open**.
 - If prompted, choose **Trust Project**.

The `business_site` project window should look similar to what's shown in *Figure 5.3*. Click the dropdown that currently says **Current File** and choose **Edit Configurations...**

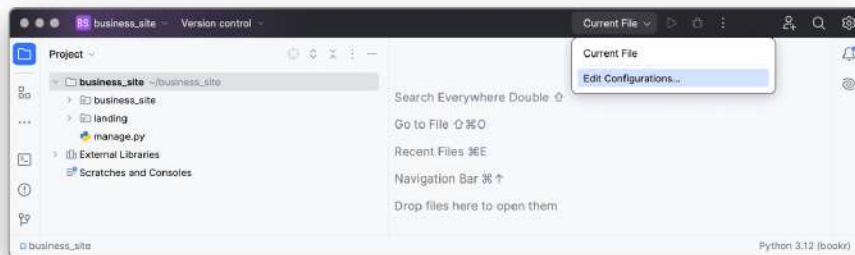


Figure 5.3 – The `business_site` project

6. Then follow the **Add new run Configuration...** link.

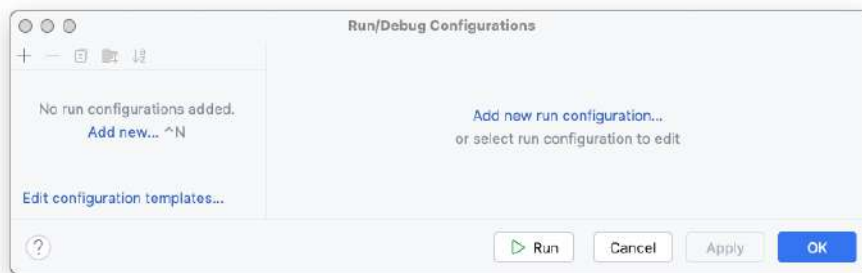


Figure 5.4 – Run/Debug Configurations

7. Create a new run configuration to execute `manage.py runserver` for the project. You can reuse the Bookr virtual environment again. The **Run/Debug Configurations** window should look similar to what's shown in Figure 5.5 when you're done:

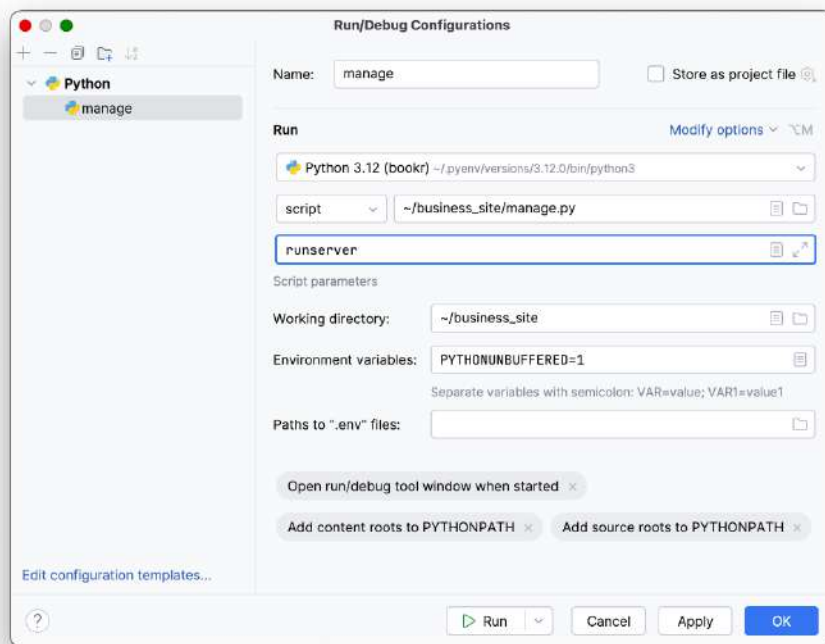


Figure 5.5 – Run/Debug Configurations for runserver

You can test that the configuration has been set up correctly by clicking the **Run** button and then visiting `http://127.0.0.1:8000/` in your browser. You should see the Django

welcome screen. If the debug server fails to start or you see the Bookr main page, then you probably still have the Bookr project running. Try stopping the Bookr runserver process (press *Ctrl* + *C* in the terminal that's running it) and then starting the new one you just set up.

Now that we have completed the project configuration in PyCharm, we will begin the coding steps for the exercise.

1. Open `settings.py` in the `business_site` directory and add 'landing' to the `INSTALLED_APPS` setting. We learned how to do this in *Step 1 of Exercise 1.05 – creating a templates directory and base template*, in *Chapter 1, An Introduction to Django*.
2. In PyCharm, right-click on the `landing` directory in the project panel and select **New | Directory**.
3. Enter `static` and press the *Enter* key:



Figure 5.6 – Naming the directory `static`

4. Right-click on the `static` directory you just created and select **New | Directory** again.
5. Enter `landing` and press *Enter*. This will implement namespacing of the static files directory, as we discussed earlier:

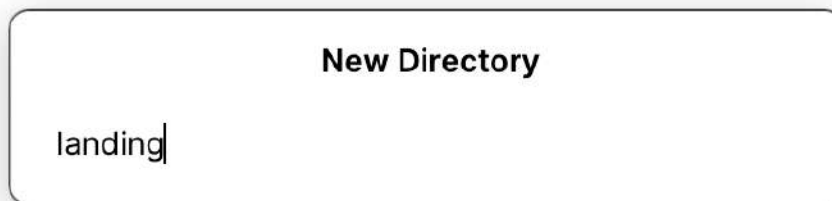


Figure 5.7 – Naming the new directory `reviews` to implement namespacing

6. Download the `logo.png` file from the `Chapter05` folder in this book's GitHub repo and move it into the `landing/static/landing` directory.
7. Start the Django dev server. If it is not already running, then navigate to `http://127.0.0.1:8000/static/landing/logo.png`. You should see the image being served in your browser, illustrating that you have set up static file serving correctly:



Figure 5.8 – Image served by Django

Now that you've seen that loading static images works correctly, we'll learn more about how to prevent hardcoding these URLs. In the next section, you'll see how Django can automatically generate correct static URLs for use in templates.

Generating static URLs with the static template tag

In *Exercise 5.01 – serving a file from an app directory*, you set up an image file to be served by Django. We saw that the URL of the image was `http://127.0.0.1:8000/static/landing/logo.png`, which you could use inside an HTML template. For example, to display the image with an `img` tag, you could use this code in your template:

```

```

Or, since Django is also serving the media and has the same host as the dynamic template response, you can simplify this by just including the path, as follows:

```

```

Both addresses (URLs and paths) have been hardcoded into the template – that is, we include the full path to the static file and make assumptions about where the file is being hosted. This

works fine with the Django development server or if you host your static files and Django website on the same domain. For better performance as your site becomes more popular, you might consider serving static files from their own domain or **Content Delivery Network (CDN)**.

Note

A CDN is a service that can host parts or all of your website for you. It provides several web servers and can seamlessly speed up the loading of your website. For example, it might serve files to a user from the server that is geographically closest to them. There are several CDN providers, and depending on how they are set up, they might require you to specify a certain domain from which to serve your static files.

Take, for instance, a common separation approach: using a different domain for static file serving. You host your main website at `https://www.example.com` but want to serve static files from `https://static.example.com`. During development, we could use just the path to the logo file, as in the example we just saw. But when we deploy to the production server, our URLs would need to change so that they include the domain, like so:

```

```

Since all the links are hardcoded, this would need to be done for every URL throughout our templates, every time we deploy to production. Once they have been changed, though, the URL will no longer work in the Django dev server. Luckily, Django provides a solution to this problem.

The static template tag

The `staticfiles` app provides a template tag, `static`, for dynamically generating the URL to a static file inside a template. Since the URLs are all being dynamically generated, we can change the URL for all of them by changing just one setting (`STATIC_URL` in `settings.py` – more on this soon). Furthermore, later, we will introduce a method for invalidating browser caches for static files that relies on the use of the `static` template tag.

The `static` tag is very simple – it takes a single argument, which is the project-relative path to a static asset. It will then output this path, prepended by the `STATIC_URL` setting. But first, it must be loaded into the template with the `{% load static %}` template tag.

Django has a set of default template tags and filters (or tag sets) that it automatically makes available to every template. Django (and third-party libraries) also provide tag sets that are not automatically loaded. In these cases, we need to load these extra template tags and filters into

a template before we can use them. This can be done with the `load template` tag, which should come near the start of a template (although it must be after the `extends template` tag if one is used). The `load template` tag takes one or more packages/libraries to load. Here's an example:

```
{% load package_one package_two package_three %}
```

This would load the template tag and filters set provided by the (made up) `package_one`, `package_two`, and `package_three` packages.

The `load template` tag must be used in the actual template that requires the loaded package. For example, if your template extends another template, and that base template has loaded a certain package, your dependent template does not automatically have access to that package. Your template must still load the package to access the new tag set. The `static template` tag is not part of the default set, which is why we need to load it.

Then, it can be used to interpolate anywhere inside the template file. For example, by default, Django uses `/static/` as `STATIC_URL`. If we wanted to generate the static URL for our `logo.png` file, we would use the tag in a template like this:

```
{% static 'landing/logo.png' %}
```

The output inside the template would be this:

```
/static/landing/logo.png
```

An example of the static template tag

This will become clearer with an example, so let's look at how the `static` tag could be used to generate a URL for several different assets.

We can include the logo as an image on the page with an `img` tag, as follows:

```

```

This is rendered in the template like so:

```

```

Alternatively, we could use the `static` tag to generate the URL for a linked CSS file, as follows:

```
<link href="{% static 'path/to/file.css' %}" rel="stylesheet">
```

This will be rendered like so:

```
<link href="/static/path/to/file.css" rel="stylesheet">
```

It can be used in a script tag to include a JavaScript file, using the following line of code:

```
<script src="{% static 'path/to/file.js' %}"></script>
```

This is rendered as follows:

```
<script src="/static/path/to/file.js"></script>
```

You can even use it to generate a link to a static file for download, as we've done here:

```
<a href="{% static 'path/to/document.pdf' %}">Download PDF</a>
```

Note

Note that this won't generate the actual PDF content – it will just create a link to an already existing file. We will cover the generation of PDF and image files in *Chapter 13*.

This is rendered as follows:

```
<a href="/static/path/to/document.pdf">Download PDF</a>
```

While referring to these examples, we can now demonstrate the advantage of using the static tag instead of hardcoding. When we are ready to deploy to production, we can just change the `STATIC_URL` value in `settings.py`. None of the values in the templates need to be changed.

For example, we can change `STATIC_URL` to `https://static.example.com/`, and then when the page gets rendered next, the examples we've seen will automatically update. The following line shows this for the image:

```

```

The following is for the CSS link:

```
<link href="https://static.example.com/path/to/files.css" rel="stylesheet">
```

For the script, it's as follows:

```
<script src="https://static.example.com/path/to/file.js"></script>
```

And finally, the following is for the link:

```
<a href="https://static.example.com/path/to/document.pdf">Download PDF</a>
```

Note that in all these examples, a literal string is being passed as an argument (it is quoted). You can also use a variable as an argument – for example, if you were rendering a template with a context, such as in this example code:

```
def view_function(request):
    context = {"image_file": "logfile.png"}
    return render(request, "example.html", context)
```

We are rendering the `example.html` template with a variable called `image_file`. This variable has a value of `logo.png`.

You would pass this variable to the `static` tag without quotes:

```

```

It would render like this (assuming we changed `STATIC_URL` back to `/static/`):

```

```

The template tag can also be used with the `as [variable]` suffix to assign the result to a variable for use later in the template. This can be useful if the static file lookup takes a long time and you want to refer to the same static file multiple times (such as by including an image in multiple places).

The first time you refer to the static URL, give it a variable name to assign to. In this case, we are creating the `logo_path` variable:

```

```

This renders the same as the examples we've seen before:

```

```

However, we can then use the assigned variable (`logo_path`) again later in the template:

```

```

This renders the same again:

```

```

This variable is now just a normal context variable in the template scope and can be used anywhere in the template. Be careful, though, as you might override a variable that has already been defined – although this is a general warning when using any of the template tags that assign variables (for example, `{% with %}`).

In the next exercise, we will put the static template into practice by adding a template to the `business_site` project, then including the example image.

Exercise 5.02 – using the static template tag

In *Exercise 5.01 – serving a file from an app directory*, you tested serving `logo.png` from the `static` directory. In this exercise, you will continue with the `business site` project and create an `index.html` file as the template for our landing page. Then, you'll include the logo in this page using the `{% static %}` template tag:

1. In PyCharm (make sure you're in the `business_site` project), right-click on the `landing` directory and create a new folder called `templates`.
2. Right-click on the new `templates` directory and select **New | HTML File**. Select **HTML 5 file** and name it `index.html`:



Figure 5.9 – New `index.html`

3. Open the `index.html` file and, first, load the static tag library to make the static tag available in the template. Do this with the `load template` tag. On the second line of the file (just after `<!DOCTYPE html>`), add this line to load the static library:

```
{% load static %}
```

4. You can also make the template a bit nicer with some extra content. Enter the Business Site text inside the `<title>` tags:

```
<title>Business Site</title>
```

Then, inside the body, add an `<h1>` element with the `Welcome to my Business Site` text:

```
<h1>Welcome to my Business Site</h1>
```

5. Underneath the heading text, use the `{% static %}` template tag to set the source of an `<img>`. You will use it to refer to the logo from *Exercise 5.01 – serving a file from an app directory*:

```

```

6. Finally, to flesh out the site a bit, add a `<p>` element under `<img>`. Give it some text about the business:

```
<p>Welcome to the site for my Business. For all your Business needs!</p>
```

Although the extra text and title are not too important, they give us an idea of how to use the `{% static %}` template tag around the rest of the content. Save the file. It should look like in the `Chapter05` folder in this book's GitHub repo.

7. Next, set up a URL to use to render the template. You will also use the built-in `TemplateView` to render the template without having to create a view. Open `urls.py` in the `business_site` package directory. At the start of the file, import `TemplateView` as follows:

```
from django.views.generic import TemplateView
```

You can also remove this Django `import admin` line since we're not using it in this project:

```
from django.contrib import admin
```

8. Add a URL map from the root URL to a `TemplateView`. The `as_view` method of `TemplateView` takes `template_name` as an argument, which is used in the same way as a path that you might pass to the render function. Your `urlpatterns` should look like this:

```
urlpatterns = [  
    path("",  
        TemplateView.as_view(template_name="index.html")),  
]
```

Save the `urls.py` file. Once complete, it should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

9. Start the Django dev server, if it's not already running. Navigate to `http://127.0.0.1:8000/` in your browser. You should see your new landing page, as shown in *Figure 5.10*:



Figure 5.10 – My business site with a logo

In this exercise, we added a base template for landing and loaded the static library into the template. Once the static library was loaded, we were able to use the `static` template tag to load an image. Then, we were able to see our business logo rendered in the browser.

So far, the static file loading has used `AppDirectoriesFinder` because it required no extra configuration to use it. In the next section, we will look at `FileSystemFinder`, which is more flexible but requires a small amount of configuration to use it.

FileSystemFinder

So far, we've learned about `AppDirectoriesFinder`, which loads static files inside Django app directories. However, we expect well-designed apps to be self-contained, and therefore, they should only contain static files that they rely on. If we have other static files that are used throughout the website or across different apps, we should store them outside the app directory.

Note

As a general rule, your CSS is probably consistent throughout your site and could be kept in a global directory. Some images and JavaScript code could be specific to apps, so these would be stored in the static directory for that application. This is just general advice, though: you can store static files anywhere that makes the most sense for your project.

In our business site application, we will be storing a CSS file in a site static directory, as it will be used not only in the landing app but also throughout the site as we add more apps.

Django provides support for serving static files from arbitrary directories using its `FileSystemFinder` static file finder. The directories can be anywhere on the disk. Usually, you will have a static directory inside your project directory, but if your company has a global static directory that is used in many different projects (including non-Django web applications), then you could use this as well.

`FileSystemFinder` uses the `STATICFILES_DIRS` setting in the `settings.py` file to determine which directories to search for static files in. This is not present when the project is created and must be set by the developer. We will add it in the next exercise. There are two options for building this list:

- Setting a list of directories
- Setting a list of tuples in the form of `(prefix, directory)`

The second use case will be easier to understand once we have covered some more of the fundamentals, so we will return to it after explaining and demonstrating the first case. This will be covered after *Exercise 5.04 – collecting static files for production*, in the `STATICFILES_DIRS` *prefixed mode* section. For now, we will just explain the first use case, which is just a list of one or more directories.

In `business_site`, we will add a `static` directory inside the project directory (that is, in the same directory that contains the landing app and `manage.py` file). We can use the `BASE_DIR` setting when building the list to assign to `STATICFILES_DIRS`:

```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

We also mentioned earlier in this section that you might want to set multiple directory paths in this list; for example, if you had some company-wide static data shared by multiple web projects. Simply add extra directories to the `STATICFILES_DIRS` list:

```
STATICFILES_DIRS = [BASE_DIR / "static",  
                    "/Users/username/projects/company-static/"  
]
```

Each of these directories would be checked, in the order specified, to find a matching file. If a file existed in both directories, the first one found would be served. For example, if the `static/main.css` (inside the `business_site` project directory) and `/Users/username/projects/company-static/bar/main.css` files both existed, a request for `/static/main.css` would serve the `business_site` project's `main.css` as it is first in the list. Consider then, when deciding the order in which you add directories to `STATICFILES_DIRS`, you may choose to prioritize your project static files over the global ones or vice versa.

In our business site (and later with Bookr), we will only use one static directory in this list, so we won't have to worry about this problem.

In the next exercise, we will add a static directory with a CSS file inside. Then, we will configure the `STATICFILES_DIRS` setting so that it can be served from the `static` directory.

Exercise 5.03 – serving from a project static directory

We showed an example of serving an application-specific image file in *Exercise 5.01 – serving a file from an app directory*. Now, we want to serve a CSS file that is to be used throughout our project to set styles, so we will serve this from a static directory right inside the project folder.

In this exercise, you'll set up your project to serve static files from a specific directory, and then use the `{% static %}` template tag again to include it in the template. This will be done using the `business_site` example project:

1. Open the `business_site` project in PyCharm if it's not already open. Then, right-click on the `business_site` project directory (the top-level `business_site` directory, not the `business_site` package directory) and select **New | Directory**.
2. In the **New Directory** dialog, enter `static` and click **OK**.
3. Right-click on the `static` directory you just created and select **New | File**.
4. In the **Name New File** dialog, enter `main.css` and click **OK**.
5. The blank `main.css` file should open automatically. Enter a couple of simple CSS rules to center the text and set a font and background color, like so:

```
body {  
    font-family: Arial, sans-serif;  
    text-align: center;  
    background-color: #f0f0f0;  
}
```

Now save and close `main.css`.

You can also take a look at the complete file for reference in the `Chapter05` folder in this book's GitHub repo.

6. Next, open `business_site/settings.py`. Here, set a list of directories to the `STATICFILES_DIRS` settings. In this case, the list will have just one item. Define a new variable called `STATICFILES_DIRS` at the bottom of the `settings.py` file using the following code:

```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

In the `settings.py` file, `BASE_DIR` is a variable that contains the path to the project directory. You can build the full path to the static directory you created in *Step 2* by joining `static` to `BASE_DIR`. Then, you can put this inside a list. The complete `settings.py` file should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

7. Start the Django dev server if it is not running. You can verify that the settings are correct by checking if you can load the `main.css` file. Note that this is not namespaced,

so the URL is `http://127.0.0.1:8000/static/main.css`. Open this URL in your browser and check that the content matches what you just entered and saved:



Figure 5.11 – CSS served by Django

If the file does not load, check your `STATICFILES_DIRS` settings. You may need to restart the Django dev server if it was running while you made changes to `settings.py`.

8. Now, you need to include `main.css` in your `index` template. Open `index.html` in the `templates` folder. Before the closing `</head>` tag, add this `<link>` tag to load the CSS:

```
<link rel="stylesheet" href="{% static 'main.css' %}">
```

This links in the `main.css` file, using the `{% static %}` template tag. As mentioned earlier, since `main.css` is not namespaced, you can just include its name. Save the file; it should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

9. Load `http://127.0.0.1:8000/` in your browser; you should see the background color, fonts, and alignment all change:



Figure 5.12 – CSS applied with custom fonts visible

Your business landing page should look like what's shown in *Figure 5.12*. Since you included the CSS in the `index.html` template, it will be available in all the templates that extend this template (although none do at the moment, it's good planning for the future).

In this exercise, we put some CSS rules into a file and served them using Django's `FileSystemFinder`. This was accomplished by creating a `static` directory inside the `business_site` project directory and specifying it in the Django settings (the `settings.py` file) using the `STATICFILES_DIRS` setting. We linked the `main.css` file to the `index.html` template using the `static` template tag. We loaded the main page in our browser and saw the font and color changes applied.

We've now covered how static file finders are used during a request (to load a specific static file when given a URL). We'll now look at their other use case: finding and copying static files for production deployment when running the `collectstatic` management command.

Static file finders – use during `collectstatic`

Once we have finished working on our static files, they need to be moved into a specific directory that can be served by our production web server. We can then deploy our website by copying our Django code and static files to our production web server. In the case of `business_site`, we will want to move `logo.png` and `main.css` (along with other static files that Django itself includes) into a single directory that can be copied to the production web server. This is the role of the `collectstatic` management command.

Without having to run `collectstatic`, a web server would not be able to map a URL back to a path. For example, it would not know that `main.css` must be loaded from the project static

directory while `logo.png` is to be loaded from the landing app directory – it has no concept of the Django directory layout. Upon running the `collectstatic` management command, Django uses each finder to list static files on the disk. Every static file that is found is then copied into the `STATIC_ROOT` directory (also defined in `settings.py`). This is a little bit like the reverse of handling a request. Instead of getting a URL path and mapping it to a filesystem path, the filesystem path is copied to a location that can be predicted by the frontend web server. This allows the frontend web server to handle a request for a static file independently of Django.

Note

A frontend web server is a web server that's designed to route requests to applications (such as Django) or read static files from disk. They can handle requests faster, but aren't able to generate dynamic content in the same way as something such as Django. Frontend web servers are software such as Apache HTTPD, nginx, and lighttpd.

For some specific examples of how `collectstatic` works, we'll use the two files from *Exercise 5.01 – serving a file from an app directory* and *Exercise 5.03 – serving from a project status directory*, respectively: `landing/static/landing/logo.png` and `static/main.css`.

Let's assume that `STATIC_ROOT` has been set to a directory being served by a normal web server – this would be something such as `/var/www/business_site/static`. The destination for these files would be `/var/www/business_site/static/reviews/logo.png` and `/var/www/business_site/static/main.css`, respectively.

Now, when a request for a static file comes in, the web server will be able to serve it easily because the paths are mapped consistently:

- `/static/main.css` is served from the `/var/www/business_site/static/main.css` file
- `/static/reviews/logo.png` is served from the `/var/www/business_site/static/reviews/logo.png` file

This means the web server root is `/var/www/business_site/` and static paths are loaded directly from disk in the usual manner that a web server would load files.

With that, we have demonstrated how Django locates static files during development and can serve them itself. In production, we need the frontend web server to be able to serve static files without involving Django, for both safety and speed.

You might be tempted to serve files directly from the Django project directory by setting your web server root to this directory – **don't do this**. There is a **security risk** in sharing your entire

Django project directory, as it would make it possible to download `settings.py` or other sensitive files. Running `collectstatic` will copy the files to a directory that can be moved outside the Django project directory to the web server root for security.

Here, we have talked about using `collectstatic` to copy static files directly to the web server root. You could also have Django copy them to an intermediary directory and have your deployment process move to a CDN or another server afterward. However, we won't go into detail on specific deployment processes; how you choose to copy static files to the web server will depend on you or your company's existing setup (for example, a continuous delivery pipeline).

Note

The `collectstatic` command does not take into consideration the use of `static` template tags. It will collect all the static files inside static directories, even those that your project does not include inside a template.

In the next exercise, we will see the `collectstatic` command in action. We'll use it to copy all the `business_site` static files that we have so far into a temporary directory.

Exercise 5.04 – collecting static files for production

While we won't be covering deployment to a web server in this chapter, we can still use the `collectstatic` management command and see its result. In this exercise, we will create a temporary holding location for the static files to be copied into. This directory will be called `static_production_test` and will be located inside the `business_site` project directory.

As part of the deployment process, you could copy this directory to your production web server. However, since we won't be setting up a web server until *Chapter 18, Deploying Django*, we will just examine its contents to understand how files are copied and organized.

Here's how to get started:

1. In PyCharm, create a temporary directory to put the collected files in. Right-click on the `business_site` project directory (this is the top-level folder, not the `business_site` module) and select **New | Directory**.
2. In the **New Directory** dialog, enter `static_production_test` and hit *Enter*.
3. Open `settings.py` and, at the bottom of the file, define a new setting for `STATIC_ROOT`. Set it to the path of the directory you just created:

```
STATIC_ROOT = BASE_DIR / "static_production_test"
```

This will join `static_dir` to `BASE_DIR` (the `business_site` project path) to generate the full path. Save the `settings.py` file. It should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

4. In a terminal, run the `collectstatic` manage command:

```
python manage.py collectstatic
```

You should see an output like the following:

```
129 static files copied to '/Users/chrisquest/business_site/
static_production_test'.
```

This might seem like a lot if you were expecting it to copy just two files, but remember that it will copy all the files for all installed apps. In this case, as you have the Django admin app installed, most of the 129 files are to support that.

5. Let's look through the `static_production_test` directory to check what has been created. An expanded view of this directory (from the PyCharm project page) is shown in *Figure 5.13*, for reference. Yours should be similar:



Figure 5.13 – Destination directory of the collectstatic command

You should notice three items inside it:

- The `admin` directory: This contains files from the Django admin app. If you look inside this, you'll see it has been organized into subfolders for `css`, `fonts`, `img`, and `js`.

- The `landing` directory: This is the static directory from your landing app. Inside it is the `logo.png` file. This directory has been created to match the namespacing of the directory that we created.
- The `main.css` file: This is from your project static directory. Since you didn't place it inside a namespacing directory, this has been placed directly inside `STATIC_ROOT`.

If you want, you can open any of these files and verify that their content matches the files you have just been working on – they should do as they are simply copies of the original files.

In this exercise, we collected all the static files from `business_site` (including the admin static files that Django includes). They were copied into the directory defined by the `STATIC_ROOT` setting (`static_production_test` inside the `business_site` project directory). We saw that `main.css` was directly inside this folder, but that other static files were namespaced inside their app directories (admin and reviews). This folder could have been copied to a production web server to deploy our project.

In the next section, we'll look at how a prefix can be added to each static file directory source to customize how static files are organized after being collected.

STATICFILES_DIRS prefixed mode

As mentioned earlier, the `STATICFILES_DIRS` setting also accepts items as tuples in the form of `(prefix, directory)`. These modes of operation are not mutually exclusive; `STATICFILES_DIRS` may contain both non-prefixed (string) or prefixed (tuple) items. Essentially, this allows you to map a certain URL prefix to a directory.

In Bookr, we do not have enough static assets to warrant setting this up, but it can be useful if you want to organize your static assets differently. For example, you can keep all your images in a certain directory, and all your CSS in another directory. You might need to do this if you use a third-party CSS generation tool such as Node.js with LESS.

Note

LESS is a CSS pre-processor that uses Node.js. It allows you to write CSS using variables and other programming-like concepts that don't exist natively. Node.js will then compile this to CSS. A more in-depth explanation is outside the scope of this book, though the site lesscss.org offers a comprehensive introduction and further documentation. If you use it (or a similar tool), then you might want to serve directly from the directory to which it saves its compiled output.

The easiest way to explain how prefixed mode works is with a short example. This will expand on the `STATICFILES_DIRS` setting we created in *Exercise 5.03 – serving from a project static directory*; however, this is not an exercise. In this example, two prefixed directories are being added to this setting – one for serving images and one for serving CSS:

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
    ("images", BASE_DIR / "static_images"),  
    ("css", BASE_DIR / "static_css"),  
]
```

As well as the `static` directory that was already being served with no prefix, we have added serving of the `static_images` directory inside the `business_site` project directory. This has a prefix of `images`. We have also added serving for the `static_css` directory inside the `Bookr` project directory, with a prefix of `css`.

Then, we can serve three files – `main.js`, `main.css`, and `main.jpg` – from the `static`, `static_css`, and `static_images` directories, respectively. The directory layout will be as shown in *Figure 5.14*:

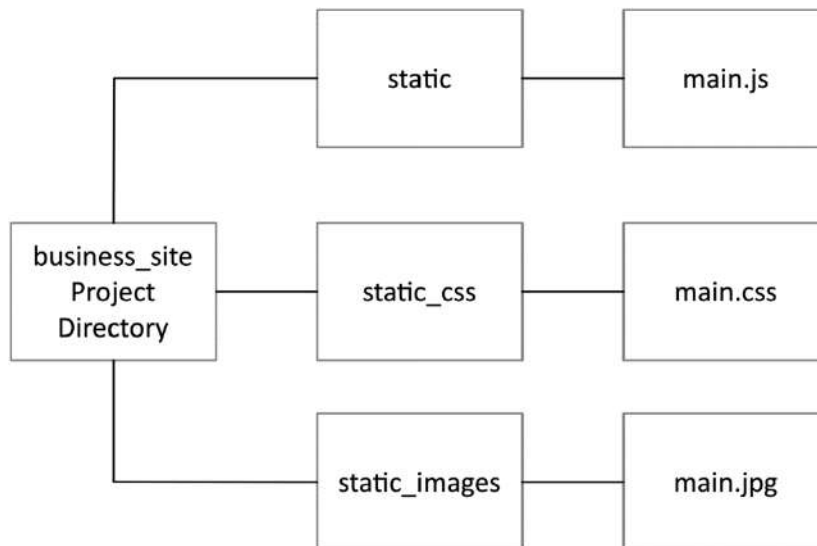


Figure 5.14 – Directory layout for use with prefixed URLs

In terms of accessing these via a URL, the mapping is as shown in *Figure 5.15*:

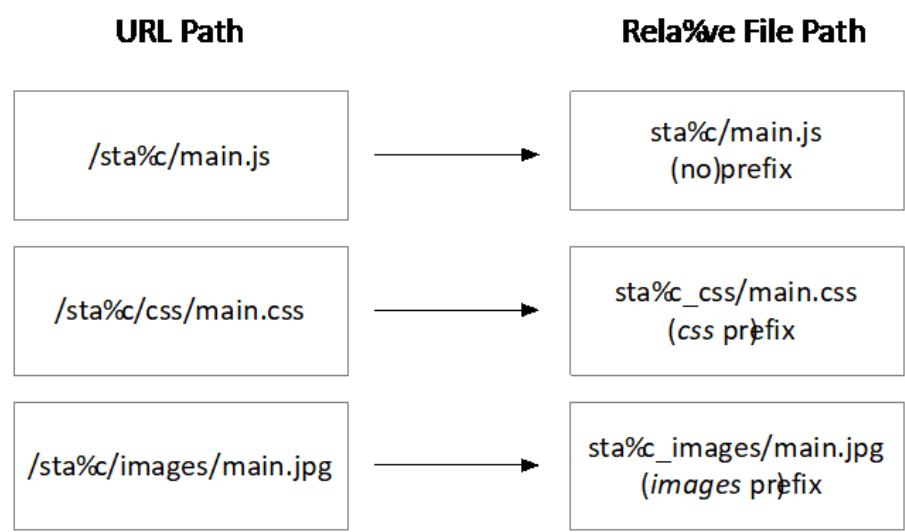


Figure 5.15 – Mapping a URL to a file based on the prefix

Django routes any static URL with a prefix to the directory that matches that prefix. When used with the static template tag, use the prefix and filename, not the directory name. Here's an example:

```
{% static 'images/main.jpg' %}
```

When the static files are gathered using the collectstatic command, they are moved into a directory with the prefix name, inside STATIC_ROOT. The source paths and the target paths inside the STATIC_ROOT directory are shown in Figure 5.16:

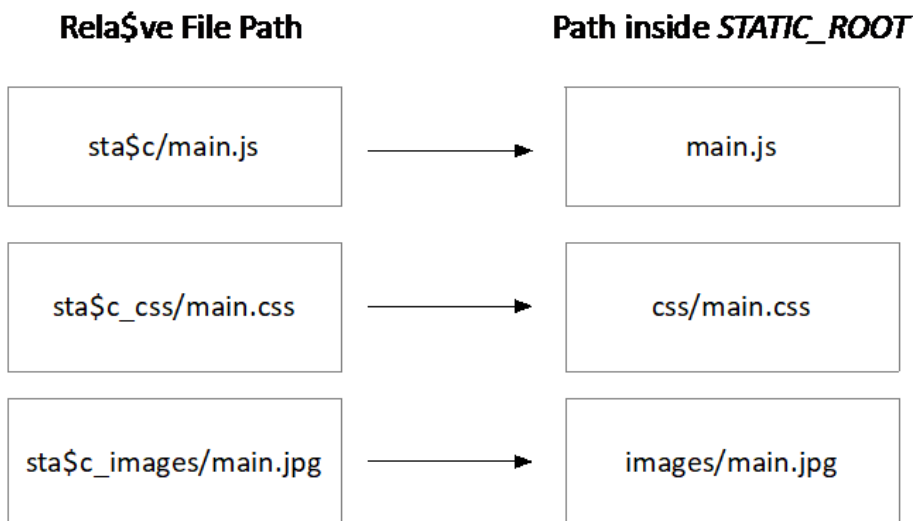


Figure 5.16 – Mapping from the path in the project directory to the path in *STATIC_ROOT*

Django creates the prefix directories inside *STATIC_ROOT*. Because of this, the paths can be kept consistent, even when using a web server and not routing the URL lookup through Django.

Next, we'll look at a management command that's useful for troubleshooting when static files aren't being loaded as you expect: `findstatic`.

The `findstatic` command

The *staticfiles* application also provides one more management command: `findstatic`. This command allows you to enter the relative path to a static file (the same as what would be used inside a static template tag) and Django will tell you where that file was located. It can also be used in verbose mode to output the directories it is searching through.

Note

You may not be familiar with the concept of verbosity or verbose mode. Having a higher verbosity (or simply turning on verbose mode) will cause a command to generate more output. Many command-line applications can be executed with more or less verbosity. This can be helpful when you're trying to debug programs you are using. To see an example of verbose mode in action, you can try running the Python shell in verbose mode. Enter `python -v` (instead of just `python`) and press *Enter*. Python will start in verbose mode, which prints out the path of every file it imports.

This command is mostly useful for debugging/troubleshooting purposes. If the wrong file is loading, or a particular file can't be found, you can use this command to try to find out why. The command will display which file on disk is being loaded for a specific path, or let you know that the file can't be found and what directories were searched.

This can help solve issues where multiple files have the same name, and the precedence is not what you expect. See the *FileSystemFinder* section for a note about precedence in the `STATICFILES_DIRS` setting. You may also see that Django is not searching in a directory you expect for the file, in which case the static directory might need to be added to the `STATICFILES_DIRS` setting.

In the next exercise, you will execute the `findstatic` management command so that you are familiar with what the outputs are for good (file found correctly) and bad (file missing) scenarios.

Exercise 5.05 – finding files using `findstatic`

You will now run the `findstatic` command with a variety of options and understand what its output means. First, we will use it to find a file that exists and see that it displays the path to the file. Then, we will try to find a file that doesn't exist and check the error that is output. After that, we will repeat this process with multiple levels of verbosity and different ways of interacting with the command.

While this exercise will not make changes to or progress the Bookr project, it is good to be familiar with the command in case you need to use it when working on your own Django applications.

Let's get started:

1. Open a Terminal and navigate to the `business_site` project directory.
2. Execute the `findstatic` command with no options. It will output some help explaining how it is used:

```
python manage.py findstatic
```

The following help output will be displayed:

```
usage: manage.py findstatic [-h] [--first] [--version]
                             [-v {0,1,2,3}]
                             [--settings SETTINGS]
                             [--pythonpath PYTHONPATH]
                             [--traceback] [--no-color]
                             [--force-color]
```

```
        [--skip-checks]
        staticfile [staticfile ...]
manage.py findstatic: error: Enter at least one label.
```

3. You can find one or more files at a time. Let's start with one that we know exists: `main.css`. We'll use the `findstatic` management command to locate it on disk by entering the following command:

```
python3 manage.py findstatic main.css
```

The preceding command outputs the path at which `main.css` was found:

```
Found 'main.css' here:
/Users/chrisquest/business_site/static/main.css
```

Your full path will be different (unless you are also called Chris), but you can see that when Django locates `main.css` in a request, it will load the `main.css` file from the project static directory.

This can be useful if a third-party application you have installed has not namespaced its static files correctly and is conflicting with one of your files.

4. Let's try finding a file that does not exist, `logo.png`:

```
python manage.py findstatic logo.png
```

Django will display an error stating that the file could not be found:

```
No matching file found for 'logo.png'.
```

Django is unable to locate this file because we have namespaced it – we must include the full relative path, the same way we used it in the static template tag.

5. Try finding the `logo.png` again, but this time using the full path:

```
python manage.py findstatic landing/logo.png
```

Django can find the file now:

```
Found 'landing/logo.png' here:
/Users/chrisquest/business_site/landing/static/landing/logo.png
```

6. You can find multiple files at once by adding each file as an argument:

```
python manage.py findstatic landing/logo.png  
missing-file.js main.css
```

The location status for each file is shown as follows:

```
No matching file found for 'missing-file.js'.  
Found 'landing/logo.png' here:  
  /Users/chrisguest/business_site/landing/static/landing/logo.png  
Found 'main.css' here:  
  /Users/chrisguest/business_site/static/main.css
```

7. The command can be executed with a verbosity of 0, 1, or 2. By default, it executes at verbosity 1. To set the verbosity, use the `--verbosity` or `-v` flag. Decreasing the verbosity to 0 only outputs the paths it locates without any extra information. No errors are displayed for missing paths:

```
python manage.py findstatic -v0 landing/logo.png missing-file.js main.css
```

The output shows only found paths – notice that no error is shown for the missing file, `missing-file.js`:

```
/Users/chrisguest/business_site/landing/static/landing/logo.png  
/Users/chrisguest/business_site/static/main.css
```

This level of verbosity can be useful if you are piping the output to another file or command.

8. To get more information about which directories Django is searching in for the file you have requested, increase the verbosity to 2:

```
python manage.py findstatic -v2 landing/logo.png  
missing-file.js main.css
```

The output contains much more information, including the directories that have been searched for the requested file. You can see that as the admin application is installed, Django is also searching in the Django admin application directory for static files:

```
> python manage.py findstatic -v2 landing/logo.png missing-file.js main.css
No matching file found for 'missing-file.js'.

Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
Found 'landing/logo.png' here:
/Users/chrisquest/business_site/landing/static/landing/logo.png
Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
Found 'main.css' here:
/Users/chrisquest/business_site/static/main.css
Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
```

Figure 5.17 – `findstatic` executed with verbosity 2, showing exactly which directories were searched

The `findstatic` command is not something that you will use day-to-day when working with Django, but it is useful to know about when you're trying to troubleshoot problems with static files. Here, we saw the command's output, the full path to a file that existed, as well as the error messages when files did not exist. We also ran the command and supplied multiple files at once, and saw that information about all the files was output. Finally, we ran the command with different levels of verbosity. The `-v0` flag suppressed errors about missing files. `-v1` was the default and displayed found paths and errors. Increasing the verbosity using the `-v2` flag also printed out the directories that were being searched through for a particular static file.

In the next section, we'll take a look at caching static files and how Django helps us ensure our users are served the latest versions.

Serving the latest files (for cache invalidation)

If you are not familiar with caching, the basic idea is that some operations can take a long time to perform. We can speed up a system by storing the results of the operation in a place that is faster to access so that the next time we need them, they can be retrieved quickly. The "operation" that takes a long time can be anything – a function that takes a long time to run, an image that takes a long time to render, or a large asset that takes a long time to download over the internet. We are interested in this last scenario.

You might have noticed that the first time you ever visit a particular website, it can be slow to load, but then the next time, it loads much faster. This is because your browser has cached some (or all) of the static files the site needs to load.

To use our business site as an example, we have a page that includes the `logo.png` file. The first time we visit the business site, we must download the dynamic HTML, which is small and quick to transfer. Our browser parses the HTML and sees that the `logo.png` file should be included. It can then download this file too, which is much larger and can take longer to download. Note that this scenario assumes that our business site is now hosted on a remote server and not on our local machine – which is very fast for us to access.

If the web server has been set up correctly, your browser will store `logo.png` on your computer. The next time we visit the *landing* page (or indeed any page that includes the `logo.png` file), our browser will recognize the URL and can load the file from disk instead of having to download it again, thus speeding up the browsing experience.

Note

We said that the browser will cache "if the web server is set up correctly." What does this mean? The frontend web server should be configured to send special HTTP headers as part of a static file response. It can send a `Cache-Control` header, which can have values such as `no-cache` (the file should never be cached – that is, the latest version should be requested every time), or `max-age=seconds` (the file should only be downloaded again if it was last retrieved more than *seconds* ago). The response could also contain the `Expires` header, with the value being a date. The file is considered to be "stale" once this date has been reached, and at that point, the new version should be requested.

One of the hardest problems in computer science is cache invalidation. For instance, if we change `logo.png`, how does our browser know it should download the new version?

The only surefire way of knowing it had changed would be to download the file again and compare it with the version we had already saved every time. Of course, this defeats the purpose of caching since we would still be downloading every time the file changed (or not).

We can cache for an arbitrary or server-specified amount of time, but if the static file changed before that time was up, we wouldn't know. We would use the old version until we considered it expired, at which time we'd download the new version. If we had a 1-week expiry and the static file changed the next day, we'd still be using the old one for 6 days. Of course, the browser can be made to reload the page without using the cache (how this is done depends on the browser; for example, *Cmd/Ctrl + Shift + R*) if you want to forcefully download all static assets again.

There is no need to try to cache our dynamic responses (rendered templates). Since they are designed to be dynamic, we would want to make sure that the user gets the latest version on every page load, so they should not be cached. They are also quite a small size (compared to assets such as images), so there is not much speed advantage when caching them.

Django provides a built-in solution. During the `collectstatic` phase, when the files are copied, Django can append a hash of their content to the filename. For example, the source file, `logo.png`, will be copied to `static_production_test/landing/logo.f30ba08c60ba.png`. This is only done when using the `ManifestFilesStorage` storage engine.

Since the filename only changes when the content changes, the browser will always download the new content.

Using `ManifestFilesStorage` is just one way of invalidating caches. There may be other options that are more suitable for your particular application.

Note

A hash is a one-way function that generates a string of a fixed length, regardless of the length of the input. There are several different hash functions available, and Django uses **MD5** for content hashing. While no longer cryptographically secure, it is adequate for this purpose. To illustrate the fixed length property, the MD5 hash of the a string is `0cc175b9c0f1b6a831c399e269772661`. The MD5 hash of the a much longer string is `69fc4316c18cdd594a58ec2d59462b97`. They are both 32 characters long.

You can choose the storage engine by changing the `STATICFILES_STORAGE` value in `settings.py`. This is a string with a dotted path to the module and class to use. The class that implements the hash-addition functionality is `django.contrib.staticfiles.storage.ManifestStaticFilesStorage`.

Using this storage engine doesn't require you to make any changes to your HTML templates, provided you are including static assets with the static template tag. Django generates a manifest file (`staticfiles.json`, in JSON format) that contains a mapping between the original filename and the hashed filename. It will automatically insert the hashed filename when using the static template tag. If you are including your static files without using the static tag and instead just manually inserting the static URL, then your browser will attempt to load the non-hashed path, and the URL will not automatically be updated when the cache should be invalidated.

For example, you can include `logo.png` with the static tag:

```

```

When the page is rendered, the latest hash will be retrieved from `staticfiles.json` and the output will be like this:

```

```

Whereas, if we had not used the static tag and instead hardcoded the path, it would always appear as written:

```

```

Since this does not contain a hash, our browser will not see the path changing and thus never attempt to download the new file.

Django retains the previous version of files with the old hash when running `collectstatic`, so older versions of your application can still refer to it if they need to. The latest version of the file is also copied with no hash so that non-Django applications can refer to it without needing to look up the hash.

In the next exercise, we will change our project settings to use the `ManifestFilesStorage` engine, and then run the `collectstatic` management command. This will copy all the static assets as in *Exercise 5.04 – collecting static files for production*; however, they will now have their hash included in the filename.

Exercise 5.06 – exploring the ManifestFilesStorage storage engine

In this exercise, you will temporarily update `settings.py` to use `ManifestFilesStorage`, then run `collectstatic` to see how the files are generated with a hash:

1. In PyCharm (still in the `business_site` project), open `settings.py` and add a `STATICFILES_STORAGE` setting at the bottom of the file:

```
STATICFILES_STORAGE =  
    "django.contrib.staticfiles.storage.ManifestStaticFilesStorage"
```

The completed file should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

2. Open a terminal, navigate to the `business_site` project directory, and run the `collectstatic` command like before:

```
python manage.py collectstatic
```

If your `static_production_test` directory is not empty (which will probably be the case as files were moved there in *Exercise 5.04 – collecting static files for production*), then you will be prompted to allow the overwrite of the existing files:

```
> python manage.py collectstatic  
  
You have requested to collect static files at the destination  
location as specified in your settings:  
  
    /Users/chrisquest/business_site/static_production_test  
  
This will overwrite existing files!  
Are you sure you want to do this?  
  
Type 'yes' to continue, or 'no' to cancel:
```

Figure 5.18 – Prompt to allow overwrite during static collection

Just type `yes` and then press `Enter` to allow the overwrite.

The output from this command will tell you the number of files that were copied, as well as the number that were processed and had the hash added to the filename:

```
0 static files copied to '/Users/chrisquest/business_site/  
static_production_test', 129 unmodified, 101 post-processed.
```

Since you haven't changed any files since we last ran `collectstatic`, no files have been copied. Instead, Django post-processes the files (28 of them) – that is, it generates their hashes and appends them to the filename.

3. The static files were copied into the `static_production_test` directory as they were before; however, there are now two copies of each file: one named with the hash and one without. For example, `static/main.css` has been copied to `static_production_test/main.856c74fb7029.css` (this filename might be different if your CSS file contents differ; for example, if it has extra spaces or newlines):



Figure 5.19 – Expanded `static_production_test` directory with hashed filenames

Figure 5.19 shows the expanded `static_production_test` directory layout. You can see two copies of each file's static file, as well as the `staticfiles.json` manifest file. To take `logo.png` as an example, you can see that `landing/static/landing/logo.png` has been copied to `static_production_test/landing/logo.0745d1077c9e.png`.

Let's make a change to the `main.css` file and see how the hash changes.

4. Add some blank lines to the end of the file and save it (make sure that you edit the source file and not the one that was already copied to the `static_production_test` directory). This won't change the effect of the CSS, but the change in the file will affect its hash. Rerun the `collectstatic` command in a terminal:

```
python3 manage.py collectstatic
```

Once again, you may have to enter yes to confirm the overwrite, which will give you the following output:

```
You have requested to collect static files at the destination location as
specified in your settings:
    /Users/chrisgust/business_site/static_production_test

This will overwrite existing files!
Are you sure you want to do this?

Type 'yes' to continue, or 'no' to cancel: yes

1 static file copied to '/Users/chrisgust/business_site/
static_production_test', 128 unmodified, 101 post-processed.
```

Since only one file was changed, only one static file was copied (`main.css`).

5. Look inside the `static_production_test` directory again. You should see that the old file with the old hash was retained and that a new file with a new hash has been added:



Figure 5.20 – Another `main.css` file with the latest hash was added

In this case, we have `main.856c74fb7029.css` (existing), `main.efb556103718.css` (new), and `main.css`. Your hashes may differ.

The `main.css` file (no hash) always contains the newest content – that is, the contents of the `main.efb556103718.css` and `main.css` files are identical. During the execution of `collectstatic`, Django will copy the file with a hash, as well as without a hash.

6. Now, examine the `staticfiles.json` file that Django generates. This is the mapping that allows Django to look up the hashed path from the normal path. Open

`static_production_test/staticfiles.json`. All the content may appear in one line. If it does, enable text soft wrapping from the **View** menu | **Active Editor** | **Soft Wrap**. Scroll to the end of the file; you should see an entry for the `main.css` file. Here's an example:

```
"main.css": " main.efb556103718.css"
```

This is how Django can populate the correct URL in a template when using the static template tag: by looking up the hashed path in this mapping file.

7. We've finished using `business_site`, which we were just using for testing. You can delete the project or keep it around for reference during the activities.

Note

Unfortunately, we can't examine how the hashed URL is interpolated in the template because, when running in debug mode, Django does not look up the hashed version of the file. As we know, the Django dev server only runs in debug mode, so if we were to turn debug mode off to try to view the hashed interpolation, the Django dev server would not start. You will need to examine this interpolation yourself when going to production and when using a frontend web server.

In this exercise, we configured Django to use `ManifestFilesStorage` for its static file storage by adding the `STATICFILES_STORAGE` setting to `settings.py`. Then, we executed the `collectstatic` command to see how the hashes are generated and added to the filename of the copied files. We saw the `staticfiles.json` manifest file, which stored a lookup from the original path to the hashed path. Finally, we cleaned up the settings and directories that we added in this exercise and *Exercise 5.04 – collecting static files for production*. These were the `STATIC_ROOT` setting, the `STATICFILES_STORAGE` setting, and the `static_production_test` directory.

So far, we've only seen how to locate static files stored on our disk. Django can be customized to locate static files stored in other locations (such as in the cloud). In the next section, we'll briefly look at how to use custom storage engines.

Custom storage engines

In the previous section, we set the storage engine to `ManifestFilesStorage`. This class is provided by Django, but it is also possible to write a custom storage engine. For example, you could write a storage engine that uploads your static files to a CDN, Amazon S3, or Google Cloud bucket when you run `collectstatic`.

Writing a custom storage engine is beyond the scope of this book. Third-party libraries already exist that support uploading to a variety of cloud services. One such library is `Django Storages`, which can be found at <https://django-storages.readthedocs.io/>.

The following code is a short skeleton indicating which methods you should implement to create a custom file storage engine:

```
from django.conf import settings
from django.contrib.staticfiles import storage

class CustomFilesStorage(storage.StaticFilesStorage):
    def __init__(self):
        """The class must be able to be instantiated
        without any arguments.
        Create custom settings in settings.py and read them
        instead."""
        self.setting = settings.CUSTOM_STORAGE_SETTING
```

The class you define (in this example, it's called `CustomFilesStorage`) must be able to be instantiated without any arguments. The `__init__` function must be able to load any settings from global identifiers (in this case, from our Django settings):

```
def delete(self, name):
    """Implement delete of the file from the remote
    service."""
```

The `delete` method should be able to delete the file specified by the `name` argument from the remote service:

```
def exists(self, name):
    """Return True if a file with name exists in the
    remote service."""
```

The `exists` method should query the remote service to check if the file specified by name exists. It should return `True` if the file exists, or `False` if it doesn't:

```
def listdir(self, path):  
    """List a directory in the remote service. Return  
    should be a 2-tuple of lists, the first a list  
    of directories, the second a list of files."""
```

The `listdir` method should query the remote service to list the directory at `path`. It should then return a 2-tuple of lists. The first element of this tuple is a list of directories inside `path`, while the second element is a list of files. Here's an example:

```
return (["directory1", "directory2"], ["main.css", "document.txt", "image.jpg"])
```

If `path` contains no directories or no files, then an empty list should be returned for that element. You would return two empty lists if the directory was empty:

```
def size(self, name):  
    """Return the size in bytes of the file with  
    name."""
```

The `size` method should query the remote service and get the size of the file specified by name:

```
def url(self, name):  
    """Return the URL where the file of with name can  
    be access on the remote service. For example,  
    this might be URL of the file after it has been  
    uploaded to a specific remote host with a  
    specific domain."""
```

The `url` method should determine the URL to access the file specified by name. This can be built by appending name to a specific static hosting URL:

```
def _open(self, name, mode="rb"):  
    """Return a File-like object pointing to file with  
    name. For example, this could be a URL handle  
    for a remote file."""
```

The `_open` method will provide a remote file handle, specified by name. How you implement this will depend on the type of remote service. You might have to download the file and then use a memory buffer (such as an `io.BytesIO` object) to simulate opening the file:

```
def _save(self, name, content):  
    """Write the content for a file with name. In this  
    method you might upload the content to a remote  
    service."""
```

The `_save` method should save content to the remote file at `name`. The method of implementing this will depend on your remote service. It might transfer the file over SFTP or upload it to a CDN.

While this example does not implement any transferring to or from a remote service, you can refer to it to get an idea of how to implement a custom storage engine.

After implementing your custom storage engine, you can make it active by setting its dotted module path in the `STATICFILES_STORAGE` setting in `settings.py`. We won't proceed further with custom storage, but will take some time to practice what we have learned about static files with activities that implement features of the Bookr project.

```
STATICFILES_STORAGE = "myapp.storages.CustomFilesStorage"
```

You'll now work on the first activity in this chapter. In this activity, you'll add a static logo image to the Bookr application. The logo will be displayed in the **Reviews** section of the website.

Activity 5.01 – adding a Reviews logo

The Bookr app should have a logo that is specific to pages in the Reviews app. This will involve adding a base template just for the Reviews app and updating our current Reviews templates to inherit from it. Then, you'll include the Bookr Reviews logo on this base template. These steps will help you complete this activity:

1. Add a CSS rule to position the logo. Put this rule into the existing `base.html` file, after the `.navbar-brand` rule:

```
.navbar-brand > img {  
    height: 60px;  
}
```

2. Add a brand block template tag that inheriting templates can override. Put this inside the `<a>` element with the `navbar-brand` class. The default contents of block should be left as *Book Review*.
3. Add a static directory inside the Reviews app, containing a namespaced directory. Download the `Reviews logo.png` file from the `Chapter05` folder in this book's GitHub repo and put it inside this directory.
4. Create a `templates` directory for the Bookr project (inside the Bookr project directory). Then, move the Reviews app's current `base.html` into this directory so that it becomes a base template for the whole project.
5. Add the new templates directory's path to the `TEMPLATES["DIRS"]` setting in `settings.py`.
6. Create another `base.html` template specifically for the Reviews app. Put it inside the Reviews app's templates directory. The new template should extend the existing (now global) `base.html` file.
7. The new `base.html` file should override the content of the brand block. This block should contain just an `<img>` whose `src` attribute is set using the `{% static %}` template tag. The image source should be the logo we added in *Step 2*.

Refer to the following screenshots to see what your pages should look like after these changes. Note that, although you are making changes to the base template, it will not change the layout of the main page:

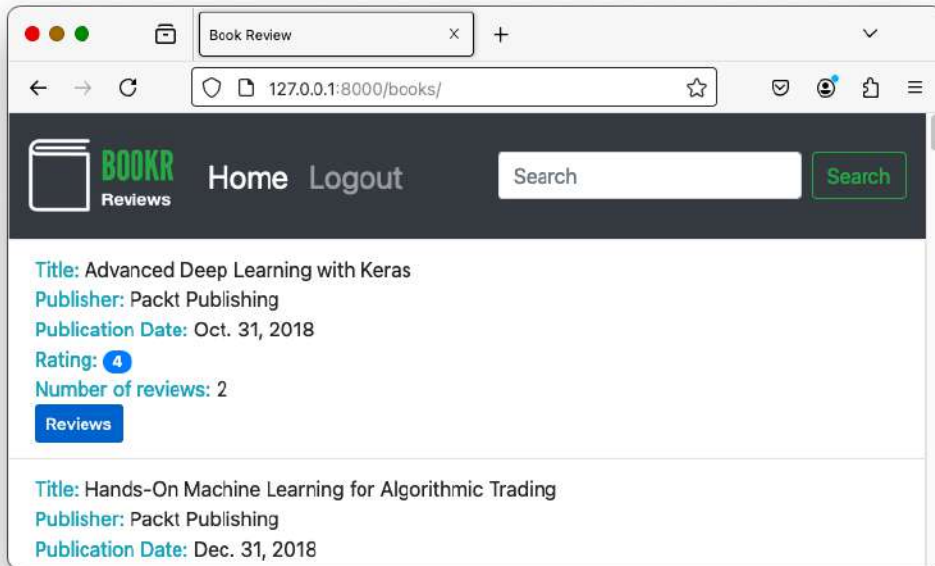


Figure 5.21 – The Book List page after adding the Reviews logo

On the **Book Details** page, you will also see the Reviews logo:

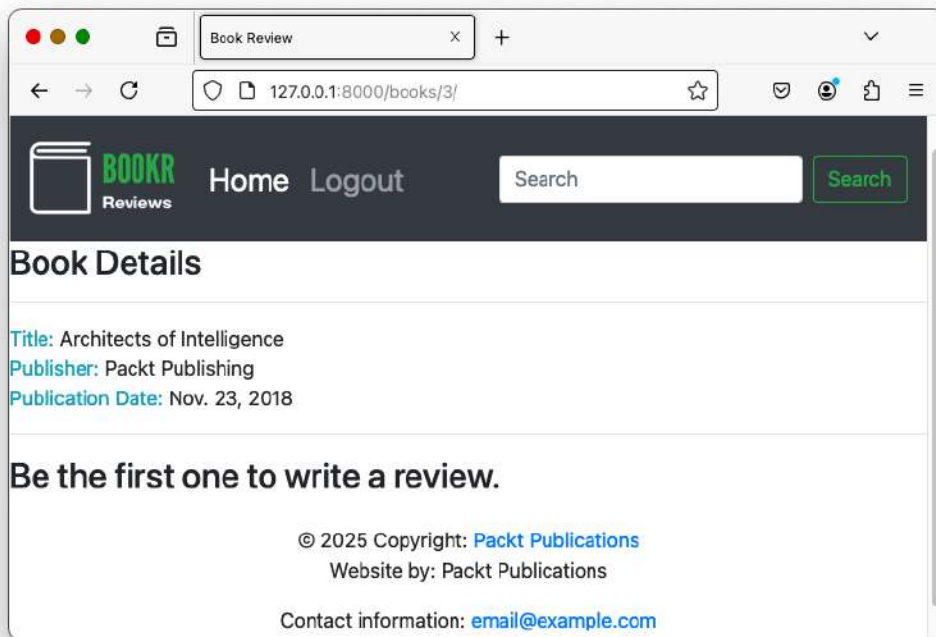


Figure 5.22 – The Book Details page after adding the Reviews logo

Activity 5.02 – CSS enhancements

Currently, the CSS is kept inline in the `base.html` template. As a best practice, it should be moved into its own file so that it can be cached separately and decrease the size of the HTML downloads. As part of this, you'll also add some CSS enhancements, such as fonts and colors, and link in Google Fonts CSS to support these changes. These steps will help you complete this activity:

1. Create a directory named `static` in the Bookr project directory. Then, create a new file inside it named `main.css`.

2. Copy the contents of the `<style>` element from the main `base.html` template into the new `main.css` file, then remove the `<style>` element from the template. Add these extra rules to the end of the CSS file:

```
body {  
    font-family: 'Source Sans Pro', sans-serif;  
    background-color: #e6efe8  
    color: #393939;  
}  
  
h1, h2, h3, h4, h5, h6 {  
    font-family: 'Libre Baskerville', serif;  
}
```

3. Link to the new `main.css` file with a `<link rel="stylesheet" href="...">` tag. Use the `{% static %}` template tag to generate the URL for the href attribute, and don't forget to load the static library.
4. Link in the Google Fonts CSS by adding this code to the base template:

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css?  
family=Libre+Baskerville|Source+Sans+Pro&display=swap">
```

(You will need to have an active internet connection so that your browser can include this remote CSS file.)

5. Update your Django settings to add `STATICFILES_DIRS`, which is set to the static directory you created in *Step 1*. When you're finished, your Bookr application should look like *Figure 5.23*:

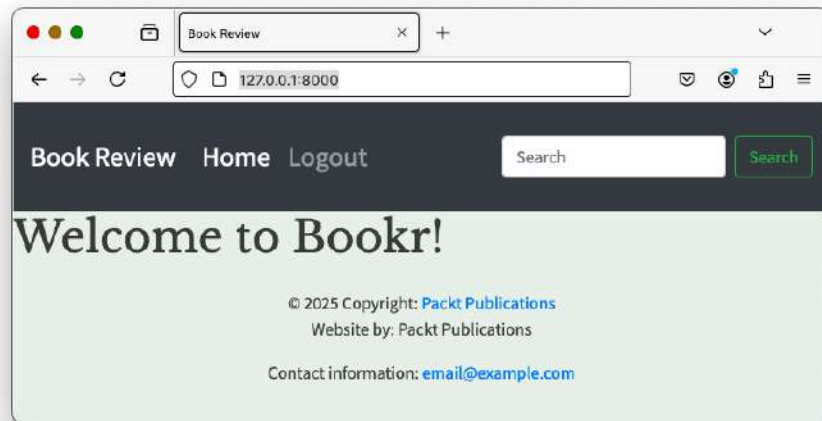


Figure 5.23 – Main page with a new font and background color

Notice the new font and background color. These should be displayed on all the Bookr pages.

Activity 5.03 – adding a global logo

You have already added a logo that is served on pages for the Reviews app. We have another logo to be used globally by default, but other apps will be able to override it:

1. Download the Bookr logo (logo.png) from the Chapter05 folder in this book's GitHub repo.
2. Save it in the main static directory for the project.
3. Edit the main base.html file. We already have a block for the logo (*brand*), so an `<img>` can be placed inside here. Use the static template tag to refer to the logo you just downloaded.
4. Check that your pages work. On the main URL, you should see the Bookr logo, but on the **Book List** and **Book Details** pages, you should see the Bookr Reviews logo.

When you're finished, you should see the Bookr logo on the main page:

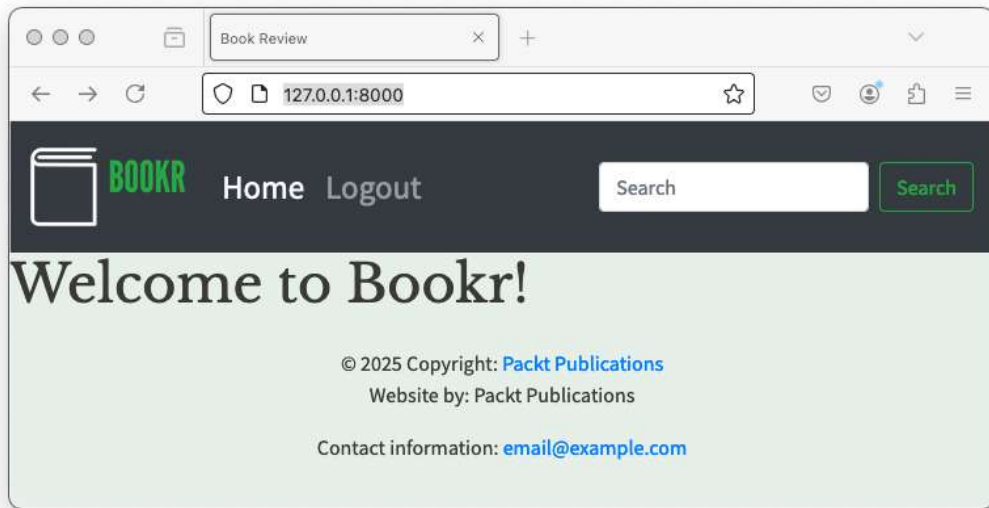


Figure 5.24 – Bookr logo on the main page

When you visit a page that had the Bookr Reviews logo before, such as the **Book List** page, it should still show the Bookr Reviews logo:

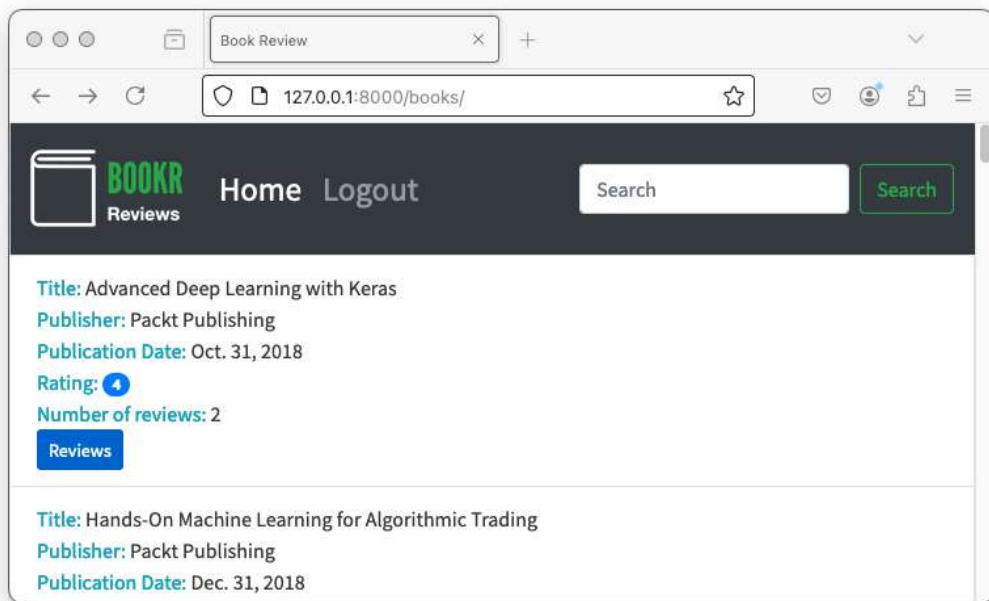


Figure 5.25 – Bookr Reviews logo still appears on the Reviews pages

Summary

In this chapter, we showed you how to use Django's `staticfiles` app to find and serve static files. We used the built-in `static` view to serve these files with the Django dev server in `DEBUG` mode. We showed different places to store static files by using a directory that is global to the project or in a specific directory for the application; global resources should be stored in the former while application-specific resources should be stored in the latter. We showed the importance of namespacing static file directories to prevent conflicts. After serving the assets, we used the `static` tag to include them in our template. We then demoed how the `collectstatic` command copies all the assets into the `STATIC_ROOT` directory, for production deployment. We showed how to use the `findstatic` command to debug the loading of static files. To invalidate caches automatically, we looked at using `ManifestFilesStorage` to add a hash of the file's content to the static file URL. Finally, we briefly talked about using a custom file storage engine.

So far, we have only fetched web pages using content that already exists. In the next chapter, we will start adding forms so that we can interact with web pages by sending data to them over HTTP.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

6

Forms

This chapter introduces **web forms**, a method of sending information from the browser to a web server. We will start by introducing forms in general and discussing how data is encoded to be sent to the server.

So far, the views we have been building for Django have been one-way only. Our browser is retrieving data from the views we have written, but it doesn't send any data back to them. In *Chapter 4, Introduction to Django Admin*, we created model instances by using the Django admin and submitting forms, but those were using views built into Django, not created by us. In this chapter, we will use the Django Forms library to start accepting user-submitted data. The data will be provided through GET requests in the URL parameters and/or POST requests in the body of the request. But before we get into the details, first, let's understand what forms are in Django. You'll learn about the differences between sending form data in a GET HTTP request and sending it in a POST HTTP request, and how to choose which one to use.

In this chapter, we will be covering the following topics:

- Background to forms
- Form security
- The Django Forms library
- Validating forms and retrieving Python values
- Activity 1 – Book search

By the end of this chapter, you will have learned how Django's Forms library is used to build and validate forms automatically and how it cuts down on the amount of manual HTML to write.

Technical requirements

Find the solution in the `Chapter06` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Background to forms

Django forms act as a helper that does three jobs: they build the HTML fields, check that the user's input is valid, and provide clean data to use in a view. They are like a gatekeeper between what a user types and what the application accepts.

When working with an interactive web app, we not only want to provide data to users but also accept data from them to either customize the responses we're generating or to let them submit data to the site. A form is made up of inputs that define key-value pairs of data to submit to the server. For example, when logging in to a website, the data being sent would have the keys `username` and `password`, with values of your username and your password, respectively. Each input in the form has a *name*, and this is how its data is identified on the server side (in a Django view). There can be multiple inputs with the same name, whose data is available in a list containing all the posted values with this name – for example, a list of checkboxes with permissions to apply to users. Each checkbox would have the same name but a different value. The form has attributes that specify which URL the browser should submit the data to and what method it should use to submit the data (browsers only support GET or POST).

The form element

All inputs used during form submission must be contained inside a `<form>` element. There are three HTML attributes that you must use to modify the behavior of the form:

- **Method:** This is the HTTP method for submitting the form; it's either GET or POST. If omitted, this defaults to `get` (because this is the default method when typing in a URL in the browser and hitting *Enter*).
- **Action:** This refers to the URL (or path) to send the form data to. If omitted, the data gets sent back to the current page.
- **Enctype:** This sets the encoding type of the form. You only need to change this if you are using the form to upload files. The most common values are `application/x-www-form-urlencoded` (the default if this value is omitted) or `multipart/form-data` (set this if you're uploading files). Note that you don't have to worry about the encoding type in your view – Django handles the different types automatically.

Here is an example of a form without any of its attributes set:

```
<form>
    <!-- Input elements go here -->
</form>
```

It will submit its data using a GET request to the current URL that the form is being displayed on, using the `application/x-www-form-urlencoded` encoding type.

In the following example, we'll set all three attributes on a form:

```
<form method="post" action="/form-submit" enctype="multipart/form-data">
    <!-- Input elements go here -->
</form>
```

This form will submit its data with a POST request to the `/form-submit` path, encoding the data as `multipart/form-data`.

How do GET and POST requests differ in how the data is sent? Recall in *Chapter 1, Introduction to Django*, that we discussed what the underlying HTTP request and response data that your browser sends look like. In the following two examples, we will submit the same form twice: the first time using GET and the second time using POST. The form will have two inputs: a first name and a last name.

A form submitted using GET sends its data in the URL like this:

```
GET /form-submit?first_name=Joe&last_name=Bloggs HTTP/1.1
Host: www.example.com
```

However, a form submitted using POST sends its data in the body of the request, like this:

```
POST /form-submit HTTP/1.1
Host: www.example.com
Content-Length: 31
Content-Type: application/x-www-form-urlencoded

first_name=Joe&last_name=Bloggs
```

You'll notice that the form data is encoded the same way in both cases; it is just placed differently for the GET and POST requests. In the *Choosing between GET or POST* section, we'll discuss how to choose between these two types of requests.

Form security

As HTML forms are a source of external data to reach an app, we need to consider the security implications that they pose.

We will briefly discuss the main security considerations that HTML forms introduce to an app and will address them in this and later chapters. This is by no means a complete list of security considerations, as new threats emerge over time.

Note

For further information on security, consider *Attacking and Exploiting Modern Web Applications*, by Simone and Donato Onofri (Packt Publishing, 2023, ISBN: 9781801816298): <https://www.packtpub.com/en-us/product/attacking-and-exploiting-modern-web-applications-9781801816298>.

The **Open Worldwide Application Security Project (OWASP)** provides excellent online resources for application security education, including the OWASP Top Ten vulnerabilities: <https://owasp.org/www-project-top-ten/>.

Input validation

We are accustomed to seeing client-side form validation, such as JavaScript checks, within the HTML page that is executed by the browser, but it is also crucial to carry out these checks server-side. If form validation is only carried out client-side, this can be sidestepped by either switching off JavaScript in the browser or bypassing the browser by submitting the form data using a script or dev tool.

Traditionally, this secure approach has involved duplicated logic by including the frontend validation as well as backend validation. Fortunately, in Django, the `django.forms.Form` class provides a single point of truth for defining validation logic, which we will learn about in the *The Django Forms library* section.

Cross-site request forgery protection

Cross-site request forgery (CSRF) protection prevents a web application from receiving form posts from another site.

Historically, the form action attribute could specify any web resource as a URL, and it was possible to create a remote form that could even masquerade for another purpose, particularly by including malicious content in hidden fields. In fact, the CSRF attack could even be

implemented without a form, being an HTTP POST request implemented in JavaScript on the remote page.

CSRF protection requires a token to be included in the form that is expected on the server side. The CSRF token is stored as a cookie in the browser and is unique on a user-per-session basis.

As we will see in *Exercise 6.01*, Django apps are configured by default with CSRF protection enabled.

Django also gives us a mechanism for including the CSRF token in the form.

The CSRF token must be added to the HTML for every form being sent, and is done with the `{% csrf_token %}` template tag. When the template is rendered, this tag will be interpolated similar to the following:

```
<input type="hidden" name="csrfmiddlewaretoken"
value="tETZjLDUXev1tiYqGCSbMQkhWiesHCnutxpt6mutHI6YH64F0nin5k2JW3B68IeJ">
```

Since this is a hidden field, the form on the page does not look any different from how it did before.

The CSRF token is unique to every visitor on the site. If an attacker were to copy the HTML from our site, they would get their own CSRF token, which would not match that of any other user, so Django would reject the form when it was posted by someone else.

CSRF tokens also change periodically. This limits how long the attacker would have to take advantage of a particular user and token combination. Even if they were able to get the CSRF token of a user that they were trying to exploit, they would have a short window of time to use it.

We will implement a CSRF token in a form in *Exercise 6.02*.

SQL injection prevention

SQL injection attacks were a prevalent security weakness in the early days of the web. A user could inject SQL into a form by carefully crafting inputs that would add additional SQL statements to the database call that is made on the server side during the HTTP response.

This approach relied on the server-side code being poorly written so that user input was sent to the database without any control characters being escaped.

By developing our code with an ORM or other database tool, the coder is taken away from constructing raw SQL from user input, and we can eliminate this risk.

We saw how to construct queries using the Django ORM in *Chapter 3*.

Content Security Policy (CSP) and XSS attacks

Cross-site scripting (XSS) attacks are a broad group of malicious acts that involve including HTML-tagged JavaScript content in input fields that can then be rendered in the display pages of the site for malicious purposes.

Consider a text field in a form on a website that is vulnerable to XSS attacks. A malicious user might enter the following:

```
"><script>malicious_function()</script><p class="
```

If the vulnerable site does not escape the HTML tag syntax, this will be rendered as follows:

```
<input type="text" name="fname" value=""> <script>malicious_function()</script><p  
class="">
```

Fortunately, the Django Forms library has been designed to protect against this kind of attack. This input would be escaped in the following way, and the HTML elements in the input would be escaped so that no script tags are injected into the web page:

```
"><script>malicious_function()</script><p class="
```

However, it would be possible to defeat this security design if a developer were to develop a custom HTML template that specifically allowed unsafe tagged content without escaping it:

```
<input type="text" name="fname" value="UNTRUSTED DATA ">
```

HTTPS and security headers

Hosting an application over HTTP makes it vulnerable to man-in-the-middle attacks, where a third party observes traffic between the client and server and compromises security by capturing that data and replaying it to the server.

By hosting our web app over HTTPS, we are preventing this vulnerability. Our development environment uses HTTP, but we will learn more about deploying our application to HTTPS in *Chapter 18*.

A Django project comes pre-configured with the middleware that supports best practice security.

The main security middleware package is `django.middleware.security.SecurityMiddleware`, which is responsible for configuring

several important security headers in HTTP responses. The headers are configurable with variables in the project's `settings.py` file.

It implements **HTTP Strict Transport Security (HSTS)** for restricting requests to HTTPS only. When in development mode, `SECURE_HSTS_SECONDS` is set to 0, but upon deployment to production, if HTTPS is configured, then it should be set to a small value, such as 60.

Content type sniffing is a technique to ascertain the content type of a file by examining a part of the data that it contains. It can be a useful technique in some file upload situations, but it is also associated with some security exploits. By default, `SECURE_CONTENT_TYPE_NOSNIFF` is set to `True`. This is the more secure option.

Django also includes `django.middleware.clickjacking.XFrameOptionsMiddleware`. In its default mode, it will add this header to HTTP responses:

```
X-Frame-Options: DENY
```

This setting is necessary for preventing clickjacking attacks on a website.

Clickjacking is an attack that involves setting up an external website that wraps the target website with inobtrusive and often invisible HTML content. The target site is loaded as an `iframe` and the clickjack wrapper site overlays malicious options onto the target site's display, so that the user is unaware that they are clicking on an external overlaid site.

Note

You can read more about security headers in the Django documentation at <https://docs.djangoproject.com/en/5.1/topics/security/>.

With this very brief introduction to form security complete, we will commence an exercise to build a Django project with a form that we can use to demonstrate some of the concepts that we have discussed.

Exercise 6.01 – Building a form in HTML

For the first few exercises of this chapter, we will need an HTML form to test with. In this exercise, we will add an HTML form to a Django project. This will also allow you to experiment with how different fields are validated and submitted. This will be done in a new Django project so that we don't interfere with Bookr. You can refer to *Chapter 1* to refresh your memory on creating a Django project:

1. We'll start by creating the new Django project. You can reuse the Bookr virtual environment that already has Django installed. Open a new terminal and activate the

virtual environment. Then, use `django-admin` to start a Django project named `form_project`. To do this, run the following command:

```
django-admin startproject form_project
```

This will scaffold the Django project in a directory named `form_project`.

2. Create a new Django app in this project by using the `startapp` management command. The app should be called `form_example`. To do this, `cd` into the `form_project` directory, then run the following:

```
python manage.py startapp form_example
```

This will create the `form_example` app directory inside the `form_project` directory.

3. Launch PyCharm, then open the `form_project` directory. If you already have a project open, you can do this by choosing **File | Open**; otherwise, just click **Open** in the **Welcome to PyCharm** window. Navigate to the `form_project` directory, select it, then click **Open**.

If prompted, be sure to choose **Trust Project**.

The `form_project` window should be shown like this:



Figure 6.1: The `form_project` project open

4. Create a new run configuration to execute `manage.py runserver` for the project by following the **Edit Configuration** option under the **Current File** menu at the top of the window. You can reuse the virtual environment that you have already created. The **Run/Debug Configurations** window should look similar to the following figure when you're done:

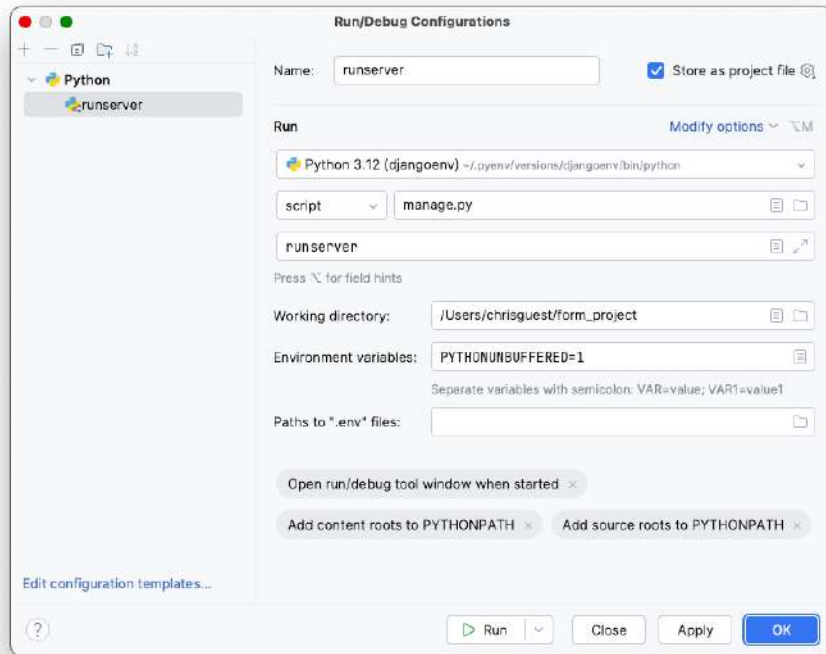


Figure 6.2: Run/Debug Configurations for runserver

You can test that the configuration has been set up correctly by clicking the **Run** button, and then visiting `http://127.0.0.1:8000/` in your browser. You should see the Django welcome screen. If the debug server fails to start or you see the Bookr main page, then you probably still have the Bookr project running. Try stopping the Bookr runserver process and then starting the new one you just set up.

5. Open `settings.py` in the `form_project` directory and add `'form_example'` to the `INSTALLED_APPS` setting.

6. The last step in setting up this new project is to create a `templates` directory for the `form_example` app. Right-click on the `form_example` directory, then select **New | Directory**. Name it `templates`.
7. We need an HTML template to display our form. We can copy the HTML for the form example from the Git repository by visiting this URL and saving the file to the `form_example` directory in the `Chapter06` folder in this book's GitHub repo. .
8. The code for this html page appears in *Figure 6.3*. Its `method` attribute is set to `post`. The `action` attribute is omitted, which means the form will be submitted back to the same URL on which it was loaded:

```
<form method="post">
```

9. Inside the `form` tag are several `<p>` tags, which wrap the form inputs and create some spacing to give the form a visually pleasing layout:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
<form method="post">
  <p><label for="id_text_input">Text Input</label><br>
    <input id="id_text_input" type="text" name="text_input"
      value="" placeholder="Enter some text"></p>
  <p><label for="id_password_input">Password Input</label><br>
    <input id="id_password_input" type="password" name="password_input"
      value="" placeholder="Your password"></p>
  <p><input id="id_checkbox_input" type="checkbox" name="checkbox_on"
    value="Checkbox Checked" checked>
    <label for="id_checkbox_input">Checkbox</label></p>
  <p><input id="id_radio_one_input" type="radio" name="radio_input" value="Value One">
    <label for="id_radio_one_input">Value One</label>
    <input id="id_radio_two_input" type="radio" name="radio_input" value="Value Two" checked>
    <label for="id_radio_two_input">Value Two</label>
    <input id="id_radio_three_input" type="radio" name="radio_input" value="Value Three">
    <label for="id_radio_three_input">Value Three</label></p>
  <p><label for="id_favorite_book">Favorite Book</label><br>
    <select id="id_favorite_book" name="favorite_book">
      <optgroup label="Non-Fiction">
        <option value="1">Deep Learning with Keras</option>
        <option value="2">Web Development with Django</option>
      </optgroup>
      <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
      </optgroup>
    </select></p>
  <p><label for="id_books_you_own">Books You Own</label><br>
    <select id="id_books_you_own" name="books_you_own" multiple>
      <optgroup label="Non-Fiction">
        <option value="1">Deep Learning with Keras</option>
        <option value="2">Web Development with Django</option>
      </optgroup>
      <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
      </optgroup>
    </select></p>
  <p><label for="id_text_area">Text Area</label><br>
    <textarea name="text_area" id="id_text_area"
      placeholder="Enter multiple lines of text"></textarea></p>
  <p><label for="id_number_input">Number Input</label><br>
    <input id="id_number_input" type="number" name="number_input"
      value="" step="any" placeholder="A number"></p>
  <p><label for="id_email_input">Email Input</label><br>
    <input id="id_email_input" type="email" name="email_input"
      value="" placeholder="Your email address"></p>
  <p><label for="id_date_input">Date Input</label><br>
    <input id="id_date_input" type="date" name="date_input" value="2019-11-23"></p>
  <p><input type="submit" name="submit_input" value="Submit Input"></p>
  <p>
    <button type="submit" name="button_element" value="Button Element">
      Button With <strong>Styled</strong> Text
    </button></p>
  <input type="hidden" name="hidden_input" value="Hidden Value">
</form>
</body>
</html>

```

Figure 6.3: form-example.html

10. As with any template, we can't see it unless we have a view to render it. Open the `form_example` app's `views.py` file and add a new view called `form_example`. It should render and return the template you just created, like so:

```
def form_example(request):  
    return render(request, "form-example.html")
```

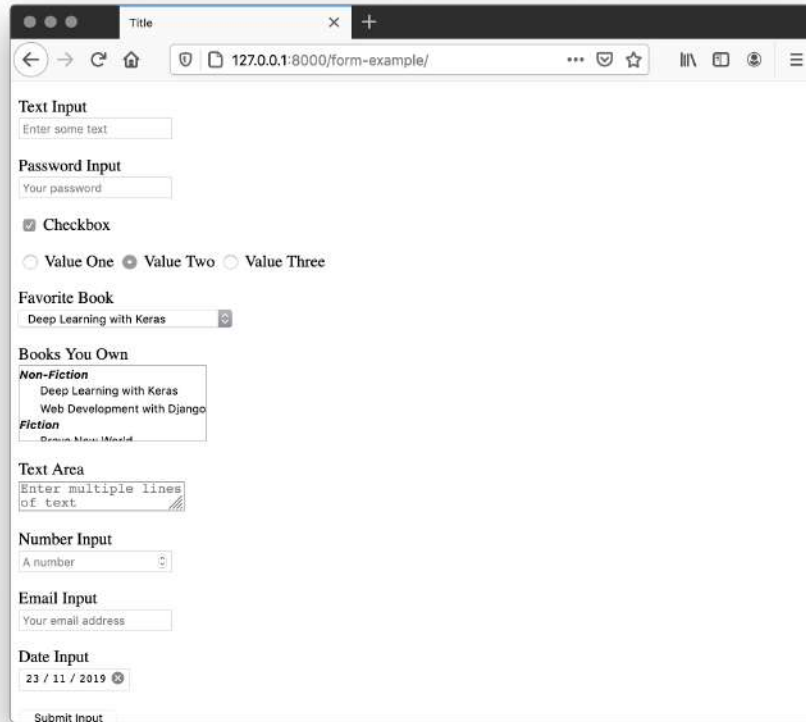
You can now save and close `views.py`.

11. You should be familiar with the next step now, which is to add a URL mapping to the view. Open the `urls.py` file in the `form_project` package directory. Add a mapping for the `form-example` path to your `form_example` view, to the `urlpatterns` variable. It should look like this:

```
path('form-example/', form_example.views.form_example)
```

Make sure you also add an import of `form_example.views`. Save and close `urls.py`.

12. Start the Django dev server (if it is not already running), then load your new view in your web browser; the address is `http://127.0.0.1:8000/form-example/`. Your page should look like this:



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/form-example/'. The form contains the following elements:

- Text Input:** A text box with the placeholder 'Enter some text'.
- Password Input:** A text box with the placeholder 'Your password'.
- Checkbox:** A checked checkbox labeled 'Checkbox'.
- Radio Buttons:** Three radio buttons labeled 'Value One', 'Value Two' (selected), and 'Value Three'.
- Favorite Book:** A dropdown menu showing 'Deep Learning with Keras'.
- Books You Own:** A section with two categories: 'Non-Fiction' (listing 'Deep Learning with Keras' and 'Web Development with Django') and 'Fiction' (listing 'Django: Keras: Mincel').
- Text Area:** A text area with the placeholder 'Enter multiple lines of text'.
- Number Input:** A text box with the placeholder 'A number'.
- Email Input:** A text box with the placeholder 'Your email address'.
- Date Input:** A date picker showing '23 / 11 / 2019'.
- Submit Input:** A button labeled 'Submit Input'.

Figure 6.4: Example inputs page

We have not yet set up everything for form submission, so if you correct all the errors in the form and try to submit it (by clicking either of the **Submit** buttons), you will receive an error stating **CSRF verification failed**, as we can see in the following figure. This demonstrates Django's CSRF protection, which we discussed in the previous section. When you receive this error, just go back in your browser to return to the input example page.



Figure 6.5: CSRF verification error

In this exercise, you created an example page showcasing many HTML inputs, then created a view to render it and a URL to map to it. You loaded the page in your browser and experimented with changing data and trying to submit the form when it contained errors.

We just saw an example of Django's CSRF protection. In the following exercise, we will fix the CSRF token error and learn about how Django handles form submissions. Before we get to that, let's have a quick refresh about accessing data in a request using QueryDict objects.

Accessing data in the view

As we discussed in *Chapter 1*, Django provides two QueryDict objects on the HTTPRequest instances that are passed to the view function. These are `request.GET`, which contains parameters passed in the URL, and `request.POST`, which contains parameters in the HTTP request body. Even though `request.GET` has GET in its name; this variable is populated even for non-GET HTTP requests. This is because the data it contains is parsed from the URL. Since all HTTP requests have a URL, all HTTP requests may contain GET data, even if they are POST or PUT, and so on.

In the next exercise, we will add code to our view to read and display the POST data.

Exercise 6.02 – Working with POST data in a view

We will now add some code to our example view to print out the received POST data to the console. We will also insert the HTTP method that was used to generate the page into the HTML output. This will allow us to be sure of what method was used to generate the page (GET or POST) and see how the form differs for each type:

1. First, in PyCharm, open the `form_example` app's `views.py` file. Alter the `form_example` view so that it prints each value in the POST to the console by adding this code inside the function:

```
for name in request.POST:
    print(f"{name}: {request.POST.getlist(name)}")
```

This code iterates over each key in the POST request data's `QueryDict` object and prints the key and list of values to the console. We already know that each `QueryDict` object can have multiple values for a key, so we use the `getlist` function to get them all.

2. Pass the template's `request.method` into a context variable named `method`. Do this by updating the call to `render` in the view so that it's like this:

```
return render(request, "form-example.html",
              {"method": request.method})
```

3. We will now display the `method` variable in the template. Open the `form-example.html` template and use a `<h4>` tag to show the `method` variable. Put this just after the opening `<body>` tag, like so:

```
<body>
    <h4>Method: {{ method }}</h4>
```

Note that we could access the `method` directly inside the template without passing it in a context dictionary, by using the `request.method` variable. We know from *Chapter 3, URL Mapping, Views, and Templates*, that by using the `render` shortcut function, the `request` is always available in the template. We just demonstrated how to access the `method` in the view here because, later on, we will change the behavior of the page based on the `method`.

4. We also need to add the CSRF token to the form HTML. We do this by putting the `{% csrf_token %}` template tag after the opening `<form>` tag. The start of the form should look like this:

```
<form method="post">
    {% csrf_token %}
```

Now, save the file.

5. Start the Django dev server if it's not already running. Load the example page (`http://127.0.0.1:8000/form-example/`) in your browser; you should see that it now displays the method at the top of the page (**GET**):



Figure 6.6: Method at the top of the page

6. Enter some text or data in each of the inputs and submit the form by clicking the **Submit Input** button:



Figure 6.7: Clicking the Submit Input button to submit the form

You should see the page reload and the method display change to **POST**:



Figure 6.8: Method updated to POST after form is submitted

7. Switch back to PyCharm and look in the **Run** console at the bottom of the window. If it is not visible, click the **Run** button at the bottom of the window to show it:



Figure 6.9: Clicking the Run button at the bottom of the window to display the console

If you're running the dev server in **Debug** mode, click **Debug** instead.

Inside the **Run** console, the list of values that were posted to the server should be displayed:



```

Quit the server with CONTROL-C.
csrfmiddlewaretoken: ['tTX0YsXDB0nSho9wcIWN557sqrWvJQeD5fVcc0J70JPamSAf6RHDk3AfJKl1raHQ']
text_input: ['Text']
password_input: ['Password']
checkbox_on: ['Checkbox Checked']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
number_input: ['154']
email_input: ['email@example.com']
date_input: ['2024-10-27']
submit_input: ['Submit Input']
hidden_input: ['Hidden Value']
[27/Oct/2024 03:58:57] "POST /form-example/ HTTP/1.1" 200 3479

```

Figure 6.10: Input values shown in the Run console

Here are some things you should note:

- All values are sent as text, even number and date inputs.
- For the select inputs, the selected value attributes of the selected options are sent, not the text content of the option tag.
- If you select multiple options for `books_you_own`, then you will see multiple values in the request. This is why we use the `getList` method since multiple values are sent for the same input name.
- If the checkbox was checked, you will have a `checkbox_on` input in the debug output. If it was not checked, then the key will not exist at all (that is, there is no key instead of having the key existing with an empty string or a `None` value).
- We have a value for the `submit_input` name, which is the text **Submit Input**. You submitted the form by clicking the **Submit Input** button, so we received its value. Notice that no value is set for the `button_element` input since that button was not clicked.

8. We will experiment with two other ways of submitting the form – first, by typing Enter when your cursor is in a text-like input (*Text*, *Password*, *Date*, or *Email*, but not *Text Area*, as typing Enter will add a new line).

If you submit a form in this way, the form will act as though you had clicked the first **Submit** button on the form, so the `submit_input` input value will be included. The output you see should match that of the previous figure.

The other way to submit the form is by clicking the `Button Element` submit input, where we will try clicking this button to submit the form. You should see that `submit_button` is no longer in the list of posted values, while `button_element` is now present:

```
csrfmiddlewaretoken: ['tTX0YsxDb0nsHo9wcIwN557sqrWvJQeD5fVCC0J70JPaMSAf6RHDk3AfJk1irahQ']
text_input: ['Text']
password_input: ['']
checkbox_on: ['Checkbox Checked']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
number_input: ['154']
email_input: ['email@example.com']
date_input: ['2024-10-27']
submit_input: ['Submit Input']
hidden_input: ['Hidden Value']
```

Figure 6.11: `submit_button` is now gone from the inputs, and `button_element` has been added

You can use this multiple-submit technique to alter how your view behaves, depending on which button was clicked. You can even have multiple submit buttons with the same name attribute to make the logic easier to write.

In this exercise, you added a CSRF token to your form element by using the `{% csrf_token %}` template tag. This means that your form can be submitted to Django successfully without it generating an **HTTP Permission Denied** response. We then added some code to output the values that our form contained when it was submitted. We tried submitting the form with various values to see how they are parsed into Python variables on the `request.POST` part of `QueryDict`. We will now discuss some more of the theory around the difference between GET and POST requests, then move on to the Django Forms library, which makes designing and validating forms easier.

Choosing between GET and POST

Choosing when to use a GET or POST request requires you to consider several factors. The most important thing is deciding whether the request should be idempotent. A request can be said to be idempotent if it can be repeated and produce the same result each time. In other words, if running it multiple times should always leave things the same, use GET. If repeating it could change data or create duplicates, use POST. Let's look at some examples.

If you type in any web address into your browser (such as any of the Bookr pages we have built so far), it will do a GET request to fetch the information. You can refresh the page, and no matter how many times you click **Refresh**, you will get the same data back. The request you are making is not affecting the content on the server. You would say these requests are idempotent.

Now, remember when you added data through the Django admin interface (in *Chapter 4*). You typed the information for the new book into a form, then clicked **Save**. Your browser made a POST request to create a new book on the server. If you repeated that POST request, the server would create *another* book and would do so each time you repeated the request. Since the request is updating information, it is not idempotent. Your browser will warn you about this. If you've ever tried to refresh a page that you were sent to after submitting a form, you may have received a message asking whether you want to "repost form data" (or something more verbose, as shown in the following figure). This is a warning that you're sending the form data again, which might cause the action you just undertook to be repeated:

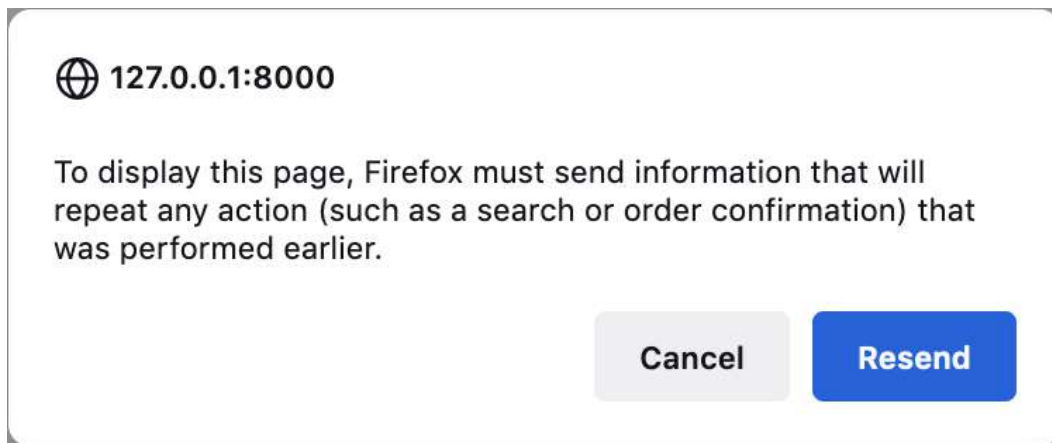


Figure 6.12: Firefox confirming whether information should be resent

This is not to suggest that all GET requests are idempotent and all POST requests are not – your backend application can be designed in any way you want. Although it is not best practice, a

developer might have decided to make data get updated during a GET request in their web application. When you are building your applications, you should try to make sure GET requests are idempotent and leave data-altering to POST requests only. Stick to these principles unless you have a good reason not to.

Another point to consider is that Django only applies CSRF protection to POST requests. Any GET request, including one that alters data, can be accessed without a CSRF token.

Sometimes, it can be hard to decide whether a request is idempotent or not – for example, a login form. Before you submitted your username and password, you were not logged in, and afterward, the server considered you to be logged in, so could we consider that non-idempotent, as it changed your authentication status with the server? On the other hand, once logged in, if you were able to send your credentials again, you would remain logged in. This implies that the request is idempotent and repeatable. So, should the request be GET or POST? This brings us to the second point to consider when choosing what method to use: form data visibility.

If sending form data with a GET request, the form parameters will be visible in the URL. For example, if we made a login form using a GET request, the login URL might be `https://www.example.com/login?username=user&password=password1`. The username, and worse, the password, are visible in the web browser's address bar. It would also be stored in the browser history, so anyone who used the browser after the real user could log in to the site. The URL is often stored in web server log files as well, meaning the credentials would be visible there, too. In short, regardless of the idempotency of a request, don't pass sensitive data through URL parameters.

Sometimes, knowing that the parameter will be visible in the URL might be something you desire. For example, when searching with a search engine, the search parameter will be visible in the URL. To see this in action, try visiting `https://www.google.com` and searching for something. You'll notice that the page with the results has your search term as the `q` parameter. For example, a search for Django will take you to the URL `https://www.google.com/search?q=Django`. This allows you to share search results with someone else by sending them this URL. In *Activity 1*, you will add a search form that passes a parameter similarly.

Another consideration is that the maximum length of a URL allowed by a browser can be short compared to the size of a POST body – sometimes, only around 2,000 characters (or about 2 KB) compared to many megabytes or gigabytes that a POST body can be (assuming your server is set up to allow these size requests).

As we mentioned earlier, URL parameters are available in `request.GET`, regardless of the type of request being made (GET, POST, PUT, and so on). You might find it useful to send some data in

URL parameters and others in the request body (available in `request.POST`). For example, you could specify a `format` argument in the URL that sets what format some output data will be transformed to, but the input data is provided in the `POST` body.

Setting parameters in the `GET` query string might seem redundant when values can be passed as part of the URL path and then parsed by Django to route the URL. Next, we'll look at some reasons why you would prefer using the query string.

Why use GET when we can put parameters in the URL mappings?

Django allows us to easily define URL maps that contain variables. We could, for example, set up a URL mapping for a search view like this:

```
path("/search/<str:search>/", reviews.views.search)
```

This probably looks like a good approach at first, but when we start wanting to customize the results view with arguments, it can get complicated quickly. For example, we might want to be able to move from one results page to the next, so we'll add a `page` argument:

```
path("/search/<str:search>/<int:page>",  
      reviews.views.search)
```

And then, we might also want to order the search results by a specific category, such as author name or date of publishing, so we'll add another argument for that:

```
path("/search/<str:search>/<int:page>/<str:order >",  
      reviews.views.search)
```

You might be able to see the problem with this approach – we can't order the results without providing a `page`. If we wanted to add a `results_per_page` argument too, we wouldn't be able to use that without setting a `page` and `order` key.

Contrast this to using query parameters – all of them are optional, so you could search like this:

```
?search=search+term:
```

Or you could set a `page` like this:

```
?search=search+term&page=2
```

Or just set the results ordering like this:

```
?search=search+term&order=author
```

Or you could combine them all:

```
?search=search+term&page=2&order=author
```

Another reason to use URL query parameters is that when submitting a form, the browser always sends the input values in this manner – it cannot be changed so that parameters are submitted as path components in the URL. Therefore, when submitting a form using GET, the URL query parameters must be used as the input data.

Now that we've introduced the fundamentals of forms in HTTP requests and HTML, let's look at the Django Forms library, which makes dealing with forms and requests much easier.

The Django Forms library

We've looked at how to manually write forms in HTML and how to access the data on the request object using `QueryDict`. We saw that the browser provides some validation for us for certain field types (such as email or numbers), but we have not tried validating the data in the Python view. We should validate the form in the Python view for two reasons:

- It is not safe to rely solely on browser-based validation of input data. A browser may not implement certain validation features, meaning the user could post any type of data. For example, older browsers don't validate number fields, so a user can type in a number outside the range we are expecting. Further, a malicious user could try to send harmful data without using a browser at all. The browser validation should be considered a nicety for the user, and that's all.
- The browser does not allow us to do cross-field validation. For example, we can use the `required` attribute for inputs that are mandatory to be filled in. Often, though, we want to set the `required` attribute based on the value of another input. For example, the email address input should only be set as required if the user has checked the **Register My Email** checkbox.

The Django Forms library allows you to quickly define a form using a Python class. This is done by creating a subclass of the base Django Form class. You can then use an instance of this class to render the form in your template and validate the input data. We refer to our classes as forms, similar to how we subclass Django models to create `Model` classes. Forms contain one or more fields of a certain type (such as text fields, number fields, or email fields). You'll notice

this sounds like Django models, and forms *are* similar to models but use different field classes. You can even automatically create a form from a model – we will cover this in *Chapter 7*.

Defining a form

Creating a Django form is similar to creating a Django model: you define a class that inherits from the `django.forms.Form` class. The class has attributes that are instances of different `django.forms.Field` subclasses. When rendered, the attribute name in the class corresponds to its input name in HTML. To give you a quick idea of what fields there are, some examples are `CharField`, `IntegerField`, `BooleanField`, `ChoiceField`, and `DateField`. Each field generally corresponds to one input when rendered in HTML, but there's not always a one-to-one mapping between a form field class and an input type. Form fields are more coupled to the type of data they collect rather than how they are displayed.

To illustrate this, consider a text input and a password input. They both accept some typed-in text data, but the main difference between them is that the text is displayed in a text input, while in a password input, the text is obscured. In a Django form, both of these fields are represented using `CharField`. Their difference in how they are displayed is set by changing the widget the field is using.

Note

If you're not familiar with the word *widget*, it's a term to describe the actual input that is being interacted with and how it is displayed. Text inputs, password inputs, select menus, checkboxes, and buttons are all examples of different widgets. The inputs we have seen in HTML correspond one-to-one with widgets. In Django, this is not the case, and the same type of `Field` class can be rendered in multiple ways, depending on the widget that is specified.

Django defines several widget classes that define how a field should be rendered as HTML. They inherit from `django.forms.widgets.Widget`. A widget can be passed to the `Field` constructor to change how it is rendered. For example, `CharField` renders as text `<input>` by default. If we use the `PasswordInput` widget, it will instead render as password `<input>`. The other widgets we will use are as follows:

- `RadioSelect`, which renders `ChoiceField` as radio buttons instead of a `<select>` menu
- `Textarea`, which renders `CharField` as `<textarea>`
- `HiddenInput`, which renders a field as a hidden input

We'll look at an example form and add fields and features one by one. First, let's just create a `forms.py` file containing a form with a text input and a password input:

```
from django import forms

class ExampleForm(forms.Form):
    text_input = forms.CharField()
    password_input = forms.CharField(
        widget=forms.PasswordInput
    )
```

The widget argument can be just a widget subclass, which can be fine a lot of the time. If you want to further customize the display of the input and its attributes, you can set the widget argument to an instance of the widget class instead. We will look at customizing the widget displays further soon. In this case, we're using just the `PasswordInput` class, since we are not customizing beyond changing the type of input being displayed.

To provide a form to the user, it must be made available to the template through a view's context. Here is an example:

```
from django.shortcuts import render
from .forms import ExampleForm

def example_view(request):
    form = ExampleForm() # empty form for GET
    return render(request, "example.html", {"form": form})
```

When the form is rendered in a template, it looks like this:

Text input:

Password input:

Figure 6.13: Django form rendered in the browser

Note that the inputs do not contain any content when the page loads; the text has been entered to illustrate the different input types.

If we examine the page source, we will see the HTML that Django generates. For the first two fields, it looks like this (some spacing has been added for readability):

```
<p>
  <label for="id_text_input">Text input:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</p>
<p>
  <label for="id_password_input">Password input:</label>
  <input type="password" name="password_input" required
    id="id_password_input">
</p>
```

Notice that Django has automatically generated a label with text derived from the field name. The name and id attributes have been set automatically. Django also automatically adds the required attribute to the input. Similar to model fields, form field constructors also accept a required argument – this defaults to True. Setting this to False removes the required attribute from the generated HTML.

Next, we'll look at how a checkbox is added to the form.

A checkbox is represented with BooleanField as it can have only two values: checked or unchecked. It's added to the form in the same way as the other field:

```
class ExampleForm(forms.Form):
    ...
    checkbox_on = forms.BooleanField()
```

The HTML Django generates for this new field is similar to the previous two fields:

```
<label for="id_checkbox_on">Checkbox on:</label>
<input type="checkbox" name="checkbox_on" required
  id="id_checkbox_on">
```

Next are select inputs:

- We need to provide a list of choices to display in the <select> dropdown.
- The field class constructor takes a choices argument. The choices are provided as a tuple of two-element tuples. The first element in each sub-tuple is the value of the

choice, while the second element is the text or description of the choice. For example, choices could be defined like this:

```
BOOK_CHOICES = (  
    ("1", "Deep Learning with Keras"),  
    ("2", "Web Development with Django"),  
    ("3", "Brave New World"),  
    ("4", "The Great Gatsby")  
)
```

Note that you can use lists instead of tuples if you want (or a combination of the two). This can be useful if you want your choices to be mutable:

```
BOOK_CHOICES = (  
    ["1", "Deep Learning with Keras"],  
    ["2", "Web Development with Django"],  
    ["3", "Brave New World"],  
    ["4", "The Great Gatsby"]  
)
```

- To implement `optgroup`, we can nest the choices. To implement the choices as we did in our previous examples, we can use a structure like this:

```
BOOK_CHOICES = (  
    (  
        "Non-Fiction", (  
            ("1", "Deep Learning with Keras"),  
            ("2", "Web Development with Django")  
        )  
    ),  
    (  
        "Fiction", (  
            ("3", "Brave New World"),  
            ("4", "The Great Gatsby")  
        )  
    )  
)
```

The select element is added to the form by using `ChoiceField`. The widget defaults to a select input, so that no configuration is necessary apart from setting choices:

```
class ExampleForm(forms.Form):
    ...
    favorite_book =
        forms.ChoiceField(choices=BOOK_CHOICES)
```

This is the HTML that is generated:

```
<label for="id_favorite_book">Favorite book:</label>
<select name="favorite_book" id="id_favorite_book">
  <optgroup label="Non-Fiction">
    <option value="1">Deep Learning with Keras
    </option>
    <option value="2">Web Development with Django
    </option>
  </optgroup>
  <optgroup label="Fiction">
    <option value="3">Brave New World</option>
    <option value="4">The Great Gatsby</option>
  </optgroup>
</select>
```

- Making a multiple select requires the use of `MultipleChoiceField`. It takes a choices argument in the same format as the regular `ChoiceField` for single selects:

```
class ExampleForm(forms.Form):
    ...
    books_you_own =
        forms.MultipleChoiceField(choices=BOOK_CHOICES)
```

And its HTML is similar to that of the single select, except it has the `multiple` attribute added:

```
<label for="id_books_you_own">Books you own:</label>
<select name="books_you_own" required
  id="id_books_you_own" multiple>
  <optgroup label="Non-Fiction">
    <option value="1">Deep Learning with Keras
```

```
        </option>
        <option value="2">Web Development with Django
        </option>
    </optgroup>
    <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
    </optgroup>
</select>
```

- Choices can also be set after the form has been instantiated. You may want to generate the list/tuple choice inside your view dynamically and then assign it to the field's choices attribute, like so:

```
form = ExampleForm()
form.fields["books_you_own"].choices = [("1", "Deep
Learning with Keras"), ...]
```

Next are radio inputs, which are similar to selects:

- Like selects, radio inputs use ChoiceField, as they provide a single choice between multiple options.
- The options to choose between are passed into the field constructor with the choices argument.
- The choices are provided as a tuple of two-element tuples, also like selects:

```
choices = (
    ("1", "Option One"),
    ("2", "Option Two"),
    ("3", "Option Three")
)
```

- ChoiceField defaults to displaying as a select input, so the widget must be set to RadioSelect to have it rendered as radio buttons. Putting the choice setting together with this, we can add radio buttons to the form like this:

```
RADIO_CHOICES = (
    ("Value One", "Value One"),
    ("Value Two", "Value Two"),
    ("Value Three", "Value Three")
)
```

```
)

class ExampleForm(forms.Form):
    ...
    radio_input =
        forms.ChoiceField(choices=RADIO_CHOICES,
                           widget=forms.RadioSelect)
```

Here is the HTML that is generated:

```
<label for="id_radio_input_0">Radio input:</label>
<ul id="id_radio_input">
<li>
    <label for="id_radio_input_0">
        <input type="radio" name="radio_input"
            value="Value One" required
            id="id_radio_input_0">
        Value One
    </label>
</li>
<li>
    <label for="id_radio_input_1">
        <input type="radio" name="radio_input"
            value="Value Two" required
            id="id_radio_input_1">
        Value Two
    </label>
</li>
<li>
    <label for="id_radio_input_2">
        <input type="radio" name="radio_input"
            value="Value Three" required
            id="id_radio_input_2">
        Value Three
    </label>
</li>
</ul>
```

Django automatically generates a unique label and ID for each of the three radio buttons.

- To create a text area, use CharField with a Textarea widget:

```
class ExampleForm(forms.Form):  
    ...  
    text_area = forms.CharField(widget=forms.Textarea)
```

You might notice that this text area is much larger than the previous ones we have seen (see the following figure):



Figure 6.14: Normal text area (top) versus Django default text area (bottom)

This is because Django automatically adds `cols` and `rows` attributes. These set the number of columns and rows that the text field displays, respectively:

```
<label for="id_text_area">Text area:</label>  
<textarea name="text_area" cols="40" rows="10"  
required id="id_text_area"></textarea>
```

- Note that the `cols` and `rows` settings do not affect the amount of text that can be entered into a field, only the amount that is displayed at a time. Also, note that the size of `textarea` can be set using CSS (for example, the `height` and `width` properties). This will override the `cols` and `rows` settings.

To create number inputs, you might expect Django to have a `NumberField` type, but it does not.

Remember that the Django form fields are data-centric rather than display-centric, so instead, Django provides different `Field` classes, depending on what type of numeric data you want to store:

- For integers, use `IntegerField`.
- For floating point numbers, use `FloatField` or `DecimalField`. The latter two differ in how they convert their data into a Python value. `FloatField` will convert into a float, while `DecimalField` is a decimal.
- Decimal values offer better accuracy in representing numbers than float values, but may not integrate well into your existing Python code.

We'll add all three fields to the form at once:

```
class ExampleForm(forms.Form):  
    ...  
    integer_input = forms.IntegerField()  
    float_input = forms.FloatField()  
    decimal_input = forms.DecimalField()
```

Here's the HTML for all three:

```
<p>  
    <label for="id_integer_input">Integer input:</label>  
    <input type="number" name="integer_input" required  
        id="id_integer_input">  
</p>  
<p>  
    <label for="id_float_input">Float input:</label>  
    <input type="number" name="float_input" step="any"  
        required id="id_float_input">  
</p>  
<p>  
    <label for="id_decimal_input">Decimal input:</label>  
    <input type="number" name="decimal_input" step="any"  
        required id="id_decimal_input">  
</p>
```

The generated HTML of `IntegerField` is missing the `step` attribute that the other two have, which means the widget will only accept integer values.

The other two fields (`FloatField` and `DecimalField`) generate very similar HTML; their behavior is the same in the browser, and they differ only when their values are used in Django code.

As you might have guessed, an email input can be created with `EmailField`:

```
class ExampleForm(forms.Form):  
    ...  
    email_input = forms.EmailField()
```

Its HTML is similar to the email input we created manually:

```
<label for="id_email_input">Email input:</label>  
<input type="email" name="email_input" required  
      id="id_email_input">
```

Continuing with our manually created form, the next field we will look at is `DateField`:

- By default, Django will render `DateField` as a text input, and the browser will not show a calendar popup when the field is clicked on.

We can add `DateField` to the form with no arguments, like this:

```
class ExampleForm(forms.Form):  
    ...  
    date_input = forms.DateField()
```

When rendered, it just looks like a normal text input:

Date input:

Figure 6.15: Default `DateField` display in the form

Here is the HTML generated by default:

```
<label for="id_date_input">Date input:</label>  
<input type="text" name="date_input" required  
      id="id_date_input">
```

- The reason for using a text input is that it allows the user to enter the date in several different formats. For example, by default, the user can type in the date in *Year-Month-Day* (dash-separated) or *Month/Day/Year* (slash-separated) formats.

- The accepted formats can be specified by passing a list of formats to the `DateField` constructor using the `input_formats` argument. For example, we could accept dates in the *Day/Month/Year* or *Day/Month/Year-with-century* format, like this:

```
DateField(input_formats = ["%d/m/%y", "%d/%m/%Y"])
```

- We can override any attributes on a field's widget by passing the `attrs` argument to the widget constructor. This accepts a dictionary of attribute keys/values that will be rendered into the input's HTML.
- We have not used this yet, but we will see it again in the next chapter when we customize the field rendering further. For now, we'll just set one attribute, `type`, that will overwrite the default input type:

```
class ExampleForm(forms.Form):  
    ...  
    date_input = forms.DateField(  
        widget=forms.DateInput(  
            attrs={"type": "date"})
```

When rendered, it now looks like the date field we had before, and clicking on it brings up the calendar date picker:

Date input:

Figure 6.16: `DateField` with date input

Examining the generated HTML now, we can see it uses the date type:

```
<label for="id_date_input">Date input:</label>  
<input type="date" name="date_input" required  
      id="id_date_input">
```

The final input that we are missing is the hidden input:

- Once again, due to the data-centric nature of Django forms, there is no `HiddenField`.
- Instead, we choose the type of field that needs to be hidden and set its widget to `HiddenInput`. We can then set the value of the field using the field constructor's `initial` argument:

```
class ExampleForm(forms.Form):  
    ...  
    hidden_input = forms.CharField(  
        widget=forms.HiddenInput, initial="Hidden Value")
```

Here is the generated HTML:

```
<input type="hidden" name="hidden_input"  
value="Hidden Value" id="id_hidden_input">
```

Note that, as this is a hidden input, Django does not generate a label or surrounding `p` elements.

There are other form fields that Django provides that work in similar ways. These range from `DateTimeField` (for capturing a date and a time) to `GenericIPAddressField` (for either IPv4 or IPv6 addresses) and `URLField` (for URLs). A full list of fields is available at <https://docs.djangoproject.com/en/5.1/ref/forms/fields/>.

Rendering a form in a template

We've now seen how to create a form and add fields, and we've seen what the form looks like and what HTML is generated. But how is the form rendered in the template? We simply instantiate the form class and pass it to the render function in a view, using the context, just like any other variable.

For example, here's how we can pass `ExampleForm` to a template:

```
def view_function(request):  
    form = ExampleForm()  
    return render(request, "template.html", {"form": form})
```

Django does not add the `<form>` element or submit button(s) for you when rendering the template; you should add these around where your form is placed in the template. The form can be rendered like any other variable.

We mentioned briefly earlier that the form was being rendered in the template using the `as_p` method. This layout method was chosen as it most closely matches the example form we built manually. Django offers four layout methods that can be used:

- `as_table`: The form is rendered as table rows, with each input on its own row. Django does not generate the surrounding table element, so you should wrap the form yourself. Here is an example:

```
<form method="post">
  <table>
    {{ form.as_table }}
  </table>
</form>
```

`as_table` is the default rendering method, so `{{ form.as_table }}` and `{{ form }}` are equivalent.

When rendered, the form looks like this:

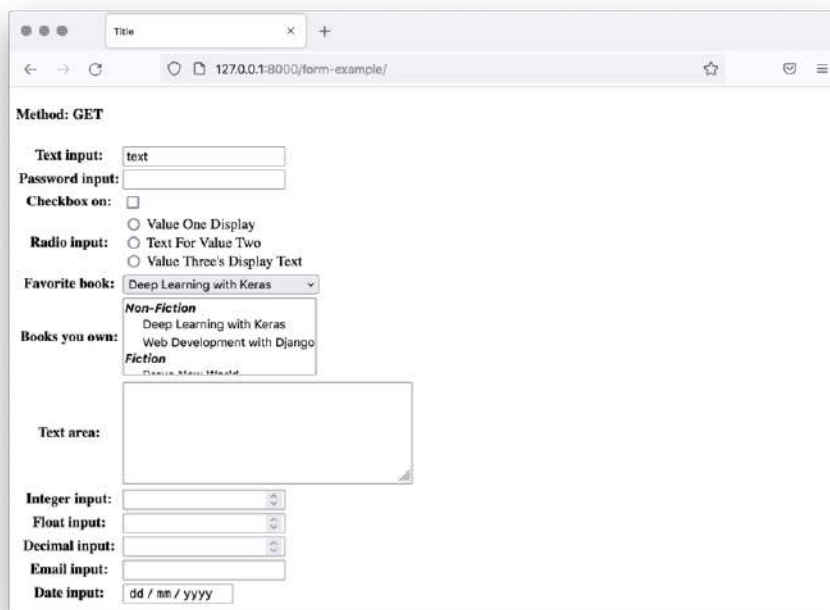


Figure 6.17: Form rendered as a table

Here is a small sample of HTML that is generated:

```
<tr>
  <th>
    <label for="id_text_input">Text input:</label>
  </th>
  <td>
    <input type="text" name="text_input" required
      id="id_text_input">
    </td>
</tr>
<tr>
  <th>
    <label for="id_password_input">Password input:</label>
  </th>
  <td>
    <input type="password" name="password_input"
      required id="id_password_input">
    </td>
</tr>
```

- `as_ul`: This renders the form fields as list items (`li`) inside either a `ul` or `ol` element. Like with `as_table`, the containing element (`<ul>` or `<ol>`) is not created by Django and must be added by you:

```
<form method="post">
  <ul>
    {{ form.as_ul }}
  </ul>
</form>
```

Here's how the form renders using `as_ul`:

Method: GET

- Text input:
- Password input:
- Checkbox on: ☐
- Radio input:
 - ☐ Value One Display
 - ☐ Text For Value Two
 - ☐ Value Three's Display Text
- Favorite book:
 - Non-Fiction
 - Deep Learning with Keras
 - Web Development with Django
 - Fiction
 - ...
- Books you own:
- Text area:
- Integer input:
- Float input:
- Decimal input:
- Email input:
- Date input:

Submit Input

Figure 6.18: Form rendered using `as_ul`

And here's a sample of the generated HTML:

```
<li>
  <label for="id_text_input">Text input:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</li>
<li>
  <label for="id_password_input">Password input:</label>
  <input type="password" name="password_input"
    required id="id_password_input">
</li>
```

- `as_p`: Then there's the `as_p` method, which we used in our previous examples. Each input is wrapped within `p` tags, which means that you don't have to wrap the form manually (in `<table>` or `<ul>`) as you did with the previous methods:

```
<form method="post">
    {{ form.as_p }}
</form>
```

Here's what the rendered form looks like:

Figure 6.19: Form rendered using `as_p`

You've seen this before, but once again, here's a sample of the HTML generated:

```
<p>
  <label for="id_text_input">Text input:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</p>
<p>
  <label for="id_password_input">Password input:</label>
```



```
<input type="password" name="password_input"
      required id="id_password_input">
</p>
```

- `as_field_group`: Django also offers more fine-grained control of layout so that individual form elements can be laid out in a custom design without constraining them to the table, ul, or p structures. `as_field_group` is a method of the field object rather than the form object. It can be called from the individual fields in the form like this:

```
<form method="post">
  {{ form.field.text_input.as_field_group }} <br />
  {{ form.field.password_input.as_field_group }} <br />
</form>
```

This will be rendered as follows:

```
<form method="post">
<label for="id_text_input">Text input:</label>
<input type="text" name="text_input" required id="id_text_input"> <br />
<label for="id_password_input">Password input:</label>
<input type="password" name="password_input" required
id="id_password_input"> <br />
</form>
```

If you wanted to generate a form with fields separated by a div element, you could iterate through the fields of the form like this in the template:

```
<form method="post">
  {% csrf_token %}
  {% for field in form %}
    <div class="field_wrapper"
      id="field_group_{{field.name}}">
      {{ field.as_field_group }}
    </div>
  {% endfor %}
</form>
```

It is up to you to decide which method you want to use to render your form, depending on which suits your application best. In terms of their behavior and use in your view, they are all

identical. In *Chapter 15*, we will also introduce a method of rendering forms that will make use of the Bootstrap CSS classes.

Now that you have been introduced to Django forms, we can update our example form page to use a Django form instead of manually writing all the HTML. We'll do this in the next exercise by replacing hand-written HTML with a Django form.

Exercise 6.03 – Building and rendering a Django form

In this exercise, you will build a Django form using all the fields we have seen. The form and view will behave similarly to the form built manually; however, you will be able to see how much less code is required when writing forms using Django. Your form will also automatically get field validation, and if we make changes to the form, we don't have to then make changes to the HTML, as it will update dynamically based on the form definition:

1. In PyCharm, create a new file called `forms.py` inside the `form_example` app directory.
2. Import the Django forms library at the top of your `forms.py` file:

```
from django import forms
```

3. Define the choices for the radio buttons by creating a `RADIO_CHOICES` variable. Populate it as follows:

```
RADIO_CHOICES = (  
    ("Value One", "Value One Display"),  
    ("Value Two", "Text For Value Two"),  
    ("Value Three", "Value Three's Display Text"),  
)
```

You will use this soon when you create a `ChoiceField` called `radio_input`.

4. Define the nested choices for the book and select inputs by creating a `BOOK_CHOICES` variable. Populate it as follows:

```
BOOK_CHOICES = (  
    (  
        "Non-Fiction", (  
            ("1", "Deep Learning with Keras"),  
            ("2", "Web Development with Django")  
        )  
    ),  
    (  

```

```
        "Fiction", (  
            ("3", "Brave New World"),  
            ("4", "The Great Gatsby")  
        )  
    )  
)
```

5. Create a class called `ExampleForm`, which inherits from the `forms.Form` class:

```
class ExampleForm(forms.Form):
```

Add the following fields as attributes to the class:

```
    text_input = forms.CharField()  
    password_input = forms.CharField(  
        widget=forms.PasswordInput)  
    checkbox_on = forms.BooleanField()  
    radio_input = forms.ChoiceField(  
        choices=RADIO_CHOICES,  
        widget=forms.RadioSelect)  
    favorite_book = forms.ChoiceField(  
        choices=BOOK_CHOICES)  
    books_you_own = forms.MultipleChoiceField(  
        choices=BOOK_CHOICES)  
    text_area = forms.CharField(widget=forms.Textarea)  
    integer_input = forms.IntegerField()  
    float_input = forms.FloatField()  
    decimal_input = forms.DecimalField()  
    email_input = forms.EmailField()  
    date_input = forms.DateField(  
        widget=forms.DateInput(  
            attrs={"type": "date"})  
        )  
    hidden_input = forms.CharField(  
        widget=forms.HiddenInput,  
        initial="Hidden Value")
```

Save the file.

6. Open your `form_example` app's `views.py` file. At the top of the file, add a line to import `ExampleForm` from your `forms.py` file:

```
from .forms import ExampleForm
```

7. Inside the `form_example` view, instantiate the `ExampleForm` class and assign it to the `form` variable:

```
form = ExampleForm()
```

8. Add the `form` variable to the context dictionary using the `form` key. The return line should look like this:

```
return render(request, "form-example.html",
              {"method": request.method, "form": form})
```

Save the file.

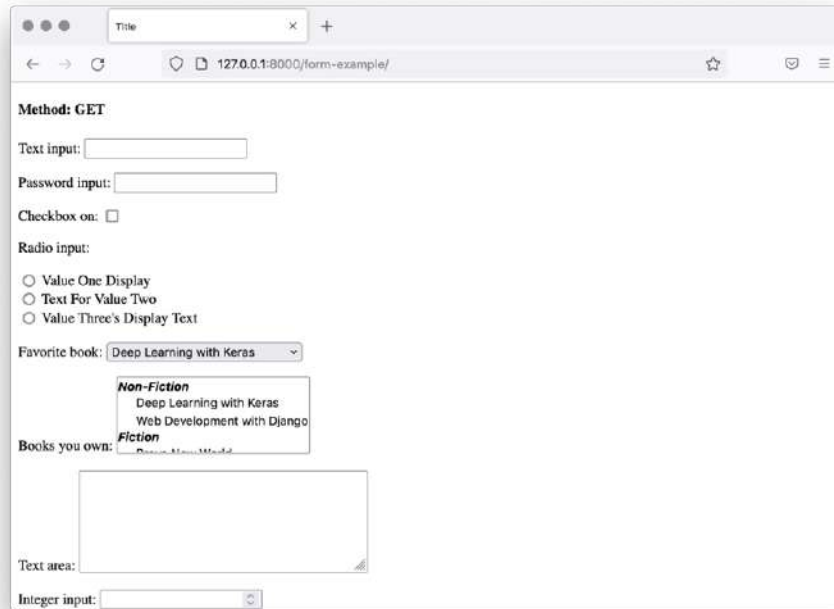
Make sure you haven't removed the code that prints out the data the form has sent, as we will use it again later in this exercise.

9. Open the `form-example.html` file inside the `form_example` app's templates directory. You can remove nearly all of the contents of the `form` element, except the `{% csrf_token %}` template tag and the submit buttons. Add a rendering of the `form` variable using the `as_p` method after the `{% csrf_token %}` template tag. The whole `form` element should now look like this:

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <p>
    <input type="submit" name="submit_input"
      value="Submit Input">
  </p>
  <p>
    <button type="submit" name="button_element"
      value="Button Element">
      Button With <strong>Styled</strong> Text
    </button>
```

```
</p>  
</form>
```

10. Start the Django dev server if it is not already running, then visit the form example page in your browser, at `http://127.0.0.1:8000/form-example/`. It should look as follows:



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000/form-example/`. The page content is as follows:

- Method: GET**
- Text input:** A single-line text input field.
- Password input:** A single-line password input field.
- Checkbox on:** A checkbox that is currently unchecked.
- Radio input:** Three radio buttons with labels: "Value One Display", "Text For Value Two", and "Value Three's Display Text".
- Favorite book:** A dropdown menu with "Deep Learning with Keras" selected. A tooltip is visible showing a list of books: "Non-Fiction" (Deep Learning with Keras, Web Development with Django) and "Fiction" (The Great Gatsby, The Catcher in the Rye).
- Books you own:** A text area for a multi-line input.
- Text area:** A single-line text input field.
- Integer input:** A single-line integer input field.

Figure 6.20: Django example form rendered in the browser

11. Enter some data into the form – since Django marks all fields as required, you will need to enter some text or select values for all fields, including ensuring that the checkbox is checked. Submit the form.
12. Switch back to PyCharm and look in the **Debug** console at the bottom of the window. You should see that all the values being submitted by the form are printed out to the console, similar to in *Exercise 6.02*:

```
csrfmiddlewaretoken: ['HZ222KpXGzT2PLxMSIfxwHXcgyTtk3qSjl0Eg6Brvu1KUfYvmRqnLFqZzrYg2nt5']
text_input: ['Text']
password_input: ['Password']
checkbox_on: ['on']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
integer_input: ['154']
float_input: ['11.0']
decimal_input: ['1234']
email_input: ['user@example.com']
date_input: ['2024-10-27']
hidden_input: ['Hidden Value']
submit_input: ['Submit Input']
```

Figure 6.21: Values as submitted by the Django form

You can see that the values are still strings, and the names match those of the attributes of the `ExampleForm` class. Notice that the submit button that you clicked is included, as well as the CSRF token. The form you submit can be a mix of Django form fields and arbitrary fields you add; both will be contained in the `request.POST` `QueryDict` object.

In this exercise, you created a Django form with many different types of form fields. You instantiated it into a variable in your view, then passed it to `form-example.html`, where it was rendered as HTML. Finally, you submitted the form and looked at the values it posted. Notice that the amount of code we had to write to generate the same form was greatly reduced. We did not have to manually code any HTML, and we now have one place that defines how the form will display and how it will validate.

In the next section, we will examine how Django forms can automatically validate the submitted data, as well as how the data is converted from strings into Python objects.

Validating forms and retrieving Python values

So far, we have seen how Django forms make it much simpler to define a form using Python code and have it automatically rendered. We will now look at the other part of what makes Django forms useful: their ability to automatically validate the form and then retrieve native Python objects and values from them.

In Django, a form can either be *unbound* or *bound*. These terms describe whether or not the form has had the submitted POST data sent to it for validation. So far, we have only seen unbound forms – they are instantiated without arguments, like this:

```
form = ExampleForm()
```

A form is bound if it is called with some data to be used for validation, such as the POST data. A bound form can be created like this:

```
form = ExampleForm(request.POST)
```

A bound form allows us to start using built-in validation-related tools on the form instance. First, there's the `is_valid` method, which checks the form's validity, and then the `cleaned_data` attribute, which contains the values converted from strings into Python objects. The `cleaned_data` attribute is only available after the form has been *cleaned*, which is the process of cleaning up the data and converting it from strings into Python objects. The cleaning process runs during the `is_valid` call. An `AttributeError` will be raised if you try to access `cleaned_data` before calling `is_valid`.

A short example of how to access the cleaned data of `ExampleForm` looks like this:

```
form = ExampleForm(request.POST)

if form.is_valid():
    # cleaned_data is only populated if the form is valid
    if form.cleaned_data["integer_input"] > 5:
        do_something()
```

In this example, `form.cleaned_data["integer_input"]` is the integer value, 10, so it can be compared to the number 5. Compare this to the value that was posted, which is the string 10. The cleaning process performs this conversion for us. Other fields, such as dates or Booleans, are converted accordingly.

The cleaning process also sets any errors on the form and fields, which will be displayed when the form is rendered again. Let's see all this in action. Modern browsers provide a large amount of client-side validation, so they prevent forms from being submitted unless their basic validation rules are met. You might have already seen this if you tried to submit the form in the previous exercise with empty fields:

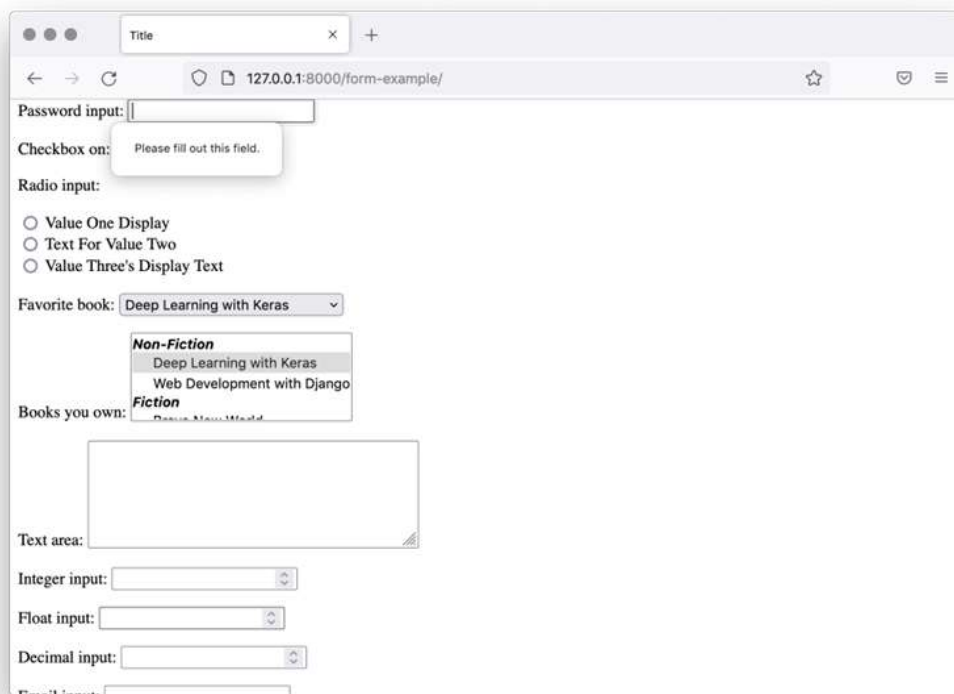


Figure 6.22: Form submission prevented by the browser

Figure 6.22 shows the browser preventing form submission. Since the browser is preventing the submission, Django never gets the opportunity to validate the form itself. To allow the form to be submitted, we need to add some more advanced validation that the browser is unable to validate itself. We will discuss the different types of validations that can be applied to form fields in the next section, but for now, we will just add a `max_digits` setting of 3 to `decimal_input` of `ExampleForm`. This means the user should not enter more than three digits in the form.

Note

Why should Django validate the form if the browser is already doing this and preventing submission? A server-side application should never trust input from the user: the user might be using an older browser or other HTTP clients to send the request, thus not receiving any errors from their browser. Also, as we have just mentioned, there are types of validation that the browser does not understand, so Django must validate these on its end.

ExampleForm can be updated like this:

```
class ExampleForm(forms.Form):  
    ...  
    decimal_input = forms.DecimalField(max_digits=3)  
    ...
```

Now, the view should be updated to pass `request.POST` to the form class when the method is `POST`, for example, like this:

```
if request.method == "POST":  
    form = ExampleForm(request.POST)  
else:  
    form = ExampleForm()
```

If you pass `request.POST` into the form constructor when the method is not `POST`, then the form will always contain errors when first rendered, as `request.POST` will be empty.

Now, the browser will let us submit the form, but we will get an error displayed if `decimal_input` contains more than three digits:

- **Ensure that there are no more than 3 digits in total.**

Decimal input:

Figure 6.23: An error is displayed when a field is not valid

Django automatically renders the form differently in the template when it has errors. But how can we make the view behave differently depending on the validity of the form? As we

mentioned earlier, we should use the form's `is_valid` method. A view using this check might contain code like this:

```
form = ExampleForm(request.POST)

if form.is_valid():
    # perform operations from with data from
    # form.cleaned_data
    return redirect("/success-page")
    # redirect to a success page
```

In this example, we are redirecting to a success page if the form is valid. Otherwise, we assume the execution flow continues as before and pass the invalid form back to the render function to be displayed to the user with errors.

Note

Why do we return a redirect on success? For two reasons: first, an early return prevents the execution of the rest of the view (that is, the failure branch); second, it prevents the message about resending the form data if the user then reloads the page.

In the next exercise, we will see the form validation in action and change the view's execution flow based on the validity of the form.

Exercise 6.04 – Validating forms in a view

In this exercise, we will update the example view to instantiate the form differently depending on the HTTP method. We will also change the form so that it prints out the cleaned data instead of the raw POST data, but only if the form is valid:

1. In PyCharm, open the `forms.py` file inside the `form_example` app directory. Add a `max_digits=3` argument to the `decimal_input` of `ExampleForm`:

```
class ExampleForm(forms.Form):
    ...
    decimal_input = forms.DecimalField(max_digits=3)
```

Once this argument has been added, we can submit the form since the browser does not know how to validate this rule, but Django does.

2. Open the reviews app's `views.py` file. We need to update the `form_example` view so that if the request's method is `POST`, `ExampleForm` is instantiated with the `POST` data; otherwise, it's instantiated without arguments. Replace the current form initialization with this code:

```
def form_example(request):
    if request.method == "POST":
        form = ExampleForm(request.POST)
    else:
        form = ExampleForm()
```

3. Next, also for the `POST` request method, we will check whether the form is valid using the `is_valid` method. If the form is valid, we will print out all the cleaned data. Add a condition after the `ExampleForm` instantiation to check `form.is_valid()`, then move the debug print loop inside this condition. Your `POST` branch should look like this:

```
if request.method == "POST":
    form = ExampleForm(request.POST)
    if form.is_valid():
        for name in request.POST:
            print(
                f"{name}: {request.POST.getlist(name)}")
```

4. Instead of iterating over the raw `request.POST` `QueryDict` (in which all the data is strings), we will iterate over the form's `cleaned_data`. This is a normal dictionary and contains the values converted into Python objects. Replace the `for` line and `print` line with these two:

```
for name, value in form.cleaned_data.items():
    print(f"{name}: ({type(value)}) {value}")
```

We don't need to use `getlist()` anymore since `cleaned_data` has already converted multi-value fields into lists.

5. Start the Django dev server, if it's not already running. Switch to your browser and browse to the example form page at `http://127.0.0.1:8000/form-example/`. The form should look as it did before. Fill in all the fields, but make sure that you enter four or more numbers into the **Decimal input** field to make the form invalid. Submit the

form; you should see the error message for the **Decimal input** field show up when the page refreshes:

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/form-example/'. The form contains the following elements:

- Favorite book:** A dropdown menu with 'Deep Learning with Keras' selected. A tooltip shows options: 'Non-Fiction' (with sub-items 'Deep Learning with Keras' and 'Web Development with Django') and 'Fiction'.
- Books you own:** A dropdown menu with 'Fiction' selected.
- Text area:** A text input field containing 'Test Value'.
- Integer input:** A numeric input field containing '10'.
- Float input:** A numeric input field containing '11.0'.
- Decimal input:** A numeric input field containing '1234'. Below it is an error message: '• Ensure that there are no more than 3 digits in total.'
- Email input:** A text input field containing 'user@example.com'.
- Date input:** A date input field showing '20 / 04 / 2022'.
- Buttons:** 'Submit Input' and 'Button With **Styled** Text'.

Figure 6.24: Decimal input error displayed after the form is submitted

- Fix the form errors by making sure only three digits are in the **Decimal input** field, then submit the form again. Switch back to PyCharm and check the **Debug** console. You should see that all the cleaned data has been printed out:

```
text_input: (<class 'str'>) Text
password_input: (<class 'str'>) Password
checkbox_on: (<class 'bool'>) True
radio_input: (<class 'str'>) Value Two
favorite_book: (<class 'str'>) 1
books_you_own: (<class 'list'>) ['1', '3']
text_area: (<class 'str'>) Text Area
integer_input: (<class 'int'>) 154
float_input: (<class 'float'>) 11.0
decimal_input: (<class 'decimal.Decimal'>) 1234
email_input: (<class 'str'>) user@example.com
date_input: (<class 'datetime.date'>) 2024-10-27
hidden_input: (<class 'str'>) Hidden Value
```

Figure 6.25: The cleaned data from the form has been printed out

Notice the conversions that have taken place. `CharField` has been converted into `str`, `BooleanField` into `bool`, and `IntegerField`, `FloatField`, and `DecimalField` into `int`, `float`, and `Decimal`, respectively. `DateTimeField` becomes a `datetime.date`, and the choice fields will retain the string values of their initial choice values. Notice that `books_you_own` is automatically converted into a list of `str`.

Also, note that, unlike when we iterated over all of the POST data, `cleaned_data` only contains form fields. The other data (such as the CSRF token and the submit button that was clicked) is present in the POST request's `QueryDict` object but is not included, as it is not form fields.

In this exercise, you updated `ExampleForm` so that the browser allowed it to be submitted even though Django would consider it to be invalid. This allowed Django to perform its validation on the form. You then updated the `form_example` view to instantiate the `ExampleForm` class differently, depending on the HTTP method, passing in the request's POST data for a POST request. The view also had its debug output code updated to print out the `cleaned_data` dictionary. Finally, you tested submitting valid and invalid form data to see the different execution paths and the types of data that the form generated. We saw that Django automatically converted the POST data from strings into Python types based on the field class.

Next, we will look at how to add more validation options to fields, which will allow us to more tightly control the values that can be entered.

Built-in field validation

We have not yet discussed the standard validation arguments that can be used on fields. Although we already mentioned the `required` argument (which is `True` by default), many others can be used to control the data being entered into a field more tightly. Here are a few useful ones:

- `max_length`: Sets the maximum number of characters that can be entered into the field, available on `CharField` (and `FileField` – which we will cover in *Chapter 8*).
- `min_length`: Sets the minimum number of characters that must be entered into the field, available on `CharField` (and `FileField` – again, more about it in *Chapter 8*).
- `max_value`: Sets the maximum value that can be entered into a numeric field, available on `IntegerField`, `FloatField`, and `DecimalField`.
- `min_value`: Sets the minimum value that can be entered into a numeric field, available on `IntegerField`, `FloatField`, and `DecimalField`.

4. The user may have no books, so the `books_you_own` field should not be required. Add a `required=False` argument to the `MultipleChoiceField` constructor:

```
books_you_own = forms.MultipleChoiceField(
    required=False, choices=BOOK_CHOICES)
```

5. The user should only be able to enter a value between 1 and 10 in `integer_input`. Add `min_value=1` and `max_value=10` arguments to the `IntegerField` constructor:

```
integer_input = forms.IntegerField(
    min_value=1, max_value=10)
```

6. Finally, add `max_digits=5` and `decimal_places=3` to the `DecimalField` constructor:

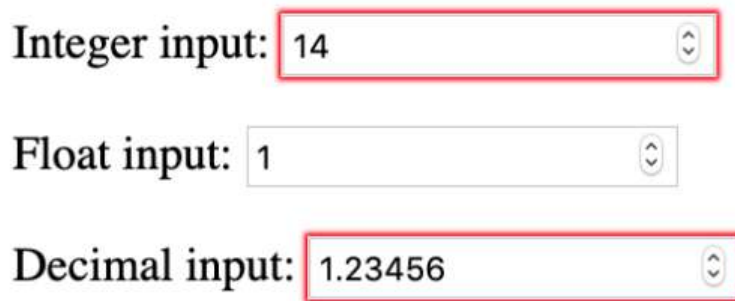
```
decimal_input = forms.DecimalField(
    max_digits=5, decimal_places=3)
```

Save the file.

7. Start the Django dev server if it's not already running. We do not have to make any changes to any other files to get these new validation rules since Django automatically updates the HTML generation and validation logic. This is a great benefit you get from using Django forms. Just visit or refresh `http://127.0.0.1:8000/form-example/` in your browser, and the new validation will be automatically added. The form should not look any different until you try to submit it with incorrect values, in which case your browser can automatically show errors. Some things to try are as follows:
- Entering more than three characters into the **Text input** field is something you will not be able to do.
 - Type less than eight characters into the **Password** field, and then clicking away from it might cause your browser to show an error indicating that this is not valid (only some browsers support this).
 - Do not select any values for the **Books you own** field. This will not prevent you from submitting the form anymore.

- Use the stepper buttons on **Integer input**. You will only be able to enter a value between 1 and 10. If you type in a value outside this range, your browser should show an error.
- **Decimal input** is the only field that does not validate the Django rules in the browser. You will need to type in an invalid value (such as 123.456) and submit the form before an error (generated by Django) is displayed.

The following figure shows some of the fields that the browser can use to validate itself:



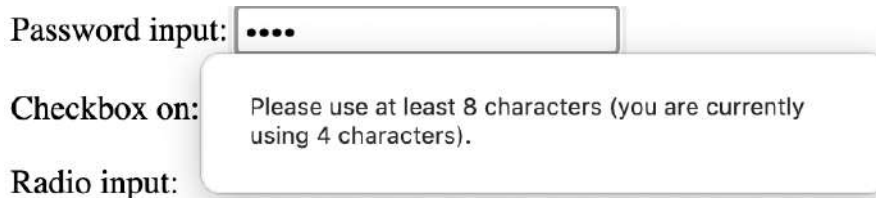
Integer input: 14

Float input: 1

Decimal input: 1.23456

Figure 6.26: Some browsers highlight invalid fields

Figure 6.27 shows an alternate way that validation errors might be presented by the browser – by placing an error message next to a particular field on form submission:



Password input:

Checkbox on: Please use at least 8 characters (you are currently using 4 characters).

Radio input:

Figure 6.27: Browsers might show messages next to fields on submit

Figure 6.28 shows an error that can only be generated by Django, as the browser does not understand the `DecimalField` validation rules:

- Ensure that there are no more than 3 digits in total.



Decimal input: 1234

Figure 6.28: The browser considers the form valid, but Django does not

In this exercise, we implemented some basic validation rules on our form fields. We then loaded the form example page in the browser without having to make any changes to our template or view. We tried to submit the form with different values to check how the browser can validate the form compared to Django.

In the activity for this chapter, we will implement the Book Search view using a Django Form.

Activity 6.01 – Book Search

In this activity, you will finish the Book Search view that was started in *Chapter 1*. You will build a `SearchForm` that submits and accepts a search string from `request.GET`. It will have a select field to choose to search for a title or contributor. It will then search for all books containing the given text in their title or the contributor's first names or last names. You will then render this list of books in the `search-results.html` template. The search term should not be required, but if it exists, it should have a length of more than three characters. Since the view will search even when using the GET method, the form will always have its validation checked. If we made the field required, it would always show an error whenever the page loads.

There will be two ways of performing the search. The first is by submitting the search form that is in the `base.html` template and thus in the top-right corner of every page. This will only search through Book titles. The other method is by submitting a `SearchForm` that is rendered on the `search-results.html` page. This form will display a `ChoiceField` for selecting between a title or contributor search.

These steps will help you complete this activity:

1. Create a `forms.py` file, import the Django forms library, then create a `SearchForm` class.
2. `SearchForm` should have two fields. The first is `CharField` with the name `search`. This field should not be required, but should have a minimum length of 3.
3. The second field on `SearchForm` is `ChoiceField` with the name `search_in`. This will allow you to select between title and contributor (with labels of **Title** and **Contributor**, respectively). It should not be required.
4. Update the `book_search` view so that it instantiates a `SearchForm` using data from `request.GET`.
5. Add code to search for Book models using `title__icontains` (for case-insensitive searching). This should be done if searching by *title*. The search should only be performed if the form is valid and contains some search text. The `search_in` value

should be retrieved from `cleaned_data` using the `GET` method since it might not exist, as it's not required. Default it to `title`.

6. When searching for contributors, use `first_names__icontains` or `last_names__icontains`, then iterate the contributors and retrieve the books for each contributor. This should be done if you're searching by *contributor*. The search should only be performed if the form is valid and contains some search text. There are many ways to combine the search results for first or last names. The easiest method using the techniques that you have been introduced to so far is to perform two queries, one for matching first names and then for last names, and iterate them separately.
7. Update the render call so that it includes the `form` variable and books that were retrieved in the context (as well as `search_text`, which was already being passed). The location of the template was changed in *Chapter 3*, so update the second argument to render accordingly.
8. The `search-results.html` template we created in *Chapter 1* is essentially redundant now, so you can clear its content. Update `search-results.html` so that it extends from `base.html` instead of being a standalone template file.
9. Add a title block that will display **Search Results for <search_text>** if the form is valid and `search_text` was set; otherwise, just display **Book Search**. This block will also be added to `base.html` later in this activity.
10. Add a content block, which should show a `<h2>` heading with the text **Search for Books** under the `<h2>` render the form. The `<form>` element can have no attributes, and it will default to making a GET request to the same URL it's on. Add a submit button, as we have used in previous activities, with the `btn btn-primary` class.
11. Under the form, show a message stating **Search results for <search_text>** if the form is valid and search text was entered; otherwise, show no message. This should be displayed in a `<h3>` heading, and the search text should be wrapped in an `<em>`.
12. Iterate over the search results and render each one. Show the book title and contributor(s) first and last names. The book title should link to the `book_detail` page. If the books list is empty, show the text **No results found**. You should wrap the results in a `<ul>` with a class called `list-group`; each result should be a `<li>` with a class of `list-group-item`. This will be like the `book_list` page, but we won't show as much information (just the title and contributors).
13. Update the project-level `base.html` file so that it includes an action attribute in the search `<form>` tag. Use the `url` template tag to generate the URL for this attribute.

14. Set the name attribute of the search field to search, and the value attribute to the entered search_text. Also, ensure that the minimum length of the field is 3.
15. While in base.html, add a title block to the title tag that was overridden by other templates (as in *Step 9*). Add a block template tag inside the <title> HTML element. It should contain the content Bookr.

After completing this activity, you should be able to open the Book Search page at `http://127.0.0.1:8000/book-search/`; it will look like *Figure 6.29*:

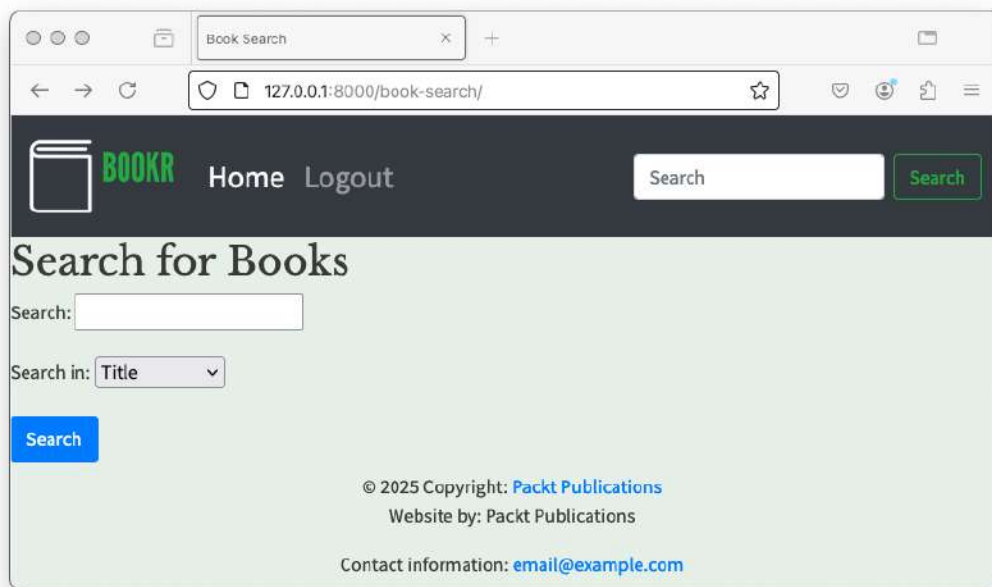


Figure 6.29: The Book Search page without a search

When searching for something using just two characters, your browser should prevent you from submitting either of the search fields.

If you search for something that returns no results, you will see a message that there were no results. Searching by title (this can be done with either field) will show matching results:

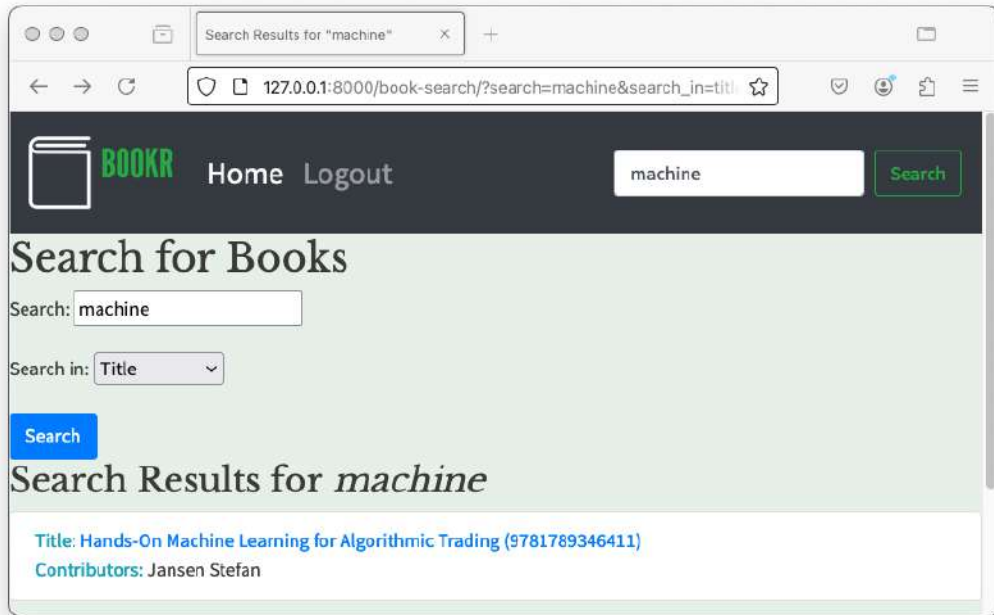


Figure 6.30: Title search

Similarly, when searching by contributor (although this can only be done in the lower form), you'll see matching results:

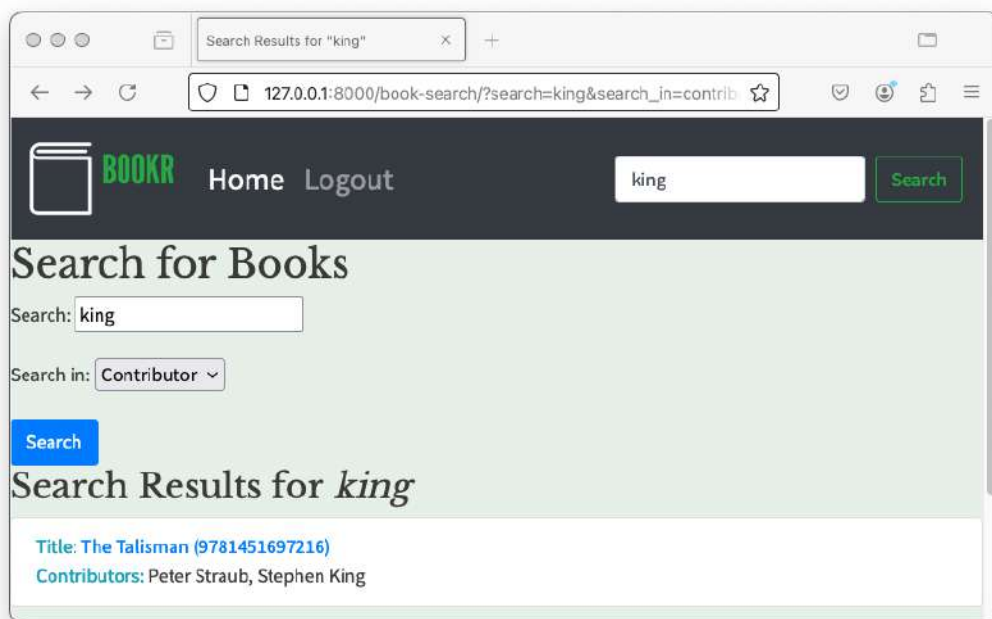


Figure 6.31: A contributor search

Summary

This chapter provided an introduction to forms in Django. We introduced several HTML inputs for entering data onto a web page. We talked about how data is submitted to a web application and when to use GET and POST requests. We then looked at how Django's form classes can make generating the form HTML simpler, as well as allow us to automatically build forms using models. Finally, we enhanced Bookr further by building the Book Search functionality.

In the next chapter, we will dive deeper into forms and learn how to customize the display of form fields, how to add more advanced validation to a form, and how to automatically save model instances by using the `ModelForm` class.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

7

Advanced Form Validation and Model Forms

Continuing your journey with the `form_project` application you started in the previous chapter, you'll begin this chapter by adding a new form to your app with custom multi-field validation and form cleaning. You'll learn how to set the initial values on your form and customize the widgets (the HTML input elements that are being generated). Then, you'll be introduced to the `ModelForm` class, which allows a form to be automatically created from a model. You'll use it in a view to automatically save the new or changed `Model` instance.

In this chapter, we will take our knowledge of Django form validation further by introducing concepts that form the fundamentals of most production websites.

For instance, a certain field might only be required if another field is set. Let's say we want to add a checkbox to allow users to sign up for our monthly newsletter. It has a text box below it that lets them enter their email address. With some basic validation, we can check the following:

- Whether the user has checked the checkbox
- Whether the user has entered their email address

When the user clicks the **Submit** button, we'll be able to validate whether both fields are actioned. But what if the user doesn't want to sign up for our newsletter? If they click the **Submit** button, ideally, both fields should be blank. That's where validating each field might not work.

Another example could be a case where we have two fields where each has a maximum value of, say, 50. But the total values added to each one must be less than 75. We'll start this chapter by looking at how to write custom validation rules to solve such problems.

Later, as we progress in this chapter, we will look at how to set initial values on a form. This can be useful when automatically filling out information that is already known to the user. For example, we can automatically put a user's contact information into a form if that user is logged in.

We'll finish this chapter by looking at model forms, which will let us automatically create a form from a Django `Model` class. This cuts down the amount of code that needs to be written to create a new `Model` instance.

In this chapter, we will cover the following topics:

- Custom field validation and cleaning
- Adding placeholders and initial values
- Creating or editing Django models

By the end of this chapter, you will know how to add extra multi-field validation to Django forms, how to customize and set form widgets for fields, how to use `ModelForms` to automatically create a form from a Django model, and how to automatically create `Model` instances from `ModelForms`.

Technical requirements

Find the solution in the `Chapter07` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Custom field validation and cleaning

In the previous chapter, we saw how a Django form converts values from an HTTP request, which are strings, into Python objects. In a non-custom Django form, the target type is dependent on the field class. For example, the Python type derived from `IntegerField` is `int`, and string values are given to us verbatim, as the user enters them. But we can also implement methods on our `Form` class to alter the output values from our fields in any way we choose. This allows us to clean or filter the user's input data to fit what we expect better. We could round an integer to the nearest multiple of 10 so that it fits into a batch size for ordering specific items. Or we could transform an email address into lowercase so that the data is consistent for searching.

We can also implement some custom validators. We'll look at a couple of different ways of validating fields: by writing a custom validator and by writing a custom `clean` method for the field. Each method has its pros and cons: a custom validator can be applied to different fields and forms, so you don't have to write the validation logic for each field; a custom `clean`

method must be implemented on each form you want to clean, but is more powerful and allows validation since you can use other fields in the form or change the cleaned value that the field returns.

Custom validators

A validator is simply a function that accepts a value and raises `django.core.exceptions.ValidationError` if the value is invalid – the validity is determined by the code you write. The value is a Python object (that is, `cleaned_data`, which has already been converted from the POST request string).

Here's a simple example that validates whether a value is in lowercase:

```
from django.core.exceptions import ValidationError

def validate_lowercase(value):
    if value.lower() != value:
        raise ValidationError(f"{value} is not lowercase.")
```

Notice the function does not return anything, for either success or failure. It will just raise `ValidationError` if the value is not valid.

Note

Note that the behavior and handling of `ValidationError` are different from how we've seen other exceptions behave in Django. Normally, if you raise an exception in your view, you'll end up with an HTTP response with a 500 status code from Django (if you don't handle the exception in your code).

When raising `ValidationError` in your validation/cleaning code, the Django form class will catch the error for you; then, the `is_valid` method of form will return `False`. You don't have to write try/except handlers around the code that might raise `ValidationError`.

The validator can be passed to the `validators` argument of a field constructor on a form, inside a list – for example, to our `text_input` field from `ExampleForm`:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField(
        validators=[validate_lowercase])
```

Now, if we submit the form and the fields contain uppercase values, we'll get an error, as shown in the following figure:

- Text is not lowercase.

Text input:

Figure 7.1 – Lowercase text validator in action

The validator function can be used on any number of fields. In our example, if we wanted lots of fields to have lowercase enforced, `validate_lowercase` could be passed to all of them. Now, let's look at how we could implement this another way, with a custom `clean` method.

Cleaning methods

A `clean` method is created on the `Form` class and is named in the `clean_field_name` format. For example, the `clean` method for `text_input` would be called `clean_text_input`, the `clean` method for `books_you_own` would be called `clean_books_you_own`, and so on.

Cleaning methods take no arguments; instead, they should use the `cleaned_data` attribute on `self` to access the field data. This dictionary will contain the data after being cleaned in the standard Django way, as we saw in the previous example. The `clean` method must return the cleaned value, which will replace the original value in the `cleaned_data` dictionary. Even if the method does not change the value, a value must be returned. You can also use the `clean` method to raise `ValidationError`, and the error will be attached to the field (the same as with a validator).

Let's re-implement the lowercase validator as a `clean` method, like this:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    ...

    def clean_text_input(self):
        value = self.cleaned_data["text_input"]
        if value.lower() != value:
            raise ValidationError(
                f"{value} is not lowercase.")
        return value
```

You can see that the logic is essentially the same, except we must return the validated value at the end. If we submit the form, we will get the same result as the previous time we tried (Figure 7.1).

Let's look at one more cleaning example. Instead of raising an exception when the value is invalid, we could just convert the value into lowercase. We would implement that with this code:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    ...

    def clean_text_input(self):
        value = self.cleaned_data['text_input']
        return value.lower()
```

Now, let's say we enter text into the input as uppercase:

Text input:

Figure 7.2 – ALL UPPERCASE text entered

If we were to examine the cleaned data using our debug output from the view, we would see that it is lowercase:

text_input: (<class 'str'>) all uppercase

Figure 7.3 – The cleaned data has been transformed into lowercase

These were just a couple of simple examples of how to validate fields using both validators and clean methods. You can, of course, make each type of validation much more complex if you wish, and transform the data in more complex ways using a clean method.

So far, you've only learned simple methods for form validation, where you've treated each field independently. A field is valid (or not) based only on the information it contains and nothing else. What if the validity of one field depends on what the user entered into another field? An example of this might be that you have an email field to collect someone's email address if they want to be signed up for a mailing list. The field is only required if they check a checkbox that indicates they want to be signed up. Neither of these fields is required on its own – we do not want the checkbox to be required to be checked, but if it is checked, then the email field should be required too.

In the next section, we will show how you can validate a form whose fields depend on each other by overriding the `clean` method in your form.

Multi-field validation

We've just looked at the `clean_field_name` methods that can be added to a Django form to clean a specific field. Django also allows us to override the `clean` method, in which we can access all cleaned data from all fields, and we know that all custom field methods have been called. This allows the fields to be validated based on another field's data.

Referring to our previous example with a form that has an email address that is only required if a checkbox is checked, we'll see how we can implement this using the `clean` method.

First, create a Form class and add two fields – make them both optional with the `required=False` argument:

```
class NewsletterSignupForm(forms.Form):
    signup = forms.BooleanField(
        label="Sign up to newsletter?", required=False)
    email = forms.EmailField(
        help_text="Enter your email address to subscribe",
        required=False)
```

We've also introduced two new arguments that can be used for any field:

- `label`: This allows you to set the label text for a field. As we've seen, Django will automatically generate label text from the field name. If you set the `label` argument, you can override this default. Use this argument if you want to have a more descriptive label.
- `help_text`: If you need more information to be displayed regarding what input a field requires, you can use this argument. By default, it's displayed after the field.

When rendered, the form looks like this:

Sign up to newsletter? ☐

Email: Enter your email address to subscribe

Figure 7.4 – Email signup form with a custom label and help text

If we were to submit the form now, without entering any data, nothing would happen. Neither field is required, so the form validates fine.

Now, we can add the multi-field validation to the `clean` method. We will check whether the **Sign up to newsletter?** checkbox is checked, and then check that the **Email** field has a value. The built-in Django methods have already validated that the email address is valid at this point, so we just need to check that a value exists for it. We will then use the `add_error` method to set an error for the email field. This is a method you haven't seen before, but it's very simple; it takes two arguments – the name of the field to set the error on, and the text of the error.

Here's the code for the `clean` method:

```
class NewsletterSignupForm(forms.Form):
    ...

    def clean(self):
        cleaned_data = super().clean()

        if (cleaned_data["signup"] and not
            cleaned_data.get("email")):
            self.add_error("email", "Your email address is required "
                             "if signing up for the newsletter.")
```

Your `clean` method must always call the `super().clean()` method to retrieve the cleaned data. When `add_error` is called to add errors to the form, the form will no longer validate (the `is_valid` method returns `False`).

Now, if we submit the form without the checkbox checked, still, no error is generated, but if you check the checkbox without an email address, you will receive the error we just wrote the code for:

Sign up to newsletter? ☒

- Enter your email address to subscribe

Email: Enter your email address to subscribe

Submit Input

Figure 7.5 – Error displayed when attempting to sign up with no email address

You might notice that we are retrieving the email from the `cleaned_data` dictionary using the `get` method. The reason for doing this is that if the `email` value in the form is invalid, then the `email` key will not exist in the dictionary. The browser should prevent the user from submitting the form if an invalid email has been entered, but a user might be using an older browser that does not support this client-side validation, so, for safety, we use the `get` method. Since the `signup` field is a `BooleanField` type and not required, it will only be invalid if a custom validation function is used. We're not using one here, so it's safe to access its value using square bracket notation.

There's one more validation scenario to consider before moving on to our first exercise, and that's adding errors that are not specific to any field. Django calls these **non-field errors**. There are many scenarios where you might want to use these when multiple fields are dependent on each other.

Take, for example, a shopping website. Your order form could have two numeric fields whose totals can't exceed a certain value. If the total were exceeded, the value of either field could be decreased to bring the total below the maximum value, so the error is not specific to either of the fields. To add a non-field error, call the `add_error` method with `None` as the first argument.

Let's look at how to implement this. In this example, we'll have a form where the user can specify a certain number of items to order, for item A or item B. The user cannot order more than 100 items in total. The fields will have a maximum value of 100, and a minimum value of 0, but custom validation in the `clean` method will need to be written to handle the validation of the total amount:

```
class OrderForm(forms.Form):
    item_a = forms.IntegerField(min_value=0, max_value=100)
    item_b = forms.IntegerField(min_value=0, max_value=100)
```

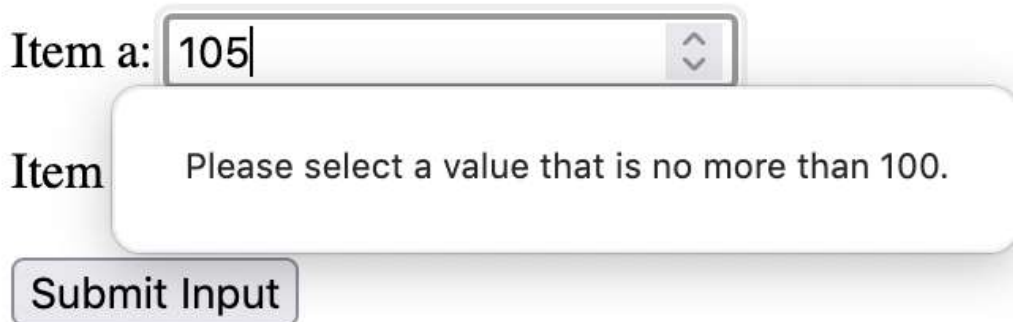
```
def clean(self):
    cleaned_data = super().clean()
    if (cleaned_data.get("item_a", 0) +
        cleaned_data.get("item_b", 0) > 100):
        self.add_error(None, "The total number of items "
                             "must be 100 or less.")
```

The fields (`item_a` and `item_b`) are added in the normal way, with standard validation rules. You can see that we have used the `clean` method the same way we used it before. Moreover, we have implemented the maximum item logic inside this method. The following line is what registers the non-field error if the maximum number of items is exceeded:

```
self.add_error(None, "The total number of items must be 100 or less.")
```

Once again, we access the values of `item_a` and `item_b` using the `get` method, with a default value of `0`. This is in case the user has an older browser (from 2011 or earlier) and is able to submit the form with invalid values.

In a browser, the field-level validation ensures values between `0` and `100` have been entered in each field, and prevents the form from being submitted otherwise:



Item a: 105

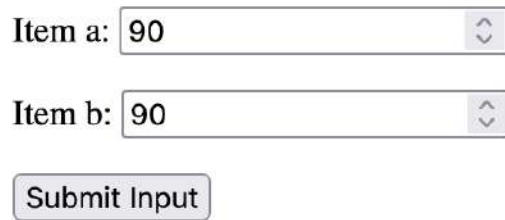
Item Please select a value that is no more than 100.

Submit Input

Figure 7.6 – The form cannot be submitted if one field exceeds the maximum value

However, if we put in two values that sum to more than 100, we can see how Django displays the non-field error:

- The total number of items must be 100 or less.



Item a:

Item b:

Figure 7.7 – Django non-field error displayed at the start of the form

Django non-field errors are always displayed at the start of a form, before other fields or errors.

In the following exercise, we will build a form that implements a validation function, a field clean method, and a form clean method.

Exercise 7.01 – Custom clean and validation methods

In this exercise, you will build a new form that allows the user to create an order for books or magazines. It must have the following validation criteria:

- The user may order up to 80 magazines and/or 50 books, but the total number of items must not be more than 100
- The user can choose to receive an order confirmation, and if they do, they must enter an email address
- The user should not enter an email address if they have not chosen to receive an order confirmation
- To ensure they are part of our company, the email address must be part of our company domain (in our case, we'll just use `example.com`)
- For consistency with other email addresses in our fictional company, the address should be converted to lowercase

This sounds like a lot of rules, but with Django, it's simple if we tackle them one by one. We'll carry on with the `form_project` app you started in *Chapter 6, Forms*. If you haven't completed *Chapter 6, Forms*, you can download the code from the `Chapter07` folder in this book's GitHub repo.

Follow these steps:

1. In PyCharm, open the `form_example` app's `forms.py` file.

The function splits the email address on the @ symbol, and then checks whether the part after it is equal to `example.com`. This function alone would validate non-email addresses. For example, the `not-valid@some other domain@example.com` string would not cause `ValidationError` to be raised in this function. This is acceptable in our case because, since we're using `EmailField`, the other standard field validators will check the email address's validity.

5. Add the `validate_email_domain` function as a validator to the email field on `OrderForm`. Update the `EmailField` constructor call to add a `validators` argument, passing in a list containing the validation function:

```
class OrderForm(forms.Form):
    ...
    email = forms.EmailField(required=False,
                             validators=[validate_email_domain])
```

6. Add a `clean_email` method to the form to make sure the email address is lowercase:

```
class OrderForm(forms.Form):
    # truncated for brevity

    def clean_email(self):
        return self.cleaned_data['email'].lower()
```

7. Now, add the `clean` method to perform all the cross-field validation. First, we will just add the logic for making sure that an email address is only entered if an order confirmation is requested:

```
class OrderForm(forms.Form):
    # truncated for brevity

    def clean(self):
        cleaned_data = super().clean()
        if (cleaned_data["send_confirmation"] and
            not cleaned_data.get("email")):
            self.add_error("email",
                           "Please enter an email address to "
                           "receive the confirmation message.")

        elif (cleaned_data.get("email") and
              not cleaned_data["send_confirmation"]):
```

```
self.add_error("send_confirmation",
               "Please check this if you want to "
               "receive a confirmation email.")
```

This will add an error to the **Email** field if **Send confirmation** is checked but no email address is added:

Method: POST

Magazine count: 10

Book count: 20

Send confirmation: ☒

- Please enter an email address to receive the confirmation message.

Email:

Submit Input

Button With **Styled** Text

Figure 7.8 – Error if Send confirmation is checked but no email address is added

Similarly, an error will be added to **Email** if an email address is entered but **Send confirmation** is not checked:

Method: POST

Magazine count: 10

Book count: 20

- Please check this if you want to receive a confirmation email.

Send confirmation: ☐

Email:

Submit Input

Button With **Styled** Text

Figure 7.9 – Error because an email has been entered but the user has not chosen to receive confirmation

8. Add the final check, also inside the `clean` method. The total number of items should not be more than 100. We will add a non-field error if the sum of `magazine_count` and `book_count` is greater than 100:

```
class OrderForm(forms.Form):
    ...
```

```
def clean(self):
    ...
    item_total = cleaned_data.get(
        "magazine_count", 0) + cleaned_data.get(
        "book_count", 0)

    if item_total > 100:
        self.add_error(None,
            "The total number of items must be "
            "100 or less.")
```

This will add a non-field error by passing `None` as the first argument to the `add_error` call.

Note

Refer to `/form_project/form_example/forms.py` folder in the Chapter07 folder in this book's GitHub repo for the complete code.

Save `forms.py`.

9. Open the reviews app's `views.py` file. We will change the form's import so that `OrderForm` is being imported instead of `ExampleForm`. Consider the following import line:

```
from .forms import ExampleForm
```

Change it as follows:

```
from .forms import OrderForm
```

10. In the `form_example` view, change the two lines that use `ExampleForm` to use `OrderForm` instead. Consider the following line of code:

```
form = ExampleForm(request.POST)
```

Change this as follows:

```
form = OrderForm(request.POST)
```

Similarly, consider the following line of code:

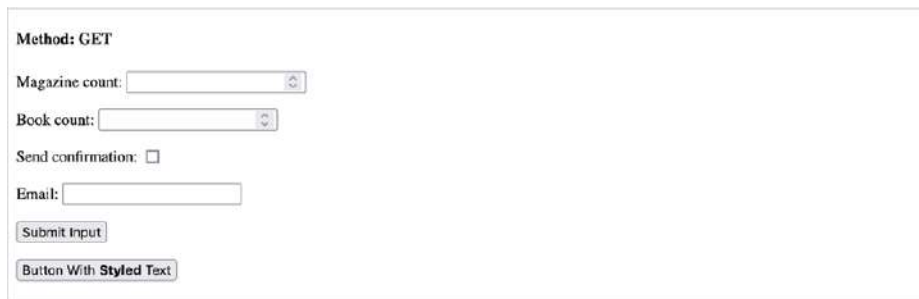
```
form = ExampleForm()
```

Change this as follows:

```
form = OrderForm()
```

The rest of the function can stay as-is.

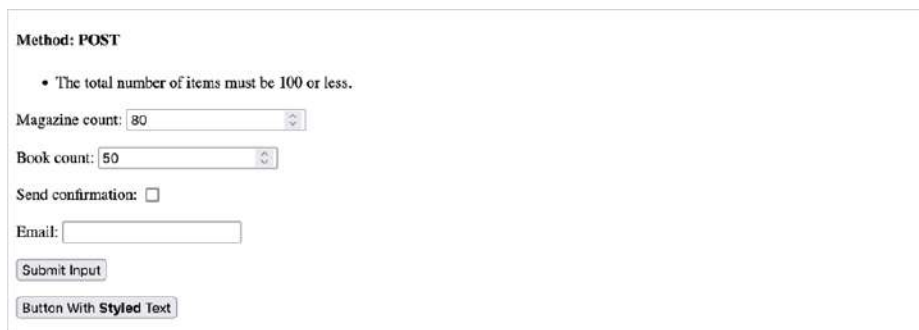
We don't have to make changes to the template. Start the Django dev server and navigate to `http://127.0.0.1:8000/form-example/` in your browser. You should see the form rendered similarly to what's shown in *Figure 7.10*:



The screenshot shows a web browser window displaying a form titled "Method: GET". The form contains the following elements: a "Magazine count:" label followed by a text input field with a value of 80; a "Book count:" label followed by a text input field with a value of 50; a "Send confirmation:" label followed by an unchecked checkbox; an "Email:" label followed by an empty text input field; a "Submit Input" button; and a "Button With **Styled Text**" button.

Figure 7.10 – OrderForm in the browser

11. Try submitting the form with a **Magazine count** value of 80 and a **Book count** value of 50. The browser will allow this, but since they sum to more than 100, an error will be triggered by the `clean` method in the form and displayed on the page:



The screenshot shows a web browser window displaying the form after a POST request. At the top, it says "Method: POST". Below this, a non-field error is displayed: "• The total number of items must be 100 or less." The form fields are: "Magazine count:" with a value of 80; "Book count:" with a value of 50; "Send confirmation:" with an unchecked checkbox; "Email:" with an empty text input field; "Submit Input" button; and "Button With **Styled Text**" button.

Figure 7.11 – A non-field error displayed on the form when the maximum number of allowed items is exceeded

12. Try submitting the form with **Send confirmation** checked but the **Email** field blank. Then, fill in the **Email** text box, but uncheck **Send confirmation**. Either combination will give an error stating that both must be present. The error will differ based on which field is missing:

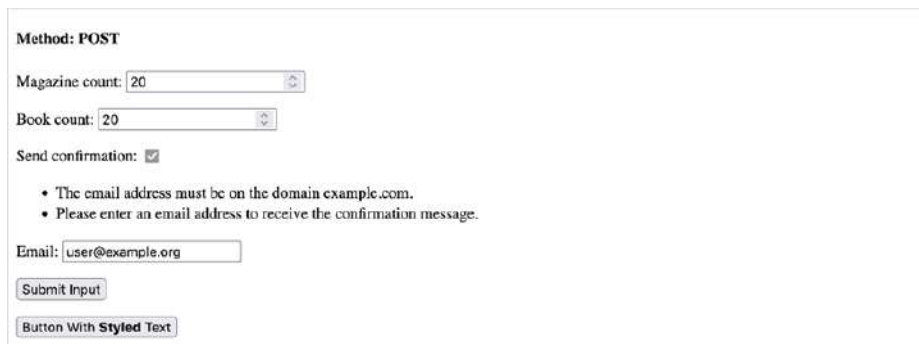


The screenshot shows a web form with the following elements:

- Method: POST**
- Magazine count:** 20 (with a spinner)
- Book count:** 20 (with a spinner)
- Send confirmation:** ☒
- Email:** (empty text box)
- Submit Input** button
- Button With Styled Text** button
- Error message:**
 - Please enter an email address to receive the confirmation message.

Figure 7.12 – Error message if no email address is present

13. Now, try submitting the form with **Send confirmation** checked, and an email address that is not on the example.com domain. You should receive a message that your email address must have the example.com domain. You should also receive a message that **Email** must be set – this is because the email does not end up in the `cleaned_data` dictionary. After all, it is not valid:

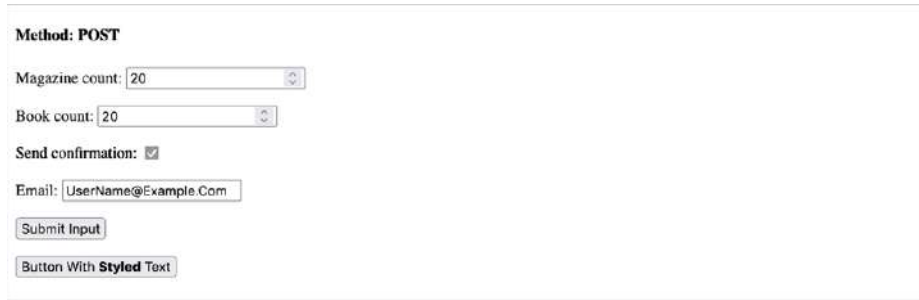


The screenshot shows the same web form as Figure 7.12, but with the following changes:

- Email:** user@example.org
- Send confirmation:** ☒
- Error messages:**
 - The email address must be on the domain example.com.
 - Please enter an email address to receive the confirmation message.

Figure 7.13 – The error message that's shown when the email domain is not example.com

14. Finally, enter valid values for **Magazine count** and **Book count** (such as 20 and 20). Check **Send confirmation** and enter `UserName@Example.Com` as the email (make sure you match the letter case, including the mixed uppercase and lowercase characters):



Method: POST

Magazine count: 20

Book count: 20

Send confirmation: ☒

Email: UserName@Example.Com

Submit Input

Button With **Styled** Text

Figure 7.14 – The form after being submitted with valid values

15. Switch to PyCharm and look in the debug console. You'll see that the email is converted into lowercase when it is printed by our debug code:

```
magazine_count: (<class 'int'>) 20
book_count: (<class 'int'>) 20
send_confirmation: (<class 'bool'>) True
email: (<class 'str'>) username@example.com
```

Figure 7.15 – Email in lowercase, as well as other fields

This is our `clean_email` method in action – even though we entered data in both uppercase and lowercase, it has been converted into all lowercase.

In this exercise, we created a new `OrderForm` that implemented form and field `clean` methods. We used a custom validator to ensure that the **Email** field met our specific validation rules – only a specific domain was allowed. We used a custom field cleaning method (`clean_email`) to convert the email address into lowercase. We then implemented a `clean` method to validate the forms that were dependent on each other. In this method, we added both field and non-field errors.

So far, we have learned about multi-field validations and how to implement them to make sure that we can validate the form in one go rather than highlighting errors one by one.

In the next section, we will cover how to add placeholders and initial values to the form.

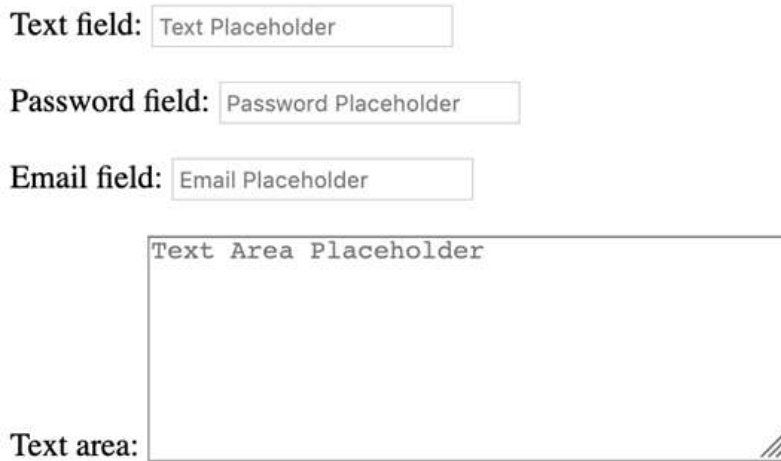
Adding placeholders and initial values

There are two things our first manually-built form had that our current Django form still does not have – placeholders and initial values. Adding placeholders is simple; they are just added as attributes to the widget constructor for the form field. This is similar to what we've already seen for setting the type of `DateField` in our previous examples.

Here's an example:

```
class ExampleForm(forms.Form):
    text_field = forms.CharField(
        widget=forms.TextInput(
            attrs={"placeholder": "Text Placeholder"})
    )
    password_field = forms.CharField(
        widget=forms.PasswordInput(
            attrs={"placeholder": "Password Placeholder"})
    )
    email_field = forms.EmailField(
        widget=forms.EmailInput(
            attrs={"placeholder": "Email Placeholder"})
    )
    text_area = forms.CharField(
        widget=forms.Textarea(
            attrs={"placeholder": "Text Area Placeholder"})
    )
```

Here is what the preceding form looks like when rendered in the browser:



The image shows a Django form with four fields, each with a placeholder text:

- Text field:** A single-line text input with the placeholder "Text Placeholder".
- Password field:** A single-line password input with the placeholder "Password Placeholder".
- Email field:** A single-line email input with the placeholder "Email Placeholder".
- Text area:** A multi-line text area with the placeholder "Text Area Placeholder".

Figure 7.16 – Django form with placeholders

Of course, if we're manually setting `Widget` for each field, we need to know which `Widget` class to use. The ones that support placeholders are `TextInput`, `NumberInput`, `EmailInput`, `URLInput`, `PasswordInput`, and `Textarea`.

While we're examining the `Form` class itself, we'll look at the first of two ways of setting an initial value for a field. We can do this by using the `initial` argument on a `Field` constructor, like this:

```
text_field = forms.CharField(initial="Initial Value", ...)
```

The other method is to pass in a dictionary of data when instantiating the form in our view. The keys are the field names. The dictionary should have zero or more items (that is, an empty dictionary is valid). Any extra keys are ignored. This dictionary should be supplied as the `initial` argument in our view, as follows:

```
initial = {"text_field": "Text Value", "email_field": "user@example.com"}
form = ExampleForm(initial=initial)
```

Or, for a POST request, pass in `request.POST` as the first argument, as usual:

```
initial = {"text_field": "Text Value", "email_field": "user@example.com"}
form = ExampleForm(request.POST, initial=initial)
```

Values in `request.POST` will override values in `initial`. This means that even if we have an initial value for a required field, if it's left blank when submitted, then it will not validate. The field will not fall back to the value in `initial`.

Whether you decide to set initial values in the `Form` class itself or in the view is up to you and depends on your use case. If you had a form that was used in multiple views but usually had the same value, it would be better to set the initial value in the form. Otherwise, it can be more flexible to use setting in the view.

In the next exercise, we will add placeholders and initial values to the `OrderForm` class from the previous exercise.

Exercise 7.02 – Placeholders and initial values

In this exercise, you will enhance the `OrderForm` class by adding placeholder text. You will simulate passing an initial email address to the form. It will be a hardcoded address, but once the user can log in, it could be an email address associated with their account – you will learn about sessions and authentication in *Chapter 9, Sessions and Authentication*.

Follow these steps:

1. In PyCharm, open the reviews app's `forms.py` file. You will add placeholders to the `magazine_count`, `book_count`, and `email` fields on `OrderForm`, which means also setting a widget.

To the `magazine_count` field, add a `NumberInput` widget with placeholder in the `attrs` dictionary. This placeholder value should be set to `Number of Magazines`. Write the following code:

```
magazine_count = forms.IntegerField(
    min_value=0,
    max_value=80,
    widget=forms.NumberInput(
        attrs={"placeholder": "Number of Magazines"}),
)
```

2. Add a placeholder to the `book_count` field in the same manner. The placeholder text should be `Number of Books`:

```
book_count = forms.IntegerField(
    min_value=0,
    max_value=50,
    widget=forms.NumberInput(
```

```
        attrs={"placeholder": "Number of Books"}),  
    )
```

3. The final change to `OrderForm` is to add a placeholder to the email field. This time, the widget is `EmailInput`. The placeholder text should be `Your company email address`:

```
email = forms.EmailField(  
    required=False,  
    validators=[validate_email_domain],  
    widget=forms.EmailInput(attrs={  
        "placeholder": "Your company email address"}),  
)
```

Note that the `clean_email` and `clean` methods should remain as they were in *Exercise 7.01 – Custom clean and validation methods*. Save the file.

4. Open the `form_project` app's `views.py` file. In the `form_example` view function, create a new dictionary variable called `initial` with one key, `email`, like this:

```
initial = {"email": "user@example.com"}
```

5. In the two places that you are instantiating `OrderForm`, also pass in the `initial` variable using the `initial` keyword argument. The first instance is as follows:

```
form = OrderForm(request.POST, initial=initial)
```

The second instance is as follows:

```
form = OrderForm(initial=initial)
```

The complete code for `views.py` can be found in the `Chapter07` folder in this book's GitHub repo.

Save the `views.py` file.

6. Start the Django dev server if it is not already running. Browse to `http://127.0.0.1:8000/form-example/` in your browser. You should see that your form now has placeholders and an initial value set:

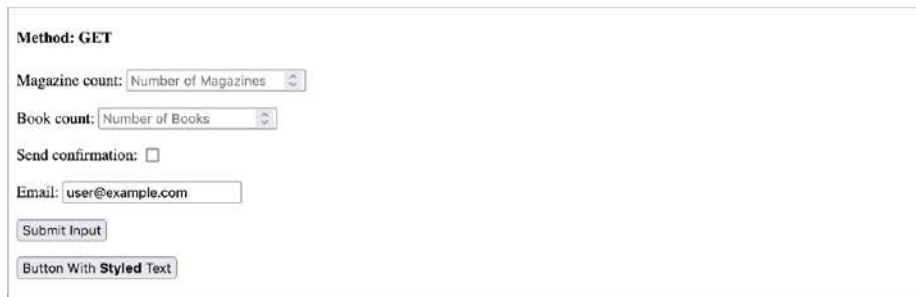


Figure 7.17 – Order form with initial values and placeholders

In this exercise, we added placeholders to form fields. This was done by setting a form widget when defining the form field on the form class and setting a placeholder value in the `attrs` dictionary. We also set an initial value for the form using a dictionary and passed it to the form instance using the `initial` keyword argument.

In the next section, we will talk about how to work with Django models using data from forms, and how `ModelForm` makes this easier.

Creating or editing Django models

We've seen how to define a form, and in *Chapter 2, Models and Migrations*, you learned how to create Django model instances. By using these things together, you can build a view that displays a form and also saves a model instance to the database. This gives you an easy way to save data without having to write a lot of boilerplate code or create custom forms. In *Bookr*, we will use this method to allow users to add reviews without requiring access to the Django admin site. Without using `ModelForm`, you can do something like this:

- You can create a form based on an existing model, for example, `Publisher`. The form would be called `PublisherForm`.
- You can manually define the fields on `PublisherForm`, using the same rules defined on the `Publisher` model, as shown here:

```
class PublisherForm(forms.Form):
    name = forms.CharField(max_length=50)
    website = forms.URLField()
    ...
```

- In the view, the initial values would be retrieved from the model queried from the database, and then passed to the form using the initial argument. If you were creating a new instance, the initial value would be blank – something like this:

```
if create:
    initial = {}
else:
    publisher = Publisher.objects.get(pk=pk)
    initial = {"name": publisher.name,
              "website": publisher.website, ...}

form = PublisherForm(initial=initial)
```

- Then, in the POST flow of the view, you can either create or update the model based on `cleaned_data`:

```
form = PublisherForm(request.POST, initial=initial)
if create:
    publisher = Publisher()
else:
    publisher = Publisher.objects.get(pk=pk)

publisher.name = form.cleaned_data['name']
publisher.website = forms.cleaned_data['website']
...
publisher.save()
```

This is a lot of work, and we have to consider how much duplicated logic we have. For example, we're defining the length of the name in the name form field. If we made a mistake here, we could allow a longer name in the field than the model allows. We also have to remember to set all the fields in the initial dictionary, as well as set the values on the new or updated model with `cleaned_data` from the form. There are many opportunities to make mistakes here, as well as to remember to add or remove field setting data for each of these steps if the model changes. All this code would have to be duplicated for each Django model you work with as well, exacerbating the duplication problem.

The `ModelForm` class

Luckily, Django provides a method of building `Model` instances from forms much more simply, with the `ModelForm` class. `ModelForm` is a form that is built automatically from a particular model. It will inherit the validation rules from the model (such as whether fields are required, the maximum length of `CharField` instances, and so on). It provides an extra `__init__` argument (called `instance`) for automatically populating the initial values from an existing model. It also adds a `save` method to automatically persist the form data to the database. All you need to do to set up `ModelForm` is specify its model and what fields should be used: this is done on the class `Meta` attribute of the form class. Let's see how to build a form from `Publisher`.

Inside the file that contains the form (for example, the `forms.py` file we have been working with), the only change is that the model must be imported:

```
from .models import Publisher
```

Then, the `Form` class can be defined. The class requires a class `Meta` attribute, which, in turn, must define a `model` attribute and either the `fields` or `excludes` attributes:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = ("name", "website", "email")
```

`fields` is a list or tuple of the fields to include in the form. When manually setting the list of fields, if you add extra fields to the model, you must also add their name here to have them displayed on the form.

You can also use the special `__all__` value instead of a list or tuple to automatically include all the fields, like this:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
```

If the `model` field has its `editable` attribute set to `False`, then it will not be automatically included.

On the contrary, the `exclude` attribute sets the fields to not display in the form. Any fields added to the model will automatically be added to the form. We could define the preceding form using `exclude` with any empty tuple, as we want all the fields. The code is like this:

```
class PublisherForm(forms.ModelForm):  
    class Meta:  
        model = Publisher  
        exclude = ()
```

This saves you some work because you don't need to add a field to both the model and the fields list; however, it is not as safe since you might automatically expose fields to the end user that you don't want to. For example, if you had a `User` model with `UserForm`, you might add an `is_admin` field to the `User` model to give admin users extra privileges. If this field were not in `exclude`, it would be displayed to the user. A user would then be able to make themselves an administrator, which is something you probably wouldn't want.

Whichever of these three approaches to choosing the forms to display we decide to use, in our case, they will display the same in the browser. This is because we are choosing to display *all* the fields. They all look like this when rendered in the browser:

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Figure 7.18 – *PublisherForm*

Note that `help_text` from the `Publisher` model is automatically rendered as well.

Usage in a view is similar to the other forms we have seen. Also, as mentioned, there is an extra argument that can be provided, called `instance`. This can be set to `None`, which will render an empty form.

Assuming, in your view function, you have some method of determining whether you are creating or editing a model instance (we will discuss how to do this later), this will determine a

variable called `is_create` (True if creating an instance, or False if editing an existing one). Your view function to create the form could then be written like this:

```
if is_create:
    instance = None
else:
    instance = get_object_or_404(Publisher, pk=pk)

if request.method == "POST":
    form = PublisherForm(request.POST, instance=instance)
    if form.is_valid():
        # we'll cover this branch soon
    else:
        form = PublisherForm(instance=instance)
```

As you can see, in either branch, the instance is passed to the `PublisherForm` constructor, although it is `None` if we're in create mode.

If the form is valid, we can save the model instance. This can be done simply by calling the `save` method on the form. This will automatically create the instance, or simply save changes to the old one:

```
if form.is_valid():
    form.save()
    return redirect(success_url)
```

The `save` method returns the model instance that was saved. It takes one optional argument, `commit`, which determines whether the changes should be written to the database. You can pass `False` instead, which allows you to make additional changes to the instance before manually saving the changes. This would be required to set attributes that have not been included in the form. As we mentioned, maybe you would set the `is_admin` flag to `False` on a `User` instance:

```
if form.is_valid():
    new_user = form.save(False)
    new_user.is_admin = False
    new_user.save()
    return redirect(success_url)
```

In *Activity 7.02 – Review creation UI*, at the end of this chapter, we will be using this feature as well.

If your model uses `ManyToMany` fields, and you call `form.save(False)`, you should also call `form.save_m2m()` to save any many-to-many relationships that have been set. It's not necessary to call this method if you call the `form.save` method with `commit` set to `True` (that is, the default).

Model forms can be customized by making changes to their Meta attributes. The `widgets` attribute can be set. It can contain a dictionary keyed on the field names, with widget classes or instances as the values. For example, this is how to set up `PublisherForm` to have placeholders:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
        widgets = {
            "name": forms.TextInput(attrs={
                "placeholder": "The publisher's name."
            })
        }
```

The values behave the same as setting the `kwargs` widget in the field definition; they can be a class or an instance. For example, to display `CharField` as a password input, the `PasswordInput` class can be used; it does not need to be instantiated:

```
widgets = {
    "password": forms.PasswordInput
}
```

Model forms can also be augmented with extra fields added in the same way as they are added to a normal form. For example, suppose we wanted to provide the option of sending a notification email after saving a `Publisher`. We can add an `email_on_save` field to `PublisherForm` like this:

```
class PublisherForm(forms.ModelForm):
    email_on_save = forms.BooleanField(required=False,
        help_text="Send notification email on save")

    class Meta:
```

```
model = Publisher
fields = "__all__"
```

When rendered, the form looks like this:

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Email on save: ☒ Send notification email on save

Figure 7.19 – PublisherForm with an additional field

Additional fields are placed after the Model fields. The extra fields are not handled automatically – they do not exist on the model, so Django won't attempt to save them on the model instance. Instead, you should handle how their values are saved by examining the `cleaned_data` values of the form, as you would with a standard form, for example (inside your view function):

```
if form.is_valid():
    if form.cleaned_data.get("email_on_save"):
        send_email()
        # assume this function is defined elsewhere
    form.save()
    # save the instance regardless of sending
    # the email or not
    return redirect(success_url)
```

In the following exercise, you will write a new view function in Bookr that will allow you to create and edit a Publisher.

Exercise 7.03 – Creating and editing a publisher

In this exercise, we will return to Bookr. We want to add the ability to create and edit a Publisher without using the Django admin. To do this, we'll add a `ModelForm` for the Publisher model. It will be used in a new view function. This view function will take an optional argument, `pk`, which will either be the ID of the Publisher object being edited or `None` to create a new Publisher. We will add two new URL maps to facilitate this. When this is complete, we will be able to see and update any publisher using their ID. For example, information for *Publisher 1* will be viewable/editable at the `/publishers/1` URL path:

1. In PyCharm, open the reviews app's `forms.py` file. After the forms import, import the Publisher model:

```
from .models import Publisher
```

2. Create a `PublisherForm` class, inheriting from `forms.ModelForm`:

```
class PublisherForm(forms.ModelForm):
```

3. Define the class `Meta` attribute on `PublisherForm`. The attributes that `Meta` requires are the model (`Publisher`) and fields (`"__all__"`):

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
```

Save `forms.py`.

Note

The full file can be found in the `Chapter07` folder in this book's GitHub repo.

4. Open the reviews app's `views.py` file. At the top of the file, import `PublisherForm`:

```
from .forms import PublisherForm, SearchForm
```

5. Make sure you import the `get_object_or_404` and `redirect` functions from `django.shortcuts`, if you aren't already:

```
from django.shortcuts import (
    render, get_object_or_404, redirect)
```

6. Also, make sure you're importing the `Publisher` model if you aren't already. You may already be importing this and other models:

```
from .models import Book, Contributor, Publisher
```

7. The final import you will need is the `messages` module. This will allow us to register a message letting the user know that a `Publisher` object was edited or created:

```
from django.contrib import messages
```

Once again, add this import if you do not already have it.

8. Create a new view function called `publisher_edit`. It takes two arguments: `request` and `pk` (the ID of the `Publisher` object to edit). This is optional, and if it is `None`, then a `Publisher` object will be created instead:

```
def publisher_edit(request, pk=None):
```

9. Inside the view function, we need to try to load the existing `Publisher` instance if `pk` is not `None`. Otherwise, `publisher` should be `None`:

```
def publisher_edit(request, pk=None):
    if pk is not None:
        publisher = get_object_or_404(Publisher, pk=pk)
    else:
        publisher = None
```

10. After getting a `Publisher` instance or `None`, complete the branch for a POST request. Instantiate the form in the same way as you saw earlier in this chapter, but now, make sure that it takes `instance` as a kwarg. Then, if the form is valid, save it using the `form.save()` method. This method will return the updated `Publisher` instance, which is stored in the `updated_publisher` variable. Then, register a different success message depending on whether the `Publisher` was created or updated. Finally, redirect back to

this `publisher_edit` view, since `updated_publisher` will always have an ID at this point:

```
def publisher_edit(request, pk=None):
    ...
    if request.method == "POST":
        form = PublisherForm(request.POST, instance=publisher)
        if form.is_valid():
            updated_publisher = form.save()
            if publisher is None:
                messages.success(request,
                                f'Publisher "{updated_publisher}" 'f'was created.')
            else:
                messages.success(request, "Publisher
                                f'Publisher \"{updated_publisher}\" 'f'was updated.')
        return redirect("publisher_edit",
                        updated_publisher.pk)
```

If the form is not valid, the execution will fall through and just return the render function call with the invalid form (this will be implemented in *Step 12*). The redirect uses a named URL map, which will be added later in this exercise.

11. Next, fill in the non-POST branch of the code. In this case, just instantiate the form with your instance:

```
def publisher_edit(request, pk=None):
    ...
    if request.method == "POST":
        ...
    else:
        form = PublisherForm(instance=publisher)
```

12. In *Step 14*, you'll create a `form-example.html` file to render, but we can add the call to the render function now. Render it in the view with the render function by passing in the HTTP method and form as the context:

```
def publisher_edit(request, pk=None):
    ...
    return render(request, "form-example.html",
                  {"method": request.method, "form": form})
```

Save this file. You can refer to this in the Chapter07 folder in this book's GitHub repo.

13. Open `urls.py` in the reviews directory. Add two new URL maps; they will both go to the `publisher_edit` view. One will capture the ID of `Publisher`, which we will want to edit and pass into the view as the `pk` argument. The other will use `new` instead, and will not pass the `pk` argument, which will indicate that we want to create a new `Publisher`. To your `urlpatterns` variable, add the `"publishers/<int:pk>/"` path mapping to the `reviews.views.publisher_edit` view with the name `"publisher_edit"`.

Also, add the `"publishers/new/"` path mapping to the `reviews.views.publisher_edit` view with the name `"publisher_create"`:

```
urlpatterns = [
    ...
    path("publishers/<int:pk>",
        views.publisher_edit, name="publisher_edit"),
    path("publishers/new/",
        views.publisher_edit, name="publisher_create")
]
```

Since the second mapping does not capture anything, whichever `pk` is passed to the `publisher_detail` view function is `None`.

Save the `urls.py` file. The completed version for reference is at the Chapter07 folder in this book's GitHub repo.

14. Create a `form-example.html` file inside the reviews app's templates directory by using the **New HTML File** function in PyCharm. Since this is a standalone template (it does not extend any other templates), we need to render the messages inside it. Add this code just after the opening `<body>` tag to iterate through all the messages and display them:

```
{% for message in messages %}
<p><em>{{ message.level_tag|title }}:</em>
{{ message }}</p>
{% endfor %}
```

This will loop over the messages we have added and display the tag (in our case, `Success`) and then the message.

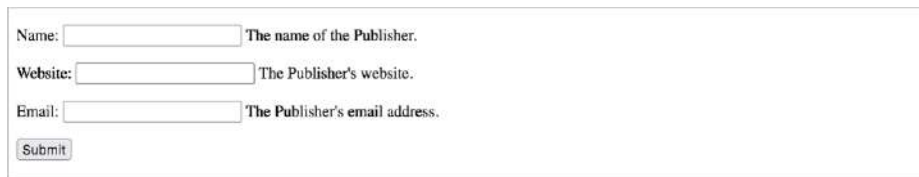
15. Then, add the normal form rendering and submission code:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <p>
        <input type="submit" value="Submit">
    </p>
</form>
```

Save and close this file.

You can refer to the full version of this file at the Chapter07 folder in this book's GitHub repo.

16. Start the Django dev server, then navigate to `http://127.0.0.1:8000/publishers/new/`. You should see a blank PublisherForm being displayed:



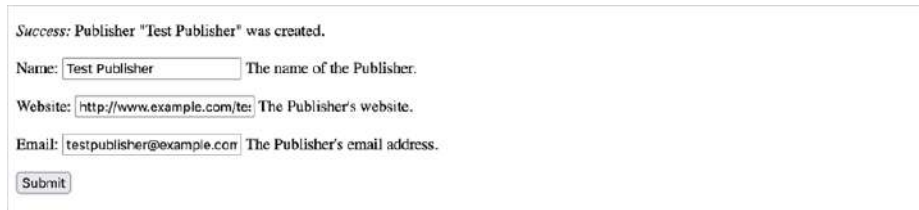
Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Figure 7.20 – Blank publisher form

17. The form has inherited the model's validation rules, so you cannot submit the form with too many characters for **Name** or with an invalid **Website** or **Email** value. Put in some valid information, then submit the form. After submission, you should see the **Success** message; the form will be populated with information that was saved to the database:



Success: Publisher "Test Publisher" was created.

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

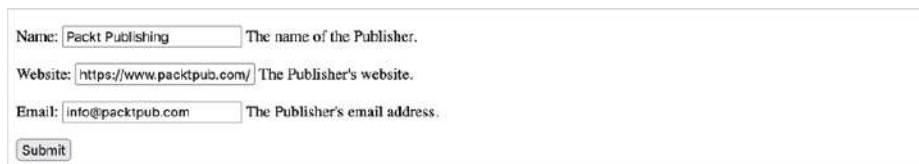
Figure 7.21 – Form after submission

Notice that the URL has also been updated and now includes the ID of the publisher that was created. In this case, it is `http://127.0.0.1:8000/publishers/10/`; the ID in

your setup will depend on how many Publisher instances were already in your database.

Notice that if you refresh the page, you will not receive a message confirming whether you want to resend the form data. This is because we redirected after saving, so it is safe to refresh this page as many times as you want, and no new Publisher instances will be created. If you had not redirected it, then every time the page was refreshed, a new Publisher instance would be created.

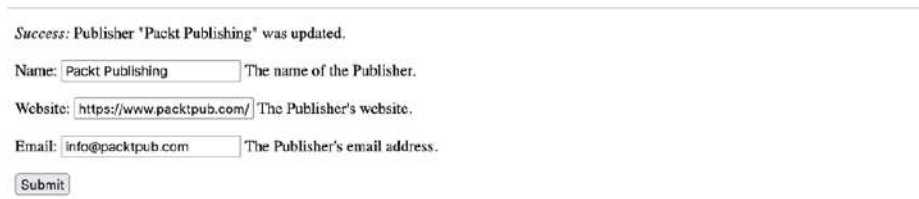
If you have other Publisher instances in your database, you can change the ID in the URL to edit other ones. Since the ID in this instance is 3, we can assume that *Publisher 1* and *Publisher 2* already exist and can substitute their IDs to see the existing data. Here is the view of the existing *Publisher 1* information (at `http://127.0.0.1:8000/publishers/1/`) – your information may be different:



A screenshot of a web form for editing a Publisher instance. The form contains three text input fields with labels and a Submit button. The first field is labeled 'Name:' and contains 'Packt Publishing'. The second field is labeled 'Website:' and contains 'https://www.packtpub.com/'. The third field is labeled 'Email:' and contains 'info@packtpub.com'. Each field has a small text label to its right: 'The name of the Publisher.', 'The Publisher's website.', and 'The Publisher's email address.' respectively. The Submit button is at the bottom left of the form.

Figure 7.22 – Existing Publisher 1 information

Try making changes to the existing Publisher instance. Notice that after you save, the message is different – it is telling the user that the Publisher instance was *updated* rather than *created*:



A screenshot of the same web form as in Figure 7.22, but with a success message at the top. The message reads: 'Success: Publisher "Packt Publishing" was updated.' Below the message, the form fields and labels are identical to the previous figure, showing the current state of the Publisher instance after an update.

Figure 7.23 – Publisher after updating instead of creating

In this exercise, we implemented a `ModelForm` from a model (`PublisherForm` was created from `Publisher`) and saw how Django automatically generated the form fields with the correct validation rules. We then used the form's built-in save method to save changes to the Publisher instance (or automatically create it) inside the `publisher_edit` view. We mapped two URLs to the view. The first was for editing an existing Publisher and passing a `pk` to the view. The other did not pass this `pk` to the view, indicating that the Publisher instance should

be created. Finally, we used the browser to experiment with creating a new Publisher instance and then editing an existing one.

Activity 7.01 – Styling and integrating the publisher form

In *Exercise 7.03 – Creating and editing a publisher*, you added `PublisherForm` to create and edit Publisher instances. You built this with a standalone template that did not extend any other templates, so it lacked the global styles. In this activity, you will build a generic form detail page that will display a Django form, similar to `form-example.html`, but extending from a base template. The template will accept a variable to display the type of model being edited. You will also update the main `base.html` template to render the Django messages, using Bootstrap styling.

These steps will help you complete this activity:

1. Start by editing the project's `base.html` file. Wrap the content block in a `div` container for a nicer layout with some spacing. Surround the existing content block with a `<div>` element with `class="container-fluid"`.
2. Render each message in `messages` (similar to *Step 14 of Exercise 7.03 – Creating and editing a publisher*). Add the `{% for %}` block after the `<div>` container you just created, but before the content block. You should use the Bootstrap framework classes – this snippet will help you:

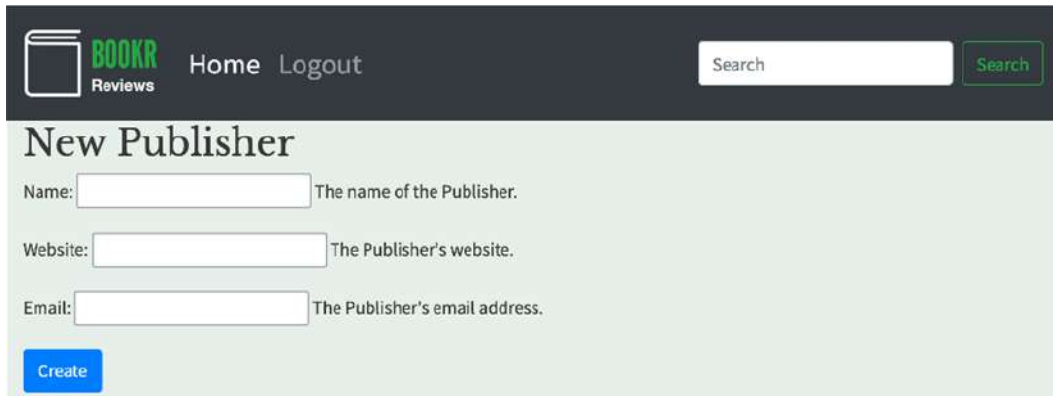
```
<div role="alert"
      class="alert alert-{{ {'error': 'danger'}.get(level_tag, level_tag)}}"
>
    {{ message }}
</div>
```

The Bootstrap class and Django message tags have corresponding names for the most part (for example, `success` and `alert-success`). The exception is Django's `error` tag. The corresponding Bootstrap class is `alert-danger`. See more information about Bootstrap alerts at <https://getbootstrap.com/docs/5.3/components/alerts/>. This is why you need to use the `if` template tag in this snippet.

3. Create a new template called `instance-form.html`, inside the `reviews` app's namespaced templates directory.
4. Now, `instance-form.html` should extend from the `reviews` app's `base.html` file.

5. The context that's being passed to this template will contain a variable called `instance`. This will be the `Publisher` instance being edited, or `None` if we are creating a new `Publisher` instance. The context will also contain a `model_type` variable, which is a string indicating the model type (in this case, `Publisher`). Use these two variables to populate the `title` block template tag:
 - If the instance is `None`, the title should be `New Publisher`
 - Otherwise, the title should be `Editing Publisher <Publisher Name>`
6. `instance-form.html` should contain a content block template tag to override the `base.html` content block.
7. Add an `<h2>` element inside the content block and populate it using the same logic as the title. For better styling, wrap the publisher name in an `<em>` element.
8. Add a `<form>` element to the template with a `method` value of `post`. Since we are posting back to the same URL, an `action` value does not need to be specified.
9. Include the CSRF token template tag in the `<form>` body.
10. Render the Django form (its context variable will be `form`) inside `<form>`, using the `as_p` method.
11. Add a submit `<button>` element to the form. Its text should depend on whether you're editing or creating. Use the `Save` text for editing or `Create` for creating. You can use the Bootstrap classes for the button styling here. It should have the `class="btn btn-primary"` attribute.
12. In `reviews/views.py`, the `publisher_edit` view does not need many changes. Update the render call to render `instance-form.html` instead of `form-example.html`.
13. Update the context dictionary being passed to the render call. It should include the `Publisher` instance (the `publisher` variable that was already defined) and the `model_type` string. The context dictionary already includes `form` (a `PublisherForm` instance). You can remove the `method` key.
14. Since we're finished with the `form-example.html` template, it can be deleted.

When you've finished, the `Publisher` creation page (at `http://127.0.0.1:8000/publishers/new/`) should look like what's shown in *Figure 7.24*:



The screenshot shows the 'New Publisher' form in the BOOKR Reviews application. The header bar is dark grey with the BOOKR Reviews logo on the left, 'Home' and 'Logout' links in the center, and a search bar on the right. The main content area has a light green background. The form title 'New Publisher' is in a large, bold, serif font. Below the title are three input fields: 'Name:', 'Website:', and 'Email:'. Each field has a placeholder text to its right: 'The name of the Publisher.', 'The Publisher's website.', and 'The Publisher's email address.' respectively. At the bottom left of the form is a blue 'Create' button.

Figure 7.24 – The Publisher creation page

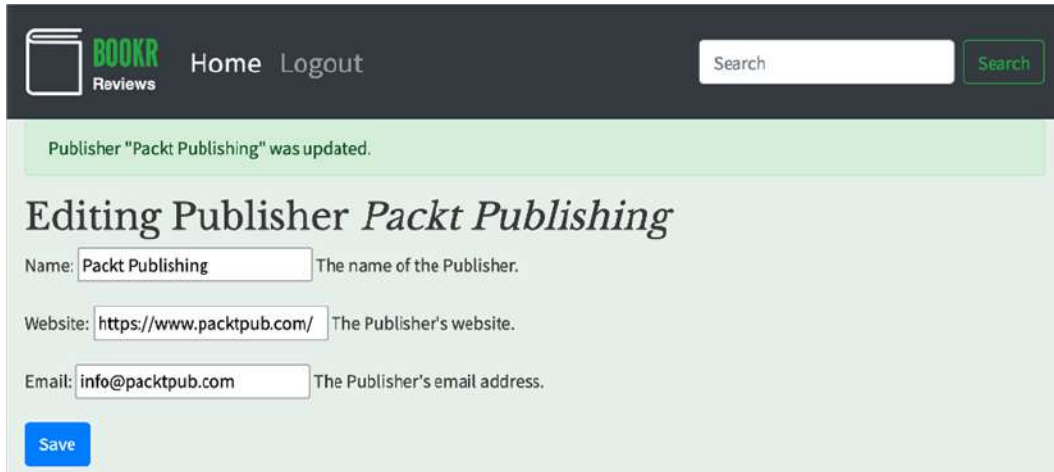
When editing a Publisher (for example, at <http://127.0.0.1:8000/publishers/1/>), your page should look like what's shown in Figure 7.25:



The screenshot shows the 'Editing Publisher' form in the BOOKR Reviews application. The header bar is dark grey with the BOOKR Reviews logo on the left, 'Home' and 'Logout' links in the center, and a search bar on the right. The main content area has a light green background. The form title 'Editing Publisher' is in a large, bold, serif font, followed by the publisher's name 'Packt Publishing' in a smaller, italicized serif font. Below the title are three input fields: 'Name:', 'Website:', and 'Email:'. Each field has a placeholder text to its right: 'The name of the Publisher.', 'The Publisher's website.', and 'The Publisher's email address.' respectively. The 'Name' field contains the text 'Packt Publishing', the 'Website' field contains 'https://www.packtpub.com/', and the 'Email' field contains 'info@packtpub.com'. At the bottom left of the form is a blue 'Save' button.

Figure 7.25 – The Editing Publisher page

After saving a Publisher instance, whether creating or editing, you should see a success message at the top of the page (Figure 7.26):



The screenshot shows a web application interface. At the top, there is a dark navigation bar with a logo on the left that says "BOOKR Reviews" (with "BOOKR" in green and "Reviews" in white). To the right of the logo are links for "Home" and "Logout". Further right is a search bar with a "Search" button. Below the navigation bar, a light green alert box displays the message: "Publisher 'Packt Publishing' was updated." Below the alert, the page title is "Editing Publisher *Packt Publishing*". The form contains three input fields: "Name:" with the value "Packt Publishing" and the label "The name of the Publisher.", "Website:" with the value "https://www.packtpub.com/" and the label "The Publisher's website.", and "Email:" with the value "info@packtpub.com" and the label "The Publisher's email address.". A blue "Save" button is located at the bottom left of the form area.

Figure 7.26 – Success message rendered as a Bootstrap alert

Activity 7.02 – Review creation UI

Activity 7.01 – Styling and integrating the publisher form, was quite extensive; however, by completing it, you have created a foundation that makes it easier to add other edit and create views. You will experience this first-hand in this activity when you build forms for creating and editing reviews. Because the `instance-form.html` template was made generically, you can reuse it in other views.

In this activity, you will create a `review ModelForm` and then add a `review_edit` view to create or edit a `Review` instance. You can reuse `instance-form.html` from *Activity 7.01 – Styling and integrating the Publisher form*, and pass in different context variables to make it work with the `Review` model. When working with reviews, you'll operate within the context of a book – that is, the `review_edit` view must accept a `Book pk` as an argument. You'll fetch this `Book` separately and assign it to the `Review` instance that you create.

These steps will help you complete this activity:

1. In `forms.py`, add a `ReviewForm` subclass of `ModelForm`; its model should be `Review` (make sure you import the `Review` model).
`ReviewForm` should exclude the `date_edited` and `book` fields since the user should not be setting these in the form. The database allows any rating, but we can override the rating field with an `IntegerField` that requires a minimum value of 0 and a maximum value of 5.

2. Create a new view called `review_edit`. It should accept two arguments after request: `book_pk`, which is required, and `review_pk`, which is optional (defaults to `None`). Fetch the `Book` instance and `Review` instance using the `get_object_or_404` shortcut (call it once for each type). When fetching the review, make sure the review belongs to the book. If `review_pk` is `None`, then the `Review` instance should be `None` too.
3. If the request method is `POST`, then instantiate a `ReviewForm` using `request.POST` and the `Review` instance. Make sure you import this `ReviewForm`.
If the form is valid, save the form but change the `commit` argument from `save` to `False`. Then, set the `book` attribute on the returned `Review` instance to the book that was fetched in *Step 2*.

If the `Review` instance is being updated instead of created, then you should also set the `date_edited` attribute to the current date and time. Use the `from django.utils.timezone.now()` function. Then, save the `Review` instance.
4. Finish the valid form branch by registering a success message and redirecting back to the `book_detail` view. Since the `Review` model doesn't contain a meaningful text description, use the `Book` title in the message – for example, `Review for "<book title>" created`.
5. If the request method is not `POST`, instantiate a `ReviewForm` and just pass in the `Review` instance.
6. Render the `instance-form.html` template. In the context dictionary, include the same items as were used in `publisher_view`: `form`, `instance`, and `model_type` (`Review`). Include two extra items – `related_model_type`, which should be `Book`, and `related_instance`, which will be the `Book` instance.
7. Edit `instance-form.html` to add a place to display the related instance information added in *Step 6*. Under the `<h2>` element, add a `<p>` element that is only displayed if both `related_model_type` and `related_instance` are set. It should show the text `For <related_model_type> <related_instance>` – for example, `For Book Advanced Deep Learning with Keras`. Put the `related_instance` output in an `<em>` element for better readability.
8. In the `reviews` app's `urls.py`, add URL maps to the `review_edit` view. The `/books/` and `/books/<pk>/` URLs have already been configured. Add `/books/<book_pk>/reviews/new/` to create a review and `/books/<book_pk>/reviews/<review_pk>/` to

edit a review. Make sure you give these names, such as `review_create` and `review_edit`.

9. Inside the `book_detail.html` template, add links that a user can click on to create or edit a review. Add a link inside the content block, just before the `endblock` closing template tag. It should use the `url` template tag to link to the `review_edit` view when in creation mode. Also, use the `class="btn btn-primary"` attribute to make the link display like a Bootstrap button. The link text should be `Add Review`.
10. Finally, add a link to edit a review, inside the `for` loop that iterates over `Reviews` for `Book`. After all the instances of `text-info <span>`, add a link to the `review_edit` view using the `url` template tag. You will need to provide `book.pk` and `review.pk` as arguments. The text of the link should be `Edit Review`.

When you're finished, the **Review Comments** page should look like what's shown in *Figure 7.27*:

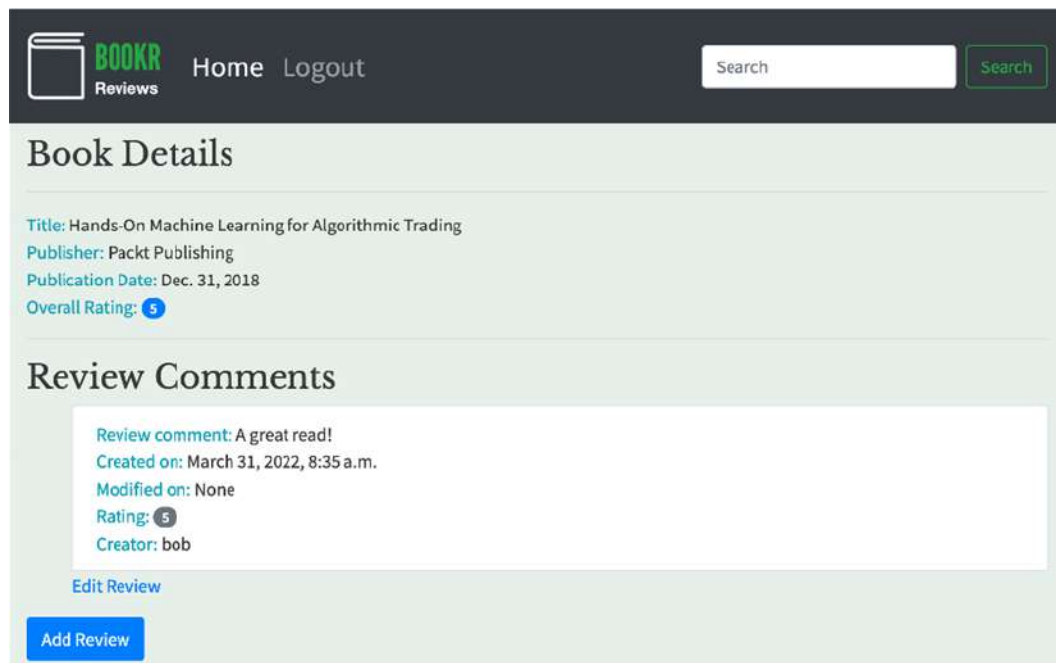
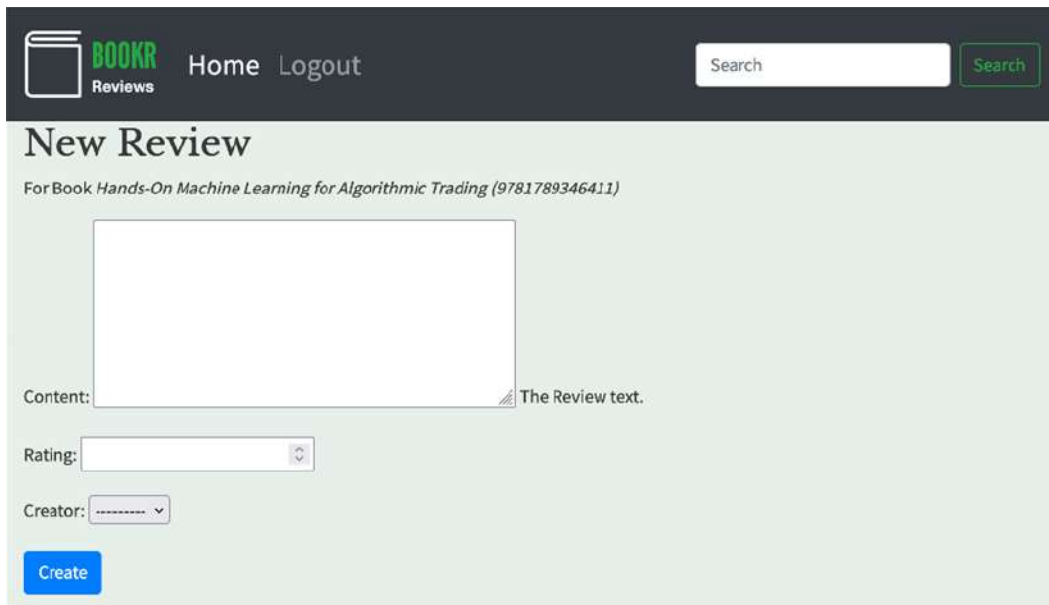


Figure 7.27 – The Book Details page with the added Add Review button

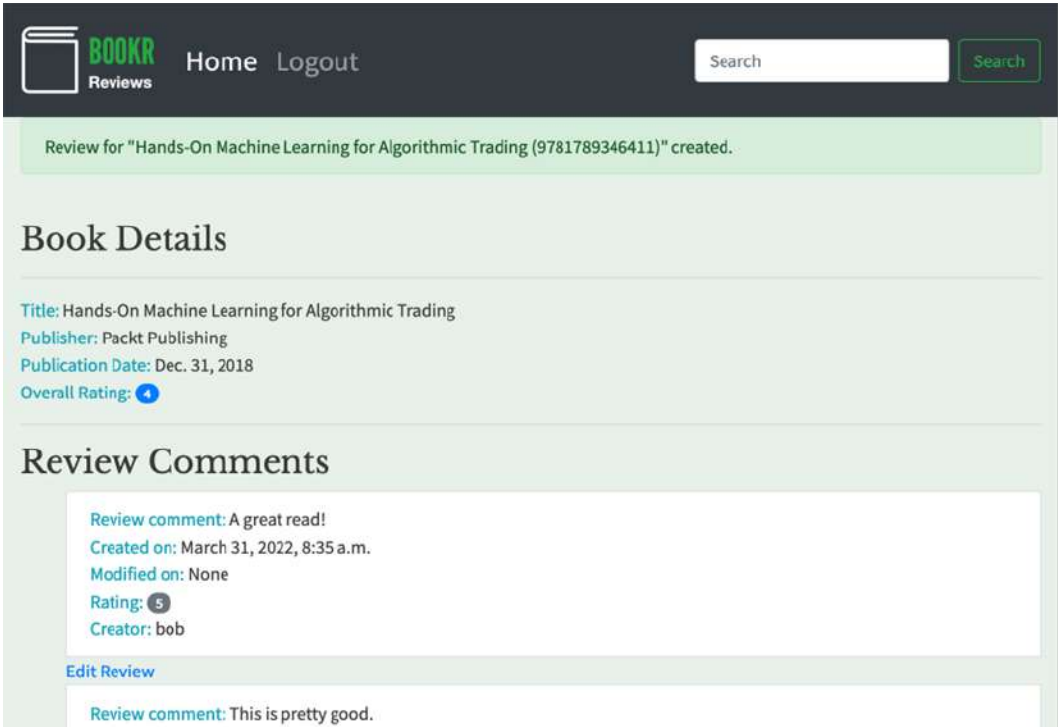
You will see the **Add Review** button. Clicking it will take you to the **New Review** page, which should look like what's shown in *Figure 7.28*:



The screenshot shows a web application interface for adding a new review. At the top is a dark navigation bar with a logo on the left that says "BOOKR Reviews" next to a book icon. To the right of the logo are links for "Home" and "Logout". On the far right of the navigation bar is a search bar with the text "Search" inside and a green "Search" button to its right. Below the navigation bar, the main content area has a light green background. It starts with the heading "New Review" in a large, bold font. Underneath the heading is a line of text: "For Book Hands-On Machine Learning for Algorithmic Trading (9781789346411)". The main form consists of three fields: a large text area labeled "Content:" on the left, a "Rating:" field with a dropdown arrow, and a "Creator:" field with a dropdown arrow. To the right of the "Content:" field, there is a small icon of a document with a diagonal line and the text "The Review text." At the bottom left of the form is a blue "Create" button.

Figure 7.28 – The New Review page

Enter some details in the form and click **Create**. You'll be redirected to the **Book Details** page. You should see the success message and your review, as shown in *Figure 7.29*:



The screenshot shows the BOOKR Reviews website interface. At the top, there is a dark navigation bar with the BOOKR Reviews logo, links for Home and Logout, and a search bar. Below the navigation bar, a green success message states: "Review for 'Hands-On Machine Learning for Algorithmic Trading (9781789346411)' created." The main content area is titled "Book Details" and lists the following information: Title: Hands-On Machine Learning for Algorithmic Trading, Publisher: Packt Publishing, Publication Date: Dec. 31, 2018, and Overall Rating: 4. Below this, the "Review Comments" section displays a single review comment: "A great read!" with a rating of 5, created on March 31, 2022, at 8:35 a.m., by user bob. An "Edit Review" link is visible below the comment.

BOOKR Reviews Home Logout Search

Review for "Hands-On Machine Learning for Algorithmic Trading (9781789346411)" created.

Book Details

Title: Hands-On Machine Learning for Algorithmic Trading
Publisher: Packt Publishing
Publication Date: Dec. 31, 2018
Overall Rating: 4

Review Comments

Review comment: A great read!
Created on: March 31, 2022, 8:35 a.m.
Modified on: None
Rating: 5
Creator: bob

[Edit Review](#)

Review comment: This is pretty good.

Figure 7.29 – The Book Details page with a review added

You will also see the **Edit Review** link. If you click it, you will be taken to a form that has been pre-populated with your review data (see *Figure 7.30*):

The screenshot shows a web application interface for editing a book review. At the top is a dark navigation bar with a 'BOOKR Reviews' logo, 'Home' and 'Logout' links, and a search bar with a 'Search' button. Below the navigation bar, the page title is 'Editing Review *bob - Hands-On Machine Learning for Algorithmic Trading*'. Underneath the title is a subtitle: 'For Book *Hands-On Machine Learning for Algorithmic Trading* (9781789346411)'. The main form area has a light green background. It contains a large text input field with the placeholder text 'A great read!'. Below this field is a label 'Content:' followed by a small icon and the text 'The Review text.'. Below the content field is a 'Rating:' label followed by a dropdown menu showing the value '5'. Below the rating is a 'Creator:' label followed by a dropdown menu showing the value 'bob'. At the bottom left of the form is a blue 'Save' button.

Figure 7.30 – Review form when editing a review

After saving an existing review, you should see that the **Modified on** date has been updated on the **Book Details** page (Figure 7.31):

BOOKR Reviews Home Logout Search

Publisher: Packt Publishing
Publication Date: Dec. 31, 2018
Overall Rating: 4

Review Comments

Review comment: A great read!
Created on: March 31, 2022, 8:35 a.m.
Modified on: March 31, 2022, 8:50 a.m.
Rating: 4
Creator: bob

[Edit Review](#)

Review comment: This is pretty good.
Created on: March 31, 2022, 8:36 a.m.
Modified on: None
Rating: 3
Creator: carol

[Edit Review](#)

[Add Review](#)

Figure 7.31 – The Modified on date field has now been populated

Summary

This chapter provided a deep dive into forms. We saw how to enhance Django forms with custom validation, advanced rules for cleaning data, and validating fields. We also saw how custom cleaning methods can transform the data that we get out of forms. A nice feature we saw that can be added to forms is the ability to set initial and placeholder values on fields so that the user does not have to fill them out.

We then looked at how to use the `ModelForm` class to automatically create a form from a Django model. We saw how to only show some fields to the user and how to apply custom form validation rules to `ModelForm`. We also saw how Django can automatically save the new or updated model instance to the database inside the view. In the activities for this chapter, we enhanced Bookr further by adding forms for creating and editing publishers and submitting reviews.

In the next chapter, we will continue with the theme of submitting user input, and we'll discuss how Django handles file uploads and downloads.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

8

Media Serving and File Uploads

Media files refer to extra files that can be added after deployment to enrich your Django application. Usually, they are extra images that you would use on your site, but any type of file (including video, audio, PDF, text, documents, or even HTML) can be served as media.

You can think of them as somewhere between dynamic data and static assets. They aren't dynamic data that Django generates on the fly, such as when rendering a template. They also aren't the static files that are included by the site developer when the site is deployed. Instead, they are extra files that can be uploaded by users or generated by your application for later retrieval.

Some common examples of media files (which you'll see in *Activity 8.01 – image and PDF upload for books*, later in this chapter) are book covers and preview PDFs that can be attached to a Book object. You can also use media files to allow users to upload images for a blog post or avatars for a social media site. If you wanted to use Django to build your own video-sharing platform, you would store the uploaded videos as media. Your website will not function well if all these files are static files, as users won't be able to upload their own book covers, videos, and so on, and will be stuck with the ones you deployed.

In this chapter, we will be covering the following topics:

- Settings for media uploads and serving
- File uploads using HTML forms
- File uploads with Django forms
- Image uploads with Django forms
- Serving uploaded (and other) files using Django
- `ModelForm` and file uploads

- Activity 8.01 – image and PDF upload for books
- Activity 8.02 – displaying the cover and sample link

Technical requirements

Find the solution in the `Chapter08` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code file* section in the *Preface*.

Settings for media uploads and serving

In *Chapter 5, Serving Static Files*, we looked at how Django can be used to serve static files. Serving media files is quite similar. Two settings must be configured in `settings.py`: `MEDIA_ROOT` and `MEDIA_URL`. These are analogous to `STATIC_ROOT` and `STATIC_URL` for serving static files. They are as follows:

- `MEDIA_ROOT`: This is the path on the disk where the media (such as uploaded files) will be stored. As with static files, your web server should be configured to serve directly from this directory to take the load off Django.
- `MEDIA_URL`: This is similar to `STATIC_URL`, but as you might guess, it's the URL that should be used to serve media. It must end in `/`. Generally, you will use something such as `/media/`.

Note

For security reasons, the path for `MEDIA_ROOT` must not be the same as the path for `STATIC_ROOT`, and `MEDIA_URL` must not be the same as `STATIC_URL`. If they were the same, a user might replace your static files (such as JavaScript or CSS files) with malicious code and exploit your users.

`MEDIA_URL` is designed to be used in templates so that you are not hardcoding the URL, and it can be changed easily. For example, you might want to set it to a specific host or **content delivery network (CDN)** when you deploy to production. We will discuss the use of `MEDIA_URL` in templates in the next section.

Serving media files in development

As with static files, when serving media in production, your web server should be configured to serve directly from the `MEDIA_ROOT` directory to prevent Django from being tied up in servicing the request. The Django dev server can serve media files in development. However, unlike static files, the URL mapping and view are not set up automatically for media files.

Django provides the static URL mapping, which can be added to your existing URL maps to serve media files. It is added to your `urls.py` file like this:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # your existing URL maps
]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                          document_root=settings.MEDIA_ROOT)
```

This will serve the `MEDIA_ROOT` setting defined in `settings.py` to the `MEDIA_URL` setting that is also defined there. The reason we check for `settings.DEBUG` before appending the map is so that we don't add this map in production.

For example, if your `MEDIA_ROOT` was set to `/var/www/bookr/media`, and your `MEDIA_URL` was set to `/media/`, then the `/var/www/bookr/media/image.jpg` file would be available at `http://127.0.0.1:8000/media/image.jpg`.

The static URL map does not work when the Django `DEBUG` setting is `False`, so it can't be used in production. However, as has been mentioned, in production, your web server should be serving these requests, so Django will not need to handle them.

In the first exercise, you will add `MEDIA_ROOT` and `MEDIA_URL` to your `settings.py`. You will then add the static media-serving URL map and add a test file to ensure media serving is configured correctly.

Exercise 8.01 – configuring media storage and serving

In this exercise, you will set up a new Django project as an example project to use throughout this chapter. Then, you'll configure it to be able to serve media files. You'll do this by creating a media directory and adding the `MEDIA_ROOT` and `MEDIA_URL` settings. Then, you'll set up the URL mapping for `MEDIA_URL`.

To check that everything is configured and being served correctly, you will put a test file inside the media directory:

1. As with the previous example Django projects you've set up, you can reuse the existing djangoenv virtual environment. In a terminal, activate the djangoenv virtual environment, using `pyenv local djangoenv`. Then, start a new project named `media_project` using `django-admin.py`:

```
django-admin startproject media_project
```

Note

To learn how to create and activate a virtual environment using pyenv, refer to *Chapter 1* of this book.

Change (or `cd`) into the `media_project` directory that was created, then use the `startapp` management command to start an app called `media_example`:

```
python manage.py startapp media_example
```

2. Open the `media_project` directory in PyCharm. Set up a run configuration for the `runserver` command in the same manner as for the other Django projects you've opened:

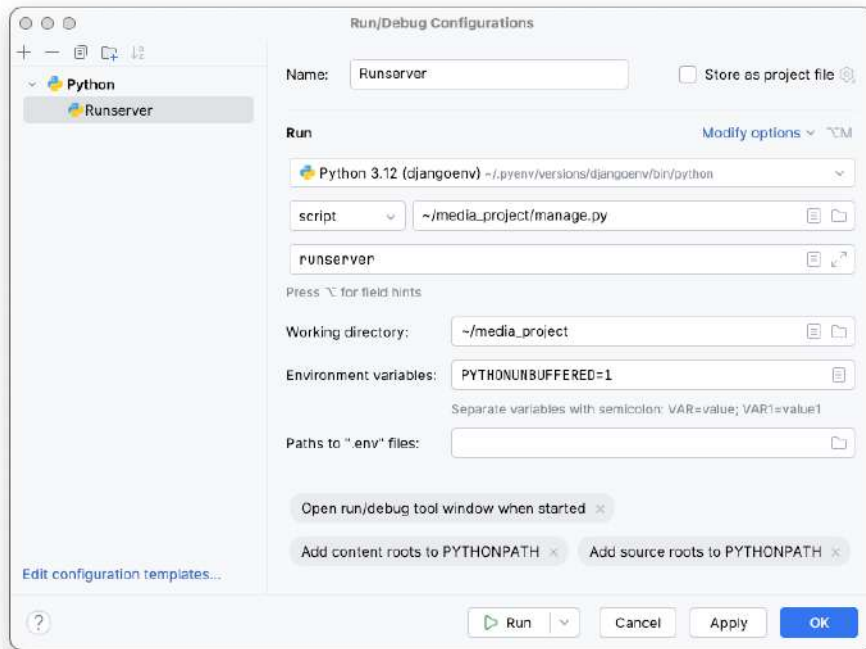


Figure 8.1: Runserver configuration

Figure 8.1 shows the **Runserver** configuration of the project in PyCharm.

3. Create a new directory named `media` inside the `media_project` project directory. Then, create a new file in this directory named `test.txt`. The directory structure of this will look like Figure 8.2:



Figure 8.2: The media directory and the test.txt layout

4. The `test.txt` file will also open automatically. Enter `Hello world!` into it, then you can save and close the file.

5. Open `settings.py` inside the `media_project` package directory. At the end of the file, add a setting for `MEDIA_ROOT` using the path to the media directory you just created. Join the name of the directory (`media`) onto `BASE_DIR`:

```
MEDIA_ROOT = BASE_DIR / "media"
```

6. Directly below the line added in step 5, add another setting for `MEDIA_URL`; this should just be `"/media/"`:

```
MEDIA_URL = "/media/"
```

With these changes made, `settings.py` should look like the file placed in the `Chapter08` folder in this book's GitHub repo.

7. Open the `media_project` package's `urls.py` file. After the `urlpatterns` definition, add the following code to add the media-serving URL if running in the `DEBUG` mode. First, you will need to import the Django settings and static serving view by adding the highlighted import lines above the `urlpatterns` definition:

```
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path("admin/", admin.site.urls),
]
```

8. Then, add the following code right after your `urlpatterns` definition (refer to the code block in the previous step) to conditionally add a mapping from the `MEDIA_URL` setting to the static view, which will serve from `MEDIA_ROOT`:

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT)
```

You can now save this file.

9. Start the Django dev server, if it is not already running, then visit `http://127.0.0.1:8000/media/test.txt`. If you did everything correctly, you should see the **Hello world!** text in your browser:



Figure 8.3: Serving a media file

If your browser looks like *Figure 8.3*, it means that the media files are being served from the `MEDIA_ROOT` directory.

The `test.txt` file we created was just for testing, but we will use it in *Exercise 8.02 – template settings and using `MEDIA_URL` in templates*, so don't delete it yet.

In this exercise, we configured Django to serve media files. We served a test file just to make sure everything worked as expected, and it did. We'll now look at how we can automatically generate media URLs in templates.

Context processors and using `MEDIA_URL` in templates

To use `MEDIA_URL` in a template, we could pass it in through the rendering context dictionary in our view. An example is shown in the following code block:

```
from django.conf import settings

def my_view(request):
    return render(request, "template.html",
                  {"MEDIA_URL": settings.MEDIA_URL,
                   "username": "jbloggs"})
```

This will work, but the problem is that `MEDIA_URL` is a common variable that we might want to use in many places, so we'd have to pass it through in practically every view.

Instead, we can use a **context processor**, which is a way of adding one or more variables automatically to the context dictionary on every render call.

A context processor is a function that accepts one argument, the current request. It returns a dictionary of context information that will be merged with the dictionary that was passed to the render call.

We can look at the source code of the `media` context processor, which illustrates how it works:

```
def media(request):  
    """  
    Add media-related context variables to the context.  
    """  
    return {"MEDIA_URL": settings.MEDIA_URL}
```

With the `media` context processor activated, `MEDIA_URL` will be added to your context dictionaries. We could change our render call, seen previously, to this:

```
return render(request, "template.html",  
              {"username": "jbloggs"})
```

The same data would be sent to the template, as the context processor would add `MEDIA_URL`.

The full module path to the `media` context processor is

`django.template.context_processors.media`. Some examples of other context processors that Django provides are the following:

- `django.template.context_processors.debug`: This returns the `{"DEBUG": settings.DEBUG}` dictionary
- `django.template.context_processors.request`: This returns the `{"request": request}` dictionary; that is, it just adds the current HTTP request to the context

To enable a context processor, its module path must be added to the `context_processors` option of your `TEMPLATES` setting. For example, to enable the `media` context processor, add `django.template.context_processors.media`. We will cover how to do this in detail in *Exercise 8.02 – template settings and using MEDIA_URL in templates*.

Once the `media` context processor is enabled, the `MEDIA_URL` variable can be accessed inside a template just like a normal variable:

```
{{ MEDIA_URL }}
```

You could use it, for example, to source an image:

```

```

Note that, unlike with static files, there is no template tag for loading media files (i.e., there is no equivalent to the `{% static %}` template tag).

Custom context processors can also be written. For example, referring back to the Bookr application that we built, we might want to show a list of the five latest reviews in a sidebar that's on every page. A context processor like this would perform the following code:

```
from reviews.models import Review

def latest_reviews(request):
    return {"latest_reviews": Review.objects.order_by(
        "-date_created")[:5]}.
```

This would be saved in a file named `context_processors.py` in the Bookr project directory, then referred to in the `context_processors` setting by its module path, `context_processors.latest_reviews`. Or we could save it inside the reviews app and refer to it as `reviews.context_processors.latest_reviews`. It is up to you to decide whether a context processor should be considered project-wide or app-specific. However, bear in mind that regardless of where it is stored, once activated, it applies to all the render calls for all apps.

A context processor can return a dictionary with multiple items or even zero items. It would do this if it had conditions to only add items if certain criteria were met. For example, show the latest reviews only if the user is logged in.

Let us explore this in detail in the next exercise.

Exercise 8.02 – template settings and using MEDIA_URL in templates

In this exercise, you will continue with `media_project` and configure Django to automatically add the `MEDIA_URL` setting to every template. You do this by adding `django.template.context_processors.media` to the `TEMPLATES` `context_processors` setting. You'll then add a template that uses this new variable and an example view to render it. You will make changes to the view and template throughout the exercises in this chapter:

1. In PyCharm, open `settings.py`. First, you'll need to add `media_example` to the `INSTALLED_APPS` setting since it wasn't done when the project was set up:

```
INSTALLED_APPS = [  
    # other apps truncated for brevity  
    "media_example",  
]
```

2. About halfway down the file, you'll find the `TEMPLATES` setting, which is a dictionary. Inside it is the `OPTIONS` item (another dictionary). Inside `OPTIONS` is the `context_processors` setting. Add the following to the end of this list:

```
"django.template.context_processors.media"
```

The full list should look like this:

```
TEMPLATES = [{  
    "BACKEND":  
        "django.template.backends.django.DjangoTemplates",  
    "DIRS": [],  
    "APP_DIRS": True,  
    "OPTIONS": {  
        "context_processors": [  
            "django.template.context_processors.debug",  
            "django.template.context_processors.request",  
            "django.contrib.auth.context_processors.auth",  
            "django.contrib.messages.context_processors.messages",  
            "django.template.context_processors.media",  
        ],  
    },  
}]
```

```
    },  
    },]
```

3. Open the `media_example` app's `views.py` file and create a new view called `media_example`. For now, it can just render a template named `media-example.html` (you will create this in step 5). The entire code of the view function is like this:

```
def media_example(request):  
    return render(request, "media-example.html")
```

4. Save `views.py`. You need a URL mapping to the `media_example` view. Open the `media_project` package's `urls.py` file.
First, import `media_example.views` with the other imports in the file:

```
import media_example.views
```

Then add `path` into `urlpatterns` to map `media-example/` to the `media_example` view:

```
path('media-example/', media_example.views.media_example)
```

Your full `urlpatterns` should look like this code block:

```
from django.conf.urls.static import static  
  
import media_example.views  
  
urlpatterns = [  
    path("admin/", admin.site.urls),  
    path("media-example/",  
        media_example.views.media_example),  
]  
  
if settings.DEBUG:  
    urlpatterns += static(settings.MEDIA_URL,  
                           document_root=settings.MEDIA_ROOT)
```

You can save and close the file.

5. Create a templates directory inside the media_example app directory. Then, create a new HTML file inside the media_project project's templates directory. Select **HTML File** and name the file media-example.html.
6. The media-example.html file should open automatically. You are just going to add a link inside the file to the test.txt file you created in *Exercise 8.01 – configuring media storage and serving*. Inside the <body> element, add the highlighted code:

```
<body>  
  <a href="{ { MEDIA_URL }}test.txt">Test Text  
  File</a>  
</body>
```

7. Note that there is no / between MEDIA_URL and the filename – this is because we already added a trailing slash when we defined it in settings.py. You can save the file. Start the Django dev server if it's not already running, then visit `http://127.0.0.1:8000/media-example/`. You should see a simple page like in *Figure 8.4*:



Figure 8.4: Basic media link page

If you click the link, you will be taken to the test.txt display and see the *Hello world!* text you created in *Exercise 8.01 – configuring media storage and serving* (Figure 8.3). This means you have configured the Django context_processors settings correctly.

We have finished with test.txt, so you can delete the file now. We will use the media_example view and template in the other exercises, so retain them. In the next section, we will talk about how to upload files using a web browser and how Django accesses them in a view.

File uploads using HTML forms

In *Chapter 6, Forms*, we learned about HTML forms. We discussed how to use the method attribute of <form> for the GET or POST requests. So far, we have only submitted text data using a form, but it is also possible to submit one or more files using a form.

When submitting files, we must ensure that there are at least two attributes on the form: method and enctype. You may still also need other attributes, such as action. A form that supports file uploads might look like this:

```
<form method="post" enctype="multipart/form-data">
```

File uploads are only available for POST requests. They are not possible with GET requests, as it would be impossible to send all the data for a file through a URL. The enctype attribute must be set to let the browser know it should send the form data as multiple parts, one part for the text data of the form and separate parts for each of the files that have been attached to the form. This encoding is seamless to the user; they don't know how the browser is encoding the form, and they do not need to do anything different.

To attach files to a form, you need to create an input of the file type. You can manually write the HTML code like this:

```
<input type="file" name="file-upload-name">
```

When the input is rendered in the browser, it looks like this when empty:

No file selected.

Figure 8.5: Empty file input

The title of the button might be different depending on your browser.

Clicking the **Browse...** button will display a *file open* dialog box:

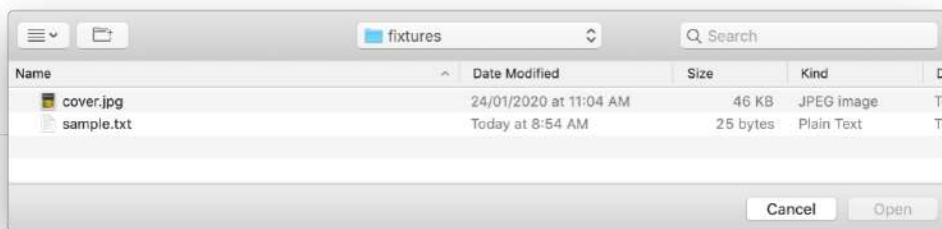


Figure 8.6: File browser on macOS

After selecting a file, the name of the file is shown in the field:



Figure 8.7: File input with *cover.jpg* selected

Figure 8.7 shows a file input with a file named *cover.jpg* having been selected.

We briefly looked at how file upload fields look in the browser. Let's start looking at the Django side of the file upload process.

Working with uploaded files in a view

In addition to text data, if a form also contains file uploads, Django will populate the `request.FILES` attribute with uploaded files. The `request.FILES` attribute is a dictionary-like object that is keyed on the name attribute given to the file input.

In the form example in the previous section, the file input had the name `file-upload-name`. So, the file would be accessible in the view using `request.FILES["file-upload-name"]`.

The objects that `request.FILES` contains are file-like objects (specifically a `django.core.files.uploadedfile.UploadFile` instance), so to use them, you must read their data. For example, to get the content of an uploaded file in your view, you can write the following:

```
content = request.FILES["file-upload-name"].read()
```

A more common action is to write the file contents to disk. When files are uploaded, they are stored in a temporary location (in memory if they are under 2.5 MB, otherwise in a temporary file on disk). To store the file data in a known location, the contents must be read and then written to disk at that location. `UploadFile` has a `chunks` method that will read the file data one chunk at a time to prevent too much memory from being used by reading the entirety of the file at once.

So, instead of simply using the `read` and `write` functions, use the `chunks` method to only read small chunks of the file into memory at a time:

```
with open("/path/to/output.jpg", "wb+") as output_file:
    uploaded_file = request.FILES["file-upload-name"]
    for chunk in uploaded_file.chunks():
        output_file.write(chunk)
```

Note that in some of the upcoming examples, we will refer to this code as the `save_file_upload` function. Assume that the function is defined like this:

```
def save_file_upload(upload, save_path):  
    with open(save_path, "wb+") as output_file:  
        for chunk in upload.chunks():  
            output_file.write(chunk)
```

The previous example code could then be refactored to call the function:

```
uploaded_file = request.FILES["file-upload-name"]  
save_file_upload(uploaded_file, "/path/to/output.jpg")
```

Each `UploadedFile` object (the `uploaded_file` variable in the previous example code snippets) also contains extra metadata about the uploaded file, such as the file's name, size, and content type. The attributes you'll find most useful are the following:

- **size:** As the name suggests, this is the size of the uploaded file in bytes.
- **name:** This refers to the name of the uploaded file, for example, `image.jpg`, `file.txt`, `document.pdf`, and so on. This value is sent by the browser.
- **content_type:** The content type (MIME type) of the uploaded file, for example, `image/jpeg`, `text/plain`, `application/pdf`, and so on. Like `name`, this value is sent by the browser.
- **charset:** This refers to the charset or text encoding of the uploaded file for text files. This will be something such as `UTF-8` or `ASCII`. Once again, this value is also determined and sent by the browser.

Here is a quick example of accessing these attributes (such as inside a view):

```
upload = request.FILES["file-upload-name"]  
size = upload.size  
name = upload.name  
content_type = upload.content_type  
charset = upload.charset
```

Security and trust of browsers' sent values

As we just described, `UploadedFile` values for `name`, `content_type`, and `charset` are determined by the browser. This is important to consider because a malicious user could send fake values in place of real ones to disguise the actual files being uploaded. Django does not automatically try to determine the content type or charset of the uploaded file, so it relies on the client to be accurate when it sends this information.

If we manually handle the saving of file uploads without suitable checks, then a scenario such as this could happen:

1. A user of the site uploads a malicious `malware.exe` executable but sends the `image/jpeg` content type.
2. Our code checks the content type and considers it to be safe, so it saves `malware.exe` to the `MEDIA_ROOT` file.
3. Another user of the site downloads what they think is a book cover image but is actually the `malware.exe` executable. They open the file, and their computer is infected with malware.

This scenario has been simplified; the malicious file would probably have a name that wasn't so obvious (maybe something such as `cover.jpg.exe`), but the general process has been illustrated.

How you choose to handle the security of your uploads will depend on the specific use case, but for most cases, these tips will help:

- When you save the file to disk, generate a name instead of using the one provided by the uploader. You should replace the file extension with what you expect. For example, if a file is named `cover.exe` but the content type is `image/jpeg`, save the file as `cover.jpg`. You could also generate a completely random filename for extra security.
- Check that the filename extension matches the content type. This method is not foolproof, as there are so many MIME types that if you are handling uncommon files, you might not get a match. The built-in Python `mimetypes` module can help you here. Its `guess_type` function takes a filename and returns a `mimetype` tuple (content type) and encoding. Here's a short snippet showing its use in a Python console:

```
>>> import mimetypes
>>> mimetypes.guess_type('file.jpg')
('image/jpeg', None)
>>> mimetypes.guess_type('text.html')
```

```
('text/html', None)
>>> mimetypes.guess_type('unknownfile.abc')
(None, None)
>>> mimetypes.guess_type('archive.tar.gz')
('application/x-tar', 'gzip')
```

Either element of the tuple might be `None` if the type or encoding can't be guessed. Once imported into your file by doing `import mimetypes`, you can use it like this in your view function:

```
upload = request.FILES["file-upload-name"]
mimetype, encoding = mimetypes.guess_type(upload.name)
if mimetype != upload.content_type:
    raise TypeError("Mimetype doesn't match file extension.")
```

This method will work for common file types such as images, but as mentioned, many uncommon types may return `None` for `mimetype`.

- If you are expecting image uploads, use the Pillow library to try to open the uploaded file as an image. If it is not a valid image, then Pillow will be unable to open it. This is what Django does when using `ImageField` to upload images. We will show how to use this technique to open and manipulate an image in *Exercise 8.05 – image uploads using Django forms*.
- You can also consider the `python-magic` Python package, which examines the actual content of files to try to determine their type. It is installed using `pip`, and its GitHub project is found here: <https://github.com/ahupp/python-magic>. Once installed and imported into your file with `import magic`, you can use it like this in your view function:

```
upload = request.FILES["field_name"]
mimetype = magic.from_buffer(upload.read(2048), mime=True)
```

You can then verify that `mimetype` is in a list of allowed types.

This is not a definitive list of all the ways of protecting against malicious file uploads. The best approach will depend on what type of application you are building. You might build a site for hosting arbitrary files, in which case you wouldn't need any kind of content checking at all.

Let us now see how we can build an HTML form and view that allows files to be uploaded. We will then store them inside the media directory and retrieve the downloaded files in our browser.

Exercise 8.03 – file upload and download

In this exercise, you will add a form with a file field to the `media-example.html` template. This will allow you to upload a file to the `media_example` view using your browser. You'll also update the `media_example` view to save the file to the `MEDIA_ROOT` directory so that it's available for download. You will then test that this all works by downloading the file again:

1. In PyCharm, open the `media-example.html` template located inside the templates folder. Inside the `<body>` element, remove the `<a>` link that was added in step 6 of *Exercise 8.02 – template settings and using MEDIA_URL in templates*. Replace it with a `<form>` element (highlighted in the following code snippet). Make sure the opening tag has `method="post"` and `enctype="multipart/form-data"`:

```
</head>
<body>
    <form method="post" enctype="multipart/form-data">
    </form>
</body>
```

2. Insert the `{% csrf_token %}` template tag inside the `<form>` body.
3. After `{% csrf_token %}`, add an `<input>` element, with `type="file"` and `name="file_upload"`:

```
<input type="file" name="file_upload">
```

4. Finally, before the closing `</form>` tag, add a `<button>` element with `type="submit"` and the Submit text content:

```
<button type="submit">Submit</button>
```

Your HTML body should now look like this:

```
<body>
    <form method="post" enctype="multipart/form-data">
        {% csrf_token %}
```

```
<input type="file" name="file_upload">
<button type="submit">Submit</button>
</form>
</body>
```

Now, save and close the file.

5. Open the `media_example` app's `views.py` file. Inside the `media_example` view, add code to save the uploaded file to the `MEDIA_ROOT` directory. For this, you need access to `MEDIA_ROOT` from the settings, so import the Django settings at the top of the file:

```
from django.conf import settings
```

6. The uploaded file should only be saved if the request method is POST. Inside the `media_example` view, add an if statement to validate that `request.method` is POST:

```
def media_example(request):
    if request.method == "POST":
        ...
```

7. Inside the if statement added in the previous step, generate the output path by joining the uploaded filename to the `MEDIA_ROOT` directory. Then, open this path in the `wb` mode and iterate over the uploaded file using the `chunks` method. Finally, write each chunk to the saved file:

```
def media_example(request):
    if request.method == 'POST':
        save_path = (settings.MEDIA_ROOT /
                     request.FILES["file_upload"].name)

        with open(save_path, "wb") as output_file:
            for chunk in request.FILES[
                "file_upload"].chunks():
                output_file.write(chunk)

        return render(request, "media-example.html")
```

Note that the uploaded file and its metadata are being accessed from the `request.FILES` dictionary using the key that matches the name given to the file input (in our case, this is `file_upload`). You can save and close `views.py`.

8. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the file upload field and the **Submit** button, as can be seen here:



Figure 8.8: The file upload form

Click **Browse...** (or the equivalent on your browser) and select a file to upload. The name of the file will appear in the file input. Then, click **Submit**.

The page will reload, and the form will be empty again. This is normal – in the background, the file should have been saved.

9. Try to download the file you uploaded using `MEDIA_URL`. In this example, a file named `cover.jpg` was uploaded. It will be downloadable at `http://127.0.0.1:8000/media/cover.jpg`. Your URL will depend on the name of the file you uploaded:

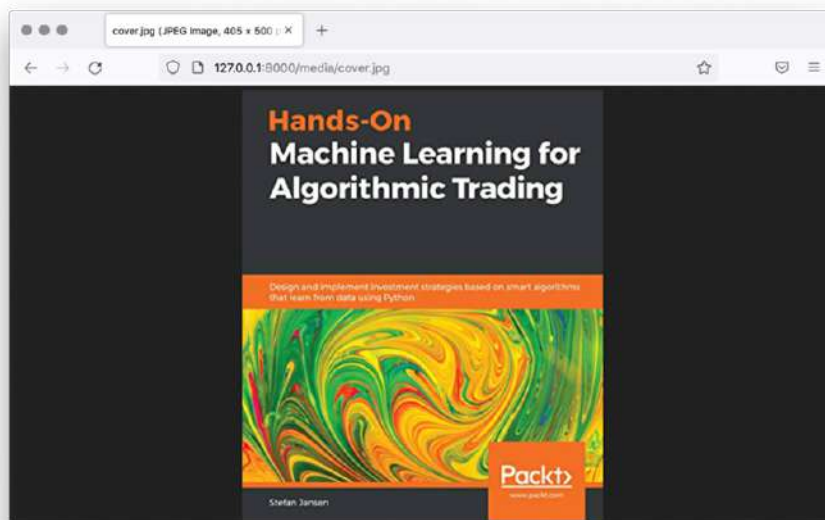


Figure 8.9: Uploaded file visible inside `MEDIA_URL`

If you upload an image file, HTML file, or another type of file that your browser can display, you will be able to view it inside the browser. Otherwise, your browser will just download it to disk again. In both cases, it means the upload was successful.

You can also confirm that the upload was successful by looking inside the media directory in the `media_project` project directory:



Figure 8.10: `cover.jpg` inside the media directory

Figure 8.10 shows the `cover.jpg` file inside the `media` directory in PyCharm.

In this exercise, you added an HTML form with `enctype` set to `multipart/form-data` so that it allowed file uploads. It contained a file input to select a file to upload. You then added a saving functionality to the `media_example` view to save the uploaded file to disk.

In the next section, we will look at how to simplify form generation and add validation using Django forms for file upload handling.

File uploads with Django forms

In *Chapter 6, Forms*, we saw how Django makes it easy to define forms and automatically render them to HTML. In the previous example, we defined our form manually and wrote the HTML. We can replace this with a Django form and implement the file input with a `FileField` constructor.

Here's how `FileField` is defined on a form:

```
from django import forms

class ExampleForm(forms.Form):
    file_upload = forms.FileField()
```

The `FileField` constructor can take the following keyword arguments:

- **Required:** This should be `True` for required fields and `False` if the field is optional

- `max_length`: This refers to the maximum length of the filename of the file being uploaded
- `allow_empty_file`: A field with this argument is considered to be valid even if the uploaded file is empty (has a size of 0)

Apart from these three keyword arguments, the constructor can also accept the standard `Field` arguments, such as `widget`. The default widget class for `FileField` is `ClearableFileInput`; this is a file input that can display a checkbox that can be checked to send a null value and clear the saved file on a `Model` field.

Using a form with a `FileField` constructor in a view is similar to other forms, but when the form has been submitted (i.e., `request.METHOD` is `POST`), then `request.FILES` should be passed into the form constructor as well. This is because Django needs to access `request.FILES` to find information about uploaded files when validating the form.

The basic flow in a view function is, therefore, similar to this:

```
def view(request):
    if request.method == "POST":
        # instantiate the form with POST data and files
        form = ExampleForm(request.POST, request.FILES)
        if form.is_valid():
            # process the form and save files
            form.save()
            return redirect("success-url")
    else:
        # instantiate an empty form as we've seen before
        form = ExampleForm()

    # render a template, the same as for other forms
    return render(request, "template.html", {"form": form})
```

When working with uploaded files and forms, you can interact with the uploaded files by accessing them through `request.FILES`, or through `form.cleaned_data`; the values will return to the same object. In the preceding example, we could process the uploaded file like this:

```
if form.is_valid():
    save_file_upload("/path/to/save.jpg", request.FILES["file_upload"])
    return redirect("/success-url/")
```

Or, since they contain the same object, you can use `form.cleaned_data`:

```
if form.is_valid():
    save_file_upload("/path/to/save.jpg", form.cleaned_data["file_upload"])
    return redirect("/success-url/")
```

The data that is saved will be the same.

Note

In *Chapter 6, Forms*, you experimented with forms and submitted them with invalid values. When the page refreshed to show the form errors, the data that you had previously entered was populated when the page reloaded. This does not occur with file fields; instead, the user will have to navigate and select the file again if the form is invalid.

In the next exercise, we will put what we have seen with `FileField`s into practice by building an example form and then modifying our view to save the file only if the form is valid.

Exercise 8.04 – file uploads with a Django form

In the previous exercise, you created a form in HTML and used it to upload a file to a Django view. If you tried submitting the form without selecting a file, you would get a Django exception screen. You did not do any validation on the form, so this method is quite fragile.

In this exercise, you will create a Django form with `FileField`, which will allow you to use form validation functions to make the view more robust, as well as reducing the amount of code:

1. In PyCharm, inside the `media_example` app, create a new file named `forms.py`. It will open automatically. At the start of the file, import the Django forms library:

```
from django import forms
```

Then, create a `forms.Form` subclass, and name it `UploadForm`. Add to it a `FileField` field named `file_upload`. Your class should have this code:

```
class UploadForm(forms.Form):
    file_upload = forms.FileField()
```

You can save and close this file.

2. Open the `form_example` app's `views.py` file. At the start of the file, right below the existing import statements, you will need to import your new class like this:

```
from .forms import UploadForm
```

3. If you are in the POST branch of the view, `UploadForm` needs to be instantiated with both `request.POST` and `request.FILES`. If you don't pass in `request.FILES`, then the form instance will not be able to access the uploaded files. Under the `if request.method == "POST"` check, instantiate `UploadForm` with these two arguments:

```
form = UploadForm(request.POST, request.FILES)
```

4. The existing lines that define `save_path` and store the file contents can be retained, but they should be indented by one block and put inside a form validity check, so they are only executed if the form is valid. Add the `if form.is_valid():` line, and then indent the other lines, so the code looks like this:

```
if form.is_valid():
    save_path = os.path.join(settings.MEDIA_ROOT,
                             request.FILES["file_upload"].name)

    with open(save_path, "wb") as output_file:
        for chunk in request.FILES["file_upload"].chunks():
            output_file.write(chunk)
```

5. Since you are using a form now, you can access the file upload through the form. Replace usages of `request.FILES["file_upload"]` with `form.cleaned_data["file_upload"]`:

```
if form.is_valid():
    save_path = settings.MEDIA_ROOT /
    form.cleaned_data["file_upload"].name

    with open(save_path, "wb") as output_file:
        for chunk in form.cleaned_data["file_upload"].chunks():
            output_file.write(chunk)
```

6. Finally, add an else branch to handle non-POST requests, which simply instantiates a form without any arguments:

```
if request.method == "POST":
    ...
else:
    form = UploadForm()
```

7. Add a context dictionary argument to the render call and set the form variable in the form key:

```
return render(request, "media-example.html", {"form": form})
```

You can now save and close this file.

8. Finally, open the `media-example.html` template and remove your manually defined `<input>` file. Replace it with `form`, rendered using the `as_p` method (highlighted):

```
<body>
  <form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
  </form>
</body>
```

You should not change any other parts of the file. You can save and close this file.

9. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the file upload field and the **Submit** button as follows:



Figure 8.11: The file upload Django form rendered in the browser

10. Since we're using a Django form, we get its built-in validation automatically. If you try to submit the form without selecting a file, your browser should prevent it and show an error, as can be seen here:



Figure 8.12: Form submission prevented by the browser

11. Finally, repeat the upload test that you performed in *Exercise 8.03 – file upload and download*, by selecting a file and submitting the form. You should then be able to retrieve the file using `MEDIA_URL`. In this case, a file named `cover.jpg` is being uploaded again (see the following screenshot):



Figure 8.13: Uploading a file named `cover.jpg`

You can then retrieve the file at `http://127.0.0.1:8000/media/cover.jpg`, and you can see it in the browser as follows:

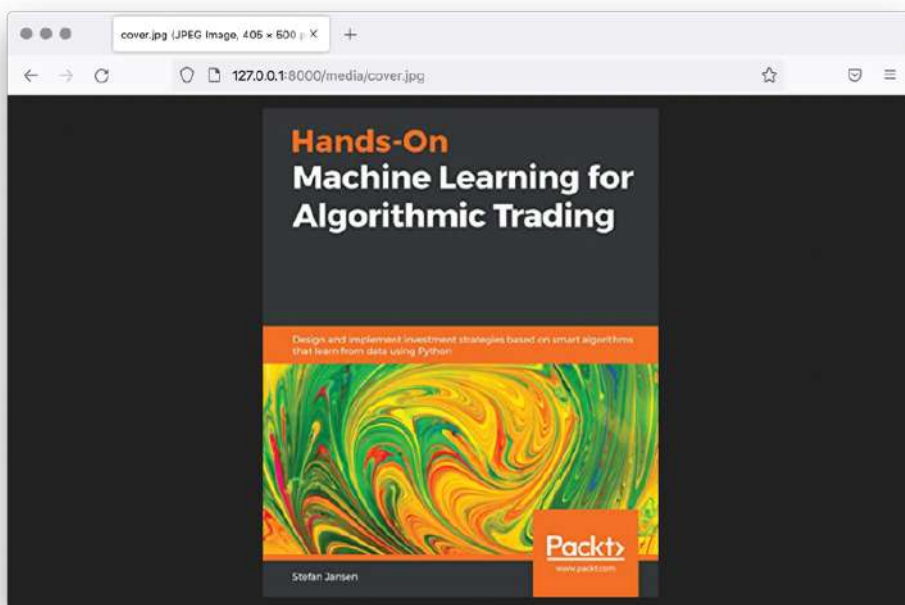


Figure 8.14: The file uploaded using the Django form is also visible in the browser

In this exercise, we replaced a manually built form with a Django form containing `FileField`. We instantiated the form in the view by passing in both `request.POST` and `request.FILES`. We then used the standard `is_valid` method to check the validity of the form and only saved the file upload if the form was valid. We tested the file uploading and saw that we were able to retrieve uploaded files using `MEDIA_URL`.

In the next section, we will look at `ImageField`, which is like `FileField` but specifically for images.

Image uploads with Django forms

If you want to work with images in Python, the most common library that you'll use is called Pillow, and this is the library Django uses to validate images. The veteran Python developer Fredrik Lundh first released the **Python Imaging Library (PIL)** in 1995. It was not kept up to date, and eventually, a fork of the library was created and is still maintained, called Pillow. The original project has been discontinued, but development on a fork of the project called Pillow has continued. Pillow strives to maintain a high degree of compatibility with the original PIL package. For this reason, Pillow contains a very similar namespace to PIL, and the package is still called PIL when installed. For example, the Image object is imported from PIL as follows:

```
from PIL import Image
```

The terms *Python Imaging Library*, *PIL*, and *Pillow* are often used interchangeably. You can assume that if someone refers to PIL, they mean the latest Pillow library.

Pillow provides various methods of retrieving data about images or manipulating them. You can find out the width and height of images and scale, crop, and apply transformations to them. There are too many operations available to cover in this chapter, so we will just introduce a simple example (scaling an image), which you will use in the next exercise.

Since images are one of the most common types of files that a user may want to upload, Django also includes `ImageField`. This behaves similarly to `FileField` but also automatically validates that the data is an image file. This helps mitigate against security issues where we expect an image but the user uploads a malicious file.

An `UploadedFile` from an `ImageField` has all the same attributes and methods as `FileField` (size, `content_type`, `name`, `chunks()`, etc.) but adds an extra attribute: `image`. This is an instance of the PIL Image object that is used to verify that the file being uploaded is a valid image.

After checking that the form is valid, the underlying PIL Image object is closed. This is done to free up memory and prevent the Python process from holding too many files open, which could cause performance issues. What this means for the developer is that you can access some of the metadata about the image (such as width, height, and `format`), but you can't access the actual image data without reopening the image.

To illustrate, we'll have a form with `ImageField`, named `picture`:

```
class ExampleForm(forms.Form):  
    picture = forms.ImageField()
```

Inside the view function, the picture field can be accessed in the form's `cleaned_data`:

```
if form.is_valid():  
    picture_field = form.cleaned_data["picture"]
```

Then, the picture field's Image object can be retrieved:

```
image = picture_field.image
```

Now that we have a reference to the image in the view, we can get some metadata:

```
w = image.width # an integer, e.g. 600  
h = image.height # also an integer, e.g. 420  
f = image.format # the format of the image as a string,  
# e.g. "PNG"
```

Django will also automatically update the `content_type` attribute of `UploadedFile` to the correct type for the picture field. This overwrites the value that the browser has sent when uploading the file.

Attempting to use a method that accesses the actual image data (rather than just the metadata) will cause an exception to be raised. This is because Django has already closed the underlying image file.

For example, the following code snippet will raise `AttributeError`:

```
image.getdata()
```

Instead, we need to reopen the image. The image data can be opened with the `ImageField` reference after importing the `Image` class:

```
from PIL import Image  
  
image = Image.open(picture_field)
```

Now that the image has been opened, you can perform operations on it. In the next section, we will look at a simple example – resizing the uploaded image.

Resizing an image with Pillow

Pillow supports many operations that you might want to perform on an image before saving it. We can't explain them all in this book, so we will just use a common operation: resizing an image to a specific size before saving it. This will help us save storage space and improve download speed. For example, a user may upload large cover images in Bookr that are bigger than are needed for our purposes. When saving the file (writing it back to disk), we must specify the format to use. We could determine the type of image that was uploaded with a number of methods (such as checking `content_type` of the uploaded file or `format` from the `Image` object), but in our example, we'll always just save it as a JPEG file.

The `PIL Image` class has a `thumbnail` method that will resize an image to a maximum size while retaining the aspect ratio. For example, we could set a maximum size of 50 **pixels (px)** by 50 px. A 200 px by 100 px image would be resized to 50 px by 25 px; the aspect ratio is retained by setting the maximum dimension to 50 px. Each dimension is scaled by a factor of 0.25:

```
from PIL import Image

size = 50, 50 # a tuple of width, height to resize to
image = Image.open(image_field) # open the image as before
image.thumbnail(size) # perform the resize
```

At this point, the resize has been done in memory only. The change is not saved to disk until the `save` method is called, like so:

```
image.save("path/to/file.jpg")
```

The output format is automatically determined from the file extension used, in this case, JPEG. The `save` method can also take a `format` argument to override it, as in this example:

```
image.save("path/to/file.png", "JPEG")
```

Despite having the `.png` extension, the format is specified as JPEG, and so the output will be in JPEG format. As you might imagine, this can be very confusing, so you might decide to stick with specifying the extension only.

In the next exercise, we will change the `UploadForm` we have been working with to use `ImageField` instead of `FileField`, then implement resizing of an uploaded image before saving it to the media directory.

Exercise 8.05 – image uploads using Django forms

In this exercise, you will update the `UploadForm` class you created in *Exercise 8.04 – file uploads with a Django form*, to use `ImageField` instead of `FileField` (this will involve simply changing the field's class). You will then see that the form renders the same in the browser. Next, you will try uploading some non-image files and see how Django validates the form to disallow them. Finally, you will update your view to use PIL to resize the image before saving it, and then test it in action:

1. Open the `media_example` app's `forms.py` file. In the `UploadForm` class, change `file_upload` so that it's an instance of `ImageField` instead of `FileField`. After the update, `UploadForm` should look like this:

```
class UploadForm(forms.Form):  
    file_upload = forms.ImageField()
```

Save and close the file.

2. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the form rendered, and it will look identical to when we used `FileField` (see the following screenshot):



Figure 8.15: `ImageField` looks the same as `FileField`

3. You will notice the difference when you try to upload a non-image file. Click the **Browse...** button and try to select a non-image file. Depending on your browser or operating system, you might not be able to select anything other than an image file, as in *Figure 8.16*:

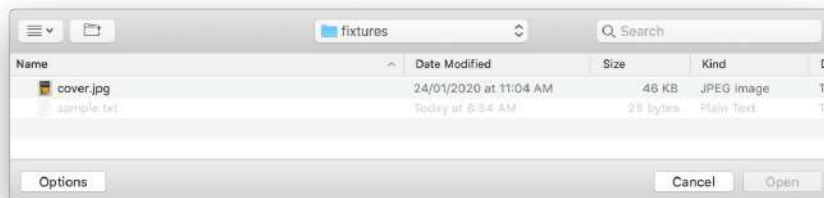


Figure 8.16: Only image files are selectable

Your browser may allow you to select an image but show an error in the form after selection. Or your browser may allow you to select a file and submit the form, and Django will raise `ValidationError`. Regardless, you can be sure that in your view, the form's `is_valid` view will only return `True` if an image has been uploaded.

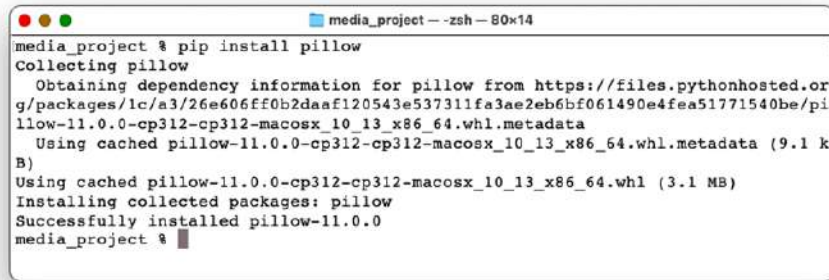
Note

You don't need to test uploading a file at this point, as the result would be the same as in *Exercise 8.04 – file uploads with a Django form*.

4. The first thing you will need to do is make sure the Pillow library is installed. In a terminal (making sure your virtual environment has been activated), run the following command:

```
pip3 install pillow
```

In Windows, this is `pip install pillow`. You'll get output like *Figure 8.17*:



```
media_project -- zsh -- 80x14
media_project % pip install pillow
Collecting pillow
  Obtaining dependency information for pillow from https://files.pythonhosted.org/packages/1c/a3/26e606ff0b2daaf120543e537311fa3ae2eb6bf061490e4fea51771540be/pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl.metadata
    Using cached pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl.metadata (9.1 kB)
Using cached pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl (3.1 MB)
Installing collected packages: pillow
Successfully installed pillow-11.0.0
media_project %
```

Figure 8.17: pip3 installing Pillow

If Pillow is already installed, you'll see the Requirement already satisfied output message.

- Now, we can update the `media_example` view to resize the image before saving it. Switch back to PyCharm and open the `media_example` app's `views.py` file, then import PIL's `Image` class by adding this import line near the top of the file:

```
from PIL import Image
```

- Go to the `media_example` view. Under the line that generates `save_path`, take out the three lines that open the output file, iterate over the uploaded file, and write out its chunks. Replace this with the code that opens the uploaded file with PIL, resizes it, and then saves it:

```
image = Image.open(form.cleaned_data["file_upload"])
image.thumbnail((50, 50))
image.save(save_path)
```

The first line creates an `Image` instance by opening the uploaded file, the next performs the thumbnail conversion (to a maximum size of 50 px by 50 px), and the third line saves the file to the same save path that we have been generating in previous exercises. You can save the file.

- The Django dev server should still be running from step 2, but you should start it if it is not. Then, navigate to `http://127.0.0.1:8000/media-example/`. You'll see the

familiar `UploadForm`. Select an image and submit the form. If the upload and resize were successful, the form will refresh and be empty again.

8. View the uploaded image using `MEDIA_URL`. For example, a file named `cover.jpg` will be downloadable from `http://127.0.0.1:8000/media/cover.jpg`. You should see that the image has been resized to have a maximum dimension of just 50 px:



Figure 8.18: The resized cover

While a thumbnail this size might not be that useful, it at least lets us be sure that the image resize has worked correctly.

In this exercise, we changed `FileField` on `UploadForm` to `ImageField`. We saw that the browser wouldn't let us upload anything other than images. We then added code to the `media_example` view to resize the uploaded image using PIL.

We have encouraged the use of a separate web server to serve static and media files for performance reasons. However, in some cases, you might want to use Django to serve files, for example, to provide authentication before allowing access. In the next section, we will discuss how to use Django to serve media files.

Serving uploaded (and other) files using Django

Throughout this chapter and *Chapter 5, Serving Static Files*, we have discouraged serving files using Django. This is because it would needlessly tie up a Python process just to serve a file – something that the web server is capable of handling. Unfortunately, web servers do not usually provide dynamic access control, that is, allowing only authenticated users to download a file. Depending on the web server used in production, you might be able to have it authenticate against Django and then serve the file itself; however, the specific configuration of specific web servers is outside the scope of this book.

One approach you can take is to specify a subdirectory of your `MEDIA_ROOT` directory and have your web server prevent access to just this specific folder. Any protected media should be stored inside it. If you do this, only Django will be able to read the files inside. For example,

your web server could serve everything in the MEDIA_ROOT directory except for a MEDIA_ROOT/protected directory.

Another approach would be to configure a Django view to serve a specific file from disk. The view will determine the path of the file on disk to send, then send it using the `FileResponse` class. The `FileResponse` class takes an open file handle as an argument and tries to determine the correct content type from the file's content. Django will close the file handle after the request completes.

The view function will accept the request and a relative path to the file to be downloaded as parameters. This relative path is the path inside the MEDIA_ROOT/protected folder.

In our case, we will just check whether the user is anonymous (not logged in). We'll do this by checking the `request.user.is_anonymous` property. If they aren't logged in, then we will raise a `django.core.exceptions.PermissionDenied` exception, which returns an **HTTP 403 Forbidden** response to the browser. This will stop the execution of the view and not return any file:

```
import os.path
from django.conf import settings
from django.http import FileResponse
from django.core.exceptions import PermissionDenied

def download_view(request, relative_path):
    if request.user.is_anonymous:
        raise PermissionDenied
    full_path = os.path.join(settings.MEDIA_ROOT,
                             "protected", relative_path)
    file_handle = open(full_path, "rb")
    # Django sends the file then closes the handle
    return FileResponse(file_handle)
```

The URL mapping to this view could be like this, using the `path` path converter inside your `urls.py` file:

```
urlpatterns = [
    ...
    path("downloads/<path:relative_path>", views.download_view)
]
```


There are many ways that you could choose to implement a view that sends files. The important thing is that you use the `FileResponse` class, which is designed to stream the file to the client in chunks instead of loading it all into memory. This will reduce the load on the server and lessen the impact on resource usage if you must resort to sending files with Django.

Now that we know how to upload and process images, let's look at how to store them on models.

Storing files on model instances

So far, we have manually managed the uploading and saving of files. You can also associate a file with a model instance by assigning the path to which it was saved to `CharField`. However, as with much of Django, this capability (and more) is already provided with the `models.FileField` class. A `FileField` instance does not actually store the file data; instead, it stores the path where the file is stored (like `CharField` would), but it also provides helper methods. These methods assist with loading files (so you don't have to open them manually) and generating disk paths for you based on the ID of the instance (or other attributes).

`FileField` can accept two specific optional arguments in its constructor (as well as the base `Field` arguments, such as `required`, `unique`, `help_text`, etc.):

- `max_length`: Like `max_length` in the form's `ImageField`, this is the maximum length of the filename that is allowed.
- `upload_to`: The `upload_to` argument has three different behaviors depending on what type of variable is passed to it. Its simplest use is with a string or a `pathlib.Path` object. The path is simply appended to `MEDIA_ROOT`.

In this example, `upload_to` is just defined as a string:

```
class ExampleModel(models.Model):  
    file_field = models.FileField(upload_to="files/")
```

Files saved to `FileField` will be stored in the `MEDIA_ROOT/files` directory.

You can achieve the same result using a `pathlib.Path` instance, too:

```
import pathlib  
  
class ExampleModel(models.Model):  
    file_field = models.FileField(upload_to=pathlib.Path("files/"))
```

The next way of using `upload_to` is with a string that contains the `strftime` formatting directives (for example, `%Y` to substitute the current year, `%m` for the current month, and `%d` for the current day of the month). The full list of these directives is extensive and can be found at <https://docs.python.org/3/library/time.html#time.strftime>. Django will automatically interpolate these values when saving the file.

For example, say you defined the model and `FileField` like this:

```
class ExampleModel(models.Model):  
    file_field = models.FileField(upload_to="files/%Y/%m/%d/")
```

For the first file uploaded on a specific day, Django would create the directory structure for that day. For example, for the first file uploaded on January 1st, 2025, Django would create the `MEDIA_ROOT/2025/01/01` directory and then store the uploaded file there. The next file (and all subsequent ones) uploaded on the same day would also be stored in that directory. Similarly, on January 2nd, 2025, Django will create the `MEDIA_ROOT/2025/01/02` directory, and files will be stored there.

If you have many thousands of files being uploaded every day, you could even have the files split up further by including the hour and minute in the `upload_to` argument (`upload_to="files/%Y/%m/%d/%H/%M/"`). This may not be necessary if you only have a small volume of uploads, though.

By utilizing this method of the `upload_to` argument, you can have Django automatically segregate uploads and prevent too many files from being stored within a single directory (which can be hard to manage).

The final method of using `upload_to` is by passing a function that will be called to generate the storage path. Note that this is different from the other uses of `upload_to`, as it should generate the full path, including the filename, rather than just the directory. The function takes two arguments: `instance` and `filename`. The `instance` argument is the model instance that `FileField` is attached to, and `filename` is the name of the uploaded file.

Here is an example function that takes the first two characters of a filename to generate the save directory. This means that each uploaded file will be grouped into parent directories, which can help organize files and prevent there from being too many in one directory:

```
def user_grouped_file_path(instance, filename):  
    return "/".join([instance.username,  
        filename[0].lower(), filename[1].lower(), filename])
```

If this function is called with the `Test.jpg` filename, it will return `<username>/t/e/test.jpg`. If called with `example.txt`, it will return `<username>/e/x/example.txt`, and so on. `username` is retrieved from the instance that is being saved. To illustrate, here is a model with `FileField` that uses this function. It also has a `username`, which is `CharField`:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to=user_grouped_file_path)
    username = models.CharField(unique=True)
```

You can use any attribute of the instance in the `upload_to` function, but be aware that if this instance is in the process of being created, then the file save function will be called before it is saved to the database. Therefore, some of the automatically generated attributes on the instance (such as `id/pk`) will not yet be populated and shouldn't be used to generate a path.

Whatever path is returned from the `upload_to` function is appended to `MEDIA_ROOT`, so the uploaded files would be saved at `MEDIA_ROOT/<username>/t/e/test.jpg` and `MEDIA_ROOT/<username>/e/x/example.txt`, respectively.

Note that `user_grouped_file_path` is just an illustrative function that has intentionally been kept short, so it will not work correctly with single-character filenames or if the username has invalid characters. For example, if the username has a `/` character in it, then this would act as a directory separator in the generated path.

Now, we've done a deep dive into setting up `FileField` on a model, but how do we actually save an uploaded file to it? It's as easy as assigning the uploaded file to the attribute of the model, as you would with any type of value. Here's a quick example with a view and the simple `ExampleModel` we were using as an example earlier in this section:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to="files/")

def view(request):
    if request.method == "POST":
        # Create a new ExampleModel Instance
        m = ExampleModel()
        m.file_field = request.FILES["uploaded_file"]
        m.save()
    return render(request, "template.html")
```

In this example, we create a new `ExampleModel` and assign the uploaded file (called `uploaded_file` on the form) to its `file_field` attribute. When we saved the model instance, Django automatically wrote the file with its name to the `upload_to` directory path. If the uploaded file had the name `image.jpg`, the save path would be `MEDIA_ROOT/upload_to/image.jpg`.

We could just as easily have updated the file field on an existing model or used a form (validating it before saving). Here is another simple example demonstrating this:

```
class ExampleForm(forms.Form):
    uploaded_file = forms.FileField()

def view(request, model_pk):
    form = ExampleForm(request.POST, request.FILES)
    if form.is_valid():
        # Get an existing model instance
        m = ExampleModel.objects.get(pk=model_pk)

        # store the uploaded file on the instance
        m.file_field = form.cleaned_data["uploaded_file"]
        m.save()
    return render(request, "template.html")
```

You can see that updating `FileField` on an existing model instance is the same process as setting it on a new instance, and if you choose to use a Django form or just access `request.FILES` directly, the process is just as simple.

Now we'll look at how to store images on models using `ImageField`.

Storing images on model instances

While `FileField` can store any type of file, including images, there also exists `ImageField`. As you would expect, this is only for storing images. The relationship between the models' `forms.FileField` and `forms.ImageField` is similar to that between `models.FileField` and `models.ImageField`, meaning `ImageField` extends `FileField` and adds extra methods for working with images.

The `ImageField` constructor takes the same arguments as `FileField` and adds two extra optional arguments:

- `height_field`: This is the name of the field on the model that will be updated with the height of the image every time the model instance is saved
- `width_field`: This is the width counterpart to `height_field`: the field that stores the width of the image that is updated every time the model instance is saved

Both of these arguments are optional, but the fields they name must exist if used. That is, it is valid to have `height_field` or `width_field` unset, but if they are set to the name of a field that does not exist, then an error will occur. The purpose of this is to assist with searching the database for files of a particular dimension.

Here is an example model using `ImageField`, which updates the image dimension fields:

```
class ExampleModel(models.Model):
    image = models.ImageField(
        upload_to="images/%Y/%m/%d/",
        height_field="image_height",
        width_field="image_width")
    image_height = models.IntegerField()
    image_width = models.IntegerField()
```

Notice that `ImageField` is using the `upload_to` parameter with date formatting directives that are updated on save. The behavior of `upload_to` is identical to that of `FileField`.

Upon saving an `ExampleModel` instance, its `image_height` field would be updated with the height of the image and `image_width` with the width of the image.

We won't show examples for setting the `ImageField` values in a view, as the process is the same as for a plain `FileField`.

Working with FieldFile

When you access the `FileField` or `ImageField` attribute of a model instance, you will not get a native Python file object. Instead, you will be working with a `FieldFile` object. The `FieldFile` class is a wrapper around file that adds extra methods. Yes, it can be confusing to have classes called `FileField` and `FieldFile`.

The reason that Django uses `FieldFile` instead of just a file object is twofold. First, it adds extra methods to open, read, delete, and generate the URL of the file. Second, it provides an abstraction to allow alternative storage engines to be used.

Custom storage engines

We looked at custom storage engines in *Chapter 5, Serving Static Files*, regarding storing static files. We won't examine custom storage engines in detail for media files since the code outlined in *Chapter 5, Serving Static Files*, for static files also applies to media files. The important thing to note is that which storage engine you're using can be changed without updating your other code. This means that you can have your media files stored on your local drive during development and then save them to a CDN when your application is deployed to production.

The default `storage_engine` class can be set with `DEFAULT_FILE_STORAGE` in `settings.py`. The storage engine can also be specified on a per-field basis (for `FileField` or `ImageField`) with the `storage` argument, as in this example:

```
storage_engine = CustomStorageEngine()

class ExampleModel(models.Model):
    image_field = ImageField(storage=storage_engine)
```

This demonstrates what actually happens when you upload or retrieve a file. Django delegates to the storage engine to write or read it, respectively. This happens even while saving to disk; however, this is automated and is invisible to the user.

Reading a stored FieldFile

Now that we've learned about custom storage engines, let's look at reading from `FieldFile`. In the previous sections, we saw how to set the file on the model instance. Reading the data back again is just as easy – we have a couple of different methods that can help us depending on our use case.

In the following few code snippets, assume we are inside a view and have retrieved our model instance in some manner, and it's stored in an `m` variable:

```
m = ExampleModel.objects.get(pk=model_pk)
```

We can read all the data from the file with the `read` method:

```
data = m.file_field.read()
```

Or, we can manually open the file with the `open` method. This might be useful if we want to write our own generated data to the file:

```
with m.file_field.open("wb") as f:
    chunk = f.write(b"test") # write bytes to the file
```

If we want to read the file in chunks, we can use the `chunks` method. This works the same as reading chunks from the uploaded file, as we saw earlier:

```
for chunk in m.file_field.chunks():
    # assume this method is defined somewhere
    write_chunk(open_file, chunk)
```

We can also manually open the file ourselves by using its `path` attribute:

```
open(m.file_field.path)
```

If we want to stream `FileField` for download, the best way is by using the `FileResponse` class, as we saw earlier. Combine this with the `open` method on `FileField`. Note that if we're just trying to serve a media file, we should only implement a view to do this if we're trying to restrict access to the file. Otherwise, we should just serve the file using `MEDIA_URL` and allow the web server to handle the request. Here's how we'd write `download_view` to use `FileField` instead of the manually specified path:

```
def download_view(request, model_pk):
    if request.user.is_anonymous:
        raise PermissionDenied
    m = ExampleModel.objects.get(pk=model_pk)
    # Django sends the file then closes the handle
    return FileResponse(m.file_field.open())
```

Django opens the correct path and closes it after the response. Django will also attempt to determine the correct mime type for the file. We assume that `FileField` has its `upload_to` attribute set to a protected directory that the web server is preventing direct access to.

Storing existing files or content on `FileField`

We've seen how to store an uploaded file on an image field; simply assign it to the field like so:

```
m.file_field = request.FILES["file_upload"]
```

But how can we set the field value to that of an existing file that we might already have on disk? You might think you can use a standard Python file object, but this won't work:

```
# Don't do this
m.file_field = open("/path/to/file.txt", "rb")
```

You might also try setting the file using some content:

```
# Don't do this
m.file_field = "new file content"
```

This won't work either.

You instead need to use the `FileField` save method, which accepts an instance of a Django `File` object or a `ContentFile` object (these classes' full paths are `django.core.files.File` and `django.core.files.base.ContentFile`, respectively). We'll briefly discuss the save method and its arguments and then return to these classes.

The `FileField` save method takes three arguments:

- **name:** This is the name of the file you are saving and is the name the file will have when saved to the storage engine (in our case, to disk, inside `MEDIA_ROOT`).
- **content:** This is an instance of `File` or `ContentFile` that we just saw; again, we will discuss these soon.
- **save:** This argument is optional and defaults to `True`. This indicates whether or not to save the model instance to the database after saving the file. If set to `False` (i.e., the model is not saved), then the file will still be written to the storage engine (to disk), but the association is not stored on the model. The previous file path (or no file if one was not set) will still be stored in the database until the model instance's save method is called manually. You should only set this argument to `False` if you intend to make other changes to the model instance and then save it manually.

Back to `File` and `ContentFile`: the one to use depends on what you want to store in `FileField`.

`File` is used as a wrapper around a Python file object, and you should use it if you have an existing file or file-like object that you want to save. File-like objects include `io.BytesIO` or

`io.StringIO` instances. To instantiate a `File` instance, just pass the native file object to the constructor, as in this example:

```
f = open("/path/to/file.txt", "rb")
file_wrapper = File(f)
f.close()
```

Use `ContentFile` when you already have some data loaded, either a `str` object or a `bytes` object. Pass the data to the `ContentFile` constructor:

```
string_content = ContentFile("A string value")
bytes_content = ContentField(b"A bytes value")
```

Now that you have either a `File` instance or a `ContentFile` instance, saving the data to `FileField` is easy, using the `save` method:

```
m = ExampleModel.objects.first()
with open("/path/to/file.txt") as f:
    file_wrapper = File(f)
    m.file_field.save("file.txt", f)
```

Since we did not pass a value for `save` to the `save` method, it will default to `True`, so the model instance is automatically persisted to the database.

Next, we will look at how to store an image that has been manipulated with a PIL in an image field.

Writing PIL images to ImageField

In *Exercise 8.05 – image uploads using Django forms*, you used PIL to resize an image and save it to disk. When working with a model, you might want to perform a similar operation but have Django handle the file storage using `ImageField` so that you don't have to do it manually. As in the exercise, you could save the image to disk and then use the `File` class to wrap the stored path – something like this:

```
image = Image.open(request.FILES["image_field"])
image.thumbnail((150, 150))
# save thumbnail to temp location
image.save("/tmp/thumbnail.jpg")

with open("/tmp/thumbnail.jpg", "rb") as f:
```

```
image_wrapper = File(f)
m.image_field.save("thumbnail.jpg", image_wrapper, save=True)
# clean up temp file
os.unlink("/tmp/thumbnail.jpg")
```

In this example, we're storing the PIL store in a temporary location with the `Image.save()` method, and then reopening the file.

This method works but is not ideal, as it involves writing the file to disk and then reading it out again, which can sometimes be slow. Instead, we can perform this whole process in memory.

Note

`io.BytesIO` and `io.StringIO` are useful objects. They behave like files but exist in memory only. `BytesIO` is used for storing raw bytes, and `StringIO` accepts Python 3's native Unicode strings. You can use `read`, `write`, and `seek`, just like a normal file. Unlike a normal file, though, they don't get written to disk and, instead, will disappear when your program terminates, or they go out of scope and are garbage collected. They are very useful if a function wants to write to something like a file, but you want to access the data immediately.

First, we will save the image data to an `io.BytesIO` object. Then, we'll wrap the `BytesIO` object in a `django.core.files.images.ImageField` instance (a subclass of `File` that is specifically for images and provides the width and height attributes). Once we have this `ImageFile` instance, we can use it in the `save` method of `ImageField`.

Note

`ImageFile` is a file or file-like wrapper just like `File`. It provides two extra attributes: `width` and `height`. `ImageFile` does not generate any errors if you use it to wrap a non-image. For example, you could use `open()` to open a text file and pass the file handle to the `ImageFile` constructor without issue. You can check whether the image file you passed in was valid by trying to access the `width` or `height` attribute; if these are `None`, then PIL was unable to decode the image data. You can check for the validity of these values yourself and throw an exception if they are `None`.

Let's have a look at this in practice in a view:

```
from io import BytesIO
from PIL import Image
from django.core.files.images import ImageFile

def index(request, pk):
    # trim out logic for checking if method is POST

    # get a model instance, or create a new one
    m = ExampleModel.objects.get(pk=pk)

    # store the uploaded image in a variable for shorter
    # code
    uploaded_image = request.FILES["image_field"]

    # Load a PIL image instance from the uploaded file
    image = Image.open(uploaded)

    # perform the image resize
    image.thumbnail((150, 150))

    # Create a BytesIO file-like object to store
    image_data = BytesIO()

    # Write the Image data back out to the BytesIO object
    # Retain the existing format from the uploaded image
    image.save(fp=image_data, uploaded_image.format)

    # Wrap the BytesIO containing the image data
    image_file = ImageFile(image_data)

    # Save the wrapped image file data with the original
    # name
    m.image_field.save(uploaded_image.name, image_file)
    # this also saves the model instance
    return redirect("/success-url/")
```

You can see that this is a little bit more code, but it saves on writing the data to disk. You can choose to use either method (or another one that you come up with) depending on your needs.

Referring to media in templates

Once we have uploaded a file, we want to be able to refer to it in a template. For an uploaded image, such as a book cover, we will want to display the image on the page. We saw in *Exercise 8.02 – template settings and using MEDIA_URL in templates*, how to build a URL using MEDIA_URL in a template. When working with FileField or ImageField on a model instance, it is not necessary to do this, as Django provides this functionality for you.

The url attribute of FileField will automatically generate the full URL to the media file based on the MEDIA_URL in your settings.

Note

Note that references we make to FileField in this section also apply to ImageField, as it is a subclass of FileField.

This can be used anywhere that you have access to the instance and field, such as in a view or a template. The following example is in a view:

```
instance = ExampleModel.objects.first()
url = instance.file_field.url # Get the URL
```

The following example is in a template (assuming instance has been passed to the template context):

```

```

In the next exercise, we will create a new model with FileField and ImageField and then show how Django can automatically save these. We'll also demonstrate how to retrieve the URL for an uploaded file.

Exercise 8.06 – FileField and ImageField on models

In this exercise, we will create a model with FileField and ImageField. After doing this, we will have to generate a migration and apply it. We'll then change the UploadForm we've been using so it has both FileField and ImageField. The media_example view will be updated to store the uploaded files on the model instance. Finally, we'll add into the example template to show the previously uploaded image:

1. In PyCharm, open the media_example app's models.py file. Create a new model called ExampleModel with two fields: an ImageField named image_field, and a FileField

called `file_field`. `ImageField` should have `upload_to` set to `images/`, and `FileField` should have `upload_to` set to `files/`. The finished model should look like this:

```
class ExampleModel(models.Model):
    image_field = models.ImageField(upload_to="images/")
    file_field = models.FileField(upload_to="files/")
```

2. Open a terminal and navigate to the `media_project` project directory. Make sure your `djangoenv` virtual environment is active. Run the `makemigrations` management command to generate the migrations for this new model:

```
python manage.py makemigrations
```

Note

To learn how to create and activate a virtual environment, refer to the *Preface* section of the book.

The output should be like the following:

```
% python3 manage.py makemigrations
Migrations for 'media_example':
  media_example/migrations/0001_initial.py
  - Create model ExampleModel
```

3. Apply the migration by running the `migrate` management command:

```
python3 manage.py migrate
```

The output will be as follows:

```
% python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes,
  reviews, sessions
Running migrations:
  # output trimmed for brevity
  Applying media_example.0001_initial... OK
```

Note that all the initial Django migrations will also be applied since we did not apply those after creating the project.

4. Switch back to PyCharm and open the `media_example` app's `forms.py` file. Rename the existing `ImageField` from `file_upload` to `image_upload`. Then, add a new `FileField` named `file_upload`. After making these changes, your `UploadForm` code should look like this:

```
class UploadForm(forms.Form):
    image_upload = forms.ImageField()
    file_upload = forms.FileField()
```

You can save and close the file.

5. Open the `media_example` app's `views.py` file. First, import `ExampleModel` into the file. To do this, add this line at the top of the file after the existing import statements:

```
from .models import ExampleModel
```

Some imports will no longer be required, so you can remove these lines:

```
from PIL import Image
from django.conf import settings
```

6. In the `media_example` view, set a default for the instance you will render in case one is not created. After the function definition, define a variable called `instance` and set it to `None`:

```
def media_example(request):
    instance = None
```

7. You can completely remove the contents of the `form.is_valid()` branch as you no longer need to manually save the file. Instead, it will automatically be saved when the `ExampleModel` instance is saved. You will instantiate an `ExampleModel` instance and set the file and image fields from the uploaded form. Add this code under the `if form.is_valid():` line:

```
instance = ExampleModel()
instance.image_field = form.cleaned_data["image_upload"]
```

```
instance.file_field = form.cleaned_data["file_upload"]
instance.save()
```

8. Pass the instance through to the template in the context dictionary that is passed to render. Use the instance key:

```
return render(request, "media-example.html",
               {"form": form, "instance": instance})
```

You can now save and close this file.

9. Open the `media-example.html` template. Add an `<img>` element that displays the last uploaded image. Under the closing `</form>` tag, add an `if` template tag that checks whether instance has been provided. If so, display `<img>` with a `src` attribute of `instance.image_field.url`:

```
{% if instance %}
    
{% endif %}
```

10. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the form rendered with two fields:



Figure 8.19: UploadForm with two fields

11. Select a file for each field – for `ImageField`, you must select an image, but any type of file is allowed for `FileField`. See Figure 8.20, which shows the fields with files selected:



Figure 8.20: ImageField and FileField with files selected

Then, submit the form. If the submission is successful, the page will reload, and the last image we uploaded will be displayed (Figure 8.21):

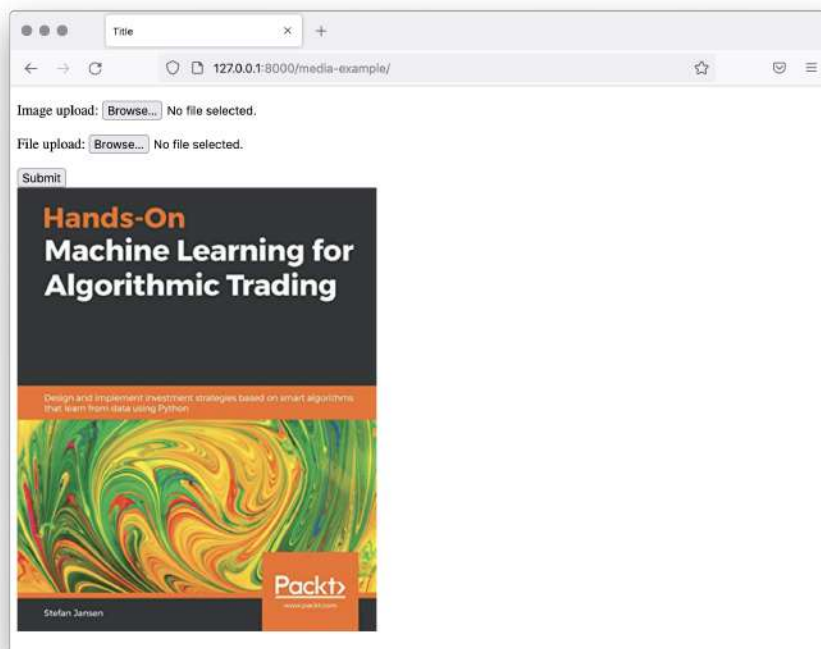


Figure 8.21: The last image that was uploaded is displayed

12. You can see how Django stores the files by looking in the MEDIA_ROOT directory. Figure 8.22 shows the directory layout in PyCharm:

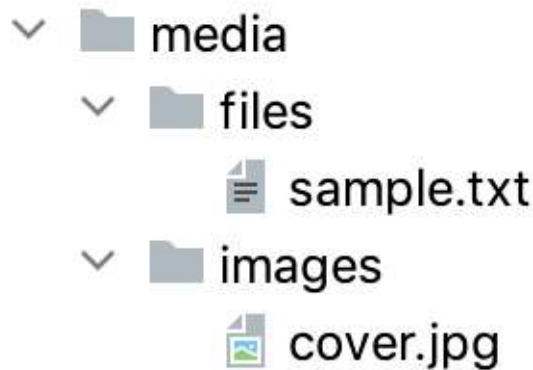


Figure 8.22: Uploaded files that Django has created

You can see that Django has created the `files` and `images` directories. These were what you set in the `upload_to` arguments on `ImageField` and `FileField` of the model.

We could also verify these uploads by attempting to download them, for example, at `http://127.0.0.1:8000/media/files/sample.txt` or `http://127.0.0.1:8000/media/images/cover.jpg`.

In this exercise, we created `ExampleModel` with `FileField` and `ImageField` and saw how to store uploaded files on it. We saw how to generate a URL to an uploaded file for use in a template. We tried uploading some files and saw that Django automatically created the `upload_to` directories (`media/files` and `media/images`) and then stored the files inside.

In the next section, we will look at how we can simplify the process even further by using `ModelForm` to generate the form and save the model without having to manually set the files in the view.

ModelForm and file uploads

We've seen how using `form.ImageField` on a form can prevent non-images from being uploaded. We've also seen how `models.ImageField` makes it easy to store an image for a model. But we need to be aware that Django does not stop us from setting a non-image file to `ImageField`. For example, consider a form that has both `FileField` and `ImageField`:

```
class ExampleForm(forms.Form):
    uploaded_file = forms.FileField()
    uploaded_image = forms.ImageField()
```

In the following view, the form would not validate if the `uploaded_image` field on the form was not an image, so some data validity is ensured for uploaded data:

```
def view(request):
    form = ExampleForm(request.POST, request.FILES)
    if form.is_valid():
        m = ExampleModel()
        m.file_field = form.cleaned_data["uploaded_file"]
        m.image_field = forms.cleaned_data["uploaded_image"]
        m.save()
    return render(request, "template.html")
```

Since we're sure that the form is valid, we know that `forms.cleaned_data["uploaded_image"]` must contain an image. Therefore, we would never assign a non-image to the model instance's `image_field`.

However, say we made a mistake in our code and wrote something such as the following:

```
m.image_field = forms.cleaned_data["uploaded_file"]
```

That is, if we accidentally reference `FileField` by mistake, Django does not validate that a (potential) non-image is being assigned to `ImageField`, and so it does not throw an exception or generate any kind of error. We can mitigate the potential for issues such as this by using `ModelForm`.

We introduced `ModelForm` in *Chapter 7, Advanced Form Validation and Model Forms*. This is a type of form whose fields are automatically defined from a model. We saw that `ModelForm` has a `save` method that automatically creates or updates the model data in the database. When used with a model that has `FileField` or `ImageField`, then the `save` method of `ModelForm` will also save uploaded files.

Here's an example of using `ModelForm` to save a new model instance in a view. Here, we are just making sure to pass `request.FILES` to the `ModelForm` constructor:

```
class ExampleModelForm(forms.ModelForm):
    class Meta:
        # The same ExampleModel class we've seen previously
        model = ExampleModel
        fields = "__all__"

    def view(request):
```

```
if request.method == "POST":
    form = ExampleModelForm(request.POST,
                             request.FILES)

    form.save()
    return redirect("/success-page")
else:
    form = ExampleModelForm()
    return (request, "template.html", {"form": form})
```

As with any `ModelForm`, the `save` method can be called with the `commit` argument set to `False`. Then, the model instance will not be saved to the database, and the `FileField` and `ImageField` files will not be saved to disk. The `save` method should be called on the model instance itself – this will commit changes to the database and save the files. In this next short example, we set a value on the model instance before saving it:

```
def view(request):
    if request.method == "POST":
        form = ExampleModelForm(request.POST,
                                 request.FILES)
        m = form.save(False)
        # Set arbitrary value on the model instance
        # before save
        m.attribute = "value"
        # save the model instance, also write the
        # files to disk
        m.save()
        return redirect("/success-page/")
    else:
        form = ExampleModelForm()
        return (request, "template.html", {"form": form})
```

Calling the `save` method on the model instance saves both the model data to the database and the uploaded files to disk. In the next exercise, we will build `ModelForm` from the `ExampleModel` we created in *Exercise 8.06 – FileField and ImageField on models*, then test uploading files with it.

Exercise 8.07 – file and image uploads using ModelForm

In this exercise, you will update `UploadForm` to be a subclass of `ModelForm` and have it built automatically from `ExampleModel`. You will then change the `media_example` view to save the instance automatically from the form, so you can see how the amount of code can be reduced:

1. In PyCharm, open the `media_example` app's `forms.py` file. You need to use `ExampleModel` in this chapter, so use `import` to import it at the top of the file after `from django import forms` statement. Insert this line:

```
from .models import ExampleModel
```

2. Change `UploadForm` to be a subclass of `forms.ModelForm`. Remove the class body and replace it with a `class Meta` definition. Its `model` setting should be `ExampleModel`. Set the `fields` attribute to `__all__`. After completing this step, `UploadForm` should look like this:

```
class UploadForm(forms.ModelForm):  
    class Meta:  
        model = ExampleModel  
        fields = "__all__"
```

Save and close the file.

3. Open the `media_example` app's `views.py` file. Since you no longer need to reference `ExampleModel` directly, you can remove its import at the top of the file. Remove the following line:

```
from .models import ExampleModel
```

4. In the `media_example` view, remove the entirety of the `form.is_valid()` branch and replace it with a single line:

```
instance = form.save()
```

The form's `save` method will handle persisting the instance to the database and saving the files. It will return an instance of `ExampleModel`, the same as the other `ModelForm` that we worked with in *Chapter 7, Advanced Form Validation and Model Forms*.

Save and close `views.py`.

5. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the form rendered with two fields, **Image field** and **File field** (Figure 8.23):



Figure 8.23: UploadForm as a ModelForm rendered in the browser

Note that the names of these fields now match those of the model rather than the form, since the form just uses the model's fields.

6. Browse and select an image and file (Figure 8.24), then submit the form:



Figure 8.24: Image and file selected

7. The page will reload, and as in *Exercise 8.06 – FileField and ImageField on models*, you will see the previously uploaded image (Figure 8.25):

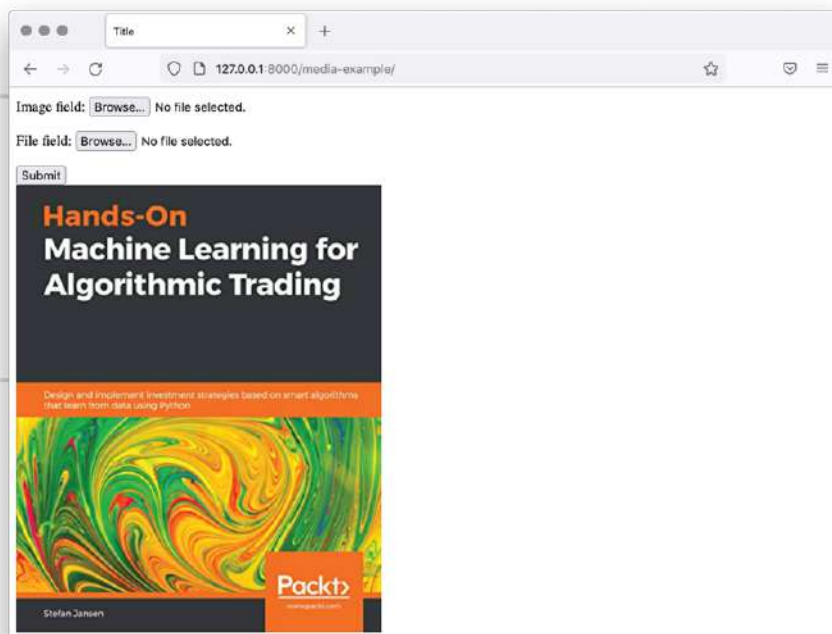


Figure 8.25: Image being displayed after upload

8. Finally, examine the contents of the `media` directory. You should see that the directory layout matches that of *Exercise 8.06 – FileField and ImageField on models*, with images inside the `images` directory and files inside the `files` directory:

```

✓ media
  ✓ files
    sample.txt
    sample_oMGMUVy.txt
  ✓ images
    cover.jpg
    cover_uf1yjUz.jpg

```

Figure 8.26: The uploaded files directory matches Exercise 6

In this exercise, we changed `UploadForm` to a `ModelForm` subclass, which allowed us to automatically generate the upload fields. We could replace the code that stored the uploaded files on the models with a call to the form's `save` method.

Before moving on to the activities for this chapter, we need to briefly discuss how to handle file saving when using an instance.

Handling file saving

As you know from working with `ModelForms` in *Chapter 7*, you can pass an `instance` argument to `ModelForm` to cause it to update an existing model instead of creating a new one when saving.

For example, do the following to update an example model in a view:

```
def update_view(request, pk):
    instance = ExampleModel.objects.get(pk=pk)
    form = UploadForm(request.POST, request.FILES,
                      instance=instance)
    if form.valid():
        form.save()
```

When accessing the `cleaned_data` attribute of the form, we will get back the data that was contained in `request.POST`. If there was no value for that key, then we'll get the value from the instance instead.

This normally doesn't cause any problems, but when working with `ImageField` or `FileField`, the type of data being returned will differ from the request or the instance.

If a file was uploaded, then the cleaned data will be `InMemoryUploadedFile` or `TemporaryUploadedFile` (depending on its size; only small files will fit in memory). If no file was uploaded, then the type will be `FieldFile` for `FileField` or `ImageFieldFile` for `ImageField`.

To explain, consider this code:

```
def update_view(request, pk):
    instance = ExampleModel.objects.get(pk=pk)
    form = UploadForm(request.POST, request.FILES,
                      instance=instance)
    if form.is_valid():
```

```
form_file = form.cleaned_data["file_field"]
form_image = form.cleaned_data["image_field"]
```

In the case where data was uploaded (that is, the user selected an image and file in the form), then `form_file` and `form_image` will be `InMemoryUploadedFile` or `TemporaryUploadedFile`. However, if the user did not upload anything for the fields (assuming the fields are not required in the form), then `form_file` will be a `FieldFile` instance, and `form_image` will be an `ImageFieldFile` instance.

To differentiate between them, there are a few different checks you could do. One of the simplest methods is to check for the existence of a `path` attribute. Since uploaded files haven't been saved to disk yet, they have no `path`:

```
if hasattr(form_file, "path"):
    print("This is a FieldFile")
else:
    print("This is an uploaded file")
```

Sometimes you will need to check what kind of file you are working with before performing some action, and perhaps skip that action if the user hasn't uploaded a new file. For example, we wouldn't want to resize an image that was already assigned to a model. We'd only do the image resize on newly uploaded images:

```
if hasattr(image_file, "path"):
    print("This image was on the instances so no "
          "need to resize")
else:
    perform_resize(image_file)
```

You will need to make a check like this in the first activity for this chapter.

We've now covered everything you need to start enhancing Bookr with file uploads. In the activity for this chapter, we'll add support for uploading a cover image and sample document (PDF, text file, and more) for a book. The book cover will be resized using PIL before it is saved.

Activity 8.01 – image and PDF upload for books

In this activity, you will start by cleaning up (deleting) the example views, templates, forms, models, and URL maps that we were using throughout the exercises in this chapter. You'll then need to generate and apply a migration to delete `ExampleModel` from the database.

You can then start adding the Bookr enhancements, first by adding `ImageField` and `FileField` to the `Book` model to store the book cover and sample. Then you will create a migration and apply it to add these fields to the database. You can then build a form that will display just these new fields. You'll add a view that uses this form to save the model instance with the uploaded files after first resizing the image to a thumbnail size. You will be able to reuse the `instance-form.html` template from *Chapter 7, Advanced Form Validation and Model Forms*, with a minor change to allow file uploads.

These steps will help you complete the activity:

1. Update the Django settings to add the `MEDIA_ROOT` and `MEDIA_URL` settings.
2. The `/media/` URL mapping should be added to the main `urls.py` file. Use the static view and utilize `MEDIA_ROOT` and `MEDIA_URL` from the Django settings. Remember, this mapping should only be added if `DEBUG` is `True`.
3. Add `ImageField` (named `cover`) and `FileField` (named `sample`) to the `Book` model. The fields should upload to `book_covers/` and `book_samples/`, respectively. They should both allow the `null` and `blank` values.
4. Run `makemigrations` and `migrate` again to apply the `Book` model changes to the database.
5. Create `BookMediaForm` as a subclass of `ModelForm`. Its model should be `Book`, and the fields should only be the fields you added in step 3.
6. Add a `book_media` view. This will not allow you to create `Book`; instead, it will only allow you to add media to an existing `Book` (so it must take `pk` as a required argument).
7. The `book_media` view should validate the form and use `save` to save it, but not commit the instance. The uploaded cover should first be resized using the `thumbnail` method, as demonstrated in the *Writing PIL images to ImageField* section. The maximum size should be 300 px by 300 px. It should then be stored on the instance, and the instance should be saved. Remember that the `cover` field is not required, so you should check this before trying to manipulate the image, and you won't need to resize the image if one is already on the instance. On a successful POST, register a success message that `Book` was updated, then redirect to the `book_detail` view.
8. Render `instance-form.html`, passing a context dictionary containing `form`, `model_type`, and `instance`, as you did in *Chapter 6, Forms*. Also, pass another item, `is_file_upload`, set to `True`. This variable will be used in the next step.

9. In the `instance-form.html` template, use the `is_file_upload` variable to add the correct enctype attribute to the form. This will allow you to switch the modes for the form to enable file uploads when required.
10. Finally, add a URL map that maps `/books/<pk>/media/` to the `book_media` view.

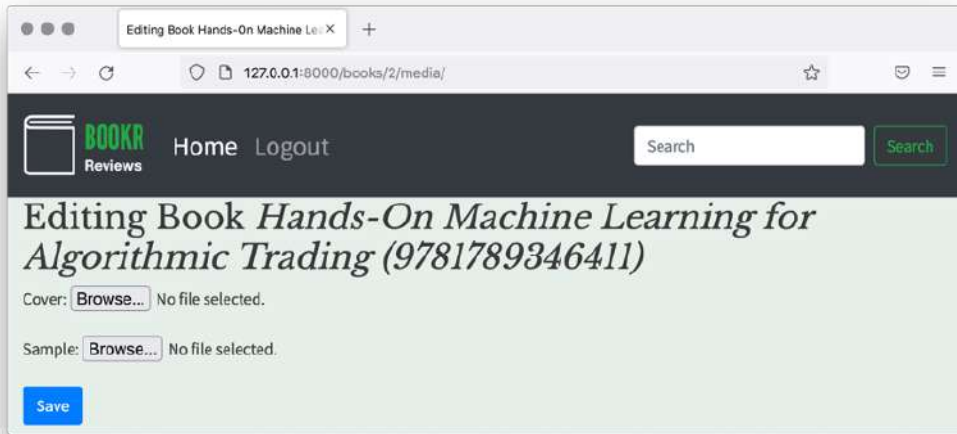


Figure 8.27: BookMediaForm in the browser

Select a cover image and a sample file for the book. You can use the image at `Chapter08/Activity8.01/bookr/media/book_covers/machine-learning-for-algorithmic-trading.png` and the PDF at `Chapter08/Activity8.01/bookr/media/book_samples/machine-learning-for-trading.pdf` (or you can use any other image/PDF of your choosing).

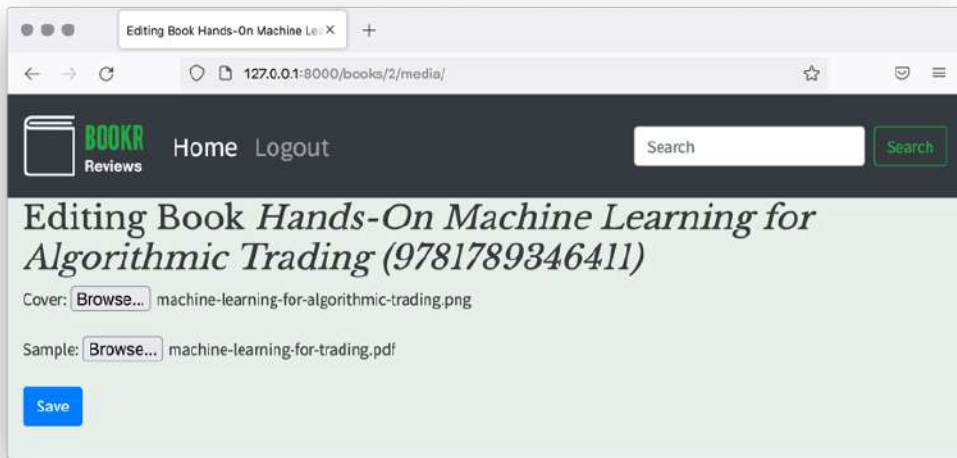


Figure 8.28: Book cover and sample selected

After submitting the form, you will be redirected to the **Book Details** view and see the success message (Figure 8.29):

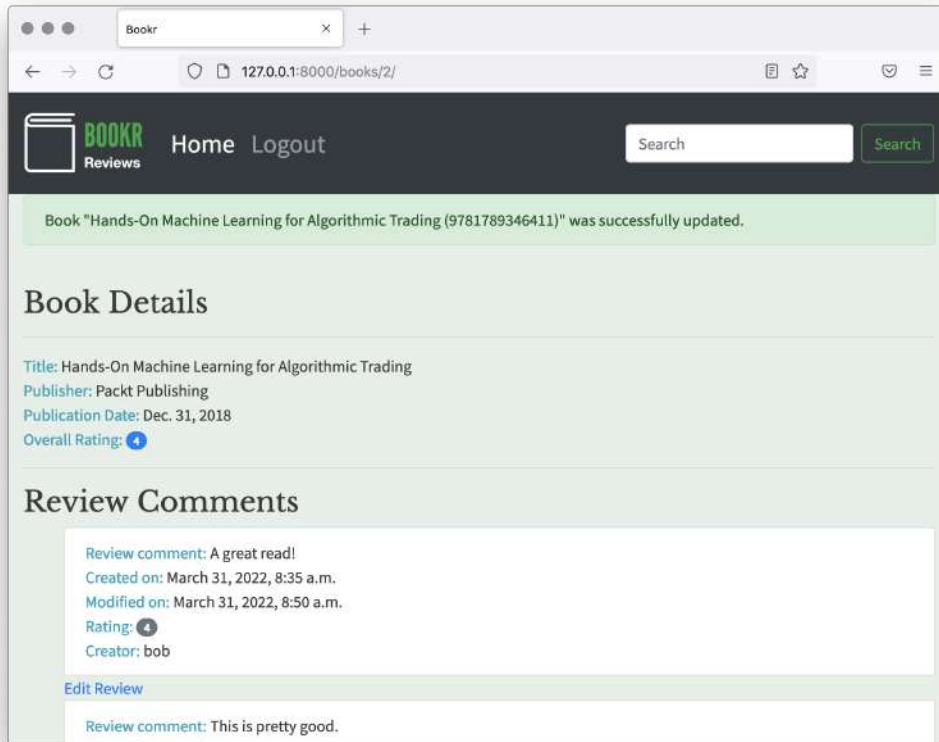


Figure 8.29: Success message on the Book Details page

If you go back to the same book's media page, you should see that the fields are now filled in with an option to clear the data from them:

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/books/2/media/'. The page title is 'Editing Book Hands-On Machine Learning for Algorithmic Trading (9781789346411)'. The page has a dark header with 'BOOKR Reviews' logo, 'Home', 'Logout', and a search bar. The main content area is light green and contains the following form elements:

- Cover:** Currently: [book_covers/machine-learning-for-algorithmic-trading.png](#) ☐ Clear
- Change:** machine-learning-for-trading.pdf
- Sample:** Currently: [book_samples/machine-learning-for-trading.pdf](#) ☐ Clear
- Change:** No file selected.
-

Figure 8.30: BookMediaForm with existing values

In *Activity 8.02 – displaying the cover and sample link*, you will add these uploaded files to the **Book Details** view, but for now, if you want to check that uploads have worked, you can look inside the media directory in the Bookr project:

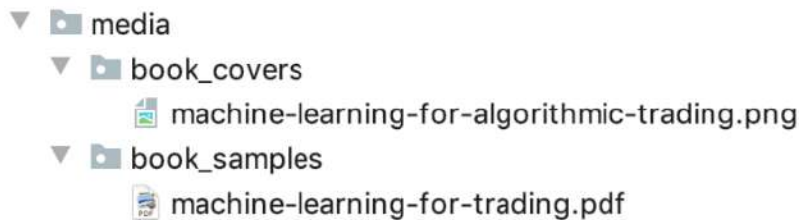


Figure 8.31: Book media

You should see the directories that were created and the uploaded files, as per *Figure 8.31*. Open an uploaded image, and you should see that its maximum dimension is 300 px.

Activity 8.02 – displaying the cover and sample link

In this activity, you will update the `book_detail.html` template to show the cover for Book (if one is set). You will also add a link to download the sample again, only if one is set. You will use the `FileField` and `ImageField` `url` attributes to generate the URLs to the media files.

These steps will help you complete this activity:

1. Inside the **Book Details** display in the `book_detail.html` view, add an `<img>` element if the book has a cover image. Then, display the cover of the book inside it. Use `<br>` after the `<img>` tag, so the image is on its own line.
2. After the **Publication Date** display, add a link to the sample file. It should only be displayed if a sample file has been uploaded. Make sure you add another `<br>` tag, so it displays correctly.
3. In the section that has a link to add a review, add another link that goes to the media page for the book. Follow the same styling as the **Add Review** link.

When you've completed these steps, you should be able to load a **Book Details** page. If the book has no cover or sample, then the page should look very similar to what it did before, except you should see the new link to the **Media** page at the bottom (*Figure 8.32*):

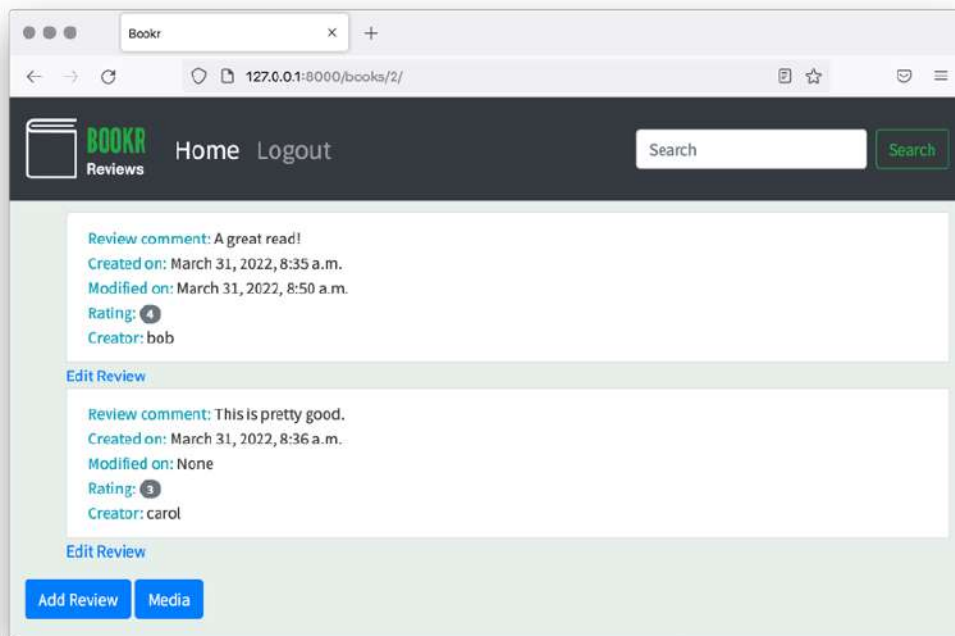


Figure 8.32: The new Media button visible on the Book Details page

Once you have uploaded a cover and/or sample for a book, the cover image and the sample link should be displayed (*Figure 8.33*):

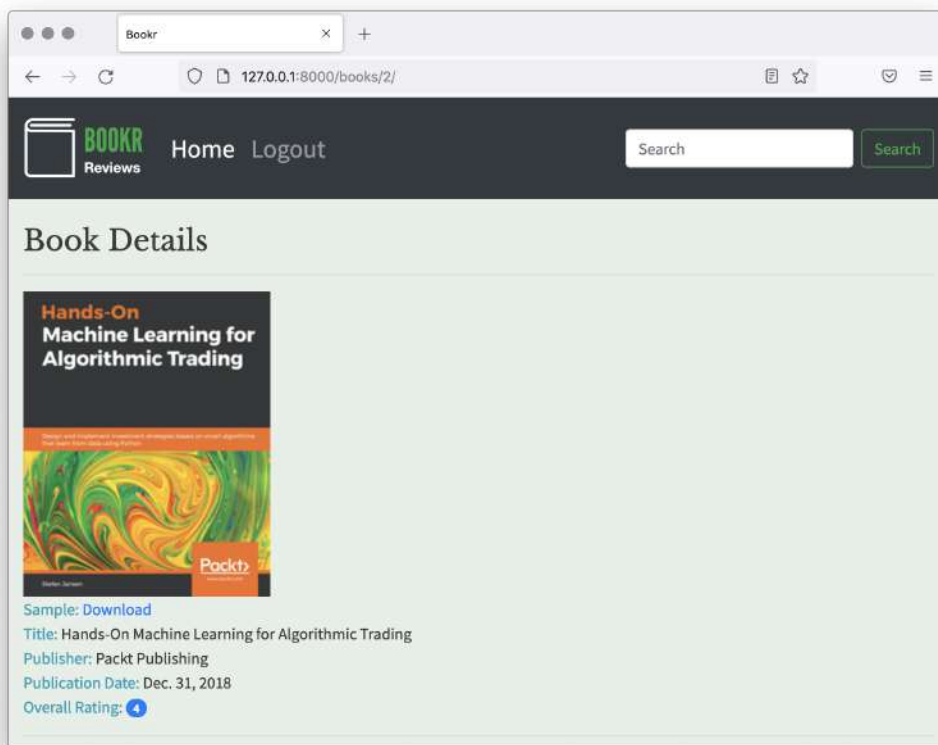


Figure 8.33: The book cover and the Sample link displayed

Summary

In this chapter, we added the `MEDIA_ROOT` and `MEDIA_URL` settings and a special URL map to serve media files. We then created a form and a view to upload files and save them to the media directory. We saw how to add the media context processor to automatically have access to the `MEDIA_URL` setting in all our templates. We then enhanced and simplified our form code by using a Django form with `FileField` or `ImageField`, instead of manually defining one in HTML.

We looked at some of the enhancements Django provides for images with `ImageField`, and how to interact with an image using Pillow. We showed an example view that would be able to serve files that required authentication using the `FileResponse` class. Then, we saw how to store files on models using `FileField` and `ImageField` and refer to them in a template using

the `FileField.url` attribute. We were able to reduce the amount of code we had to write by automatically building `ModelForm` from a model instance. Finally, we carried out two activities to enhance Bookr by adding a cover image and a sample file to the Book model.

In *Chapter 9, Sessions and Authentication*, we will learn how to add authentication to a Django application to protect it from unauthorized users.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

9

Sessions and Authentication

So far, we have used Django to develop dynamic applications that allow users to interact with application models, but we have not attempted to secure these applications from unwanted use. For example, our bookr project allows unauthenticated users to add reviews and upload media. This is a critical security issue for any online web app, as it leaves the site open to the posting of spam or other inappropriate material and the vandalism of existing content. We want the creation and modification of content to be strictly limited to authenticated users who have registered with the site.

The **authentication app** supplies Django with the models for representing users, groups, and permissions. It also provides middleware, utility functions, decorators, and mixins that help us integrate user authentication into our apps. Furthermore, the authentication app allows us to group and name certain sets of users.

In *Chapter 4, Introduction to Django Admin*, we used the Admin app to create a help desk user group with the Can view log entry, the Can view permission, the Can change user, and the Can view user permissions. Those permissions could be referenced in our code using their corresponding codenames: `view_logentry`, `view_permissions`, `change_user`, and `view_user`. In this chapter, we will learn how to customize Django behavior based on specific user permissions.

Permissions control what different types of users can do in your application. Permissions can be assigned either to groups or directly to individual users. From an administrative point of view, it is cleaner to assign permissions to groups. Groups make it easier to model roles and organizational structures. If a new permission is created, it is less time-consuming to modify a few groups than to remember to assign it to a subset of users.

We are already familiar with creating users and groups and assigning permissions using several methods, such as the option of instantiating users and groups through the model using

scripts and the convenience of creating them through the Django Admin app. The authentication app also offers us programmatic ways of creating and deleting users, groups, and permissions, and assigning relationships between them.

As we go through this chapter, we'll learn how to use authentication and permissions to implement application security and how to store user-specific data to customize the user's experience. This will help us secure the bookr project from unauthorized content changes and make it contextually relevant for different types of users. Adding this basic security to our bookr project is crucial before we consider deploying it on the internet.

This chapter begins with a brief introduction to **middleware** before delving into the concepts of **authentication models** and **session engines**. You will implement Django's authentication model to restrict permissions to only a specific set of users. Then, you will learn how to leverage Django authentication to provide a flexible approach to application security. After that, you will learn how Django supports multiple session engines to retain user data. By the end of this chapter, you will be proficient at using sessions to retain information on past user interactions and to maintain user preferences for when pages are revisited.

We will cover the following topics in this chapter:

- Middleware
- Password storage in Django
- Authentication decorators and redirection
- Enhancing templates with authentication data
- Sessions

Technical requirements

Find the solution in the Chapter09 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Middleware

Authentication, as well as session management (which we'll learn about in the *Sessions* section), is handled by something known as a **middleware stack**. Before we implement authentication in our bookr project, let's learn a bit about this middleware stack and its modules.

In *Chapter 3, URL Mapping, Views, and Templates*, we discussed Django's implementation of the request/response process, along with its view and rendering functionality. In addition to these, another feature that plays an extremely important role when it comes to Django's core web

processing is **middleware**. Django's middleware refers to a variety of software components that intervene in this request/response process to integrate important functionalities such as security, session management, and authentication.

So, when we write a view in Django, we don't have to explicitly set a series of important security features in the response header. These additions to the response object are automatically made by the `SecurityMiddleware` instance after the view returns its response. As middleware components wrap the view and perform a series of preprocesses on the request and post-processes on the response, the view is not cluttered with a lot of repetitive code, and we can concentrate on coding application logic rather than worrying about low-level server behavior. Rather than building these functionalities into the Django core, Django's implementation of a middleware stack allows these components to be both optional and replaceable.

Middleware modules

When we run the `startproject` subcommand, a default list of middleware modules is added to the `MIDDLEWARE` variable in the `<project>/settings.py` file, as follows:

```
MIDDLEWARE = [  
    "django.middleware.security.SecurityMiddleware",  
    "django.contrib.sessions.middleware.SessionMiddleware",  
    "django.middleware.common.CommonMiddleware",  
    "django.middleware.csrf.CsrfViewMiddleware",  
    "django.contrib.auth.middleware.AuthenticationMiddleware",  
    "django.contrib.messages.middleware.MessageMiddleware",  
    "django.middleware.clickjacking.XframeOptionsMiddleware",  
]
```

This is a minimal middleware stack that is suitable for most Django applications. The following list elaborates on the general purpose of each module:

- `SecurityMiddleware` provides common security enhancements, such as handling SSL redirects and adding response headers to prevent common hacks
- `SessionMiddleware` enables session support and seamlessly associates a stored session with the current request
- `CommonMiddleware` implements a lot of miscellaneous features, such as rejecting requests from the `DISALLOWED_USER_AGENTS` list, implementing URL rewrite rules, and setting the Content-Length header
- `CsrfViewMiddleware` adds protection against **cross-site request forgery (CSRF)**

- `AuthenticationMiddleware` adds the user attribute to the request object
- `MessageMiddleware` adds "flash" message support
- `XFrameOptionsMiddleware` protects against X-Frame-Options header clickjacking attacks

The middleware modules are loaded in the order that they appear in the `MIDDLEWARE` list. This makes sense because we want to call the middleware that deals with initial security issues first so that dangerous requests are rejected before further processing occurs. Django also comes with several other middleware modules that perform important functions, such as using `gzip` file compression, redirect configuration, and web cache configuration.

This chapter is devoted to discussing two important aspects of stateful application development that are implemented as middleware components – `SessionMiddleware` and `AuthenticationMiddleware`.

The `process_request` method of `SessionMiddleware` adds a session object as an attribute of the request object. The `process_request` method of `AuthenticationMiddleware` adds a user object as an attribute of the request object.

It is possible to write a Django project without these layers of the middleware stack if a project does not require user authentication or a means of preserving the state of individual interactions. However, most of the default middleware plays an important role in application security. If you don't have a good reason for changing the middleware components, it is best to maintain these initial settings. The Admin app requires `SessionMiddleware`, `AuthenticationMiddleware`, and `MessageMiddleware` to run, and the Django server will throw errors such as these if the Admin app is installed without them:

```
django.core.management.base.SystemCheckError: SystemCheckError: System check
identified some issues:

ERRORS:
?: (admin.E408) 'django.contrib.auth.middleware.AuthenticationMiddleware' must be
in MIDDLEWARE in order to use the admin application.
?: (admin.E409) 'django.contrib.messages.middleware.MessageMiddleware' must be in
MIDDLEWARE in order to use the admin application.
?: (admin.E410) 'django.contrib.sessions.middleware.SessionMiddleware' must be in
MIDDLEWARE in order to use the admin application.
```

Now that we know about the middleware modules, let's look at one way we can enable authentication in our project using the authentication app's views and templates.

Implementing authentication views and templates

We encountered the login form on the Admin app in *Chapter 4, Introduction to Django Admin*. This is the authentication entry point for staff users who have access to the Admin app. We also need to create a login capability for ordinary users who want to give book reviews. Fortunately, the authentication app comes with the tools to make this possible.

As we work through the forms and views of the authentication app, we will encounter a lot of flexibility in its implementation. We are free to implement our own login pages, define either very simple or fine-grained security policies at the view level, and authenticate against external authorities.

The authentication app exists to accommodate a lot of different approaches to authentication so that Django doesn't rigidly enforce a single mechanism. For a first-time user encountering the documentation, this can be quite bewildering. For the most part in this chapter, we will follow Django's defaults, but some of the important configuration options will be noted.

A Django project's settings object contains attributes for login behavior. `LOGIN_URL` specifies the URL of the login page. `"/accounts/login/"` is the default value. `LOGIN_REDIRECT_URL` specifies the path to where a successful login is redirected. The default path is `"/accounts/profile/"`.

The authentication app supplies standard forms and views for carrying out typical authentication tasks. These forms are located in `django.contrib.auth.forms`, while the views are located in `django.contrib.auth.views`.

The views are referenced by these URL patterns present in `django.contrib.auth.urls`:

```
urlpatterns = [
    path('login/', views.LoginView.as_view(),
         name='login'),
    path('logout/', views.LogoutView.as_view(),
         name='logout'),
    path('password_change/',
         views.PasswordChangeView.as_view(),
         name='password_change'),
    path('password_change/done/',
         views.PasswordChangeDoneView.as_view(),
         name='password_change_done'),
    path('password_reset/',
         views.PasswordResetView.as_view(),
         name='password_reset'),
```

```

    path('password_reset/done/',
         views.PasswordResetDoneView.as_view(),
         name='password_reset_done'),
    path('reset/<uidb64>/<token>/',
         views.PasswordResetConfirmView.as_view(),
         name='password_reset_confirm'),
    path('reset/done/',
         views.PasswordResetCompleteView.as_view(),
         name='password_reset_complete'),
]

```

If this style of views looks unfamiliar, it is because they are class-based views rather than the function-based views that we have previously encountered. We will learn more about class-based views in *Chapter 11, Advanced Templates and Class-Based Views*. For now, note that the authentication app makes use of class inheritance to group the functionality of views and prevent a lot of repetitive coding.

If we want to maintain the default URLs and views that are presupposed by the authentication app and Django settings, we can include the authentication app's URLs in our project's `urlpatterns`.

By taking this approach, we have saved a lot of work. We only need to include the authentication app's URLs in our `<project>/urls.py` file and assign it to the `'accounts'` namespace. Designating this namespace ensures that our reverse URLs correspond to the default template values of the views. The `authlib` views contain code references to templates named `password_reset_done` and `password_reset_complete`, so their paths need to be explicitly included in `urlpatterns` too:

```

from django.contrib import admin, auth
...

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/",
         include(("django.contrib.auth.urls", "auth"),
                 namespace="accounts")),
    path("accounts/password_reset/done/",
         auth.views.PasswordResetDoneView.as_view(),
         name="password_reset_done"),
    path("accounts/reset/done/",
         auth.views.PasswordResetCompleteView.as_view(),

```

```
        name="password_reset_complete",),
    path("", reviews.views.index),
    path("book-search/", reviews.views.book_search,
        name="book_search"),
    path("", include("reviews.urls")),
]
```

Though the authentication app comes with forms and views, it lacks the templates needed to render these components as HTML. *Figure 9.1* lists the templates that we require to implement the authentication functionality in our project. Fortunately, the Admin app does implement a set of templates that we can utilize for our purposes.

Note

When we say *Django source code*, it's the directory where your Django installation resides. If you installed Django in a virtual environment (as detailed in *Chapter 1*), you can find these template files at <name of your virtual environment>/lib/python3.X/site-packages/django/contrib/admin/templates/registration/. Provided your virtual environment is activated and Django is installed in it, you can also retrieve the complete path to the site-packages directory by running the python `-c "import sys; print(sys.path)"` command in a terminal.

We could just copy the template files from the Django source code in the `django/contrib/admin/templates/registration` directory and `django/contrib/admin/templates/admin/login.html` into our project's `templates/registration` directory:

Template Path
<code>templates/registration/login.html</code>
<code>templates/registration/password_reset_email.html</code>
<code>templates/registration/password_change_form.html</code>
<code>templates/registration/password_change_done.html</code>
<code>templates/registration/password_reset_form.html</code>
<code>templates/registration/password_reset_done.html</code>
<code>templates/registration/password_reset_confirm.html</code>
<code>templates/registration/password_reset_complete.html</code>
<code>templates/registration/logged_out.html</code>

Figure 9.1: Default paths for authentication templates

Note

We only need to copy the templates that are dependencies for the views, and we should avoid copying the `base.html` or `base_site.html` files.

This gives a promising result at first, but as they stand, the admin templates do not meet our precise needs, as we can see from the login page (*Figure 9.2*):

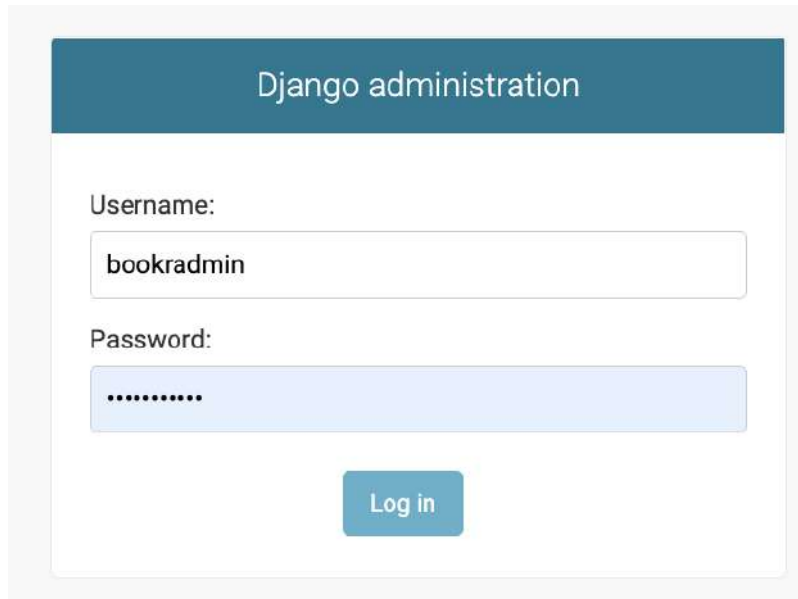


Figure 9.2: A first attempt at a user login screen

As these authentication pages inherit from the Admin app's `admin/base_site.html` template, they follow the style of the Admin app. We would prefer for these pages to follow the style of the bookr project that we have developed. We can do this by following these three steps on each Django template that we have copied from the Admin app to our project:

1. The first change that needs to be made is to replace the `{% extends "admin/base_site.html" %}` tag with `{% extends "base.html" %}`, as we want to inherit from the bookr project's base template.
2. Given that `template/base.html` only contains the title, brand, and content block definitions, we should examine the additional block substitutions from our templates in the bookr folder. We are not using the content from the `userlinks` and `breadcrumbs` blocks in our app's template layouts, so these blocks can be removed entirely. Some of these blocks, such as `content_title` and `reset_link`, contain HTML content that is relevant to our application, so they should be retained.

For example, the `password_change_done.html` template contains the `userlinks` and `breadcrumbs` blocks:

```
{% extends "admin/base_site.html" %}
{% load i18n %}
```

```
{% block userlinks %}
    {% url 'django-admindocs-docroot' as docsroot %}{% if docsroot %}<a
href="{% docsroot %}">{% translate 'Documentation' %}</a> / {% endif %}{%
translate 'Change password' %} /
    <form id="logout-form" ... ></form>
    {% include "admin/color_theme_toggle.html" %}
{% endblock %}
{% block breadcrumbs %} <div class="breadcrumbs">
<a href="{% url 'admin:index' %}">Home</a>
> Password change</div> {% endblock %}
{% block content %}
<p>{% translate 'Your password was changed.' %}</p>
{% endblock %}
```

It will be simplified to this template in the bookr project:

```
{% extends "base.html" %}
{% load i18n %}

{% block content %}
<p>{% translate 'Your password was changed.' %}</p>
{% endblock %}
```

3. Likewise, there are reverse URL patterns that need to change to reflect the current path, so {% url 'login' %} gets replaced with {% url 'accounts:login' %}.

Given these considerations, the next exercise will focus on transforming the Admin app's login template into a login template for the bookr project.

Note

The `i18n` module is used for creating multilingual content. If you intend to develop multilingual content for your website, leave the `i18n` import, `translate` tags, and `translateblock` statements in the templates. For brevity, we will not be covering those in detail in this chapter.

Exercise 9.01 — Repurposing the Admin app login template

We started this chapter without a login page for our project. By adding the URL patterns for authentication and copying the templates from the Admin app to our project, we can implement the functionality of a login page. But this login page is not satisfactory as it is directly copied from the Admin app and is disconnected from the Bookr design. In this exercise, we will follow the steps needed to repurpose the Admin app's login template for our project. The new login template will need to inherit its style and format directly from the bookr project's `templates/base.html` file:

1. Create a directory inside your project for `templates/registration`.

The Admin login template is located in the Django source directory at the `django/contrib/admin/templates/admin/login.html` path. It begins with an `extends` tag, a `load` tag, importing the `i18n` and `static` modules, and a series of block extensions that override the blocks defined in the child template, `django/contrib/admin/templates/admin/base.html`. A truncated snippet of the `login.html` file is shown in the following code block:

```
{% extends "admin/base_site.html" %}
{% load i18n static %}
{% block title %}{% if form.errors %}
{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block extrastyle %}...{% endblock %}
{% block bodyclass %}...{% endblock %}
{% block usertools %}{% endblock %}
{% block nav-global %}{% endblock %}
{% block content_title %}{% endblock %}
{% block breadcrumbs %}{% endblock %}
{% block content %}
{% if form.errors and not form.non_field_errors %}
... {% endif %}
<div id="content-main">
...
<form action="{% app_path ...>{% csrf_token %}
...
</form>
</div>
{% endblock %}
```

2. Copy this Admin login template, `django/contrib/admin/templates/admin/login.html`, into `templates/registration` and begin editing the file using PyCharm.
3. As the login template you are editing is located at `templates/registration/login.html` and extends the base template (`templates/base.html`), replace the argument of the `extends` tag at the top of `templates/registration/login.html` with the following:

```
{% extends "base.html" %}
```

4. We don't need most of the contents of this file. Just retain the content block, which contains the login form. The remainder of the template will consist of loading the `i18n` and static tag libraries:

```
{% load i18n static %}
{% block title %}{% if form.errors %}{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block content %}
...
{% endblock %}
```

5. Now, you must replace the paths and reverse URL patterns in `templates/registration/login.html` with ones that are appropriate to your project. As you don't have an `app_path` variable defined in your template, it needs to be replaced with the reverse URL for the login, `'accounts:login'`. So, consider the following line:

```
<form action="{{ app_path }}" method="post"
id="login-form">
```

This line changes as follows:

```
<form action="{% url 'accounts:login' %}"
method="post" id="login-form">
```

No `'admin_password_reset'` has been defined in your project paths, so it will be replaced with `'accounts:password_reset'`.

Consider the following line:

```
{% url 'admin_password_reset' as password_reset_url %}
```

This changes to the following line:

```
{% url 'accounts:password_reset'
as password_reset_url %}
```

Your login template will look as follows:

```
{% extends "base.html" %}
{% load i18n static %}
{% block title %}{% if form.errors %}{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block content %}
{% if form.errors and not form.non_field_errors %}
<p class="errornote">
{% if form.errors.items|length == 1 %}{% translate
"Please correct the error
below." %}{% else %}{% translate "Please correct
the errors below." %}{% endif %}
</p>
{% endif %}
```

You can find the complete code for this file in the Chapter09 folder in this book's GitHub repo.

6. As mentioned previously, to use the standard Django authentication views, we must add URL mappings to them. Open the `urls.py` file in the bookr project directory, ensure that `auth` is imported from `django.contrib`, and add the three paths to the URL patterns:

```
from django.contrib import admin, auth
...
urlpatterns = [
    ...
    path("accounts/",
        include(("django.contrib.auth.urls", "auth"),
            namespace="accounts")),
    path("accounts/password_reset/done/",
```

```
auth.views.PasswordResetDoneView.as_view(),
name="password_reset_done",),
path("accounts/reset/done/",
auth.views.PasswordResetCompleteView.as_view(),
name="password_reset_complete",),
...
]
```

7. Now, when you visit the login link at `http://127.0.0.1:8000/accounts/login/`, you will see this page:

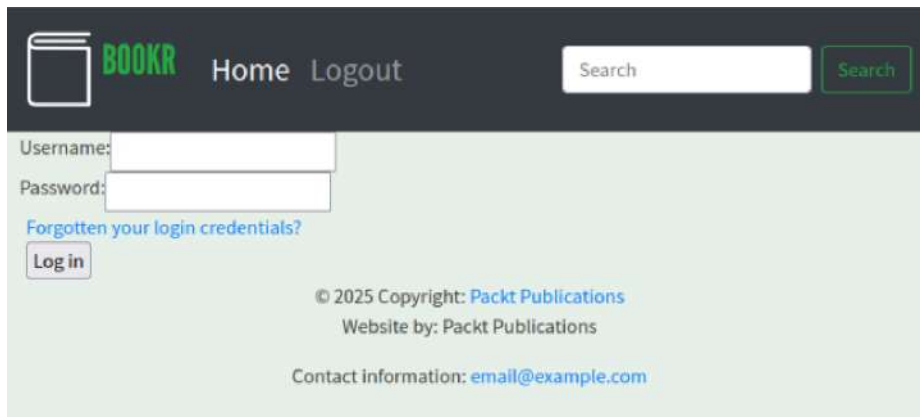


Figure 9.3: The Bookr login page

By completing this exercise, you have created the template required for non-admin authentication in your project.

Note

Before you proceed, you'll need to make sure the rest of the templates in the registration directory follow the bookr project's style; that is, they do not inherit from the Admin app's `admin/base_site.html` template. You've already seen this done with `password_change_done.html` and the `login.html` templates. Go ahead and apply what you've learned in this exercise (and the section before it) to the rest of the files in the registration directory. Alternatively, you may download the modified files from the Chapter09 folder in this book's GitHub repo.

Now that we have modified our project to enable authentication, we will learn about password and session storage in Django.

Password storage in Django

Django does not store passwords in plain text form in the database. Instead, passwords are digested with a hashing algorithm, such as **PBKDF2/SHA256**, **BCrypt/SHA256**, or **Argon2**. As hashing algorithms are a one-way transformation, this prevents a user's password from being decrypted from the hash stored in the database. This often comes as a surprise to users who expect a system administrator to retrieve their forgotten password, but it is best practice in security design. So, if we query the database for the password, we will see something like this:

```
sqlite> select password from auth_user;
pbkdf2_sha256$870000$81tMmVyCdDp4Kl3fe2khaF$XopYZzmVFwO4pZe7h014cy0sLBUzllYtG+uFsy9RG10=
pbkdf2_sha256$1000000$io5fNjVKzW74uvQB9L15LJ$V1NAajdh2r6dZhdvcvqptmvWd143N3Ce3qp19evf0H88=
pbkdf2_sha256$870000$UWL4u5af6xn637DmbUyY1j$jfDS1wfnYFwHejCkFF32E8yGnphK4tm0xALQ1ho/Gh4=
```

Figure 9.4 – Password hashes in a Django SQLite3 database

The components of this string are `<algorithm>$<iterations>$<salt>$<hash>`. As several hashing algorithms have been compromised over time and we sometimes need to work with mandated security requirements, Django is flexible enough to accommodate new algorithms and can maintain data encrypted in multiple algorithms.

The profile page and the `request.user` object

When a login is successful, the login view redirects to `/accounts/profile`. However, this path is not included in the existing `auth.url`, nor does the authentication app provide a template for it. To avoid a `Page not Found` error, a view and an appropriate URL pattern are required.

Each Django request has a `request.user` object. If the request is made by an unauthenticated user, `request.user` will be an `AnonymousUser` object. If the request is made by an authenticated user, then `request.user` will be a `User` object. This makes it easy to retrieve personalized user information in a Django view and render it in a template.

In the next exercise, we will add a profile page to our bookr project.

Exercise 9.02 – Adding a profile page

In this exercise, we will add a profile page to our project. To do so, we need to include the path to it in our URL patterns and also include it in our views and templates. The profile page will simply display the following attributes from the `request.user` object:

- `username`
- `first_name` and `last_name`
- `date_joined`

- email
- last_login

Perform the following steps to complete this exercise:

1. Add `bookr/views.py` to the project. It needs a trivial profile function to define our view:

```
from django.shortcuts import render

def profile(request):
    return render(request, 'profile.html')
```

2. In the templates folder of your main bookr project folder, create a new file called `profile.html`. In this template, the attributes of the `request.user` object can easily be referenced by using a notation such as `{{ request.user.username }}`:

```
{% extends "base.html" %}
{% block title %}Bookr{% endblock %}
{% block content %}
<h2>Profile</h2>
<div>
  <p>
    Username: {{ request.user.username }} <br>
    Name: {{ request.user.first_name }}
    {{ request.user.last_name }}<br>
    Date Joined: {{ request.user.date_joined }} <br>
    Email: {{ request.user.email }}<br>
    Last Login: {{ request.user.last_login }}<br>
  </p>
</div>
{% endblock %}
```

We also added a block containing the profile details of the user. More importantly, we made sure that `profile.html` extends `base.html`.

3. Finally, this path needs to be added to the top of the `urlpatterns` list in `bookr/urls.py`. First, import the new views and then add a path linking the `accounts/profile/` URL to `bookr.views.profile`:

```
from bookr.views import profile
```

```
urlpatterns = [...  
    path('accounts/profile/', profile,  
        name='profile'),  
    path("", reviews.views.index),  
    path("book-search/",  
        reviews.views.book_search,  
        name="book_search"),  
    path('', include('reviews.urls'))]
```

This is a good start on a user profile page. We created an account for Alice in the *CRUD operations using the Django admin app* section in *Chapter 4*. When Alice logs in and visits `http://localhost:8000/accounts/profile/`, it is rendered as shown in *Figure 9.5*. Remember, if the server needs to be started, use the `python manage.py runserver` command:

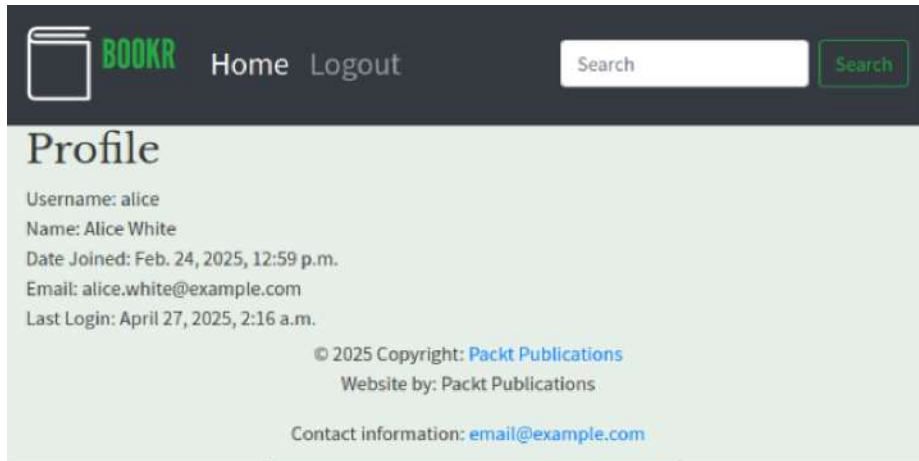


Figure 9.5: Alice visits her user profile

With that, we've seen how we can redirect a user to their profile page, once they've successfully logged in. Now, let's discuss how we can give content access to specific users only.

Authentication decorators and redirection

Now that we have learned how to allow ordinary users to log in to our project, we can discover how to restrict content to authenticated users. The authentication module comes with some useful decorators that can be used to secure views according to the current user's authentication or access.

At present, if Alice were to log out of Bookr, the profile page would still render and display empty details. Instead of this happening, it would be preferable for any unauthenticated visitor to be directed to the login screen:

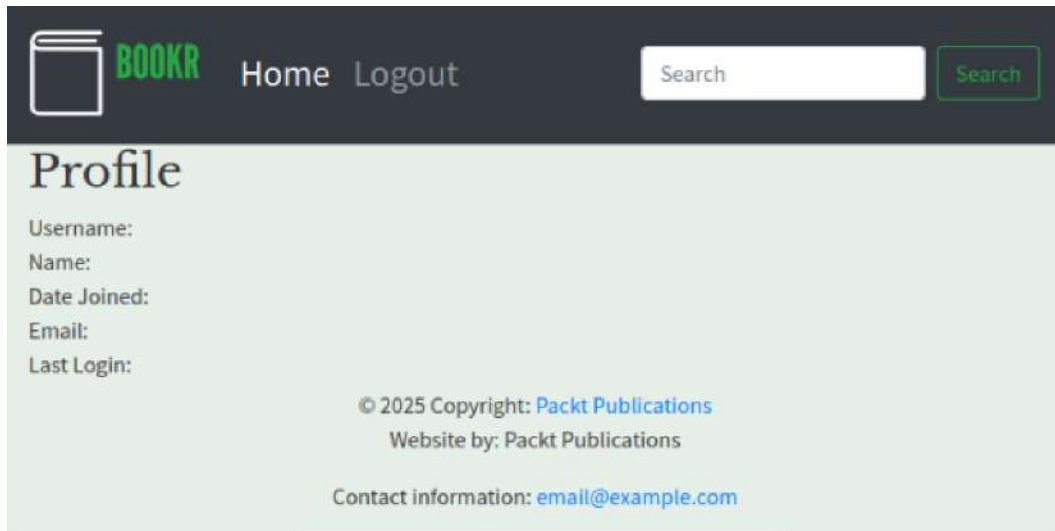


Figure 9.6: An unauthenticated user visits a user profile

The authentication app comes with useful decorators for adding authentication behavior to Django views. In this situation of securing our profile view, we can use the `login_required` decorator:

```
from django.contrib.auth.decorators import login_required

@login_required
def profile(request):
    ...
```

Now, if an unauthenticated user visits the `/accounts/profile` URL, they will be redirected to `http://localhost:8000/accounts/login/?next=/accounts/profile/`.

This URL takes the user to the login URL. The `next` parameter in the GET variables tells the login view where to redirect after a successful login. The default behavior is to redirect back to the current view, but this can be overridden by specifying the `login_url` argument to the

login_required decorator. For example, if we had some need to redirect to a different page after login, we could have explicitly stated it in the decorator call like this:

```
@login_required(login_url='/accounts/profile2')
```

If we had rewritten our login view to expect the redirection URL to be specified in a different URL argument to 'next', we could explicate this in the decorator call with the redirect_field_name argument:

```
@login_required(redirect_field_name='redirect_to')
```

There are often situations where a URL should be restricted to users or groups holding a specific condition. Consider the case where we have a page for staff users to view any user profile. We don't want this URL to be accessible to all users, so we want to limit this URL to users or groups with the 'view_user' permission and forward the unauthorized requests to the login URL:

```
from django.contrib.auth.decorators import (login_required,
                                             permission_required)
...

@permission_required('view_group')
def user_profile(request, uid):
    user = get_object_or_404(User, id=uid)
    permissions = user.get_all_permissions()
    return render(request, 'user_profile.html',
                  {'user': user, 'permissions': permissions})
```

So with this decorator applied to our user_profile view, an unauthorized user visiting `http://localhost:8000/accounts/users/123/profile/` would be redirected to `http://localhost:8000/accounts/login/?next=/accounts/users/123/profile/`.

Since Django 5.1, an additional middleware component called `LoginRequiredMiddleware` is available. This can be activated by adding `"django.contrib.auth.middleware.LoginRequiredMiddleware"` to the `MIDDLEWARE` list in a project's `settings.py` file. This will enforce the login requirement by default for views in the project. In order to allow a view to be called by non-validated users, the `@login_not_required` decorator will need to be employed.

Sometimes, though, we need to structure more subtle conditional permissions that don't fall into the scope of these two decorators. For this purpose, Django provides a custom decorator that takes an arbitrary function as an argument. The `user_passes_test` decorator requires a `test_func` argument:

```
user_passes_test(test_func, login_url=None,
                 redirect_field_name='next')
```

Here's an example where we have a view, `veteran_features`, that is only available to users who have been registered on the site for more than a year:

```
from django.contrib.auth.decorators import (login_required,
                                           permission_required, user_passes_test)
from django.utils import timezone...

def veteran_user(user):
    now = timezone.now()
    if user.date_joined is None:
        return False
    return now - user.date_joined >
        datetime.timedelta(days=365)

@user_passes_test(veteran_user)
def veteran_features(request):
    user = request.user
    permissions = user.get_all_permissions()
    return render(request, 'veteran_profile.html',
                  {'user': user, 'permissions': permissions})
```

Sometimes, the logic in our views cannot be handled with one of these decorators, and we need to apply the redirect within the control flow of the view. We can do this using the `redirect_to_login` helper function. It takes the same arguments as the decorators, as shown in the following snippet:

```
redirect_to_login(next, login_url=None,
                 redirect_field_name='next')
```

We will consolidate our knowledge of authentication decorators by applying them to the bookr project in the following exercise.

Exercise 9.03 – Adding authentication decorators to the views

Having learned about the flexibility of the authentication app's permission and authentication decorators, we will now set about putting them to use in the **Reviews** app. We need to ensure that only authenticated users can edit reviews and that only staff users can edit publishers. There are several ways of doing this, so we will attempt a few approaches. All the code in these steps is in the `reviews/views.py` file:

1. Your first instinct to solve this problem would be to think that the `publisher_edit` method needs an appropriate decorator to enforce that the user has `edit_publisher` permission. For this, you could easily do something like this:

```
from django.contrib.auth.decorators import \
    permission_required
...

@permission_required('edit_publisher')
def publisher_edit(request, pk=None):
    ...
```

2. Using this method is fine, and it's one way to add permissions checking to a view. You can also use a slightly more complicated but more flexible method. Instead of using a permission decorator to enforce permission rights on the `publisher_edit` method, you can create a test function that requires a staff user and apply this test function to `publisher_edit` with the `user_passes_test` decorator. Writing a test function allows more customization on how you validate users' access rights or permissions. If you made changes to your `views.py` file in *Step 1*, feel free to comment the decorator out (or delete it) and write the following test function instead:

```
from django.contrib.auth.decorators import \
    user_passes_test
...

def is_staff_user(user):
    return user.is_staff

@user_passes_test(is_staff_user)
...

```

3. Ensure that login is required for the `review_edit` and `book_media` functions by adding the appropriate decorator:

```
...

from django.contrib.auth.decorators import (
    login_required, user_passes_test)
...
@login_required
def review_edit(request, book_pk, review_pk=None):

@login_required
def book_media(request, pk):
...
```

4. In the `review_edit` method, add logic to the view that requires that the user be either a staff user or the owner of the review. The `review_edit` view controls the behavior of both review creation and review updates. The constraint that we are developing only applies to the case where an existing review is being updated. So, the place to add code is after a `Review` object has been successfully retrieved. If the user is not a staff account or the review's creator doesn't match the current user, we need to raise a `PermissionDenied` error:

```
...

from django.core.exceptions import PermissionDenied
from PIL import Image
from django.contrib import messages
...

@login_required
def review_edit(request, book_pk, review_pk=None):
    book = get_object_or_404(Book, pk=book_pk)

    if review_pk is not None:
        review = get_object_or_404(Review,
                                   book_id=book_pk,
                                   pk=review_pk)

        user = request.user
        if (not user.is_staff and
```

```
        review.creator.id != user.id):  
            raise PermissionDenied  
    else:  
        review = None  
    ...
```

Now, when a non-staff user attempts to edit another user's review, a Forbidden error will be thrown, as in *Figure 9.7*. In the next section, we will look at applying conditional logic in templates so that users aren't taken to pages that they don't have sufficient permission to access:



403 Forbidden

Figure 9.7: Access is forbidden to non-staff users

In this exercise, we used authentication decorators to secure views in a Django app. The authentication decorators that were applied provided a simple mechanism to restrict views from users lacking necessary permissions, non-staff users, and unauthenticated users. Django's authentication decorators provide a robust mechanism that follows Django's role and permission framework, while the `user_passes_test` decorator provides an option to develop custom authentication.

Now that we have used decorators to control the authentication flow of the application, we can customize templates with user authentication data.

Enhancing templates with authentication data

In *Exercise 9.02 – Adding a profile page*, we saw that we can pass the `request.user` object to the template to render the current user's attributes in the HTML. We can also take the approach of giving different template renderings according to the user type or the permissions held by a user. Consider that we want to add an edit link that only appears to staff users. We might apply an `if` condition to achieve this:

```
{% if user.is_staff %}
    <p><a href="{% url 'review:edit' %}">Edit this Review</a>
</p>
{% endif %}
```

If we didn't take the time to conditionally render links based on permissions, users would have a frustrating experience navigating the application, as many of the links that they click on would lead to 403 Forbidden pages. The following exercise will show how we can use templates and authentication to present contextually appropriate links in our project.

Exercise 9.04 – Toggling login and logout links in the base template

In the bookr project's base template, located in `templates/base.html`, we have a placeholder logout link in the header. It is coded in HTML as follows:

```
<li class="nav-item">
    <a class="nav-link" href="#">Logout</a>
</li>
```

We don't want the logout link to appear after a user has logged out. So, this exercise aims to apply conditional logic to the template so that the Login and Logout links are toggled, depending on whether the user is authenticated:

1. Edit the `templates/base.html` file. Copy the structure of the Logout list element and create a Login list element. Then, replace the placeholder links with the correct URLs for the Logout and Login pages – `/accounts/logout` and `/accounts/login`, respectively – as follows:

```
<li class="nav-item">
    <a class="nav-link" href="/accounts/logout">Logout
</a>
```

```
</li>
<li class="nav-item">
  <a class="nav-link" href="/accounts/login">Login</a>
</li>
```

2. Now, put our two `li` elements inside an `if ... else ... endif` conditional block. The logic condition that we are applying is `if user.is_authenticated`:

```
{% if user.is_authenticated %}
  <li class="nav-item">
    <a class="nav-link" href="/accounts/logout">Logout
  </a>
</li>
{% else %}
  <li class="nav-item">
    <a class="nav-link" href="/accounts/login">Login
  </a>
</li>
{% endif %}
```

3. Now, visit the user profile page at `http://localhost:8000/accounts/profile/`. When authenticated, you will see the **Logout** link:

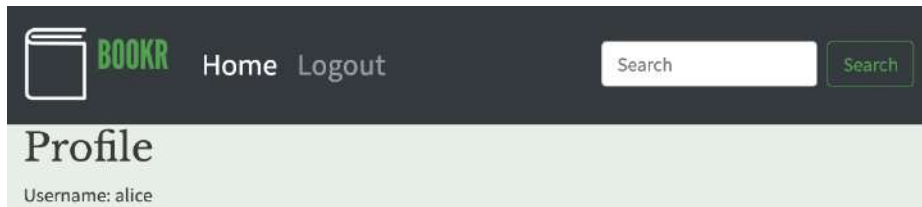


Figure 9.8: An authenticated user sees the Logout link

4. Now, click the **Logout** link; you will be taken to the `/accounts/logout` page. The **Login** link appears in the menu, confirming that the link is contextually dependent on the authentication state of the user:

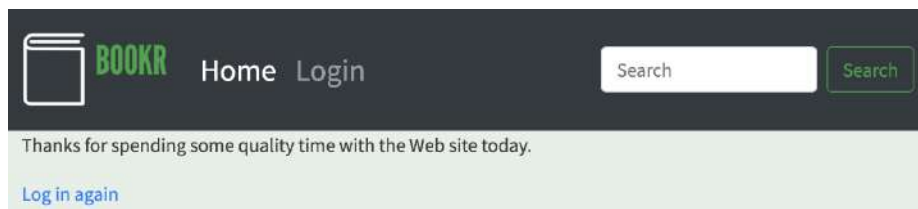


Figure 9.9: An unauthenticated user sees the Login link

Note

Requests to `/accounts/logout` require a POST request since Django 5.0. So, you will see that, in the example template in GitHub, there is a form with a button to handle this.

This exercise was a simple example of how Django templates can be used with authentication information to create a stateful and contextual user experience. We also do not want to provide links that a user does not have access to or actions that are not permissible for the user's permission level. The following activity will use this templating technique to fix some of these problems in Bookr.

Activity 9.01 – Authentication-based content using conditional blocks in templates

In this activity, you will apply conditional blocks to templates that modify content based on user authentication and user status. Users should not be presented with links that they are not permitted to visit or actions that they are not authorized to carry out. The following steps will help you complete this activity:

1. In the `book_detail` template, in the file at `reviews/templates/reviews/book_detail.html`, hide the **Add Review** and **Media** buttons from non-authenticated users.
2. Also, hide the heading that says, **Be the first one to write a review**, as that is not an option for non-authenticated users.
3. In the same template, make the **Edit Review** link only appear for the staff or the user who wrote the review. The conditional logic for the template block is very similar to the conditional logic that we used in the `review_edit` view in the previous section:

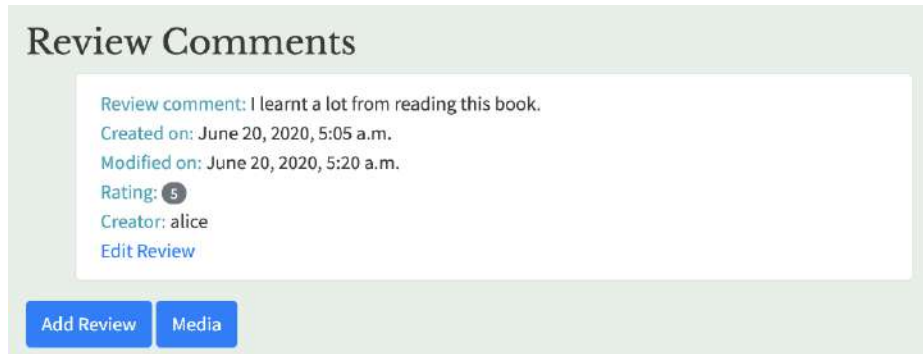


Figure 9.10: The **Edit Review** link appears on Alice's review when Alice is logged in

When a different user is logged in, the **Edit Review** link will not be present:

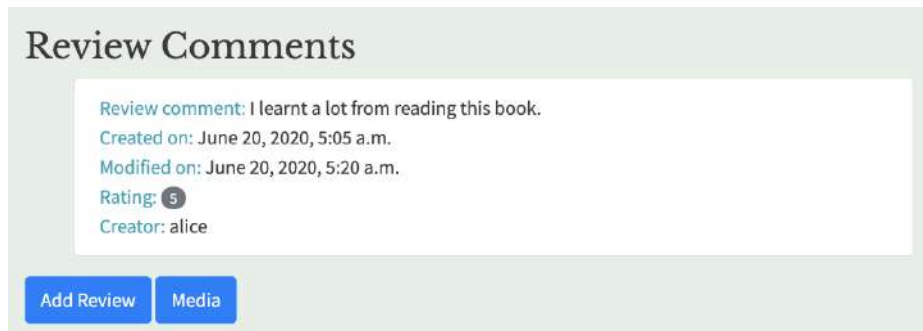


Figure 9.11: There is no **Edit Review** link on Alice's review when Bob is logged in

4. Modify `template/base.html` so that it displays the currently authenticated user's username to the right of the search form in the header, linking to the user profile page. By completing this activity, you will have added dynamic content to the template that reflects the authentication state and identity of the current user, as can be seen from the following screenshot:

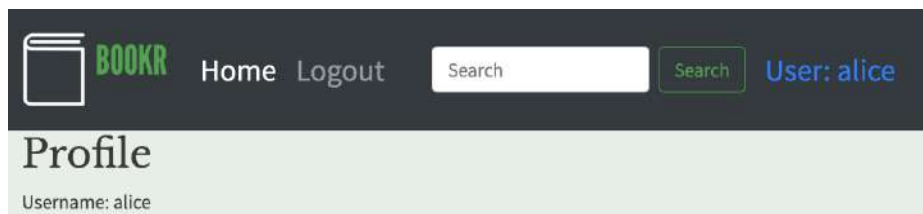


Figure 9.12: An authenticated user's name appears after the search form

With that, we have reviewed the available authentication mechanisms in Django and can examine how Django implements sessions.

Sessions

It is worth looking at some theory to understand why sessions are a common solution in web applications for managing user content. The HTTP protocol defines the interactions between a client and a server. It is said to be a "stateless" protocol as no stateful information is retained by the server between requests. This protocol design worked well for delivering hypertextual information in the early days of the World Wide Web, but it did not suit the needs of secure web applications delivering customized information to specific users.

We are now acquainted with seeing websites adapt to our viewing habits. Shopping sites recommend similar products to the ones that we have recently viewed and tell us about products that are popular in our region. These features all required a stateful approach to website development. One of the most common ways to implement a stateful web experience is through **sessions**. A session refers to a user's current interaction with a web server or application and requires that data be persisted for the duration of the interaction. This may include information about the links that the user has visited, the actions that they have performed, and the preferences that they have made in their interactions.

If a user sets a blogging site to a dark theme on one page, there is an expectation that the next page will use the same theme as well. We describe this behavior as "maintaining state." A session key is stored client-side as a browser cookie, which can be identified with server-side information that persists while the user is logged in.

In Django, sessions are implemented as a form of middleware. When we initially created the app in *Chapter 4, Introduction to Django Admin*, session support was activated by default.

The session engine

Information about current and expired sessions needs to be stored somewhere. In the early days of the World Wide Web, this was done through saving session information in files on the server, but as web server architectures have become more elaborate and their performance demands have increased, other, more efficient strategies, such as a database or in-memory storage, have become the norm. By default, in Django, session information is stored in a project's database.

This is a reasonable default for most small projects. However, Django's middleware implementation of sessions gives us the flexibility to store our project's session information in a variety of ways to suit our system architecture and performance requirements. Each of these

different implementations is called a **session engine**. If we want to change the session configuration, we need to specify the `SESSION_ENGINE` setting in the project's `settings.py` file:

- **Cached sessions:** In some environments, caching session information in memory or a database is an approach that is suited to high performance. Django provides the `django.contrib.sessions.backends.cache` and `django.contrib.sessions.backends.cached_db` session engines for this purpose.
- **File-based sessions:** As stated earlier, this is a somewhat antiquated way of maintaining session information, but it may suit some sites where performance is not an issue, and there are reasons not to store dynamic information in a database.
- **Cookie-based sessions:** Rather than keeping session information server-side, you can keep them entirely in the web browser client by serializing the content of the session as JSON and storing it in a browser-based cookie.

Do you need to flag cookie content?

All of Django's implementations of sessions require storing a session ID in a cookie on the user's web browser.

Regardless of the session engine employed, all these middleware implementations involve storing a site-specific cookie in the web browser. In the early days of web development, it was not uncommon to pass session IDs as URL arguments, but this approach has been eschewed in Django for reasons of security.

In many jurisdictions, including the European Union, websites are legally required to warn users if the site sets cookies in their browsers. If there are such legislative requirements in the region where you intend to operate your site, it is your responsibility to ensure that the code meets these obligations. Be sure to use up-to-date implementations and avoid using abandoned projects that have not kept pace with legislative changes.

Note

To cater to these changes and legislative requirements, there are many useful apps, such as **Django Cookie Consent** and **Django Cookie Law**, that are designed to work with several legislative frameworks. You can find out more by going to the following links:

- <https://github.com/jazzband/django-cookie-consent>
- <https://github.com/TyMaszWeb/django-cookie-law>

Many JavaScript modules exist that implement similar cookie consent mechanisms.

With this in mind, we can examine how session data is implemented in Django.

Pickle or JSON storage?

Python provides the **pickle** module in its standard library for serializing Python objects into a byte stream representation. A pickle is a binary structure that has the benefit of being interoperable between different architectures and different versions of Python, so that a Python object can be serialized to a pickle on a Windows PC and deserialized to a Python object on a Linux Raspberry Pi.

This flexibility comes with security vulnerabilities, and it is not recommended that it be used to represent untrusted data. Consider the following Python object, which contains several types of data. It can be serialized using pickle:

```
import datetime
data = dict(
    viewed_books=[17, 18, 3, 2, 1],
    search_history=['1981', 'Machine Learning', 'Bronte'],
    background_rgb=(96, 91, 92),
    foreground_rgb=(17, 17, 17),
    last_login_login=datetime.datetime(2019,12,3,15,30,30),
    password_change=datetime.datetime(2019,9,2,8,41,25),
    user_class='Veteran',
    average_rating=4.75,
    reviewed_books={18, 3, 7}
)
```

Using the `dumps` (dump string) method of the `pickle` module, we can serialize the data object to produce a byte representation:

```
import pickle
data_pickle = pickle.dumps(data)
```

JSON stands for **JavaScript Object Notation**. The syntax of JSON is a small subset of the JavaScript language. It is a widespread standard for messaging and data exchange, commonly used for transferring data between web browsers and servers. We can serialize JSON with a similar approach to the one that we outlined with the `pickle` format:

```
import json
data_json = json.dumps(data)
```

Because data contains Python datetime and set objects, which aren't serializable with JSON, when we attempt to serialize the structure, a type error will be thrown:

```
TypeError: Object of type datetime is not JSON serializable
```

For serializing to JSON, we could convert the datetime objects into the string type and set them in a list:

```
data['last_login_login'] = \
    data['last_login_login'].strftime("%Y%d%m%H%M%S")
data['password_change'] = \
    data['password_change'].strftime("%Y%d%m%H%M%S")
data['reviewed_books'] = list(data['reviewed_books'])
```

As JSON data is human-readable, it is easy to examine:

```
{"viewed_books": [17, 18, 3, 2, 1], "search_history": ["1981", "Machine
Learning", "Bronte"], "background_rgb": [96, 91, 92], "foreground_rgb": [17, 17,
17], "last_login_login": "20190312153030", "password_change": "20190209084125",
"user_class": "Veteran", "average_rating": 4.75, "reviewed_books": [18, 3, 7]}
```

Note that we had to explicitly convert the datetime and set objects, but the tuple is automatically converted into a list by the JSON. Django ships with `PickleSerializer` and `JSONSerializer`. If the serializer needs to be altered, it can be changed by setting the `SESSION_SERIALIZER` variable in the project's `settings.py` file:

```
SESSION_SERIALIZER = 'django.contrib.sessions.serializers.JSONSerializer'
```

Exercise 9.06 – Examining the session key

The purpose of this exercise is to query the project's SQLite database and perform queries on the session table to become familiar with how session data is stored. You will then create a Django management command for examining session data that is stored using `JSONSerializer`:

1. Start the **DB Browser for SQLite** application and open your `db.sqlite3` database:

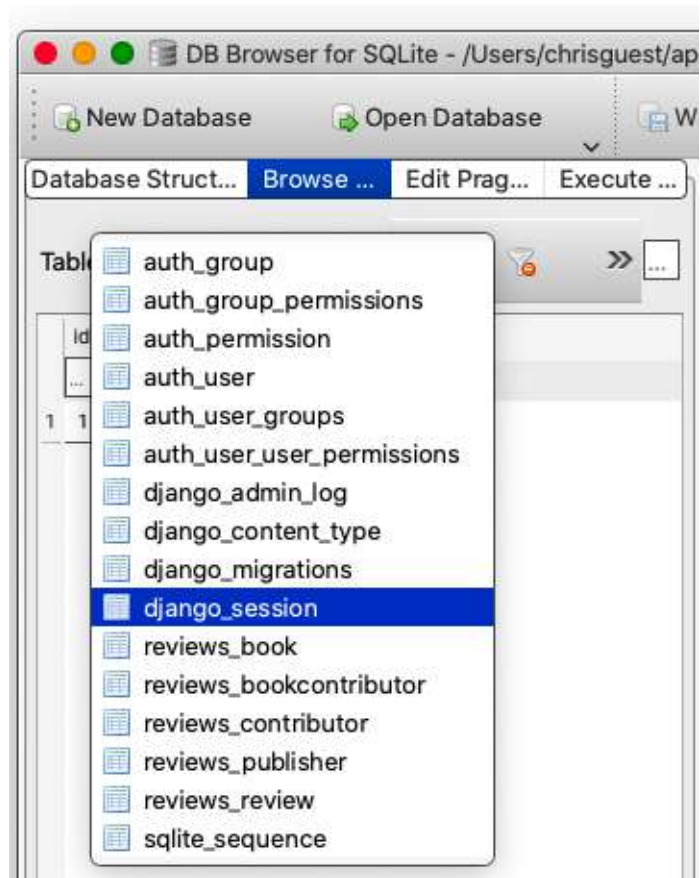


Figure 9.13: Table selection in DB Browser for SQLite

2. Under the **Database Structure** tab, expand the `django_session` table:

Name	Type	Schema
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "name" va
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMEN
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "cont
auth_user		CREATE TABLE "auth_user" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "password"
auth_user_groups		CREATE TABLE "auth_user_groups" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "use
auth_user_user_permissions		CREATE TABLE "auth_user_user_permissions" ("id" integer NOT NULL PRIMARY KEY AUTOINCREN
django_admin_log		CREATE TABLE "django_admin_log" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "act
django_content_type		CREATE TABLE "django_content_type" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "
django_migrations		CREATE TABLE "django_migrations" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "ap
django_session		CREATE TABLE "django_session" ("session_key" varchar(40) NOT NULL PRIMARY KEY, "session_d
session_key	varchar(40)	"session_key" varchar(40) NOT NULL
session_data	text	"session_data" text NOT NULL
expire_date	datetime	"expire_date" datetime NOT NULL
reviews_book		CREATE TABLE "reviews_book" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" va

Figure 9.14: Table definitions in DB Browser for SQLite

This reveals that the `django_session` table in the database stores session information in the `session_key`, `session_data`, and `expire_date` fields.

3. Click on the **Browse Data** tab and select the `django_session` table:

	session_key	session_data	expire_date
Filter			Filter
1	ecgo7ypa201tue9z0r1ffrog0u0d3	.a..VjDE0xjAMRe-...	2022-06-19 07:28:26.390874
2	y2r3d73wtprsmm2azch2a01ehdn0nr	.a..VjEEDWIAQRe.C2pAyQAsu3xs0Ms0AVa0kp0Z765Nu...	2022-06-19 07:29:15.509410
3	gyreicm0m8t8vm3c0iem6m2n1tgi	.a..VjDE0WYAQBf9CMAgZDxCkTJ630UD4CQCydVib0_hil...	2022-06-19 07:30:26.166834

Figure 9.15: Querying data in the `django_session` table

4. Django stores the session data using cryptographically signed values. Because, by default, we are using the database to store our sessions, we will import the `Session` model, as well as the `SessionStore` class of `django.contrib.sessions.backends.db`. `SessionStore` has a `decode` method that we can use to read the encoded `session_data` from the `Session` object.
5. This code snippet can be executed using the Django shell by entering `python manage.py shell` at the command line to get an interactive prompt:

```
>>> from django.contrib.sessions.backends.db import SessionStore
>>> from django.contrib.sessions.models import Session
>>> session_store = SessionStore()
>>> sessions = Session.objects.all()
>>> session_store.decode(sessions[0].session_data)
```

```
{'_auth_user_id': '2', '_auth_user_backend':  
'django.contrib.auth.backends.ModelBackend', '_auth_user_hash':  
'eb5de6bf54460bafc9e0f555c65d16b17b3fd754617d1d73529d9605fea2f0c'}
```

6. Using this approach, we can create a management command for examining session data. Sensitive data may be stored in sessions, so creating a management command ensures that the data is only accessible by a user with console access to the server that is hosting Django.
7. Using **PyCharm**, in the `reviews/management/commands` directory of the `bookr` project, create a Python file called `sessioninfo.py`.
8. Let's start by importing the necessary modules. We will use the `pformat` command from the `pprint` module from Python's Standard Library to format session data. We will need the `User` model from `django.contrib.auth.models` and the `Session` model from `django.contrib.sessions.models` to query `User` and `Session` objects. The `BaseCommand` class from `django.core.management.base` provides the scaffolding for our Django Admin command:

```
from pprint import pformat  
  
from django.contrib.auth.models import User  
from django.contrib.sessions.models import Session  
from django.contrib.sessions.backends.db import SessionStore  
from django.core.management.base import BaseCommand
```

9. At a minimum, a user command needs to be defined by subclassing `BaseCommand` as the `Command` class and defining a `handle` method. Typically, additional command-line parameters would be defined in an `add_arguments` method, but we are aiming to create a very simple example. A `help` attribute of the `Command` class is used to provide information when the user enters `python manage.py --help` or `python manage.py sessioninfo --help`:

```
class Command(BaseCommand):  
    help = "List all user sessions and the data that they contain."
```

10. Write a `handle` method of the `Command` class. First, instantiate a `SessionStore` object, then iterate through the `Session` objects and decode the data for each using the `SessionStore.decode` method. The `_auth_user_id` key of the decrypted data

references a `User.id` from the database so that we can retrieve the user using the `User` model.

Now, for each session, write the session key, user ID, username, and email address to the console. We will develop a simple Python utility for decrypting this session information. Note that it is best practice to call `self.stdout.write` in a management command instead of `print` when writing to the console:

```
def handle(self, *args, **options):
    session_store = SessionStore()
    for session in Session.objects.all():
        data =
    session_store.decode(session.session_data)
    user =
    User.objects.get(id=data['_auth_user_id'])
    self.stdout.write(
        f"Session Key: {session.session_key} "
        f"User: {user.id} {user.username}
        {user.email} ")
    self.stdout.write(pformat(data))
```

11. Now, you can use this management command to examine session data that is stored in the database. You can call it on the command line as follows:

```
python manage.py sessioninfo
```

This will be useful for debugging session behavior when you attempt the final activity:

```
> python manage.py sessioninfo
Session Key: ocgo7sy paz01tlw92dr1ffkrogy0ux03 User: 2 carol carol.brown@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': 'eb5de6bf54460bafc9e0f555c65d16b17b3fdf754617d1d73529d9605fea2f0c',
 '_auth_user_id': '2'}
Session Key: y2r3d3fwtpmsmrn2azch2a01ehdnbhr User: 1 bob bob.black@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': '322625efba7788f598d02fca5ad86bb14db4893f3c92bc4099822ea3bf4d1e4f',
 '_auth_user_id': '1'}
Session Key: gyfciciqm8ht8vmx3c8i8m8m6zh11tgi User: 3 bookradmin bookradmin@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': 'a30d9315c9b5dda38802da5e8e64f94b0b063f0136fad8456205ed8cced8ff30',
 '_auth_user_id': '3'}
```

Figure 9.16: Python script

This script outputs the decoded session information. At present, the session only contains three keys:

- `_auth_user_backend` is a string representation of the class of the user backend. As our project stores user credentials in the model, `ModelBackend` is used.
- `_auth_user_hash` is a hash of the user's password.
- `_auth_user_id` is the user ID obtained from the model's `User.id` attribute.

This exercise helped you become familiar with how session data is stored in Django. We will now turn our attention to adding additional information to Django sessions.

Storing data in sessions

We've already covered the way sessions are implemented in Django. Now, we are going to briefly examine some of the ways that we can make use of sessions to enrich our user experience. In Django, `session` is an attribute of the request object. It is implemented as a dictionary-like object. In our views, we can assign keys to the `session` object like a typical dictionary, as shown here:

```
request.session['books_reviewed_count'] = 39
```

But there are some restrictions. Firstly, the keys in the session must be strings, so integers and timestamps are not allowed. Secondly, keys starting with an underscore are reserved for internal system use. Data is limited to values that can be encoded as JSON, so some byte sequences that can't be decoded as UTF-8, such as `binary_key` listed previously, can't be stored as JSON data. The other warning is to avoid reassigning `request.session` to a different value. We should only assign or delete keys. So, *don't do* this:

```
request.session = {'books_read_count':30,  
                  'books_reviewed_count': 39}
```

Instead, *do* this:

```
request.session['books_read_count'] = 30  
request.session['books_reviewed_count'] = 39
```

With those restrictions in mind, we will investigate how we can make use of session data in our Reviews application.

Exercise 9.06 – Storing recently viewed books in sessions

The purpose of this exercise is to use the session to keep information about the 10 books that have been most recently browsed by the authenticated user. This information will be displayed on the profile page of the bookr project. When a book is browsed, the `book_detail` view is called. In this exercise, we will edit `reviews/views.py` and add some additional logic to the `book_detail` method. We will add a key to the session called `viewed_books`. Using basic knowledge of HTML and CSS, the page can be created to show the profile details and viewed books stored in separate divisions of the page, as follows:

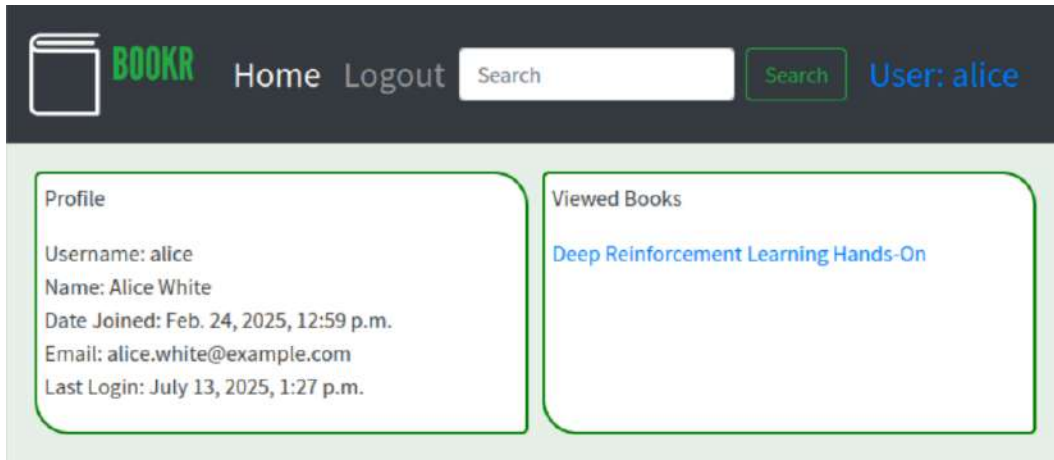


Figure 9.17: The Profile page incorporating Viewed Books

1. Edit `reviews/views.py` and the `book_detail` method. We are only interested in adding session information for authenticated users, so add a conditional statement to check whether the user is authenticated and set `max_viewed_books_length`, the maximum length of the viewed books list, to 10:

```
def book_detail(request, pk):
    ...
    if request.user.is_authenticated:
        max_viewed_books_length = 10
```

2. Within the same conditional block, add code to retrieve the current value of `request.session['viewed_books']`. If this key isn't present in the session, start with an empty list:

```
viewed_books = request.session.get(
    'viewed_books', [])
```

3. If the current book's primary key is already present in `viewed_books`, the following code will remove it:

```
viewed_book = [book.id, book.title]
if viewed_book in viewed_books:
    viewed_books.pop(viewed_books.index(viewed_book))
```

4. The following code inserts the current book's primary key at the start of the `viewed_books` list:

```
viewed_books.insert(0, viewed_book)
```

5. Add the following key to only keep the first 10 elements of the list:

```
viewed_books = viewed_books[:
    max_viewed_books_length]
```

6. The following code will add our `viewed_books` back to `session['viewed_books']` so that it is available in subsequent requests:

```
request.session['viewed_books'] = viewed_books
```

7. As before, at the end of the `book_detail` function, render the `reviews/book_detail.html` template given the request and context data:

```
return render(request, "reviews/book_detail.html",
    context)
```

Once complete, the `book_detail` view will have this conditional block:

```
def book_detail(request, pk):
    ...
    if request.user.is_authenticated:
```

```

max_viewed_books_length = 10
viewed_books =
    request.session.get('viewed_books', [])
viewed_book = [book.id, book.title]
if viewed_book in viewed_books:
    viewed_books.pop(viewed_books.index(viewed_book))
viewed_books.insert(0, viewed_book)
viewed_books = viewed_books[:max_viewed_books_length]
request.session['viewed_books'] = viewed_books
return render(request, "reviews/book_detail.html",
               context)

```

8. Modify the page layout and CSS of templates/profile.html to accommodate the viewed_books division. As we may add more divisions to this page in the future, one convenient layout concept is the **flexbox**. We will add this CSS and separate the content into nested div instances that will be arranged horizontally on the page. We will refer to the internal div instances as infocell instances and style them with green borders and rounded edges:

```

<style>
.flexrow { display: flex;
           border: 2px solid black;
        }
.flexrow > div { flex: 1; }

.infocell {
    border: 2px solid green;
    border-radius: 5px 25px;
    background-color: white;
    padding: 5px;
    margin: 20px 5px 5px 5px;
}

</style>

<div class="flexrow" >
    <div class="infocell" >
        <p>Profile</p>
        ...
    </div>

```



```

<div class="info cell" >
  <p>Viewed Books</p>
  ...
</div>
</div>

```

9. Modify the `viewed_books` div element in `templates/profile.html` so that if there are books present, their titles are displayed and linked to the individual book detail pages. This will be rendered as follows:

```

<a href="/books/1">Advanced Deep Learning with Keras </a><br>

```

A message should be displayed if the list is empty. The entire div, including the iteration through `request.session.viewed_books`, will look like this:

```

<div class="info cell" >
  <p>Viewed Books</p>
  <p>
    {% for book_id, book_title in
      request.session.viewed_books %}
    <a href="/books/{{ book_id }}">{{ book_title }}
    </a><br>
    {% empty %}
      No recently viewed books found.
    {% endfor %}
  </p>
</div>

```

You can find the complete code for the `templates/profile.html` in the `Chapter09` folder in this book's GitHub repo.

This exercise has enhanced the profile page by adding a list of recently viewed books. Now, when you visit the **Login** link at `http://127.0.0.1:8000/accounts/profile/`, you will see this page:

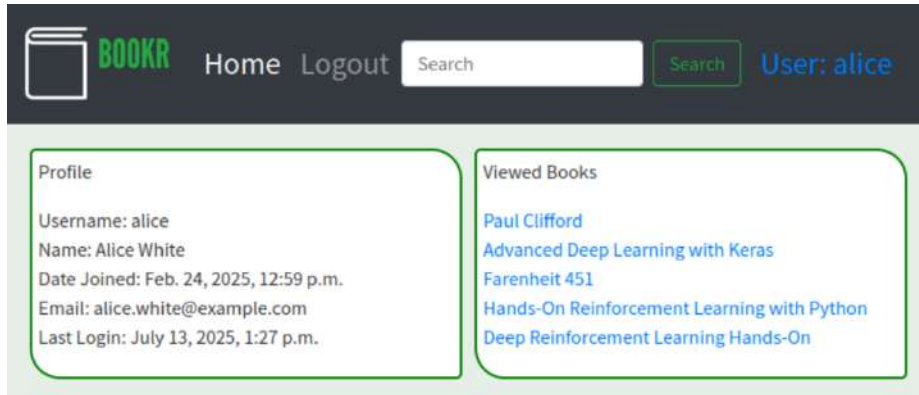


Figure 9.18: Recently viewed books

We can use the `sessioninfo` management command that we developed in *Exercise 9.06 – Examining the session key*, to examine the user's session once this feature is implemented. It can be called on the command line by passing the session data as an argument:

```
python manage.py sessioninfo
```

We can see that the book IDs and titles are listed in the `viewed_books` key. Remember that the encoded data is obtained by querying the `django_session` table in the SQLite database:

```
> python manage.py sessioninfo
Session Key: mjbbpj6e6y0qayh1mj6sgw27kfugklqc User: 4 alice alice.white@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': '384340e0724b154062dcad2c94a9fc94380cd323b9e1e433aff3aef799b99a3e',
 '_auth_user_id': '4',
 'viewed_books': [[17, 'Paul Clifford'],
 [1, 'Advanced Deep Learning with Keras'],
 [13, 'Fahrenheit 451'],
 [6, 'Hands-On Reinforcement Learning with Python'],
 [4, 'Deep Reinforcement Learning Hands-On']]}
```

Figure 9.19: The viewed books are stored in the session data

In this exercise, we used Django's session mechanism to store ephemeral information about user interactions with the Django project. We have learned how this information can be retrieved from the user session and displayed in a view that informs users about their recent activity.

Exercise 9.07 – Using session storage for the Search for Books page

Sessions are a useful way to store short-lived information that assists in maintaining a stateful experience on a site. Users frequently revisit pages such as search forms, and it would be convenient to store their most recently used form settings when they return to those pages. In *Chapter 3, URL Mapping, Views, and Templates*, we developed a Book Search feature for the bookr project. The **Book Search** page has two options for **Search in: Title** and **Contributor**. Currently, each time the page is visited, it defaults to **Title**:

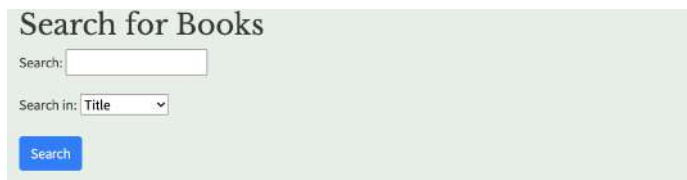
A screenshot of a web form titled "Search for Books". It features a text input field labeled "Search:" followed by a dropdown menu labeled "Search in:" with "Title" selected. Below these is a blue "Search" button.

Figure 9.20: The Search and Search in fields of the Book Search form

In this activity, you will use session storage so that when the book search page, `/book-search`, is visited, it will default to the most recently used search option. You will also add a third `infoCell` to the profile page that contains a list of links to the most recently used search terms. Follow these steps to complete this activity:

1. Edit the `book_search` view and retrieve `search_history` from the session.
2. When the form has received valid input and a user is logged in, append the search option and search text to the session's search history list.

If the form hasn't been filled (for example, when the page is first visited), render the form with the previously used **Search in** option selected – that is, either **Title** or **Contributor** (Figure 9.21):

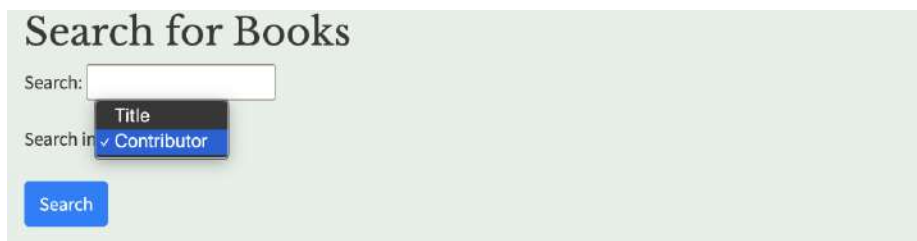
A screenshot of the "Search for Books" form. The "Search in:" dropdown menu is open, showing two options: "Title" and "Contributor". The "Contributor" option is highlighted with a blue background and a checkmark, indicating it is the selected option. The "Search" button remains visible below.

Figure 9.21: Selecting Contributor in the Search field

3. In the profile template, include an additional `infocell` division for Search History.
4. List the search history as a series of links to the book search page. The links will take this form: `/book-search?search=Python&search_in=title`.

This activity will challenge you to apply session data to solve a usability issue in a web form. This approach will have applicability in many real-world situations and will give you some idea of the use of sessions in creating a stateful web experience. After completing this activity, the profile page will contain the third `infocell`, as shown in *Figure 9.22*:

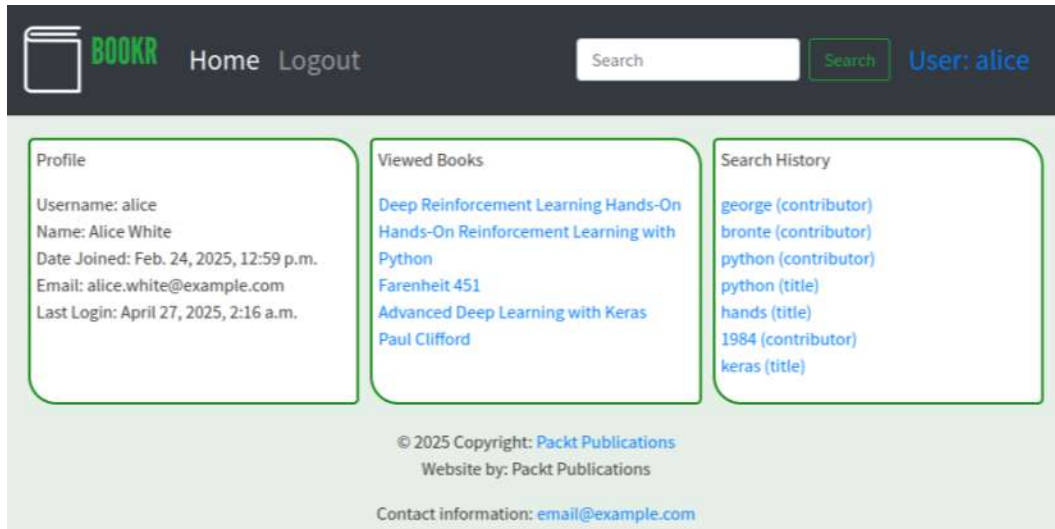


Figure 9.22: The profile page with the Search History infocell

Summary

In this chapter, we examined Django's middleware implementation of authentication and sessions. We also learned how to incorporate authentication and permission logic into views and templates. Now, we can set permissions on specific pages and limit their access to authenticated users. We also examined how to store data in a user's session and render it on subsequent pages.

With that, you have the skills to customize a Django project to deliver a personalized web experience. You can limit the content to authenticated or privileged users, and you can personalize a user's experience based on their prior interactions.

In the next chapter, we will revisit the Admin app and learn about some advanced techniques we can use to customize our user model and apply fine-grained changes to the admin interface for our models.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

10

Advanced Django Admin and Customizations

Let's say we want to customize the front page of a large organization's admin site. We want to show the health of the different systems in the organization and see any active high-priority alerts. If this were an internal website built on top of Django, we would need to customize it. Adding these kinds of functionalities will require the developers in the IT team to customize the default admin panel and create their own custom `AdminSite` module, which will render a different index page in comparison to what is provided by the default admin site. Fortunately, Django makes these kinds of customizations easy.

In this chapter, we will look at how we can leverage Django's framework and its extensibility to customize Django's default admin interface. We'll not just learn how to make the interface more personal; we will also learn how we can control the different aspects of the admin site to make Django load a custom admin site, instead of the one that ships with the default framework. Such customization can come in handy when we want to introduce features into the admin site that are not present by default.

This chapter builds upon the skills we practiced in *Chapter 4, Introduction to Django Admin*. Just to recap, we learned how to use the Django admin site to take control of the administration and authorization for our Bookr app. We also learned how to register models to read and edit their contents, and how to customize Django's admin interface using the `admin.site` properties. We will finish up by learning about how to configure an email server with Django and the use of the Django Tasks framework.

The following topics are covered in this chapter:

- Customizing the user model
- Form enhancements with `ModelAdmin` attributes

- Adding views to the Django admin app
- Configuring an email server with Django
- Automating admin with Django Tasks

Now, let's expand our knowledge further by taking a look at how we can start customizing the admin site by utilizing Django's `AdminSite` module to add powerful new functionalities to the admin portal of our web application.

Technical requirements

Find the solution in the `Chapter10` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Customizing the admin site

Django, as a web framework, provides a lot of customization options for building web applications. We will be using this same freedom provided by Django when we work on building the admin application for our project.

In *Chapter 4, Introduction to Django Admin*, we looked at how we can use the `admin.site` properties to customize the elements of Django's admin interface. However, what if we require more control over how our admin site behaves? For example, what if we wanted to show a custom template for the login page or the logout page when the user comes to the Bookr admin panel? In this case, the `admin.site` properties provided might not be enough, and we will need to build customizations that can extend the default admin site's behavior.

A customized user model

The user model that is provided with the authentication app comes with very general features. It is often the case that we want to retain more specific information about a user, such as address or phone number, or we may want to implement a different authentication pattern than the username/password approach. These are important design considerations. It is best to consider it now before we accumulate a lot of user data and need to replace the user model at a later date.

We will look at customizing the user model by either extending the user model or by subclassing one of its inherited abstract models.

Extending the user model

The first approach involves adding a model to contain supplementary information about our users. This doesn't involve modifications to the auth app or its user model. Instead, a model in a 1-to-1 relationship with the user model is created.

To illustrate this, we will add a `ReviewerProfile` model to the `reviews/models.py` file. This model contains additional details about our user, such as their favorite book and their profile picture. To define fields in models, we need to use the `settings.AUTH_USER_MODEL` attribute.

We will create a `BookrUser` model.

```
from django.contrib.auth.models import User
...

class ReviewerProfile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE)
    location = models.CharField(max_length=100)
    favourite_book = models.ForeignKey(
        Book, null=True, on_delete=models.SET_NULL,
        help_text="Your favourite book (if it is present in Bookr)")
    profile_photo = models.ImageField(
        null=True, blank=True, upload_to="profile_photos/")
```

Note that we have added a `favourite_book` attribute so that a reviewer can optionally select their favorite book from the reviews app if it is present. We've also added a `profile_photo` attribute, so that the user can add a display picture. We have set `on_delete=models.SET_NULL` because we want to ensure that the record deletion of that book would lead to the unsetting of the `favourite_book` attribute rather than the deletion of `ReviewerProfile`.

Currently, we are using the auth user model with our app, but we might want to future-proof the code so that our app does not make this assumption. This is particularly important if the app is intended to be reused in other projects where no assumptions about the user model can be made.

To do so, we can use the `get_user_model` function from the `auth` app to determine which user model is being used in the project.

As we saw in the previous chapters, the `on_delete=models.CASCADE` parameter ensures that when the `User` object is deleted, `ReviewerProfile` will be deleted too. Similarly, in order to ensure that `ReviewerProfile` is created after a `User` object is created, we can employ an easy-to-code Django signal. This could be written to `signals.py`, which can then be imported in the `ready` function of the `AppConfig` class in `apps.py`.

```
from django.dispatch import receiver
from django.db.models.signals import post_save

...

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_reviewer_profile(sender, created, instance,
                           **kwargs):
    if created:
        profile = ReviewerProfile(user=instance)
        profile.save()
```

Here, we are using the `sender=settings.AUTH_USER_MODEL` argument to the receiver decorator. This is because we are not presuming that the `auth` user model will be employed in the app.

One slight disadvantage of a 1-to-1 extension of the `User` class is that by using two models for user and profile data, we no longer have a single place in the admin app to maintain this data.

Subclassing `AbstractUser`

There are situations where it is preferable to replace the user in the `auth` application with a more appropriate one. This can include the case where the primary fields used in authentication are different from the username and password, or where we need to override the password mechanism or permission structures.

It is advised to do this early in the development if it is apparent that such a need exists, as we don't want to create complicated migration scenarios if we don't have to.

First, we should look in more detail at how the user model is implemented in the auth app.

The auth app user model is implemented in `django/contrib/auth/models.py`. `User` is a subclass of `AbstractUser`, which in turn subclasses `AbstractBaseUser` (located in `django/contrib/auth/base_user.py`) and `PermissionMixin`.

Figure 10.1 depicts the attributes of these classes and where they come from. It is a useful guide to establishing how to approach overriding the user model without incurring any unnecessary coding complexity.

Before you implement your own alternative user or authentication protocols, it is worth reviewing what existing apps have already been developed for these needs. Many apps to meet common needs, such as OAuth2 client implementations, exist, as well as established Django apps. It is crucial to thoroughly review what options are available before devising your own customization, as adopting an established app will save a lot of unnecessary duplication of effort.

If you do start customizing the `User` subclass, you will be responsible for modifying it when the Authentication module is updated. You will also be responsible for ensuring that new Django security fixes are not blocked by your customizations.

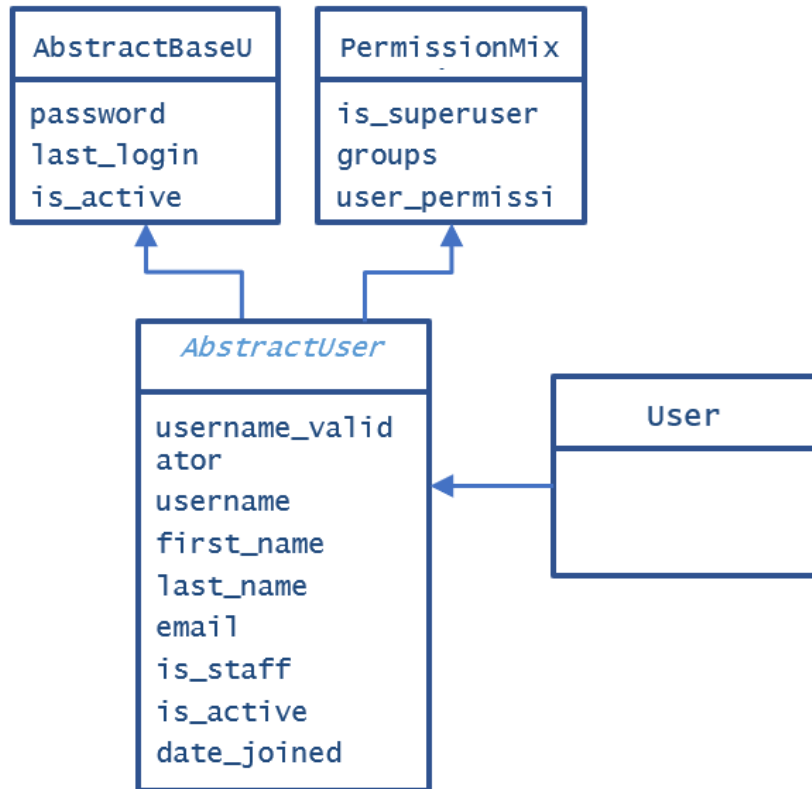


Figure 10.1: The user model's inheritance

Let us look at three different scenarios for subclassing `AbstractUser` or `AbstractBaseUser`.

Scenario 1: Adding additional fields to the User class

In this case, we would subclass `AbstractUser` and include the additional fields.

Sometimes the fields that define the user are insufficient for cultural or professional reasons. In some cultures, the first name/last name distinction is not a natural way to view a person's name. For example, in many parts of Russia and Eastern Europe, a middle name derived from a mother's or a father's first name, a matronym or patronym, respectively, is used. It may make more sense to store this in a separate field in the user model.

Similarly, if your project will be used to generate formal correspondence, it may make sense to store the honorifics Mrs, Ms, Mr, Miss, and so on in a separate honorific field. Professional titles such as Dr, Prof, and Colonel may warrant being stored in a separate field from the honorifics.

If a phone number is used as an essential part of the user registration process and is uniquely tied to the user's identity, it may be worth including it in this class too. Otherwise, consider adding a separate telephone model that is in a many-to-one relationship with the user profile method.

Scenario 2: Replacing the Username key

It is a frequent requirement to implement user authentication that consists of an email address and password, and to dispense with an additional username.

This would involve subclassing `AbstractUser` and replacing the username and username validation fields. New login-related methods may need to be devised.

Scenario 3: Modifying the password

While we can modify the way that the password is stored in the database by changing the middleware, changing the definition of the password field is a different proposition altogether.

If, for some reason, we needed to replace the 128-character password field or replace password authentication with a novel protocol, then it would make sense to override `AbstractBaseUser`. This approach would add a lot of complexity and is not recommended for novices.

Note

The scenarios presented are not mutually exclusive. There may be situations where the best course of action is to include an app-based user extension, such as the `ReviewerProfile` depicted, as well as sub-classing the `AbstractUser`.

Steps in replacing the auth user model

The following list of steps is used to create a custom user model. We will implement these in the next exercise.

1. Create a new app for the custom user model.
2. Define a custom user model class, inheriting from `User`, `AbstractUser`, or `AbstractBaseUser` as the case merits, based on the scenarios outlined previously.
3. Include the app in `INSTALLED_APPS`.
4. Set this model as `AUTH_USER_MODEL` in `<project>/settings.py`.
5. Ensure that any references to the user model are replaced with references to `settings.AUTH_USER_MODEL` or `get_user_model`.
6. Replace any templates or create appropriate extensions that are invalidated by the changes.

7. Register the model with the admin app.
8. Create and run a migration script for these model changes.

Exercise 10.1: Replacing User with a TitledUser model

In this exercise, we will create a `TitledUser` class that includes an honorific and title field for each user.

These options are to be defined as `CharField` with fixed choice lists. The `TitledUser.get_full_name` method will override `AbstractUser.get_full_name`. It will follow using the following logic:

- If a title is present, return a string of "<title> <firstname> <secondname>".
- If an honorific is present, return a string of "<honorific> <firstname> <secondname>".
- Otherwise, return a string of "<firstname> <secondname>".

Create a separate app named `titledusers`.

```
python manage.py startapp titledusers
```

Subclass the User model.

```
class TitledUser(User):
```

Include a Meta class with `verbose_name` and `verbose_name_plural` attributes inside the `TitledUser` class.

```
class Meta:
    verbose_name = "titled user"
    verbose_name_plural = "titled users"
```

Define the `TextChoice` classes for `TitleChoice` and `HonorificChoice` inside the `TitledUser` class.

```
class TitleChoices(models.TextChoices):
    PROF = "PROF", "Prof"
    DR = "DR", "Dr"

class HonorificChoices(models.TextChoices):
    MR = "MR", "Mr"
    MISS = "MISS", "Miss"
```

```
MRS = "MRS", "Mrs"  
MS = "MS", "Ms"
```

Add the two additional fields.

```
title = models.CharField(verbose_name="Title",  
                          blank=True, choices=TitleChoices.choices)  
honorific = models.CharField(  
    verbose_name="Honorific",  
    blank=True, choices=HonorificChoices.choices)
```

Add the `get_full_name` method.

```
def get_full_name(self):  
    if self.title:  
        return f'{self.title} '  
            f'{self.firstname} {self.secondname}'  
    elif self.honorific:  
        return f'{self.honorific} '  
            f'{self.firstname} {self.secondname}'  
    else:  
        return f'{self.firstname} '  
            f'{self.secondname}'
```

After the preceding steps, our model in `titledusers/models.py` will look like this:

```
from django.db import models  
from django.contrib.auth.models import User  
  
class TitledUser(User):  
    class Meta:  
        verbose_name = _('titled user')  
        verbose_name_plural = _('titled users')  
  
    class TitleChoices(models.TextChoices):  
        PROF = "PROF", "Prof"  
        DR = "DR", "Dr"  
  
    class HonorificChoices(models.TextChoices):  
        MR = "MR", "Mr"  
        MISS = "MISS", "Miss"
```

```

MRS = "MRS", "Mrs"
MS = "MS", "MS"

title = models.CharField(verbose_name="Title",
                          blank=True, choices=TitleChoices.choices)
honorific = models.CharField(verbose_name="Honorific",
                              blank=True, choices=HonorificChoices.choices)

def get_full_name(self):
    if self.title:
        return f'{self.title} '
            f'{self.firstname} {self.secondname}'
    elif self.honorific:
        return f'{self.honorific} '
            f'{self.firstname} {self.secondname}'
    else:
        return f'{self.firstname} '
            f'{self.secondname}'

```

Include the new app in the list of installed apps and set `AUTH_USER_MODEL`.

```

INSTALLED_APPS = [
    ...
    'reviews',
    'titledusers',
]
AUTH_USER_MODEL = 'titledusers.TitledUser'

```

Ensure that any references to the user model are replaced with references to `settings.AUTH_USER_MODEL` or `get_user_model`.

A search of the code reveals that these only occur in `reviews/management/commands/loadcsv.py` and the original code for `reviews/models.py`.

Replace any templates or create appropriate extensions that are invalidated by the changes. Currently, since these changes do not affect any username, email, or login behaviors, we do not need to replace any templates.

Register the model with the admin app.

At a minimum, the `titledusers/admin.py` file will look like this:

```
from django.contrib import admin

# Register your models here.
admin.site.register(TitledUser)
```

Create and run a migration script for these model changes.

We have now created a custom user, `TitledUser`.

Activity 10.1: Implementing the reviewer profile model

The aim of this activity is to follow through the necessary steps to implement the `ReviewerProfile` model that was listed in the *Extending the user model* section.

As we are keeping `auth.User` as the auth model, we do not have to create a new user. However, we need to explicitly define this in the project's settings file.

Replace any templates or create appropriate extensions that are invalidated by the changes. At this point, we will add custom admin templates for profile and registration. The existing templates will be modified to include the `ReviewerProfile` form.

1. Register the model with the admin app in `admin.py` by importing the model and then registering it with `admin.site.register(ReviewerProfile)`.
2. Create and run a migration script for these changes.
3. The outcome of this activity is that we have integrated our `ReviewerProfile` model into the Bookr project, and we can modify `ReviewerProfiles` through the admin interface.

The screenshot shows the 'Bookr administration' interface. At the top, there's a dark blue header with the title 'Bookr administration' and links for 'WELCOME, BOOKRADMIN', 'VIEW SITE / CHANGE PASSWORD / LOG OUT'. Below this is a breadcrumb trail: 'Home > Reviews > Reviewer profiles > Add reviewer profile'. The main content area is titled 'Add reviewer profile'. It contains several form fields: 'User:' with a dropdown menu showing 'bob' and edit/delete icons; 'Location:' with an empty text input; 'Favourite book:' with a dropdown menu and edit/delete icons, with a note below it saying 'Your favourite book (if it is present in Bookr)'; and 'Profile photo:' with a 'Choose file' button and the text 'No file chosen'. At the bottom right, there are three buttons: 'Save and add another', 'Save and continue editing', and 'SAVE'.

Figure 10.2: Editing the reviewer profile in the admin app

Form enhancements with ModelAdmin attributes

This section addresses some of the additional attributes of the `ModelAdmin` class. We first encountered the `ModelAdmin` class in *Chapter 4* and learned how to subclass it to produce customizations in our admin app.

`filter_horizontal` / `filter_vertical`

Recall from the user change page that there was a widget consisting of two lists used for assigning permissions to the user. This kind of multiple selection is easier to use when navigating long lists of options than a typical dropdown. As the selected options shift from the left-hand list to the right-hand list, it is easy to keep track of which options have been selected.

If we want to assign this kind of widget to any of our multiple selection fields, we can set the `filter_horizontal` attribute in the `ModelAdmin` subclass. The `filter_horizontal` attribute takes a tuple of strings as arguments. The strings are references to fields in the model or fields in foreign keys.

There is also an option to display the two lists on top of each other.

If we don't want to display the two lists side by side, we have the option of displaying them as top and bottom. This might be the case if we want the selection to be displayed on a mobile phone. In this case, we add the field string to the `filter_vertical` attribute.

Inline forms

Sometimes we also want to represent models that are associated through a ManyToMany or OneToMany relationship.

For example, on the user page, we have a list of groups that the user is in and the option to assign them to additional groups. This is achieved using the `inlines` attribute.

There are two formats of inline form elements, `TabularInline` and `StackedInline`.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0

class BookAdmin(admin.ModelAdmin):
    date_hierarchy = 'publication_date'
    list_display = ('title', 'isbn')
    list_filter = ('publisher', 'publication_date')
    search_fields = ('title', 'isbn', 'publisher__name')
    inlines = (ContributorInline,)
```

Change book

HISTORY

Title:1984

The title of the book.

Date the book was published:2017-04-04Today

Note: You are 11 hours ahead of server time.

ISBN number of the book.9781328869333

Publisher:Houghton Mifflin Harcourt

Cover:Choose fileNo file chosen

Sample:Choose fileNo file chosen

BOOK CONTRIBUTORS

Book contributor: Orwell, G (AUTHOR)

Delete

Contributor:Orwell, G

The role this contributor had in the book.Author

Book contributor: #2

Contributor:

The role this contributor had in the book.

Book contributor: #3

Contributor:

The role this contributor had in the book.

Book contributor: #4

Contributor:

The role this contributor had in the book.

+ Add another Book contributor

Delete

Save and add another

Save and continue editing

SAVE

Figure 10.3: StackedInline forms

Perhaps having those blank extra inline forms looks too busy and confusing. They can be removed using the `extra` attribute.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0
```

Figure 10.4: StackedInline forms without blank forms

Alternatively, if we want the parent form to enforce a maximum number of inline forms, we can add the `max_num` attribute. Observe that there will no longer be + Add another Book contributor at the bottom of the inline forms. Of course, setting a maximum number of contributors would not make sense for books, given that they can have multiple authors, editors, and other collaborators, but there are situations where this form constraint is perfectly reasonable.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0
    max_num = 2
```

BOOK CONTRIBUTORS

Book contributor: Straub, P (CO_AUTHOR) ☐ Delete

Contributor:

The role this contributor had in the book.

Book contributor: King, S (CO_AUTHOR) ☐ Delete

Contributor:

The role this contributor had in the book.

Figure 10.5: StackedInline forms with a maximum constraint

This StackedInline format emphasizes that the inline elements are separate elements. Sometimes, we might prefer the TabularInline rendering, which is more compact and arranges fields in columns. This is particularly useful for numerical data such as invoice lines.

```
class ContributorInline(admin.TabularInline):
    model = BookContributor
    extra = 0
```

BOOK CONTRIBUTORS

CONTRIBUTOR	THE ROLE THIS CONTRIBUTOR HAD IN THE BOOK.	DELETE?
Straub, P (CO_AUTHOR)	Co-Author	☐
King, S (CO_AUTHOR)	Co-Author	☐

Add another Book contributor

Figure 10.6: TabularInline forms

Be mindful when adding inline (or `filter_horizontal` / `filter_vertical`) fields to an admin form, as this will increase the underlying SQL query complexity used to generate the page. It may have performance repercussions if you have large datasets.

radio_fields

By default, the admin app renders choices or foreign key relationships as dropdowns. Sometimes radio buttons are an appropriate way to represent a short list of discrete options. By using the `radio_fields` attribute, we render some fields as radio buttons.

We might consider rendering the `role` attribute of our `BookContributor` class using `radio_fields`. We need to specify `radio_fields` as a dictionary, specifying the field name as the key, and the orientation (either `admin.HORIZONTAL` or `admin.VERTICAL`) as the value.

```
class BookContributorAdmin(admin.ModelAdmin):
    role = {'rating': admin.HORIZONTAL}

admin.site.register(BookContributor, BookContributorAdmin)
```

This setting works well when there is a small list of options. Otherwise, the layout can look somewhat confusing or cluttered.

readonly_fields

Sometimes we want to protect specific fields from being modified in the admin interfaces. We can designate them as being read-only by using the `readonly_fields` attribute. For example, you might want to set the fields showing the date of creation and date of modification of a record as read-only. The following line of code shows how this can be done using `readonly_fields`.

```
readonly_fields = ('date_created', 'date_modified')
```

For example, in this situation, we have specified a tuple of read-only fields, but there are some fields that are only read-only when the object is being modified. Using the conditional is a common way to achieve this, and we will revisit this logic in the exercises and activities.

get_readonly_fields

Sometimes we need to assign more dynamic criteria to specify when a field is read-only. For this reason, there is a `get_readonly_fields` method that gives us the flexibility to include criteria such as a user's permissions or whether the object is being created or updated.

Let's suppose we only wanted the creator and book fields of the review to be editable upon creation, but read-only fields when the review is being edited. We could achieve this by specifying an `if obj is None` condition.

```
def get_readonly_fields(self, request, obj=None):
    if obj is None or request.user.is_superuser:
        return self.readonly_fields
    return ['creator', 'book',] + self.readonly_fields
```

list_editable

There are occasions when you view a list of objects when it would be handy to be able to edit the display fields on the change list without having to edit a change page for each record.

Fortunately, the `ModelAdmin` object provides an option for this. We can assign fields to the `list_editable` attribute.

Exercise 10.2

In this exercise, we will use some of the `ModelAdmin` attributes that we have learned about to carry out some fine-grained customizations of the admin interface. In our modification to the `Review` model, reviews can be assigned a star ranking. We would like to modify the display of reviews in their respective admin form so that they appear as vertical radio fields.

Include the rating in `ReviewsAdmin list_display`. Note that the `CharFields` choices representation appears in `list_display`.

Add a method called `has_profile_photo` to `ReviewerProfile`. It should return an empty string if no profile photo has been uploaded or an appropriate emoji if one is available. We might use a camera, painting palette, or framed picture emoji (Unicode U+1F5BC, HTML 🖼️).

In order to display formatted HTML in `list_display` rather than escaping it, we need to use the `format_html` function in `reviews/admin.py`.

```
from django.utils.html import format_html
```

Create a `ReviewerProfileAdmin` class in `reviews/admin.py`, then adjust the model registration to include the custom admin class:

```
def has_profile_photo(self, obj):
    return obj.profile_photo and format_html(
        '<h1>🖼️ </h1>') or ''
```

Change lists on reviewer profiles should display username, location, and has_profile_photo.

Make the location field of ReviewerProfile editable from an admin display page.

The reviews/admin.py file should now look like this:

```
from django.utils.html import format_html

class ReviewerProfileAdmin(admin.ModelAdmin):
    list_display = ('user', 'location',
                   'has_profile_photo')
    list_editable= ('location',)

    def has_profile_photo(self, obj):
        return obj.profile_photo and format_html(
            '<h1>📷</h1>') or ''
    ...
admin.site.register(ReviewerProfile, ReviewerProfileAdmin)
```

Now, once you have created some reviewer profiles, the change list will look something like *Figure 10.7*.

☐ USER	LOCATION	HAS PROFILE PHOTO
☐ marksandler@test.com	Bukit Merah, Singapore	
☐ alice	Melbourne, Australia	
☐ bob	Shanghai, China	

3 reviewer profiles Save

Figure 10.7: The reviewer profile change list

We've looked at how we can enhance form layout in the admin interface. Now we will examine pagination, search, and filter customizations.

Pagination customizations

So far, we have been dealing with relatively small datasets in our example, but as our new data is accumulated, we have to render more and more information on our pages. Fortunately, the admin app implements options for pagination so that we can return small segments of retrieved data at a time. This is necessary for performance, as large query sets may lead to database performance issues and will slow down web server responsiveness.

By default, records are paginated into sets of 100. For many applications, we would prefer to have a smaller set that doesn't require page scrolling to navigate.

We can alter this by using the `list_per_page` attribute of the `ModelAdmin` class.

```
class ReviewAdmin(admin.ModelAdmin):
    list_display = ('creator', 'book', 'rating')
    list_per_page = 10
```

We can see in *Figure 10.8* that records are now paginated into 10 objects per page.

(Try this with 5 items.)

Action:		Go	0 of 10 selected
☐	CREATOR	BOOK	RATING
☐	david	Paul Clifford (9781719053167)	2
☐	david	Fahrenheit 451 (9781451673319)	2
☐	david	The Catcher in the Rye (9780316769174)	3
☐	david	The Great Gatsby (9780743273565)	2
☐	david	To Kill A Mocking Bird (9780060935467)	3
☐	david	Brave New World (9780060850524)	3
☐	david	Deep Reinforcement Learning Hands-On (9781788834247)	3
☐	david	Architects of Intelligence (9781789954531)	3
☐	david	Hands-On Machine Learning for Algorithmic Trading (9781789346411)	4
☐	carol	Animal Farm: A Fairy Story (9780151002177)	5
1	2	3	4
40 reviews Show all			

Figure 10.8: The ratings result frame

At the bottom of the result set, there is a Show all link. Clicking this link will return all records without pagination set. By default, the Show all link will only appear if there are 200 or fewer records to be returned. By using the `list_max_show_all` attribute, we can alter this number.

Search and filter customizations

We reviewed Django's search filter capability in *Chapter 4*. There are additional options available for customizing the filter's performance and appearance.

By default, when we retrieve a result set from a filter operation, the result set record count is displayed. What might not be apparent is that in order to display this number, Django has to make a secondary database query after the primary query that retrieves the dataset.

As pagination is an efficient way to retrieve a small portion of the results at a time, this second query that returns the record count can often be the one that creates performance issues for the site, especially with large record sets or datasets that involve multiple relations with other models. By default, `show_full_result_count=True`. If you are finding that Django is taking too long to retrieve and render your filtered result set, it may be worth setting `show_full_result_count=False` and seeing if this improves the filter's performance.

By default, after we edit an object that we retrieve from a filtered result set, we are returned to the filter set. This is usually the desired behavior, though sometimes it is a more natural workflow to return to the model index page without any filter settings. If this is the case, it is appropriate to specify the `preserve_filters=False` option in our `ModelAdmin` subclass.

These behaviors will work the same if the result set is derived from a search bar query or from a filter selection.

```
class ReviewAdmin(admin.ModelAdmin):
    ...
    list_filter = ('content', 'creator__username',
                  'book__title')
    search_fields = ('creator__username',)
    show_full_result_count = False
    preserve_filters = False
```

Activity 10.2

This activity will consolidate our knowledge of pagination, search, and filter customizations.

1. Standardize the pagination on the Reviews pages so that all objects are returned with a maximum of 10 records.
2. Remove the Show all link for result sets that have more than 50 records.
3. Add a search bar for the Review admin page. Allow searching on Book title, Creator username, and Review content.

4. Add a method to ReviewAdmin called `rating_stars`. It will return a number of stars (Unicode character U+2606) based on the value of rating. Use this method for display formatting of review ratings.
5. Add a filter on the review admin page for review rating.
6. Because the Review model has foreign key relations with Book and Contributor, the structure of the result count query may well cause performance issues for large query sets. Modify ModelAdmin for the Review class so that the full result count is not displayed.
7. By completing this activity, we have completed our customizations of the reviews app admin pages.

The modified change list page for Reviews will look like *Figure 10.9*.



Figure 10.9: Search and filters on the review change list

Adding views to the admin site

Just like general applications inside Django, which can have multiple views associated with them, Django allows developers to add custom views to the admin site, also, which allows a developer to increase the scope of what the admin site interface can do.

The ability to add your own views to the admin site provides a lot of extensibility to the admin panel of the website, which can be leveraged for several additional use cases. For example, as we discussed at the start of the chapter, an IT team of a big organization can add a custom view to the admin site, which can then be used to monitor the health of the different IT systems at the organization, as well as to provide the IT team with the ability to quickly look at any urgent alerts that need to be addressed.

Now, the next question we need to answer is, *How can we add a custom view to the admin site?*

As it turns out, adding a new view inside the admin template is quite easy and follows the same approach we used while creating views for our applications, with some minor modifications. So, let's look at how we can add a new view to our Django admin dashboard.

Creating the view function

The first step to adding a new view to the Django application is to create a view function that implements the logic to handle the view. In the previous chapters, we created view functions inside a separate file known as `views.py`, which was used to hold all our method- and class-based views.

In the context of adding a new view to the Django admin dashboard, to create a new view, we need to define a new view function inside our custom `AdminSite` class. For example, to add a new view that renders a page showing the health of the different IT systems inside an organization, we can create a new view function named `system_health_dashboard()` inside our custom `AdminSite` class implementation, as shown in the following code snippet:

```
class SysAdminSite(admin.AdminSite):
    def system_health_dashboard(self, request):
        # View function logic
```

Inside the view function, we can perform any operations we want in order to generate a view and, finally, use that response to render a template. Inside this view function, there are some important pieces of logic we need to make sure are implemented correctly.

The first one is to set the `current_app` property for the request field inside the view function. This is required in order to allow Django's URL resolver inside the templates to correctly resolve the view functions for an application. To set this value inside the custom view function we just created, we need to set the `current_app` property, as shown in the following code snippet:

```
request.current_app = self.name
```

The `self.name` field is automatically populated by Django's `AdminSite` class, and we don't need to initialize it explicitly. With this, our minimal custom view implementation will appear, as shown in the following code snippet:

```
class SysAdminSite(admin.AdminSite):
    def system_health_dashboard(self, request):
        request.current_app = self.name
        # View function logic
```

Accessing common template variables

When creating a custom view function, we might want access to the common template variables, such as `site_header` and `site_title`, in order to render them correctly in the template associated with our view function. As it turns out, this is quite easy to achieve with the use of the `each_context()` method provided by the `AdminSite` class.

The `each_context()` method of the `AdminSite` class takes a single parameter, `request`, which is the current request context, and returns the template variables that are to be inserted in all the admin site templates.

For example, if we wanted to access the template variables inside our custom view function, we could implement code similar to the following code snippet:

```
def system_health_dashboard(self, request):
    request.current_app = self.name
    context = self.each_context(request)
    # view function logic
```

The value returned by the `each_context()` method is a dictionary containing the name of the variable and the associated value.

Mapping URLs for the custom view

Once the view function is defined, the next step involves mapping this view function to a URL so that a user can access it or allow the other views to link to it. For the views defined inside `AdminSite`, this URL mapping to views is controlled by the `get_urls()` method implemented by the `AdminSite` class. The `get_urls()` method returns the `urlpatterns` list that maps to the `AdminSite` views.

If we want to add a URL mapping for our custom view, the preferred approach includes overriding the implementation of `get_urls()` in our custom `AdminSite` class and adding the URL mapping there. This approach is demonstrated in the following code snippet:

```
class SysAdminSite(admin.AdminSite):
    def get_urls(self):
        base_urls = super().get_urls(). # Get the existing
                                         set of URLs
        # Define our URL patterns for custom views
        urlpatterns = [
            path("health_dashboard/",
                self.system_health_dashboard)
```

```
]
    return base_urls + urlpatterns. # Return the
                                    updated mapping
```

The `get_urls()` method is generally called automatically by Django, and there is no need to perform any manual processing on it.

Once this is done, the last step involves making sure that our custom admin view is only accessible through the admin site, and non-admin users should not be able to access it. Let's take a look at how that can be achieved.

Restricting custom views to the admin site

If you followed all the previous headings thoroughly, you will now have a custom `AdminSite` view ready for use. However, there is a small glitch. This view is also directly accessible to any user who is not on the admin site.

To ensure that such a situation does not arise, we need to restrict this view to the admin site. This can be achieved quite simply by wrapping our URL path inside the `admin_view()` call, as shown in the following code snippet:

```
urlpatterns = [
    self.admin_view(path("health_dashboard/",
                        self.system_health_dashboard))
]
```

The `admin_view` function makes sure the path provided to it is restricted just to the admin dashboard and that no non-admin-privilege user can access it.

Now, let's add a new custom view to our admin site.

Exercise 10.03 – adding custom views to the admin site

In this exercise, you will add a custom view to the admin site, which will render a user profile and will show the user the options to modify their email and add a new profile picture. To build this custom view, follow the steps described:

1. Open the `admin.py` file under the `bookr_admin` directory and add the following imports. These will be required to build our custom view inside the admin site application:

```
from django.template.response import TemplateResponse
from django.urls import path
```

2. Open the `admin.py` file under the `bookr_admin` directory and create a new method named `profile_view`, which takes in a request variable as its parameter, inside the `BookrAdmin` class:

```
def profile_view(self, request):
```

Next, inside the method, get the name of the current application and set that in the request context. For this, you can use the `name` property of the class, which is auto-populated by Django. To get this property and set it in your request context, you need to add the following line:

```
request.current_app = self.name
```

Once you have the application name populated in the request context, the next step is to fetch the template variables, which are required to render the contents, such as `site_title` and `site_header`, in the admin templates. For this, leverage the `each_context()` method of the `AdminSite` class, which provides the dictionary of the admin site template variables from the class:

```
context = self.each_context(request)
```

Once you have the data in place, the last step is to return a `TemplateResponse` object, which will render the custom profile template when someone visits the URL endpoint mapped to your custom view:

```
return TemplateResponse(request,  
    "admin/admin_profile.html", context)
```

3. With the view function now created, the next step is to make `AdminSite` return the URLs mapping the view to a path inside `AdminSite`. To do this, you need to create a new method with the name `get_urls()`, which overrides the `AdminSite.get_urls()` method and returns the mapping of your new view. This can be done by first creating a new method named `get_urls()` inside the `BookrAdmin` class you created for your custom admin site:

```
def get_urls(self):
```

Inside this method, the first thing you need to do is to get the list of the URLs that are already mapped to the admin endpoint. This is a required step; otherwise, your custom

admin site will not be able to load any results associated with the model editing pages, the logout page, and so on if this mapping is lost. To get this mapping, call the `get_urls()` method of the base class from which the `BookrAdmin` class is derived:

```
urls = super().get_urls()
```

Once the URLs from the base class are captured, the next step is to create a list of URLs that map our custom view to a URL endpoint in the admin site. To do this, we will create a new list named `url_patterns` and map our `profile_view` method to the `admin_profile` endpoint. This is done by using the `path` utility function from Django, which allows us to map the view function with a string-based API endpoint path:

```
url_patterns = [  
    path("admin_profile", self.profile_view)  
]  
return url_patterns + urls
```

Save the `admin.py` file. It should look like `Exercise10.03/bookr/admin.py` in the `Chapter10` folder in this book's GitHub repo. .

Note that the value returned by `get_urls()` is a non-modifiable list. In the preceding code snippet, if we try to switch the order between the merging of the `url_patterns` and `urls` objects, the method won't work, and the `admin_profile` URL won't be created.

4. Now, with the `BookrAdmin` class configured for the new view, the next step is to create your template for the admin profile page. To do this, create a new file named `admin_profile.html` under the `templates/admin` directory of your project root. Inside this file, first, add an `extends` tag to make sure that you are extending from the default admin template:

```
{% extends "admin/index.html" %}
```

This step ensures that all of your admin template style sheets and HTML are available for use inside your custom view template. For example, without having this `extends` tag, your custom view will not show any specific content already mapped to the admin site, such as `site_header`, `site_title`, or any links to log out or go to another page.

Once the `extends` tag is added, add a `block` tag and provide it with the value of content. This makes sure that the code you add between the pair of `{% block content %}...{%`

`endblock %}` segments overrides whatever value is present in the `index.html` template that comes prepackaged with the Django admin module:

```
{% block content %}
```

Inside the `block` tag, add the HTML required to render the profile view that was created in *step 2* of this exercise:

```
<p>Welcome to your profile, {{ username }}</p>
<p>You can do the following operations</p>
<ul>
  <li><a href="#">Change E-Mail Address</a></li>
  <li><a href="#">Add Profile Picture</a></li>
</ul>
{% endblock %}
```

5. Now, with the preceding steps complete, reload your application server by running `python manage.py runserver localhost:8000` and then visiting `http://localhost:8000/admin/admin_profile`.

When the page opens, you can expect to see something like the following screenshot:

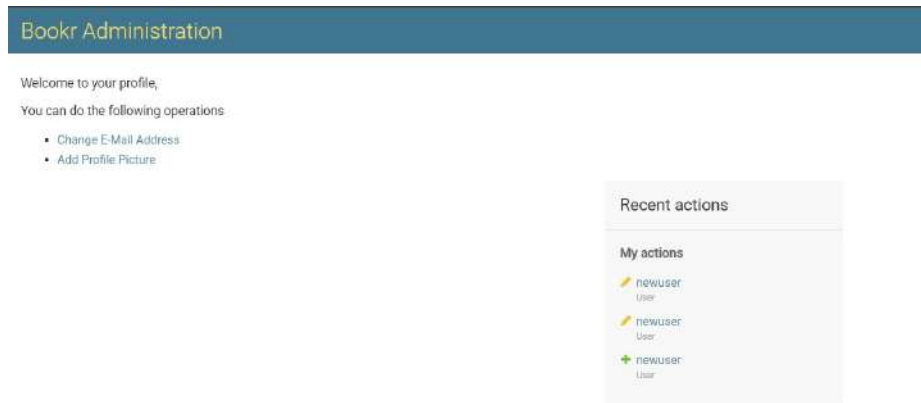


Figure 10.10: The profile page view on the Bookr Administration site

Note

The view created so far will render just fine, irrespective of whether a user is logged into the admin application.

To make sure that this view is only accessible to the logged-in admins, you need to make a small modification inside your `get_urls()` method, which you defined in *step 2* of this exercise.

Inside the `get_urls()` method, modify the `url_patterns` list to look something like the one shown here:

```
url_patterns = [  
    path("admin_profile",  
        self.admin_view(self.profile_view)),  
]
```

In the preceding code, you wrapped your `profile_view` method inside the `admin_view()` method.

The `AdminSite.admin_view()` method causes the view to be restricted to those users who are logged in. If a user who is currently not logged into the admin site tries to visit the URL directly, they will be redirected to the login page, and only in the event of a successful login will they be allowed to see the contents of our custom page.

During this exercise, we leveraged our existing understanding of writing views for Django applications, merging it with the context of the `AdminSite` class to build a custom view for our admin dashboard. With this knowledge, we can now move on and add useful functionalities to our Django admin site to supercharge its usefulness.

Passing additional keys to templates using template variables

Inside the admin site, the variable values passed to the templates are done so by using template variables. These template variables are prepared and returned by the `AdminSite.each_context()` method.

Now, if there is a value that you want to pass to all the templates of your admin site, you can override the `AdminSite.each_context()` method and add the required fields to the request context. Let's look at an example to see how we can achieve this outcome.

Consider the username field, which we passed to our `admin_profile` template earlier. If we want to pass it to every template inside our custom admin site, we first need to override the `each_context()` method inside our `BookrAdmin` class, as shown here:

```
def each_context(self, request):
    context = super().each_context(request)
    context['username'] = request.user.username
    return context
```

The `each_context()` method takes a single argument (we're not considering `self` here) of the `HttpRequest` type, which it uses to evaluate certain other values.

Now, inside our overridden `each_context()` method, we first make a call to the base class `each_context()` method to retrieve the context dictionary for the admin site:

```
context = super().each_context(request)
```

Once that is done, the next thing to do is to add our username field to context and set its value to the value of the `request.user.username` field:

```
context['username'] = request.user.username
```

Once this is done, the last thing that remains is to return this modified context.

Now, whenever a template is rendered by our custom admin site, the template will be passed with this additional username variable.

Configuring an email server with Django

The Django Admin app contains powerful features that enhance the rapid and robust development of applications. It also provides a mechanism to deal with forgotten passwords by emailing the user a secure link to reset the password.

In order to implement this email functionality in Django, we need to configure an SMTP server in the Django bookr/`settings.py` file.

It is recommended to install the `python-decouple` module so that sensitive mail server account details are not saved in the bookr/`settings.py` file. We can install it as follows.

```
pip install python-decouple
```

Now we can add appropriate SMTP settings to the `<projectname>/setting.py` file. The settings that are applicable for SMTP configuration are `EMAIL_BACKEND`, `EMAIL_HOST`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_PORT`, `EMAIL_USE_TLS`, and `EMAIL_USE_SSL`.

The following are the current settings that work with a Gmail account. Google accounts have recently stopped third-party apps from accepting email account passwords. Instead, they have introduced a system of generating "application passwords." Security policies are prone to frequent changes, but at the time of writing, application passwords can be generated for accounts with two-step verification enabled by visiting this link: <https://myaccount.google.com/apppasswords>.

```
from decouple import config

EMAIL_BACKEND = \
    'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_HOST_USER = config('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD')
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_USE_SSL = False
```

These changes to the `settings.py` file include calls to the `decouple.config` function. So, rather than including the email account username and password in the `settings.py` file (which might be included in the source code distribution or public repository), we can set these environment variables at runtime or by including them in the top-level project folder as either an `.env` or a `settings.ini` file. Invoking them at runtime could involve putting them in a `.bashrc` or `.profile` file, or defining them at the command line when the Django server is started:

```
EMAIL_HOST_USER=bookr2026 EMAIL_HOST_PASSWORD='vnpw zgkl ixpq pbcz' python
manage.py runserver
```

Alternatively, an `.env` or a `settings.ini` file can be added to the project directory. The `.env` format is more commonly used on macOS and Linux, while the `settings.ini` format is typically used in Windows environments. Decouple can read both formats on all platforms.

`EMAIL_HOST_USER` is set to the username on the SMTP server, and `EMAIL_HOST_PASSWORD` is set to either the email password or the app password (in the case of Gmail SMTP and other security-conscious email providers).

.env

```
EMAIL_HOST_USER=bookr2026
EMAIL_HOST_PASSWORD=vnpw zgkl ixpq pbcz
```

The settings file is very similar to the .env format, but requires a tab.

settings.ini

```
[settings]
EMAIL_HOST_USER=bookr2026
EMAIL_HOST_PASSWORD=vnpw zgkl ixpq pbcz
```

Other SMTP servers will have different configuration settings.

Once these settings are added, the Django apps are able to use the email server to send emails to users.

Django provides functions such as `send_mail` and `send_mass_mail`, which can be used to send emails from within a Django app. You can read more about this functionality in the Django documentation at <https://docs.djangoproject.com/en/6.0/topics/email/> and learn by doing in the upcoming *Exercise 10.05*.

Automating admin with Django Tasks

Many jobs that we need to repeat in a web application do not suit the limitations of the HTTP request and response cycle. We do not want to limit a web server's capacity by keeping HTTP connections open while lengthy database updates or batches of network activity are occurring. There are many automated tasks involved in supporting a web application that are best carried out without the overhead of web interactions.

Until recently, these would typically have been implemented as cron jobs on Linux or Mac, or as scheduled tasks on Windows or using a standalone scheduling and task processor such as Celery.

Django 6.0 introduces the Django Tasks framework. Its functionality is somewhat limited at present, but it offers us a way to maintain scheduling tasks within a familiar Django paradigm.

The Django Tasks framework provides a mechanism to define units of work and their scheduling parameters, along with a mechanism to enqueue them. The API defines an interface to backends that actually carry the work.

A task can be defined in a Django application using the task decorator. This defines the properties of the task, including its priority, the queue that it will be scheduled in, and the backend that will be delegated to run the task.

If we had tasks to schedule that fell into groups such as AI image generation, database transformations, email batching, and data backup, it would make sense to configure a separate task queue for each of these task groups as a means of allocating resources. In the example and exercise that follow, we will only use the default queue and a trivial backend, but more complicated setups would be appropriate with a significant production deployment.

The Django Tasks application comes with two trivial backends: `ImmediateBackend` and `DummyBackend`. `ImmediateBackend` operates by running tasks as soon as they are enqueued, while `DummyBackend` returns a task result when enqueued without any work being passed to a worker. `DummyBackend` is useful for including in tests of a Django application.

The backend is specified in the `<project>/settings.py` file. A simple configuration with `ImmediateBackend` will look like this:

```
TASKS = {
    "default": {
        "BACKEND": "django.tasks.backends.immediate."
                    "ImmediateBackend"
    }
}
```

`DummyBackend` can be configured by specifying `"django.tasks.backends.dummy.DummyBackend"`.

Tasks are typically defined inside the app folder, in a file called `tasks.py`.

A task is defined by using the `@task` decorator. The decorator doesn't require any arguments, but has the optional arguments of:

- **priority:** An integer value that can be used by backends to determine the scheduling priority of the task, with a default of 0.
- **queue_name:** A string corresponding to a queue defined in the Tasks specification. The default value is "default".

- **backend**: A string corresponding to a backend defined in the Tasks specification. The default value is "default".
- **takes_context**: A Boolean value specifying whether the first argument to the decorated task function is a TaskContext object. TaskContext has two attributes defined, the attempt number and task_result. The default is False.

Here is an example of a task that is used for generating an image and writing it to the file system.

myapp/tasks.py

```
from django.tasks import task
from .img_utils import _generate_cake_jpg

@task(priority=3, queue_name="img_generator")
def generate_birthday_greeting_jpg(user, background_img,
    favourite_cake):
    img = _generate_cake_jpg(user, background_img,
        favourite_cake)
    filename = f"media/{user.username}/birthday.jpg"
    img.save(filename)
```

The task will be scheduled to run by calling the enqueue method of the task. The enqueue method receives the arguments of the decorated task method.

```
>>> from myapp.tasks import generate_birthday_greeting_jpg
>>> generate_birthday_greeting_jpg.enqueue(user,
... "media/background/hundreds_and_thousands.jpg",
... "tiramisu")
```

Once the task runs, it will produce a TaskResult that will look something like that displayed in *Figure 10.12*.

```
TaskResult(task=Task(priority=3, func=<function generate_birthday_greeting_jpg at 0x7d6cc5df3b00>, backend='default', queue_name='img_generator', run_after=None, takes_context=False), id='CoAhMGuktRkrYTciUqy5FoBWu6KAhp8R', status=TaskResultStatus.SUCCESSFUL, enqueued_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154716, tzinfo=datetime.timezone.utc), started_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154860, tzinfo=datetime.timezone.utc), finished_at=datetime.datetime(2026, 1, 29, 13, 25, 6, 288324, tzinfo=datetime.timezone.utc), last_attempted_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154860, tzinfo=datetime.timezone.utc), args=[<User: bob>, "media/background/hundreds_and_thousands.jpg", "tiramisu"], kwargs={}, backend='default', errors=[], worker_ids=['4mUrIPeXMGeZgU4l6AHUrgd5nbQEmK2m'], _return_value=1)
```

Figure 10.12: The TaskResult output

With a brief explanation of the Django Tasks framework, we will develop a functional task in the concluding exercise of this chapter.

Activity 10.03 – building a custom admin dashboard with a built-in search

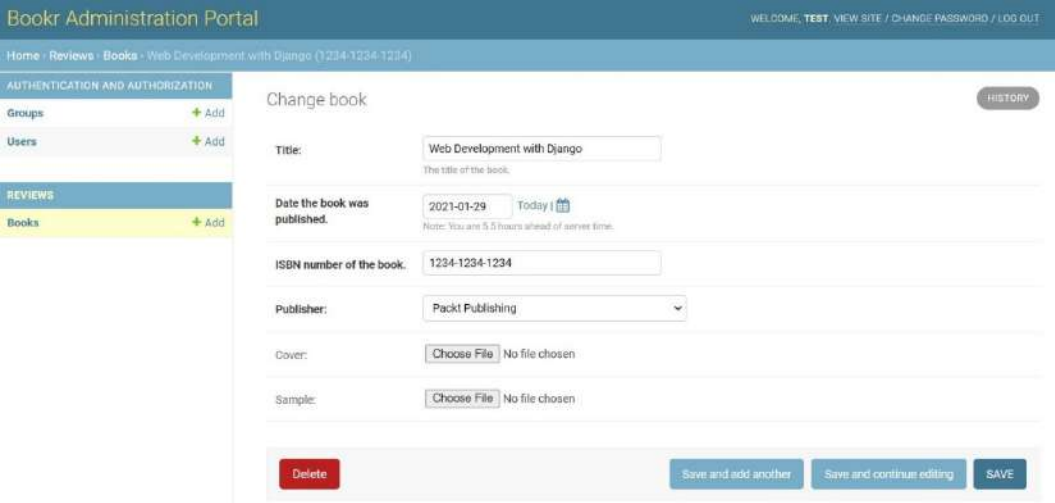
In this activity, you will use the knowledge gained about the different aspects of creating a custom admin site to build a custom admin dashboard for Bookr. Inside this dashboard, you will introduce the capability of allowing a user to search for books by using either the name of the book or the name of the book publisher, and allowing the user to modify or delete these book records.

The following steps will help you build a custom admin dashboard and add the ability to search a book record by using the name of the publisher:

1. Create a new application inside the Bookr project named `bookr_admin`, if not created already. This is going to store the logic for our custom admin site.
2. Inside the `admin.py` file under the `bookr_admin` directory, create a new class, `BookrAdmin`, which inherits from the `AdminSite` class of Django's admin module.
3. Inside the newly created `BookrAdmin` class from *step 2*, add any customizations for the site title or any other branding component of the admin dashboard.
4. Inside the `apps.py` file under the `bookr_admin` directory, create a new `BookrAdminConfig` class, and inside this new `BookrAdminConfig` class, set the default site attribute to the fully qualified module name for our custom admin site class, `BookrAdmin`.
5. Inside the `settings.py` file of your Django project, add the fully qualified path of the `BookrAdminConfig` class created in *step 4* as the first installed application.

6. To register the Books model from the reviews application inside Bookr, open the `admin.py` file inside the reviews directory and make sure that the Books model is registered to the admin site by using `admin.site.register(ModelClass)`.
7. To allow a search of a book according to the name of the publisher, inside the `admin.py` file of the reviews application, modify the `BookAdmin` class and add to it a property named `search_fields`, which contains `publisher_name` as a field.
8. To get the publisher's name correctly for the `search_fields` property, introduce a new method named `get_publisher` inside the `BookAdmin` class, which will return the name field of the publisher from the Book model.
9. Make sure that the `BookAdmin` class is registered as a `Model` admin class for the Book model inside our Django admin dashboard by using `admin.site.register(Book, BookModel)`.

After completing this activity, once you start the application server and visit `http://localhost:8000/admin` and navigate to the Book model, you should be able to search for books by using the publisher's name and, in the event of a successful search, see a page that resembles the one shown in the following screenshot:



The screenshot displays the 'Bookr Administration Portal' interface. The top navigation bar includes links for 'Home', 'Reviews', 'Books', and 'Web Development with Django (1234-1234-1234)'. The left sidebar contains two main sections: 'AUTHENTICATION AND AUTHORIZATION' with 'Groups' and 'Users' (each with an '+ Add' link), and 'REVIEWS' with 'Books' (with an '+ Add' link). The main content area is titled 'Change book' and features a 'HISTORY' button. The form fields are as follows: 'Title' (text input with 'Web Development with Django' and a tooltip 'The title of the book.'), 'Date the book was published.' (calendar icon with '2021-01-29' and 'Today' button, and a note 'Note: You are 5.5 hours ahead of server time.'), 'ISBN number of the book.' (text input with '1234-1234-1234'), 'Publisher.' (dropdown menu with 'Packt Publishing' selected), 'Cover.' (file upload button 'Choose File' and 'No file chosen'), and 'Sample.' (file upload button 'Choose File' and 'No file chosen'). At the bottom, there are four buttons: 'Delete' (red), 'Save and add another' (blue), 'Save and continue editing' (blue), and 'SAVE' (blue).

Figure 10.11: The book editing page inside the Bookr Administration dashboard

Note

The solution for this activity can be found in the Chapter09 folder in this book's GitHub repo.

Exercise 10.5

In this exercise, we want to use the Django Tasks framework to create a periodic task that sends an email to subscribers telling them about recent reviews on the Bookr site.

We will write this task with some flexibility so that it can be configured to run daily, weekly, monthly, or annually.

First, we will add a function called `get_daterange_dt` to `review/utils.py`. This function will take two parameters: a date string in the `YYYY-mm-dd` format and a string to describe periodicity – daily, weekly, monthly, or annual. The function will require the `monthrange` function from the `calendar` module and the `datetime` and `timedelta` types from the `datetime` module.

```
from calendar import monthrange
from datetime import datetime, timedelta
```

Then we can define the `get_daterange_dt` function, which takes a `start_day` string and a period designator that is limited to the values `'daily'`, `'weekly'`, `'monthly'`, and `'annual'`.

```
def get_daterange_dt(start_day, period):
    assert period in ['daily', 'weekly', 'monthly', 'annual']
    start_day_dt = datetime.strptime(start_day, '%Y-%m-%d')
```

Our logic for calculating the end day of the date range will differ depending on whether we are calculating a day, week, month, or year-long range. Note that there is a `try/except` block that handles issues for leap days.

```
if period == 'daily':
    end_day_dt = start_day_dt
elif period == 'weekly':
    end_day_dt = start_day_dt + timedelta(days=6)
elif period == 'monthly':
    last_day_of_month = monthrange(
        start_day_dt.year, start_day_dt.month)[1]
    end_day_dt = start_day_dt.replace(
        day=last_day_of_month)
elif period == 'annual':
    try:
        end_day_dt = (start_day_dt.replace(
            year=start_day_dt.year+1) -
```

```

        timedelta(days=1))
    except ValueError:
        # catch error caused by 29th
        end_day_dt = start_day_dt.replace(
            year=start_day_dt.year+1,
            month=2, day=28)

```

Having calculated the end day, we should calculate the final microsecond of it, as this will be used in the database query. We return the datetime types `start_date_dt` and `end_day_dt`.

Create a file in the reviews folder called `tasks.py`. We will need to import the `groupby` function from `itertools`, the Django `send_mass_mail` function, the task decorator, and the `render_to_string` function from `django.template.loader`. We also need to import the `Review` model and the `get_daterange_dt` function that we just created.

```

from itertools import groupby
from django.core.mail import send_mass_mail
from django.tasks import task
from django.template.loader import render_to_string

from .models import Review
from .utils import get_daterange_dt

```

Now we can define a function with a task decorator. We are specifying a priority (but this will only be significant if we use an external queuing system with contending tasks) while accepting the default queue and backend. The `review_report` function takes arguments for the following:

- `emails`: A list of email recipients.
- `from_email`: The email address
- `period`: String values – 'daily', 'weekly', 'monthly', 'annual'.
- `start_day`: The start date of the reporting period in the 'YYYY-dd-mm' format.
- `site`: The URL of the Bookr website. The default is 'http://127.0.0.1:8000'.

```

@task(priority=2)
def review_report(emails, from_email, period, start_day,
    site="http://127.0.0.1:8000"):

```

We call the `get_daterange_dt` utility function to derive a datetime start and end from a string-specified date, such as "2026-01-01", and a time period string. Then we create human-readable strings for the start and end day, using the locale-specific time specifier.

```
start_day_dt, end_day_dt = get_daterange_dt(
    start_day, period)
start_day_fmt = start_day_dt.strftime('%x')
end_day_fmt = end_day_dt.strftime('%x')
```

Use the reviews app's model to construct a query of the reviews that have been published in the given timeframe and order them by book title.

```
reviews = Review.objects.filter(
    date_created__range=[start_day_dt, end_day_dt]
).order_by('book__title')
```

Now we can construct a context of appropriate data that we will send to the Django template in order to create the email message. As the data from the Django model is already sorted by book name, we can use the `itertools.groupby` function so that this data is partitioned using the title as a key:

```
[('Title 1', [<Review 2>, <Review 3>]),
 ('Title 2', [<Review 1>]),
 ('Title 3', [<Review 6>, <Review 7>])
 ('Title 5', [<Review 4>, <Review 5>])]
```

The context will look like this:

```
context = dict(period=period, site=site,
    start_day_fmt=start_day_fmt,
    end_day_fmt=end_day_fmt,
    reviews_by_title=[(title, list(reviews))
        for title, reviews in
            groupby(reviews, lambda x: x.book.title)])
```

Next, we can generate a subject line for the email message. We will vary the message depending on whether it relates to reviews from a single day or a longer period of time.

```
subject = f"Your {period} Bookr reviews "
if period=='daily':
```

```

        subject += "for {start_day_fmt}"
    else:
        subject += "from {start_day_fmt} to {end_day_fmt}"

```

Finally, we will create the email message by rendering `review_report_email.txt` to a string. (We will create this template in the next step.) Then we create a list of email messages with one message for each of the emails passed to the function. We then use Django's `send_mass_mail`. Note that if we had deployed the message using the `send_mail` function and a list of email recipients, we would have had a situation where a single email went out with multiple recipients. In most situations, this would be deemed to be a privacy issue as the users had not requested to have their email addresses shared with the other recipients.

```

message = render_to_string(
    'reviews/review_report_email.txt', context)
email_messages = [(subject, message, from_email,
    [email]) for email in emails]
return send_mass_mail(
    email_messages, fail_silently=False)

```

Now this task requires a Django template to render the results. It will follow this structure:

```

{% if reviews_by_title %} <-- header --> {% endif %}
{% for title, reviews in reviews_by_title %}
    <- book title ->
    {% for review in reviews %}
        <- reviewer detail and link to review ->
    {% endfor %}
{% empty %}
    <-- Empty report message -->
{% endfor %}

```

A template for the email message needs be added to the application's templates directory. For the sake of clarity, we are using a text-only message without HTML formatting.

reviews/templates/reviews/review_report_email.txt

```

{% if reviews_by_title %}Here is your {{period}} report of Bookr reviews {% if
period == 'daily' %}on {{start_day_fmt}}. {% else %}from {{start_day_fmt}} to
{{end_day_fmt}}. {% endif %}
{% endif %}
{% for title, reviews in reviews_by_title %}

```

```
{{ title }}
{% for review in reviews %}
* {{ review.creator.username }} - {{ review.rating }}/5 - {{ site }}/reviews/
{{ review.id }}
{% endfor %}
{% empty %}
No reviews
{% if period == 'daily' %}{{ start_day_fmt }}. {% else %} the period
{{ start_day_fmt }} to {{ end_day_fmt }}. {% endif %}
{% endfor %}
```

Now that we have finished developing the task, we can enqueue the task from the command line. This approach will work with `DummyBackend` or `ImmediateBackend`. The `emails` and `from_email` parameters should be the list of recipient emails and the email address configured in the `settings.py` file, respectively. If you aren't getting any data in the current time period, be sure to add your own.

```
>>> from reviews.tasks import review_report
>>> review_report.enqueue(
... ['alice@example.com', 'bob@example.com'],
...   'bookr2026@example.com', 'annual', '2026-01-01',
...   'bookr.example.com')
```

Your email recipients will receive an email similar to the one in *Figure 10.13*.

bookr2026@gmail.com

2:49PM (0 minutes ago)

to me, chrisguest, chrisguest+1 ▼

Here is your annual report of Bookr reviews from 01/01/26 to 12/31/26.

Advanced Deep Learning with Keras 2

- * alice - 4/5 - bookr.example.com/reviews/4
- * david - 4/5 - bookr.example.com/reviews/5

Architects of Intelligence 1

- * david - 5/5 - bookr.example.com/reviews/7

Deep Reinforcement Learning Hands-On 1

- * david - 5/5 - bookr.example.com/reviews/8

Hands-On Machine Learning for Algorithmic Trading 1

- * david - 4/5 - bookr.example.com/reviews/6

Hands-On Reinforcement Learning with Python 1

- * david - 4/5 - bookr.example.com/reviews/10

Natural Language Processing with TensorFlow 1

- * david - 5/5 - bookr.example.com/reviews/9

Figure 10.13: The review report email

Although it might seem like there is a lot of code in this exercise, this is a simple example of a time period report. It is better to handle this as a Django Task rather than as a Django request, as it may take a significant amount of time to generate the report and email a number of recipients, which could lead to web server timeouts during the execution.

Summary

In this chapter, we learned about advanced customization of our Django admin app. We delved deep into features that help us enhance the look and feel of our admin app, making the user interface more intuitive and useful, before moving on to the finer points of pagination and search results filtering. In the next chapter, we will be looking at how we can use templating and class-based views for our Django app.

We also learned how to connect a Django project with an email server and how to develop non-HTTP services with the Django Tasks framework.

As we move on to the next chapter, we will get to build upon what we have learned thus far and extend that knowledge by being introduced to the concept of building our own custom tags and filters for templates, as well as the ability to build our views in an object-oriented style using the concept of class-based views.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

11

Advanced Templating and Class-Based Views

In *Chapter 3, URL Mapping, Views, and Templates*, we learned how to build views and create templates in Django. Then, we learned how to use those views to render the templates we built. In this chapter, we will build upon our knowledge of developing views by using **class-based views**, allowing us to write views that can group logical methods into a single entity. This skill comes in handy when developing a view that maps to multiple HTTP request methods for the same **application programming interface (API)** endpoint. With method-based views, we may end up using a lot of the `if-else` conditions to successfully handle the different types of HTTP request methods. In contrast, class-based views allow us to define separate methods for every HTTP request method we want to handle. Then, based on the type of request received, Django takes care of calling the correct method in the class-based view.

Beyond the ability to build views based on different development techniques, Django also comes packed with a powerful templating engine. This engine allows developers to build reusable templates for their web applications. This reusability of the templating engine is further enhanced by using **template tags**, **filters**, and **template partials**, which help easily implement commonly used features inside templates, features such as iterating over lists of data, formatting the data in a given style, extracting a piece of text from a variable to display, and overriding the content in a specific block of a template. All these features also expand the reusability of a Django template.

As we go through this chapter, we will look at how we can expand the default set of template filters and template tags provided by Django by leveraging Django's ability to define our own custom template tags, filters, and template partials. These custom template tags, filters, and partials can then be used to implement some common features in a reusable fashion across our web application. For example, while building a user profile badge that can be shown in several

places inside a web application, it is better to leverage the ability to write a custom template inclusion tag that just inserts the template of the badge in any of the views we desire, rather than rewriting the entire code for the badge template or by introducing additional complexity to the templates.

In this chapter, we will be covering the following topics:

- Template filters
- Custom template filters
- String filters
- Template tags
- Template partials
- Django views
- Class-based views

Technical requirements

Find the solution in the Chapter11 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Template filters

While developing templates, developers often just want to change the value of a template variable before rendering it to the user. For example, consider that we are building a profile page for a Bookr user. There, we want to show the number of books the user has read. Below that, we also want to show a table listing the books they have read.

To achieve this, we can pass two separate variables from our view to the HTML template. One can be named `books_read`, which denotes the number of books read by the user. The other can be `book_list`, containing the list of names of the books read by the user, for example:

```
<span class="books_read">You have read {{ books_read }}
books</span>
<ul>
  {% for book in book_list %}
    <li>{{ book }} </li>
  {% endfor %}
</ul>
```

Template filters in Django are simple Python-based functions that accept a variable as an argument (and any additional data in the context of the variable), change its value as per our requirements, and then render the changed value.

Now, the same outcome from writing the previous snippet can also be obtained without the use of two separate variables by using template filters in Django, as follows:

```
<span class="books_read">You have read
  {{ book_list|length }}</span>
<ul>
  {% for book in book_list %}
    <li>{{ book }}</li>
  {% endfor %}
</ul>
```

Here, we used the built-in length filter provided by Django. The use of this filter causes the length of the `book_list` variable to be evaluated and returned, which is then inserted into our HTML template during rendering.

Like length, there are a lot of other template filters that come pre-packaged with Django and that are ready to be used. For example, the lowercase filter converts the text to all lowercase format, the last filter can be used to return the last item in the list, and the `json_script` filter can be used to output a Python object passed to the template as a JSON value wrapped in a `<script>` tag in your template.

Note

You can refer to Django's official documentation for the complete list of template filters offered by Django here: <https://docs.djangoproject.com/en/5.2/ref/templates/builtins/>.

Now, with the knowledge of how to use template filters, let's jump into understanding how we can write our own custom filters.

Custom template filters

Django supplies a lot of useful filters that we can use in our templates while we are working on our projects. But what if someone wants to format a specific piece of text and render it with different fonts? Or, say, someone wants to translate an error code to a user-friendly error message based on the mapping of the error code in the backend. In these cases, predefined filters do not suffice, and we would like to write our own filter that we can reuse across the project.

Luckily, Django supplies an easy-to-use API that we can use to write custom filters. This API provides developers with some useful decorator functions that can be used to quickly register a Python function as a custom template filter. Once a Python function is registered as a custom filter, a developer can start using the function in templates.

An instance of this template library method is required to access the filters. This instance can be created by instantiating the `Library()` class in Django from Django's `template` module, as shown here:

```
from django import template
register = template.Library()
```

Once the instance is created, we can use the filter decorator from the template library instance to register our filters.

Creating custom template filters

There are a couple of steps we need to take to create custom template filters. Let's try to understand what these steps are and how they help us with the creation of a custom template filter in the next subsection.

Setting up the directory for storing template filters

It is important to note that when creating a custom template filter or template tag, we need to put them in the `templatetags` directory under the application directory. This requirement arises because Django is internally configured to look for custom template tags and filters when loading a web application. A failure to name the directory `templatetags` will result in Django not loading the custom template filters and tags created by us.

To create this directory, first, navigate to the application folder inside which you want to create custom template filters, and then run the following command in the terminal:

```
mkdir templatetags
```

Once the directory is created, the next step is to create a new file inside the `templatetags` directory to store the code for our custom filters. This can be done by executing the following command inside the `templatetags` directory:

```
touch custom_filter.py
```

Note

The aforementioned command won't work on Windows. You can, however, navigate to the desired directory and create a new file using Windows Explorer.

Alternatively, this can be done by using the GUI interface provided by PyCharm.

Setting up the template library

Once the file for storing the code for the custom filter is created, we can start working on implementing our custom filter code. For custom filters to work in Django, they need to be registered to Django's template library before they can be used inside templates. To that end, the first step is to set up an instance of the template library, which will be used to register our custom filters. For this, inside the `custom_filters.py` file, we created in the previous section, we first need to import the `template` module from the Django project:

```
from django import template
```

Once the import is resolved, the next step is to create an instance of the template library by adding the following line of code:

```
register = template.Library()
```

The `Library` class from Django's `template` module is implemented as a **singleton** class that returns the same object that is only initialized once at the start of the application.

Once the template library instance is set up, we can proceed with implementing our custom filter.

Implementing the custom filter function

Custom filters inside Django are nothing more than simple Python functions that essentially take the following parameters:

- The value on which the filter is being applied (mandatory)
- Any additional parameters (zero or more) that need to be passed to the filter (optional)

These functions need to be decorated with the `filter` attribute from Django's template library instance to behave as template filters. For example, the generic implementation of a custom filter will look like the following:

```
@register.filter
def my_filter(value, arg):
    # Implementation logic of the filter
```

With this, we have learned how to implement a custom filter that can be used inside templates. In the next section, we will learn how to use these custom filters.

Using custom filters inside templates

Once the filter is created, it's simple to start using it inside our templates. To do that, the filter must first be imported into the template. This can be easily done by adding the following line to the top of the template file:

```
{% load custom_filter %}
```

When Django's templating engine parses the template files, the preceding line is automatically resolved by Django to find the correct module specified under the `templatetags` directory. As a consequence, all the filters mentioned inside the `custom_filter` module are automatically made available inside the template.

Using our custom filter inside the template is as simple as adding the following line:

```
{{ some_value|generic_filter:"arg" }}
```

It is a simple process to create a custom filter inside Django and then use it in our templates. In the next section, let's take a look at another type of filter, namely, string filters, which work solely on string-type values.

String filters

In the general case, when building a custom filter, taking any type of variable argument and returning any type. For example, the `length` filter takes an iterable and returns an integer value. But what if we wanted to restrict our filter to work only with strings and not with any other type of values, such as integers?

We can use the `stringfilter` decorator provided by Django's template library to develop filters that work only on *strings*. When the `stringfilter` decorator is used to register a Python method as a filter in Django, the framework ensures that the value being passed to the filter is

converted to a string before the filter executes. This reduces any potential issues that may arise when non-string values are passed on to our filter.

The steps to implement a **string filter** are similar to the ones we followed for building a custom filter with some minor changes.

Remember the `custom_filter.py` file we created in the *Setting up the directory for storing template filters* section? Let's now add a new Python function inside it that will act as our string filter.

Before we can implement a string filter, we first need to import the `stringfilter` decorator, which demarcates a custom filter function as a string filter. You can add this decorator by adding the following `import` statement inside the `custom_filters.py` file:

```
from django.template.defaultfilters import stringfilter
```

Now, to implement our custom string filter, the following syntax can be used:

```
@register.filter
@stringfilter
def generic_string_filter(value, arg):
    # Logic for string filter implementation
```

With this approach, we can build as many string filters as we want and use them just like any other filter.

Equipped with this knowledge, let's now create our first custom string filter.

Exercise 11.01 – Creating a custom template filter

In this exercise, we will write a custom filter named `link_worldcat`, which returns a URL to the WorldCat site when provided with an ISBN or title string along with an explicit argument to say if the string is either an ISBN or title.

Here's an example:

```
value = "9781727518603"
```

You will apply the following filter to this string:

```
{{ value|link_worldcat:"isbn" }}
```


The output after applying this filter should be as follows:

```
"https://www.worldcat.org/isbn/9781727518603"
```

Alternatively, there' the value was a book title:

```
value = "The Iliad"
```

The filter can be used to construct a book title query at worldcat.org such that:

```
{{ value|link_worldcat:"title" }}
```

would produce:

```
"https://www.worldcat.org/search?q=The+Iliad"
```

Let's get started with the steps:

1. Now, create a new directory named `templatetags` inside your reviews application directory to store the code for your custom template filters. To create the directory, run the following command from inside the `filter_demo` directory from the terminal app or command prompt:

```
mkdir templatetags
```

2. Once the directory is created, create a new file named `link_worldcat_filter.py` inside the `reviews/templatetags` directory, adding the following lines to it:

```
import urllib.parse
from django import template
from django.template.defaultfilters import stringfilter

register = template.Library()
```

The preceding code creates an instance of the Django library that can be used to register our custom filter with Django.

3. Add the following code to implement the `link_worldcat` filter:

```
@register.filter
@stringfilter
```

```
def link_worldcat (value, identifier):
    if not value:
        return ""
    if identifier=='title':
        link = urllib.parse.quote_plus(value)
        return f"https://www.worldcat.org/search?q={link}"
    else:
        return f"https://www.worldcat.org/isbn/{value}"
```

The `link_worldcat` filter takes two arguments; one is `value`, on which the filter was used, and the second is `identifier`, passed from the template to the filter. The filter will create an appropriate link depending on whether `identifier` is specified as `isbn` or `title`.

4. With the custom filter ready, we can revisit the existing book detail template where this filter can be applied. Add the `load link_worldcat_filter` directive at the start of the file after the `extends` directive and before the `block` declaration. Then add new spans that will display the ISBN and use the `link_worldcat` filter to provide a link to the book's WorldCat entry.

`reviews/templates/reviews/book_detail.html`

```
{% extends 'reviews/base.html' %}
{% load link_worldcat_filter %}
{% block content %}
    ...
    <span class="text-info">Publication Date: </span>
    <span>{{ book.publication_date }}</span>
    <br>
    <span class="text-info">ISBN: </span>
    <span><a href="{{ book.isbn|link_worldcat:'isbn' }}">
        {{ book.isbn }}</a></span>
    <br>
    ...
{% endblock %}
```

Likewise, we can add similar spans for ISBN and the WorldCat link in the book list template:

reviews/templates/reviews/book_list.html

```
{% extends 'reviews/base.html' %}
{% load link_worldcat_filter %}
{% block content %}
<ul class="list-group">
  {% for item in book_list %}
  <li class="list-group-item">
    ...
    <span class="text-info">ISBN: </span>
    <span><a
      href="{{ item.book.isbn|link_worldcat:'isbn' }}">
        {{ item.book.isbn }}</a></span>
    <br>
    ...
  </li>
  {% endfor %}
</ul>
{% endblock %}
```

After the changes have been made, the code should resemble that in the bookr project directory in the Chapter11 folder in this book's GitHub repo.

5. To view whether the custom filter works, run the following command:

```
python manage.py runserver localhost:8000
```

Now, navigate to the following page in the browser: <http://localhost:8000/books/2>.

6. This page should appear as shown in *Figure 11.1*:

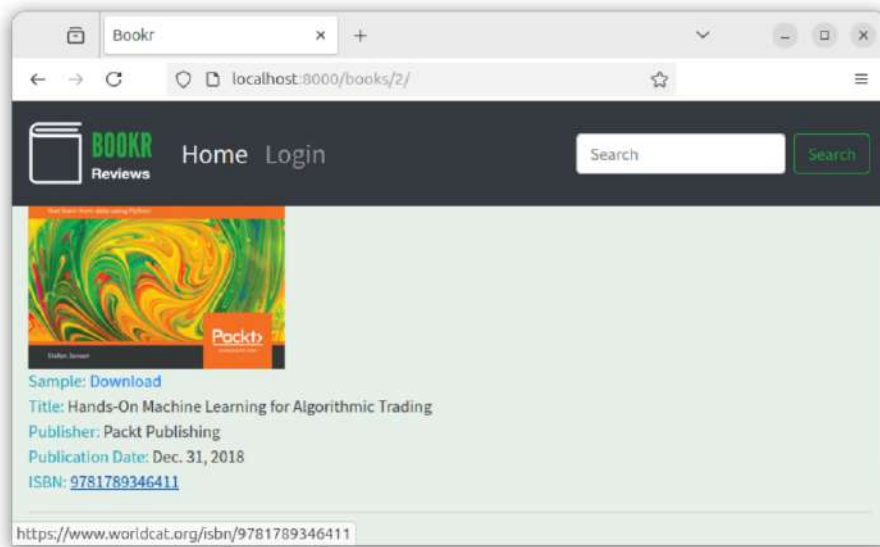


Figure 11.1 – Book detail page with ISBN link using the `link_worldcat` filter

Template tags

Template tags are a powerful feature of Django's templating engine. They allow developers to build powerful templates by generating HTML by evaluating certain conditions and help avoid the repetitive writing of common code.

One example where we may use template tags is the sign-up/login options in the navigation bar of a website. In this case, we can use template tags to evaluate whether the visitor on the current page is logged in. Based on that, we can render either a profile banner or a sign-up/login banner.

Tags are also a common occurrence while developing templates. For example, consider the following line of code, which we used to import the custom filters inside our templates in the previous section:

```
{% load explode_filter %}
```

This uses a template tag known as `load`, responsible for loading the `explode` filter into the template. Template tags are much more powerful compared to filters. While filters have access only to the values they are operating on, template tags have access to the context of the whole template and hence they can be used to build a lot of complex functionalities inside a template.

Let's now look at the different types of template tags supported by Django and how we can build our own custom template tags.

The types of template tags

Django mainly supports two types of template tags:

- **Simple tags:** These are the tags that operate on the variable data provided (and any additional variables to them) and render in the same template they have been called in. For example, one such use case can include rendering a custom welcome message to the user based on their username or displaying the user's last login time based on their username.
- **Inclusion tags:** These tags take in the provided data variables and generate an output by rendering another template. For example, the tag can take in a list of objects and iterate over them to generate an HTML list.

In the next sections, we will look at how we can create these different types of tags and use them in our application.

Simple tags

Simple tags provide a way for developers to build template tags that take in one or more variables from the template, process them, and return a response. The response returned from the template tag replaces the template tag definition provided inside the HTML template. These kinds of tags can be used to build several useful functionalities – for example, the parsing of dates or displaying any active alerts, if there are any, that we want to show to the user.

Simple tags can be created easily using the `simple_tag` decorator provided by the template library, by decorating the Python method that should act as a template tag. Now, let's look at how we can implement a custom simple tag using Django's template library.

Creating a simple template tag

Creating simple template tags follows the same conventions we discussed in the *Custom template filters* section, with some subtle differences. First, let's go over understanding how template tags can be created for use in our Django templates.

Setting up the directory

Just like custom filters, custom template tags also need to be created inside the same `templatetags` directory to make them discoverable by Django's templating engine. The directory can be created either directly using the PyCharm GUI or by running the following command inside the application directory where we want to create our custom tags:

```
mkdir templatetags
```

Once this is done, we can now create a new file that will store the code for our custom template tags by using the following command:

```
touch custom_tags.py
```

Note

The aforementioned command won't work on Windows. You can, however, create a new file using Windows Explorer.

Setting up the template library

Once the directory structure is set up and we have a file in place for keeping the code for our custom template tags, we can now start creating our template tags. But before that, we need to set up an instance of Django's template library as we did earlier. This can be done by adding the following lines of code to our `custom_tag.py` file:

```
from django import template
register = template.Library()
```

Like custom filters, the template library instance is used here to register the custom template tags for use inside Django templates.

Implementing a simple template tag

Simple template tags inside Django are Python functions that can take any number of arguments as we desire. These Python functions need to be decorated with the `simple_tag` decorator from the template library to register those functions as simple template tags. The following snippet of code shows how a simple template tag is implemented:

```
@register.simple_tag
def generic_simple_tag(arg1, arg2):
    # Logic to implement a generic simple tag
```

Next, let's use these simple tags inside the templates.

Using simple tags inside templates

Using simple tags inside Django templates is quite easy. Inside the template file, we need first to make sure that we have the tag imported inside the template by adding the following to the top of the template file:

```
{% load custom_tag %}
```

The preceding statement will load all the tags from the `custom_tag.py` file we defined earlier and make them available inside our template. Then we can use our custom simple tag by adding the following command:

```
{% custom_simple_tag "argument1" "argument2" %}
```

Wasn't that easy?! Now, let's put this knowledge into practice and create our first custom simple tag.

Exercise 11.02 – Creating a custom simple tag

Much of the existing templating code is somewhat cluttered and difficult to read because it contains control and conditional code. Consider this section from the search results template:

`reviews/templates/reviews/search-results.html`

```
{% for contributor in book.contributors.all %}
    {{ contributor.first_names }} {{ contributor.last_names }}
    {% if not forloop.last %}, {% endif %}
{% endfor %}
```

It makes sense to use simple tags as a way of keeping this conditional and control code in its own method. Now the code in the template can be replaced with two calls:

```
{% load list_items %}
{% list_items book.contributors.all 'first_names' 'last_names' %}
```

The `list_items` simple tag can be reused in our templates when we need to represent another list as a comma delimited string.

In this exercise, we will create a simple tag that takes multiple arguments: the first will be an iterable (such as a list of books or contributors), and the following arguments will be attributes

that belong to the list item. The tag will return a string rendering of the specified attributes separated by commas.

1. First, create a new file named `simple_tag.py` under the `reviews/templatetags` directory. Inside this file, add the following code:

```
from django import template

register = template.Library()

@register.simple_tag
def list_items(iterable, *fields):
    return ', '.join(
        [' '.join([getattr(item, field, '')
                    for field in fields])
         for item in iterable])
```

Here, you have created a new Python method, `list_items`, which takes an iterable argument followed by multiple arguments for the fields to be referenced. This method is then decorated with `@register.simple_tag`, indicating that this method is a simple tag and can be used as a template tag in the templates.

2. Now, edit the search results template that will use your simple tag. `reviews/templates/reviews/search-results.html`

```
{% extends 'base.html' %}
{% load simple_tag %}
{% block title %}
...
<ul class="list-group">
    {% for book in books %}
        <li class="list-group-item">
            ...
            <span class="text-info">Contributors: </span>
            {% list_items book.contributors.all
                'first_names' 'last_names' %}
        </li>
    ...
</ul>
{% endif %}
{% endblock %}
```


In the preceding code snippet, we just created a bare-bones HTML page that will use your custom simple tag. The semantics of loading a custom template tag is similar to that of loading a custom template filter and requires the use of a `{% load %}` tag in the template. The process will look for the `simple_tag.py` module under the `templatetags` directory and, if found, will load the tags that have been defined under the module.

The final set of changes should resemble the ones in the `Chapter11` folder in this book's GitHub repo.

3. To see the custom tag in action, start your web server by running the following command:

```
python manage.py runserver localhost:8000
```

When using the search form, the contributors will still be rendered as before, but the simple tag greatly simplifies the Django template.

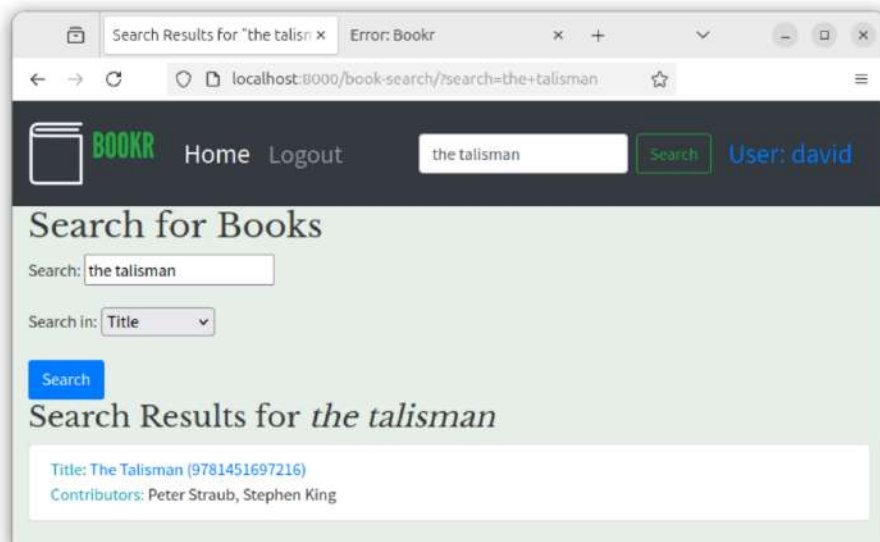


Figure 11.2 – Search form results

With this, we created our first custom template tag and used it successfully to render our template, as shown in *Figure 11.2*. Now, let's look at another important aspect of simple tags, which is associated with passing the context variables available in the template to the template tag.

Passing the template context in a custom template tag

In the previous exercise, we created a simple tag to which we passed two arguments, the greeting message and the username. But what if we wanted to pass a large number of variables to the tag? Or, simply, what if we did not want to pass the user's username explicitly to the tag?

There are times when developers would like to have access to all the variables and data that is present in the template to be available inside the custom tag. Fortunately for us, this is easy to implement.

Let's create a new tag named `contextual_greet_user` and see how we can pass the data available in the template directly to the tag instead of passing it manually as an argument.

Set up the tag as follows:

```
@register.simple_tag(takes_context=True)
def contextual_greet_user(context, message):
    username = context['username']
    return f'{message}, {username}'
```

With this, we tell Django that when our `contextual_greet_user` tag is used, Django should also pass it the template context, which has all the data passed from the view to the template. It accepts the added context as an argument and renders a greeting message to the current user.

In the preceding code example, we can see how the `contextual_greet_user()` method was modified to accept the passed context as the first argument, followed by the greeting message passed by the user.

To use this, we need the following call to the `contextual_greet_user` tag inside `simple_tag_template.html` under `filter_demo` to look like this:

```
{% contextual_greet_user "Hey" %}
```

Then, when we reload our Django web application, the output at `http://localhost:8000/filter_demo/greet` should look the same as was shown in step 5 of *Exercise 11.02, Creating a custom simple tag*.

With this, we learned how we can build a simple tag and handle passing the template context to the tag. Now, let's take a look at how we can build an inclusion tag that can be used to render data in a certain format, as described by another template.

Inclusion tags

Simple tags allow us to build tags that accept one or more input variables, do some processing on them, and return an output. This output is then inserted where the simple tag was used.

But what if we wanted to build tags that, instead of returning text output, return an HTML template, which can then be used to render the parts of the page? For example, many web applications allow users to add custom widgets to their profiles. These individual widgets can be built as an inclusion tag and rendered independently. This kind of approach keeps the code for the base page template and the individual templates separate and hence allows for easy reuse and refactoring.

Developing custom inclusion tags is a similar process to how we develop our simple tags. This involves the use of the `inclusion_tag` decorator provided by the template library. So, let's take a look at how we can do it.

Implementing inclusion tags

Inclusion tags are those tags used for rendering a template as a response to their usage inside a template. These tags can be implemented in a similar manner to how other custom template tags are implemented, with some minor modifications.

Inclusion tags are also simple Python functions that can take multiple parameters, where each parameter maps to an argument passed from the template where the tag was called. These tags are decorated using the `inclusion_tag` decorator from Django's template library. The `inclusion_tag` decorator takes a single parameter, the name of the template, which should be rendered as a response to the processing of the inclusion tag.

A generic implementation of an inclusion tag will look like the one shown in the following code snippet:

```
@register.inclusion_tag('template_file.html')
def my_inclusion_tag(arg):
    # Logic for processing
    return {'key1': 'value1'}
```

Notice the return value in this case. An inclusion tag is supposed to return a dictionary of values that will be used to render the `template_file.html` file specified as an argument in the `inclusion_tag` decorator.

Using an inclusion tag inside a template

An inclusion tag can easily be used inside a template file. This can be done by first importing the tag as follows:

```
{% load custom_tags %}
```

Then, by using the tag like any other tag:

```
{% my_inclusion_tag "argument1" %}
```

The response of the rendering of this tag will be a sub-template that will be rendered inside our primary template where the inclusion tag was used.

Exercise 11.03 – Building a custom inclusion tag

In this exercise, we are going to build a custom inclusion tag, called `book_item`, which will render the details of a book, simplifying the code in the existing book list template of the reviews app.

1. For this exercise, you will continue to use the same demo folders as in earlier exercises. First, create a new file named `inclusion_tag.py` under the `filter_demo/templatetags` directory and write the following code inside it:

```
from django import template

register = template.Library()

@register.inclusion_tag('book_list_item.html')
def book_list_item(book):
    return {'item': book}
```

The `@register.inclusion_tag` decorator is used to mark the method as a custom inclusion tag. This decorator takes the template's name as an argument that should be used to render the data returned by the tag function.

After the decorator, we defined a function that implements the logic of our custom inclusion tag. This function takes a single argument called `book`. This argument will be passed from the template file and contain a `Book` object.

This object is then returned as a context for the `book_list_item.html` template:

```
return {'item': book}
```

The value returned by this method will be passed by Django to the `book_list_item.html` template, and the contents will then be rendered.

2. Next, create the actual template, which will contain the rendering structure for the template tag. For this, create a new template file, `book_list_item.html`, under the `reviews/templates` directory, and add the following content to it:
`reviews/templates/book_list_item.html`

```
{% load link_worldcat_filter %}
<li class="list-group-item">
    <span class="text-info">Title: </span>
    <span>{{ item.book.title }}</span>
    <br>
    <span class="text-info">Publisher: </span>
    <span>{{ item.book.publisher }}</span>
    <br>
    <span class="text-info">Publication Date: </span>
    <span>{{ item.book.publication_date }}</span>
    <br>
    <span class="text-info">ISBN: </span>
    <span>
        <a href="{ { item.book.isbn|link_worldcat:'isbn' } }">
            {{ item.book.isbn }}</a></span>
    <br>
    <span class="text-info">Rating: </span>
    <span class="badge badge-primary badge-pill">
        {{ item.book_rating }}</span>
    <br>
    <span class="text-info">Number of reviews: </span>
    <span>{{ item.number_of_reviews }}</span>
    <br>
    {% if not item.book_rating %}
    <span class="text-secondary">
        Provide a rating and write the first review
        for this book.</span>
    <br>
    {% endif %}
```

```

        <a class="btn btn-primary btn-sm active" role="button"
          aria-pressed="true"
          href="/books/{{ item.book.id }}/">Reviews</a>
    </li>

```

Here, in the new template file, we used the contents of the `<li>` tag from the `reviews/templates/reviews/book_list.html` template. Note that we also included the `load link_worldcat_filter` directive as this filter is now included within the inclusion tag.

3. With the template defined for the `book_list_item` tag, modify the existing `book_list.html` template to make this tag available inside it and replace the contents of the `<li>` tag with a call to `book_list_item`.
`reviews/templates/reviews/book_list.html`

```

{% extends 'reviews/base.html' %}

{% load inclusion_tag %}

{% block content %}
<ul class="list-group">
    {% for item in book_list %}
        {% book_list_item item %}
    {% endfor %}
</ul>
{% endblock %}

```

In this snippet, the first thing you did was load the `inclusion_tag` module by writing the following:

```
{% load inclusion_tag %}
```

Once the tag is loaded, you can use it anywhere in the template. To use it, you added the `book_list_item` tag in the following format:

```
{% book_list_item item %}
```

This tag takes a single argument, a `Book` object.

Once the changes have been made, the files should resemble the ones hosted in the `Chapter11` folder in this book's GitHub repo. .

4. With the preceding changes implemented, it's time to render the modified template. To do this, restart your Django application server by running the following command:

```
python manage.py runserver localhost:8080
```

5. Navigate to `http://localhost:8080/books`, which should now render a page similar to the following screenshot:

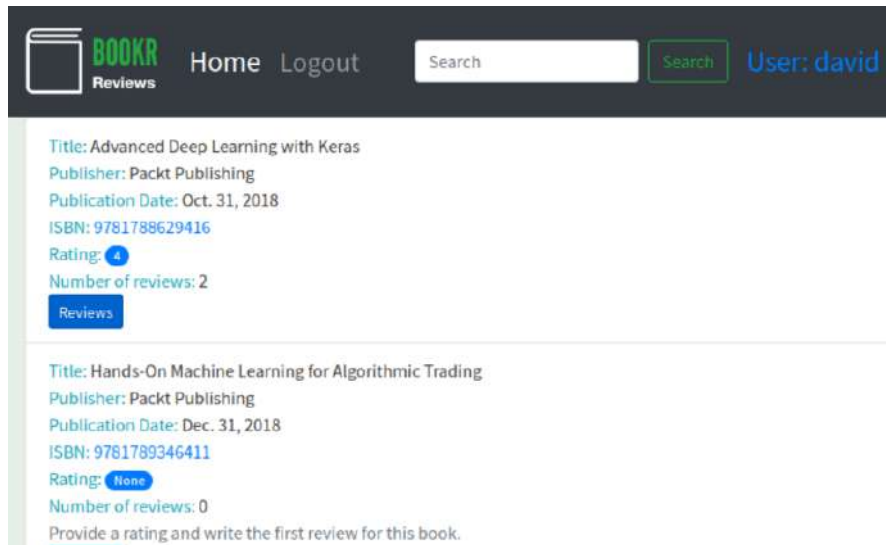


Figure 11.3 – List of books after refactoring the template code with inclusion_tag

The books endpoint still renders the same, but we have simplified the `book_list.html` template by using an inclusion tag. By modularizing the template code with inclusion tags, we make it easy to reuse components of pages without wholesale repetition of common code blocks.

Activity 11.01 – Rendering details on the user profile page using inclusion tags

In this activity, you will create a custom inclusion tag that helps to develop a user profile page that renders not only the users' details but also lists the books they have read.

The following steps should help you to complete this activity successfully:

1. Create a new `templatetags` directory under the reviews application inside the bookr project to provide a place where you can create your custom template tags.

2. In the `inclusion_tag.py` file, which you created in *Exercise 11.03*, add the following functionality:
3. Import the `Review` model from the `reviews` application to fetch the reviews written by a user. This will be used to filter the reviews for the current user to render on the user profile page.
4. Next, create a new Python function named `review_list`, which will contain the logic for your inclusion tag. This function should only take a single parameter, the username of the currently logged-in user.
5. Inside the body of the `review_list` function, add the logic for fetching the reviews for the given user and extract the review IDs and titles of the books this user has reviewed.
6. Decorate this `review_list` function with the `inclusion_tag` decorator and provide it with a template name, `review_list.html`.
7. Create a new template file named `review_list.html`, which was specified to the inclusion tag decorator in step 4. Inside this file, add code to render a list of books with hyperlinks to the user's reviews. This can be done by using a for loop construct and rendering HTML anchor and line break tags – `<a>`, `<br>` – for every item inside the list provided. The for loop will contain an empty clause that will display, "No reviews found".
8. Modify the existing `profile.html` file under the `templates` directory, which will be used to render the user profile. Inside this template file, load the `inclusion_tag` library and use `review_list` to render the list of books reviewed by the user. The code will be wrapped inside a new `div` of class `infoCell`.

Once you complete all these steps, starting the application server and visiting the user profile page should render a page that is similar to the one shown in *Figure 11.10*:

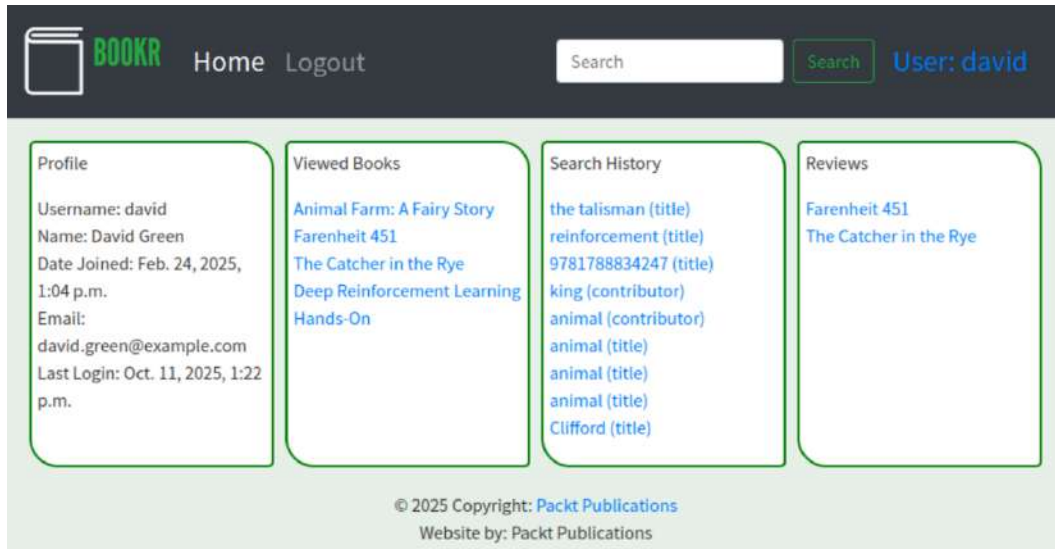


Figure 11.10 – The user profile page with the list of books reviewed by the user

Template partials

As we code our templates in a Django app, we tend to see a lot of repetition in the features that we have coded. Using inheritance from base templates is a good way to reuse common features such as headers, menus and footers but sometimes we want to reuse components within a page. **Template partials** are similar to functions that are defined within a template to define a reusable template block.

In order to create a template partial, we first define the template partial content using between the `partialdef` and `endpartialdef` template tags. We can reference context variables in the template partial as we do in the rest of a template.

```
{% partialdef "chapter" %}
    <h2>
    <a href="{{chapter.url}}" >{{chapter.number}}:
        {{chapter.title}}</a></h2>
{% endpartialdef %}
```

The template partial is called by using the partial template tag.

```
{% for chapter in chapters %}
    {% partial "chapter" %}
{% endfor %}
```

Let's say the Django view parses a context variable for chapters like this:

```
context = dict(chapters=(
    dict(url='/chap/1', number=1, title='Introduction'),
    dict(url='/chap/2', number=2,
        title='Historical Background'),
    dict(url='/chap/3', number=3,
        title='Technical Information'),
    dict(url='/chap/4',
        number=4, title='Conclusion'),
))
```

The template will be rendered as follows:

```
<h2><a href="/chap/1">1: Introduction</a></h2>
<h2><a href="/chap/2">2: Historical Background</a></h2>
<h2><a href="/chap/3">3: Technical Information</a></h2>
<h2><a href="/chap/4">4: Conclusion</a></h2>
```

This approach offers a simpler approach to reuse templating code than using an inclusion tag as the template partial is defined in the template file and doesn't require the overhead of an additional function in the `views.py` file. However, there are some cases, such as the `review_list` inclusion tag in *Activity 11.01 – Rendering details on the user profile page using inclusion tags*, where it makes sense to encapsulate database or control logic in the inclusion tag so that it can be shareable across multiple views. In the next activity, we will replace the `book_list_item` inclusion tag from *Exercise 11.03 – Building a custom inclusion tag* with a template partial.

Activity 11.02 – Rendering a book list item with template partials

In this activity, we will convert the inclusion tag defined in *Exercise 11.03* into a template partial.

1. Use the code in `reviews/templates/book_list_item.html` to form the body of a template partial definition in `reviews/templates/reviews/book_list.html`. Call the template partial `book_list_item` and include the `load link_worldcat_filter` directive and the contents of the `li` tag.
2. Now, in the content block in `reviews/templates/reviews/book_list.html`, replace the call to the inclusion tag `book_list_item` with a call to the template partial `book_list_item` using a Django template directive, `{% partial "book_list_item" %}`.
3. We can now delete the `reviews/templates/book_list_item.html` file as well as the inclusion tag, `book_list_item`, that is defined in `reviews/templatetags/inclusion_tag.py`.

The book list review will continue to render as it did in *Exercise 11.03*, but we have simplified the coding of the template.

With this, we now have the foundations on which we can build highly complex template filters, custom tags, and template partials that can be helpful in the development of the projects we want to work on.

Now, let's take a look at Django views and dive into a new territory of class-based views provided by Django to help us leverage the power of object-oriented programming that allows the reuse of code for the rendering of a view.

Django views

To recall, a view in Django is a piece of Python code that allows a request to be taken in, performs an action based on the request, and then returns a response to the user, hence forming an important part of Django applications.

Inside Django, we have the option of building our views by following two different methodologies, one of which we have already seen in the preceding examples and is known as

function-based views, while the other one, which we will be covering soon, is known as class-based views:

- **Function-based views (FBVs):** FBVs inside Django are nothing more than generic Python functions that are supposed to take an `HttpRequest` type object as their first positional parameter and return an `HttpResponse` type object, which corresponds to the action the view wants to perform once the request is processed by it. In the preceding exercise, `index()` and `greeting_view()` were examples of FBVs.
- **Class-based views (CBVs):** CBVs are views that closely adhere to the Python object-oriented principles and allow mapping of the view calls in a class-based representation. These views are specialized in nature, and a given type of CBV performs a specific operation. The benefits that CBVs provide include easy extensibility of the view and the reuse of code, which may turn out to be a complex task with FBVs.

Now, with the basic definitions clear and with knowledge of FBVs already in our arsenal, let's look at CBVs and see what they have in store for us.

Class-based views

Django provides different ways in which developers can write views for their applications. One way is to map a Python function to act as a view function to create FBVs. Another way of creating views is to use Python object instances (based on top of Python classes). These are known as CBVs. An important question that arises is, what is the need for a CBV when we can already create views using the FBV approach?

When creating FBVs, the idea is that, at times, we may be replicating the same logic again and again – for example, the processing of certain fields or logic for handling certain request types. Although it is completely possible to create logically separate functions that handle a particular piece of logic, the task becomes difficult to manage as the complexity of the application increases.

This is where CBVs come in handy, as they abstract away implementation of the common repetitive code that we need to write to handle certain tasks, such as the rendering of templates, while also making it easy to reuse pieces of code through the use of inheritance and mix-ins. For example, the following code snippet shows the implementation of a CBV:

```
from django.http import HttpResponse
from django.views import View

class IndexView(View):
```

```
def get(self, request):  
    return HttpResponse("Hey there!")
```

In the preceding example, we built a simple CBV by inheriting from the built-in `View` class that Django provides.

Using these CBVs is also quite easy. For example, let's say we wanted to map `IndexView` to a URL endpoint in our application. In this case, all we need to do is to add the following line to our `urlpatterns` list inside the `urls.py` file of the application:

```
urlpatterns = [  
    path('my_path', IndexView.as_view(), name='index_view')  
]
```

In this, as we can observe, we used the `as_view()` method of the CBV we created. Every CBV implements the `as_view()` method, which allows the `View` class to be mapped to a URL endpoint by returning the instance of the view controller from the view class.

Django provides a couple of built-in CBVs that provide the implementation of a lot of common tasks, such as how to render a template or how to process a particular request. The built-in CBVs help avoid rewriting code from scratch when handling basic functionality, thereby enabling the reusability of code. Some of these in-built views include the following:

- **View:** This is the base class for all CBVs available in Django that allows a custom CBV to be written with all the features provided and overridable. A user can implement their own definitions for different HTTP Request methods, such as GET, POST, PUT, and DELETE, and the view will automatically delegate the call to the method responsible for handling the request based on the type of request received.
- **TemplateView:** This is a view that can be used to render a template based on the parameters for the template data provided in the URL of the call. This allows developers to easily render a template without writing any logic related to how the rendering should be handled.
- **RedirectView:** This is a view that can automatically redirect a user to the correct resource based on the request they have made.
- **DetailView:** This is a view that is mapped to a Django model and can be used to render the data obtained from the model using a template of choice.

The preceding views are just some of the built-in views that Django provides by default, and we will cover more of them as we move through the chapter.

Now, to better understand how CBVs work inside Django, let's try to build our first CBV.

Exercise 11.04 – Creating a book catalog using a CBV

In this exercise, we will create a class-based form view that will help build a book catalog. This catalog will consist of the name of the book and the name of the author of the book:

1. To get started, create a new application inside the bookr project and name it `book_management`. This can be done by simply running the following command:

```
python manage.py startapp book_management
```

2. Now, before building the book catalog, you first need to define a Django model that will help you store the records inside the database. To do this, open the `models.py` file under the `book_management` app you just created and define a new model named `Book`, as shown here:

```
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=255)
    author = models.CharField(max_length=50)
```

The model contains two fields, the name of the book and the name of the author.

3. Once all the preceding steps are complete, add your `book_management` application to the `INSTALLED_APPS` list such that Django can discover it and you can use your model properly. For this, open the `settings.py` file under the `bookr` directory and add the following code at the final position in the `INSTALLED_APPS` section:

```
INSTALLED_APPS = [
    ...,
    'book_management'
]
```

4. After the changes have been made, the `settings.py` file should look like in the `Chapter11` folder in this book's GitHub repo.

5. Migrate your model to the database by running the following two commands. These will first create a Django migrations file and then create a table in your database:

```
python manage.py makemigrations
python manage.py migrate
```

6. Now, with the database model in place, let's create a new form that we will use to capture information pertaining to the books, such as the book title, author, and ISBN. For this, create a new file named `forms.py` under the `book_management` directory and add the following code inside it:

```
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['name', 'author']
```

In the preceding code snippet, we first imported Django's `forms` module, which will allow us to easily create forms and also provide the form's rendering capability. The next line imports the model that will store the data for the form:

```
from django import forms
from .models import Book
```

In the next line, we created a new class named `BookForm`, which inherits from `ModelForm`. This is nothing but a class that maps the fields of the model to the form. To successfully achieve this mapping between the model and the form, we defined a new subclass named `Meta` under the `BookForm` class and set the attribute `model` to point to the `Book` model and the attribute `fields` to the list of fields that you want to display in the form:

```
class Meta:
    model = Book
    fields = ['name', 'author']
```

This allows `ModelForm` to render the correct form of HTML when expected to do so. The `ModelForm` class provides a built-in `Form.save()` method, which, when used, writes the data in the form to the database and helps avoid having to write redundant code.

7. Now that you have both your model and the form ready, go ahead and implement a view that will render the form and accept input from the user. For this, open `views.py` under the `book_management` directory and add the following lines of code to the file:

```
from django.http import HttpResponseRedirect
from django.views.generic.edit import FormView
from django.views import View
from .forms import BookForm

class BookRecordFormView(FormView):
    template_name = 'book_form.html'
    form_class = BookForm
    success_url = '/book_management/entry_success'

    def form_valid(self, form):
        form.save()
        return FormView.form_valid(self, form)

class FormSuccessView(View):
    def get(self, request, *args, **kwargs):
        return HttpResponseRedirect(
            "Book record saved successfully")
```

In the preceding code snippet, we created two major views, `BookRecordFormView`, which is also responsible for rendering the book catalog entry form, and `FormSuccessView`, which you will use to render the success message if the form data is saved successfully. Let's now look at both views individually and understand what we are doing.

First, we created a new view named the `BookRecordFormView` CBV, which inherits from `FormView`:

```
class BookRecordFormView(FormView)
```

The `FormView` class allows us to easily create views that deal with forms. To this class, we need to provide certain parameters, such as the name of the template it will render

to show the form, the form class that it should use to render the form, and the success URL to redirect to when the form is processed successfully:

```
template_name = 'book_form.html'
form_class = BookForm
success_url = '/book_management/entry_success'
```

The `FormView` class also provides a `form_valid()` method, which is called when the form successfully finishes the validation. Inside the `form_valid()` method, we can decide what we want to do. For our use case, when the form validation completes successfully, we first call the `form.save()` method, which persists the data for our form into the database, and then call the base class `form_valid()` method, which will cause the form view to redirect to the successful URL in the event that form validation was a success:

```
def form_valid(self, form):
    form.save()
    return FormView.form_valid(self, form)
```

Note

The `form_valid()` method should always return an `HttpResponse` object.

This completes the implementation of `BookRecordFormView`. The next thing we have to do is to build a view named `FormSuccessView`, which we will use to render the success message once the data is saved successfully for the book record form we just created. This is done by creating a new view class named `FormSuccessView`, which inherits from the `View` base class of Django CBVs:

```
class FormSuccessView(View)
```

Inside this class, we override the `get()` method, which will be rendered when the form is saved successfully. Inside the `get()` method, we render a simple success message by returning a new `HttpResponse`:

```
def get(self, request, *args, **kwargs):
    return HttpResponse("Book record saved successfully")
```

8. Now, create a template that will be used to render the form. For this, create a new `templates` folder under the `book_management` directory and a new file named `book_form.html`. Add the following lines of code inside the file:

```
<html>
<head>
  <title>Book Record Insertion</title>
</head>
<body>
  <form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <input type="submit" value="Save record" />
  </form>
</body>
</html>
```

9. With the CBVs now built, go ahead and map them to URLs so that you can start using them to add new book records. To do this, create a new file named `urls.py` under the `book_management` directory and add the following code to it:

```
from django.urls import path

from .views import BookRecordFormView, FormSuccessView

urlpatterns = [
    path('new_book_record',
         BookRecordFormView.as_view(),
         name='book_record_form'),
    path('entry_success', FormSuccessView.as_view(),
         name='form_success'),
]
```

Most parts of the preceding snippet are similar to the ones you wrote earlier, but there is one thing different in the way we map our CBVs under the URL patterns. When using CBVs, instead of directly adding the function name, we use the class name and its `as_view` method, which maps the class object to the view. For example, to map `BookRecordFormView` as a view, we will use `BookRecordFormView.as_view()`.

10. With the URLs added to our `urls.py` file, the next thing is to add the application URL mapping to the `bookr` project. To do this, open the `urls.py` file under the `bookr` application and add the following line to `urlpatterns`:

```
urlpatterns = [  
    path('book_management/',  
        include('book_management.urls')),  
    ...  
]
```

11. Now, start your development server by running the following command:

```
python manage.py runserver localhost:8080
```

12. Then, visit `http://localhost:8080/book_management/new_book_record`. If everything works fine, you will see a page like the one shown here:

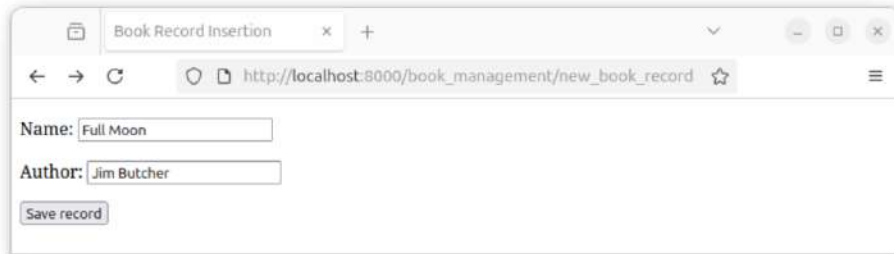
A screenshot of a web browser window with the title "Book Record Insertion". The address bar shows "http://localhost:8080/book_management/new_book_record". The form contains two text input fields: "Name:" with the value "Full Moon" and "Author:" with the value "Jim Butcher". Below the fields is a "Save record" button.

Figure 11.4 – The view for adding a new book to the database

Upon clicking **Save record**, your record will be written to the database, and the following page will appear:

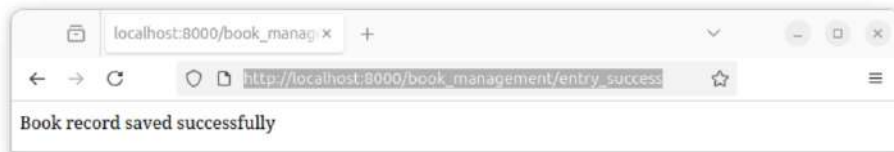


Figure 11.5 – The template is rendered when the record is successfully inserted

With this, we have created our own CBV, which allows us to save records for new books. With our knowledge of CBVs in tow, let's now look at how we can perform **create, read, update, and delete (CRUD)** operations with the help of CBVs.

CRUD operations with CBVs

While working with Django models, one of the most common patterns we run into involves creating, reading, updating, and deleting objects that are stored inside our database. The Django admin interface allows us to achieve these CRUD operations easily, but what if we wanted to build custom views to give us the same capability?

As it turns out, Django's CBVs allow us to achieve this quite easily. All we need to do is to write our custom CBVs and inherit them from the built-in base classes provided by Django. Building on our existing example of book record management, let's see how we can build CRUD-based views in Django.

The Create view

To build a view that helps in object creation, we'll need to open the `view.py` file under the `book_management` directory and add the following lines of code to it:

```
from django.views.generic.edit import CreateView
from .models import Book

class BookCreateView(CreateView):
    model = Book
    fields = ['name', 'author']
    template_name = 'book_form.html'
    success_url = '/book_management/entry_success'
```

With this, we created `CreateView` for the book resource. Before we can use it, we will need to map it to a URL. To do this, we can open the `urls.py` file and add the following entry under the `urlpatterns` list:

```
urlpatterns = [
    ...,
    path('book_record_create', BookCreateView.as_view(),
        name='book_create')
]
```

Now, when we visit `http://localhost:8080/book_management/book_record_create`, we will be greeted with the following page:

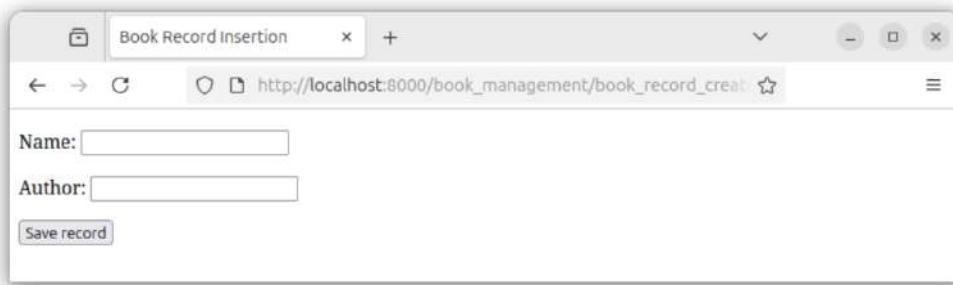


Figure 11.6 – A view to insert a new book record based on the Create view

This looks similar to the one we got when using the form view. On filling in the data and clicking **Save record**, Django will save the data to the database.

The Read view

In this view, we want to see a list of records that our database stores for the books. To achieve this, we will build a view named `DetailView`, which will render details about the book we are requesting. To build this view, we can add the following lines of code to our `views.py` file under the `book_management` directory:

```
from django.views.generic import DetailView

class BookRecordDetailView(DetailView):
    model = Book
    template_name = 'book_detail.html'
```

In the preceding code snippet, we created `DetailView`, which will help us to render the details pertaining to the book ID we are asking for. `DetailView` internally queries our database model with the book ID we provide to it and, if a record is found, renders the template with the data stored inside the record by passing it as an object variable inside the template context.

Once this is done, the next step is to create the template for our book details. For this, we'll need to create a new template file named `book_detail.html` under our `templates` directory inside the `book_management` application with the following contents:

```
<html>
<head>
  <title>Book List</title>
</head>
```

```
<body>
  <span>Book Name: {{ object.name }}</span><br />
  <span>Author: {{ object.author }}</span>
</body>
</html>
```

Now, with the template created, the last thing we need to do is to add a URL mapping for the detail view. This can be done by appending the following to the `urlpatterns` list under the `urls.py` file of the `book_management` application:

```
path('book_record_detail/<int:pk>',
      BookRecordDetailView.as_view(), name='book_detail')
```

Now, with all of this configured, if we now open `http://localhost:8080/book_management/book_record_detail/2`, we will get to see the details about our book, as shown here:

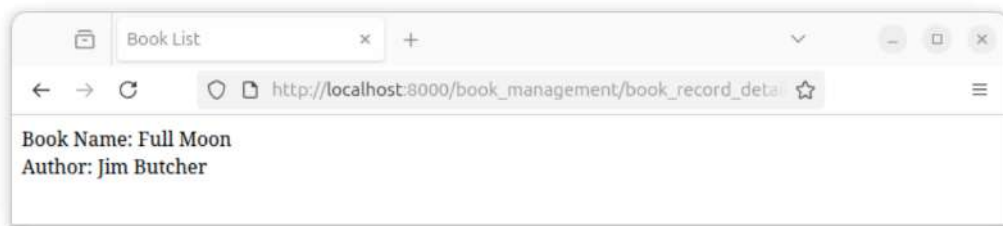


Figure 11.7 – The view rendered when we try to access a previously stored book record

With the preceding examples, we just enabled CRUD operations for our `Book` model, all while using CBVs.

The Update view

In this view, we want to update the data for a given record. To do this, we need to open the `view.py` file under the `book_management` directory and add the following lines of code to it:

```
from django.views.generic.edit import UpdateView

from .models import Book

class BookUpdateView(UpdateView):
    model = Book
    fields = ['name', 'author']
```

```
template_name = 'book_form.html'  
success_url = '/book_management/entry_success'
```

In the preceding code snippet, we used the built-in `UpdateView` template, which allows us to update the stored records. Therefore, the `fields` attribute here should take in the name of the fields that we would like to allow the user to update.

Once the view is created, the next step is to add the URL mapping. To do this, open the `urls.py` file under the `book_management` directory and add the following lines of code:

```
urlpatterns = [  
    path('book_record_update/<int:pk>',  
        BookUpdateView.as_view(), name='book_update')  
]
```

In this example, we appended `<int:pk>` to the URL field. This signifies the field input we will have to retrieve the record for. Inside Django models, Django inserts a primary key of the integer type, which uniquely identifies the records. Inside the URL mapping, this is the field that we asked to insert.

Now, when we try to open `http://localhost:8080/book_management/book_record_update/1`, it should show us a record of the first record that we inserted into our database and allow us to edit it:

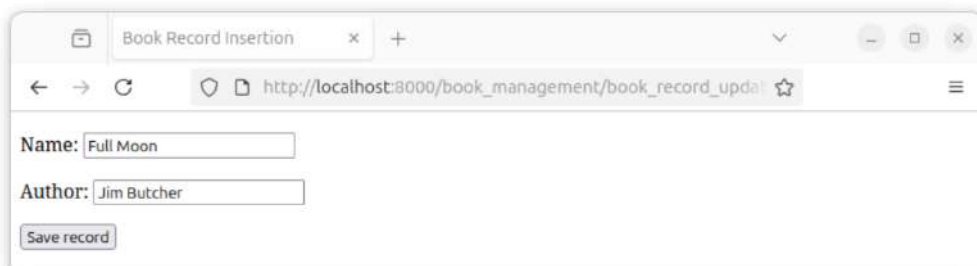


Figure 11.8 – The view displaying the book record update template based on the `UpdateView`

The Delete view

The Delete view, as the name suggests, is a view that deletes the record from our database. To implement such a view for our Book model, we will need to open the `views.py` file under the `book_management` directory and add the following code snippet to it:

```
from django.views.generic.edit import DeleteView
from .models import Book

class BookDeleteView(DeleteView):
    model = Book
    template_name = 'book_delete_form.html'
    success_url = '/book_management/delete_success'
```

With this, we just created a Delete view for our book records. As we can see, this view uses a different template, where all we would like to confirm with the user is whether they really want to delete the record or not? To achieve this, you can create a new template file, `book_delete_form.html`, and add the following code to it:

```
<html>
<head>
    <title>Delete Book Record</title>
</head>
<body>
    <p>Delete Book Record</p>
    <form method="POST">
        {% csrf_token %}
        Do you want to delete the book record?
        <input type="submit" value="Delete record" />
    </form>
</body>
</html>
```

Note

For the code related to the `/delete_success` endpoint, take a look at the code files under the `Chapter 11` directory on the book's accompanying GitHub repository.

Then we can add a mapping for our delete view by modifying the `urlpatterns` list inside the `urls.py` file under the `book_management` directory as follows:

```
urlpatterns = [
    ...,
    path('book_record_delete/<int:pk>',
        BookDeleteView.as_view(), name='book_delete')
]
```

Now, when visiting `http://localhost:8080/book_management/book_record_delete/1`, we should be greeted with the following page:

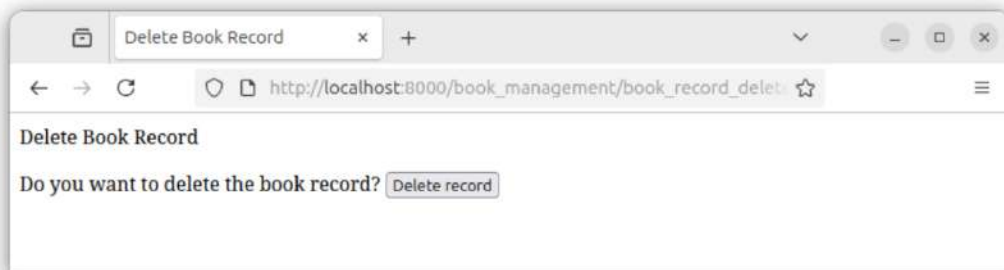


Figure 11.9 – The Delete Book Record view based on the `DeleteView` class

Upon clicking the **Delete record** button, the record will be deleted from the database, and the **Deletion success** page will be rendered.

Summary

In this chapter, we learned about the advanced templating concepts in Django and understood how we can create custom template tags, filters, and partials to fit a myriad of use cases and support the reusability of components across the application. We then examined how Django provides us with the flexibility to implement FBVs and CBVs to render our responses.

While exploring CBVs, we learned how they can help us avoid code duplication and leverage the built-in CBVs to render forms that save data, help us update existing records, and implement CRUD operations on our database resources.

As we move on to the next chapter, we will now utilize our knowledge of building CBVs to work on implementing REST APIs in Django. This will allow us to perform well-defined HTTP operations on our data inside our Bookr application without maintaining any state inside the application.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

12

Building a REST API

In the previous chapter, we learned about templates and class-based views. These concepts greatly help expand the range of functionalities we can provide to the user on the frontend (in their web browser). However, that is not sufficient to build a modern web application. Web apps typically have the frontend built with an entirely separate library, such as ReactJS or AngularJS. These libraries provide powerful tools for building dynamic user interfaces but do not communicate directly with our backend Django code or database. The frontend code simply runs in the web browser and does not have direct access to any data on our backend server. Therefore, we need to create a way for these applications to "talk" to our backend code. One of the best ways to do this in Django is by using REST APIs.

Application programming interfaces (APIs) are used to facilitate interaction between different pieces of software, and they communicate using **Hypertext Transfer Protocol (HTTP)**. This is the standard protocol for communication between servers and clients and is fundamental to information transfer on the web. APIs receive requests and send responses in HTTP format.

This chapter introduces **REST APIs** and **Django REST framework (DRF)**. You will start by implementing a simple API for the bookr project. Next, you will learn about the serialization of model instances, which is a key step in delivering data to the frontend side of Django applications. Finally, you will explore different types of API views, including both functional and class-based types.

In our use case in this chapter, an API will help facilitate interaction between our Django backend and our frontend JS code. For example, imagine that we want to create a frontend application that allows users to add new books to the bookr database. The user's web browser would send a message (an HTTP request) to our API to say that they want to create an entry for a new book and perhaps include some details about the book in that message. Our server

would send back a response to report on whether the book was successfully added or not. The web browser would then be able to display the outcome of their action to the user.

In this chapter, we will cover the following topics:

- Understanding REST APIs
- DRF
- Understanding serializers
- Simplifying the code using ViewSets
- Implementing authentication

By the end of this chapter, you will be able to implement custom API endpoints, including token-based authentication.

Technical requirements

All the code files of this chapter can be found in the Chapter12 folder in this book's GitHub repo. To access the repository link, follow the steps in the *"Download the example code files"* section in the *Preface*.

Understanding REST APIs

Most modern web APIs can be classified as **representational state transfer (REST)** APIs. REST APIs are simply a type of API that focuses on communicating and synchronizing the *state* of objects between the database server and the frontend client.

For example, imagine that you are updating your details on a website for which you are signed in to your account. When you go to the account details page, the web server tells your browser about the various details attached to your account. When you change the values on that page, the browser sends back the updated details to the web server and tells it to update these details in the database. If the action is successful, the website will show you a confirmation message.

This is a very simple example of what is known as **decoupled** architecture between frontend and backend systems. Decoupling allows greater flexibility and makes it easier to update or change components in your architecture. So, let's say you want to create a new frontend website. In such a case, you don't have to change the backend code at all, as long as your new frontend is built to make the same API requests as the old one.

REST APIs are *stateless*, meaning neither the client nor the server stores any states in between to do the communication. Every time a request is made, the data is processed, and a response is sent back without the protocol having to store any intermediate data. What this means is that the API is processing each request in isolation. It doesn't need to store information regarding

the session itself. This is in contrast to a stateful protocol (such as **Transmission Control Protocol (TCP)**), which maintains information regarding the session during its life.

So, a **RESTful web service**, as the name suggests, is a collection of REST APIs used to carry out a set of tasks. For example, if we develop a set of REST APIs for the bookr application to carry out a certain set of tasks, we can call it a RESTful web service. Next, we will start working with DRF.

DRF

DRF is an open source Python library that can be used to develop REST APIs for a Django project. DRF has most of the necessary functionality built in to help develop APIs for any Django project. Throughout this chapter, we will be using it to develop APIs for our bookr project. In the following section, we will install and configure DRF.

Installation and configuration

To install `django-rest-framework` in the `virtual env` setup along with PyCharm, follow these steps:

1. Enter the following code in your terminal app or Command Prompt to do this:

```
pip install django-rest-framework
```

2. Next, open the `settings.py` file and add `rest_framework` to `INSTALLED_APPS` as shown in the following snippet:

```
INSTALLED_APPS = [  
    'reviews.adminconfig.ReviewsAdminConfig',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'reviews',]  

```

You are now ready to start using DRF to create your first simple API.

Functional API views

In *Chapter 3, URL Mapping, Views, and Templates*, we learned about simple functional views that take a request and return a response. We can write similar functional views using DRF. However, note that class-based views are more commonly used and will be covered next. A functional view is created by simply adding the following decorator to a normal view, as follows:

```
from rest_framework.decorators import api_view

@api_view
def my_view(request):
    ...
```

This decorator takes the functional view and turns it into a subclass of the `APIView` DRF. It's a quick way to include an existing view as part of your API. Using what we have learned so far, we will create a simple REST API using DRF in the next section.

Exercise 12.01 – Creating a simple REST API

In this exercise, you will create your first REST API using DRF and implement an endpoint using a functional view. You will create this endpoint to view the total number of books in the database:

Note

You'll need to have DRF installed on your system to proceed with this exercise. If you haven't already installed it, make sure you refer to the *Installation and configuration* section earlier in this chapter.

1. Create `api_views.py` in the `bookr/reviews` folder.

REST API views work like Django's conventional views. We could have added the API views, along with the other views, in the `views.py` folder. However, having our REST API views in a separate file will help us maintain a cleaner code base.

2. Add the following code in `api_views.py`:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Book
```

```
@api_view(['GET'])
def first_api_view(request):
    num_books = Book.objects.count()
    return Response({"num_books": num_books})
```

The first line imports the `api_view` decorator, which will convert our functional view into one that can be used with DRF, and the second line imports `Response`, which will be used to return a response.

The view function returns a `Response` object containing a dictionary with the number of books in our database (see the highlighted part in the previous code snippet).

3. Open `bookr/reviews/urls.py` and import the `api_views` module. Then, add a new path to the `api_views` module in the URL patterns that we have developed throughout this course, as follows:

```
from . import views, api_views

urlpatterns = [path('api/first_api_view/',
                  api_views.first_api_view),
              ...
            ]
```

4. Start the Django service with the `python manage.py runserver` command and go to `http://127.0.0.1:8000/api/first_api_view/` to make your first API request. Your screen should appear as in *Figure 12.1*:



Figure 12.1: First Api View with the number of books

Calling this URL endpoint made a default GET request to the API endpoint, which returned a **JavaScript Object Notation (JSON)** key-value pair (`"num_books": 0`). Also, notice how DRF provides a nice interface to view and interact with the APIs.

5. We could also use the Linux `curl` (client URL) command to send an HTTP request as follows:

```
curl http://127.0.0.1:8000/api/first_api_view/  
{ "num_books": 18 }
```

Alternatively, if you are using Windows 10, you can make an equivalent HTTP request with `curl.exe` from Command Prompt as follows:

```
curl.exe http://127.0.0.1:8000/api/first_api_view/
```

In this exercise, we learned how to create an API view using DRF and a simple functional view. We will now look at a more elegant way to convert between information stored in the database and what gets returned by our API using serializers.

Understanding serializers

By now, we are well versed in the way Django works with data in our application. Broadly, the columns of a database table are defined in a class in `models.py`, and when we access a row of the table, we are working with an instance of that class. Ideally, we often just want to pass this object to our frontend application. For example, if we wanted to build a website that displayed a list of books in our bookr app, we would want to call the `title` property of each book instance to know what string to display to the user. However, our frontend application knows nothing about Python and needs to retrieve this data through an HTTP request, which just returns a string in a specific format.

This means that any information translated between Django and the frontend (via our API) must be done by representing the information in JSON format. JSON objects look similar to a Python dictionary, except there are some extra rules that constrict the exact syntax. In our previous example in *Exercise 12.01 – Creating a simple REST API*, the API returned the following JSON object containing the number of books in our database:

```
{ "num_books": 0 }
```

But what if we wanted to return the full details about an actual book in our database with our API? DRF's serializer class helps to convert complex Python objects into formats such as

JSON or XML so that they can be transmitted across the web using the HTTP protocol. The part of DRF that does this conversion is named `serializer`. Serializers also perform deserialization, which refers to converting serialized data back into Python objects so that the data can be processed in the application. In the following section, we will implement a way to display a list of books.

Exercise 12.02 – Creating an API view to display a list of books

In this exercise, you will use serializers to create an API that returns a list of all books present in the bookr application:

1. Create a file named `serializers.py` in the `bookr/reviews` folder. This is the file where we will place all the serializer code for the APIs.
2. Add the following code to `serializers.py`:

```
from rest_framework import serializers

class PublisherSerializer(serializers.Serializer):
    name = serializers.CharField()
    website = serializers.URLField()
    email = serializers.EmailField()

class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
    isbn = serializers.CharField()
    publisher = PublisherSerializer()
```

Here, the first line imports the serializers from the `rest_framework` module.

Following the imports, we have defined two classes, `PublisherSerializer` and `BookSerializer`. As the names suggest, they are serializers for the `Publisher` and `Book` models, respectively. Both these serializers are subclasses of `serializers.Serializer`, and we have defined field types for each serializer, such as `CharField`, `URLField`, `EmailField`, and so on.

Look at the `Publisher` model in the `bookr/reviews/models.py` file. The `Publisher` model has the `name`, `website`, and `email` attributes. So, to serialize a `Publisher` object,

we need the name, website, and email attributes in the serializer class, which we have defined accordingly in `PublisherSerializer`.

Similarly, for the `Book` model, we have defined title, publication_date, isbn, and publisher as the desired attributes in `BookSerializer`. Since publisher is a foreign key for the `Book` model, we have used `PublisherSerializer` as the serializer for the publisher attribute.

3. Open `bookr/reviews/api_views.py`, remove any pre-existing code, and add the following code:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

from .models import Book
from .serializers import BookSerializer

@api_view(['GET'])
def all_books(request):
    books = Book.objects.all()
    book_serializer = BookSerializer(books, many=True)
    return Response(book_serializer.data)
```

In the second line, we have imported the newly created `BookSerializer` class from the `serializers` module.

We then add a functional view, `all_books` (as in the previous exercise). This view takes a `QuerySet` containing all books and then serializes them using `BookSerializer`. The serializer class is also taking an argument, `many=True`, which indicates that the `books` object is a `QuerySet` or a list of many objects. Remember that serialization takes Python objects and returns them in a JSON serializable format, as follows:

```
[OrderedDict([('title', 'Advanced Deep Learning with Keras'),
('publication_date', '2018-10-31'), ('isbn', '9781788629416'),
('publisher', OrderedDict([('name', 'Packt Publishing'), ('website',
'https://www.packtpub.com/'), ('email', 'info@packtpub.com')]))]),
OrderedDict([('title', 'Hands-On Machine Learning for Algorithmic
Trading'), ('publication_date', '2018-12-31'), ('isbn', '9781789346411'),
```

```
('publisher', OrderedDict([('name', 'Packt Publishing'), ('website',  
'https://www.packtpub.com/'), ('email', 'info@packtpub.com')])))) ...
```

4. Open `bookr/reviews/urls.py`, remove the previous example path for `first_api_view`, and add the `all_books` path, as shown in the following code:

```
from django.urls import path  
from . import views, api_views  
  
urlpatterns = [  
    path(  
        'api/all_books/',  
        api_views.all_books, name='all_books'  
    ),  
    ...  
]
```

This newly added path calls the `all_books` view function when it comes across the `api/all_books/` path in the URL.

5. Once all the code is added, run the Django server with the `python manage.py runserver` command and navigate to `http://127.0.0.1:8000/api/all_books/`. You should see something similar to *Figure 12.2*:

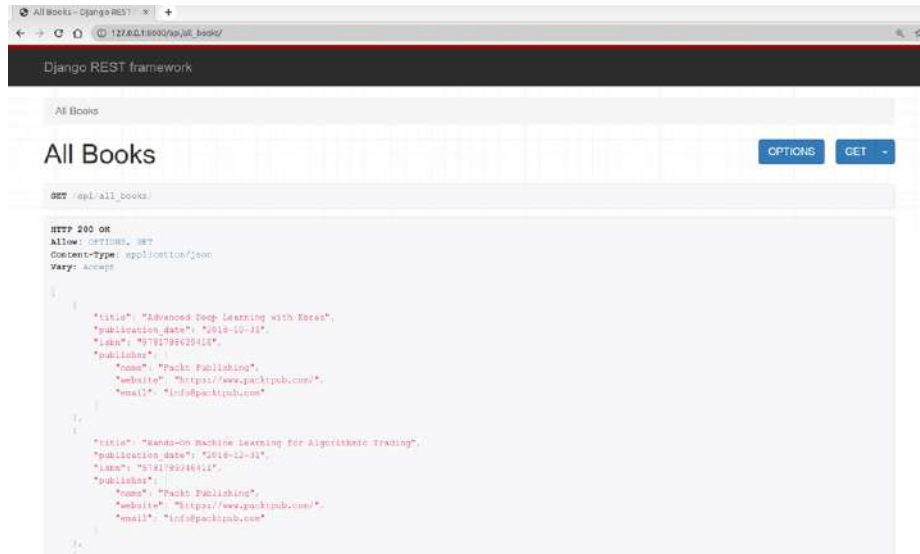


Figure 12.2: List of books shown in the `all_books` endpoint

The preceding screenshot shows that a list of all books is returned upon calling the `/api/all_books` endpoint. With that, you have successfully used a serializer to return data efficiently in your database with the help of a REST API.

Till now, we have been focusing on functional views. However, you will now learn that class-based views are more commonly used in DRF and will make your life much easier.

Class-based API views and generic views

Similar to what we learned in *Chapter 11, Advanced Templating and Class-Based Views*, we can also write class-based views for REST APIs. Class-based views are the preferred way of writing views among developers, as a lot can be achieved by writing very little code.

Just as with conventional views, DRF offers a set of generic views that make writing class-based views even simpler. Generic views are designed considering some of the most common operations needed while creating APIs. Some of the generic views offered by DRF are `ListAPIView`, `RetrieveAPIView`, and so on. In *Exercise 12.02 – Creating an API view to display a list of books*, our functional view was responsible for creating a `QuerySet` of the objects and then calling the serializer. Equivalently, we could use `ListAPIView` to do the same thing:

```

class AllBooks(ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer

```

Here, a `QuerySet` of objects is defined as a class attribute. Passing `queryset` through to `serializer` is handled by methods on `ListAPIView`.

Model serializers

In *Exercise 12.02 – Creating an API view to display a list of books*, our serializer was defined as follows:

```
class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
    isbn = serializers.CharField()
    publisher = PublisherSerializer()
```

However, our model for `Book` looks like this (note how similar the definitions of the model and serializer appear to be):

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70,
        help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        max_length=20,
        verbose_name="ISBN number of the book.")
    ...
    def __str__(self):
        return f"{self.title} ({self.isbn})"
```

We would prefer not to specify that the title must be `serializers.CharField()`. It would be easier if the serializer just looked at how `title` was defined in the model and could figure out what serializer field to use.

This is where model serializers come in. They provide shortcuts to create serializers by utilizing the definition of the fields on the model. Instead of specifying that `title` should be serialized using `CharField`, we just tell the model serializer we want to include `title`, and it uses the `CharField` serializer because the `title` field on the model is also `CharField`.

For example, suppose we wanted to create a serializer for the Contributor model in `models.py`. Instead of specifying the types of serializers that should be used for each field, we can give it a list of field names and let it figure out the rest:

```
from rest_framework import serializers

from .models import Contributor

class ContributorSerializer(serializers.ModelSerializer):

    class Meta:
        model = Contributor
        fields = ['first_names', 'last_names', 'email']
```

In the following exercise, we will see how we can use a model serializer to avoid the duplication of code in the preceding classes.

Exercise 12.03 – Creating class-based API views and model serializers

In this exercise, you will create class-based views to display a list of all books while using model serializers:

1. Open the `bookr/reviews/serializers.py` file, remove any pre-existing code, and replace it with the following code:

```
from rest_framework import serializers

from .models import Book, Publisher

class PublisherSerializer(
    serializers.ModelSerializer):

    class Meta:
        model = Publisher
        fields = ['name', 'website', 'email']

class BookSerializer(serializers.ModelSerializer):
    publisher = PublisherSerializer()
```

```
class Meta:
    model = Book
    fields = ['title', 'publication_date', 'isbn',
              'publisher']
```

Here, we have included two model serializer classes, `PublisherSerializer` and `BookSerializer`. Both these classes inherit the `serializers.ModelSerializer` parent class. We do not need to specify how each field gets serialized; instead, we can simply pass a list of field names, and the field types are inferred from the definition in `models.py`.

Although mentioning the field inside `fields` is sufficient for the model serializer, in certain special cases, such as this one, we may have to customize the field since the `publisher` field is a foreign key. Hence, we must use `PublisherSerializer` to serialize the `publisher` field.

2. Next, open `bookr/reviews/api_views.py`, remove any pre-existing code, and add the following code:

```
from rest_framework import generics

from .models import Book
from .serializers import BookSerializer

class AllBooks(generics.ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

Here, we use the DRF class-based `ListAPIView` view instead of a functional view. This means that the list of books is defined as a class attribute, and we do not have to write a function that directly handles the request and calls the serializer. The book serializer from the previous step is also imported and assigned as an attribute of this class.

Open the `bookr/reviews/urls.py` file and modify the `/api/all_books` API path to include the new class-based view as follows:

```
urlpatterns = [
    path(
        'api/all_books/',
        api_views.AllBooks.as_view(),
```



```

        name='all_books'
    ),
    ...
]

```

Since we are using a class-based view, we have to use the class name along with the `as_view()` method.

3. Once all the preceding modifications are completed, wait till the Django service restarts or start the server with the `python manage.py runserver` command, and then open the API at `http://127.0.0.1:8000/api/all_books/` in the web browser. You should see something like *Figure 12.3*:

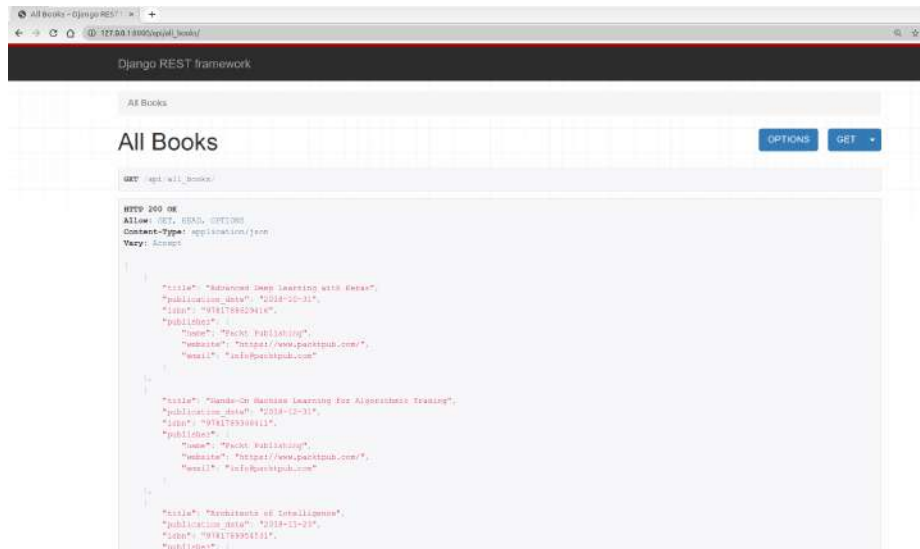


Figure 12.3: List of books shown in the `all_books` endpoint

Similar to what we saw in *Exercise 12.02 – Creating an API view to display a list of books*, this is a list of all books present in the book review application. In this exercise, we used model serializers to simplify our code and the generic class-based `ListAPIView` view to return a list of the books in our database. In the next section, you will implement an API endpoint to return a list of top contributors.

Activity 12.01 – Creating an API endpoint for a top contributors page

Imagine that your team decides to create a web page that displays the top contributors (authors, coauthors, and editors) in your database. They decide to enlist the services of an external developer to create an app in ReactJS. To integrate with the Django backend, the developer will need an endpoint that provides the following:

- A list of all contributors in the database
- For each contributor, a list of all books they contributed to
- For each contributor, the number of books they contributed to
- For each book they contributed to, their role in the book

The final API view should look like this:

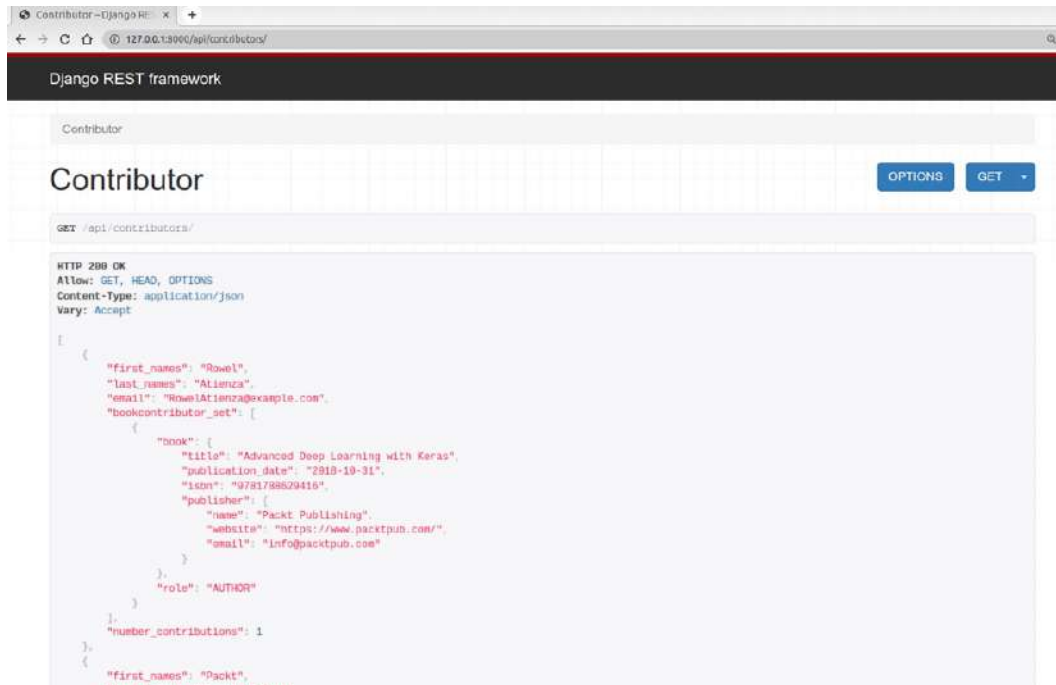


Figure 12.4: The top contributors endpoint

To perform this task, execute the following steps:

1. Add a method to the Contributor class that returns the number of contributions made.
2. Add ContributionSerializer, which serializes the BookContribution model.

3. Add `ContributorSerializer`, which serializes the `Contributor` model.
4. Add `ContributorView`, which uses `ContributorSerializer`.
5. Add a pattern to `urls.py` to enable access to `ContributorView`.

Simplifying the code using ViewSets

We have seen how we can optimize our code and make it more concise using class-based generic views. **ViewSets** and **routers** help us further simplify our code. As the name indicates, ViewSets are a set of views represented in a single class. For example, we used the `AllBooks` view to return a list of all books in the application and the `BookDetail` view to return the details of a single book. Using ViewSets, we could combine both these classes into a single class.

DRF also provides a class named `ModelViewSet`. This class not only combines the two views mentioned in the preceding discussion (list and detail) but also allows you to create, update, and delete model instances. The code needed to implement all this functionality could be as simple as specifying the serializer and `QuerySet`. For example, a view that allows you to manage all these actions for your user model could be defined as tersely as the following:

```
class UserViewSet(viewsets.ModelViewSet):
    serializer_class = UserSerializer
    queryset = User.objects.all()
```

Lastly, DRF provides a `ReadOnlyModelViewSet` class. This is a simpler version of the preceding `ModelViewSet` class. It is identical, except that it only allows you to list and retrieve specific users. You cannot create, update, or delete records. In the following section, we will learn about routers.

URL configuration using routers and ViewSets

Routers, when used along with a `ViewSet`, automatically create the required URL endpoints for the `ViewSet`. This is because a single `ViewSet` is accessed at different URLs. For example, in the preceding `UserViewSet` class, you would access a list of users at the `/api/users/` URL, and a specific user record at the `/api/users/123` URL, where 123 is the primary key of that user record. Here is a simple example of how you might use a router in the context of the previously defined `UserViewSet` class:

```
from rest_framework import routers
router = routers.SimpleRouter()
```

```
router.register(r'users', UserViewSet)
urlpatterns = router.urls
```

Now, let's try combining the concepts of routers and ViewSets in a simple exercise.

Exercise 12.04 – Using ViewSets and routers

In this exercise, we will combine the existing views to create a ViewSet and create the required routing for the ViewSet:

1. Open the `bookr/reviews/serializers.py` file, remove the pre-existing code, and add the following code snippet:

```
from django.contrib.auth.models import User
from django.utils import timezone
from rest_framework import serializers
from rest_framework.exceptions import
    NotAuthenticated, PermissionDenied
from .models import Book, Publisher, Review
from .utils import average_rating
class PublisherSerializer(serializers.ModelSerializer):
```

You can find the complete code snippet in the `Chapter12` folder in this book's GitHub repo. To access the repository link, follow the steps in the *"Download the example code files"* section in the *Preface*.

Here, we added two new fields to `BookSerializer`, namely `reviews` and `rating`. The interesting thing about these fields is that the logic behind them is defined as a method on the serializer itself. This is why we use the `serializers.SerializerMethodField` type to set the serializer class attributes.

2. Open the `bookr/reviews/api_views.py` file, remove the pre-existing code, and add the following:

```
from rest_framework import viewsets
from rest_framework.pagination import LimitOffsetPagination

from .models import Book, Review
from .serializers import (BookSerializer, ReviewSerializer)

class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
```

```
serializer_class = BookSerializer

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.order_by(
        '-date_created')
    serializer_class = ReviewSerializer
    pagination_class = LimitOffsetPagination
    authentication_classes = []
```

Here, we have removed the `AllBook` and `BookDetail` views and replaced them with `BookViewSet` and `ReviewViewSet`. In the first line, we import the `ViewSets` module from `rest_framework`. The `BookViewSet` class is a subclass of `ReadOnlyModelViewSet`, which ensures that the views are only used for the GET operation.

3. Next, open the `bookr/reviews/urls.py` file, remove the first two URL patterns starting with `api/`, and then add the following (highlighted) code:

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter

from . import views, api_views

router = DefaultRouter()
router.register(r'books', api_views.BookViewSet)
router.register(r'reviews', api_views.ReviewViewSet)

urlpatterns = [
    path(
        'api/', include((router.urls, 'api'))
    ),
    path(
        'books/', views.book_list, name='book_list'),
    ...
    path(
        'publishers/new/', views.publisher_edit,
        name='publisher_create'
    )
]
```

Here, we have combined the `all_books` and `book_detail` paths into a single path called `books`. We have also added a new endpoint under the `reviews` path, which we will need in a later chapter.

We start by importing the `DefaultRouter` class from `rest_framework.routers`. Then, we create a router object using the `DefaultRouter` class and then register the newly created `BookViewSet` and `ReviewViewSet` classes, as can be seen from the highlighted code. This ensures that `BookViewSet` is invoked whenever the API has the `/api/books/` path.

4. Save all the files, and once the Django service restarts (or you start it manually with the `python manage.py runserver` command), go to the `http://127.0.0.1:8000/api/books/` URL to get a list of all the books. You should see the following view in the API explorer:

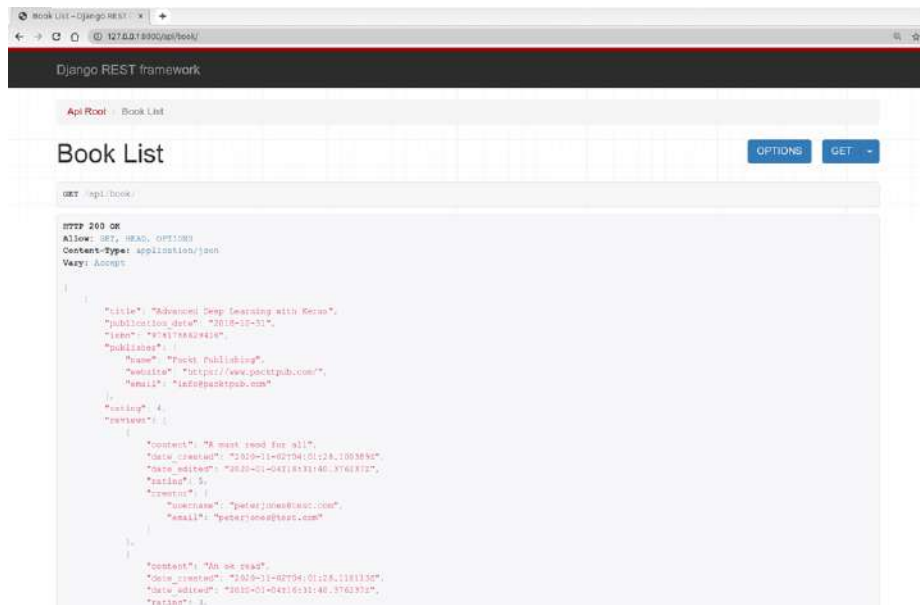


Figure 12.5: Book list at the `/api/books` path

5. You can also access the details for a specific book using the `http://127.0.0.1:8000/api/books/1/` URL. In this case, it will return details for the book with a primary key of 1 (if it exists in your database):

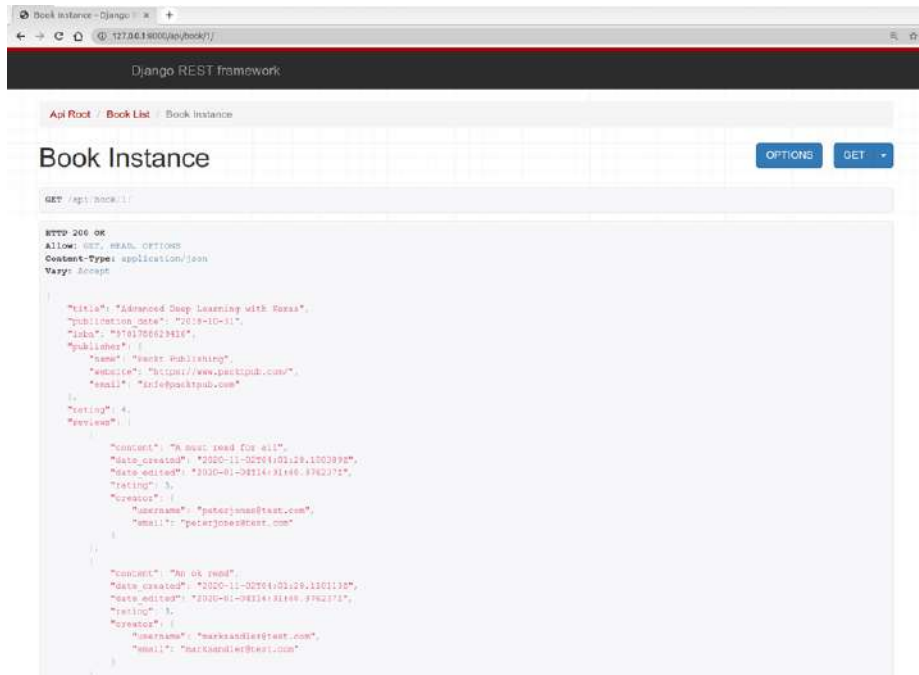


Figure 12.6: Book details for "Advanced Deep Learning with Keras"

In this exercise, we saw how we can use ViewSets and routers to combine list and detail views into a single ViewSet. Using ViewSets makes our code more consistent and idiomatic, making it easier to collaborate with other developers. This becomes particularly important when integrating with a separate frontend application. In the next section, we will learn in brief about different types of authentication and implement token-based authentication for the book review application.

Implementing authentication

As we learned in *Chapter 9, Sessions and Authentication*, it is important to authenticate the users of our application. It is good practice to only allow those users who have registered in the application to log in and access information from the application. Similarly, for REST APIs, we also need to design a way to authenticate and authorize users before any information is passed on. For example, suppose Facebook's website makes an API request to get a list of all comments for a post. If they did not have authentication on this endpoint, you could use it to get comments for any post you want programmatically. They obviously don't want to allow this, so some sort of authentication needs to be implemented.

There are different authentication schemes, such as **basic authentication**, **session authentication**, **token authentication**, **remote user authentication**, and various third-party authentication solutions. For the scope of this chapter and for our bookr application, we will use token authentication.

Note

For further reading on all the authentication schemes, please refer to the official documentation at <https://www.django-rest-framework.org/api-guide/authentication>.

Token-based authentication

Token-based authentication works by generating a unique token for a user in exchange for the user's username and password. Once the token is generated, it will be stored in the database for further reference and will be returned to the user upon every login.

This token is unique to the user, and they can then use it to authorize every API request they make. Token-based authentication eliminates the need to pass the username and password on every request. It is much safer and is best suited to client-server communication, such as a JavaScript-based web client interacting with the backend application via REST APIs.

An example of this would be a ReactJS or AngularJS application interacting with a Django backend via REST APIs.

The same architecture can be used if you are developing a mobile application to interact with the backend server via REST APIs; for instance, an Android or iOS application interacting with a Django backend via REST APIs.

Exercise 12.05 – Implementing token-based authentication for bookr APIs

In this exercise, you will implement token-based authentication for the bookr application's APIs:

1. Open the `bookr/settings.py` file and add `rest_framework.authtoken` to `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',
```



```
'rest_framework.authtoken',  
'reviews']
```

2. Since the authtoken app has associated database changes, run the migrate command in the command line/terminal as follows:

```
python manage.py migrate
```

3. Open the bookr/reviews/api_views.py file, remove any pre-existing code, and replace it with the following:

```
from django.contrib.auth import authenticate  
from rest_framework import viewsets  
from rest_framework.authentication import (  
    TokenAuthentication)  
from rest_framework.authtoken.models import Token  
from rest_framework.pagination import (  
    LimitOffsetPagination)  
from rest_framework.permissions import IsAuthenticated  
from rest_framework.response import Response  
from rest_framework.status import (  
    HTTP_404_NOT_FOUND, HTTP_200_OK)  
from rest_framework.views import APIView
```

You can find the complete code for this file You can find the complete code snippet in the Chapter12 folder in this book's GitHub repo.

Here, we have defined a view called Login. The purpose of this view is to allow a user to get (or create if it does not already exist) a token that they can use to authenticate with the API.

We override the post method of this view because we want to customize the behavior when a user sends us data (that is, their login details). First, we use the authenticate method from Django's auth library to check whether the username and password are correct. If they are correct, then we will have a user object. If not, we return an HTTP 404 error. If we do have a valid user object, then we simply get or create a token and return it to the user.

4. Next, let's add an authentication class to BookViewSet. This means that when a user tries to access this ViewSet, it will require them to authenticate using token-based

authentication. Note that it's possible to include a list of different accepted authentication methods, not just one. We also add the `permissions_classes` attribute, which just uses DRF's built-in class that checks to see whether the given user has permission to view the data in this model:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
```

Note

The preceding code (highlighted) won't match the code you see on GitHub as we'll be modifying it later in *step 9*.

5. Open the `bookr/reviews/urls.py` file and add the following path into the URL patterns:

```
path(
    'api/login', api_views.Login.as_view(),
    name='login'
)
```

6. Save the file and wait for the application to restart, or start the server manually with the `python manage.py runserver` command. Then, access the application using the `http://127.0.0.1:8000/api/login` URL. Your screen should appear as follows:

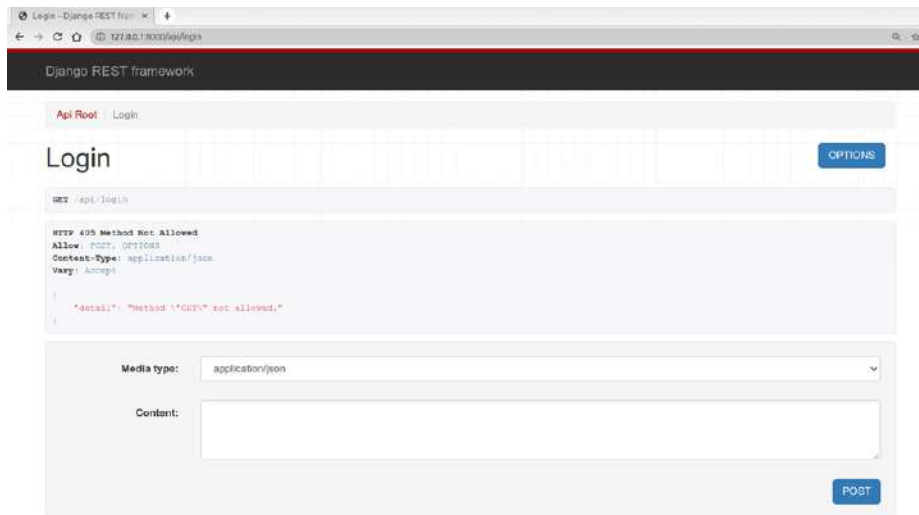


Figure 12.7: Login page

The API at `/api/login` is a POST-only message; hence, Method GET not allowed is displayed.

7. Next, enter the following snippet in the **Content** field and click on **POST**:

```
{
  "username": "Peter",
  "password": "testuserpassword"
}
```

You will need to replace this with an actual username and password for your account in the database. Now, you can see the token generated for the user. This is the token we need to use to access BookSerializer:

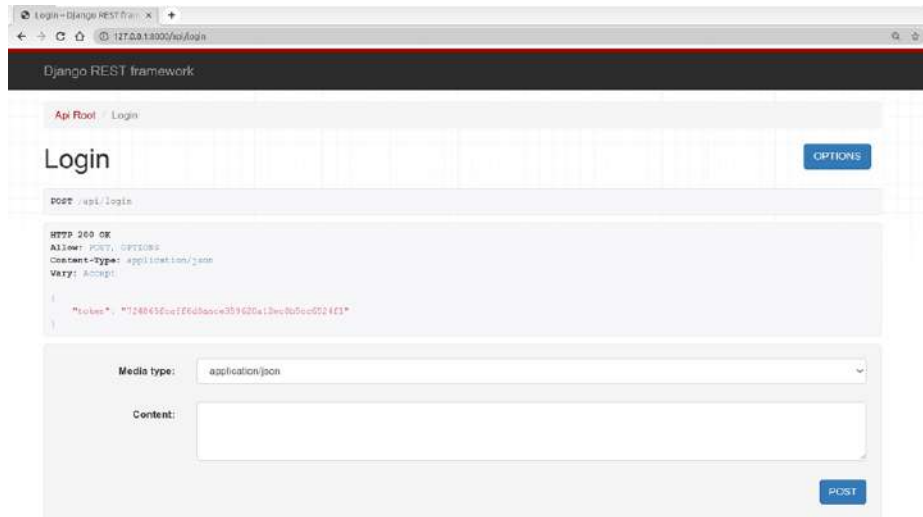


Figure 12.8: The token generated for the user

8. Try to access the list of books using the API that we previously created at `http://127.0.0.1:8000/api/books/`. Note that you are now not allowed to access it. This is because this ViewSet now requires you to use your token to authenticate. The same API can be accessed using `curl` on the command line:

```
curl -X GET http://127.0.0.1:8000/api/books/

{"detail": "Authentication credentials were not provided."}
```

Since the token was not provided, an Authentication credentials were not provided message is displayed:



Figure 12.9: Message saying that the authentication details weren't provided

Note that if you're using Windows 10, replace `curl` in the preceding command with `curl.exe` and execute it from Command Prompt.

To pass the Authorization token (obtained in *step 7*) as a header, you can use the following command (Windows users can replace `curl` with `curl.exe`):

```
curl -X GET http://127.0.0.1:8000/api/books/ -H "Authorization: Token
cd5dafa1d4361fd1502652d266eed3dcdb55c64f1"
```

Note

Before pasting this command, make sure you've replaced the token (highlighted) with the one you got when you ran *step 7* of this exercise. It will be different from the one we have shown here.

The preceding command should now return a list of books:

```
[{"title": "Advanced Deep Learning with
Keras", "publication_date": "2018-10-31", "isbn": "9781788629416", "publisher":
{"name": "Packt Publishing", "website": "https://
www.packtpub.com/", "email": "info@packtpub.com"}, "rating": 4, "reviews":
[{"content": "A must read for all", "date_created": "... (truncated)"}]
```

This operation ensured that only an existing user of the application could access and fetch the collection of all books.

9. Before moving on, set the authentication and permission classes on `BookViewSet` to an empty string. Future chapters will not utilize these authentication methods, and we will assume for the sake of simplicity that an unauthenticated user can access our API:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = []
    permission_classes = []
```

In this exercise, we implemented token-based authentication in our bookr app. We created a login view that allows us to retrieve the token for a given authenticated user. This then enabled us to make API requests from the command line by passing through the token as a header in the request.

Overall, in this section, we learned about different types of authentication, and then we learned in detail about token authentication and authorization while working with REST APIs.

Summary

This chapter introduced REST APIs, a fundamental building block in most real-world web applications. These APIs facilitate communication between the backend server and the web browser, so they are central to your growth as a Django web developer. We learned how to serialize data in our database so that it can be transmitted via an HTTP request. Next, we learned about the various options DRF gives us to simplify the code we write, taking advantage of the existing definitions of the models themselves. We also covered ViewSets and routers and saw how they can be used to condense code even further by combining the functionality of multiple views. Finally, we learned about authentication and authorization and implemented token-based authentication for the book review app.

In the next chapter, we will extend bookr's functionality for its users by learning how to generate CSVs, PDFs, and other binary file types.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

13

Generating CSV, PDF, and Other Binary Files

So far, we have learned about the various aspects of the Django framework and explored how we can build web applications using Django with all the features and customizations we want.

Let's say that, while building a web application, we need to do some analysis and prepare some reports. We may need to analyze user demographics about how the platform is being used or generate data that can be fed into machine learning systems to find patterns. We want our website to display some of the results of our analysis in a tabular format and other results as detailed graphs and charts. Furthermore, we want to allow our users to export the reports and peruse them further in applications such as Jupyter Notebook and Excel.

As we work our way through this chapter, we will learn how to bring these ideas to fruition and implement functionality in our web application that allows us to export records into structured formats such as tables through the use of **Comma-Separated Values (CSV)** files or Excel files. We will also learn how to allow our users to generate visual representations of the data we have stored inside our web application and export it in a PDF format so that it can be distributed easily for quick reference.

Let's start our journey by learning how to work with CSV files in Python. Learning this skill will help us create functionality that allows our readers to export our data for further analysis.

In this chapter, we will be covering the following topics:

- Using the csv module to render a CSV file with a Django view
- Using the zipfile module to create a ZIP file archive with a Django view
- Generating PDF files with the WeasyPrint package
- Generating Excel files with the XlsxWriter package

- Generating graphics with the Python Image Library package
- Generating graphs with Plotly
- Generating graphics with Pillow

Working with CSV files in Python

There are several reasons we may need to export the data in our application. One of the reasons may involve analyzing that data – for example, we may need to understand the demographics of users registered on the application or extract patterns of application usage. We may also need to find out how our application is working for users to design future improvements. Such use cases require data to be in a format that can be easily consumed and analyzed. Here, the CSV file format comes to the rescue.

CSV is a handy file format that can be used to quickly export data from an application in a row-and-column format. CSV files usually have data separated by simple delimiters, which are used to differentiate one column from another, and newlines, which are used to indicate the start of a new record (or row) inside the table.

Python has great support for working with CSV files in its standard library, thanks to the `csv` module. This support enables the reading, parsing, and writing of CSV files. Let's take a look at how we can leverage the `csv` module provided by Python to work on CSV files and read and write data from them.

Working with Python's `csv` module

The `csv` module from Python allows us to interact with files that are in CSV format, which is nothing but a text file format – that is, the data stored inside the CSV files is human-readable.

The `csv` module requires that the file is opened before the methods supplied by the `csv` module can be applied.

First, let's look at how we can use the Python `csv` module to create new CSV files.

Writing to CSV files using Python

Now, let's learn how we can write CSV data to files. The following steps outline the process of writing data to CSV files:

1. Open the file in writing mode by running the following script:

```
csv_file = open('path to csv file', 'w')
```

2. Obtain a CSV writer object, which can help us write data that is correctly formatted in CSV format. This can be done by calling the `writer()` method of the `csv` module, which returns a writer object, which can be used to write CSV format-compatible data to a CSV file:

```
csv_writer = csv.writer(csv_file)
```

3. Once the writer object is available, we can start writing the data. This is facilitated by the `write_row()` method of the writer object. The `write_row()` method takes in a list of values that it writes to the CSV file. The list itself indicates a single row and the values inside the list indicate the values of columns:

```
record = ['value1', 'value2', 'value3']  
csv_writer.writerow(record)
```

4. If you want to write multiple records in a single call, you can also use the `writerows()` method of the CSV writer. The `writerows()` method behaves similarly to the `writerow()` method but takes a list of lists and can write multiple rows in one go:

```
records = [['value11', 'value12', 'value13'],  
           ['value21', 'value22', 'value23']]  
csv_writer.writerows(records)
```

5. Once the records have been written, we can close the CSV file:

```
csv_file.close()
```

Fortunately, a Django `HttpResponse` object is a file-like object and can be used to create a CSV writer, so it is very easy to incorporate the `csv` module into a Django view in order to create dynamic spreadsheets from our models.

6. The `HttpResponse` constructor takes an optional argument, `content_type`, that can be used to specify a MIME type for the response. `DEFAULT_CHARSET` is 'UTF-8'. If the `content_type` and `DEFAULT_CHARSET` arguments are unspecified, Django will render the Content-Type line of the HTML response header as follows:

```
Content-Type: text/html; charset=UTF-8
```

7. The preferred MIME type for a CSV file is text/csv. We don't want to see the raw CSV data in the web browser, so we should set the content disposition to attachment, so that the CSV is downloaded with a specified filename.

```
response['Content-Disposition'] = \
    'attachment; filename="downloadfile.csv"'
```

8. Sometimes it suits our purposes to have a static filename, but in other circumstances, we might want to differentiate the filename by including a primary key or a date stamp.

```
date_stamp = datetime.datetime.now().strftime('%Y%m%d')
filename = f"download_file_{date_stamp}_{user_pk}"
response['Content-Disposition'] = f'attachment; filename="{filename}"'
```

Now let's utilize our knowledge of the csv module in order to create a summary report of the review data in CSV format.

Exercise 13.01 – Create a review summary CSV file

In this exercise, you will use the Python csv module in a Django view that returns a CSV file with a summary of book reviews.

The review summary needs to contain columns for **Book title**, **ISBN**, **Publisher name**, **Reviewer username**, and **Rating**.

We only want the book-specific information – **Book title**, **ISBN**, **Publisher name** – on the first review line of each book. Books without reviews will not be added to the report.

The column headers in the CSV file are to appear as 'title', 'isbn', 'publisher', 'reviewer' and 'rating'.

The Django view will be added to reviews/views.py.

Edit reviews/views.py and add an import statement for the csv module and make sure HttpResponse is being imported.

```
import csv

from django.http import HttpResponse
```

The data retrieval step will involve querying the Book model and searching for books with at least one review. As this data retrieval step will be used in subsequent views in this chapter, we will create it as a private function, `_review_summary`, that can be called by various view functions.

1. The first step of the view is to retrieve all of the books from the Book model.

```
def _review_summary (request):  
    # Retrieve the data  
    books = Book.objects.all()
```

2. Define the header and include it as the first row in the data structure, rows.

```
header = ['title', 'isbn', 'publisher', 'reviewer', 'rating']  
rows = [header]
```

Iterate through the books and retrieve the current book's reviews.

```
for book in books:  
    reviews = book.review_set.all()
```

If they are present, append a row containing the book title, ISBN, publisher name, and, from the first review, the reviewer username and rating.

```
if reviews:  
    rows.append([book.title, book.isbn,  
                book.publisher.name,  
                reviews[0].creator.username,  
                reviews[0].rating])
```

On subsequent reviews, leave the book title, ISBN, and publisher name blank but include the reviewer username and rating.

```
for review in reviews[1:]:  
    rows.append(['', '', '',  
                review.creator.username,  
                review.rating])  
  
return rows
```

Now that we have created the private function, `_review_summary`, that is used to retrieve the book review data, we can code the function view, `review_summary_csv`.

After the view retrieves the data from `_review_summary`, a response object is created. It needs `content_type` of `text/csv` and an attachment disposition (since we want it to be saved as a download rather than appearing in the browser. We will give the download the filename `review_summary.csv`.

```
def review_summary_csv(request):
    rows = _review_summary()
    response = HttpResponse(content_type='text/csv')
    filename = "review_summary.csv"
    response['Content-Disposition'] = f'attachment; filename="{filename}"'
```

Create a `csv.writer` object, using the response object as a file-like object.

```
# Write the CSV response
writer = csv.writer(response)
```

Write each row in `rows` to the writer.

```
for row in rows:
    writer.writerow(row)
```

Return the response.

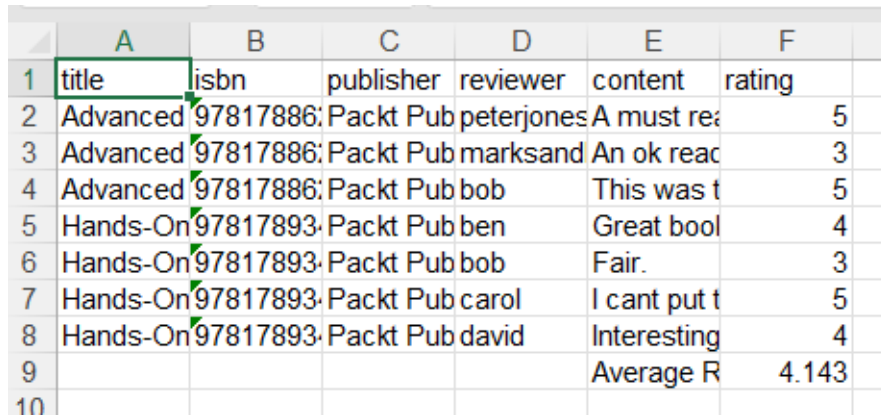
```
return response
```

The remaining step is to add the path of the summary report to `reviews/urls.py`.

```
urlpatterns = [
    ...
    path('reviews/csv/',
        views.review_summary_csv,
        name='review_summary_csv'),
]
```

The final set of changes should resemble the ones in the `Chapter13` folder in this book's GitHub repo.

Now, when you visit the `http://localhost:8000/reviews/csv/` URL, you will be prompted to download a CSV file that looks like the following one.



	A	B	C	D	E	F
1	title	isbn	publisher	reviewer	content	rating
2	Advanced	97817886	Packt Pub	peterjones	A must read	5
3	Advanced	97817886	Packt Pub	marksand	An ok read	3
4	Advanced	97817886	Packt Pub	bob	This was t	5
5	Hands-On	97817893	Packt Pub	ben	Great book	4
6	Hands-On	97817893	Packt Pub	bob	Fair.	3
7	Hands-On	97817893	Packt Pub	carol	I cant put t	5
8	Hands-On	97817893	Packt Pub	david	Interesting	4
9					Average R	4.143
10						

Figure 13.01: The CSV file imported into Microsoft Excel

Soon, we will look at how we can make our website work with the XLSX format. But before that, let's quickly take a look at the usage of binary file formats.

Binary file formats for data exports

Until now, we have worked mainly with textual data and how we can read and write it from text files. But often, text-based formats are not enough. For example, imagine you want to export an image or a graph. How will you represent an image or a graph as text, and how will you read and write these images?

To help us in these situations, binary file formats come into the picture, which can help us read and write to and from a rich and diverse set of data. All commercial operating systems provide native support for working with both text and binary file formats, and it comes as no surprise that Python provides one of the most versatile implementations to work on binary data files. A simple example of this is the `open` command, which you can use to state the format you would like to open a file in:

```
file_handler = open('path to file', 'rb')
```

Here, `b` indicates binary.

Starting from this section, we will now be dealing with how we can work on binary files and use them to represent and export data from our Django web application. The first of the formats we are going to look at is the XLSX file format made popular by Microsoft Excel.

So, let's dive into handling XLSX files with Python.

Working with XLSX files using the `XlsxWriter` package

In this section, we will learn more about the XLSX file format and understand how we can work with it using the **`XlsxWriter`** package.

While CSV has very wide usage, the degree of ambiguity in interpreting it between applications can make it a less desirable option for exchanging spreadsheet information – importantly, many of the features that we value in spreadsheets, such as worksheets, text formatting, color, and formulaic calculations.

Fortunately, it is possible to generate files in the native Excel format using Python. Microsoft Excel is the leading software application in the field of bookkeeping and tabular record management. Similarly, the XLSX file format that was adopted by Excel in 2007 has seen widespread support by many major product vendors.

XLSX files

XLSX files are binary files that are used to store tabular data. These files can be read by any software that implements support for this format. The XLSX format arranges data into two logical partitions:

- **Workbooks:** Each XLSX file is called a workbook and is supposed to contain datasets related to a particular domain. In *Figure 13.2*, `Examplefile.xlsx` is a workbook **(1)**:

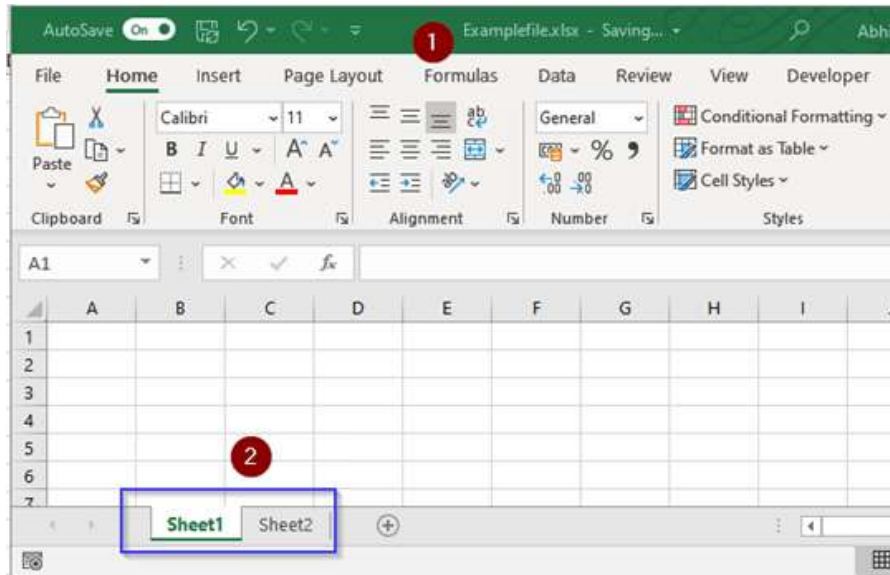


Figure 13.2: Workbooks and worksheets in Excel

- **Worksheets:** Inside each workbook, there can be one or more worksheets, which are used to store data about different but logically related datasets in a tabular format. In Figure 13.2, **Sheet1** and **Sheet2** are two worksheets (2).

When working with XLSX format, these are the two units that we generally work on. If you know about relational databases, you can think of workbooks as databases and worksheets as tables.

Now, let's try to understand how we can start working on XLSX files inside Python.

The XlsxWriter Python package

Python does not provide native support for working with XLSX files through its standard library. But thanks to the vast community of developers within the Python ecosystem, it is easy to find several packages that can help us manage our interaction with XLSX files. One popular package in this category is **XlsxWriter**.

XlsxWriter is an actively maintained package by the developer community that provides support for interacting with XLSX files. The package provides a lot of useful functionalities and supports the creation and management of workbooks, as well as worksheets in individual workbooks. You can install it by running the following command in Terminal or Command Prompt:

```
pip install XlsxWriter
```


Once installed, you can import the `xlsxwriter` module as follows:

```
import xlsxwriter
```

So, let's look at how we can start creating XLSX files with the support of the `XlsxWriter` package.

Creating a workbook

To start working on XLSX files, we first need to create them. An XLSX file is also known as a workbook and can be created by calling the `Workbook` class from the `xlsxwriter` package, as follows:

```
workbook = xlsxwriter.Workbook(filename)
```

The call to the `Workbook` class opens a binary file, specified with the `filename` argument, and returns an instance of `workbook` that can be used to further create worksheets and write data.

Creating a worksheet

Before we can start writing data to an XLSX file, we first need to create a worksheet. This can be done easily by calling the `add_worksheet()` method of the `workbook` object we obtained in the previous section:

```
worksheet = workbook.add_worksheet()
```

The `add_worksheet()` method creates a new worksheet, adds it to the `workbook`, and returns an object mapping the worksheet to a Python object, through which we can write data to the worksheet.

Writing data to the worksheet

Once a reference to the worksheet is available, we can start writing data to it by calling the `write` method of the `worksheet` object, as shown here:

```
worksheet.write(row_num, col_num, col_value)
```

As you can see, the `write()` method takes three parameters: a row number (`row_num`), a column number (`col_num`), and the data that belongs to the (`row_num`, `col_num`) pair, as represented by `col_value`. This call can be repeated to insert multiple data items into the worksheet.

Writing data to the workbook

Once all the data has been written, to finalize the written datasets and cleanly close the XLSX file, you must call the `close()` method on the workbook:

```
workbook.close()
```

This method writes any data that may be in the file buffer and finally closes the workbook. Now, let's use this knowledge to implement our own code, which will help us write data to an XLSX file.

Note

It's not possible to cover all the methods and features the `XlsxWriter` package provides in this chapter. For more information, you can read the official documentation: <https://xlsxwriter.readthedocs.io/contents.html>.

Formatting a spreadsheet

Let's revisit the spreadsheet from the CSV example and see how it could be implemented with additional XLSX formatting using `xlsxwriter`.

First, we need to install the `xlsxwriter` module. It is available in PyPi and can be installed quickly using `pip`, or it can be found on GitHub at <https://github.com/jmcnamara/XlsxWriter>.

```
pip install xlsxwriter
```

This section is not aimed at giving a comprehensive documentation of this module, but it gives a simple outline of the features that we are likely to encounter.

We begin by importing the `xlsxwriter` module, then creating a workbook by passing a filename or file-like object to the constructor.

```
import xlsxwriter
workbook = xlsxwriter.Workbook('output.xlsx')
```

We can then create specific worksheets in the workbook with the `add_worksheet` method.

```
worksheet = workbook.add_worksheet('Review Summary')
```

The worksheet class contains a lot of methods for working with worksheets. If you are aiming to print your worksheet, or at least save it to PDF, the `set_landscape` and `set_portrait` methods come in handy.

We can also set specific column widths using the `set_column` method. The default width is approximately 8.43 characters in the default font of Calibri 11. We can specify a width based on characters for a range of columns in the spreadsheet.

So, for example, if we want the first column to have the default width, but the second to fourth columns to have a width of 20 characters, followed by a width of 50 characters for the eighth column, we can use the `worksheet.set_column` method to accomplish this.

```
worksheet.set_column(1, 3, 20)    # Columns B:D width 20
worksheet.set_column(7, 7, 50)    # Column H width 50
```

As well as numbering columns in the spreadsheet from zero and specifying two integer values for the column range, it is also possible to specify a column using Excel's A1 notation, so this could equivalently be written like this:

```
worksheet.set_column('B:D', 20)   # Columns B:D width 20
worksheet.set_column('H:H', 50)   # Column H width 50
```

As a rule, it is easier to use the A1 notation if we are hardcoding specific column details in our code, but if we are writing spreadsheets where we are dynamically formatting cells, the numeric co-ordinate approach is simpler.

Fortunately, `xlsxwriter` provides utilities functions that translate between the numeric and the A1 grid co-ordinate systems.

```
>>> xlsxwriter.utility.xl_col_to_name(4)
'E'
>>> xlsxwriter.utility.xl_rowcol_to_cell(4,6)
'G5'
>>> xlsxwriter.utility.xl_rowcol_to_cell(26,3)
'D27'
>>> xlsxwriter.utility.xl_rowcol_to_cell(3,26)
'AA4'
>>> xlsxwriter.utility.xl_cell_to_rowcol('A1')
(0, 0)
>>> xlsxwriter.utility.xl_cell_to_rowcol('AA10')
(9, 26)
```

```
>>> xlswriter.utility.xl_range(0, 4, 5, 4)
'E1:E6'
```

These utilities come in very handy when we are trying to specify Excel formulae that require the A1 notation.

Note

Always use the `xlswriter.utility` functions for row col to A1 conversion. You may be tempted to write your own simple code to work out the cell name, but such code is error-prone, especially when the you utilize it for high-range values that you have not adequately tested.

Once we have a worksheet, we can assign specific values to cells, rows, and columns in the worksheet using the `worksheet.write`, `worksheet.write_row`, and `worksheet.write_column` methods. All of these methods use either the A1 notation or the coordinate notation for writing cells, as we can see in the following example, where it is easier to use the A1 notation to explicitly specify rows or columns, but if we need to dynamically determine the cell location, then it is easier to specify by (row, column) co-ordinates.

The Python code uses these write methods to produce the Excel spreadsheet.

```
import xlswriter

workbook = xlswriter.Workbook('WriteRowsAndColumns.xlsx')
worksheet = workbook.add_worksheet('Title')

worksheet.write_column('A1', list('WEB'))
worksheet.write_row('B4', list('DEVELOPMENT'))
worksheet.write_column('A5', list('WITH'))
worksheet.write_row('B9', list('DJANGO'))

for i, letter in enumerate('PACKT'):
    worksheet.write(12, i*2, letter)

workbook.close()
```

This brings us to cell formatting. `xlswriter` includes a `format` class that implements all of the formatting that we encounter in Excel spreadsheets – fonts, color, font size, bold, italic, struck-out text, date representation, numerical formatting, and so on. We can create a `Format` object by using the `add_format` method on the `worksheet` class. We can either pre-specify the format

by calling `add_format` with a dictionary argument that contains the formatting parameters, or we can create an empty format and then override the defaults with specific methods on the format object. The following two code snippets show equivalent ways of creating the same format.

```
format_dict = {'bold': True,          # set bold
               'font_size': 12,      # specify font size
               'bg_color': 'gray'}) # set background color
header_format = workbook.add_format(format_dict)

header_format = workbook.add_format()
header_format.set_bold()
header_format.set_font_size(12)
header_format.set_bg_color('gray')
```

Once we have a format object, we can specify it as the final argument of one of the write methods.

```
worksheet.writerow('A1', header, header_format)
```

For a better understanding of the `Format` class, you might read the comprehensive documentation at <https://xlsxwriter.readthedocs.io/format.html>. The remaining examples and exercises in this section will only use the properties dictionary to specify formatting.

We are not limited to populating spreadsheets with static content. `xlsxwriter` includes the ability to specify Excel formulae in specific worksheet cells.

```
worksheet.write_formula('F9', '=RAND()')
```

The `XlsxWriter` reference material at <https://xlsxwriter.readthedocs.io/> can help you unlock the power of the XSLX file format and make amazing spreadsheets. We have barely scratched the surface of `xlsxwriter`, but have learned enough to recreate the review CSV file as an Excel spreadsheet, augmenting it with formatting and dynamic calculations.

Now that we have surveyed the basic functionality of the **XlsxWriter** package, we can revisit the review summary report from *Exercise 13.01* and implement it as a formatted spreadsheet.

The format settings need to be mapped to individual columns in the report. So, title, reviewer, and content will all use the short_text format, while isbn and rating will have their own specific numeric formats.

```
# title, isbn, publisher, reviewer, content, rating
col_formats = ('short_text', 'isbn', 'short_text',
               'short_text', 'content', 'rating')
```

We create a BytesIO in-memory buffer to store the spreadsheet. The buffer will be written to by `xlsxwriter.Workbook`.

```
output = BytesIO()
workbook = xlsxwriter.Workbook(output)
```

We will call a `format_workbook` utility function that takes the workbook objects, the data rows, and our formatting information as parameters.

```
workbook = format_workbook(
    workbook, rows, format_settings,
    reader.fieldnames, col_formats, column_widths,
    footer_avg=dict(label='Average Rating',
                    id='rating'))
```

At the end of the process, we close the workbook, rewind the buffer, and attach the buffer with the appropriate disposition and content type to the HTTP response.

```
workbook.close()
# Rewind the buffer.
output.seek(0)
# Create the HTTP response.
filename = 'review_details.xlsx'
ctype = ('application/vnd.openxmlformats-
         'officedocument.spreadsheetml.sheet')
response = HttpResponse(output, content_type=ctype)
response['Content-Disposition'] = \
    f'attachment; filename={filename}'
return response
```

Now we can define the `format_workbook` method in the `reviews/utils.py` file. We begin by importing the necessary helper functions from `xlswriter.utility`, which will be used for converting from numeric co-ordinates to the A1 cell notation.

```
from xlswriter.utility import xl_rowcol_to_cell, xl_range
```

The `format_workbook` method takes arguments for workbook objects, the rows data, `format_settings`, `col_formats`, and `column_heights` that we created in the `review_summary_xlsx` function view. The optional `footer_avg` argument is used to specify whether there will be an average calculation in the footer.

```
def format_workbook(workbook, rows, format_settings,
                    header, col_formats, column_widths, footer_avg=None):
```

A worksheet called 'Review Summary' is added to the workbook.

```
worksheet = workbook.add_worksheet('Review Summary')
```

The `formats` parameters supplied by `format_settings` are used to define a dictionary of formats.

```
formats = {format_id: workbook.add_format(format_set)
            for format_id, format_set in
            format_settings.items()}
```

The column widths specified in `column_widths` are applied to the worksheet.

```
for col_range, width in column_widths:
    worksheet.set_column(col_range, width)
```

The header is written to the file and formatted.

```
worksheet.write_row('A1', header, formats['header'])
for col_num, cell_data in enumerate(header):
    worksheet.write(0, col_num, cell_data,
                   formats['header'])
```


Each row retrieved from `_review_summary` is added to the worksheet, with formats being applied based on their specification in `col_formats`.

```
for row_num, row in enumerate(rows):
    if not row:
        continue
    for col_num, col in enumerate(
        list(row.values())):
        worksheet.write(row_num+1, col_num, col,
                        formats[col_formats[col_num]])
```

We can include the average calculation in the worksheet before returning the formatted workbook.

```
# Add footer with average rating
if footer_avg:
    row_id = len(rows) + 1
    col_id = header.index(footer_avg['id'])
    col_label_id = (col_id - 1 if col_id > 0
                    else len(col_formats))
    worksheet.write(
        row_id,
        col_label_id,
        f"{footer_avg['label']}: ",
        formats['footer'])
    avg_cell = xl_rowcol_to_cell(len(rows)+1, col_id)
    avg_range = xl_range(1, col_id, row_id-1, col_id)
    worksheet.write_formula(avg_cell,
                            f"AVERAGE({avg_range})")

return workbook
```

Now the view has been created, we need to include the appropriate URL path in `reviews/urls.py`.

```
path('reviews/xlsx/', views.review_summary_xlsx,
     name='review_summary_xlsx'),
```

The final set of changes should resemble the ones in the `Chapter13` folder in this book's GitHub repo.

Now when you visit the `http://localhost:8000/reviews/xlsx/` URL, you will be prompted to download a CSV file that looks like the following one.

	A	B	C	D	E	F
1	title	isbn	publisher	reviewer	content	rating
2	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	peterjones	A must read for all	5
3	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	marksandler	An ok read	3
4	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	bob	This was the best book on Deep Learning that I have read.	5
5	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	ben	Now I am familiar with Keras.	4
6	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	bob	Great book!	4
7	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	bob	Fair.	3
8	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	carol	I cant put this one down..	5
9	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	david	Interesting book.	4
10					Average Rating:	4.143

Figure 13.3: The XLSX spreadsheet

We have seen in this example that Django's MVT architecture is flexible enough to allow the creation of a binary response. We will next apply this approach to create ZIP file archives from within Django.

ZIP files

The ZIP archive format has been in popular use since the release of PKZIP in 1989. ZIP files contain either single compressed files or collections and preserve directory structure.

The `zipfile` module is part of the Python standard library and it contains the resources to generate ZIP files programmatically. The module contains a `ZipFile` class that can be used to read, write, and append to ZIP files.

The `ZipFile` constructor takes a mandatory file argument and three optional arguments:

- **file:** This can either be the path of the ZIP file or a file-like object
- **mode:** Either `'r'`, `'w'`, `'a'`, or `'x'`, for read, write, append, or exclusive (the default is `'r'`)
- **compression:** The `zipfile` module defines five constants representing the type of compression that is applied to the ZIP archive. These are as follows:
 - `zipfile.ZIP_STORED` (no compression)
 - `zipfile.ZIP_DEFLATED` (zlib)
 - `zipfile.ZIP_BZIP2` (bzip2)
 - `zipfile.ZIP_LZMA` (LZMA)
 - `zipfile.ZIP_ZSTD` (compression.zstd)

The default is no compression. As not all ZIP software clients use the bzip2, LZMA, and compression.zstd libraries, `zipfile.ZIP_DEFLATED` is the safest option if you intend to compress your ZIP file.

- `allowZipFile64`: This is a boolean to allow files with ZIP64 extensions. By default, this is `True`.

Consider a situation where a directory contains the following files.

```
csv/book_list.csv
csv/reviews_2010.csv
csv/reviews_2011.csv
csv/reviews_2012.csv
csv/reviews_2013.csv
...
csv/reviews_2025.csv
```

We want to programmatically create a ZIP archive that only stores the files that match the wildcard `csv/reviews_20???.csv`. Fortunately, Python has a module for this kind of wildcard matching called `glob`.

This simple program solves our problem.

```
from zipfile import ZipFile, ZIP_DEFLATED
import glob

file_paths = glob.glob('csv/reviews_20???.csv')
with ZipFile('all_reviews.zip', 'w', ZIP_DEFLATED) as z:
    for file_path in file_paths:
        z.write(file_path)
```

As in the case of the CSV data, we can easily create a Django view that will take a request and write a ZIP file to a response.

```
import datetime
import glob
from io import BytesIO
from zipfile import ZipFile

from django.http import HttpResponse
```

```
def review_zip(request):
    output = BytesIO()
    file_paths = glob.glob('csv/reviews_20???.csv')
    with ZipFile(output, 'w') as z:
        for file_path in file_paths:
            z.write(file_path)

    # Create the HTTP response.
    date_stamp = datetime.datetime.now().strftime('%Y%m%d')
    filename = f'all_reviews_{date_stamp}.zip'
    ctype = 'application/zip'
    cdisposition = f'attachment; filename={filename}'
    response = HttpResponse(output, content_type=ctype)
    response['Content-Disposition'] = cdisposition
    return response
```

Working with PDF files in Python

Portable Document Format (PDF) was developed in the late 1980s by Adobe Inc. in order to make a uniform page description format for printing that would operate over a variety of operating systems, firmware, and printers.

There are numerous modules available for generating dynamic PDF content. Many of these involve learning complex APIs in order to build a PDF object programmatically. As we are in the process of learning about Django and have delved deeply into its templating mechanisms, it would be nice if we could leverage this knowledge in order to generate PDF files.

Fortunately, there are several packages that will generate PDF files based on text content such as HTML or markup. One Python package that is actively being developed at the time of writing is WeasyPrint.

Again, WeasyPrint is extensively documented and interested developers can visit <https://weasyprint.readthedocs.io/> for more information. The examples here are only a brief, simple introduction to the power of WeasyPrint.

WeasyPrint can be installed using pip (or obtained on GitHub at <https://github.com/Kozea/WeasyPrint>).

```
pip install weasyprint
```

WeasyPrint can even be used at the command line to turn an HTML page into a PDF document without any coding involved. All that is required is to specify weasyprint <url> <pdffile>.

Results may vary as WeasyPrint does not implement all HTML formatting exactly as you might see it in a browser, and it is unaware of JavaScript or other dynamic content inside a website.

```
weasyprint https://python.org python_org.pdf
```

If we wanted to achieve what we did on the command line with a Python script, we could use the following piece of code.

```
from weasyprint import HTML, CSS
html = HTML('https://python.org')
html.write_pdf('python_org.pdf')
```

The HTML class in weasyprint is also capable of taking any local files, as well as raw HTML string content, and using those files to generate PDFs. We will use this capability in the next exercise to produce a book detail PDF that is generated from within the bookr app.

Note

This is a very brief introduction to using WeasyPrint and covers little of its extensive functionality. For further information, please visit the WeasyPrint documentation at <https://weasyprint.readthedocs.io>.

Exercise 13.03 – Generating a PDF version of a web page in Python

In the next example, we will make use of this technique and generate a PDF file that contains information about a specific book and its reviews in the Bookr system. We are familiar with this code as it just follows the Django MVT pattern, but this time the view uses WeasyPrint to render the response as a PDF rather than HTML.

1. Add a `ratings_to_stars` method to `Review` in `model.py`.

```
def rating_to_stars(self):
    if not self.rating:
        return ''
    elif isinstance(self.rating, int):
        return '*' * self.rating
    return self.rating
```

2. Create a new Django template in reviews/templates/reviews/book_detail_pdf.html.

```
<html>
<head>
  <title>Bookr - {{ book.title }} </title>
</head>
<body>
  
  <div>
    <h1>{{ book.title }}</h1>
    <p>Published: {{ book.publication_date }}</p>
    <p>ISBN: {{ book.isbn }}</p>
    <p>Publisher: {{ book.publisher.name }} </p>
    <p>{% if book.contributors.all %}By {% endif %}
      {% for contributor in book.contributors.all %}
        {{ contributor.first_names }}
        {{ contributor.last_names }} {% endfor %}</p>
  </div>
  <div ></div>
  <div>
    {% if reviews %}
    <h2>Reviews:</h2>
    {% endif %}
    {% for review in reviews %}
      <p class="review-content" >{{review.content }}</p >
      <p class="review-creator">
        {{ review.creator.username }}
        {{ review.rating_to_stars }}</p>
      <p class="review-date">{{ review.date_edited }}</p>
    {% empty %}
      <h2> No reviews </h2>
    {% endfor %}
  </div>
</body>
</html>
```

3. We need to add a view function, book_detail_pdf, to reviews/views.py.

4. First, we need to import the HTML and CSS classes from weasyprint.

```
from weasyprint import HTML, CSS
...
```

5. We have an existing view function called `book_detail`. All of the code in this function is applicable to our current function, except for the final line that renders the view as HTML.

```
return render(request, "reviews/book_detail.html",
               context)
```

6. So, we will refactor this existing code so that the `book_detail` function calls a private function, `_book_detail(request, pk)`, which returns the request object and the context dictionary. We will reuse `_book_detail` when we write the `book_detail_pdf` function.

```
from django.template.loader import get_template
from weasyprint import HTML, CSS
...
def _book_detail(request, pk):
    book = get_object_or_404(Book, pk=pk)
    ...
    if request.user.is_authenticated:
        ...
        request.session['viewed_books'] = viewed_books
    return request, context

def book_detail(request, pk):
    request, context = _book_detail(request, pk)
    return render(request, "reviews/book_detail.html",
                  context)
```

7. With the package now installed, create a new file named `book_detail_pdf` that will hold the PDF generation logic. Inside this file, write the following code:

```
def book_detail_pdf(request, pk):
    request, context = _book_detail(request, pk):
    # Render the HTML
    template = get_template(
```

```

        'reviews/book_detail_pdf.html')
html = template.render(
    context=context, request=request)
css = CSS(string='')

# Create a response
fname=f"book_detail_{book.isbn or book.id}.pdf"
cdisposition = f"inline; filename={fname}"
response = HttpResponse(
    content_type='application/pdf')
response['Content-Disposition'] = cdisposition
response['Content-Transfer-Encoding'] = 'binary'

# Write the PDF binary to the response
return HTML(
    string=html,
    base_url=request.build_absolute_uri()
).write_pdf(response, stylesheets=[css])

```

Now, let's try to understand what this code does. In the first line, we imported the HTML class from the weasyprint package, which you installed in *Step 1*:

```
from weasyprint import HTML
```

This HTML class provides us with a mechanism through which we can read the HTML content of a website if we have its URL.

In the next step, we created a new method named `book_detail_pdf()` that takes in two parameters – the request object and the primary key of the book to generate a PDF for:

```
def book_detail_pdf(request, pk):
```

Next, we passed the rendered HTML to the HTML class object we imported earlier. This caused the HTML string to be parsed by the weasyprint library and caused its HTML content to be read. Once this was done, we called the `write_pdf()` method of the HTML class object and provided the name of the file to which the content should be written:

```
pdf_stream = HTML(string=html,
base_url=request.build_absolute_uri()).write_pdf(stylesheets=[css])
```

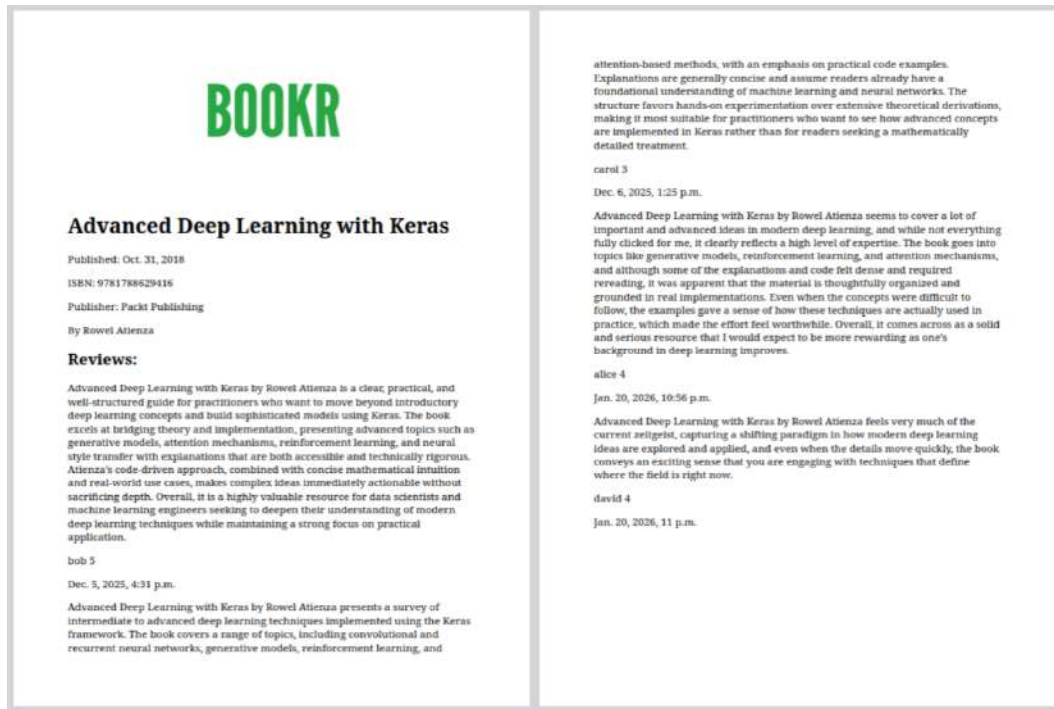



Figure 13.4: The WeasyPrint generated book detail PDF

Activity 13.01 – Adding additional functionality to a View

Including summary information in a report makes the information easier for a user to comprehend and to make comparisons with other reports.

This activity will focus on adding additional functionality to a view to include aggregated data reporting in the PDF. At the top of each Review section in the Book Detail PDF, add a simple chart that displays how many stars each movie received. This graph will be implemented as an HTML table.

The collections module in the Python Standard library provides a Counter class. You can use this class to create a function in reviews/utils.py called ratings_to_histogram. ratings_to_histogram is passed a list of reviews ratings and produces a list of values that will be used in the template to produce a chart that shows the distribution of ratings for the book.

```
>>> ratings_to_histogram([4, 5, 3, 4])
[('*****', 1), ('*****', 2), ('****', 1), ('***', 0), ('**', 0)]
```

Include `book_rating_histogram` as a key in the context of the `book_detail_pdf` view in `reviews/views.py`. It will have a value, as follows:

```
ratings = [review.rating for review in reviews]
book_rating_histogram = ratings_to_histogram(ratings)
```

In `reviews/templates/reviews/book_detail_pdf.html`, make the following changes to incorporate the chart.

Ensure that the chart only appears on books that have received at least one review.

The histogram data can be displayed in a table with two columns inside a `div` tag with an HTML ID of `histogram`. The left column will contain the stars and the right column will contain their count. This can be achieved by iterating using a `for` loop like this:

```
{% for stars, count in book_rating_histogram %}
```

Give the existing `div` blocks of data HTML IDs of `logoheader`, `bookinfo`, `bookcover`, and `reviews`. Put the title in a `div` of its own called `title`. Add a background color to the `logoheader` `div` so that the white parts of the logo are visible.

Include some CSS so that the histogram `div` floats to the right of `bookinfo`. To contrast with the rest of the report, use white text on a dark gray background.

When you have finished this activity, you should see a simple chart like this inset in the PDF.



Figure 13.5: Ratings chart

The overall design will now look something like the PDF in *Figure 13.6*.

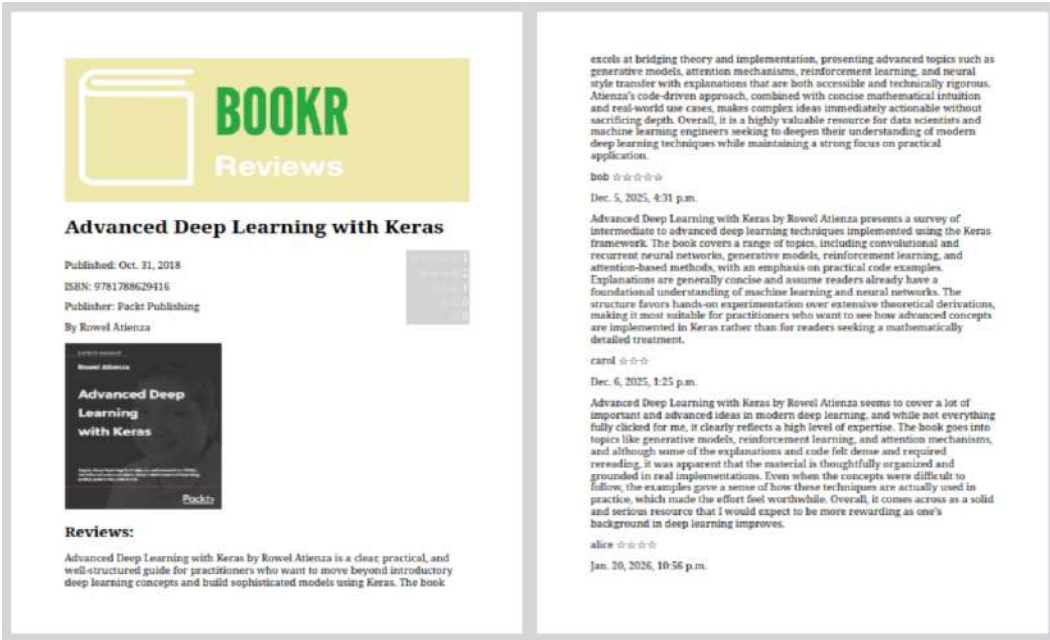


Figure 13.6: Ratings chart

So far, we have learned how we can generate different types of binary files with Python, which can help us export our data in a structured manner or help us print PDF versions of our pages. Next, we will learn how to generate graph representations of our data using Python.

Note

The solution for this activity can be found at in the Chapter13 folder in this book's GitHub repo.

Generating graphs with Plotly

Graphs are a great way to visually represent data that changes within a specific dimension. We come across graphs quite frequently in our day-to-day lives, be it weather charts for a week, stock market movements, or student performance report cards.

Plotly is a popular library for producing JavaScript enriched visualizations. It is implemented in both Python and the R programming language that is popular with statisticians and scientists. It enables the rapid visualization of data with dozens of configurable charts, including histograms, pie charts, scattergraphs, Venn diagrams, heatmaps, and many 3D plots. One of its

benefits is that `plotly` is particularly interesting to us due to its ease of integration with Django.

We will start with a simple pie chart example.

Suppose we have a dataset with the ratings that users have assigned to reviews in Bookr, as per *Figure 13.7*.

Rating	Count
*****	510
****	735
***	475
**	386
*	235

Figure 13.7: The star ranking dataset

Installation

To install it on your system, you can run the following command:

```
pip install plotly
```

Now that you've done that, let's take a look at how we can generate a graph visualization using `plotly`.

Setting up a figure

Before we can start generating a graph, we need to initialize a `plotly` `Figure` object, which essentially acts as a container for our graph. A `plotly` `Figure` object is quite easy to initialize; it can be done by using the following code snippet:

```
from plotly.graph_objs import graphs
figure = graphs.Figure()
```

The `Figure()` constructor from the `graph_objects` module of the `plotly` library returns an instance of the `Figure` graph container, inside which a graph can be generated. Once the `Figure` object is in place, we must generate a plot.

Generating a plot

A plot is a visual representation of a dataset. This plot could be a pie chart, a scatter plot, a line graph, and so on. To implement a pie chart of the data in *Figure 13.7*, you can use the following code snippet:

```
labels = ['*****', '****', '***', '**', '*']
values = [510, 735, 475, 386, 235]

pie_plot = go.Pie(labels=labels, values=values, sort=False)
figure.add_trace(pie_plot)
figure.show()
```

The `add_trace()` method is responsible for adding a plotting object to the figure and generating its visualization inside the figure. By default, when the `Figure.show()` method is called, the figure will be rendered in a web browser.

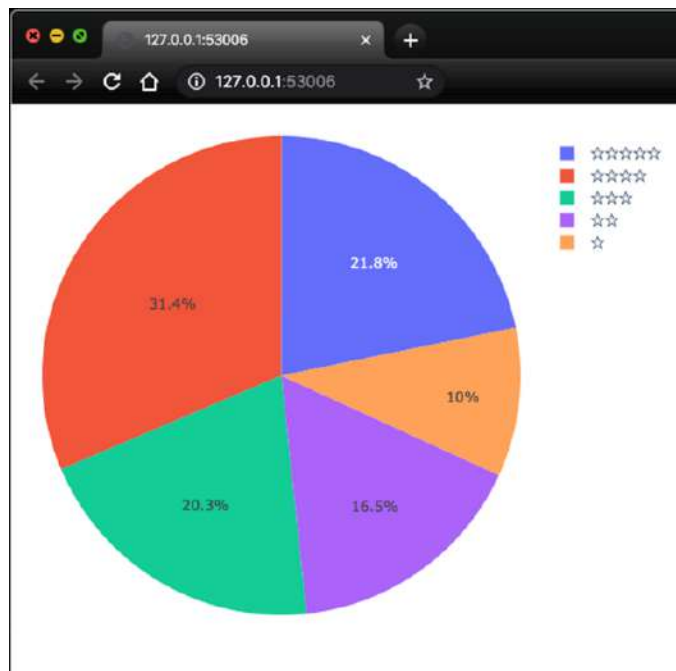


Figure 13.8: A plotly pie chart

Rendering a plot on a web page

Once the plot has been added to the figure, it can be rendered on a web page by calling the `plot()` method from the offline plotting module of the `plotly` library. This is shown in the following code snippet:

```
from plotly.offline import plot
visualization_html = plot(figure, output_type='div')
```

The `plot()` method takes two primary parameters: the first is the figure that needs to be rendered and the second is the HTML tag of the container inside which the figure HTML will be generated. The `plot` method returns fully integrated HTML that can be embedded in any web page or made a part of the template to render a graph.

Now, with this understanding of how graph plotting works, let's try a hands-on exercise where we will generate a graph for our sample dataset.

Advanced graphs – bubble plots

For our next example, we will develop a bubble plot. This is a scatter plot with weighted markers. In the dataset we are dealing with, we will be plotting average reviews for a variety of books and setting the size of the markers.

For our dataset, we will use this CSV file, which contains information on books and their ratings.

```
publication_year,average_rating,title,num_rating
2014,2.4285714285714284,'Book A',7
2016,2.25,'Book B',4
2018,4.0,'Book C',5
2016,2.3333333333333335,'Book D',3
2011,4.0,'Book E',1
2015,2.875,'Book F',16
2019,2.6315789473684212,'Book G',19
2020,3.375,'Book H',16
2018,5.0,'Book I',1
2011,2.6666666666666665,'Book J',9
2013,5.0,'Book K',1
2013,3.5384615384615383,'Book L',13
2016,2.5789473684210527,'Book M',19
2010,2.2941176470588234,'Book N',17
2014,3.3333333333333335,'Book O',12
2015,3.0526315789473686,'Book P',19
```

```
2020,2.857142857142857,'Book Q',7
2018,3.0,'Book R',16
2015,2.5,'Book S',18
2020,3.230769230769231,'Book T',13
```

We need to import `plotly` and the `graph_objects` module and then read the CSV file.

```
import csv
import plotly as py
import plotly.graph_objects as go

with open('data.csv') as f:
    reader = csv.DictReader(f)
    reviews = list(reader)
```

The `Scatter` constructor takes in the values for the *x* axis and *y* axis and returns an object that can be used to build a scatter plot. In our example we are plotting the publication year on the *x* axis and the average rating on the *y* axis. We are defining the marker using the number of ratings present in the dataset. This way, books with more ratings will look more prominent than those with fewer ratings.

```
scatter_0 = go.Scatter(
    x=[review['publication_year'] for review in reviews],
    y=[review['average_rating'] for review in reviews],
    text=[review['title'] for review in reviews],
    mode='markers+text',
    textposition='middle center',
    marker=dict(
        size=[int(review['num_rating'])
              for review in reviews],
        sizemode='area',
        sizeref=0.001,
    )
)
```

Once the scatter plot object has been generated, the next step is to add this plot to `Figure`. This can be done as follows:

```
figure = go.Figure()
figure.add_trace(scatter_0)
```

```
figure.update_xaxes(range=[2009, 2021])
figure.update_yaxes(range=[-0.2, 5.2])
figure.show()
```

The resulting bubble plot will appear as in *Figure 13.9*.

In the next exercise, we will use incorporate this kind of plotly object into a Django view that takes its data from a dynamic query of the database.

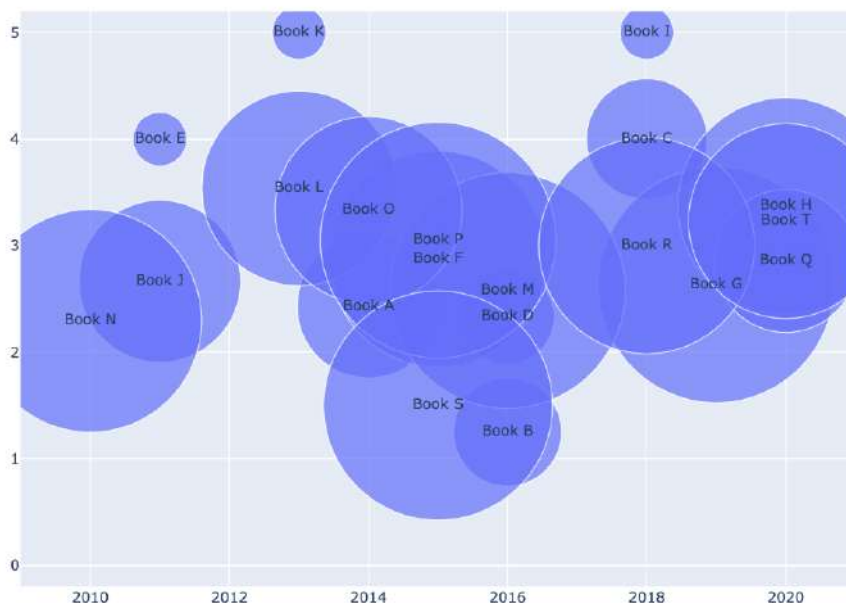


Figure 13.9: The bubble plot in Plotly

Exercise 13.4 – Generating graphs in Python

In this exercise, we will add a Django view that creates a bubble plot of the bookr app's **Review** data.

1. Edit `reviews/views.py` and include the required `import` statements.

```
import datetime

import plotly.graph_objects as go
from django.db.models import Avg
from plotly.offline import plot
```


2. Define the `review_statistics` view with the arguments for `min_publication_year` and `max_publication_year`. Calculate the start of the first year and the end of the second year so that we can do inclusive searches on this date range. (For example, if `max_publication_year` is 2025, we want our search to include publications from up to the last second of December 31, 2025.)

```
def review_statistics(request,
    min_publication_year, max_publication_year):
    min_year = datetime.datetime(
        min_publication_year, 1, 1)
    max_year = datetime.datetime(
        max_publication_year, 12, 31, 23, 59, 59)
```

3. Now we can retrieve the reviews filtered on our date range and include aggregates of the average rating and the count of ratings.

```
reviews = Review.objects.filter(
    book__publication_date__gte=min_year,
    book__publication_date__lte=max_year).values(
    'book__id', 'book__title',
    'book__publication_date'
).annotate(average_rating=Avg('rating'))
).annotate(num_rating=Count('rating'))
```

4. We can create the scatter plot using the class declaration that we discussed in the prior section.

```
scatter_0 = go.Scatter(
    x=[review['book__publication_date'].year
        for review in reviews],
    y=[review['average_rating']
        for review in reviews],
    text=[review['book__title']
        for review in reviews],
    mode='markers+text',
    textposition='middle center',
    marker=dict(
        size=[review['num_rating']
            for review in reviews],
        sizemode='area',
```

```
        sizeref=0.001,
    )
)
```

5. Now we can create the layout and figure and annotate the axes with the publication date and rating scales.

```
layout = go.Layout(autosize=True, height=900)
fig = go.Figure(data=[scatter_0], layout=layout)
fig.update_xaxes(range=[min_publication_year,
                        max_publication_year])
fig.update_yaxes(range=[-0.2, 5.2])
```

6. We will create a div element and render it using a Django template.

```
plotly_div = plot(fig, output_type='div')
return render(request, "reviews/statistics.html",
              context={'plotly_div': plotly_div})
```

7. The Django template can be defined as follows.

```
<!DOCTYPE HTML>
<html>
<head>
    <title>Review Bubbleplot</title>
</head>
<body>
    {% autoescape off %}
    {{ plotly_div }}
    {% endautoescape %}
</body>
</html>
```

8. Now we will need to add the following path to `reviews/urls.py`:

```
path('reviews/statistics/'
     '<int:min_publication_year>/'
     '<int:max_publication_year>',
```

```
views.review_statistics,  
name='review_statistics'),
```

9. By using `plotly` in a Django view, we have opened our Django applications to the sophisticated data visualizations that this library offers. Now we will learn about dynamic image generation using the Pillow library.

Generating images in Python with Pillow

We encountered Pillow in *Chapter 8, File Uploads with Django Forms*, where we used it to resize uploaded images.

While Pillow contains an extensive array of functionality for image creation and manipulation, we won't investigate these aspects in their entirety. The examples in this section look at simple image transformations such as rotation and resizing.

First, we need to import the `Image` class from the PIL module.

```
from PIL import Image
```

Now, we will define a private function that takes two image paths as arguments.

```
def _logo_transformations(logo, background):
```

We will load these images using the `Image.open` method and convert them to a common color mode.

```
bg = Image.open(background)  
bg_img = bg.convert('RGBA')  
logo_img = Image.open(logo).convert('RGBA')
```

Now we will paste the logo onto the background at position (100, 50).

```
bg_img.paste(logo_img, box=(100, 50), mask=logo_img)
```

Then we will rotate another logo image 135 degrees counterclockwise and paste it further to the right of the first logo.

```
rot = logo_img.rotate(135, Image.NEAREST, expand=1)  
bg_img.paste(rot, box=(650, 50), mask=rot)
```

Then we will resize the logo so that it has a 1:1 aspect ratio, paste it at the bottom of the background, and return the completed image.

```
resize = logo_img.resize((216, 216), Image.NEAREST)
bg_img.paste(resize, box=(100, 400), mask=resize)

return bg_img
```

Finally, we have the code to call the private method.

```
# bookr Logo: 432 x 216 pixels
logo = 'reviews/static/reviews/logo.png'
# wooden background: 1280 x 720 pixels
background = 'media/backgrounds/wood_background.jpg'
transformed_img = _logo_transformations(logo, background)
transformed_img.save('logo_transforms.png', quality=95)
```

The wood background image is available in the Chapter13 folder in this book's GitHub repo.

Running this code will create the following image.



Figure 13.10: *logo_transformation.png*

You will realize very quickly that this is not a very intuitive way to manipulate graphics, and it is very time-consuming to get even a basic effect when using this library compared to the time

taken with a standard image application such as Microsoft Paint or Adobe PhotoShop. The benefit of this approach is that it is very easy to automate this code and apply it to scripting images or dynamically rendering an image on a web server.

As we saw in the earlier examples, it is very easy to wrap this kind of binary data in a Django `HttpResponse`. Here is a view that we can add to `reviews/views.py` that uses the `_logo_transformations` private function and returns a PNG image.

```
from PIL import Image
...

def logo_transformations(request):
    logo = 'reviews/static/reviews/logo.png'
    background = 'media/backgrounds/wood_background.jpg'
    transformed_img = _logo_transformations(
        logo, background)

    response = HttpResponse(content_type="image/png;")
    transformed_img.save(response, 'PNG', quality=95)
    return response
```

Activity 13.02 – Creating dynamic graphics in Django with Pillow

The idea of this example is to generate a background image for a reviewer's profile that contains a pile of books that they have reviewed. The code is a dynamic version of our previous example.

The covers of the books that the user has reviewed are kept in a list called `covers`. Then, when we iterate through the covers, each cover is pasted onto the background at a different angle and position. This resembles a jumbled stack of books that the user has reviewed.

This activity will help consolidate your knowledge of Pillow and its use within Django. You will learn that Pillow can be used for generating JPEG images:

1. We need to include a `wood_background.jpg` file in `media/backgrounds`. You can copy the one in this activity from the Chapter13 folder in this book's GitHub repository supply your own.
2. Create a function in `reviews/views.py` called `get_review_profile_img`. It will take `image_type`, `user_id`, `background_path` as arguments and return an image.

3. In the function, create a list called `covers` that consists of the paths of the cover images relative to the `MEDIA_ROOT`. Get this list by filtering Review objects so that `creator_id` matches the parameter `user_id`.
4. Combine the `MEDIA_ROOT` path with `background_path` and open this image and convert it to 'RGBA'. We need to use RGBA rather than RGB because we are making use of the Alpha-Channel of the images in order to deal with transparency issues.
5. We will create a list of orientations and heights of the books. The former will be used to specify degree rotations, and the latter will be used to specify the Y-axis positioning of each book. These values give satisfying results, but consider trying your own:

```
orientations = [30, -15, 15, 0, -30, 45, -45]
heights = [125, 150, 175, 150, 125, 100, 150]
```

6. We will define a left and right displacement value for image positioning. These values account for the size of the cover image when calculating each position on the X-axis of the background image:

```
left_displacement = -10
right_displacement = 280
```

7. Now enumerate through the `covers` list and assign an orientation and height to the cover. The for loop will begin like this: `for i, cover in enumerate(covers):`
 - a. The X-axis position of the book is determined proportionately to the length of the image. We can assign a value like this:

```
x_pos = int(left_displacement + i*(
    bg_img-right_displacement)/len(covers))
```

- b. Open the cover image and convert it to RGBA.
 - c. Rotate the image according to the specific orientation and paste it at the X and Y positions derived above over the background image.
8. If the image type is 'jpg', we need to convert it back to RGB. Pillow's Image class can save JPEG files, but RGBA files first need to be converted to RGB.
9. Return the image.

Now that you have created a function that produces the review profile image, it needs to be wrapped in a view that takes a request argument and returns a response.

10. Create a view in `reviews/view.py` that takes `request` and `user_id` arguments called `reviewer_profile_jpg`. It will call `get_review_profile_img`, specifying `'jpg'` for `image_type`, `user_id`, and `'backgrounds/wood_background.jpg'` as the `background_path`, returning an image.
11. Create a `BytesIO` object and save the image to it.
12. Return a `HttpResponse` with the value of the `BytesIO` object and `content_type` set to `'image/jpeg'`.
13. Add a URL pattern of `'reviews/profile/<int:user_id>/background_jpg/'` for `reviewer_profile_jpg` to the `urlpatterns` in `reviews/urls.py`. By the end of this activity, you will be able to see an image like the following one when you visit a profile background image, such as `http://localhost:8000/reviews/profile/8/background_jpg`.

You have now gained some understanding of how Pillow can be used in Django to create dynamic images.



Figure 13.11: The profile background image

Note

The solution for this activity can be found at in the Chapter13 folder in this book's GitHub repo.

Summary

In this chapter, we looked at how we can deal with binary files and how Python's standard library, which comes pre-loaded with the necessary tools, allows us to handle commonly used file formats such as CSV. We then moved on to learning how to read and write CSV files in Python using Python's `csv` module. Later, we worked with the `XlsxWriter` package, which allows us to generate Microsoft Excel-compatible files right from our Python environment without us having to worry about the internal formatting of the file.

Next, we investigated the use of `ziplib` to generate ZIP files and used the `weasyprint` library to generate PDF versions of HTML pages. This skill can come in handy when we want to provide our users with an easy option to print the HTML version of our page with any added CSS styling of our choosing. Then we discussed how we can generate interactive graphs in Python and render them as HTML pages that can be viewed inside the browser using the `plotly` library. In the last section of this chapter, we took a deep dive into `Pillow` and learnt how it can be used to create a dynamic image from a Django request.

In the next chapter, we will look at how we can test the different components we have been implementing in the previous chapters to make sure that code changes do not break our website's functionality.

14

Testing Your Django Applications

In the preceding chapters, we focused on building our web application in Django by writing different components such as database models, views, and templates. We did all that to provide our users with an interactive application where they can create a profile and write reviews for the books they have read.

Apart from building and running the application, there is another important aspect of making sure that the application code works the way we expect it to work. This is ensured by a technique called **testing**. In testing, we run the different parts of the web application and check if the output of the executed component matches the output we expected. If the output matches, we say that the component was tested successfully, while if the output doesn't match, we say that the component failed to work as intended.

In this chapter, as we go through the different sections, we will learn why testing is important, the different ways to test a web application, and how we can build a strong testing strategy that will help us ensure that the web application we build is robust.

In this chapter, we will cover the following topics:

- Importance of testing
- Automation testing
- Testing in Django
- Testing Django models
- Testing Django views
- Django RequestFactory
- Test case classes in Django

Let's start our journey by first learning about the importance of testing.

Importance of testing

Making sure that an application works the way it was designed to work is an important aspect of development efforts because, otherwise, our users might keep on encountering weird behaviors that will usually drive away their engagement from the application.

The efforts we put into testing help us ensure that the different kinds of problems that we intend to solve are indeed being solved correctly. Imagine a case where a developer is building an online event scheduling platform. On this platform, users can schedule events on their calendars as per their local time zone. Now, what if, on this platform, users can schedule events as expected, but due to a bug, the events are scheduled in an incorrect time zone? It is issues like this that tend to drive many users away.

That's why a lot of enterprise companies spend a huge amount of money making sure that the applications they build undergo thorough testing. That way, they ensure that they do not release a buggy product or a product that is far from satisfying user requirements.

In brief, testing helps us achieve the following goals:

- Ensure that the components of the application work according to specifications
- Ensure interoperability on different infrastructure platforms, if an application can be deployed on a different operating system, such as Linux or Windows, and more
- Reduce the probability of introducing a bug while refactoring the application code

Now, a common assumption many people make about testing is that they have to test all the components manually as they are developed to make sure each component works according to its specifications, and repeat this exercise every time a change is made or a new component is added to the application. While this is true, this doesn't provide a complete picture of testing. Testing as a technique has grown very powerful with time, and as a developer, you can reduce a huge amount of testing effort by implementing **automated test cases**. So, what are these automated test cases? Or, in other words, what is **automation testing**? Let's find out.

Automation testing

Testing the whole application repeatedly when a single component is modified can turn out to be a challenging task, and even more so if that application consists of a large code base. The size of the code base could be due to the sheer number of features or the complexity of the problem it solves.

As we develop applications, it is important to make sure that the changes being made to these applications can be tested easily so that we can verify whether something is breaking. That's where the concept of automation testing comes in handy. The focus of automation testing is to

write tests as code so that the individual components of an application can be tested in isolation, as well as their interaction with each other.

While considering this, it becomes important for us to define the different kinds of automation tests that can be done for applications.

Automation testing can be broadly categorized into five different types:

- **Unit testing:** In this type of testing, the individual isolated units of code are tested. For example, a unit test can target a single method or a single isolated API. This kind of testing is performed to make sure the basic units of the application work according to their specification.
- **Integration testing:** In this type of testing, the individual isolated units of code are merged to form a logical grouping. Once this grouping is formed, testing is performed on this logical group to make sure that the group works in the way it is expected.
- **Functional testing:** In this kind of testing, the overall functionality of the different components of the application is tested. This may include different APIs, user interfaces, and more.
- **Smoke testing:** In this kind of testing, the stability of the deployed application is tested to make sure that the application continues to remain functional as the users interact with it, without causing a crash.
- **Regression testing:** This kind of testing is done to make sure that the changes being made to the application do not degrade the previously built functionality of the application.

As we can see, testing is a big domain that takes time to master; entire books have been written on this topic. To make sure we highlight the important aspects of testing, we are going to focus on unit testing in this chapter.

Testing in Django

Django is a feature-packed framework that aims to make web application development rapid. It is expected that it should also provide a full-featured way of testing the application.

Django provides a well-integrated module that allows application developers to write unit tests for their applications. This module is based on the Python `unittest` library, which ships with most of the Python distributions pre-packaged.

So, let's get started by understanding how we can write basic test cases in Django and how to leverage the framework-provided modules to test our application code.

Implementing test cases

While working on implementing mechanisms for testing your code, the first thing you need to understand is how this implementation can be logically grouped so that modules that are closely related to each other are tested in one logical unit.

This can be simplified by implementing a **test case**. A test case is nothing more than a logical unit that groups together tests that are related to logically similar units, so that all the common logic to initialize the environment for the test cases can be combined in the same place, hence avoiding duplication of work while implementing application testing code.

Unit testing in Django

Now that we understand tests, let's take a look at how we can do unit testing inside Django. In the context of Django, a unit test consists of two major parts:

- A `TestCase` class, which wraps the different test cases that are grouped for a given module
- An actual test case that needs to be executed to test the flow of a particular component

The class implementing a unit test should inherit from the `TestCase` class provided by Django's test module. By default, Django provides a `tests.py` file in every application directory, which can be used to store the test cases for the application module.

Once these unit tests have been written, they can be executed easily by running them directly using the provided test command in `manage.py`, as follows:

```
python manage.py test
```

Now, let's look at how assertions can help us build our unit tests and check for the expected behavior.

Utilizing assertions

An important part of writing tests is validating whether the test passed or failed. Generally, to implement such decisions in a testing environment, we utilize something known as **assertions**.

Assertions are a common concept in software testing. They take in two operands and validate if the value of the operand on the **left-hand side (LHS)** matches the value of the operand on the **right-hand side (RHS)**. If the value of the LHS matches the value of the RHS, an assertion is considered to be successful, whereas if the values differ, the assertion is considered to have failed.

An assertion that evaluates to `False` essentially causes a test case to be evaluated as a failure, which is then reported to the user.

Assertions in Python are quite easy to implement, and they use a simple keyword called `assert`. For example, the following code snippet shows a very simple assertion:

```
assert 1 == 1
```

The preceding assertion takes in a single expression, which evaluates to `True`. If this assertion were part of a test case, the test would have succeeded.

Now, let's see how we can implement test cases using the Python `unittest` library. Doing so is quite easy and can be accomplished with some easy-to-follow steps:

1. Import the `unittest` module, which allows us to build the test cases:

```
import unittest
```

2. Once the module has been imported, we can create a class whose name starts with `Test`, and which inherits from the `TestCase` class provided by the `unittest` module:

```
class TestMyModule(unittest.TestCase):  
    def test_method_a(self):  
        self.assertTrue(<expression>)
```

Only if the `TestMyModule` class inherits the `TestCase` class will Django be able to run it automatically with full integration with the framework. Once the class has been defined, we can implement a new method inside the class named `test_method_a()`, which validates an assertion. And that's it.

An important part to note here is the naming scheme for the test cases and test functions. The test cases being implemented should be prefixed with the name `Test` so that the test execution modules can detect them as valid test cases and execute them. The same rule applies to naming testing methods.

3. Once the test case has been written, it can be simply executed by running the following command:

```
python manage.py test
```

With our basic understanding of implementing test cases clarified, let's write a very simple unit test to get a feel of how the unit testing framework behaves inside Django.

Exercise 14.01 – Writing a simple unit test

In this exercise, we will write a simple unit test to understand how the Django unit testing framework works and use this knowledge to implement our first test case that validates a simple expression:

1. To get started, open the `tests.py` file under the `reviews` application of the *Bookr* project. By default, this file will contain only a single line that imports Django's `TestCase` class from the `test` module:

```
from django.test import TestCase
```

2. Add the following lines of code to the `tests.py` file you just opened:

```
class TestSimpleComponent(TestCase):  
    def test_basic_sum(self):  
        assert 1+1 == 2
```

Here, we created a new class named `TestSimpleComponent`, which inherits from the `TestCase` class provided by Django's `test` module. The `assert` statement will compare the expression on the LHS (`1+1`) with the one on the RHS (`2`).

3. Once you have written the test case, navigate back to the project folder and run the following command:

```
python manage.py test
```

The following output should be generated:

```
% ./manage.py test  
Creating test database for alias 'default'...  
System check identified no issues (0 silenced).  
.  
-----  
Ran 1 test in 0.001s  
  
OK  
Destroying test database for alias 'default'...
```

The preceding output signifies that Django's test runner executed 1 test case, which successfully passed the evaluation.

4. With the test case confirmed to be working and passing, we will now try to add another assertion at the end of the `test_basic_sum()` method, as shown in the following code snippet:

```
assert 1+1 == 3
```

5. With the `assert` statement added in *Step 4*, execute the test cases by running the following command from the project folder:

```
python manage.py test
```

At this point, you will notice Django reporting that the execution of the test cases has failed.

With this, you understand how test cases can be written in Django and how the assertions can be used to validate whether the output generated from your method calls under test is correct or not.

Now that we have basic knowledge of assertions, let's dive deep into the different types of assertions that are present and their usage.

Types of assertions

In *Exercise 14.01 – Writing a simple unit test*, we briefly encountered assertions when we came across the following `assert` statement:

```
assert 1+1 == 2
```

These assertion statements are simple and use the Python `assert` keyword. A few different types of assertions are possible that can be tested inside a unit test while using the `unittest` library. Let's look at those:

- `assertIsNone`: This assertion is used to check whether an expression evaluates to `None` or not. For example, this type of assertion can be used in cases where a query to a database returns `None` because no records were found for the specified filtering criteria.
- `assertIsInstance`: This assertion is used to validate if a provided object evaluates to an instance of the provided type. For example, this assertion can be used to validate if the value returned by a method indeed is of a specific type, such as `List`, `Dict`, `Tuple`, and others.

- `assertEqual`: This is a very basic function that takes in two arguments and checks whether the arguments provided to it are equal in value or not. This can be useful when you plan to compare the values of data structures that do not guarantee ordering.
- `assertRaises`: This method is used to validate whether the name of the method provided to it, when called, raises a specified exception or not. This is helpful when we are writing test cases where a code path that raises an exception needs to be tested. As an example, this kind of assertion can be useful when we want to make sure an exception is raised by a method performing a database query if the database connection is not yet established.

These were just a small set of useful assertions that we can make in our test cases. The `unittest` module, on top of which Django's testing library is built, provides a lot more assertions that can be tested for.

Ever wondered what to do when our unit tests require some lists to be pre-populated or databases to be pre-initialized? The next section will cover how to pre-test this setup and how it can help us avoid introducing repetitive code that must be executed before every test case.

Performing pre-test setup and cleanup after every test case run

Sometimes, while writing test cases, we may need to perform some repetitive tasks, such as setting up some variables that will be required for the test. Once the test is over, we will want to clean up all the changes we've made to the test variables so that any new test starts with a fresh instance.

Luckily, the `unittest` library provides us with a useful way to automate our repetitive efforts of setting up the environment before every test case runs and cleaning it up after the test case is finished. This can be achieved using the two methods that we can implement in `TestCase`.

The first is the `setUp()` method. This method is called before the execution of every test method inside the `TestCase` class. It implements the code required to set up the test case's environment before the test executes. This method can be a good place to set up any local database instance or test variables that may be required for the test cases.

Note

The `setUp()` method is only valid for test cases written inside the `TestCase` class.

For example, the following example illustrates a simple definition of how the `setUp()` method is used inside a `TestCase` class:

```
class MyTestCase(unittest.TestCase):
    def setUp(self):
        # Do some initialization work
    def test_method_a(self):
        # code for testing method A
    def test_method_b(self):
        # code for testing method B
```

In the preceding example, when we try to execute the test cases, the `setUp()` method we defined here will be called every time before a test method executes. In other words, the `setUp()` method will be called before the execution of the `test_method_a()` call, and then it will be called again before the `test_method_b()` call is called.

The second is the `tearDown()` method. This method is called once the test function finishes execution and cleans up the variables and their values once the test case's execution has finished. This method is executed regardless of whether the test case evaluates to `True` or `False`. An example of using the `tearDown()` method is shown here:

```
class MyTestCase(unittest.TestCase):
    def setUp(self):
        # Do some initialization work
    def test_method_a(self):
        # code for testing method A
    def test_method_b(self):
        # code for testing method B
    def tearDown(self):
        # perform cleanup
```

In the preceding example, the `tearDown()` method will be called every time a test method finishes execution – that is, once `test_method_a()` finishes executing and again once `test_method_b()` finishes executing.

Now that we are aware of the different components of writing test cases. Let's look at how we can test the different aspects of a Django application using the provided test framework.

Testing Django models

Models in Django are object-based representations of how data will be stored inside the database of an application. They provide methods that can help us to validate the data input provided for a given record, as well as to perform any processing on the data before it is inserted into the database.

It is as easy to test models in Django as it is to create them. Now, let's look at how Django models can be tested using the Django Test framework.

Exercise 14.02 – Testing Django models

In this exercise, we will create a new Django model and write test cases for it. The test case will validate whether your model can correctly insert and retrieve the data from the database.

These kinds of test cases that work on database models can turn out to be useful in cases where a team of developers is collaborating on a large project, and the same database model may be modified by multiple developers over time. Implementing test cases for the database models allows developers to pre-emptively identify potentially breaking changes that they may inadvertently introduce as part of their work. Let's get started with the steps:

1. Create a new application that you will use for the exercises in this chapter. For this, run the following command, which will set up a new application for your use case:

```
python manage.py startapp bookr_test
```

2. To make sure the `bookr_test` application behaves the same way as any other application in the Django project, add this application to the `INSTALLED_APPS` section of the `bookr` project. To do this, open the `settings.py` file in your `bookr` project and append the following code to the `INSTALLED_APPS` list:

```
INSTALLED_APPS = [  
    ... ,  
    ... ,  
    "bookr_test",  
]
```

3. Now, with the application setup complete, create a new database model that you will use for testing purposes. For this exercise, we are going to create a new model named `Publisher` that will store the details about the book publisher in our database. To

create the model, open the `models.py` file under the `bookr_test` directory and add the following code to it:

```
from django.db import models

class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
        max_length=50,
        help_text="The name of the Publisher.")

    website = models.URLField(
        help_text="The Publisher's website.")
    email = models.EmailField(
        help_text="The Publisher's email address.")

    def __str__(self):
        return self.name
```

In the preceding code snippet, you created a new class named `Publisher` that inherits from the `Model` class of Django's `models` module, defining the class as a Django model that will be used to store data about the publisher:

```
class Publisher(models.Model)
```

Inside this model, we have added three fields, which will act as the properties of the model:

- `name`: The name of the publisher
- `website`: The website belonging to the publisher
- `email`: The email address of the publisher

Once we did this, we created a class method called `__str__()` that defines what the string representation of the `Model` class will look like.

4. Now, with the model created, you need to migrate this model before you can run a test on it. To do this, run the following commands:

```
python manage.py makemigrations
python manage.py migrate
```

5. With the model now set up, write the test case with which you are going to test the model you created in *Step 3*. For this, open the `tests.py` file under the `bookr_test` directory and add the following code to it:

```
from django.test import TestCase

from .models import Publisher

class TestPublisherModel(TestCase):
    """Test the publisher model."""
    def setUp(self):
        self.p = Publisher(
            name='Packt',
            website='www.packt.com',
            email='contact@packt.com'
        )

    def test_create_publisher(self):
        self.assertIsInstance(self.p, Publisher)

    def test_str_representation(self):
        self.assertEqual(str(self.p), "Packt")
```

In the preceding code snippet, there are a couple of things worth exploring.

At the start, after importing the `TestCase` class from the Django test module, you imported the `Publisher` model from the `bookr_test` directory, which is going to be used for testing.

Once the required libraries have been imported, we must create a new class named `TestPublisherModel` that inherits the `TestCase` class and is used for grouping the unit tests related to the `Publisher` model:

```
class TestPublisherModel(TestCase):
```

Inside this class, we defined a couple of methods. First, we defined a new method named `setUp()` and added the `Model` object creation code to it so that the `Model` object is created every time a new test method is executed inside this test case. This `Model`

object is stored as a class member so that it can be accessed inside other methods without a problem:

```
def setUp(self):
    self.p = Publisher(
        name='Packt',
        website='www.packt.com',
        email='contact@packt.com'
    )
```

The first test case validates whether the `Model` object for the `Publisher` model was created successfully or not. To do this, we created a new method named `test_create_publisher()`, inside which we check whether the model object created points to an object of the `Publisher` type. If this `Model` object was not created successfully, your test will fail:

```
def test_create_publisher(self):
    self.assertIsInstance(self.p, Publisher)
```

If you check carefully, you will see that we are using the `assertIsInstance()` method of the `unittest` library here to assert whether the `Model` object belongs to the `Publisher` type or not.

The next test validates whether the string representation of the model is the same as we expected it to be. From the code definition, the string representation of the `Publisher` model should output the name of the publisher. To test this, we created a new method named `test_str_representation()` and checked if the generated string representation of the model matches the one we are expecting:

```
def test_str_representation(self):
    self.assertEqual(str(self.p), "Packt")
```

To perform this validation, we used the `assertEqual` method of the `unittest` library, which validates whether the two values provided to it are equal or not.

6. With the test cases now in place, you can run them to check what happens. To run these test cases, run the following command:

```
python manage.py test
```

Once the command finishes executing, you will see an output that resembles the one shown here:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
..
-----
Ran 2 tests in 0.002s

OK
Destroying test database for alias 'default'...
```

As you can see from the preceding output, the test cases are executed successfully, hence validating that the operations, such as creating a new `Publisher` object and its string representation when fetched, are being done correctly.

With this exercise, we saw how we can write test cases for our Django models easily and validate their functionality by creating objects, retrieving them, and representing them.

Also, notice the following important line in the output from *Exercise 14.02 – Testing Django models*:

```
"Destroying test database for alias 'default'..."
```

This happens because when there are test cases that require the data to be persisted inside a database, instead of using the production database, Django creates a new empty database for the test cases, which it uses to persist the value for the test case.

Now that we know how to test models in Django, in the next section, we will learn how to test views in Django.

Testing Django views

Views in Django control the rendering of the HTTP response for users based on the URL they visit in a web application. In this section, we will learn how to test views inside Django.

Imagine you are working on a website where a lot of **application programming interface (API)** endpoints are required. An interesting question to ask is, how will you be able to validate every new endpoint? If done manually, you will have to first deploy the application every time a new endpoint is added, then manually visit the endpoint in the browser to validate whether it is working fine or not. Such an approach may work out when the number of endpoints is small, but it may become extremely cumbersome if there are hundreds of endpoints.

Django provides a very comprehensive way of testing application views. This happens by using a testing client class provided by Django's test module. This class can be used to visit URLs mapped to the views and capture the output generated by visiting the URL endpoint. Then, we can use the captured output to test whether the URLs are generating a correct response or not. This client can be used by importing the `Client` class from the Django test module and then initializing it, as shown in the following snippet:

```
from django.test import Client

c = Client()
```

The `Client` object supports several methods that can be used to simulate the different HTTP calls a user can make – namely, GET, POST, PUT, DELETE, and others. An example of making such a request looks like this:

```
response = c.get('/welcome')
```

The response generated by the view is then captured by the client and exposed as a response object, which can then be queried to validate the output of the view.

With this knowledge, let's look at how we can write test cases for our Django views.

Exercise 14.03 – Writing unit tests for Django views

In this exercise, we will use the Django test client to write a test case for our Django view, which will be mapped to a specific URL. These test cases will help you validate whether your View function generates the correct response when visited using its mapped URL. Let's get started with the steps:

1. For this exercise, you are going to use the `bookr_test` application that was created in *Step 1 of Exercise 14.02 – Testing Django models*. To get started, open the `views.py` file under the `bookr_test` directory and add the following code to it:

```
from django.http import HttpResponse

def greeting_view(request):
    """Greet the user."""
    return HttpResponse("Hey there, welcome to Bookr!"
                       "Your one stop place to review books.")
```


Here, you have created a simple Django view that will be used to greet the user with a welcome message whenever they visit an endpoint mapped to the provided view.

2. Once the view has been created, you need to map this view to a URL endpoint that can then be visited in a browser or a test client. To do this, add a `urls.py` file under the `bookr_test` directory and include the following code with the mapping in the `urlpatterns` list:

```
from django.urls import path
from . import views

urlpatterns = [
    path('test/greeting', views.greeting_view,
        name='greeting_view'),
]
```

In the preceding code snippet, we mapped `greeting_view` to the `'test/greeting'` endpoint for the application by setting the path in the `urlpatterns` list.

3. Once this path has been set up, we need to make sure that it is also identified by our project. For this, we need to add this entry to the *bookr* project's URL mapping. To achieve that, open the `urls.py` file in the *bookr* directory and append the following highlighted line to the end of the `urlpatterns` list:

```
urlpatterns = [
    ...,
    ...,
    path('', include('bookr_test.urls')),
]
```

4. Once the view has been set up, validate that it works correctly. Do this by running the following command:

```
python manage.py runserver localhost:8080
```

Then, visit `http://localhost:8080/test/greeting` in your web browser. Once the page opens, you should see the following text you added in *Step 1* to the greeting view being displayed in the browser:

```
Hey there, welcome to Bookr! Your one stop place to review books.
```

5. Now, we are ready to write the test cases for `greeting_view`. For this exercise, we are going to write a test case that checks whether, on visiting the `/test/greeting` endpoint, you get a successful result or not. To implement this test case, open the `tests.py` file under the `bookr_test` directory and add the following code to the end of the file:

```
from django.test import TestCase

class TestGreetingView(TestCase):
    """Test the greeting view."""
    def test_greeting_view(self):
        response = self.client.get('/test/greeting')
        self.assertEqual(response.status_code, 200)
```

In the preceding code snippet, we defined a test case that helps validate whether the greeting view is working fine or not.

This is done by first importing Django's test client, which allows you to test views mapped to the URLs by making calls to them and analyzing the generated response:

```
from django.test import TestCase, Client
```

Once the import has been done, we must create a new class named `TestGreetingView` that will group the test cases related to the greeting view we created in *Step 2*:

```
class TestGreetingView(TestCase):
```

Inside this test case, we defined two methods, `setUp()` and `test_greeting_view()`. The `test_greeting_view()` method implements your test case. Inside this, we first make an HTTP GET call to the URL that is mapped to the greeting view and store the response generated by the view inside the response object created:

```
response = self.client.get('/test/greeting')
```

Once this call finishes, you will have its HTTP response code, contents, and headers available inside the response variable. Next, with the following code, we make an

assertion, validating whether the status code generated by the call matches the status code for successful HTTP calls (HTTP 200):

```
self.assertEqual(response.status_code, 200)
```

With this, we are now ready to run the tests.

6. With the test case written, look at what happens when you run the test case. This can be done by running the following command:

```
python manage.py test
```

Once the command executes, you can expect to see an output like the one shown in the following snippet:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
...
-----
Ran 3 tests in 0.006s

OK
Destroying test database for alias 'default'...
```

As you can see from the output, your test cases executed successfully, hence validating that the response generated by the `greeting_view()` method is as per your expectations.

In this exercise, we learned how we can implement a test case for a Django view function and use Django's `TestClient` to assert that the output generated by the view function matches the one the developer should see.

In the next section, we will learn how to test views that have authentication enabled.

Testing views with authentication

In the previous example, we looked at how we can test views inside Django. An important part to be highlighted about this view is that the view we created could be accessed by anyone and is not protected by any authentication or login checks. Now, imagine a case where a view should only be accessible if the user is logged in. For example, consider implementing a view function that renders a profile page of a registered user of our web application. To make sure that only logged-in users can view the profile page for their account, you might want to restrict the view to logged-in users only.

With this, we now have an important question: *How can we test views that require authentication?*

Luckily, Django's test client provides this functionality, through which we can log in to our views and then run tests over them. This result can be achieved using Django's test client `login()` method. When this method is called, Django's test client performs an authentication operation against the service. If the authentication succeeds, it stores the login cookie internally, which it can then use for further test runs. The following code snippet shows how you can set up Django's test client to simulate a logged-in user:

```
login = self.client.login(username='testuser',
                           password='testpassword')
```

The `login` method requires a username and password for the test user that we are going to test with, as will be shown in the next exercise. So, let's take a look at how we can test a flow that requires user authentication.

Exercise 14.04 – Writing test cases to validate authenticated users

In this exercise, we will write test cases for views that require the user to be authenticated. As part of this, we will validate the output generated by the view method when a user who is not logged in tries to visit the page and when a user who is logged in tries to visit the page mapped to the view function. Let's get started with the steps:

1. For this exercise, we are going to use the `bookr_test` application that you created in *Step 1 of Exercise 14.02 – Testing Django models*. To get started, open the `views.py` file under the `bookr_test` application and add the following code to it:

```
from django.http import HttpResponseRedirect
from django.contrib.auth.decorators import \
    login_required
```

2. Once the preceding code snippet has been added, create a new function called `greeting_view_user()` at the end of the file, as shown in the following code snippet:

```
@login_required
def greeting_view_user(request):
    """Greeting view for the user."""
    user = request.user
    return HttpResponse(f"Welcome to Bookr! {user}")
```

Here, we have created a simple Django view that will be used to greet the logged-in user with a welcome message whenever they visit an endpoint mapped to the provided view.

3. Once this view has been created, we need to map this view to a URL endpoint that can then be visited in a browser or a test client. To do this, open the `urls.py` file under the `bookr_test` directory and add the highlighted code to it:

```
from django.urls import path
from . import views

urlpatterns = [ ...
    path('test/greet_user', views.greeting_view_user,
         name='greeting_view_user'),
]
```

In the preceding code snippet, we mapped `greeting_view` to the `'test/greet_user'` endpoint for the application by setting the path in the `urlpatterns` list:

```
path('test/greet_user', views.greeting_view_user,
     name='greeting_view_user')
```

If you followed the previous exercises, this URL should already be set up for detection in the project, and no further steps are required to configure the URL mapping.

4. Once the view has been set up, the next thing we need to do is validate whether it works correctly. To do this, run the following command:

```
python manage.py runserver localhost:8080
```

Then, visit `http://localhost:8080/test/greet_user` in your web browser.

If you are not logged in already, by visiting the preceding URL, you will be redirected to the login page for the project.

5. Now, write the test cases for `greeting_view_user`, which check whether, on visiting the `/test/greet_user` endpoint, you get a successful result or not. To implement this test case, open the `tests.py` file under the `bookr_test` directory and add the following code to it:

```
from django.contrib.auth.models import User

class TestLoggedInGreetingView(TestCase):
    """Test the greeting view for the authenticated
    users."""
    def setUp(self):
        test_user = User.objects.create_user(
            username='testuser',
            password='test@#628password')
        test_user.save()
        self.client = Client()

    def test_user_greeting_not_authenticated(self):
        response = self.client.get('/test/greet_user/')
        self.assertEqual(response.status_code, 302)

    def test_user_authenticated(self):
        login = self.client.login(username='testuser',
            password='test@#628password')
        response = self.client.get('/test/greet_user/')
        self.assertEqual(response.status_code, 200)
```

In the preceding code snippet, we implemented a test case that checks the views that have authentication enabled before their content can be seen.

Here, first, we imported the required classes and methods that will be used to define the test case and initialize a testing client:

```
from django.test import TestCase, Client
```

The next thing you require is the `User` model from Django's `auth` module:

```
from django.contrib.auth.models import User
```

This model is required because, for the test cases requiring authentication, we will need to initialize a new test user. Next up, we created a new class named `TestLoggedInGreetingView`, which wraps your tests related to the `greeting_user` view (which requires authentication). Inside this class, you defined three methods called `setUp()`, `test_user_greeting_not_authenticated()`, and `test_user_authenticated()`. The `setUp()` method is used to initialize a test user that you will use for authentication. This is a required step because a test environment inside Django is a completely isolated environment that doesn't use data from your production application, so all the required models and objects are to be instantiated separately inside the test environment.

Then, we created the test user and initiated the test client using the following code:

```
test_user = User.objects.create_user(  
    username='testuser', password='test@#628password')  
test_user.save()  
self.client = Client()
```

Next, we wrote the test case for the `greet_user` endpoint when the user is not authenticated. Here, you should expect Django to redirect the user to the login endpoint. This redirect can be detected by checking the HTTP status code of the response, which should be set to HTTP 302, indicating a redirect operation:

```
def test_user_greeting_not_authenticated(self):  
    response = self.client.get('/test/greet_user/')  
    self.assertEqual(response.status_code, 302)
```

Next, we wrote another test case to check whether the `greet_user` endpoint renders successfully when the user is authenticated. To authenticate the user, you must first call the `login()` method of the test client and perform authentication by providing the username and password of the test user you created in the `setUp()` method, as follows:

```
login = self.client.login(username='testuser',  
    password='test@#628password')
```

Once the login has been completed, we must make an HTTP GET request to the `greet_user` endpoint and validate whether the endpoint generates a correct result or not by checking the HTTP status code of the returned response:

```
response = self.client.get('/test/greet_user/')
self.assertEqual(response.status_code, 200)
```

6. With the test cases written, it's time to check how they run. For this, run the following command:

```
python manage.py test
```

Once the execution finishes, we can expect to see a response that resembles the following one:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 5 tests in 0.366s

OK
Destroying test database for alias 'default'...
```

As we can see from the preceding output, our test cases have passed successfully, validating that the view we created generates the desired response of redirecting the user if the user is unauthenticated to the website and allowing the user to see the page if the user is authenticated.

In this exercise, we implemented a test case where we can test what output is generated by a view function based on whether the user is authenticated or not.

In the next section, we will explore how to create requests in Django unit tests without the use of a Django-provided test client, but by providing raw request data.

Django RequestFactory

So far, we have been using Django's test client to test the views that we have created for our application. The test client class simulates a browser and uses this simulation to make calls to the required APIs. But what if we didn't want to use the test client and its associated simulation of being a browser, but rather wanted to test the view functions by directly bypassing the request parameter? How can we do that?

To help us in such cases, we can leverage the RequestFactory class provided by Django. The RequestFactory class helps us provide the request object that we can pass to our view functions to evaluate whether they are working. The following object for the RequestFactory class can be created by instantiating the class as follows:

```
factory = RequestFactory()
```

The factory object we have created only supports HTTP methods such as `get()`, `post()`, `put()`, and others to simulate a call to any URL endpoint. Now, let's learn how to modify the test case that we wrote in *Exercise 14.04 – Writing test cases to validate authenticated users*, so that it uses RequestFactory.

Exercise 14.05 – Using RequestFactory to test views

In this exercise, we will use RequestFactory to test view functions in Django:

1. For this exercise, we are going to use the existing `greeting_view_user` view function that we created earlier, in *Step 1 of Exercise 14.04 – Writing test cases to validate authenticated users*, which looks like this:

```
@login_required
def greeting_view_user(request):
    """Greeting view for the user."""
    user = request.user
    return HttpResponse(f"Welcome to Bookr! {user}")
```

2. Next, modify the existing test case, `TestLoggedInGreetingView`, defined inside the `tests.py` file under the `bookr_test` directory. Open the `tests.py` file and make the following changes:

- First, we need to add the following import to use the `RequestFactory` class inside the test cases:

```
from django.test import (TestCase, Client
                          RequestFactory)
```

- The next thing we need is an import for the `AnonymousUser` class from Django's `auth` module and the `greeting_view_user` view method from the `views` module. This is required to test the view functions with a simulated user who is not logged in. This can be done by adding the following code:

```
from django.contrib.auth.models import (AnonymousUser,
                                       User)

from .views import greeting_view_user
```

3. Once the import statements have been added, modify the `setUp()` method of the `TestLoggedInGreetingView` class and change its contents so that it resembles the following:

```
def setUp(self):
    self.test_user = User.objects.create_user(
        username='testuser',
        password='test@#628password')
    self.test_user.save()
    self.factory = RequestFactory()
```

In this method, we first created a user object and stored it as a class member so that we can use it later in the tests. Once the user object was created, we instantiated a new instance of the `RequestFactory` class to test our view function.

4. With the `setUp()` method now defined, modify the existing tests so that they use the `RequestFactory` instance. For the test for a non-authenticated call to the view

function, modify the `test_user_greeting_not_authenticated` method so that it has the following contents:

```
def test_user_greeting_not_authenticated(self):
    request = self.factory.get('/test/greet_user')
    request.user = AnonymousUser()
    response = greeting_view_user(request)
    self.assertEqual(response.status_code, 302)
```

In this method, we first created a request object using the `RequestFactory` instance we defined in the `setUp()` method. Once we had done that, we assigned an `AnonymousUser()` instance to the `request.user` property. Assigning the `AnonymousUser()` instance to the property makes the view function think that the user making the request is not logged in:

```
request.user = AnonymousUser()
```

Once we did this, we made a call to the `greeting_view_user()` view method and passed to it the request object you created. Once the call is successful, we must capture the output of the method in the response variable using the following code:

```
response = greeting_view_user(request)
```

For the unauthenticated user, we expect to get a redirect response that can be tested by checking the HTTP status code of the response, as follows:

```
self.assertEqual(response.status_code, 302)
```

5. Now, go ahead and modify the other method, `test_user_authenticated()`, similarly by using the `RequestFactory` instance, as follows:

```
def test_user_authenticated(self):
    request = self.factory.get('/test/greet_user')
    request.user = self.test_user
    response = greeting_view_user(request)
    self.assertEqual(response.status_code, 200)
```

As you can see, most of the code resembles the code you wrote in the `test_user_greeting_not_authenticated` method, with a small change – in this

method, instead of using `AnonymousUser` for the `request.user` property, you are using the `test_user` that you created in the `setUp()` method:

```
request.user = self.test_user
```

With the changes done, it's time to run the tests.

6. To run the tests and validate whether `RequestFactory` works as expected or not, run the following command:

```
python manage.py test
```

Once the command executes, you can expect to see an output that resembles the one shown here:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 6 tests in 0.248s

OK
Destroying test database for alias 'default'...
```

As we can see from the output, the test cases written by us have passed successfully, hence validating the behavior of the `RequestFactory` class.

With this exercise, we learned how we can write test cases for view functions by leveraging `RequestFactory` and passing the request object directly to the view function, rather than simulating a URL visit using the test client approach, and hence allowing for more direct testing.

In the next section, we will test class-based views.

Testing class-based views

In the previous exercise, we saw how we can test views defined as methods. But what about class-based views? How can we test those?

As it turns out, it is quite easy to test class-based views. For example, if we have a class-based view defined with the name `ExampleClassView(View)`, to test the view, all we need to do is use the following syntax:

```
response = ExampleClassView.as_view()(request)
```

It is as simple as that.

A Django application generally consists of several different components that can either work in isolation, such as models, and some other components that need to interact with the URL mapping and other parts of the framework to work. Testing these different components may require some steps that are common to only those components. For example, when testing a model, first, we may want to create certain objects of the `Model` class before we start testing, or for views, first, we may want to initialize a test client with user credentials.

As it turns out, Django also provides some other classes based on the `TestCase` class that can be used to write test cases of specific types about the type of component being used. Let's look at the different classes provided by Django.

Test case classes in Django

Beyond the base `TestCase` class provided by Django, which can be used to define a multitude of test cases for different components, Django also provides some specialized classes derived from the `TestCase` class.

These classes are used for specific types of test cases based on the capabilities they provide to the developer. Let's look at the different classes provided by Django.

The `SimpleTestCase` class

This class is derived from the `TestCase` class provided by Django's test module and should be used when writing simple test cases that test the view functions. Usually, this class is not preferred when your test case involves making database queries. The class also provides a lot of useful features, such as the following:

- Ability to check for exceptions raised by a view function
- Ability to test form fields
- An inbuilt test client
- Ability to verify a redirect by a view function
- Can match the equality of two HTML, JSON, or XML outputs generated by the view functions

Now that we have a basic idea of what a `SimpleTestCase` class is, let's move on and try to understand another type of test case class that helps in writing test cases about the interaction with databases.

The `TransactionTestCase` class

This class is derived from the `SimpleTestCase` class and should be used when writing test cases that involve interaction with the database, such as database queries, model object creations, and more.

The class provides the following added features:

- Ability to reset the database to a default state before a test case runs
- Can skip tests based on database features, which can be quite handy if the database being used for testing does not support all the features of your production database

The `LiveServerTestCase` class

This class is like the `TransactionTestCase` class, but with a small difference that the test cases written in the class, instead of using the default test client, use a live server created by Django.

This ability to run the live server for testing comes in handy when writing test cases that test for the rendered web pages and any interaction with them, which is not possible while using the default test client.

Such test cases can leverage tools such as Selenium, which can be used to build interactive test cases that modify the state of the rendered page by interacting with it.

Modularizing test code

In the previous exercises, we saw how we can write test cases for different components of our project. But an important aspect to note is that, till now, we have written the test cases for all the components in a single file.

This approach is okay when the application does not have a lot of views and models, but this can become problematic as our application grows because our `single_tests.py` file will be hard to maintain.

To avoid running into such scenarios, we should try to modularize our test cases so that the test cases for models are kept separately from test cases related to the views and other properties. To achieve this modularization, all we need to do is follow two simple steps:

1. Create a new directory named `tests` inside our application directory by running the following command:

```
mkdir tests
```

2. Create a new empty file named `__init__.py` inside the `tests` directory by running the following command:

```
touch __init__.py
```

This `__init__.py` file is required by Django to correctly detect the `tests` directory we created as a module and not a regular directory.

Once you have completed the preceding steps, you can go ahead and create new testing files for the different components in your application. For example, to write test cases for your models, you can create a new file named `test_models.py` inside the `tests` directory and add any associated code for your model testing inside this file.

Also, you don't need to take any other additional steps to run your tests. The same command will work perfectly fine for your modular testing code base as well:

```
python manage.py test
```

With this, we know how we can write test cases for our projects. So, how about we test this knowledge and write test cases for the *Bookr* project we are working on?

Running tests faster using parallelization

A well-built application has several test cases that provide high code coverage and test all the different code paths in the application. This also means that every time the application is built, all these tests need to be executed. This slows down the build process and often becomes a reason for developer frustration.

By default, when we run testing with Django, it runs one test at a time. However, this can be made faster. A very simple way Django allows us to decrease the build time is by allowing the developers to run the tests in parallel.

The following command shows you how to run tests in Django in a parallel manner:

```
python manage.py test --parallel
```

When the preceding command is run, Django executes all the tests in the Django web application. The `--parallel` parameter causes Django to run N number of tests in parallel instead of in a serial manner. The number of tests that will run in parallel will be determined by the number of cores that are available on your development machine.

You can also explicitly tell Django how many tests it can run in parallel by mentioning the count just after the `--parallel` parameter, as shown here:

```
python manage.py test --parallel 4
```

This will tell Django to run only 4 test cases in parallel.

When the tests are being executed in parallel, every test case gets its own database and other environment variables to avoid problems that come up due to running code in a concurrent environment.

Now that we understand how to unit test our applications in Django, let's go ahead and implement our knowledge by implementing test cases of Bookr.

Activity 14.01 – Testing models and views in Bookr

In this activity, you will implement test cases for the *Bookr* project. You will implement test cases to validate the functioning of the models created inside the *reviews* application of the Bookr project, and then you will implement a simple test case for validating the *index view* inside the *reviews* application.

The following steps will help you work through this activity:

1. Create a directory named `tests` inside the *reviews* application directory so that all our test cases for the *reviews* application can be modularized.
2. Create an empty `__init__.py` file so that the directory is considered not as a general directory, but rather a Python module directory.
3. Create a new file named `test_models.py` for implementing the code for testing the models. Inside this file, import the models you want to test.
4. Inside the `test_models.py` file, create a new class that inherits from the `TestCase` class of the `django.test` module and implements methods to validate the process of creating and reading the `Model` objects.

5. To test the view function, create a new file named `test_views.py` inside the `tests` directory that was created in *Step 1*.
6. Inside the `test_views.py` file, import the `test Client` class from the `django.test` module and the index view function from the `reviews` application's `views.py` file.
7. Inside the `test_views.py` file you created in *Step 5*, create a new `TestCase` class and implement methods to validate the index view.
8. Inside the `TestCase` class you created in *Step 7*, create a new function called `setUp()`, inside which you should initialize an instance of `RequestFactory`, which will be used to create a request object that can be directly passed to the view function for testing.
9. Once you have completed the previous steps, write the test cases and run them by executing `python manage.py` to validate that the test cases pass.

Upon completing this activity, all test cases should pass successfully.

Summary

In this chapter, we learned how to write test cases for different components of our web application project with Django. We learned why testing plays a crucial role in the development of any web application and the different types of testing techniques that are employed in the industry to make sure the application code they ship is stable and bug-free.

Then, we looked at how we can use the `TestCase` class provided by Django's test module to implement unit tests, which can be used to test models as well as views. We also looked at how we can use Django's test client to test view functions, which require or do not require the user to be authenticated. We also glanced over another approach of using `RequestFactory` to test method views and class-based views.

We concluded this chapter by understanding the predefined classes provided by Django and where they should be used, while also looking at how we can modularize our testing code base to make it appear clean.

As we move on to the next chapter, we will learn how to deploy our Django application to a server. This will cover web server architecture, verifying a project is ready, and options available for deployment.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

15

Deploying a Django Project

Now that we have surveyed the crucial aspects of Django application development, we can examine the Django deployment process. Django has important utilities for deploying a project, and there is a range of deployment options that we will examine. Finally, we will step through the deployment of an application to a platform-as-a-service provider.

In this chapter, we will cover the following topics:

- Server architecture
- The Django deployment checklist
- Configuring a project for deployment
- Deployment options for Django

We will commence with a brief discussion of server architectures.

Technical requirement

All the code files in the Chapter15 folder in this book's GitHub repo. To access the repository link, follow the steps in the *"Download the example code files"* section in the *Preface*.

Server architecture

So far, we have been using the Django development server to both execute the Python code for Bookr and serve the static and media files. We mentioned throughout the book that the Django development server is not suitable for use in production. It is not designed to run multiple processes or handle many users, and more importantly, it doesn't run when the `DEBUG` setting is set to `True`.

We will start this chapter by looking at the architecture used in a production web server – that is, where the functions of the Django development server are split and handled by two

different applications. There is a frontend web server (for example, NGINX, `lighttpd`, or Apache), which receives the request from the browser, and the application server (for example, Gunicorn or `uWSGI`), which executes the Python code. The frontend web server decides how the request should be handled. If it is a request for a static or media file, then the frontend web server can handle the request itself; it can just read and send the file. If the request is for Python-specific code, then it will be forwarded to the application server to handle. The Django application then parses the URL and other HTTP data, generates an `HttpRequest` object using a view, and sends the response back to the frontend web server. The frontend web server then passes the response back to the browser.

The following figure elaborates on this process:

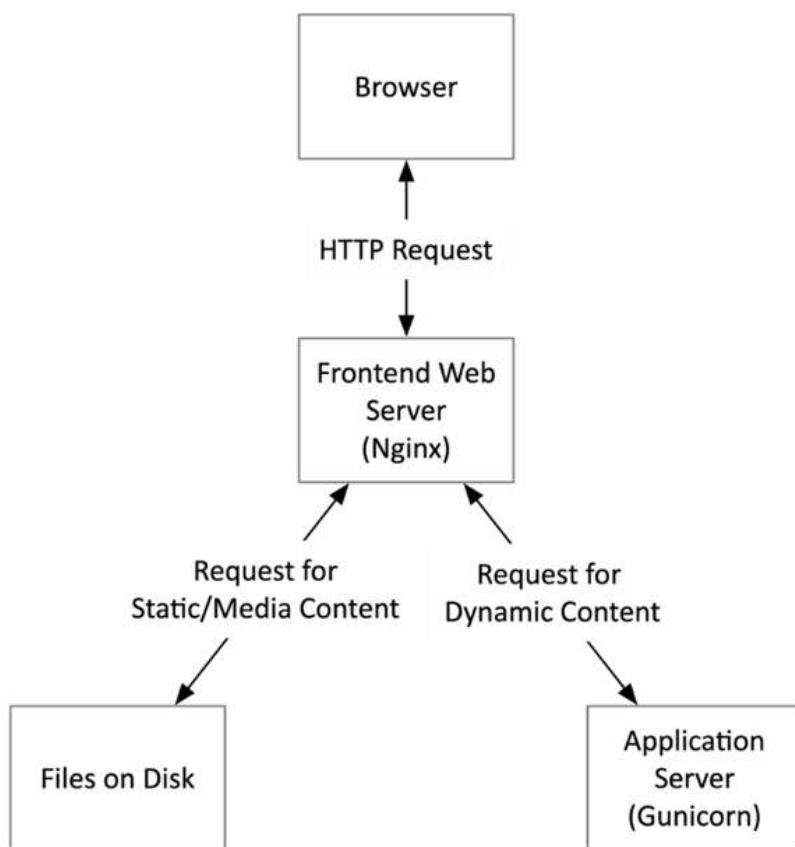


Figure 15.1: Request validation

Gunicorn

Gunicorn is an easy-to-use application server. Its settings can be controlled with command-line flags or a configuration file. In its simplest usage, it can be run like this:

```
gunicorn bookr.wsgi:application
```

This starts Gunicorn using the Django WSGI.

Note

WSGI (often pronounced *wizz-gee*) stands for **Web Server Gateway Interface**. It describes a standard for Python to communicate with web servers. This means any WSGI Python script should be able to communicate with any WSGI server, and any combination of frontend server and application server can be used together, provided they both "speak WSGI." For example, instead of using Gunicorn, we could use uWSGI. Its configuration would be different, but it serves the same purpose.

Whitenoise

For a production environment with significant traffic, a frontend web server such as NGINX is required to handle the static and media content. However, if we are setting up a site with modest demand, then we can install Django middleware called Whitenoise that will fulfill this role, and an application server (such as Gunicorn) can be run without a frontend web server.

Note

Since 2019, Django has introduced the **Asynchronous Server Gateway Interface (ASGI)**, which is used to connect with asynchronous servers such as Uvicorn and Daphne. Asynchronous servers are frequently used for long-lived connections and are frequently seen in applications such as chatbots, site notifications, live update dashboards, and streaming services. ASGI is outside the scope of this book.

In the next section, we will examine the deployment checklist in preparation for a Gunicorn/Whitenoise deployment.

The deployment checklist

Before a Django application is deployed, it needs to be configured in a more robust way, as it is no longer just accessible by the developer but is exposed to the whole internet.

The Django documentation gives a security checklist that should be followed before deploying an app to a production environment: <https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/>.

A development environment will typically fall short of the security needs of a production environment. We will discuss the tasks of the Django deployment checklist that are required to harden a Django project and then address them for the Bookr app in *Exercise 15.01 – Configuring the Bookr project for deployment*.

We will give a brief on the items in the checklist next.

Run `manage.py check --deploy`

When we start the Django development server with the `manage.py runserver` command, we might notice a line in the console that says this:

```
System check identified 0 issues (0 silenced).
```

This line is output from the `check` subcommand, which is used to detect and report on configuration issues in a Django project.

We can invoke it independently of the development server with this command:

```
$ python manage.py check
```

It can be used to assess the state of a Django application for production deployment by including the `--deploy` flag:

```
$ python manage.py check --deploy
```

This will alert us to security issues in our project configuration that we will need to address in a production environment before deployment, as seen in *Figure 15.2*.

```
$ python manage.py check --deploy
System check identified some issues:

WARNINGS:
?: (security.W004) You have not set a value for the SECURE_HSTS_SECONDS setting.
   If your entire site is served only over SSL, you may want to consider setting a
   value and enabling HTTP Strict Transport Security. Be sure to read the document
   ation first; enabling HSTS carelessly can cause serious, irreversible problems.
?: (security.W008) Your SECURE_SSL_REDIRECT setting is not set to True. Unless y
   our site should be available over both SSL and non-SSL connections, you may want
   to either set this setting True or configure a load balancer or reverse-proxy s
   erver to redirect all connections to HTTPS.
?: (security.W009) Your SECRET_KEY has less than 50 characters, less than 5 uniq
   ue characters, or it's prefixed with 'django-insecure-' indicating that it was g
   enerated automatically by Django. Please generate a long and random value, other
   wise many of Django's security-critical features will be vulnerable to attack.
?: (security.W012) SESSION_COOKIE_SECURE is not set to True. Using a secure-only
   session cookie makes it more difficult for network traffic sniffers to hijack u
   ser sessions.
?: (security.W016) You have 'django.middleware.csrf.CsrfViewMiddleware' in your
   MIDDLEWARE, but you have not set CSRF_COOKIE_SECURE to True. Using a secure-only
   CSRF cookie makes it more difficult for network traffic sniffers to steal the C
   SRF token.
?: (security.W018) You should not have DEBUG set to True in deployment.
?: (security.W020) ALLOWED_HOSTS must not be empty in deployment.

System check identified 7 issues (0 silenced).
```

Figure 15.2: Deployment warnings

We can use this command to evaluate our efforts to secure a Django configuration and will reassess it after creating a robust production environment in *Exercise 15.01*.

Switch away from manage.py runserver

Django's internal runserver functionality is a useful development tool, but as a single-process server, it cannot be deployed in a production environment. A production deployment requires a high-concurrency web server. We will examine the use of `gunicorn` for our deployment.

Critical settings

These are the most crucial settings in a Django project that need to be addressed prior to deployment.

SECRET_KEY

In the check management command, we saw the following warning:

```
?: (security.W009) Your SECRET_KEY has less than 50 characters, less than 5 unique
characters, or it's prefixed with 'django-insecure-' indicating that it was
generated automatically by Django. Please generate a long and random value,
otherwise many of Django's security-critical features will be vulnerable to
attack.
```

When a new Django project is created, a less secure SECRET_KEY is generated. Prior to deployment, it is necessary to create a more secure key for the production environment. Django comes with a function called `get_random_secret_key()` that produces a key that conforms to its current best practice standards. It can be invoked as follows:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

We also need to be mindful that this key is not present in the source code of the project, and we can enforce this using an environment variable, decouple, or a secrets file.

DEBUG

Setting DEBUG to True in a development environment is an invaluable way to receive useful information, such as tracebacks when encountering errors in web requests. However, this same information can be a security vulnerability if it is available in a production environment, as it can reveal snippets of source code, dependencies, and settings.

Environment-specific settings

ALLOWED_HOSTS

We have gotten this far leaving the value of ALLOWED_HOSTS as an empty list in `settings.py`. However, it is required that it is not empty in a production deployment:

```
?: (security.W020) ALLOWED_HOSTS must not be empty in deployment.
```

Indeed, when DEBUG is set to False, an empty list for ALLOWED_HOSTS will throw an exception:

```
$ python manage.py runserver
CommandError: You must set settings.ALLOWED_HOSTS if DEBUG is False.
```

The `ALLOWED_HOSTS` setting was introduced into Django as a mechanism to stop host header poisoning attacks. This is where an HTTP request would contain a header to a malicious site, and it would be used to generate a subsequent false link in an HTML page or email to the malicious site.

If you are deploying to a server with a known domain, this is where you should add it:

```
ALLOWED_HOST=[ 'example.com' ]
```

If you are deploying to an environment where the domain is not yet known, a wildcard can be used, but this is less secure and should be replaced when the domain is apparent:

```
ALLOWED_HOSTS=[ '*' ]
```

CACHES

Web caching is the mechanism that allows the responses of frequently made requests to be stored for repeated use in order to avoid unnecessary recomputations that would impact the CPU, filesystem, or database resources.

While we haven't addressed Django's caching abilities in this book, caching is an important strategy for a web application that can be fine-tuned post-deployment. Further information can be found at the following link: <https://docs.djangoproject.com/en/6.0/topics/cache/>.

DATABASES

During development, it is satisfactory to work with a SQLite database, but in a production environment, where multiple HTTP requests will require their own database connections, we will need to scale up from a lightweight database to a production-ready deployment.

We should avoid leaving database connection parameters in a file such as `settings.py`. The database server should be configured to only allow connections from application servers and devices that host backups.

EMAIL_BACKEND and related settings

We discussed email configuration in *Chapter 10, Configuring an Email Server with Django*, and gave an example with a Gmail account. Email configuration is required for automating administration tasks (such as password resets) and application communications.

Static files

So far, we have been using the development server to serve static files. In a production environment, we will need to define a `STATIC_ROOT` directory where `collectstatic` will copy them during the deployment.

Media files

Media files are uploaded by users and should never be trusted. So, for example, don't develop a Django app that interprets uploaded Python files.

Consider what your backup strategy is for uploaded content.

HTTPS

Data leakage over insecure HTTP connections is a perennial issue in web security. These two settings force the use of HTTPS connections to transmit sensitive data.

CSRF_COOKIE_SECURE

Setting this variable to `True` enforces the transmission of the CSRF cookie over HTTPS. We typically need this set to `False` while we are developing on `http://localhost:8000` and don't have a secure connection available, but it is imperative to set it to `True` in production so that the CSRF cookie is not transmitted over an insecure protocol.

SESSION_COOKIE_SECURE

Similarly, `SESSION_COOKIE_SECURE=True` enforces that the session cookie is transmitted over HTTPS.

Performance optimizations

These are some of the performance optimizations that should be considered prior to deployment.

Sessions

It is recommended that a cached session package be used, as it has the advantage of reducing the impact on the database. Also, when we are developing our application, we are generating a low rate of user interaction, but in a production environment, there is significant session activity, and the database tables that maintain this can grow incrementally if older session data is not periodically purged.

CONN_MAX_AGE

The `CONN_MAX_AGE` setting allows database connections to persist on a web server thread between requests. This means that if a worker thread is busy sequentially servicing requests, it does not need to make separate database connections for each, as a persistent connection has been established.

TEMPLATES

Template caching improves Django performance as it means that templates do not need to be read from disk and compiled into Python for each request, but remain in memory as Python template objects. The default behavior since Django 4.1 is to enable template caching, and it is implemented internally in Django, independently of caching mechanisms, and doesn't require the explicit configuration of a cache (as is the case with session or view caching).

This means that the default template loader is set to `django.template.loaders.cached.Loader` unless it has been explicitly disabled for debugging purposes. If you have disabled template loading in the development environment, be sure to re-enable it during deployment.

Error reporting

LOGGING

During debugging, we typically view logging in the console and traceback errors in our application browser. A production environment should produce logging that is more persistent and trackable.

ADMINS and MANAGERS

The default behaviour of Django's error logging is to send an email to everyone in the ADMINS list if there is a 500 error. ADMINS is a list of email addresses configured in the `<project>/settings.py` file. By default, it is empty, but it is good to add relevant email addresses of developers and administrators in order to receive information about 500 errors, since this information is no longer displayed in the browser when `DEBUG` is `False`.

Similarly, the MANAGERS list is configured in the `<project>/settings.py` file. Recipients in this list receive emails about broken links in the project when the `BrokenLinkEmailsMiddleware` is enabled:

```
ADMINS = ['systemadmin@bookr.example.com',
          'alice@example.com']
MANAGERS = ['contentmanager@bookr.example.com',
            'david@example.com', 'eve@example.com']
```

Customize the default error views

Django ships with default templates for 400, 403, 404, and 500 errors. It might be preferable to render these errors in a manner that is consistent with the rest of your project's design. This can be done by adding your own `400.html`, `403.html`, `404.html`, and `500.html` template files into the project's root template folder.

With these considerations in mind, we can set about configuring our Bookr project for deployment in the following exercise.

Exercise 15.01 – Configuring the Bookr project for deployment

In this exercise, we are challenged with applying the Django deployment checklist to our Bookr project before it is ready for a production environment:

1. Our first concern with the Bookr project is that the `bookr/settings.py` file contains the value of the `SECRET_PASSWORD` setting. A safer option is to load it from either an environment variable or an external file, so that it is decoupled from the source code of the Django project. We will also add a variable in our settings that acts as a switch. It will be called `ENVIRONMENT`, and its value will either be `'Development'` or `'Production'`.

In *Chapter 10*, we introduced the `decouple` module and imported it into `bookr/settings.py`. We started a `.env` file to include the sensitive or environment-specific settings:

```
from decouple import config
```

```
SECRET_KEY = config('SECRET_KEY')
ENVIRONMENT = config('ENVIRONMENT')
```

We will need to add the `SECRET_KEY` and `ENVIRONMENT` settings to the `.env` file that we created in the top-level directory of our project:

```
SECRET_KEY=...
ENVIRONMENT=development
```

While the existing `.env` file is being used in our development environment, we can create another `.env` file in a secure location where we will store values that we set for production. In this exercise, we will refer to these files as `development .env` and `production .env` in order to avoid confusion.

We can use this command to generate a strong value for the secret key in the `production .env`, before adding it to the file:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

```
SECRET_KEY=...  
ENVIRONMENT=production
```

2. Using the `ENVIRONMENT` switch, we can ensure that `DEBUG` is only set to `True` in development environments:

```
DEBUG = True if config(ENVIRONMENT)=='development' \  
        else False
```

3. At present, our project has no allowed hosts set. We can use the `config` function to load the value from the `.env` file. For now, we will just add a wildcard value for `ALLOWED_HOSTS` in production `.env`, but if we had a domain name for our production server, this is where we would add it.

By default, `decouple.config` loads values from the `.env` file as strings. As `ALLOWED_HOSTS` is a list, we need a method of casting it. We can use the `Csv()` method of `decouple` that parses a setting using the comma-separated value format that we are familiar with. So, if the `.env` file contains `ALLOWED_HOSTS=a.site,b.site`, then it will be parsed by `config` as `['a.site', 'b.site']`:

```
from decouple import config, Csv  
  
ALLOWED_HOSTS = config('ALLOWED_HOSTS', Csv())
```

The development `.env` file will contain `ALLOWED_HOSTS=`, which will be parsed as an empty list.

We will provisionally set the wildcard in the production `.env` file by adding `ALLOWED_HOSTS=*`.

4. The current database configuration points to a SQLite3 database in the project folder, `sqlite3.db`. We will keep this for development, but add a PostgreSQL configuration in our production environment. At present, the settings that we will use here are placeholders, but we will change them later during deployment. In order to connect to a PostgreSQL database, we need to install the `psycpg-binary` library:

```
pip install psycpg2-binary
```

The standard way of configuring a Django database connector in the `settings.py` file is to specify a default database that contains keys for database `ENGINE`, `NAME`, `USER`, `PASSWORD`, `HOST`, and `PORT`. Configuring six different parameters for a database is an error-prone approach, particularly if the data is being split between the `settings.py` and `.env` files, and it would be preferable to specify a single database connection URL. Although Django does not have this functionality out of the box, there is a Python module that implements it - `dj-database-url`.

We can install it using `pip`:

```
pip install dj-database-url
```

Once installed, we can specify a database string by using the `parse` function:

```
dj_database_url.parse(  
    'postgres://user:password@host:5432/database',  
    conn_max_age=600,  
    conn_health_checks=True)
```

Our intention is to use PostgreSQL in production, but we would like to have the option to run it in our development environment too, so we don't want to use the `ENVIRONMENT` variable to switch between the two.

A better strategy is to check whether the database URL is defined in the `.env` file and to follow that if it is present, or fall back to a SQLite3 database if it is absent.

We will also configure settings for the `conn_max_age` and `conn_health_checks` parameters of the `dj_database.parse` function:

bookr/settings.py

```
DATABASE_URL = config('DATABASE_URL', default='')  
CONN_MAX_AGE = config('CONN_MAX_AGE',  
                      default=0, cast=int)  
CONN_HEALTH_CHECKS = config('CONN_HEALTH_CHECKS',  
                             default=False, cast=bool)  
  
if DATABASE_URL:  
    DATABASES = {  
        'default': dj_database_url.parse(  
            DATABASE_URL,
```

```

        conn_max_age=CONN_MAX_AGE,
        conn_health_checks=CONN_HEALTH_CHECKS)
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }

```

So, if we don't intend to use a database for the development environment, we just don't include these settings in development .env, and the project will be configured with a SQLite3 database.

If we do have a PostgreSQL database, we might include something similar to what follows in the .env file (we will obtain the actual connection parameters during our Render .com deployment in *Exercise 15.02 – Deploying the Bookr project*):

.env

```

DATABASE_URL=postgres://<user>:<password>@<host>:<port>/<database>
CONN_MAX_AGE=600
CONN_HEALTH_CHECKS=True

```

5. Our development server serves the files from STATIC_DIR. However, in our production environment, this functionality is decoupled from the gunicorn server and either requires a standalone server such as NGINX to serve the static files or the use of middleware such as Whitenoise, which co-opts gunicorn for the purpose. For ease of deployment, we will install Whitenoise:

```
pip install whitenoise
```

We will also need to specify this middleware in the settings.py file and assign a location for STATIC_ROOT where the collectstatic process can save the files during deployment:

bookr/settings.py

```

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',

```



```
        'whitenoise.middleware.WhiteNoiseMiddleware',
        ...
    ]
    ...
    STATIC_ROOT = BASE_DIR / 'staticfiles'
```

While we won't make any changes to `MEDIA_ROOT` and `MEDIA_URL` in this setup, it is worth noting that not all hosted deployment environments use persistent file storage environments.

6. In the HTTPS settings, we want to ensure that CSRF cookies and sessions are only ever served over HTTPS in production. As our development environment uses HTTP – `http://localhost:8000/` – we don't want to force this behavior in a development environment: **`bookr/setting.py`**

```
if config(ENVIRONMENT)=='development':
    CSRF_COOKIE_SECURE = False
    SESSION_COOKIE_SECURE = False
else:
    CSRF_COOKIE_SECURE = True
    SESSION_COOKIE_SECURE = True
```

These are the essential changes that need to be made to the `bookr/settings.py` and `.env` files in order to prepare for deployment.

You can find the complete code in the Chapter15 folder in this book's GitHub repo.

Deployment options for Django

- While we have settled on a high-performance database and web server, such as gunicorn, we are still presented with several options for deployment of the Django project. There is a range of solutions that are listed here.

Manual deployment

- Traditionally, Django projects were manually deployed by system administrators with a knowledge of the Linux command-line environment. Databases and web servers were manually configured and maintained by these administrators. This kind of approach might be appealing if you have existing infrastructure to deploy to, but for many developers today, it requires additional resources and skill sets outside of their abilities.

Containerized deployment

Containerized deployment involves developing "containers" for the configuration of lightweight virtual server modules that specify server requirements and application code necessary for the deployment using pre-existing platform modules.

Docker is the entry-level containerization platform, and it interfaces with an ecosystem of high-performance tools such as Docker Compose, Docker Swarm, and Kubernetes.

This is an example of a Docker file specification needed to deploy and run a minimal Django project:

```
# Specify a pre-built docker container
FROM python:3.12

# Set the working directory in the container
WORKDIR /usr/src/app

# Copy project code into the container's working directory
COPY . .

# Install the dependencies from the requirements file
RUN pip install --no-cache-dir -r requirements.txt

# Expose port 8000 to the host machine
EXPOSE 8000

# Command to run the gunicorn production server
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "bookr:wsgi"]
```

Platform-as-a-service

Platform-as-a-service (PaaS) providers offer a hosting environment that can be utilized without having a background in system administration. These are cloud platforms that will host a Python environment capable of running a Django project and its auxiliary services, such as a database server, backups, and static file hosting.

Popular entry-level PaaS providers include **Render.com**, **PythonAnywhere.com**, **Seenode.com**, and **Railway.com**. The established cloud providers, including Digital Ocean, AWS, and Google Cloud Platform, also cater to PaaS configurations.

Many of these providers have detailed HOWTO documents and videos on deploying Django projects on their platforms. In the following exercise, we will conduct a walkthrough of a Django deployment with the Render.com PaaS.

Exercise 15.02 – Deploying the Bookr project

In this exercise, we will work through the steps involved in the deployment of a Django project to a PaaS environment. We will be creating a repository for the Bookr project using GitHub.com and then using Render.com to host it. This is a simple deployment strategy that involves leveraging web services and doesn't require a DevOps or system admin background.

Note

At the time of writing, Render.com has a free tier in its pricing structure. There are some limitations with this option, as it doesn't give command-line access, file storage is not persistent, and the database is reset after 30 days, with the loss of data.

There is information on the pricing tiers here at <https://render.com/pricing>.

In order to follow these instructions, you will need to create a `github.com` account if you do not have one already. The exercise has been structured so that no knowledge of the command-line Git (or a GUI) is required. However, if you have expertise with Git, you may create the repository in the way that you are most familiar with.

Note

It is assumed that you have followed the code examples in this book and have a working version of the Django Bookr project on your local computer.

If you don't, and you want to follow the deployment, you can download the code from this path in the `Chapter15` folder in this book's GitHub repo. This version was developed with Python 3.12.4.

Log in to `github.com` and create a new repository by selecting **New repository** from the + menu, as in *Figure 15.3*.

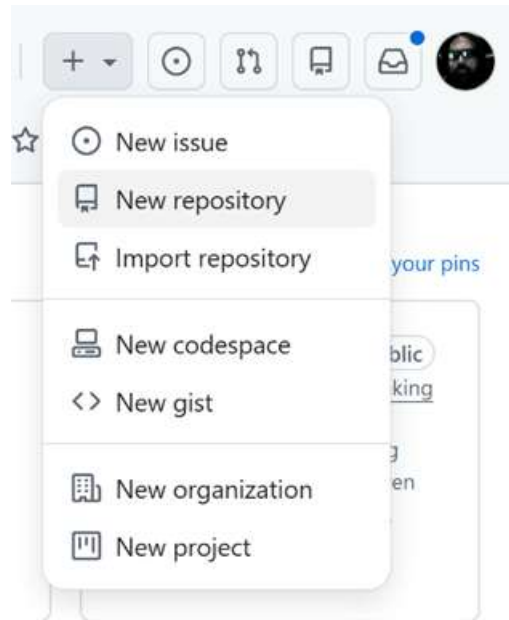


Figure 15.3: Adding a new repository in GitHub

Now, we see a blank form where we enter the details for our new repository, as in *Figure 15.4*.

In the **General** section of the form, we can give the repository a name such as `bookr`. In the **Configuraton** section, do the following:

Set **Choose visibility** to **Public** (this simplifies the `render.com` deployment process)

For **Add .gitignore**, select **Python** from the dropdown.

Now, we can click the **Create repository** button at the bottom of the screen.

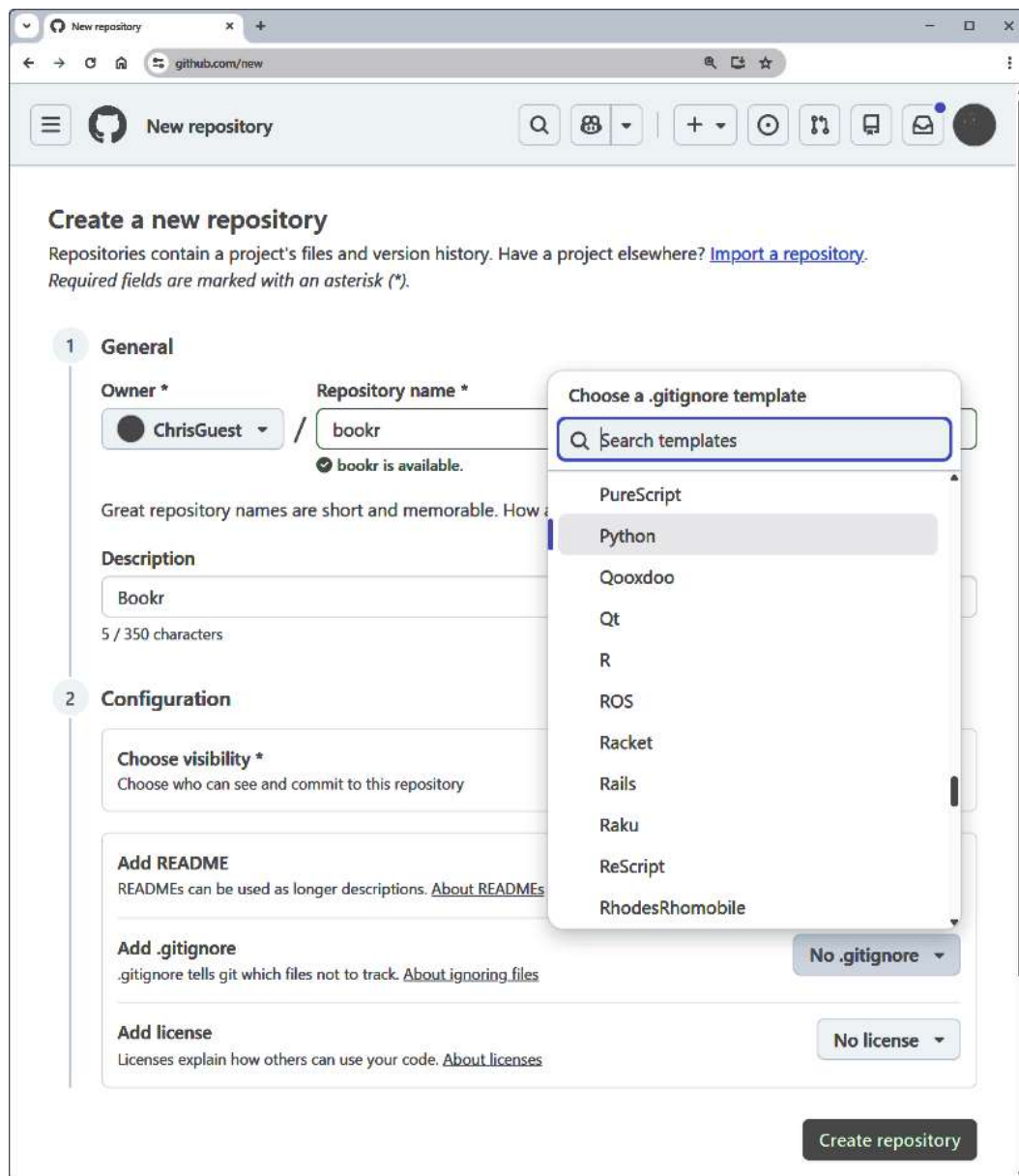


Figure 15.4: Creating a new repository

This will create a repository with a single file called `.gitignore`. This file tells Git which types of files shouldn't be tracked in the repository. The Python `.gitignore` lists SQLite database files, `__pycache__` directories, and a variety of other system files that are generated by the

runtime environment or operating system but shouldn't be maintained as part of the code repository.

If we select the **Upload files** option from the + menu at the top of *Figure 15.5*, we get to a page with a drop zone for drag and drop functionality, as in *Figure 15.6*.

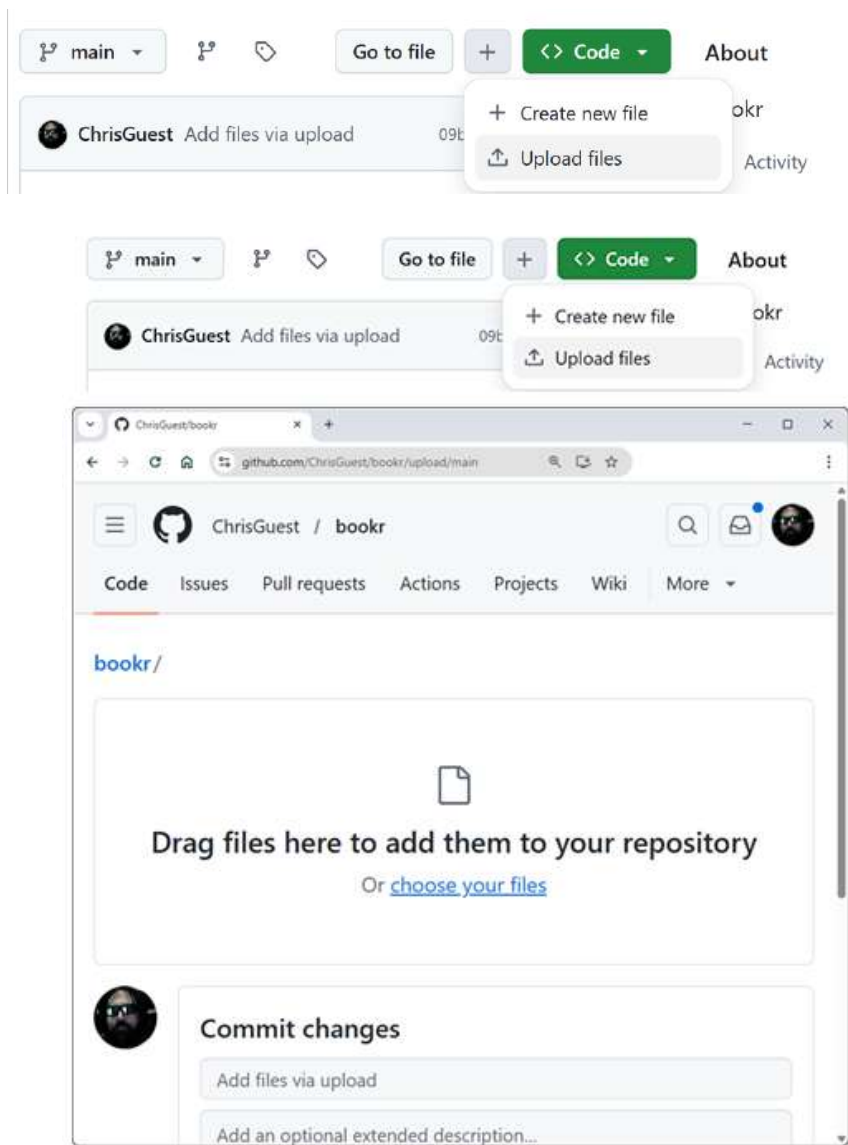


Figure 15.6: The GitHub drop zone

In our Finder or File Explorer, we can browse inside the bookr project folder, selecting all relevant files and folders except the `db.sqlite3` file, `__pycache__`, or the `.env` file, as in *Figure 15.7*. These files can be dragged into the drop zone marked **Drag files here to add them to your repository**.

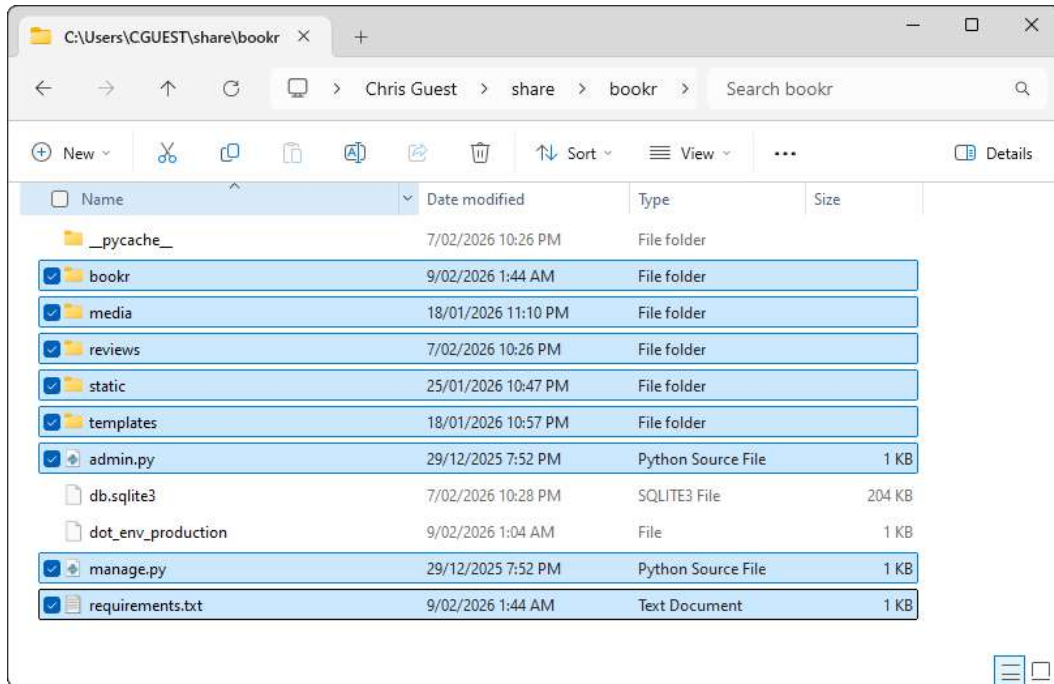


Figure 15.7: The selected files to be dragged and dropped from the filesystem

Now, we can see the files that have been added to this repository in *Figure 15.8*. Although we have a `.gitignore` file, this drag-and-drop step circumvents it, and some unwanted files will have been included, such as the `.pyc` files depicted. These can be manually deleted from the Git commit by clicking the icon link next to each file.

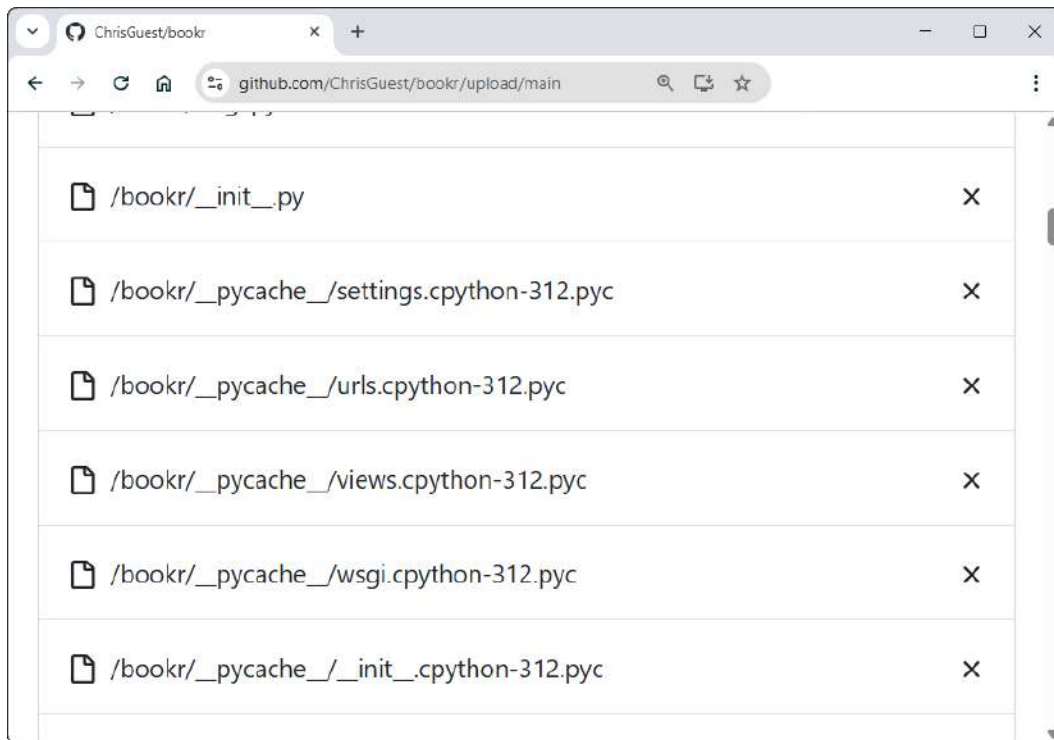


Figure 15.8: The files added to the GitHub repository

At the bottom of this page is the **Commit changes** dialog. We can add a commit message and an optional extended description before clicking the green **Commit changes** button.

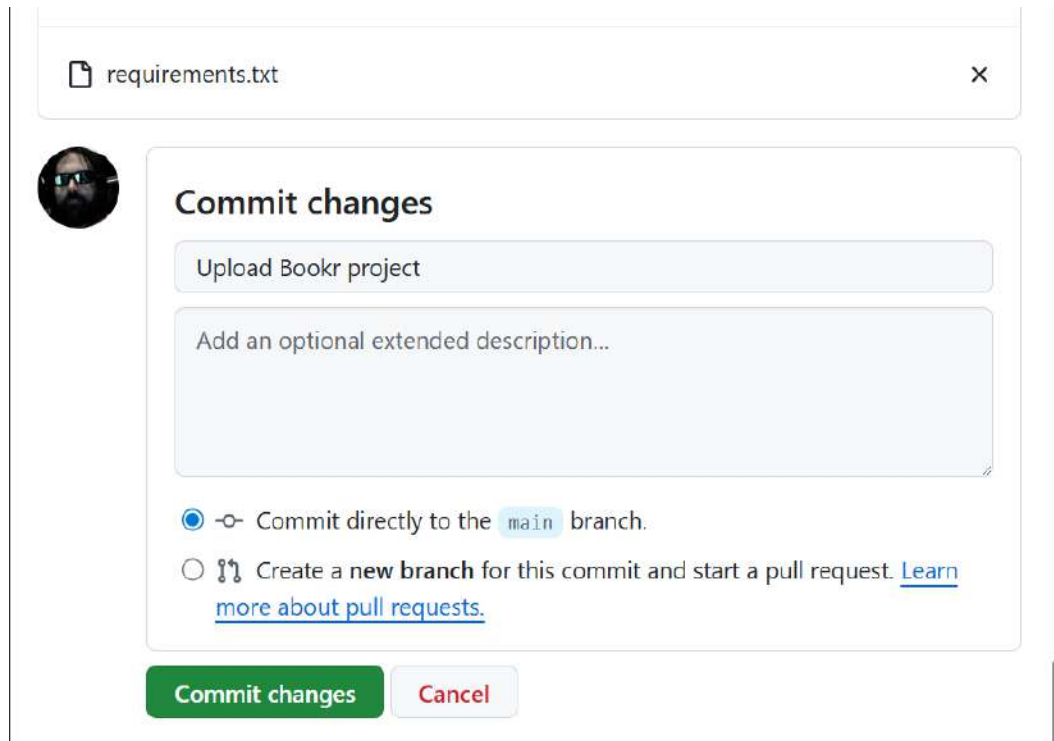


Figure 15.9: The GitHub Commit changes dialog

We have now loaded our Bookr project into a Git repository called bookr that will be publicly visible at <https://github.com/<username>/bookr>, as per *Figure 15.10*.

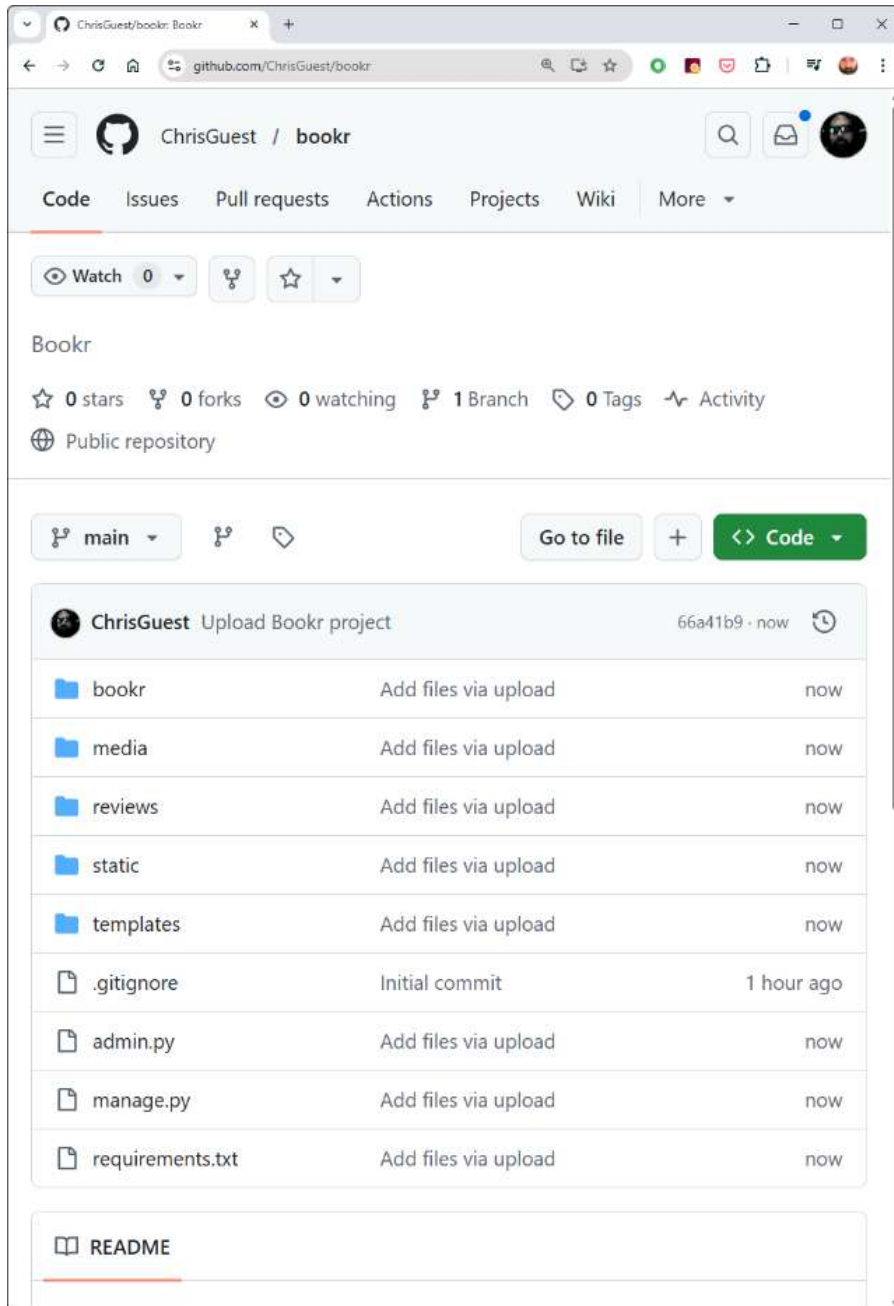


Figure 15.10: The repository containing the Bookr project

Now that we have our repository built, we can create an account on `render.com` and build the necessary services for the deployment. We can use the free account tier, **Hobby - For hobbyists and students**. After you have created your account, you will see the dashboard where you can create new services.

Render.com offers **Static Sites**, **Web Services**, **Private Services**, **Background Workers**, **Cron Jobs**, **Postgres**, and **Key Value** as its PaaS components. For this exercise, we will be deploying Postgres and a web service.

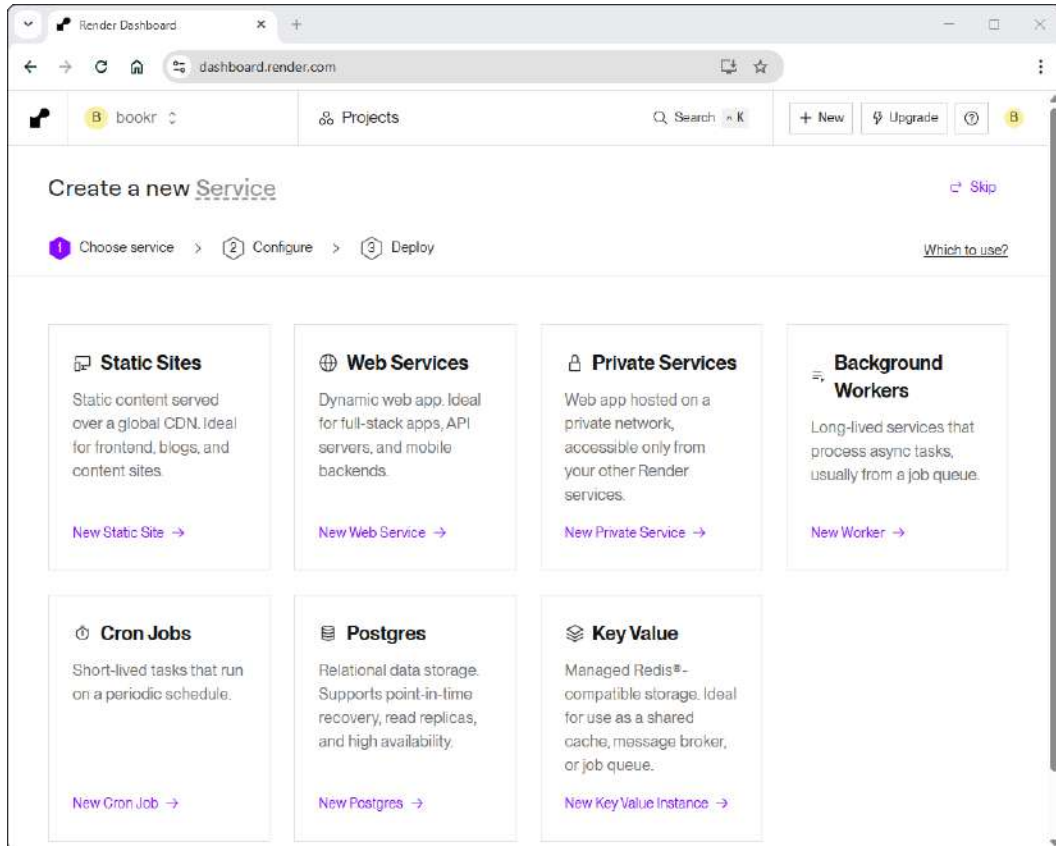
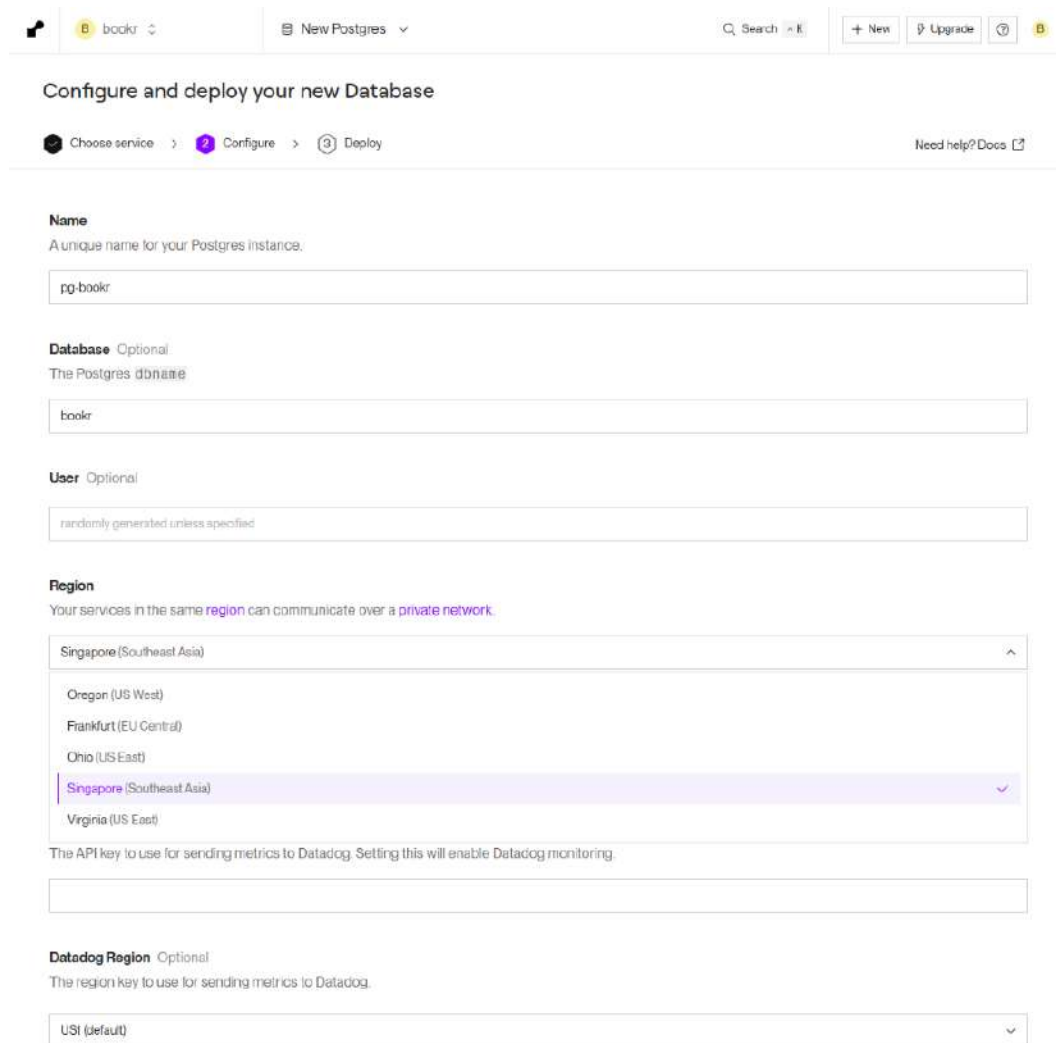


Figure 15.11: The Render.com dashboard

Select **New Postgres** from the Render dashboard. You will see a form to configure and deploy your new database, as in *Figure 15.12*. Give an appropriate instance name and database name, such as `pg-bookr` and `bookr`, respectively. Select a region that you expect will be closest to your user base. At present, Render offers hosting sites in North America, Europe, and Asia.



The screenshot shows the 'Configure and deploy your new Database' form in the Render dashboard. The form is titled 'Configure and deploy your new Database' and has a breadcrumb trail: 'Choose service' > 'Configure' > 'Deploy'. A 'Need help? Docs' link is also present. The form contains several sections:

- Name:** A unique name for your Postgres instance. The input field contains 'pg-bookr'.
- Database:** Optional. The Postgres database name. The input field contains 'bookr'.
- User:** Optional. The input field contains 'randomly generated unless specified'.
- Region:** Your services in the same region can communicate over a private network. A dropdown menu shows the following options: Singapore (Southeast Asia), Oregon (US West), Frankfurt (EU Central), Ohio (US East), Singapore (Southeast Asia) (selected), and Virginia (US East).
- Datadog Region:** Optional. The region key to use for sending metrics to Datadog. The input field is empty.

At the bottom, there is a section for 'Datadog Region' with a dropdown menu showing 'US1 (default)'.

Figure 15.12: The new database form

For now, we can scroll down and select the **Free** instance type. At the time of writing, Render does not persist Free Postgres instances after the 30-day trial period and does not give access to their hosted backup services. A Postgres instance can be upgraded to a paying tier during this time. The Free tier currently offers 1 GB of storage, which is sufficient for an initial deployment of many applications.

Plan Options

Instance Type

Set your database's RAM and CPU. You can change your instance type later.

Storage

Your database's capacity, in GB. You can increase storage later, but you can't decrease it.
Specify 1 GB or any multiple of 5 GB.

Storage Autoscaling

When your database is 90% full, automatically increase its storage by 50% (rounded up to the nearest 5 GB).
Max one increase per 12 hours. [Learn more](#)

High Availability

Run a standby instance of your database and automatically fail over to it if the primary encounters an issue. [Learn more](#)

Free

For testing out Pender Postgres

Free

\$0 / month

256 MB (RAM)

0.1 CPU

1 GB (Storage)

Basic

Reliability and performance for hobby projects. Starting at \$6 / month plus storage.

Pro

Perfect for production use cases at any scale. Starting at \$55 / month plus storage.

Accelerated

Memory-optimized plans offer significant performance improvements. Starting at \$180 / month plus storage.

Need a custom instance type? We support up to 1024 GB RAM, 128 CPUs, and 16 TB storage.

1

GB

\$0 / month

Disabled

Disabled

Only available for Pro instances and higher.

Figure 15.13: Render.com plan options

Once you have confirmed that the **Monthly Total** cost is expected, click the **Create Database** button.

Monthly Total

Database instances are billed and prorated by the second.

Instance: Free

\$0 / month

Total

\$0 / month

Create Database

Figure 15.14: Render.com monthly total

You will see a summary of the created database instance. Take note of the database expiry date, as you will need to upgrade to a paid tier by this time or risk losing your data.

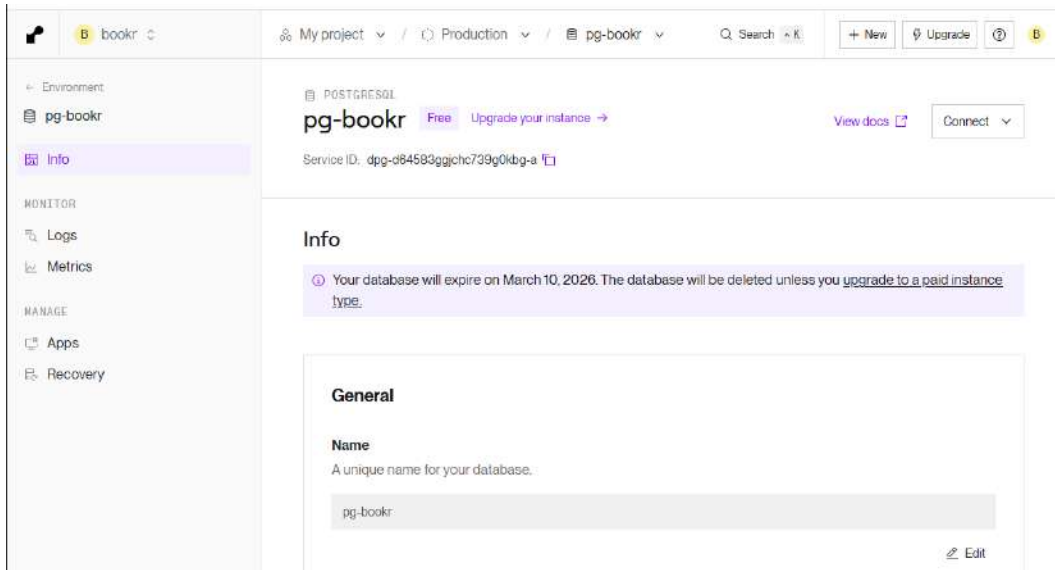


Figure 15.15: PostgreSQL database summary

Hit the *view* and *copy* icons of **Internal Database URL**.



Figure 15.16: Internal database URL

Add this string into the production `.env` file:

```
DATABASE_URL=postgresql://bookr_<...>_user:<long password>/bookr_<...>
```

Our Postgres service is now configured. We can start provisioning the web service.

We need to add a `PYTHON_VERSION` setting to our `.env` file. This reflects the version of Python that our Django project is running with. Render will use this information to provision an appropriate version of Python on the web service:

```
PYTHON_VERSION=3.12.4
```

From the Render dashboard, click **Add Web Service**. You will be taken to a form with web service details. Under **Source Code**, include your Git repository URL. Fill the **Name**, **Language**,

and **Branch** fields with bookr, Python 3, and main, respectively. Select the same region that you did for the Postgres service.

 B bookr

My project / Production / New Web Service

Search

+ New Upgrade

B

New Web Service

Source Code

Git Provider

Public Git Repository

Existing Image

PR Previews and Auto-Deploy are available only for repositories configured with Blueprints

 https://github.com/ChrisGuest/bookr/

Connect

Name

A unique name for your web service.

Bookr

Project

Optional

Add this web service to a project once it's created.

My project / Production

Language

Choose the runtime environment for this service.

Python 3

Branch

The Git branch to build and deploy.

main

Region

Your services in the same region can communicate over a private network. You currently have services running in Singapore.

 Singapore (Southeast Asia)

1 existing service

Deploy in a new region

Figure 15.17: The new web service form

Under **Environment Variables**, there is an option to add from `.env`. You will see a dialog box where you can paste your `.env` file. The environment variables will be extracted from the file and displayed in the form, as per *Figure 15.18*.

Variable Name	Value	Action
SECRET_KEY		✖
ENVIRONMENT		✖
ALLOWED_HOSTS		✖
EMAIL_HOST		✖
EMAIL_HOST_USER		✖
EMAIL_HOST_PASSWORD		✖
EMAIL_PORT		✖
EMAIL_USE_TLS		✖
EMAIL_USE_SSL		✖
DATABASE_URL		✖
SECURE_SSL_REDIRECT		✖
SECURE_HSTS_SECONDS		✖
PYTHON_VERSION	3.12.4	✖

Figure 15.18: Environment variables

Below **Environment Variables** is a section of the form containing **Build Command** and **Start Command**. Our build command will initially involve statements to install the requirements, create the database structure with the migrate subcommand, collectstatic, and create a superuser. These statements can be concatenated into a single build statement by using the `&&` operator. You can set your password in the statement by using a Linux environment variable, `DJANGO_SUPERUSER_PASSWORD`. Be sure to include your own username and password:

```
pipinstall-
rrequirements.txt&&pythonmanage.pymigrate&&pythonmanage.pycollectstatic--
```



```
noinput&&DJANGO_SUPERUSER_PASSWORD='<HardP@ssw0rd!>' python manage.py  
createsuperuser --noinput --username <yourname> --email <yourname@example.com>
```

On subsequent builds after the initial deployment, you can omit the last clause in this build command as the superuser has already been created.

For our environment, the start command will be `gunicorn bookr.wsgi`. Be sure that you included `gunicorn` in the `requirements.txt` file.

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

Build Command

Render runs this command to build your app before each deploy.

Start Command

Render runs this command to start your app with each deploy.

Figure 15.19: Root directory, build, and start commands

Now you can click the **Deploy Web Service** button at the bottom of the form, and the deployment will begin, as in *Figure 15.20*.

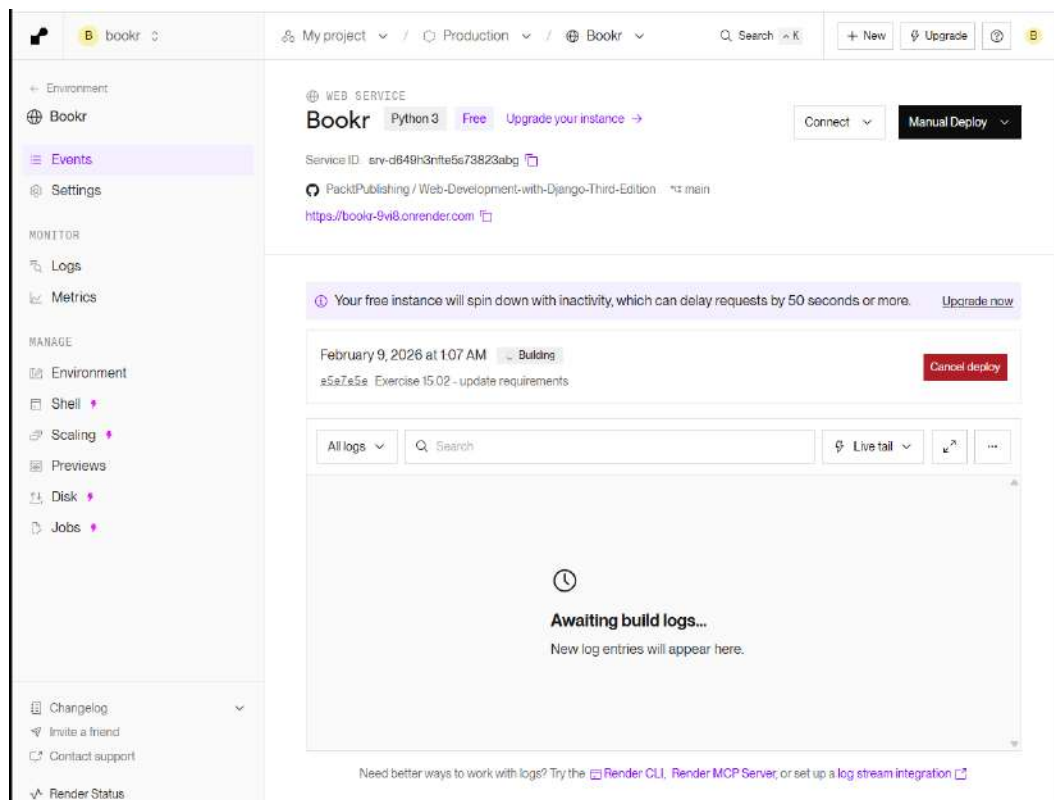


Figure 15.20: Web service build

Note that on this screen, there is a URL given for the target deployment. In the figure given, it is <https://bookr-9vi8.onrender.com>, but it will obviously differ across websites. Visiting this link during the deployment, you will see the service deployment status, as in Figure 15.21.

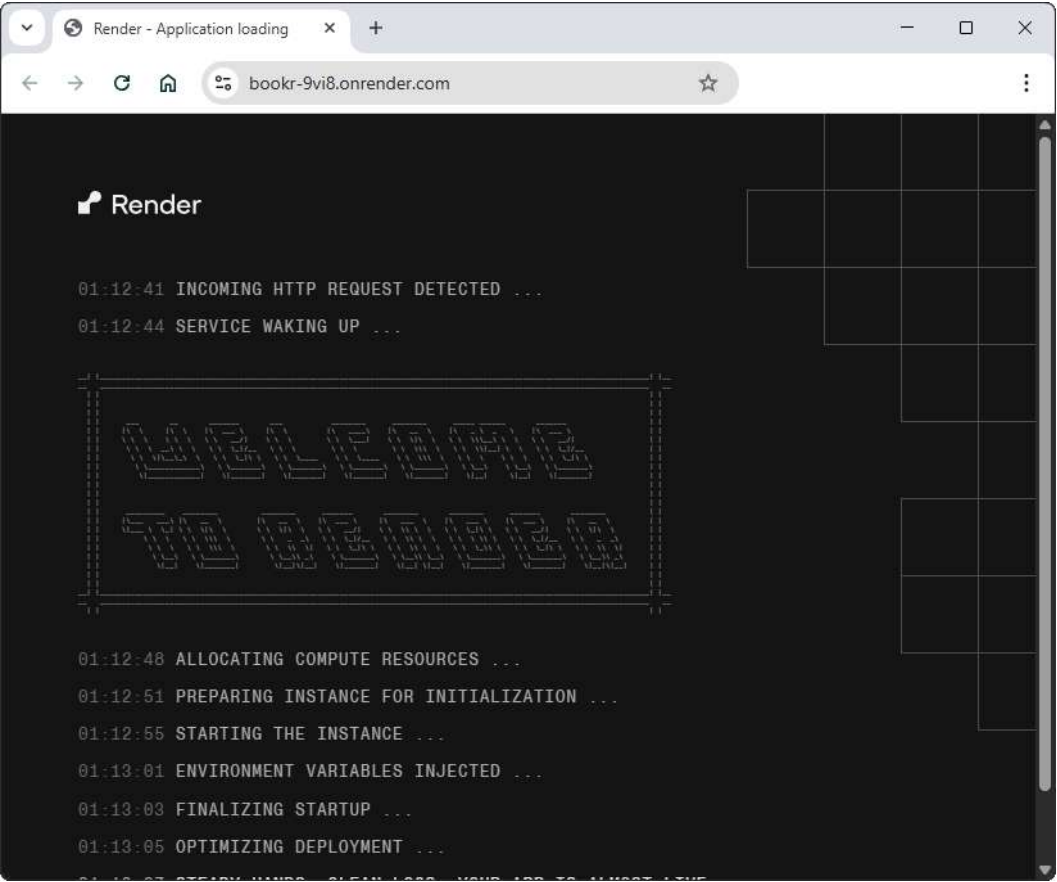


Figure 15.21: Render service deployment status

Eventually, if the deployment is successful, you will see the familiar web page that we have been developing.

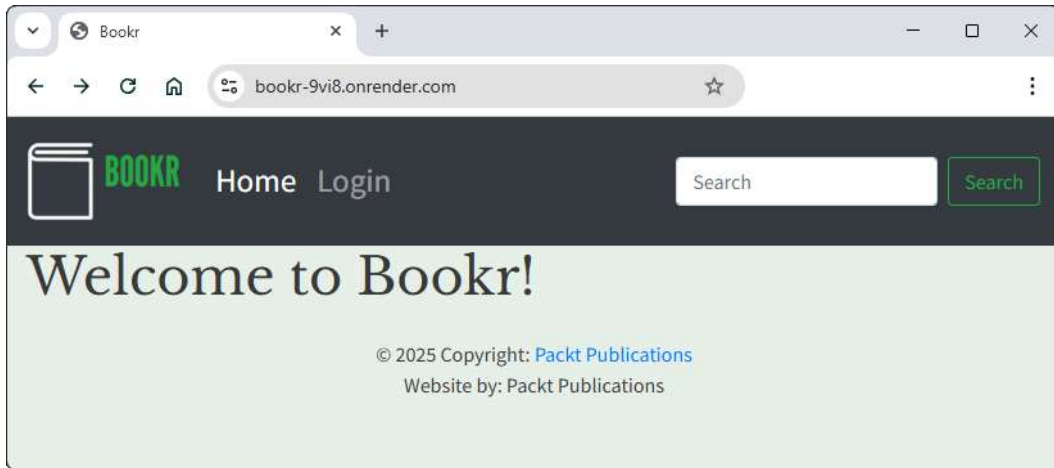


Figure 15.22: The deployed project

Keep an eye on the build screen for any errors. You may have entered a configuration parameter or environment variable incorrectly. There may even be errors in your source code that need to be fixed in the repository before redeploying. This is also the point where we want to add the allocated domain to the `ALLOWED_HOSTS` environment variable rather than use the wildcard:

```
ALLOWED_HOSTS=<yourdomain>.onrender.com
```

When rebuilding after an error or reconfiguration, the safest way to do it is by clicking on **Clear build cache & deploy** in the **Manual Deploy** menu.

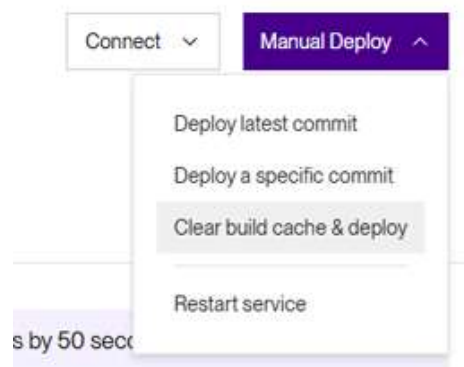


Figure 15.23: Clear build cache & deploy

If your deployment seems successful, confirm that the database has been migrated and the superuser created by logging in with the username and password details that you provided in *Step 16*.

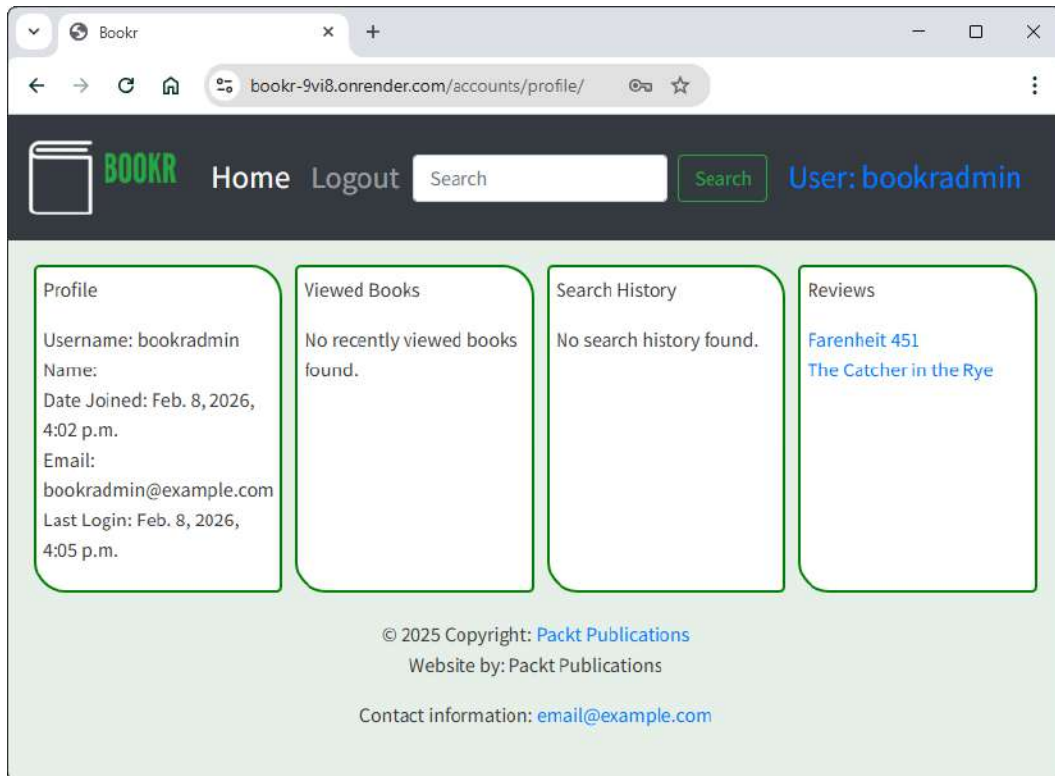


Figure 15.24: The Django project in production

With this, our production deployment is complete.

Summary

In this chapter, we have looked at simple architectures and deployment strategies for a Django project and learned the process of reconfiguring it from being a development instance to a production-ready deployment. Stepping through the deployment checklist, we have worked through the steps of a deployment to a PaaS provider, `render.com`.

Throughout this book, we have covered, in depth, everything required for setting up Python for installing Django: building an application to store data and accept data input through forms with validation, managing the system and the data through Django's admin system, and setting up a server to serve static files so that the application can be styled with CSS, enhanced

with JavaScript and display images. We have covered how to manage users and authentication, and in-depth customization to ensure Django is set up to better suit the project being undertaken. We also looked at how to provide data exports in formats such as CSV and PDF, concluding with how to make our application available to visitors by hosting the application on a server.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

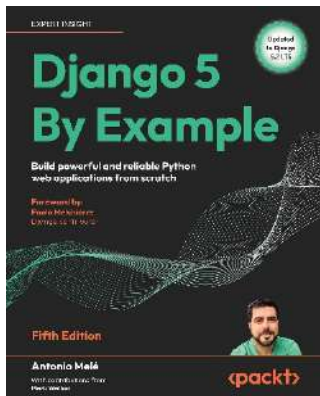
Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

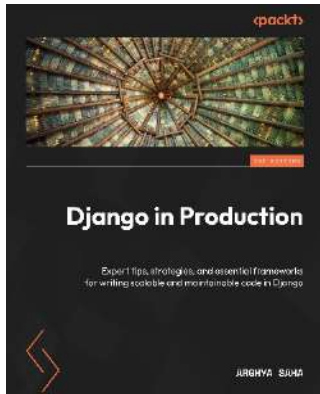


Django 5 By Example

Antonio Melé

ISBN: 9781805125457

- Use various Django modules to solve specific problems using the latest features
- Integrate third-party Django applications into your project
- Build complex web applications using Redis, Postgres, Celery/RabbitMQ and Memcached
- Set up a production environment for your project with Docker Compose
- Build a RESTful API with Django Rest Framework (DRF)



Django in Production

Arghya Saha

ISBN: 9781804610480

- Write scalable and maintainable code like a Django expert
- Become proficient in Docker for Django and experience platform-agnostic development
- Explore intelligent practices for continuous integration
- Leverage the power of AWS to seamlessly deploy your application in a production environment
- Optimize unstable systems through effective performance monitoring
- Effortlessly handle authentication and authorization issues
- Automate repetitive tasks by creating custom middleware
- Thoroughly test your code using factory_boy and craft comprehensive API tests

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packt.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Web development with Django 6*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.



<https://packt.link/r/1836202075>

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

A

ALLOWED_HOSTS 682, 683

API endpoint
creating, for top contributors page 587

API view
creating 579 – 582

AbstractUser
additional fields, adding to User class 492, 493
auth user model, replacing steps 493
password, modifying 493
subclassing 490
Username key, replacing 493

Admin app login template
repurposing 453 – 456

AdminSite
customizing 186 – 190
examining, from Python shell 181, 182
subclassing 182 – 186

AppDirectoriesFinder 217

Argon2 457

Asynchronous Server Gateway Interface (ASGI) 679

admin
automating, with Django Tasks 518 – 520

admin app 148
deletion behavior 177, 178
foreign keys 177, 178
groups, managing 160 – 165
models, registering 165
users, managing 160 – 165

admin interface
customizing 179

admin site
custom views, adding 511 – 515
custom views, restricting 511
customizing 488
views, adding 508

all() method

used, for retrieving set of objects 99, 100

app directories
HTML templates, finding 47

application programming interface (API) 531, 573, 656

assertions 646
types 649, 650
utilizing 646, 647

authentication
implementing 592
token-based authentication 593
views, testing 661

authentication app 443
templates, implementing 447 – 452
views, implementing 447 – 452

authentication data
used, for enhancing templates 466

authentication decorators 459 – 462
adding, to views 463 – 465

authentication models 444

authentication redirection 459 – 462

authentication schemes
reference link 593

authentication-based content
conditional blocks, using in templates 468, 469

automated test cases 644

automation testing 644

B

BCrypt/SHA256 457

Book Search page
session storage, using 484, 485

Book Search view
implementing, with Django Form 322 – 325

Book project's database
populating 117

Bookr 1
models and views, testing 673, 674

Bookr APIs			
used, for implementing token-based authentication	593 – 599		
Bookr project			
deploying	692		
Bookr project for deployment			
configuring	686 – 690		
Bootstrap	139		
for template styling	139, 140		
reference link	141		
base template			
creating	47 – 49		
login links, toggling	466, 467		
logout links, toggling	466, 467		
basic authentication	593		
binary file formats			
for data exports	607		
book catalog			
creating, with CBV	559 – 564		
book change page	173 – 177		
book cover			
displaying	438, 439		
book details view			
implementing	144		
book list item			
rendering, with template partials	556		
book search scaffold	63, 64		
bookr database			
entry, creating	90, 91		
browsers' sent values			
security and trust	390 – 392		
bubble plot	631		
generating	631 – 633		
built-in search			
using, to build custom admin dashboard	521, 522		
bulk_create() method	112		
used, for creating multiple records	112		
bulk_update() method	112		
used, for updating multiple records	113		
C			
CRUD operations, performing with CBVs	565		
Create view	565, 566		
Delete view	569, 570		
Read view	566		
Update view	567, 568		
CSS enhancements	260 – 262		
CSV files			
review summary CSV file, creating	604 – 606		
with Python csv module			
working with	602		
writing, with Python csv module	602 – 604		
Cascading Style Sheets (CSS)	139, 213		
Change publisher page	168 – 172		
Content Delivery Network (CDN)	225, 376		
Create operation	152 – 154		
user account, creating	154		
Create, Read, Update, and Delete (CRUD) operations	89, 147, 152		
Cross-Site Request Forgery Protection			
used, for form security	280		
cache	683		
cache invalidation			
file serving	248 – 250		
cached sessions	471		
change lists	167, 168		
class-based API views			
creating	584 – 586		
class-based views (CBVs)	120, 121, 531, 557, 558, 582		
in-built views	558		
testing	669		
used, for creating book catalog	559 – 564		
used, for performing CRUD operations	565		
clean method	332, 333		
code			
debugging	58 – 62		
simplifying, with ViewSets	588		
collectstatic management command			
static file, collecting for production	238, 239		
using	236 – 238		
comments	133		
common template variables			
accessing	510		
complex lookups			
performing, with Q objects	114		
complex query			
performing, with Q objects	114		
conditional blocks			

using, for authentication-based content in templates	468, 469	reference link	471
containerized deployment	691	Django Forms library	289, 290
context	52	building and rendering	307 – 311
context processor	381 – 383	form, defining	290 – 301
cookie-based sessions	471	form, rendering in template	301 – 307
create() method		used, for implementing Book Search view	322 – 325
used, for creating entry	92	Django ORM	68
cross-site request forgery (CSRF)	445	migration	72
custom admin dashboard		Django REST Framework (DRF)	573 – 575
building, with built-in search	521, 522	configuring	575
custom clean and validation methods		functional API views	576
implementing	338 – 345	installing	575
custom field validation	330	Django Storages	
custom filter function		reference link	255
implementing	535	Django Tasks	
custom filters		admin, automating	518 – 520
using, inside templates	536	periodic task exercise	523 – 527
custom inclusion tag		Django app	7, 25, 26
building	549 – 552	creating	7 – 9, 68 – 70
custom simple tag		Django application, deployment checklist	679
creating	544 – 546	environment-specific settings	682
template context, passing	547	manage.py check, running	680, 681
custom storage engines	255 – 257	manage.py runserver, switching	681
custom template filter		settings	681
creating	537 – 540	Django application, deployment checklist settings	
custom template filters	534	DEBUG	682
creating	534	SECRET_KEY	682
custom validators	331, 332	Django application, environment-specific settings	
custom view		ALLOWED_HOSTS	682, 683
adding, to admin site	511 – 515	cache	683
restricting, to admin site	511	databases	683
URLs, mapping	510, 511	default error views, customizing	685
D		EMAIL_BACKEND	683
DB Browser for SQLite	71	error reporting	685
reference link	71	HTTPS	684
DEBUG	682	performance optimizations	684
Delete operation	158, 159	Django development server	6, 24
Django	1	starting	7 – 9
used, for serving uploaded (and other) files	408 – 410	Django development server architecture	677, 678
Django Cookie Consent		Gunicorn	679
reference link	471	Whitenoise	679
Django Cookie Law		Django forms	
		initial values, adding	346 – 348

placeholders, adding	346 – 348
using, for file uploads	395 – 401
using, for image uploads	402 – 408
Django migration	70, 71
Django models	
creating	73, 74, 350, 351
editing	350, 351
field options	74 – 77
methods, implementing	85 – 87
ModelForm class	352 – 356
primary keys	77 – 79
publisher form, integrating	363 – 365
publisher form, styling	363 – 365
publisher, creating	357 – 362
publisher, editing	357 – 362
review creation UI	366 – 372
Review model	84, 85
testing	652 – 656
Django project	5, 6
creating	7 – 9
structure	22, 23
Django settings	44, 45
referring to, in code	45, 46
Django, deployment options	690
Bookr project, deploying	692 – 710
containerized deployment	691
manual deployment	690
Platform-as-a-service (PaaS)	691
data	
storing, in sessions	478
data exports	
binary file formats	607
database CRUD operations, Django	90
create() method, used for entry creation	92
entry, creating in bookr database	90, 91
object, creating with foreign key	92
database configuration	68 – 70
databases	683
debugging	57, 58
decoupled architecture	574
delete() method	
using	111
deletion behavior	
in admin app	177, 178
directory	
setting up, to store template filters	534, 535
double-underscore lookup	101

E

EMAIL_BACKEND	683
media files	684
static files	683
Elasticsearch	205
email server	
configuring	516, 517
error reporting	
ADMINS	685
logs	685
MANAGERS	685
errors	
debugging	53
handling	53
exceptions	53
generating	55 – 57
ImportError	54
IndentationError	53
IndexError	54
KeyError	54
NameError	54
SyntaxError	53
TypeError	54
viewing	55 – 57
exclude() method	
used, for retrieving objects	103

F

FieldFile	
custom storage engines	415
existing files, storing on	416 – 418
reading from	415, 416
working with	414
FileField	
model, creating with	421 – 426
FileSystemFinder	232, 233
field lookups	
used, for filtering	101, 102
used, for querying across many-to-many relationship	108, 109
fields	70
file	
downloading	392 – 395
storing, on model instances	410 – 413
uploading	392 – 395

-
- file saving**
 - handling 432, 433
 - file serving**
 - for cache invalidation 248 – 250
 - file uploads**
 - Django forms, using for 395 – 401
 - HTML forms, using for 386 – 388
 - ModelForm, using for 426 – 428
 - file-based sessions** 471
 - filter() method**
 - using, for object retrieval 100, 101
 - filtering**
 - field lookups, using 101, 102
 - filters** 133, 531
 - findstatic command** 243
 - used, for finding files 244 – 247
 - flexbox** 481
 - foreign keys**
 - in admin app 177, 178
 - used, for creating object 92
 - used, for querying 107
 - form** 268
 - building, in HTML 273 – 280
 - built-in field validation 318, 319
 - data, accessing in View 280
 - defining 290 – 301
 - element 268, 269
 - extra field validation, adding 319 – 322
 - query string, need for 288, 289
 - rendering, in template 301 – 307
 - selecting, between GET or POST request 286 – 288
 - validating 311 – 315
 - validating, in view 315 – 318
 - working, with POST data in view 281 – 285
 - form enhancements, ModelAdmin attributes**
 - filter_horizontal / filter_vertical 498, 499
 - get_readonly_fields 503
 - inline forms 499 – 502
 - list_editable 504
 - radio_fields 503
 - readonly_fields 503
 - function-based views (FBVs)** 120, 121, 557
 - implementing 124 – 126
 - functional API views** 576
 - functional testing** 645
- ## G
- GET request**
 - working with 39 – 42
 - GET values** 42, 43
 - GET, or POST request**
 - selecting between 286 – 288
 - Gunicorn** 679
 - generic views** 582
 - get() method**
 - used, for returning object 98, 99
 - using, to retrieve object 97, 98
 - global logo**
 - adding 262, 263
 - graphs** 628
 - bubble plot 631 – 633
 - generating, in Python 633 – 636
 - plot, generating 630
 - groups**
 - managing 159
 - managing, through admin app 160 – 165
- ## H
- HTML**
 - form, building 273, 274, 278 – 280
 - HTML forms**
 - using, for file uploads 386 – 388
 - HTML templates**
 - finding, in app directories 47
 - HTTP 403 Forbidden response** 409
 - HTTP headers** 16 – 18
 - HTTP methods** 16
 - HTTP request** 15
 - HTTP response** 15
 - HTTP status codes** 19
 - reference link 19
 - HTTPS** 684
 - CSRF_COOKIE_SECURE 684
 - SESSION_COOKIE_SECURE 684
 - Help Desk user group** 160
 - HyperText Markup Language (HTML)** 12, 213
 - Hypertext Transfer Protocol (HTTP)** 15, 573

I

ImageField	
model, creating with	421 – 426
PIL images, writing to	418 – 420
i18n module	452
image and PDF, books	
uploading	433 – 438
image uploads	
Django forms, using for	402 – 408
ModelForm, using for	426 – 428
images	
generating in Python, with Pillow	636 – 638
resizing, with Pillow	404
storing, on model instances	413, 414
inclusion tags	542, 548
implementing	548
used, for rendering details on user profile page	552, 553
using, inside template	549
initial values	
adding, to Django form	346 – 348
adding, to OrderForm class	348 – 350
integration testing	645

J

JavaScript Object Notation (JSON)	472, 473, 578
Juggler	116

L

Linux	
pyenv, installing on	4
LiveServerTestCase class	671
list of books and reviews	
displaying	134 – 136
login links	
toggling, in base template	466, 467
logout links	
toggling in base template	466, 467

M

MEDIA_URL	
using, in templates	381 – 386
ManifestFilesStorage storage engine	

exploring	251 – 254
Model View Controller (MVC)	10
Model-View-Template (MVT)	10
examples	12 – 14
ModelAdmin classes	
customizing	190, 191, 209 – 211
date list_filter, adding	200 – 202
date_hierarchy filters, adding	200 – 202
display decorator	197, 198
fields, excluding and grouping	206 – 209
filter	198 – 200
list display fields	191 – 197
search bar	203 – 206
ModelForm	
using, for file and image uploads	429 – 432
ModelForm class	352 – 356
macOS	
pyenv, installing on	4
manage.py	
commands	23
manual deployment	690
many-to-many query	
objects, using	109
set() method, using	109
many-to-many relationships	81 – 83
add() method, using	95, 96
create() method, using	96
querying, with field lookup	108
records, creating with	94, 95
set() method, using	96, 97
many-to-one relationship	80, 81
records, creating	93, 94
media files	375
media files, in development	
serving	376, 377
media serving	
configuring	377 – 381
settings	376
media storage	
configuring	377 – 381
media uploads	
settings	376
middleware	444, 445
middleware modules	445, 446
purpose	445, 446
middleware stack	444

migration	70	order_by() method	
model instances		used, for retrieving objects	103 – 106
files, storing on	410 – 414		
images, storing on	413		
model methods	85	P	
model serializers	583, 584	PBKDF2/SHA256	457
creating	584 – 586	PDF files	
models	10	additional functionality, adding to	626 – 628
registering, within admin app	165	view	
multi-field validation	334 – 337	working with	621, 622
myproject directory	23, 24	PDF version, of web pages	
		generating, in Python	622 – 625
N		PIL images	
navigation bar		writing, to ImageField	418 – 420
adding	141, 142	POST data, in view	
non-field errors	336	working with	281 – 285
		POST request	
O		working with	39 – 42
Object Relational Mapper (ORM)	10	Pillow	402, 636
OrderForm class		image, resizing with	404
initial values, adding	348 – 350	Platform-as-a-service (PaaS)	691
placeholders, adding	348 – 350	Portable Document Format (PDF)	621
object		PyCharm	476
creating, with foreign key	92	project setup	26 – 34
object creation, with foreign key		setup	26
create() method, using for many-to-	96, 97	Python	
many relationships		graphs, generating	633 – 636
many-to-many relationship, using	95, 96	images, generating with Pillow	636 – 638
add() method		used, for writing to CSV files	603
records, creating for many-to-many	94, 95	values, retrieving	311 – 315
relationships		Python Imaging Library (PIL)	402
records, creating for many-to-one	93	Python shell	
relationship		AdminSite object, examining from	181, 182
set() method, using for many-to-many	96, 97	PythonAnywhere.com	691
relationships		Pythons csv module	
object instance		CSV files, writing with	602
used, for querying across foreign key	107, 108	working with	602
relationships		pagination customizations	506
object retrieval		parallelization	
exclude() method, using for	103	using, to run tests	672, 673
filter() method, using for	100, 101	password storage, in Django	457
order_by() method, using for	103 – 106	pattern matching	
one-to-one relationships	83	used, for filtering operations	102
operation filtering		performance optimizations	684
pattern matching, using	102	CONN_MAX_AGE	684
		sessions	684

templates	685
pickle	472
placeholders	
adding, to Django form	346 – 348
adding, to OrderForm class	348 – 350
plot	630
generating	630
rendering, on web page	631
plotly	628
graphs, generating with	628
installation	629
plotly Figure object	
setting up	629
primary keys	77
profile page	457
adding	457 – 459
project management application	
models, creating	116
publisher	
creating	357 – 362
editing	357 – 362
publisher form	
integrating	363 – 365
styling	363 – 365
pyenv	
installing, on Linux	4
installing, on macOS	4
installing, on Windows	3
virtual environment, creating	3 – 5

Q

Q objects	
used, for performing complex lookups	114
used, for performing complex query	114
QueryDict object	42, 43
working with	39 – 42
querying	
across foreign key relationship, with	107, 108
object instance	
across many-to-many relationship, with	108
field lookup	
across relationships	106
foreign keys, using	107
many-to-many query, using objects	109
many-to-many query, using set() method	109
model name, using	107
update() method, using	110

queryset	
verifying, with contains method	115, 116

R

REST APIs	573, 574
creating	576 – 578
RESTful web service	575
Railway.com	691
Render.com	691, 700
RequestFactory	666
using, to test views	666 – 669
Retrieve operation	155, 156
Review model	
adding	84, 85
Reviews logo	
adding	258 – 260
Routers	588
URL configuration, using	588
using	589 – 594
read operations	
all() method, using to retrieve set of	99, 100
object	
get() method, using for returning	98, 99
object	
get() method, using to retrieve object	97, 98
performing, with get() method	97
regression testing	645
relationships, relational database	79
many to many	81 – 83
many to one	80, 81
one-to-one	83, 84
remote user authentication	593
render function	
templates, rendering	49
representational state transfer (REST)	574
request	
processing	20, 21
request.user object	457
reviewer profile model	
implementing	497
reviews app	
migrating	87 – 89
reviews model	
registering	166, 167

S

SECRET_KEY	682	soft delete	157
SQLite3	68	static URLs	
STATICFILES_DIRS prefixed mode	240 – 243	generating, with static template tag	224 – 226
Seenode.com	691	static file finders	216
SimpleTestCase class	670	AppDirectoriesFinder	217
features	670	request	217
Structured Query Language (SQL)	11	static file namespacing	218, 219
sample link		static file serving	215, 216
displaying	439	from app directory	220 – 224
search and filter customization	507	from project static directory	233 – 236
serializers	578	static template tag	225, 226
class-based views	582	example	226 – 229
generic views	582	using	229 – 232
session authentication	593	storage engine	
session engine	444, 470	global logo, adding	262
cached sessions	471	string filters	536, 537
cookie-based sessions	471	superuser account	
file-based sessions	471	creating	149 – 151
session key		T	
examining	473 – 478	TitledUser model	
session storage		User, replacing with	494 – 496
using, for Book Search page	484, 485	TransactionTestCase class	
sessions	470	features	671
cookie content, flagging	471	Transmission Control Protocol (TCP)	575
data, storing	478	template context	
recently viewed books, storing	479 – 483	passing, in custom simple tag	547
simple tags	542	template filters	532, 533
using, inside templates	544	directory, setting up to store	534, 535
simple template tag		template inheritance	138, 139
creating	542	adding	141, 142
directory, setting up	543	template language	132
implementing	543	template library	
template library, setting up	543	setting up	535
singleton class	535	template partials	531, 554, 555
site welcome screen		book list item, rendering	556
creating	63	template styling	
site-wide Django admin		with Bootstrap	139, 140
AdminSite object, examining from	181, 182	template tags	132, 531, 541
Python shell		inclusion tags	542
AdminSite, subclassing	182 – 186	simple tags	542
AdminSiteobject, customizing	186 – 190	template variables	132
customizing	179 – 181	using, to pass additional keys to	515, 516
smoke testing	645	templates	
		templates	12, 127 – 129

- custom filters, using 536
- custom simple tag, using 549
- enhancing, with authentication data 466
- greeting message, displaying 129 – 131
- media, referring in 421
- rendering, in view 49, 50
- rendering, with render function 49
- simple tags, using 544
- variables, rendering 50, 51
- variables, using 51, 52
- templates directory**
 - creating 47 – 49
- test case classes** **670**
 - LiveServerTestCase class 671
 - SimpleTestCase class 670
 - TransactionTestCase class 671
- test cases** **646**
 - implementing 646
 - pre-test setup and cleanup, performing 650, 651
- test code**
 - modularizing 671
- testing** **643**
 - automation testing 644
 - functional testing 645
 - importance 644
 - in Django 645
 - integration testing 645
 - regression testing 645
 - smoke testing 645
 - unit testing 645
- tests**
 - running, with parallelization 672, 673
- token authentication** **593**
 - implementing, for Bookr APIs 593 – 599
- top contributors page**
 - used, for creating API endpoint 587
- U**
- URL configuration** **122, 123**
 - with Routers 588
 - with ViewSets 588
- URL mapping** **36, 37, 124**
 - parameters 288, 289
- URLs**
 - mapping, for custom view 510, 511
- Update operation** **156, 157**
- unit test**
 - writing 648, 649
 - writing, for views 657 – 660
- unit testing** **645**
 - implementing 646
- update() method**
 - using 110
- uploaded files**
 - working with, in view 388, 389
- user account**
 - creating 154
- user model** **488**
 - extending 489, 490
- user profile page**
 - details, rendering with inclusion tags 552, 553
- users**
 - managing 159
 - managing, through admin app 160 – 165
- V**
- ViewSets** **588**
 - URL configuration, using 588
 - used, for simplifying code 588
 - using 589 – 592
- variables**
 - rendering, in templates 50, 51
 - using, in templates 51, 52
- view function**
 - creating 509
- views** **11, 35, 36, 120, 556**
 - adding, to admin site 508
 - class-based views 120, 121, 557, 558
 - function-based views 120, 121, 557
 - template, rendering 49, 50
 - test cases, writing, to validate authenticated users 661 – 665
 - testing 656, 657
 - testing, with authentication 661
 - testing, with RequestFactory 666 – 669
 - unit tests, writing 657 – 660
 - URL map, setting up 38
 - writing 38
- virtual environment**
 - creating, with pyenv 3 – 5

W

WeasyPrint	621
Web Server Gateway Interface (WSGI)	679
Whitenoise	679
Windows	
pyenv, installing on	3

X

XLSX files	608
data, writing to workbook	611
data, writing to worksheet	610

formatted spreadsheet, developing	615 – 618
spreadsheet, formatting	611 – 614
workbook	608
workbook, creating	610
worksheet, creating	610
worksheets	609
XlsxWriter package, used for working with	608
XlsxWriter Python package	609, 610
used, for working with XLSX files	608

Z

ZIP files	619, 620
zipfile module	619